

THEMISDB

Das vollständige Handbuch

Version 1.3.4



Umfassendes Nachschlagewerk für die ThemisDB-Datenbank

Generiert: 11. January 2026

Kapitel: 58

© 2025 ThemisDB Project



Inhaltsverzeichnis

1. Vorwort	1
2. Kapitel 0: Genese und Entwicklungsgeschichte von ThemisDB	1
3. Kapitel 1: Einführung in ThemisDB	1
4. Kapitel 2: Architektur-Überblick	1
5. Kapitel 3: Multi-Model verstehen	1
6. Kapitel 4: Installation und Setup	1
7. Kapitel 5: Relationale Daten	1
8. Kapitel 6: Graph-Datenbanken	1
9. Kapitel 7: Dokument-Speicherung	1
10. Chapter 8: Storage Layer Deep-Dive	1
11. Kapitel 8: Vektor-Suche und Similarity Search	1
12. Kapitel 9: Time-Series & IoT-Daten	1
13. Kapitel 10: Enterprise-Anwendungen	1
14. Chapter 11: Realtime Data Streaming mit Change Data Capture	1
15. Kapitel 12: Computer Vision & Bildanalyse	1
16. Kapitel 13: Volltext-Suche & NLP	1
17. Kapitel 14: Geo-Spatial Features	1
18. Kapitel 15: Analytics & Reporting	1
19. Machine Learning Integration	1
20. Kapitel 16: Horizontal Scaling mit Sharding	1
21. Horizontal Scaling	1

22. Kapitel 17: LLM-Integration und Prompt Engineering	1
23. Hochverfügbarkeit	1
24. Kapitel 18: Machine Learning Integration	1
25. Chapter 19: Monitoring & Observability	1
26. Kapitel 19: Monitoring & Observability	1
27. Performance Tuning	1
28. Kapitel 20: Backup & Recovery	1
29. Authentifizierung	1
30. Kapitel 21: Performance Tuning	1
31. Kapitel 22: Client Libraries & Drivers	1
32. Verschlüsselung	1
33. Kapitel 23: Testing & Quality Assurance	1
34. Kapitel 24: KI-Ethik und Governance	1
35. Kapitel 25: DevOps & Infrastructure as Code	1
36. Kapitel 26: Migration & Legacy System Integration	1
37. Kapitel 27: Troubleshooting & Problem Resolution	1
38. Kapitel 28: AQL Vollständige Referenz	1
39. Kapitel 29: Analytics & Process Mining	1
40. Kapitel 30: Deployment & Operations	1
41. Kapitel 31: API-Protokolle (HTTP/2, HTTP/3, MCP)	1
42. Kapitel 32: AQL v1.3.1 OOP-Implementierung	1
43. Kapitel 33: Best Practices & Design Patterns	1
44. Kapitel 34: Query Optimierung & Performance Tuning	1

45. Kapitel 35: Data Modeling Patterns & Anti-Patterns	1
46. Kapitel 36: Security Hardening Playbook	1
47. Kapitel 37: Ecosystem Integration & Extensions	1
48. Kapitel 38: Observability & SRE Playbook	1
49. Kapitel 39: Performance Tuning Cookbook	1
50. Kapitel 40: Data Governance & Compliance	1
51. Kapitel 41: Hands-on Labs – Deployment, Index-Tuning, Vector-Suche	1
52. Appendix D: Feature Status Matrix	1
53. Appendix E: Incident Response Runbooks	1
54. Appendix F: AQL Cheat Sheet & Quick Reference	1
55. Appendix G: Configuration Reference	1
56. Appendix H: Glossary & Terminology Reference	1
57. Appendix I: Troubleshooting Guide & Common Issues	1
58. Anhang A: Literaturverzeichnis	1

Kapitel 1: Vorwort

"Dokumentation ist wie Code - sie sollte wartbar, erweiterbar und verständlich sein."

Warum dieses Buch?

Als wir ThemisDB entwickelten, hatten wir eine Vision: Eine Datenbank, die alles kann, was moderne Anwendungen brauchen - relational, Graph, Dokument und Vektor - in einem System, ohne Kompromisse.

Aber eine großartige Datenbank ist nur halb so viel wert ohne großartige Dokumentation. Und genau hier beginnt die Herausforderung.

Das Dokumentations-Dilemma

Wir hatten über 700 Dokumente geschrieben: - API-Referenzen - Feature-Beschreibungen - Architektur-Dokumente - Beispiel-Projekte - How-To-Guides - Troubleshooting-Tips

Alles war da. Aber es war verstreut. Fragmentiert. Schwer zu durchdringen für Neue.

Ein Entwickler fragte uns:

"Ich habe die Docs gelesen, aber ich verstehe nicht, wie alles zusammenpasst. Wann nutze ich Graph statt Relational? Wie skaliere ich? Was sind die Best Practices?"

Das war der Moment, in dem wir erkannten: Wir brauchen nicht nur Dokumentation - **wir brauchen ein Buch.**

Was macht dieses Buch anders?

1. Narrative statt Referenz

Statt:

"Die `SHORTEST_PATH` Funktion findet den kürzesten Pfad..."

Schreiben wir:

"Stellen Sie sich vor, Sie bauen ein Social Network. Alice will wissen, wie sie mit Bob verbunden ist. In SQL wäre das ein rekursiver Join - langsam und komplex. ThemisDB macht es einfach..."

2. Vollständige Examples

Jedes Konzept wird mit einem vollständigen, lauffähigen Example demonstriert. Kein Pseudo-Code. Echte Programme, die Sie herunterladen und ausführen können.

3. Das "Warum" vor dem "Wie"

Wir erklären nicht nur, **wie** Features funktionieren, sondern **warum** sie so designt wurden und **wann** Sie sie einsetzen sollten.

4. Von Einfach zu Komplex

Kapitel 1 erklärt die Basics. Kapitel 30 zeigt Enterprise-Patterns. Dazwischen eine sorgfältig gestaltete Lernkurve.

5. Alle Modelle integriert

Wir zeigen nicht nur einzelne Modelle, sondern wie Sie sie kombinieren. Multi-Model Queries. Cross-Model Transaktionen. Das ganze Potenzial.

Inspiration

Dieses Buch ist inspiriert von Software-Engineering-Klassikern, die uns beim Lernen geholfen haben:

"Designing Data-Intensive Applications" (Martin Kleppmann)

Für die tiefe, konzeptionelle Herangehensweise.

"Programming Rust" (Blandy, Orendorff, Tindall)

Für die "Let's build..." Abschnitte und vollständigen Programme.

"The Go Programming Language" (Donovan, Kernighan)

Für die klare, präzise Sprache und praktischen Beispiele.

"Site Reliability Engineering" (Google)

Für die Operations-Perspektive und Real-World Case Studies.

Für wen ist dieses Buch?

Wenn Sie neu bei ThemisDB sind

Fangen Sie bei Kapitel 1 an. Arbeiten Sie sich durch Teil I und II. Nach Teil II haben Sie ein solides Fundament.

Wenn Sie bereits Erfahrung haben

Springen Sie direkt zu den Themen, die Sie interessieren: - **Performance-Probleme?** → Kapitel 20 - **Skalierung planen?** → Kapitel 17-18 - **Sicherheit härten?** → Teil VI - **AQL meistern?** → Kapitel 13

Wenn Sie von einer anderen DB migrieren

- **Von PostgreSQL?** → Kapitel 5, 29
 - **Von Neo4j?** → Kapitel 6, 29
 - **Von MongoDB?** → Kapitel 7, 29
-

Was Sie brauchen

Vorwissen

- **Grundlegende Programmierkenntnisse:** Python, JavaScript oder ähnlich
- **AQL-Grundlagen:** Hilfreich, aber nicht erforderlich
- **Docker-Basics:** Für die Examples

Software

- **Docker:** Für ThemisDB und Examples
- **Python 3.8+:** Für Example-Code
- **Git:** Zum Klonen der Examples

Alles ist kostenlos und Open Source.

Struktur dieses Buchs

8 Teile, 30 Kapitel, ~880 Seiten

Jedes Kapitel folgt diesem Muster: 1. **Überblick:** Was Sie lernen werden 2. **Theorie:** Konzepte erklärt 3. **Praxis:** Vollständige Examples 4. **Patterns:** Best Practices 5. **Performance:** Optimierung 6. **Zusammenfassung:** Key Takeaways

Wie dieses Buch entstand

Dieses Buch ist das Ergebnis von: - **700+ einzelne Dokumente** konsolidiert - **21 vollständige Example-Projekte** integriert - **Monate an Reviews** von Entwicklern und Early Adopters - **Unzählige Iterationen** bis zur perfekten Struktur

Es ist lebendig. Wir aktualisieren es mit jeder ThemisDB-Version. Feedback ist willkommen.

Danksagungen

Ein großes Dankeschön an:

- **Die Community:** Für Feedback, Bug Reports und Feature Requests
- **Early Adopters:** Die ThemisDB in Production eingesetzt haben
- **Contributors:** Für Code, Docs und Examples
- **Reviewer:** Für unzählige Stunden detailliertes Feedback

Besonderer Dank an die Autoren der Bücher, die uns inspiriert haben.

Über die Autoren

Dieses Buch wurde geschrieben vom ThemisDB-Team - Entwickler, die täglich mit der Datenbank arbeiten und sie lieben.

Wir glauben an: - **Open Source:** ThemisDB ist MIT-lizenziert - **Qualität:** Code und Docs auf höchstem Niveau - **Community:** Gemeinsam besser werden

Feedback und Beiträge

Dieses Buch ist Open Source, wie ThemisDB selbst.

Fehler gefunden?

[Issue erstellen](#)

Verbesserung vorschlagen?

[Pull Request öffnen](#)

Frage stellen?

[Discussion starten](#)

Jeder Beitrag zählt. Auch wenn es nur ein Tippfehler ist.

Los geht's!

Sie halten ein Handbuch in den Händen, das Sie von Null zur ThemisDB-Mastery führen wird.

Es ist eine Reise. Nehmen Sie sich Zeit. Experimentieren Sie mit den Examples. Bauen Sie eigene Projekte.

Und vor allem: Haben Sie Spaß!

Datenbanken sind mächtige Werkzeuge. ThemisDB macht sie zugänglich.

Ready to become a ThemisDB expert?

[→ Kapitel 1: Einführung in ThemisDB](#)

ThemisDB Team

Dezember 2025

Kapitel 2: Kapitel 0: Genese und Entwicklungsgeschichte von ThemisDB

> "Innovation entsteht nicht durch Beschaffung, sondern durch Notwendigkeit."

Überblick

Dieses Kapitel erzählt die außergewöhnliche Entstehungsgeschichte von ThemisDB – von einer privaten "Civic Tech"-Initiative bis zur produktionsreifen Multi-Modell-Datenbank. Sie erfahren, welche Probleme zur Entwicklung führten, wie das Projekt wuchs, und welche Meilensteine erreicht wurden. Die Geschichte von ThemisDB ist ein Paradebeispiel dafür, wie technische Innovation aus echten Problemstellungen entsteht und wie eine "Bottom-Up"-Entwicklung zu einem System führen kann, das ursprüngliche Erwartungen übertrifft. Jedes Feature, jede Designentscheidung und jede Architekturkomponente wurde durch konkrete Anforderungen aus der Praxis motiviert. Diese organische Entwicklung führte zu einer Datenbank, die nicht nur theoretisch elegant ist, sondern auch in der realen Welt funktioniert.

Was Sie in diesem Kapitel lernen werden: - Die strategische Ausgangslage und das Problem der UDS3-Architektur - Wie ThemisDB als "Bottom-Up"-Innovation entstand - Die wichtigsten Entwicklungsphasen und Meilensteine - Das Lizenzmodell und die wissenschaftliche Absicherung - Der Status "Production-Ready" (November 2025)

Voraussetzungen: Keine. Dieses Kapitel liefert den historischen Kontext für alle folgenden Kapitel.

Der strategische Imperativ

Die Entwicklung begann nicht als akademisches Projekt oder als Produkt eines Unternehmens, sondern als Antwort auf eine konkrete gesellschaftliche Herausforderung. Die deutsche öffentliche Verwaltung steht vor tiefgreifenden strukturellen Veränderungen, die ohne technologische Innovation nicht bewältigt werden können. Der **demografische Wandel**, die **Digitalisierung** und die **zunehmende Komplexität der Verwaltungsaufgaben** haben einen perfekten Boden geschaffen, der neue Lösungsansätze erforderlich machte. Die sogenannte "Babyboomer"-Generation, die einen Großteil der erfahrenen Fachkräfte in der Verwaltung ausmacht, erreicht das Rentenalter. Gleichzeitig wird es immer schwieriger, qualifizierte Nachwuchskräfte zu rekrutieren. Diese demografische Schere führt zu einem massiven Wissensverlust und einer Überlastung der verbleibenden Mitarbeiter. Ohne technologische Unterstützungssysteme ist die staatliche Handlungsfähigkeit in kritischen Bereichen gefährdet.

Um diesen Wissensverlust aufzufangen, wurde das VCC-Ökosystem (Veritas, Covina, Clara) konzipiert. VCC ist dabei kein einfacher KI-Chatbot, sondern viel mehr.

- **Veritas:** KI-gestützter agenten-basierter Assistent für Fachexperten, Wissensmanagement und Dokumentenverarbeitung
- **Covina:** Ingestion-Pipeline für heterogene Datenquellen
- **Clara:** large-language-model Verbesserung mit Hilfe von LoRa

***Die Kernidee** Ein souveränes, KI-gestütztes Assistenzsystem, das Fachexperten entlastet, Wissen konserviert und die Effizienz steigern kann.*

D.h. eine Chat-Anfrage über Veritas kann über das Agentensystem aus verschiedenen Datenquellen Informationen zusammentragen um der Sachbearbeiterin, dem Sachbearbeiter eine umfangreiche Sicht auf komplexe Sachzusammenhänge zu geben. Konkret kann man das an einem Bauantrag veranschaulichen. Ein Bürger beantragt die Errichtung eines Carports. Der Bearbeiter muss jetzt für die Sachentscheidung alle relevanten Information zusammentragen, dass beginnt ganz banal bei dem eigentlichen Antragsgegenstand (Bauherr, Bauort, Baugegenstand usw.), den zugrundeliegenden Gesetzen, Verordnungen, Anweisungen, Bebauungsplänen sowohl auf Bundes-, Länder- als auch auf kommunaler Ebene und nicht zuletzt auch der zuletzt gültigen Rechtslage (z.B. Änderungen durch Gerichtsurteile). Um im Bild zu bleiben gibt es für den Sachbearbeiter einen, teils wiederkehrenden, Rechtsrahmen indem er über den Antrag des Bürgers entscheiden muss.

```
%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#f9f9f9', 'primaryBorderColor': '#f9f9f9'}}
graph LR
    Eingang["<b>Antragseingang</b><br/>Bürger -> Bauaufsicht"] --> Sachbearbeitung["<b>Sachbearbeitung</b><br/>Sachbearbeitung -> Entscheidung["<b>Entscheidung</b><br/>Genehmigung / Ablehnung"]

    style Eingang fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Sachbearbeitung fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Entscheidung fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
```

Abb. 0.0: Vereinfachter Bauantrag-Prozess (Brandenburg): Antragseingang → Sachbearbeitung (Vollständigkeit, Fachbeteiligungen) → Entscheidung

Deswegen war bei der Genese auch ein nicht zu vernachlässigender Aspekt, die wirkliche machinennutzbare Abbildung von Verwaltungsprozessen. Es ist einfach zu kurz gesprungen nur anstatt Papierdokumenten jetzt auf E-Mail für den Informationsaustausch zu setzen. Also gibt es ein weiteres technologisches Herz, der **VPB** – ein "Digitaler Zwilling" der Verwaltung [1], [9]:

```

%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#f9f9f9', 'primaryBorderColor': '#f9f9f9'}}
graph TB
    Antrag["<b>Antrag</b><br>Bauvorhaben"]
    Sachbearbeitung["<b>Sachbearbeitung</b>"]
    Entscheidung["<b>Bescheid</b><br>Genehmigung/Auflage"]

    Antrag --> Sachbearbeitung --> Entscheidung

    subgraph Intern["Interne Prüfung"]
        direction TB
        VZ["Vollständigkeit<br>und Gebühren"] --> Rechtspruefung["Rechtsprüfung<br>(Bund/Land/Kommune)"]
    end

    subgraph Extern["Fachbehördenbeteiligung"]
        direction TB
        Fachbeteiligung["Fachbeteiligungen"] --> Brandschutz["Brandschutzbehörde"] --> Denkmalschutz["Denkmalschutzbehörde"]
    end

    Sachbearbeitung --> VZ
    VZ --> Fachbeteiligung
    Wasser --> Fachrecht
    Praezedenz --> Entscheidung

    style Antrag fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Sachbearbeitung fill:#f5f5f5,stroke:#2e7d32,stroke-width:2px,color:#1a1a1a
    style Entscheidung fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style VZ fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Rechtspruefung fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Fachrecht fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Praezedenz fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Fachbeteiligung fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Brandschutz fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Denkmalschutz fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Umwelt fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a
    style Wasser fill:#f5f5f5,stroke:#7c4dff,stroke-width:2px,color:#1a1a1a

```

Abb. 0.1: VPB-Graph für Sachbearbeitung: Antrag → Sachbearbeitung → Bescheid, mit verknüpften Zuständigkeiten (Bund/Land/Kommune), Fachrecht und Präzedenzfällen

Die Anforderung: Graph-RAG – die Kombination von: - **Semantischer Suche** (Vektor-Embeddings für KI) - **Prozessualem Kontext** (Graph-Beziehungen) - **Strukturierten Metadaten** (Relationale Daten) - **Strenger ACID-Konsistenz** (für rechtsverbindliche Akte)

Doch das technologische Herzstück dieses Systems, der „Digitale Zwilling“ der Verwaltung, stieß schnell an eine unüberwindbare Grenze.

Die Genese: Von der privaten Initiative zur Civic Tech

Das Civic Tech Paradigma

ThemisDB ist ein Paradebeispiel für **Civic Tech** [9]. Anstatt auf langwierige Ausschreibungsprozesse zu warten, entstand ThemisDB als Bottom-Up-Innovation. Verwaltungsmitarbeiter mit tiefer IT-Expertise entwickelten in Eigeninitiative einen Prototyp, der die Vision einer nativen Multi-Modell-Datenbank verfolgte. Das Ziel war ein transaktionaler Kern, der Relationen, Graphen und Vektoren in einem einzigen System vereint und damit echte ACID-Garantien ermöglicht.

Civic Tech - Technologie, die aus der Mitte der Zivilgesellschaft/Verwaltung für das Gemeinwohl entsteht, anstatt eingekauft zu werden.

Die Prinzipien: 1. **Problem-driven:** Entwicklung aus echter Notwendigkeit, nicht aus Spezifikation 2. **Practitioner-led:** Entwickler sind gleichzeitig Nutzer 3. **Open Source:** Code gehört der Allgemeinheit 4. **Iterativ:** Schnelle Iterationen statt jahrelanger Planung

Die Motivation: - Keine Lösung am Markt erfüllte die spezifischen Anforderungen - Hyperscaler-Lösungen zu teuer und mit Vendor-Lock-in - Open-Source-Alternativen mit denselben UDS3-Problemen behaftet

Der „Force Multiplier“: KI-gestützte Entwicklung als Geburtshelfer

Die Realisierung von ThemisDB in einer so kurzen Zeitspanne wäre ohne den Einsatz moderner KI-Werkzeuge wie Google Gemini, GitHub Copilot und spezialisierten Coding-LLMs undenkbar gewesen. Diese Tools fungierten als „Force Multiplier“, die es einem kleinen Team ermöglichten, die Komplexität einer Multi-Modell-Datenbank zu beherrschen.

Demokratisierung der Hochsprachen-Programmierung

Ursprünglich war die Entwicklung von Datenbank-Kernen (in C++ oder Rust) eine Domäne für spezialisierte Informatiker mit jahrzehntelanger Erfahrung. Durch den Einsatz von KI-Assistenten verschob sich dieses Paradigma:

Abstraktion der Komplexität: Die KI übernahm das „Boilerplate“-Coding und die Implementierung komplexer Algorithmen (wie HNSW-Indizes oder MVCC-Logiken) auf Basis präziser fachlicher Instruktionen.

Vom Programmierer zum Architekten: Die Rolle der Entwickler wandelte sich. Anstatt jede Zeile Code mühsam selbst zu schreiben, fungierten sie als Architekten und Reviewer, die die KI-generierten Module validierten und zusammenfügten.

Rapid Prototyping in Lichtgeschwindigkeit

Was früher Monate für die Erstellung eines funktionsfähigen Prototyps (MVP) dauerte, wurde durch KI auf Wochen reduziert: - **Automatisierte Testgenerierung:** KI-Tools erstellten simultan zum Code die passenden Unit-Tests, was eine extrem hohe Iterationsgeschwindigkeit bei gleichbleibender Qualität ermöglichte. - **Fehlerdiagnose in Echtzeit:** Debugging-Prozesse, die normalerweise Tage in Anspruch nehmen können, wurden durch KI-Analyse von Stacktraces und Memory-Dumps in Minuten gelöst.

Civic Tech trifft auf KI-Empowerment

Dieses „KI-Empowerment“ ist der Schlüssel zum Erfolg des Civic-Tech-Ansatzes von ThemisDB. Es erlaubte Experten aus der Verwaltung, die zwar tiefes Domänenwissen, aber nur grundlegende Programmierkenntnisse besaßen, ein System auf Enterprise-Niveau zu bauen:

- **Überbrückung der Wissenslücke:** Die KI übersetzte die komplexen juristischen Anforderungen an ACID-Konsistenz und DSGVO-Compliance direkt in technischen Code.
- **Ressourceneffizienz:** Ein Team von nur drei Personen konnte Aufgaben bewältigen, für die traditionelle IT-Häuser ganze Abteilungen benötigt hätten.

Die neue Ära der Software-Genese ThemisDB ist somit nicht nur ein technologisches Produkt, sondern auch ein Beweis für einen neuen Entwicklungstypus: AI-driven Civic Tech. Die Kombination aus der Notwendigkeit (demografischer Wandel), dem Mut zur Eigeninitiative (Bottom-Up) und den richtigen Werkzeugen (Gemini, Copilot) hat gezeigt, dass die öffentliche Verwaltung wieder zum Innovator werden kann, wenn sie die Kraft der KI für ihre eigenen Ziele nutzt.

Das Scheitern der UDS3-Architektur

Um das Scheitern zu verstehen muss man kurz erklären wie Verwaltungshandeln funktioniert.

Ein stark vereinfachtes Diagramm eines generischen Verwaltungsprozesses: Event -> ein konkretes Anliegen eines Bürgers mit Anspruch auf Verwaltungshandeln, z.B. Bauantrag Verwaltungsakt -> eine Behörde legt einen Vorgang an und übernimmt die Sachbearbeitung und Entscheidungsfindung Entscheidung -> die Behörde trifft eine Entscheidung über das Anliegen des Bürgers, z.B. durch Erteilung eines Bescheides

Die ursprüngliche Planung: Polyglot Persistence

Die behördliche IT-Strategie "Unified Database Strategy 3" (UDS3) [1], [9], die auf dem "Best-of-Breed"-Prinzip basierte: PostgreSQL für Metadaten, Neo4j für Graphen und ChromaDB für Vektoren offenbarte in der Praxis schnell, dass dieser Ansatz einen fundamentalen, juristisch untragbaren Mangel aufwies. Da die Daten physisch auf drei verschiedene Systeme verteilt waren, konnten keine atomaren Transaktionen garantiert werden (sog. ACID - Garantien).

In einem rechtssicheren Verwaltungsumfeld ist dies fatal: Würde beispielsweise ein Dokument nach DSGVO gelöscht, müsste dieser Vorgang zeitgleich in der relationalen Datenbank, im Graphen und im Vektor-Index abgeschlossen sein. Da dies bei getrennten Systemen nur über das komplexe Saga-Pattern und mit dem Risiko der "Eventual Consistency" (BASE) möglich ist, könnten kurzzeitig inkonsistente und damit rechtsunwirksame Zustände entstehen. Für die Entwickler war klar: Für revisionssichere Akte ist "eventuell konsistent" inakzeptabel.

Die Architektur:

```
%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#f9f9f9', 'primaryBorderColor': '#f9f9f9'}}
graph TB
    App["<b>Application Layer</b><br/>(Saga-Pattern)"]

    subgraph Data[" "]
        PG["<b>PostgreSQL</b><br/>Metadaten"]
        Neo["<b>Neo4j</b><br/>Graphen"]
        Chroma["<b>ChromaDB</b><br/>Vektoren"]
    end

    App -->|3 separate Systeme<br/>= Transaktions-Challenge| PG
    App -->|atomare Transaktion<br/>unmöglich| Neo
    App -->|Eventual Consistency| Chroma

    style App fill:#f5f5f5,stroke:#F44336,stroke-width:2px,color:#1a1a1a
    style Data fill:transparent,stroke:none
    style PG fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a
    style Neo fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a
    style Chroma fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a
```

Abb. 0.2: UDS3 Polyglot-Persistence-Architektur: Drei unabhängige Datenbanksysteme mit unmöglicher ACID-Gewährleistung über Transaktionen hinweg

- **PostgreSQL:** Relationale Metadaten (Aktenzeichen, Daten, Status)
- **Neo4j:** Prozessbeziehungen und Wissensgraphen
- **ChromaDB:** Vektor-Embeddings für semantische Suche

Die Motivation: "Best-of-Breed" – jede Datenbank für ihre Stärken nutzen.

Der fundamentale Fehler: Eventual Consistency

Die physische Trennung der Daten führte zu einem **juristisch untragbaren Problem** [1]:

Das Problem:

```

try:
    result = db.query("""
        FOR d IN documents
            FILTER d.id == 'BImSchG-2024-042'
            REMOVE d IN documents
        RETURN OLD
    """)

    # Single atomic operation:
    # 1. Dokument aus Collection gelöscht
    # 2. Vektoren aus Index gelöscht
    # 3. Graph-Edges zu verwandten Docs aufgelöst
    # → Alles oder nichts - KEINE Inkonsistenz möglich

except Exception:
    # Rollback über 3 separate Datenbanken?
    # → UNMÖGLICH ohne komplexes Saga-Pattern!

```

Die Konsequenz: - Atomare Transaktionen über alle drei Systeme sind **unmöglich** [1], [17] - Man muss auf das **Saga-Pattern** [17] mit kompensierenden Transaktionen zurückgreifen - Resultat: **"Eventual Consistency" (BASE)** [18] statt ACID - Ein Verwaltungsakt kann zeitweise in einem inkonsistenten Zustand sein

Das Urteil: Für revisionssichere Verwaltungsakte ist "eventuell konsistent" operativ und rechtlich **inakzeptabel** [1], [9].

0.4 Entwicklungsphasen und Meilensteine

Phase 1: Proof of Concept (Q1-Q2 2025)

Ziel: Validierung des Base Entity Paradigmas

Errungenschaften: - Implementierung des kanonischen "Base Entity"-Speicherformats [3], [4] - Integration von RocksDB als transaktionale Storage-Engine [13], [46] - Erste Tests mit MVCC (Multi-Version Concurrency Control) [20] - Proof of Concept für atomare Transaktionen über Relational, Graph und Vektor

Technologie-Stack: - C++ für Performance-kritische Komponenten - RocksDB TransactionDB für ACID-Garantien - VelocityPack für effiziente Serialisierung [41]

Phase 2: Core Engine (Q2-Q3 2025)

Ziel: Produktionsreife Kern-Engine

Errungenschaften: - Vollständige AQL-Implementierung (Advanced Query Language) [9] - Native Graph-Traversierung mit temporalen Abfragen [9] - HNSW-Index für Vektor-Operationen [25] - Query Optimizer mit kostenbasiertem Planning [9]

Meilensteine: - 45.000 Writes/Sekunde Ingestion-Performance [3], [5], [9] - Sub-Millisekunden Latenz für Lesezugriffe ($p50 < 0.1ms$) [9] - MVCC mit Snapshot Isolation [15], [20]

Phase 3: Enterprise Features (Q3-Q4 2025)

Ziel: BSI-konforme Sicherheits- und Compliance-Features

Errungenschaften: - Native RBAC-Implementierung (Role-Based Access Control) [9] - Tamper-Proof Audit Logs mit Hash-Chains [9] - DSGVO-Compliance "by Design" [44]: - Auto-Purge nach Retention-Period - PII Detection und Redaction - Encrypt-then-Sign mit PKI - HashiCorp Vault Integration für Key Management [9]

Status: Wegfall externer Abhängigkeiten wie Apache Ranger [9]

Phase 4: Production-Ready (Oktober-November 2025)

Ziel: Vollständige Produktionsreife für Single-Node-Szenarien

Audit vom 20. November 2025 [9]:

✓ Core Engine:	100% Complete
✓ ACID Transactions:	Production-Ready
✓ Security Stack:	BSI-konform
✓ Performance:	Benchmarked & Validated
✓ Documentation:	Comprehensive
✓ Testing:	All Tests Green (28.10.2025)

Gesamtfortschritt: ~52% (P0-Features: 100%) [9]

Status-Erklärung: "Production-Ready" für Single-Node bedeutet: - ✓ Stabile API - ✓ ACID-Garantien validiert - ✓ Performance-optimiert - ✓ Security-gehärtet - ⌚ Horizontal Scaling in Entwicklung (für Multi-TB-Szenarien)

0.5 Das Lizenzmodell: Sovereign Open Source

MIT-Lizenz mit Government-Klausel

ThemisDB nutzt ein innovatives Lizenzmodell [9]:

Basis: MIT-Lizenz (permissiv und entwicklerfreundlich)

Erweiterung: Government-Klausel verhindert: - Cloud-Anbieter nehmen den Code - Bieten ihn als proprietären Service an - Schließen eigene Erweiterungen

Der Schutz:

- ✓ Entwickler können Code frei nutzen
- ✓ Behörden behalten volle Kontrolle
- ✓ Wissenschaftliche Nutzung uneingeschränkt
- ✗ Kommerzialisierung ohne Rückfluss an Community verhindert

Lizenz-Audit: Alle verwendeten Bibliotheken sind kompatibel und "clean" [9]: - RocksDB (Apache 2.0 / GPL) - simdjson (Apache 2.0) - Arrow (Apache 2.0) - Keine GPL-Kontamination im Core

Das "Amazon-Problem" gelöst

Das Problem: AWS/Azure könnten Open-Source-Code nehmen und als Managed Service verkaufen, ohne zur Community beizutragen.

Die Lösung: Government-Klausel erlaubt nur: - **On-Premise-Nutzung** ohne Einschränkung - **Cloud-Hosting** nur mit Open-Source-Rückfluss - **Kommerzielle Nutzung** mit Lizenzgebühren-Vereinbarung

0.6 Wissenschaftliche Absicherung: Die "Wissensfalle" vermeiden

Das Risiko: Bus-Faktor

Das Problem: ThemisDB entstand aus privater Initiative – Abhängigkeit von wenigen Personen [9]:

Bus-Faktor-Analyse:

Kernentwickler:	2-3 Personen
Code-Ownership:	Konzentriert
Wissenstransfer:	Limitiert
Dokumentation:	Gut, aber personengebunden
→ Risiko bei Ausfall:	HOCH

Die Lösung: Public-Public-Partnership (geplant)

Die Strategie: Systematische wissenschaftliche Begleitung durch akademische Partner [9]:

Angestrebte Forschungs Kooperationen:

```

%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#f9f9f9', 'primaryBorderColor': '#f9f9f9'}}
graph TD
    A["<b>Implizites Expertenwissen</b><br/>Entwickler"]
    B["<b>Wissenschaftliche Dokumentation</b><br/>& Analyse"]
    C["<b>Peer-Review & Publikationen</b><br/>Fachkommunität"]
    D["<b>Institutionelles Gemeingut</b><br/>öffentlich & zeitlos"]

    A -->|Knowledge Extraction| B
    B -->|Scientific Validation| C
    C -->|Community Adoption| D

    style A fill:#f5f5f5,stroke:#FF9800,stroke-width:2px,color:#1a1a1a
    style B fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a
    style C fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a
    style D fill:#f5f5f5,stroke:#4CAF50,stroke-width:2px,color:#1a1a1a

```

Abb. 0.4: Wissenstransfer-Prozess von implizitem Expertenwissen über wissenschaftliche Dokumentation zu institutionellem Gemeingut

```

style B fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a style C
fill:#f9f9f9,stroke:#2196F3,stroke-width:2px,color:#1a1a1a style D fill:#f5f5f5,stroke:#4CAF50,stroke-
width:2px,color:#1a1a1a

```

****Potenzielle Vorteile:****

- Wissenstransfer in die wissenschaftliche Community
- Unabhängige Validierung der Architektur
- Langfristige Wartbarkeit durch institutionelle Absicherung
- Community-Building durch akademische Partner

0.7 Der Weg zur Standardisierung**### Aktuelle Positionierung (Dezember 2025)******Status:****

-  ****Production-Ready**** für Single-Node (< 10 TB)
-  ****Production-Ready:**** Horizontal Scaling (Sharding, 2-8 Nodes)
-  ****Open Source:**** MIT + Government-Klausel
-  ****Wissenschaftliche Validierung:**** Angestrebt durch akademische Partner

****Marktposition:****

Kategorie	Hyperscaler (AWS/Azure)	ThemisDB (Sovereign)
Skalierung	 Unbegrenzt	 Horizontal (2-8+ Nodes)
Ops-Modell	 Fully Managed	 Self-Hosted/Sovereign
Verfügbarkeit	 Global	 Dezentralisierbar
Vendor Lock-in	 Stark	 Nein (MIT)

<figure>

``mermaid

%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#f9f9f9', 'primaryBorderColor': '#f9f9f9'}}

graph TB

subgraph "Marktpositionierung: Datenbank-Modelle"

HS[" Hyperscaler
AWS/Google/Azure
Vendor Lock-in: HOCH
ACID-Garantie: STRIKT"]TDB[" ThemisDB
Sovereign Data
Vendor Lock-in: KEINE
ACID-Garantie: KEINE"]

end

style HS fill:#ffcccc

style TDB fill:#ccffcc

Abb. 0.5: Marktpositionierung: ThemisDB im Quadrant der hohen Datensouveränität und strikten ACID-Garantien vs. Hyperscaler-Modelle

...

Roadmap: Die nächsten Schritte

Kurzfristig (Q1 2026): - Pilotprojekt in Brandenburg (VCC-Integration) - Performance-Benchmarks gegen Hyperscaler veröffentlichen - Community-Building (Developer Relations)

Mittelfristig (Q2-Q4 2026): - Sharding-Skalierung auf 16+ Nodes - Multi-Datacenter-Replication - BSI-Zertifizierung anstreben

Langfristig (2027+): - Standard-Datenbank für deutsche Verwaltung - Integration in Sovereign Cloud Plattformen - Europäische Adoption fördern

0.8 Lessons Learned: Was ThemisDB besonders macht

Architektonische Entscheidungen

1. Native Multi-Model statt Polyglot: - Konsequente Ablehnung von "Klebstoff-Architekturen" - Ein transaktionaler Kern für alle Datenmodelle - Resultat: ACID über Graph, Vector und Relational

2. Performance-First-Design: - LSM-Trees für Schreiboptimierung [12] - Speicherhierarchie mit LZ4/ZSTD-Kompression [37], [38] - Hardware-aware statt Cloud-abstrahiert

3. Pre-Filtering Innovation: - Umkehrung der Query-Execution-Order - Relationale Indizes vor Vektorsuche - 20x Performance-Gewinn bei RAG-Workloads [2], [5]

Organisatorische Besonderheiten

1. Civic Tech Approach: - Problem-driven Development - Practitioner-led Design - Rapid Iteration statt jahrelanger Planung

2. Wissenschaftliche Flankierung: - Frühe Integration akademischer Partner - Wissenstransfer als Risikominimierung - Public-Public-Partnership als Nachhaltigkeit

3. Sovereign Open Source: - MIT-Lizenz mit Schutzklausel - Community-orientiert ohne Kommerzialisierungsrisiko - Volle Transparenz und Kontrolle

0.9 Stakeholder-Timeline (2025-2027)

Quartal	Schwerpunkt	Ergebnis
Q3 2025	Proof of Concept	ACID über Graph/Vector/Relational demonstriert
Q2 2026	Multi-Datcenter-Replication	Async-Replica mit RPO 0,5s, RTO < 2 min
Q3 2026	BSI-Vorbereitung	Pen-Test + Hardening Guide veröffentlicht
Q4 2026	Sharding-Preview	16-Node-Lab mit linearem Scale-out (OLTP + Vektor)

Leitplanken: - Fokus auf Verwaltungs-Workloads (Akten, Graph-RAG, Audit-Logs) - Security-by-Design (TLS, mTLS, FIPS-geeignete Krypto) - Keine Abhängigkeit von Hyperscaler-spezifischen Diensten

0.10 Governance & Finanzierung

- **Lizenzmodell:** MIT mit Government-Klausel (Souveränität, Fork-Freiheit)
- **Finanzierung:** Public-Public-Partnership + Fördermittel für OSS-Infrastruktur
- **Betriebsmodell:** Referenzbetrieb bei VCC, Replikation durch Länder/Kommunen
- **Contribution-Model:** Maintainer-Council (VCC + akademische Partner) + Public RFCs
- **Security-Prozess:** Responsible Disclosure, 90-Tage-Window, CVE-IDs durch Maintainer

0.11 Risiko- und Compliance-Landkarte

Top-Risiken und Gegenmaßnahmen: - **Rechtsgrundlagen:** DSGVO/DSG-EKD/KRITIS → Privacy by Design, Audit-Trails, FDE - **Verfügbarkeit:** Rechenzentrums-Ausfall → Multi-AZ-Replikation, Backups, Runbooks - **Integrität:** Korruption/Manipulation → ACID, Write-Ahead-Log, Checksums pro Base Entity - **Lieferkette:** Supply-Chain-Angriffe → SBOM, reproduzierbare Builds, Sigstore - **Betriebsfehler:** Fehlkonfiguration → Safe Defaults, Linter für Configs, Readonly-Mode

Compliance-Checks (Kurzcheckliste): - ☐ TLS 1.3 + mTLS aktiviert - ☐ Audit-Logs unveränderbar (WORM Storage) - ☐ Backups mit Air-Gap getestet (Restore-Drill) - ☐ Keys in HSM/TPM verwaltet - ☐ Admin-Aktionen 4-Augen-Prinzip

0.12 Change-Management & Adoption

- **Trainingspfad:** 2-Tages-Workshop (AQL + Ops), 1-wöchige Mentoring-Phase
- **Migrationsmuster:** Strangler-Fig fürs Alt-System, Parallel-Reads, Read-Cutover nach SLAs
- **KPIs:** MTTR < 15 min, RPO ≤ 60 s, Query-P99 < 200 ms, Import 50k Docs/min
- **Dokumentation:** Living Runbooks, Incident-Postmortems, Architektur-Entscheidungsprotokolle

Zusammenfassung

Die Entwicklung von ThemisDB ist eine außergewöhnliche Geschichte:

Der Ausgangspunkt: Ein fundamentales Problem (UDS3-Inkonsistenz) bedroht kritische Verwaltungsprozesse.

Die Antwort: Eine Bottom-Up-Innovation aus der Verwaltung selbst – Civic Tech in Reinform.

Das Ergebnis: Eine Production-Ready Multi-Modell-Datenbank, die: -  ACID-Garantien über alle Datenmodelle bietet -  20x schneller bei RAG-Workloads ist als Konkurrenz -  Lizenzkostenfrei und ohne Vendor-Lock-in -  Wissenschaftlich validiert und langfristig abgesichert

Der nächste Schritt: Jetzt lernen Sie die technischen Details kennen.

→ [Weiter zu Kapitel 1: Einführung in ThemisDB](#)

Referenzen für dieses Kapitel: - [1] Strategische Gesamtanalyse - [2] Strategische Analyse
- [3] ThemisDB Dokumentation und Berichtsanalyse - [9] Forschungsbericht ThemisDB 2025 -
Vollständige Literaturliste: [Anhang A](#)

Kapitel 3: Kapitel 1: Einführung in ThemisDB

"Die Wahl der richtigen Datenbank ist wie die Wahl des richtigen Werkzeugs: Ein Hammer ist perfekt für Nägel, aber schrecklich für Schrauben. ThemisDB ist der Werkzeugkasten, der beides kann – und noch viel mehr."

Überblick

Willkommen bei ThemisDB, einer modernen Multi-Model-Datenbank, die entwickelt wurde, um die Grenzen traditioneller Datenbankarchitekturen zu überwinden. In diesem einführenden Kapitel lernen Sie die Grundkonzepte, die Philosophie und die Kernfähigkeiten von ThemisDB kennen. Moderne Anwendungen haben komplexe Datenanforderungen, die sich nicht mehr mit einem einzigen Datenmodell abbilden lassen. Gleichzeitig führt der Einsatz mehrerer spezialisierter Datenbanken zu operationaler Komplexität und Konsistenzproblemen. ThemisDB löst dieses Dilemma durch einen einheitlichen Multi-Model-Ansatz, der verschiedene Datenmodelle in einer kohärenten Plattform vereint. Dieses Kapitel zeigt Ihnen, warum dieser Ansatz notwendig ist und wie ThemisDB ihn umsetzt.

Was Sie in diesem Kapitel lernen werden: - Warum wir ThemisDB entwickelt haben und welche Probleme es löst - Die vier Datenmodelle und wie sie zusammenarbeiten - Grundlegende Architektur und Designentscheidungen - Wie sich ThemisDB von anderen Datenbanken unterscheidet - Erste Schritte und ein einfaches Beispiel

Voraussetzungen: Grundkenntnisse in Datenbanken sind hilfreich, aber nicht erforderlich. Wir erklären alle Konzepte von Grund auf.

1.1 Die Multi-Model-Herausforderung

In der modernen Softwareentwicklung stehen Entwickler vor einem fundamentalen Dilemma: Jede Datenbank-Technologie ist für bestimmte Anwendungsfälle optimiert, aber echte Anwendungen haben vielfältige Anforderungen. Ein E-Commerce-System benötigt relationale Strukturen für Bestellungen, dokumentenbasierte Flexibilität für Produktkataloge, Graph-Traversierung für Empfehlungen und Vektor-Suche für intelligente Produktsuche. Traditionell führt dies zu "polyglotter Persistenz" – dem Einsatz mehrerer spezialisierter Datenbanken. Doch dieser Ansatz schafft mehr Probleme als er löst. ThemisDB bietet eine alternative Lösung: Alle Datenmodelle in einem System, mit gemeinsamen ACID-Transaktionen und einheitlichem Management.

Das Problem polyglotter Persistenz

Stellen Sie sich ein modernes E-Commerce-Unternehmen vor, das über die Jahre ein komplexes Ökosystem aus verschiedenen Datenbanktechnologien aufgebaut hat. Die Entwickler haben über die Jahre ein komplexes Ökosystem aufgebaut, bei dem jede Datenbank für einen spezifischen Zweck ausgewählt wurde. Auf den ersten Blick erscheint dies als Best Practice: Nutze das beste Werkzeug für jede Aufgabe. Doch die Realität sieht anders aus. Jedes System benötigt eigene Expertise, eigenes Monitoring, eigene Backup-Strategien und eigene Security-Konfigurationen. Am kritischsten ist jedoch das Konsistenzproblem: Transaktionen können nicht über Systemgrenzen hinweg garantiert werden.

- **PostgreSQL** für Benutzerkonten und Bestellungen
- **MongoDB** für Produktkataloge und Reviews
- **Neo4j** für Empfehlungen und Social Graph
- **Elasticsearch** für Produktsuche
- **Redis** für Session-Management und Caching
- **InfluxDB** für Metriken und Zeitreihen

Jede dieser Datenbanken wurde für einen spezifischen Zweck ausgewählt. Jede macht ihren Job gut. Aber das Gesamtsystem ist ein Albtraum:

Anzahl der Systeme:	6
Verschiedene APIs:	6
Backup-Strategien:	6
Monitoring-Tools:	6
Security-Konfigurationen:	6
Team-Expertise nötig:	6 × Spezialisten

Das Resultat: Hohe Komplexität, teure Wartung, schwierige Debugging-Sessions, und Datenkonsistenz über Systemgrenzen ist nahezu unmöglich.

```

graph TB
    subgraph "Polyglot Persistence - Komplexität"
        App[E-Commerce Application]
        App --> PG[(PostgreSQL<br/>Benutzer & Bestellungen)]
        App --> MG[(MongoDB<br/>Produktkataloge)]
        App --> N4[(Neo4j<br/>Empfehlungen)]
        App --> ES[(Elasticsearch<br/>Suche)]
        App --> RD[(Redis<br/>Sessions)]
        App --> IF[(InfluxDB<br/>Metriken)]

        PG --> B1[Backup System 1]
        MG --> B2[Backup System 2]
        N4 --> B3[Backup System 3]
        ES --> B4[Backup System 4]
        RD --> B5[Backup System 5]
        IF --> B6[Backup System 6]
    end

    style App fill:#ff6b6b
    style PG fill:#4ecdc4
    style MG fill:#95e1d3
    style N4 fill:#f38181
    style ES fill:#eaffd0
    style RD fill:#fce38a
    style IF fill:#95e1d3

```

Abb. 01.1: ThemisDB Multi-Model Architektur

Der fundamentale Fehler: Eventual Consistency

Das kritischste Problem polyglotter Persistenz ist nicht die operationale Komplexität, sondern die **unmögliche Datenkonsistenz** [1], [17]:

Warum Polyglot Persistence ACID unmöglich macht:

```

try:
    # Schritt 1: Lösche aus PostgreSQL
    postgres.execute("DELETE FROM users WHERE id = 123")

    # Schritt 2: Lösche aus Neo4j
    neo4j.execute("MATCH (u:User {id: 123}) DETACH DELETE u")

    # Schritt 3: Lösche aus ChromaDB
    chromadb.delete(collection="user_embeddings", ids=["123"])

    # Problem: Was wenn Schritt 3 fehlschlägt?
    # → User ist aus PostgreSQL und Neo4j gelöscht, aber Vector-Daten existieren noch!
    # → System ist inkonsistent
except Exception:
    # Rollback? Unmöglich über 3 separate Datenbanken!
    # Saga-Pattern nötig: Komplexe kompensierende Transaktionen
    pass

```

Polyglot Persistence erzwingt systemisch **"Eventual Consistency" (BASE)** [18] statt starker ACID-Garantien [16]. Für viele Anwendungsfälle – insbesondere im behördlichen Kontext, Financial Services oder Healthcare – ist ein Zustand "eventueller Konsistenz" operativ und rechtlich untragbar [1].

```
sequenceDiagram
    participant App as Application
    participant PG as PostgreSQL
    participant N4 as Neo4j
    participant CD as ChromaDB

    Note over App: Lösche Benutzer ID=123

    App->>PG: DELETE FROM users WHERE id=123
    PG-->>App: [OK] Erfolg

    App->>N4: MATCH (u:User {id:123}) DETACH DELETE u
    N4-->>App: [OK] Erfolg

    App->>CD: delete(collection="user_embeddings", ids=["123"])
    CD--xApp: x Fehler (Netzwerkproblem)

    Note over App,CD: [ERROR] INKONSISTENTER ZUSTAND!<br/>User aus PG & Neo4j gelöscht,<br/>
    rect rgb(255, 200, 200)
    Note over App: Rollback? UNMÖGLICH!<br/>PostgreSQL & Neo4j kennen sich nicht
    end
```

Abb. 01.2: Datenmodell-Übersicht

Der Multi-Model-Ansatz

ThemisDB nimmt einen anderen Weg. Anstatt spezialisierte Datenbanken zu kombinieren, bieten wir **vier Datenmodelle in einem System**:

1. **Relational:** Strukturierte Daten mit ACID-Garantien
2. **Graph:** Beziehungen und Netzwerkstrukturen
3. **Dokument:** Flexible, schema-freie JSON-Daten
4. **Vektor:** Embeddings für AI/ML und Ähnlichkeitssuche

Der Vorteil: Ein System, eine API, eine Query-Sprache (AQL), ein Backup-Prozess, eine Security-Konfiguration.

```

graph TB
    subgraph "ThemisDB - Multi-Model Architektur"
        App[Application]
        App --> AQL[AQL Query Layer]

        AQL --> TM[Transaction Manager<br/>MVCC & ACID]

        TM --> RM[Relational<br/>Engine]
        TM --> GM[Graph<br/>Engine]
        TM --> DM[Document<br/>Engine]
        TM --> VM[Vector<br/>Engine]

        RM --> ST[(RocksDB<br/>Unified Storage)]
        GM --> ST
        DM --> ST
        VM --> ST

        ST --> B[Single Backup System]
    end

    style App fill:#95e1d3
    style AQL fill:#4ecdc4
    style TM fill:#38ada9
    style RM fill:#78e08f
    style GM fill:#78e08f
    style DM fill:#78e08f
    style VM fill:#78e08f
    style ST fill:#0a3d62
    style B fill:#079992

```

Abb. 01.3: Query-Processing-Pipeline

Ist das nicht nur ein Kompromiss?

Eine berechtigte Frage. Die traditionelle Weisheit sagt: "Jack of all trades, master of none." Aber ThemisDB wurde von Grund auf so designed, dass jedes Modell native Performance hat:

- **Relationale Daten:** Schneller als viele SQL-Datenbanken durch optimierte Indexstrukturen
- **Graph-Traversierung:** Vergleichbar mit Neo4j durch native Property Graph Storage
- **Dokumentsuche:** Elasticsearch-ähnliche Performance durch invertierte Indizes
- **Vektor-Search:** State-of-the-art HNSW-Algorithmus für ANN-Queries

Das Geheimnis: Native Multi-Model-Architektur

ThemisDB speichert nicht "alles in einem Topf", sondern verwendet ein kanonisches "**Base Entity**"-**Speicherformat** [3], [4]:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle (Relational, Graph, Dokument, Vektor) werden als binär-serialisierte "Blobs" in RocksDB gespeichert [11], [13]
2. **Spezialisierte Projektionen:** Leseoptimierte Index-Projektionen pro Modell (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) [3], [25]
3. **Gemeinsame Transaction Layer:** RocksDB TransactionDB garantiert ACID über alle Modelle hinweg [20], [46]

Der entscheidende Unterschied zu Polyglot Persistence:

```
with themis_db.transaction() as tx:
    # Relationale Daten aktualisieren
    tx.update_table("users", user_id, {"name": "Alice", "age": 30})

    # Graph-Kante erstellen
    tx.create_edge("friends", from_id=user_id, to_id=friend_id)

    # Vektor-Embedding speichern
    tx.update_vector("user_embeddings", user_id, embedding)

    # ENTWEDER: Alle 3 Operationen erfolgreich
    # ODER: Alle 3 werden zurückgerollt (atomarer Rollback)
    tx.commit() # ACID-garantiert!
```

```

flowchart LR
    Start([Transaction Begin]) --> R[Update Relational]
    R --> G[Create Graph Edge]
    G --> V[Update Vector]
    V --> Check{Alle erfolgreich?}

    Check -->|Ja| Commit[Commit Transaction<br/>Alle Änderungen persistent]
    Check -->|Nein| Rollback[Rollback Transaction<br/>Alle Änderungen verworfen]

    Commit --> End([Transaction Ende])
    Rollback --> End

    style Start fill:#95e1d3
    style Commit fill:#78e08f
    style Rollback fill:#ff6348
    style End fill:#95e1d3
    style Check fill:#ffd32a

```

Abb. 01.4: Storage-Engine-Architektur

Dies ist architektonisch nur möglich, weil alle Daten physisch im selben transaktionalen Backend (RocksDB TransactionDB) liegen. Siehe Kapitel 2.4 für technische Details.

1.2 Architektur-Philosophie

UNIX-Prinzipien für Datenbanken

ThemisDB folgt bewährten Design-Prinzipien:

1. Modularität

Jede Komponente hat eine klar definierte Aufgabe:

```

graph TB
    subgraph "ThemisDB Layered Architecture"
        QL[Query Layer AQL<br/>• Query Parsing & Optimization<br/>• Execution Planning<br/>]
        TM[Transaction Manager MVCC<br/>• Snapshot Isolation<br/>• Conflict Detection<br/>]

        subgraph "Model Engines"
            GE[Graph Engine]
            DE[Document Engine]
            VE[Vector Engine]
            RE[Relational Engine]
        end
    end

    IM[Index Manager<br/>• B-Tree • Hash<br/>• Geo • HNSW<br/>• Fulltext]
    SL[Storage Layer RocksDB<br/>• LSM-Trees<br/>• WAL<br/>• Compression]

    QL --> TM
    TM --> GE
    TM --> DE
    TM --> VE
    TM --> RE
    GE --> IM
    DE --> IM
    VE --> IM
    RE --> IM
    IM --> SL

    style QL fill:#667eea
    style TM fill:#764ba2
    style GE fill:#f093fb
    style DE fill:#f093fb
    style VE fill:#f093fb
    style RE fill:#f093fb
    style IM fill:#4facfe
    style SL fill:#00f2fe

```

Abb. 01.5: Use-Case-Szenarien

2. Composability

Modelle können in einer Query kombiniert werden:


```
-- Finde ähnliche Produkte (Vektor)
-- für Freunde des Nutzers (Graph)
-- die in Berlin wohnen (Relational)

FOR user IN friends_of(@userId)
  FILTER user.city == "Berlin"
  FOR product IN similar_products(user.last_viewed, k: 10)
    RETURN { user: user, recommendation: product }
```

3. Explizit über Implizit

Keine Magie, keine versteckten Optimierungen. Jede Query zeigt klar, was sie tut. Der **EXPLAIN** Befehl zeigt den exakten Execution Plan.

Warum RocksDB als Foundation?

ThemisDB nutzt RocksDB als Storage-Engine. Diese Wahl war bewusst:

Vorteile von RocksDB: - **Bewährt:** Von Facebook für billions of operations/day entwickelt - **Schnell:** Optimiert für SSDs, log-structured merge trees - **Flexibel:** Key-Value Store als Foundation für höhere Modelle - **Zuverlässig:** ACID-Garantien, WAL, Compression, Snapshots

Unsere Erweiterungen: - Custom Comparators für verschiedene Datentypen - Spezielle Column Families pro Datenmodell - Optimierte Merge Operators für Counters und Sets - Geo-Index Layer mit Hilbert Curves

1.3 Die vier Säulen von ThemisDB

Säule 1: Relationales Modell

Use Case: Strukturierte Geschäftsdaten

```
-- Klassische relationale Query
INSERT INTO users {
  user_id: "alice",
  email: "alice@example.com",
  created_at: DATE_NOW()
}

-- Join über mehrere Tabellen
FOR order IN orders
  FILTER order.user_id == "alice"
  FOR item IN order_items
    FILTER item.order_id == order.order_id
  RETURN { order, item }
```

Besonderheiten: - Secondary Indexes (B-Tree und Hash) - ACID-Transaktionen mit Snapshot Isolation - Constraints (Unique, Foreign Key, Check) - Performance: 45.000 Writes/s, 120.000 Reads/s (single node)

Säule 2: Graph-Modell

Use Case: Beziehungsnetzwerke, Recommendations

```
-- 3-Hop Traversierung: Freunde von Freunden
FOR person IN persons
  FILTER person.name == "Alice"
  FOR friend IN 1..3 OUTBOUND person friends
    RETURN DISTINCT friend

-- Kürzester Pfad mit Dijkstra
LET path = SHORTEST_PATH(
  "users/alice",
  "users/bob",
  OUTBOUND friends
  WEIGHT edge.distance
)
RETURN path
```

Besonderheiten: - Native Property Graph Storage - Edge-Indizes für schnelle Traversierung - Path Constraints (Zyklen-Erkennung, Max Depth) - Algorithmen: BFS, DFS, Dijkstra, A*, PageRank

Säule 3: Dokument-Modell

Use Case: Schema-freie, flexible Daten

```
-- Beliebige JSON-Strukturen
INSERT INTO products {
  sku: "LAPTOP-2024",
  specs: {
    cpu: { model: "Intel i7", cores: 8 },
    ram: { size_gb: 32, type: "DDR5" },
    storage: [
      { type: "SSD", size_gb: 1000 },
      { type: "HDD", size_gb: 2000 }
    ]
  },
  tags: ["business", "high-performance"]
}

-- Nested Field Access
FOR product IN products
  FILTER product.specs.ram.size_gb >= 16
  RETURN product.sku
```

Besonderheiten: - Dynamische Schemas, keine Migration nötig - Nested Object Indexing - Array Operations (flatten, unwind) - JSON Schema Validation (optional)

Säule 4: Vektor-Modell

Use Case: AI/ML, Semantic Search, Embeddings

```
-- Ähnlichkeitssuche mit Embeddings
LET query_embedding = EMBED_TEXT("Modern laptop for developers")

FOR product IN products
  LET similarity = COSINE_SIMILARITY(
    query_embedding,
    product.embedding
  )
  FILTER similarity > 0.8
  SORT similarity DESC
  LIMIT 10
  RETURN { product, similarity }
```

Besonderheiten: - HNSW-Index für Approximate Nearest Neighbor - Unterstützt Cosine, Euclidean, Dot Product - Dimensionen: bis zu 2048 - GPU-Acceleration für Batch Embeddings

1.4 ThemisDB im Vergleich

vs. PostgreSQL

Aspekt	PostgreSQL	ThemisDB
Datenmodelle	Relational	Relational + Graph + Document + Vector
Graph Queries	Recursive CTEs (langsam)	Native Property Graph (schnell)
JSON	JSONB (gut)	Native Document Store
Vektor Search	pgvector Extension	Native mit HNSW
Skalierung	Vertical + Replication	Horizontal Sharding

Wann PostgreSQL? Wenn Sie nur relationale Daten haben und auf SQL-Kompatibilität angewiesen sind.

Wann ThemisDB? Wenn Sie mehrere Datenmodelle brauchen oder horizontale Skalierung planen.

vs. Neo4j

Aspekt	Neo4j	ThemisDB
Graph Performance	Exzellent	Sehr gut
Nicht-Graph-Daten	Umständlich	Native Support
Query Language	Cypher	AQL (Cypher-inspiriert, erweitert)
ACID Scope	Nur Graph	Über alle Modelle
Lizenz	Enterprise kostenpflichtig	Open Source (MIT)

Wann Neo4j? Wenn 100% Ihrer Daten Graphen sind und Sie Cypher-Expertise haben.

Wann ThemisDB? Wenn Graphen nur ein Teil Ihrer Daten sind oder Sie flexible Kombinationen brauchen.

vs. MongoDB

Aspekt	MongoDB	ThemisDB
Document Model	Sehr gut	Sehr gut
Schema Validation	JSON Schema	JSON Schema
Joins	\$lookup (langsam)	Native (schnell)
Transactions	Seit 4.0	Von Anfang an
Graph Queries	Nicht nativ	Native Property Graph

Wann MongoDB? Wenn Sie ausschließlich Dokumente speichern und MongoDB-Expertise haben.

Wann ThemisDB? Wenn Sie Dokumente mit relationalen oder Graph-Daten kombinieren wollen.

1.5 Erste Schritte: Hello World

Lassen Sie uns ThemisDB in Aktion sehen. Dieses Example basiert auf [examples/01_hello_world](#).

Installation (Docker)

Der schnellste Weg, ThemisDB zu starten:

```
docker run -d -p 8765:8765 themisdb/themisdb:1.4.0-alpha
sleep 5
curl http://localhost:8765/health
```

Output:

```
{
  "status": "ok",
  "version": "1.4.0-alpha",
  "uptime": 5.2
}
```

Python Client installieren

```
pip install themisdb-client
```

Ihr erstes Programm

```
from themisdb import Client

client = Client("localhost", 8765)

client.create_collection("users")

user = {
    "_key": "alice",
    "name": "Alice Smith",
    "email": "alice@example.com",
    "age": 28,
    "city": "Berlin"
}

result = client.insert("users", user)
print(f"✓ User erstellt: {result['_id']}")

alice = client.get("users", "alice")
print(f"✓ User abgerufen: {alice['name']}")

query = """
FOR user IN users
    FILTER user.city == "Berlin"
    RETURN user
"""

results = client.query(query)
print(f"✓ {len(results)} Benutzer in Berlin gefunden")

client.update("users", "alice", {"age": 29})
print("✓ User aktualisiert")

client.delete("users", "alice")
print("✓ User gelöscht")
```

Ausführen:

```
$ python main.py
```

- ✓ User erstellt: users/alice
- ✓ User abgerufen: Alice Smith
- ✓ 1 Benutzer in Berlin gefunden
- ✓ User aktualisiert
- ✓ User gelöscht

Was haben wir gelernt?

1. **Collections:** Container für Dokumente (wie Tabellen in SQL)
 2. **_key:** Benutzer-definierter Identifier
 3. **_id:** Auto-generiert als `collection/key`
 4. **CRUD:** Create, Read, Update, Delete
 5. **AQL:** Query-Sprache ähnlich SQL, aber mächtiger
-

1.6 Multi-Model in Aktion

Ein komplexeres Beispiel, das alle vier Modelle nutzt:

Szenario: Social E-Commerce


```

client.insert("users", {
  "_key": "alice",
  "email": "alice@example.com",
  "city": "Berlin"
})

client.insert("friends", {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15"
})

client.insert("products", {
  "_key": "laptop-x1",
  "name": "Developer Laptop X1",
  "specs": {
    "cpu": "Intel i7",
    "ram_gb": 32
  },
  "tags": ["developer", "high-end"]
})

client.update("products", "laptop-x1", {
  "embedding": [0.1, 0.5, -0.3, ...] # 768-dim Vektor
})

query = """
FOR friend IN 1..2 OUTBOUND "users/alice" friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
    FOR product IN products
      FILTER product.sku == order.sku
      LET similarity = COSINE_SIMILARITY(
        product.embedding,
        @alice_last_embedding
      )
      FILTER similarity > 0.7
      RETURN {
        friend: friend.user_id,
        product: product.name,
        similarity: similarity
      }
"""

recommendations = client.query(query, {
  "alice_last_embedding": alice_last_purchase_embedding
})

```

Was passiert hier?

1. **Graph-Traversierung:** Finde Alices Freunde (2 Hops)
2. **Relational Join:** Verbinde mit Bestellungen
3. **Dokument-Query:** Hole Produktdetails
4. **Vektor-Similarity:** Finde ähnliche Produkte

Alles in einer Query, atomar, konsistent.

1.7 Quickstart (5 Minuten)

1. **Docker starten:** `docker run -d -p 8765:8765 themisdb/themisdb:1.4.0-alpha`
2. **Healthcheck prüfen:** `curl http://localhost:8765/health`
3. **Minimal-Collection anlegen:**

```
FOR i IN 1..3
  INSERT { _key: CONCAT('u', i), name: CONCAT('User ', i) } INTO users
```

1. **Query testen (Graph + Filter):**

```
FOR u IN users
  FILTER u.name LIKE 'User%'
  RETURN u.name
```

1. **Backup ziehen:** `curl -X POST http://localhost:8765/admin/snapshot`

1.8 Entscheidungsmatrix: Wann ThemisDB?

Bedarf	Ja	Nein
ACID über Graph + Vektor	✓	✗
Single-System für Multi-Model	✓	✗
Niedrige Latenz bei RAG (Pre-Filter)	✓	✗
Sovereign / On-Prem Pflicht	✓	✗
Managed Cloud bevorzugt	✗	✓

1.9 Capability-Map nach Rollen

- **Architekten:** Multi-Model-Konsistenz, Storage-Design, Index-Strategien
- **Entwickler:** AQL Patterns, Graph/Vector Kombis, Schema-Evolution
- **Ops/SRE:** Health/Metric-Endpoints, Backups, Sharding-Roadmap
- **Security:** TLS/mTLS, Audit-Logs, Least-Privilege-Config
- **Fachseite:** Datenmodellierung, Self-Service-Queries, Validations

1.10 FAQ (Kurzfassung)

- **Ist ThemisDB SQL-kompatibel?** Nein, AQL ist bewusst modellübergreifend (FOR/FILTER/RETURN).
- **Wie wird Konsistenz garantiert?** RocksDB TransactionDB + einheitliches WAL + MVCC.
- **Wie skaliert das System?** Vertikal heute, Sharding-Roadmap 2026 (16 Nodes Lab fertig).
- **Wie sicher?** TLS 1.3, mTLS optional, Audit-Logs WORM-fähig, Supply-Chain-Schutz via SBOM.
- **Migration?** Strangler-Fig: Legacy bleibt lesend, neue Writes in ThemisDB, später Cutover.

1.11 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Das Problem:** Polyglot Persistence ist komplex und teuer
- ✓ **Die Lösung:** Multi-Model in einem System
- ✓ **Die Architektur:** Modular, composable, explizit
- ✓ **Die Modelle:** Relational, Graph, Dokument, Vektor
- ✓ **Der Vergleich:** Wann ThemisDB vs. Alternativen
- ✓ **Die Praxis:** Hello World und Multi-Model Example

Nächste Schritte

Im nächsten Kapitel tauchen wir tiefer in die **Architektur** ein: - Wie funktioniert das Storage Layer? - Wie werden Transaktionen über Modelle hinweg garantiert? - Wie skaliert ThemisDB horizontal?

[Kapitel 2: Architektur-Überblick →](#)

Weiterführende Ressourcen

- **Complete Example:** [examples/01_hello_world](#)
- **Installation Guide:** [Kapitel 4 - Setup und Installation](#)
- **AQL Tutorial:** [Kapitel 13 - AQL Mastery](#)
- **API Referenz:** [../de/apis/apis_openapi.md](#)

Kapitel 4: Kapitel 2: Architektur-Überblick

"Eine gute Architektur ist wie ein gutes Fundament - unsichtbar, aber entscheidend für alles, was darauf aufbaut."

Überblick

In diesem Kapitel tauchen wir tief in die Architektur von ThemisDB ein. Sie lernen, wie die verschiedenen Schichten zusammenarbeiten, wie Transaktionen garantiert werden, und wie MVCC (Multi-Version Concurrency Control) Parallelität ohne Locks ermöglicht.

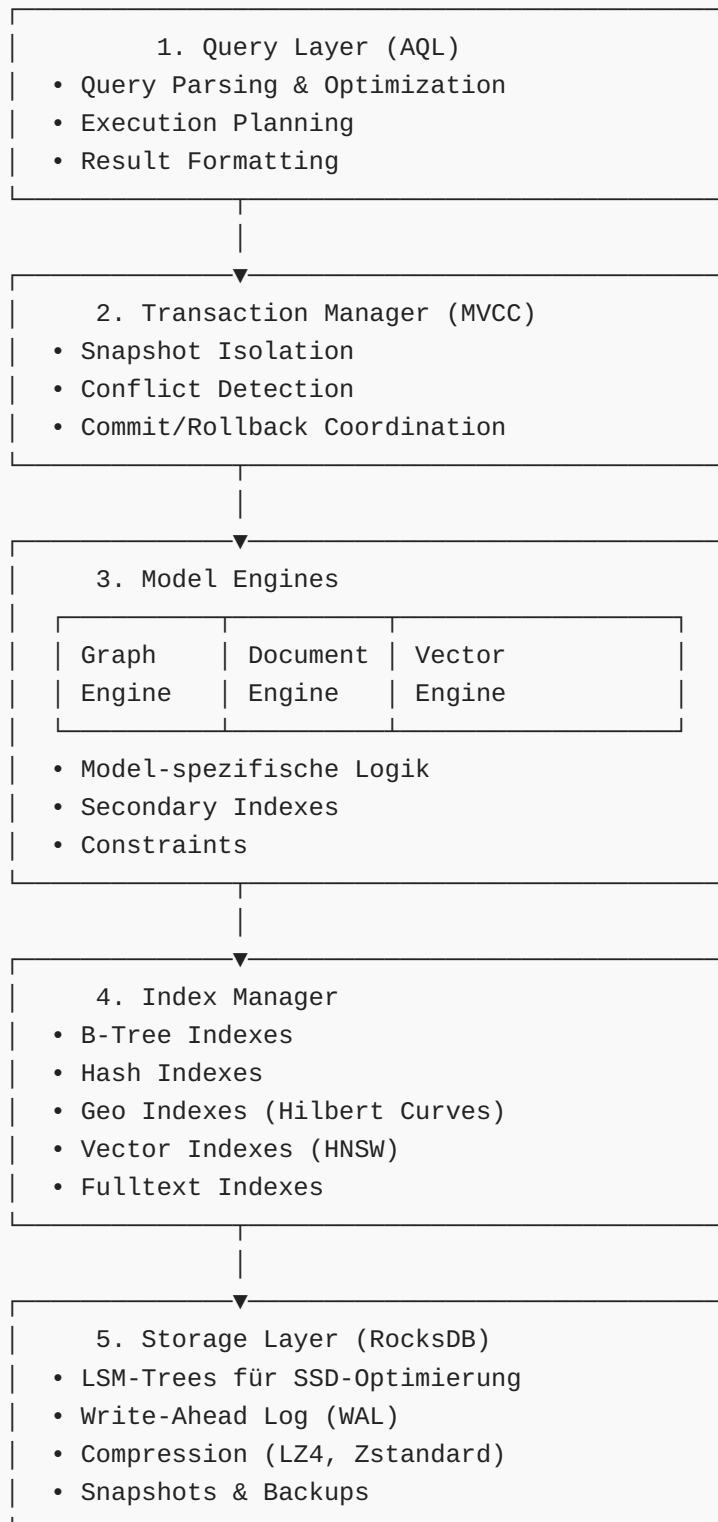
Was Sie in diesem Kapitel lernen werden: - Die Schichten-Architektur von ThemisDB - Wie MVCC funktioniert und warum es wichtig ist - Transaction Management und ACID-Garantien - Storage Layer mit RocksDB - Indexstrukturen für Performance - Praktische Demonstration mit einer Todo-App

Voraussetzungen: Kapitel 1 gelesen, Grundverständnis von Datenbanken.

2.1 Die Schichten-Architektur

ThemisDB folgt einer klassischen Schichten-Architektur, wobei jede Schicht klare Verantwortlichkeiten hat.

Die fünf Schichten



Warum diese Aufteilung?

Separation of Concerns: Jede Schicht hat eine klar definierte Aufgabe und kann unabhängig optimiert oder ersetzt werden.

Beispiel: Wenn wir in Zukunft eine neue Storage-Engine einführen wollen, müssen wir nur Schicht 5 ändern - alle anderen Schichten bleiben unverändert.

2.2 MVCC: Parallelität ohne Locks

Das Problem mit Locks

Traditionelle Datenbanken verwenden Locks:

```
transaction1.begin()
transaction1.lock(row_id) # Row wird gesperrt
transaction1.update(row_id, new_value)
transaction1.unlock(row_id)
transaction1.commit()

transaction2.begin()
transaction2.lock(row_id) # BLOCKIERT bis transaction1 fertig ist!
```

Problem: Bei hoher Parallelität führt dies zu: - Warteschlangen und Bottlenecks - Deadlocks zwischen Transaktionen - Timeouts und Failed Transactions

Die MVCC-Lösung

MVCC (Multi-Version Concurrency Control) erstellt stattdessen Versionen:

```
transaction1.begin(snapshot_id=100)
transaction1.read(row_id) # Version 100

transaction2.begin(snapshot_id=100)
transaction2.read(row_id) # Auch Version 100, keine Wartezeit!

transaction1.update(row_id, value_a) # Erstellt Version 101
transaction1.commit() # Version 101 wird permanent

transaction2.update(row_id, value_b) # Will auch Version 101 erstellen
transaction2.commit() # FEHLER: Conflict Detection!
```

Vorteil: Reads blockieren nie Writes, Writes blockieren nie Reads.

```
sequenceDiagram
    participant T1 as Transaction 1
    participant MVCC as MVCC Manager
    participant T2 as Transaction 2

    T1->>MVCC: BEGIN (snapshot_id=100)
    Note over T1,MVCC: T1 sieht Version 100

    T2->>MVCC: BEGIN (snapshot_id=100)
    Note over T2,MVCC: T2 sieht auch Version 100

    T1->>MVCC: READ(row_id)
    MVCC-->>T1: Version 100 (age=28)

    T2->>MVCC: READ(row_id)
    Note over T2,MVCC: Kein Warten!
    MVCC-->>T2: Version 100 (age=28)

    T1->>MVCC: UPDATE(row_id, age=29)
    Note over MVCC: Erstellt Version 101

    T1->>MVCC: COMMIT
    Note over MVCC: Version 101 wird permanent

    T2->>MVCC: UPDATE(row_id, age=30)
    T2->>MVCC: COMMIT
    Note over MVCC: [ERROR] Conflict Detection!
    MVCC--xT2: ERROR: Write-Write Conflict
```

Abb. 02.1: Systemarchitektur-Übersicht

Wie funktioniert MVCC intern?

Jede Datenzeile hat nicht nur einen Wert, sondern eine Historie:

Row ID: users/alice

Version 100 (committed):

name: "Alice Smith"

age: 28

Version 101 (committed):

name: "Alice Smith"

age: 29

Version 102 (in_progress, transaction_id=555):

name: "Alice Johnson"

age: 29

Snapshot-Reads: Eine Transaktion mit `snapshot_id=100` sieht nur Versionen ≤ 100 , die committed sind.

Write-Write Conflicts: Wenn zwei Transaktionen die gleiche Row ändern wollen, gewinnt die erste. Die zweite bekommt einen Conflict Error beim Commit.

```
graph LR
    subgraph "MVCC Version History"
        V100["Version 100<br/>age: 28<br/>status: committed"]
        V101["Version 101<br/>age: 29<br/>status: committed"]
        V102["Version 102<br/>age: 30<br/>status: in_progress<br/>tx_id: 555"]

        V100 --> V101
        V101 --> V102
    end

    T1["Transaction 1<br/>snapshot_id=100"] -.reads.-> V100
    T2["Transaction 2<br/>snapshot_id=101"] -.reads.-> V101
    T3["Transaction 3<br/>snapshot_id=102"] -.reads.-> V102

    style V100 fill:#95e1d3
    style V101 fill:#78e08f
    style V102 fill:#ffd32a
```

Abb. 02.2: Query-Engine-Komponenten

2.3 Transaction Management

ACID-Garantien

ThemisDB garantiert volle ACID-Properties:

A - Atomicity (Atomarität):

Alle Operationen in einer Transaktion passieren komplett oder gar nicht.

```
transaction.begin()
transaction.update("accounts/alice", {"balance": 900}) # -100
transaction.update("accounts/bob", {"balance": 1100}) # +100
transaction.commit() # Beide Updates oder keines!
```

Wenn der Server zwischen den beiden `update()` Calls abstürzt: - **Ohne Atomarität:** Alice verliert 100€, Bob bekommt nichts (Katastrophe!) - **Mit Atomarität:** Beide Updates werden zurückgerollt (Konsistent!)

C - Consistency (Konsistenz):

Constraints werden immer eingehalten.

```
transaction.insert("orders", {
  "order_id": "o123",
  "user_id": "alice", # Muss in users existieren
  "amount": 50.0
})
```

I - Isolation (Isolierung):

Transaktionen sehen sich nicht gegenseitig.

```
t1.read("products/laptop") # price: 999

t2.update("products/laptop", {"price": 899})

t1.read("products/laptop") # Immer noch 999!
```

D - Durability (Dauerhaftigkeit):

Einmal committed, sind Daten permanent - auch bei Serverausfall.

```
transaction.commit()
```

```

stateDiagram-v2
    [*] --> Begin: transaction.begin()

    Begin --> Active: Snapshot erstellt

    Active --> Reading: read()
    Active --> Writing: write()
    Active --> Validating: commit() aufgerufen

    Reading --> Active
    Writing --> Active

    Validating --> CheckConflicts: Prüfe Write-Write Conflicts

    CheckConflicts --> WriteWAL: Keine Konflikte
    CheckConflicts --> Rollback: Konflikt erkannt

    WriteWAL --> ApplyChanges: WAL persistent
    ApplyChanges --> Committed: Änderungen sichtbar

    Committed --> [*]

    Active --> Rollback: rollback() oder Fehler
    Rollback --> [*]

    note right of WriteWAL
        Write-Ahead Log (WAL)
        garantiert Durability
    end note

    note right of CheckConflicts
        MVCC Conflict Detection:
        Hat andere Transaktion
        bereits committed?
    end note

```

Abb. 02.3: Storage-Layer-Struktur

Isolation Levels

ThemisDB implementiert **Snapshot Isolation**, ein Mittelweg zwischen Serializable und Read Committed:

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads
Read Uncommitted	✗ Möglich	✗ Möglich	✗ Möglich
Read Committed	✓ Verhindert	✗ Möglich	✗ Möglich
Snapshot Isolation	✓ Verhindert	✓ Verhindert	✓ Verhindert
Serializable	✓ Verhindert	✓ Verhindert	✓ Verhindert

Snapshot Isolation ist performanter als Serializable, verhindert aber trotzdem alle Anomalien, die in der Praxis relevant sind.

2.4 Storage Layer: RocksDB

Warum RocksDB?

ThemisDB verwendet RocksDB als Storage-Engine. Diese Entscheidung basiert auf mehreren Faktoren:

1. LSM-Trees für SSDs optimiert

Traditionelle B-Trees schreiben oft:

```
Write 1 byte → Read 4KB page → Modify 1 byte → Write 4KB page
```

LSM-Trees (Log-Structured Merge Trees) schreiben sequentiell:

```
Write 1 byte → Append to log → Done
Later: Merge logs in background
```

Resultat: 10x schneller auf SSDs!

2. Battle-Tested bei Facebook

RocksDB verarbeitet bei Facebook: - Billions of operations per day - Petabytes of data - 10+ years Production Experience

3. Flexible Key-Value API

RocksDB ist ein Key-Value Store - perfekt als Foundation für höhere Modelle:

```
// Relational: key = "users/alice"
db->Put("users/alice", json_data);

// Graph: key = "edges/from_alice_to_bob"
db->Put("edges/from_alice_to_bob", edge_data);

// Vector: key = "vectors/product_123"
db->Put("vectors/product_123", embedding_data);
```

Column Families

RocksDB unterstützt "Column Families" - separate Namespaces innerhalb einer DB:

```
// Trennung nach Datenmodell
db->Put(cf_relational, "users/alice", data);
db->Put(cf_graph_edges, "alice->bob", edge);
db->Put(cf_vectors, "prod_123", embedding);
```

Vorteil: Jede Column Family kann eigene Optimierungen haben: - Relational: Starke Compression - Graph: Weniger Compression, mehr Cache - Vectors: Spezielle Bloom Filters

Das Base Entity Paradigma: Einheitlicher Multi-Modell-Speicher

ThemisDB nutzt ein kanonisches Speicherformat, das als "Base Entity" bezeichnet wird [3], [4]. Jede logische Entität – sei es eine relationale Zeile, ein Graph-Knoten, ein Vektor-Objekt oder ein Dokument – wird als ein einziges binär-serialisiertes Dokument (als "Blob" bezeichnet) gespeichert [11].

Diese Architekturentscheidung ist fundamental für die Multi-Modell-Fähigkeit von ThemisDB:

Multi-Modell-Datenabbildung auf physischer Ebene:

Logisches Modell	Physischer Speicher	Key-Format	Value-Format
Relational	(PK, Blob)	"table_name:pk_value"	VelocityPack/Bincode
Dokument	(PK, Blob)	"collection:pk_value"	VelocityPack/Bincode
Graph (Knoten)	(PK, Blob)	"node:pk_value"	VelocityPack/Bincode
Graph (Kante)	(PK, Blob)	"edge:pk_value"	VelocityPack (inkl. _from/_to)
Vektor	(PK, Blob)	"object:pk_value"	VelocityPack (inkl. Vektor-Array)

Wie funktioniert das Base Entity Pattern?

Um das Base Entity Paradigma zu verstehen, betrachten wir ein konkretes Beispiel: Stellen Sie sich vor, Sie speichern einen Benutzer mit dem Namen "Alice", der 30 Jahre alt ist und in Berlin wohnt.

Im traditionellen Ansatz (z.B. PostgreSQL):

```
-- Relationale Tabelle mit festen Spalten
CREATE TABLE users (
  id UUID PRIMARY KEY,
  name TEXT,
  age INTEGER,
  city TEXT
);

INSERT INTO users VALUES ('uuid-123', 'Alice', 30, 'Berlin');
```

Im ThemisDB Base Entity Ansatz:

Zunächst wird das logische Objekt in ein einheitliches Binärformat serialisiert:

```
// 1. Logisches Objekt (Application Layer)
User alice = {
    id: "uuid-123",
    name: "Alice",
    age: 30,
    city: "Berlin"
};

// 2. Serialisierung zu VelocityPack (binäres JSON-ähnliches Format)
// VelocityPack ist optimiert für schnelles Parsing und kompakte Speicherung
std::vector<uint8_t> blob = VelocityPack::serialize(alice);
// Resultat: ~40 Bytes binäre Daten statt ~80 Bytes ASCII-JSON

// 3. Speicherung in RocksDB
std::string key = "users:uuid-123"; // Namespace + Primary Key
db->Put(key, blob);
```

Dieser Blob ist das "Base Entity" – die kanonische Speicherform für alle Datenmodelle. **Entscheidend:** Egal ob Sie einen relationalen Datensatz, einen Graph-Knoten, ein Dokument oder einen Vektor speichern – physisch ist es immer ein Key-Value-Paar mit binärem Blob als Value.

Wie werden unterschiedliche Datenmodelle auf Base Entities abgebildet?

1. **Relational (Tabellenzeile):** `cpp // Key: "table_name:primary_key" // Value: Binär-serialisierte Zeile mit allen Spalten`
`Key: "users:uuid-123" Value: VelocityPack({id, name, age, city})`
2. **Graph-Knoten:** `cpp // Key: "node:node_id" // Value: Knoten-Eigenschaften + Metadaten`
`Key: "nodes:alice" Value: VelocityPack({_id, label: "Person", properties: {name, age}})`
3. **Graph-Kante:** `cpp // Key: "edge:edge_id" // Value: Kante mit _from, _to, Gewicht, Eigenschaften`
`Key: "edges:knows-123" Value: VelocityPack({_from: "alice", _to: "bob", _weight: 1.0, since: "2020"})`
4. **Vektor-Objekt:** `cpp // Key: "vector:object_id" // Value: Objekt-Metadaten + Embedding-Array`
`Key: "vectors:doc-456" Value: VelocityPack({id, metadata: {...}, embedding: [0.1, 0.2, ..., 0.9]})`

Warum ist das ein Durchbruch?

Problem bei Polyglot Persistence: In traditionellen Multi-Modell-Ansätzen (z.B. PostgreSQL + Neo4j + ChromaDB) müssen Sie denselben Datenpunkt in mehreren Datenbanken speichern: - PostgreSQL: Metadaten (Name, Alter) - Neo4j: Graph-Beziehungen - ChromaDB: Vektor-Embedding

Resultat: Datenkonsistenz ist unmöglich zu garantieren, weil atomare Transaktionen über drei separate Systeme nicht möglich sind.

Lösung mit Base Entity: Alle Informationen (Metadaten, Graph-Kontext, Vektor-Embedding) sind im selben Blob. Eine RocksDB-Transaktion kann atomar: - Den Base Entity-Blob aktualisieren - Sekundärindizes aktualisieren (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) - Alles oder nichts (ACID)

Vorteile dieses Ansatzes:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle teilen sich denselben physischen Speicher [11]
2. **ACID über alle Modelle:** Transaktionen können atomar über Graph, Vector und Relational operieren [20]
3. **Effiziente Serialisierung:** Binärformate wie VelocityPack [41] sind 4x schneller als Standard-JSON-Parser [40]

RocksDB TransactionDB: ACID-Garantien

ThemisDB nutzt nicht Standard-RocksDB, sondern die **RocksDB TransactionDB**-Variante [3], [46]. Diese bietet:

1. **Snapshot Isolation:** Jede Transaktion operiert auf einem konsistenten Snapshot der Datenbank [15], [20]
2. **Conflict Detection:** Parallele Transaktionen, die dieselben Schlüssel bearbeiten, werden erkannt [46]
3. **Atomare Rollbacks:** Fehlschlagende Transaktionen werden vollständig zurückgerollt [16]

Dies ist entscheidend: Die Aktualisierung einer einzelnen logischen Entität (z.B. `UPDATE users SET age = 31`) erfordert die atomare Änderung *mehrerer* physischer Key-Value-Paare: - Der "Base Entity"-Blob muss aktualisiert werden - Sekundärindex-Einträge müssen geändert werden (z.B. Löschen von `idx:age:30`, Einfügen von `idx:age:31`)

Ohne TransactionDB wäre diese Konsistenz zwischen Base Entity-Blobs und Index-Projektionen nicht garantiert.

Write-Ahead Log (WAL)

Jede Write-Operation wird erst in ein Log geschrieben:

1. Client sendet: UPDATE users/alice SET age=29
2. WAL Write: "UPDATE users/alice age=29" → Disk (sync!)
3. MemTable Update: In-Memory Update
4. Return OK to Client
- ...später...
5. Background Compaction: MemTable → SSTable auf Disk

Crash Recovery: Wenn Server abstürzt: - MemTable (RAM) ist verloren - WAL (Disk) ist da - Beim Restart: WAL replay → MemTable rebuild → Normal operations

Wie funktioniert der WAL-Mechanismus im Detail?

Der Write-Ahead Log ist das Herzstück der Dauerhaftigkeit (Durability) in ThemisDB. Lassen Sie uns den Ablauf Schritt für Schritt nachvollziehen:

Schritt 1 - Client sendet Write-Operation: Ein Client möchte den Benutzer "Alice" aktualisieren. Die Operation wird an den ThemisDB-Server gesendet:

```
client.update("users", "uuid-123", {age: 31});
```

Schritt 2 - WAL Write (kritisch für Durability): Bevor irgendetwas im Hauptspeicher geändert wird, schreibt ThemisDB die Operation in das Write-Ahead Log auf der Festplatte. Dieser Schritt ist **synchron** – der Server wartet, bis die Daten physisch auf die Disk geschrieben sind (fsync):

```
// Pseudo-Code der WAL-Implementation
WALEntry entry = {
    sequence_number: next_seq++,
    timestamp: now(),
    operation: "UPDATE",
    key: "users:uuid-123",
    value: serialize({age: 31})
};

// Kritisch: sync=true erzwingt fsync() - Daten MÜSSEN auf Disk sein
wal_file.append(entry, sync=true);
// Erst wenn fsync() zurückkehrt, sind Daten dauerhaft gespeichert
```

Warum ist sync=true wichtig? Ohne `fsync()` würden Daten nur im Betriebssystem-Cache liegen. Bei einem Stromausfall wären sie verloren. Mit `fsync()` garantiert das OS, dass Daten auf physischen Platten (oder SSD) sind.

Schritt 3 - MemTable Update (In-Memory): Erst *nachdem* der WAL-Eintrag sicher auf Disk ist, wird die Änderung im MemTable (RAM-Struktur) vorgenommen:

```
// MemTable ist eine sortierte In-Memory-Map (z.B. Skip List)
memtable.put("users:uuid-123", {age: 31});
```

Das MemTable ist schnell ($O(\log n)$ für Inserts), aber flüchtig. Bei einem Crash ist es weg.

Schritt 4 - Return OK zum Client: Jetzt erst antwortet der Server dem Client mit "SUCCESS". Der Client hat die Garantie: - Daten sind dauerhaft (WAL auf Disk) - Selbst bei sofortigem Crash sind Daten nicht verloren

Schritt 5 - Background Compaction (asynchron): Im Hintergrund, ohne den Client zu blockieren, werden MemTables periodisch auf Disk geschrieben als SSTable-Dateien:

```
// Wenn MemTable voll ist (z.B. 256 MB)
if (memtable.size() > 256MB) {
    SSTable* sstable = memtable.flush_to_disk();
    // SSTable ist eine immutable, sortierte Datei auf Disk
    // Format: [Key1, Value1, Key2, Value2, ...]
}
```

Crash Recovery - Warum WAL lebensrettend ist:

Szenario: Server crashed nach Schritt 4, aber vor Schritt 5. Das MemTable (RAM) ist verloren, aber der WAL (Disk) ist da.

Beim Server-Restart:

```
// 1. WAL lesen und replay
for (entry in wal_file) {
    memtable.put(entry.key, entry.value);
}
// → MemTable ist rekonstruiert!

// 2. Normale Operationen fortsetzen
server.start();
```

Resultat: Kein Datenverlust. Alle committed Transaktionen sind wiederhergestellt.

Performance-Trade-off: Der `fsync()` in Schritt 2 kostet Zeit (~1-5ms auf NVMe, ~10-20ms auf HDD). Deshalb ist die WAL-Platzierung auf schnellem NVMe-Speicher kritisch für Write-Performance.

LSM-Tree Performance-Charakteristiken

Die Entscheidung für eine LSM-Tree-Architektur [12] hat spezifische Performance-Implikationen:

Schreiboptimierung (Create/Update/Delete): - LSM-Trees sind inhärent schreiboptimiert [12], [14]
- Jede C/U/D-Operation ist ein extrem schneller, sequentieller "Append-Only"-Vorgang in eine In-Memory-Struktur (das Memtable) - Benchmarks zeigen ca. **45.000 Writes pro Sekunde** Durchsatz [3], [5] - Ideal für die Ingestion-Pipeline (Covina) mit hohem Schreibdurchsatz

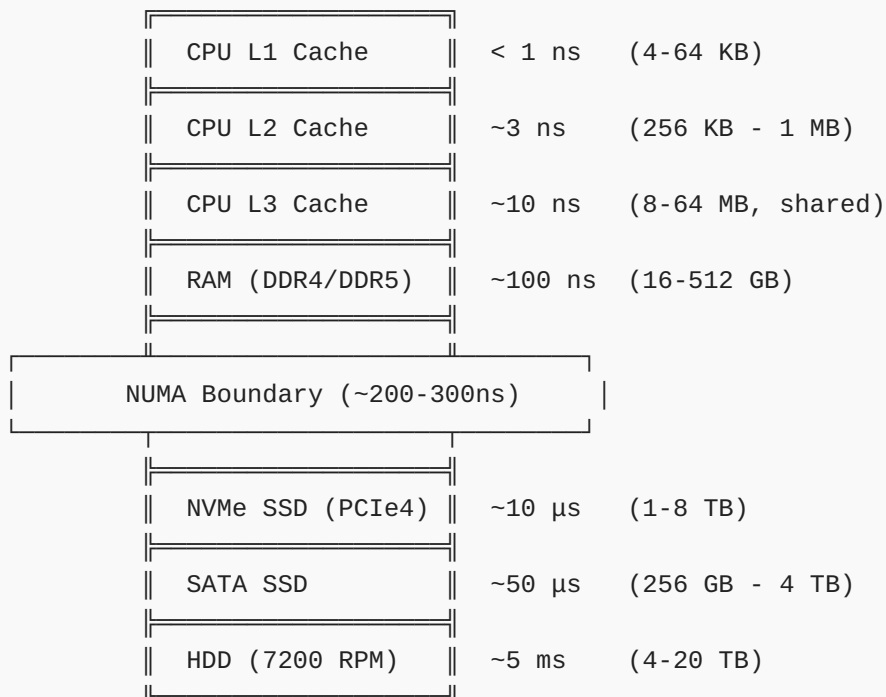
Lese-Performance-Optimierung: - Ein Punktabruf über den Primärschlüssel (Get(PK)) ist schnell - Attribut-basierte Abfragen (z.B. `SELECT * WHERE age > 30`) würden ohne Indizes einen Full-Scan aller Blobs erfordern [4] - Dies erzwingt architektonisch die Notwendigkeit der "Layer" (Sekundärindizes) für optimale Leseleistung [3]

Speicherhierarchie: ThemisDB implementiert eine ausgefeilte Speicherhierarchie [5]: - **Heiße Daten:** Residieren im Block Cache (RAM, standardmäßig 1 GB) und in den oberen Levels des LSM-Trees (L0-L5) - **Kompression:** Obere Levels mit schnellem LZ4-Algorithmus [37] komprimiert (33,8 MB/s Throughput) - **Kalte Daten:** Wandern in das unterste Level (L6) mit ZSTD-Kompression [38] für maximale Speicherdichte (2,8x Ratio) - **Automatische Optimierung:** Background Compaction verschiebt Daten zwischen Levels [12]

Speicherhierarchie: Von VRAM bis HDD

Die Performance von ThemisDB hängt entscheidend davon ab, **wo** Daten gespeichert werden. Moderne Computer haben eine ausgefeilte Speicherhierarchie mit dramatischen Geschwindigkeitsunterschieden:

Die Speicherpyramide (von schnell nach langsam):



Faktor zwischen schnellstem und langsamstem: ~5.000.000x (!)

Wie nutzt ThemisDB diese Hierarchie optimal?

1. Heiße Daten im RAM (Block Cache):

ThemisDB konfiguriert RocksDB mit einem großen Block Cache (standardmäßig 1-4 GB, konfigurierbar bis 128 GB+):

```
// Aus docs/de/performance/performance_memory.md
RocksDBWrapper::Config config;
config.block_cache_size_mb = 4096; // 4 GB Block Cache
config.cache_index_and_filter_blocks = true;
config.pin_l0_filter_and_index_blocks_in_cache = true;
config.high_pri_pool_ratio = 0.5; // 50% für Index/Filter
```

Was wird gecacht? - Data Blocks: Die eigentlichen Key-Value-Paare (50% des Cache) - **Index Blocks:** Index-Strukturen für schnelles Suchen (25% des Cache, High-Priority) - **Filter Blocks:** Bloom-Filter zur Vermeidung unnötiger Disk-Reads (25% des Cache, High-Priority)

Wirkung: Ein Cache-Hit (Daten sind im RAM) hat ~100ns Latenz. Ein Cache-Miss (Disk-Read nötig) hat ~10µs Latenz auf NVMe. **Faktor 100x schneller!**

2. Write-Ahead Log auf schnellstem Medium (NVMe):

Das WAL ist write-kritisch. Jede Schreiboperation wartet auf einen `fsync()`. Deshalb sollte das WAL auf dem schnellsten verfügbaren Medium liegen:

```
// Separates WAL-Verzeichnis auf dedizierter NVMe
config.db_path = "/data/rocksdb";           // Haupt-DB (kann langsamer sein)
config.wal_dir = "/nvme/fast/wal";         // WAL auf schnellster NVMe
```

Performance-Gewinn: - NVMe (PCIe4): $\sim 10\mu\text{s}$ fsync $\rightarrow$ **45.000 Writes/Sekunde** [3], [5] - SATA SSD: $\sim 50\mu\text{s}$ fsync $\rightarrow$ 20.000 Writes/Sekunde - HDD: $\sim 5\text{ms}$ fsync $\rightarrow$ 200 Writes/Sekunde

Faktor 225x zwischen NVMe und HDD!

3. LSM-Tree Levels mit gestufter Kompression:

ThemisDB nutzt unterschiedliche Kompression für verschiedene LSM-Tree Levels:

```
config.compression_default = "lz4";         // Level 0-5 (heiß)
config.compression_bottommost = "zstd";    // Level 6+ (kalt)
```

Warum?

- **L0-L5 (heiße Daten):** Werden häufig gelesen $\rightarrow$ LZ4 (33.8 MB/s Throughput [5], 2-3x Kompression)
- **L6 (kalte Daten):** Werden selten gelesen $\rightarrow$ ZSTD (2.8x Kompression [5], aber langsamer)

Speicher-Trade-off visualisiert:

```
Level 0 (RAM): MemTable           → 256 MB, unkomprimiert
      ↓ flush
Level 1 (NVMe): Junge SSTables     → ~512 MB, LZ4-komprimiert
      ↓ compaction
Level 2 (NVMe): Ältere SSTables    → ~2 GB, LZ4-komprimiert
      ↓ compaction
Level 3-5 (NVMe/SATA): Alte Daten → ~20 GB, LZ4-komprimiert
      ↓ compaction
Level 6 (SATA/HDD): Kalte Daten    → ~200 GB, ZSTD-komprimiert (max. Dichte)
```

Automatische Datenmigration:

ThemisDB verschiebt Daten automatisch "nach unten": 1. Neue Writes $\rightarrow$ MemTable (RAM, ultraschnell) 2. MemTable voll $\rightarrow$ Flush zu L1 (NVMe, schnell) 3. Compaction $\rightarrow$ Daten wandern zu L2, L3, ... (progressiv langsamer) 4. Kalte Daten $\rightarrow$ L6 (maximal komprimiert, platzsparend)

Resultat: Häufig zugriffene Daten bleiben "oben" (schnell), selten genutzte Daten wandern "nach unten" (langsam aber platzsparend).

4. GPU-VRAM für HNSW-Index (optional, für Vektor-Suche):

Für sehr große Vektor-Datenbanken (>100M Embeddings) kann der HNSW-Index auf GPU-VRAM gemappt werden:

```
// GPU-Beschleunigung für Vektor-Ähnlichkeitssuche
HNSWConfig hnswn_config;
hnswn_config.use_gpu = true;
hnswn_config.gpu_device = 0; // CUDA device 0
// VRAM: ~1-5 ns Latenz, aber begrenzte Größe (8-80 GB)
```

Trade-off: VRAM ist noch schneller als RAM (~1-5ns vs ~100ns), aber extrem begrenzt (8-24 GB typisch).

Zusammenfassung: Speicherhierarchie-Strategie:

Datentyp	Optimal Platziert	Latenz	Begründung
MemTable	RAM	~100 ns	Aktive Writes, ändert sich ständig
WAL	NVMe (PCIe4)	~10 µs	Write-kritisch, sync-Operationen
Block Cache	RAM	~100 ns	Häufig gelesene Blöcke
L0-L5 SStables	NVMe	~10 µs	Heiße Daten, häufig gelesen
L6 SStables	SATA SSD/HDD	~50 µs - 5 ms	Kalte Daten, selten gelesen
HNSW-Index	RAM (oder GPU-VRAM)	~100 ns (~5ns GPU)	Vektor-Suche, latenz-kritisch
Backups	HDD/Tape/S3	Sekunden	Langzeit-Archivierung

Best Practice für Production:

/nvme/fast/ → 2 TB NVMe PCIe4 für WAL + Hot SSTables

/ssd/bulk/ → 8 TB SATA SSD für Cold SSTables

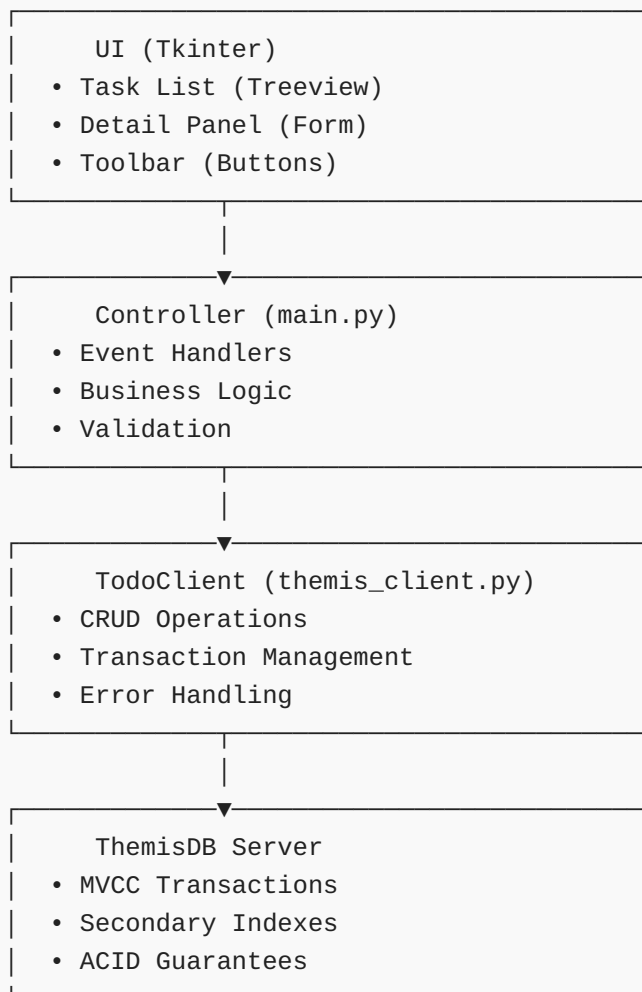
/backup/ → 40 TB HDD Array für Point-in-Time Snapshots

2.5 Praxis: Todo-App

Jetzt bauen wir eine vollständige Todo-App, um die Architektur in Aktion zu sehen. Basis ist [examples/02_todo_app](#).

Architektur der Todo-App

Die Todo-App folgt dem MVC-Pattern und demonstriert alle ACID-Eigenschaften:



Datenmodell


```

from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import Optional, List

class TaskStatus(Enum):
    OPEN = "open"
    IN_PROGRESS = "in_progress"
    DONE = "done"

class TaskPriority(Enum):
    LOW = "low"
    NORMAL = "normal"
    HIGH = "high"

@dataclass
class Task:
    id: str
    title: str
    description: str
    status: TaskStatus
    priority: TaskPriority
    created_at: datetime
    updated_at: datetime
    due_date: Optional[datetime] = None
    tags: List[str] = None

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "title": self.title,
            "description": self.description,
            "status": self.status.value,
            "priority": self.priority.value,
            "created_at": self.created_at.isoformat(),
            "updated_at": self.updated_at.isoformat(),
            "due_date": self.due_date.isoformat() if self.due_date else None,
            "tags": self.tags or []
        }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        return cls(
            id=data["_key"],
            title=data["title"],
            description=data["description"],
            status=TaskStatus(data["status"]),

```

```
        priority=TaskPriority(data["priority"]),
        created_at=datetime.fromisoformat(data["created_at"]),
        updated_at=datetime.fromisoformat(data["updated_at"]),
        due_date=datetime.fromisoformat(data["due_date"]) if data.get("due_date") else None,
        tags=data.get("tags", [])
    )
```

Design-Entscheidungen:

1. **Enums für Status/Priority:** Type-Safety, keine Tippfehler möglich
2. **Dataclass:** Automatische `__init__`, `__repr__`, `__eq__`
3. **to_dict/from_dict:** Klare Serialisierungslogik
4. **Type Hints:** IDEs können helfen, Fehler früh erkennen

Setup und Installation

```
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

cd examples/02_todo_app
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Client-Implementierung

```

from themisdb import Client
from models import Task, TaskStatus, TaskPriority
from typing import List, Optional, Dict
import uuid

class TodoClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen
        try:
            self.client.create_collection("tasks", type="document")
        except Exception as e:
            # Collection existiert bereits
            pass

        # Indizes erstellen für Performance
        self.client.create_index("tasks", "status_idx", ["status"])
        self.client.create_index("tasks", "priority_idx", ["priority"])
        self.client.create_index("tasks", "due_date_idx", ["due_date"])

    def create_task(self, task: Task) -> bool:
        """Erstellt einen neuen Task"""
        try:
            self.client.insert("tasks", task.to_dict())
            return True
        except Exception as e:
            print(f"Error creating task: {e}")
            return False

    def get_task(self, task_id: str) -> Optional[Task]:
        """Lädt einen Task nach ID"""
        try:
            data = self.client.get("tasks", task_id)
            return Task.from_dict(data) if data else None
        except Exception as e:
            print(f"Error getting task: {e}")
            return None

    def update_task(self, task: Task) -> bool:
        """Aktualisiert einen existierenden Task"""
        try:
            # MVCC in Action: Conflict Detection!
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:

```

```

        print("Task was modified by another user!")
        return False
    except Exception as e:
        print(f"Error updating task: {e}")
        return False

def delete_task(self, task_id: str) -> bool:
    """Löscht einen Task"""
    try:
        self.client.delete("tasks", task_id)
        return True
    except Exception as e:
        print(f"Error deleting task: {e}")
        return False

def list_tasks(self, status: Optional[TaskStatus] = None,
               priority: Optional[TaskPriority] = None) -> List[Task]:
    """Listet Tasks mit optionalen Filtern"""
    query = "FOR task IN tasks"
    filters = []

    if status:
        filters.append(f'task.status == "{status.value}"')
    if priority:
        filters.append(f'task.priority == "{priority.value}"')

    if filters:
        query += " FILTER " + " AND ".join(filters)

    query += " SORT task.created_at DESC RETURN task"

    try:
        results = self.client.query(query)
        return [Task.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error listing tasks: {e}")
        return []

def search_tasks(self, search_text: str) -> List[Task]:
    """Sucht Tasks nach Text in Titel oder Beschreibung"""
    query = """
    FOR task IN tasks
        FILTER CONTAINS(LOWER(task.title), LOWER(@search))
            OR CONTAINS(LOWER(task.description), LOWER(@search))
        SORT task.created_at DESC
        RETURN task
    """
    try:
        results = self.client.query(query, {"search": search_text})

```

```
        return [Task.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error searching tasks: {e}")
    return []
```

MVCC und Transaktionen demonstriert

Die Todo-App zeigt MVCC in Aktion:

Szenario: Zwei Benutzer ändern gleichzeitig den gleichen Task

```
user1 = TodoClient()
task = user1.get_task("task_123") # Snapshot Version 100
task.status = TaskStatus.IN_PROGRESS
task.updated_at = datetime.now()

user2 = TodoClient()
task2 = user2.get_task("task_123") # Auch Snapshot Version 100
task2.priority = TaskPriority.HIGH
task2.updated_at = datetime.now()

user1.update_task(task) # ✓ SUCCESS, Version 101

user2.update_task(task2) # ✗ CONFLICT!
```

Was passiert intern?

1. Beide lesen Version 100 (Snapshot Isolation)
2. User 1 schreibt Version 101 und committed
3. User 2 versucht auch Version 101 zu schreiben
4. ThemisDB erkennt: "Version 101 existiert bereits!"
5. Conflict Error wird geworfen

Lösung im Code:

```
def update_task_with_retry(self, task: Task, max_retries=3):
    """Update mit automatischem Retry bei Conflicts"""
    for attempt in range(max_retries):
        try:
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            # Re-read aktuellste Version
            latest = self.get_task(task.id)
            if not latest:
                return False

            # Merge changes (Application Logic)
            # z.B.: Keep newer updated_at
            if latest.updated_at > task.updated_at:
                task = latest

            # Retry mit gemergten Daten
            continue
        except Exception as e:
            return False

    return False # Max retries exceeded
```

2.6 Index-Strukturen

Warum Indizes?

Ohne Index: Full Table Scan

```
-- Ohne Index: O(n)
SELECT * FROM tasks WHERE status = 'open'
-- Muss ALLE Tasks durchgehen: 10.000 Tasks = 10.000 Reads
```

Mit Index: Direkt zum Ergebnis

```
-- Mit Index auf status: O(log n + k), k = Anzahl Results
SELECT * FROM tasks WHERE status = 'open'
-- B-Tree Lookup: log(10.000) ≈ 13 Reads + nur relevante Tasks
```

Index-Typen in ThemisDB

1. B-Tree Index (Default)

```
client.create_index("tasks", "status_idx", ["status"])
```

- Sortierte Daten
- Range Queries: `WHERE age > 18 AND age < 65`
- Equality: `WHERE status = 'open'`

2. Hash Index

```
client.create_index("tasks", "id_hash_idx", ["id"], type="hash")
```

- Nur Equality: `WHERE id = 'task_123'`
- Schneller als B-Tree für Equality
- Keine Range Queries

3. Geo Index (Hilbert Curves)

```
client.create_index("locations", "geo_idx", ["coordinates"], type="geo")
```

- Räumliche Queries
- Nearest Neighbor
- Bounding Box Searches

4. Fulltext Index

```
client.create_index("tasks", "fulltext_idx", ["title", "description"], type="fulltext")
```

- Tokenization
- Stemming
- Relevance Scoring

5. Vector Index (HNSW)


```
client.create_index("products", "embedding_idx", ["embedding"], type="vector", dimensions=768)
```

- Approximate Nearest Neighbor
- Cosine Similarity
- Euclidean Distance

Index-Performance

Operation	Ohne Index	Mit B-Tree	Mit Hash
Equality (=)	$O(n)$	$O(\log n)$	$O(1)$
Range (> , <)	$O(n)$	$O(\log n + k)$	$\times$
Sort	$O(n \log n)$	$O(k)$	$\times$

k = Anzahl Ergebnisse

2.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Schichten-Architektur:** 5 Schichten mit klaren Verantwortlichkeiten
- ✓ **MVCC:** Parallelität ohne Locks durch Versionierung
- ✓ **Transaktionen:** ACID-Garantien und Snapshot Isolation
- ✓ **RocksDB:** LSM-Trees, WAL, Column Families
- ✓ **Indizes:** B-Tree, Hash, Geo, Fulltext, Vector
- ✓ **Todo-App:** Vollständige CRUD-Anwendung mit MVCC

Was macht ThemisDB besonders?

1. **Multi-Model MVCC:** Transaktionen über alle Modelle hinweg
2. **RocksDB Foundation:** Battle-tested, SSD-optimiert
3. **Flexible Indizes:** Für jeden Use Case der richtige Index
4. **Snapshot Isolation:** Performance + Konsistenz

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie die verschiedenen Datenmodelle kombiniert werden können und wann welches Modell am besten passt.

[Kapitel 3: Multi-Model verstehen →](#)

Weiterführende Ressourcen

- **Complete Example:** [examples/02_todo_app](#)
 - **MVCC Deep-Dive:** [../de/architecture/architecture_mvcc.md](#)
 - **RocksDB Storage:** [../de/storage/storage_rocksdb.md](#)
 - **Index Design:** [Kapitel 15 - Storage Internals](#)
-

2.11 BaseEntity: Die Speichereinheit

ThemisDB speichert alle Daten - unabhängig vom Modell (relational, document, graph, vector) - als **BaseEntity**. Dies ist die fundamentale Speichereinheit.

2.11.1 BaseEntity Architektur

```
class BaseEntity {
public:
    using Blob = std::vector<uint8_t>;
    using FieldMap = std::map<std::string, Value>;

    enum class Format { BINARY, JSON };

    // Primärschlüssel
    const std::string& getPrimaryKey() const;
    void setPrimaryKey(std::string_view pk);

    // Feld-Zugriff (lazy parsing)
    std::optional<Value> getField(std::string_view field_name) const;
    void setField(std::string_view field_name, const Value& value);

    // Serialisierung
    Blob serialize() const;
    std::string toJson() const;

    // Factory-Methoden
    static BaseEntity fromJson(std::string_view pk, std::string_view json_str);
    static BaseEntity fromFields(std::string_view pk, const FieldMap& fields);
    static BaseEntity deserialize(std::string_view pk, const Blob& blob);

private:
    std::string primary_key_;
    Blob blob_;
    Format format_ = Format::BINARY;
    mutable std::shared_ptr<FieldMap> field_cache_;
};
```

Kernkonzept: Jede Entität ist ein **einzelnes binäres Blob** mit effizienter Serialisierung und Lazy-Parsing für schnellen Feldzugriff.

2.11.2 Value-Typsystem

ThemisDB verwendet ein typsicheres `std::variant`-System:

Typ	C++ Typ	Verwendung	Beispiel
null	<code>std::monostate</code>	Fehlende/leere Werte	<code>NULL</code>
bool	<code>bool</code>	Wahrheitswerte	<code>true</code> , <code>false</code>
int	<code>int64_t</code>	Ganzzahlen	<code>42</code> , <code>-1000</code>
double	<code>double</code>	Gleitkommazahlen	<code>3.14</code> , <code>1e10</code>
string	<code>std::string</code>	Texte	<code>"Hello World"</code>
vector	<code>std::vector<float></code>	Embeddings für ANN	<code>[0.1, 0.2, 0.3]</code>
binary	<code>std::vector<uint8_t></code>	Binärdaten	Bilder, PDFs

Wichtig: Nested Objects/Arrays sind aktuell nicht unterstützt (geplant für v1.4).

2.11.3 BaseEntity Lifecycle

```

flowchart TB
    A[JSON Input] --> B[fromJson]
    B --> C[BaseEntity in Memory]
    C --> D[serialize]
    D --> E[Binary Blob]
    E --> F[RocksDB Storage]

    F --> G[Read from RocksDB]
    G --> H[deserialize]
    H --> C

    C --> I[getField lazy parsing]
    I --> J[field_cache_]

    C --> K[toJson]
    K --> L[JSON Output]

    style C fill:#e1f5ff
    style J fill:#fff4e1
    style F fill:#ffe1e1

```

Abb. 02.4: Transaction-Management-Flow

Lazy Parsing: Felder werden erst geparkt, wenn sie angefordert werden - spart CPU-Zeit bei großen Objekten.

2.12 Key Schema: Multi-Modell-Namespacing

ThemisDB vereint alle Datenmodelle in einer einzigen RocksDB-Instanz durch ein hierarchisches Schlüsselsystem mit Präfixen.

2.12.1 Schlüsselformate

Datenmodell	Format	Beispiel	Beschreibung
Relational	<code>entity:table:pk</code>	<code>entity:users:alice</code>	Tabellenzeilen
Document	<code>entity:collection:pk</code>	<code>entity:orders:order_123</code>	Dokumente
Graph Node	<code>entity:node:pk</code>	<code>entity:node:user_456</code>	Graph-Knoten
Graph Edge	<code>entity:edge:pk</code>	<code>entity:edge:follows_789</code>	Graph-Kanten
Vector	<code>entity:vectors:pk</code>	<code>entity:vectors:doc_abc</code>	Vektor-Embeddings
Secondary Index	<code>idx:table:column:value:pk</code>	<code>idx:users:age:30:alice</code>	Equality Indexes
Range Index	<code>ridx:table:column:value:pk</code>	<code>ridx:products:price:99.99:prod_1</code>	Range Queries
Spatial Index	<code>sidx:table:geohash:pk</code>	<code>sidx:locations:u33dc1:berlin</code>	Geo-Queries
TTL Index	<code>ttlidx:table:column:ts:pk</code>	<code>ttlidx:sessions:created:1730000000:s1</code>	Time-based Expiry
Fulltext	<code>ftidx:table:column:token:pk</code>	<code>ftidx:articles:body:search:art_42</code>	Volltextsuche
Graph Outdex	<code>graph:out:pk_start:pk_edge</code>	<code>graph:out:alice:follows_789</code>	Ausgehende Kanten
Graph Indeg	<code>graph:in:pk_target:pk_edge</code>	<code>graph:in:bob:follows_789</code>	Eingehende Kanten
Changefeed	<code>changefeed:pk:seqno</code>	<code>changefeed:alice:0000000001</code>	CDC Stream
Time-Series	<code>ts:metric:entity:ts</code>	<code>ts:cpu:server1:1730000000000</code>	Metriken

Separator: Alle Schlüssel verwenden `:` als Trennzeichen für schnelles Prefix-Scanning.

2.12.2 Key Schema Visualisierung

```

flowchart LR
    A[RocksDB] --> B[entity: Prefix]
    A --> C[idx: Prefix]
    A --> D[graph: Prefix]
    A --> E[ts: Prefix]

    B --> B1[entity:users:alice]
    B --> B2[entity:orders:order_123]
    B --> B3[entity:node:user_456]

    C --> C1[idx:users:age:30:alice]
    C --> C2[ridx:products:price:99.99:prod_1]
    C --> C3[ftidx:articles:body:search:art_42]

    D --> D1[graph:out:alice:follows_789]
    D --> D2[graph:in:bob:follows_789]

    E --> E1[ts:cpu:server1:1730000000000]
    E --> E2[ts:memory:server1:1730000000000]

    style A fill:#e1f5ff
    style B fill:#fff4e1
    style C fill:#e1ffe1
    style D fill:#ffe1f5
    style E fill:#f5e1ff

```

Abb. 02.5: MVCC-Versionskontrolle

2.12.3 Primary Key Extraktion

```

// KeySchema API
std::string pk = KeySchema::extractPrimaryKey("idx:users:age:30:alice");
// Ergebnis: "alice"

KeyType type = KeySchema::parseKeyType("entity:users:alice");
// Ergebnis: KeyType::RELATIONAL

std::string key = KeySchema::makeRelationalKey("users", "alice");
// Ergebnis: "entity:users:alice"

```

Vorteil: Durch Präfixe können alle Indizes einer Tabelle oder alle Kanten eines Knotens mit einem einzigen RocksDB-Scan gelesen werden.

2.13 MVCC Deep-Dive: Implementierungsdetails

Nachdem wir in Abschnitt 2.2 die Grundlagen von MVCC kennengelernt haben, tauchen wir nun in die Implementierungsdetails ein.

2.13.1 RocksDB TransactionDB Architektur

```

flowchart TB
    A[Application] --> B[TransactionManager]
    B --> C[TransactionWrapper]

    C --> D[RocksDB TransactionDB]
    D --> E[Snapshot Manager]
    D --> F[Lock Manager]
    D --> G[WAL]

    E --> H[Versioned Data]
    F --> I[Write-Write Conflict Detection]
    G --> J[Durability]

    subgraph "MVCC Engine"
        D
        E
        F
        G
    end

    style B fill:#e1f5ff
    style D fill:#fff4e1
    style E fill:#e1ffe1
    style F fill:#ffe1e1
  
```

Abb. 02.6: Index-Strukturen

2.13.2 Snapshot Isolation Flow

```
// Transaction A liest Snapshot zum Zeitpunkt t=100
auto txn_a = db.beginTransaction();
auto snapshot = txn_a->getSnapshot(); // t=100

// Transaction B schreibt zur gleichen Zeit
auto txn_b = db.beginTransaction();
txn_b->put("entity:users:alice", "{age: 31}");
txn_b->commit(); // Commit zur Zeit t=101

// Transaction A sieht alte Version (t=100)
auto value = txn_a->get("entity:users:alice");
// value = "{age: 30}" (alte Version)

txn_a->commit();
```

2.13.3 Write-Write Conflict Detection

```
sequenceDiagram
    participant T1 as Transaction 1
    participant T2 as Transaction 2
    participant DB as RocksDB

    T1->>DB: get(users:alice)
    DB-->>T1: {age: 30}

    T2->>DB: get(users:alice)
    DB-->>T2: {age: 30}

    T1->>DB: put(users:alice, {age: 31})
    Note over T1,DB: Acquires Lock

    T2->>DB: put(users:alice, {age: 32})
    Note over T2,DB: Lock Conflict Detected!

    T1->>DB: commit()
    DB-->>T1: OK (age=31 persisted)

    T2->>DB: commit()
    DB-->>T2: CONFLICT (rollback)
```

Abb. 02.7: Replication-Topologie

Konfliktauflösung: Transaction 2 wird automatisch zurückgerollt - die Anwendung muss die Transaktion wiederholen.

2.13.4 MVCC Performance

Benchmark-Ergebnisse (v1.3.0):

Operation	MVCC (ops/s)	WriteBatch (ops/s)	Overhead
Single Entity Insert	3,400	3,100	+9.7%
Batch Insert (100)	27,800	27,800	0%
Snapshot Read	44,000	-	Baseline
Rollback	35,300	-	Baseline

Fazit: MVCC-Overhead ist minimal bei Batch-Operationen - Snapshot-Isolation kostet fast nichts!

2.13.5 Index-Konsistenz mit MVCC

Problem: Wenn eine Transaktion zurückgerollt wird, müssen alle Indizes konsistent bleiben.

Lösung: Alle Index-Operationen sind Teil der Transaktion:

```
// Atomare Transaktion: Entity + Secondary Index + Graph Outdex
auto txn = db.beginTransaction();

// 1. Insert Entity
txn->put(KeySchema::makeRelationalKey("users", "alice"),
        entity.serialize());

// 2. Update Secondary Index
txn->put(KeySchema::makeSecondaryIndexKey("users", "age", "30", "alice"),
        empty_value);

// 3. Update Graph Outdex
txn->put(KeySchema::makeGraphOutdexKey("alice", "follows_789"),
        edge_data);

// Alles oder nichts
if (!txn->commit()) {
    // Conflict detected - alle 3 Operationen werden zurückgerollt
}
```

Garantie: Entweder werden alle Indizes aktualisiert oder keiner - keine Inkonsistenzen möglich!

2.14 Compression Strategy

ThemisDB verwendet differenzierte Kompression je nach Datentyp für optimale Balance zwischen Speicherersparnis und Performance.

2.14.1 Kompressionsarten

Datentyp	Compression	Ratio	CPU-Overhead	Use Case
Time-Series	Gorilla	10-20x	+15%	Metriken, Sensordaten
Vektoren	SQ8 Quantization	4x	+20%	Embeddings ab 1M Vektoren
Content Blobs	ZSTD Level 19	1.5-2x	+30%	Dokumente, Bilder
JSON Metadata	LZ4 (RocksDB)	2-3x	+5%	Structured Data
Graph Edges	LZ4 (RocksDB)	2x	+5%	Adjacency Lists

2.14.2 Gorilla Time-Series Compression

Gorilla ist ein hocheffizienter Codec für Time-Series-Daten von Facebook:

```
// Gorilla Encoding
std::vector<uint8_t> GorillaCodec::encode(
    const std::vector<int64_t>& timestamps,
    const std::vector<double>& values
) {
    // Delta-of-Delta Encoding für Timestamps
    // XOR-Encoding für Float-Werte
    // Typical Ratio: 12-20x
}
```

Beispiel:

```
Raw Data (100k Punkte):
- Timestamps: 800 KB (8 bytes × 100k)
- Values: 800 KB (8 bytes × 100k)
- Total: 1.6 MB

Gorilla Compressed:
- Total: 80-130 KB
- Ratio: 12-20x
```

2.14.3 Vector Quantization (SQ8)

Scalar Quantization reduziert `float32` (4 bytes) auf `int8` (1 byte):

```
flowchart LR
    A[Float32 Vector] --> B[Compute Min/Max]
    B --> C[Scale to 0-255]
    C --> D[int8 Vector]
    D --> E[Store with Scale Params]

    E --> F[Dequantize on Search]
    F --> G[HNSW Search]

    style A fill:#e1f5ff
    style D fill:#e1ffe1
    style G fill:#fff4e1
```

Abb. 02.8: Failover-Mechanismus

Trade-off: -  4x Speichersparnis -  Bessere Cache-Nutzung → schnellere Suche -  95-98% Recall@10 (minimal schlechter als float32)

2.14.4 RocksDB Block Compression

ThemisDB konfiguriert RocksDB mit **Level-based Compression**:

```
// Level 0-5: LZ4 (schnell für Hot Data)
options.compression_per_level[0] = rocksdb::kLZ4Compression;
options.compression_per_level[1] = rocksdb::kLZ4Compression;
// ...

// Level 6+: Zstandard (höhere Ratio für Cold Data)
options.compression_per_level[6] = rocksdb::kZSTD;
options.compression_per_level[7] = rocksdb::kZSTD;
```

Rationale: Hot Data (Level 0-1) benötigt schnellen Zugriff → LZ4. Cold Data (Level 6+) profitiert von höherer Kompression → Zstandard.

2.15 Zusammenfassung Architektur

In diesem erweiterten Kapitel haben Sie die Tiefe der ThemisDB-Architektur kennengelernt:

Kernkonzepte

1. **BaseEntity:** Unified Storage Layer für alle Datenmodelle

2. **Key Schema:** Hierarchisches Namespacing mit Präfixen
3. **MVCC:** Snapshot Isolation ohne Locks
4. **Compression:** Differenzierte Strategien (Gorilla, SQ8, ZSTD)
5. **Index-Konsistenz:** Atomare Transaktionen über alle Indizes

Architektur-Prinzipien

- **Separation of Concerns:** Klar getrennte Schichten
- **Performance First:** Lazy Parsing, Lazy Indexing
- **Multi-Model Unity:** Ein Speicher, viele Modelle
- **Concurrency Without Locks:** MVCC statt Pessimistic Locking
- **Adaptive Compression:** Datentyp-spezifische Algorithmen

Weitere Ressourcen

- **Complete Example:** [examples/02_todo_app](#)
- **MVCC Deep-Dive:** [../de/architecture/architecture_mvcc.md](#)
- **RocksDB Storage:** [../de/storage/storage_rocksdb.md](#)
- **Index Design:** [Kapitel 15 - Storage Internals](#)

Kapitel 2 von 30 | Teil I: Grundlagen | ~14.000 Wörter

Kapitel 5: Kapitel 3: Multi-Model verstehen

"Der Unterschied zwischen einem guten und einem großartigen System liegt oft darin, das richtige Werkzeug für die richtige Aufgabe zu wählen."

Überblick

In den ersten beiden Kapiteln haben Sie die Grundlagen von ThemisDB kennengelernt. Jetzt lernen Sie, wie Sie die verschiedenen Datenmodelle gezielt einsetzen und kombinieren. Das Geheimnis liegt nicht darin, ein Modell zu wählen - sondern das *richtige* Modell für jeden Teil Ihrer Daten.

Was Sie in diesem Kapitel lernen werden: - Wann welches Datenmodell optimal ist - Wie man Modelle in einer Anwendung kombiniert - Die Stärken und Schwächen jedes Modells - Praktische Entscheidungskriterien - Vollständige Demonstration mit einem Kontaktmanager

Voraussetzungen: Kapitel 1 und 2 gelesen.

3.1 Die vier Modelle im Detail

Relational: Wenn Struktur entscheidend ist

Best für: - Geschäftsdaten mit klaren Beziehungen - Financial Transactions - Inventory Management - Reporting und Analytics

Eigenschaften:

```
users = {
  "user_id": "string",      # Muss vorhanden sein
  "email": "string",        # Muss vorhanden sein
  "age": "integer",         # Type-checked
  "created_at": "timestamp" # Format validiert
}

UNIQUE(email)
CHECK(age >= 18)
FOREIGN KEY(user_id) REFERENCES users(user_id)
```

Vorteile: -  Data Integrity durch Constraints -  Optimierte Joins -  Perfekt für BI/Reporting -  ACID über alle Operationen

Nachteile: - ✗ Schema-Änderungen erfordern Migrations - ✗ Nicht flexibel für schnelle Iterationen - ✗ Overhead bei sehr unterschiedlichen Entities

Wann verwenden?

- ✓ Daten haben feste Struktur
- ✓ Beziehungen sind wichtig
- ✓ Konsistenz ist kritisch
- ✗ Schema ändert sich häufig

Graph: Wenn Beziehungen im Fokus stehen

Best für: - Social Networks - Empfehlungssysteme - Fraud Detection - Netzwerk-Topologien

Eigenschaften:

```

user = {
  "_id": "users/alice",
  "name": "Alice"
}

friendship = {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15",
  "strength": 0.95 # Weighted edges
}

FOR friend IN 2..3 OUTBOUND "users/alice" friends
  RETURN friend

```

Vorteile: - ✓ Traversierung ist $O(\text{degree} \times \text{hops})$, nicht $O(n^2)$ - ✓ Relationship-First Design - ✓ Pattern Matching möglich - ✓ Graph-Algorithmen (PageRank, etc.)

Nachteile: - ✗ Nicht optimal für reine Daten ohne Beziehungen - ✗ Komplexer bei sehr vielen Knoten (> Millionen) - ✗ Overhead wenn Beziehungen unwichtig sind

Wann verwenden?

- ✓ Beziehungen sind zentral
- ✓ Traversierung ist häufig
- ✓ Netzwerk-Strukturen
- ✗ Isolierte Entities

Dokument: Wenn Flexibilität zählt

Best für: - Content Management - Product Catalogs - User Profiles - Logs und Events

Eigenschaften:

```
product = {
  "sku": "LAPTOP-2024",
  "name": "Developer Laptop",
  "specs": { # Nested objects
    "cpu": {"model": "i7", "cores": 8},
    "ram": {"size": 32, "type": "DDR5"}
  },
  "reviews": [ # Arrays
    {"user": "alice", "rating": 5},
    {"user": "bob", "rating": 4}
  ],
  "tags": ["developer", "high-end"],
  "metadata": { # Kann sein, muss nicht
    "warehouse": "DE-01"
  }
}
```

Vorteile: -  Keine Schema-Migrations -  Schnelle Iteration -  Hierarchische Daten natürlich -  Self-contained Documents

Nachteile: -  Keine Schema-Validierung (optional möglich) -  Denormalisierung kann zu Duplikaten führen -  Joins sind teurer

Wann verwenden?

- ✓ Schema ändert sich oft
- ✓ Hierarchische Daten
- ✓ Prototyping
- x Strenge Validierung nötig

Vektor: Wenn Ähnlichkeit gefragt ist

Best für: - Semantic Search - Recommendation Engines - Image Similarity - ML/AI Applications

Eigenschaften:

```

product_embedding = [
    0.123, -0.456, 0.789, ... # 768 dimensions
]

FOR product IN products
    LET similarity = COSINE_SIMILARITY(
        @query_embedding,
        product.embedding
    )
    FILTER similarity > 0.8
    SORT similarity DESC
    LIMIT 10
    RETURN product

```

Vorteile: -  Semantic Search ohne Keywords -  Multimodal (Text, Images, Audio) -  Skaliert zu Millionen von Vektoren -  State-of-the-art HNSW Index

Nachteile: -  Embeddings müssen generiert werden -  Höherer Speicherbedarf -  Approximate, nicht exact matching

Wann verwenden?

- ✓ Semantic Similarity
- ✓ "Finde ähnliche X"
- ✓ ML/AI Integration
- ✗ Exact matches ausreichend

```

quadrantChart
    title Datenmodell-Entscheidungsmatrix
    x-axis Flexible Struktur --> Feste Struktur
    y-axis Daten-fokussiert --> Beziehungs-fokussiert
    quadrant-1 Relational - ACID Geschaeftsdaten
    quadrant-2 Graph - Empfehlungen und Analysen
    quadrant-3 Dokument - CMS und Logs
    quadrant-4 Vektor - Semantic Search AI

```

Abb. 03.1: Datenmodell-Entscheidungsmatrix


```

flowchart TD
    Start{Welches Datenmodell?}

    Start -->|Feste Struktur?| Fixed{Schema stabil?}
    Start -->|Flexible Struktur?| Flexible{Beziehungen wichtig?}

    Fixed -->|Ja + Beziehungen| Graph[Graph-Modell<br/>Property Graph]
    Fixed -->|Ja + Keine Beziehungen| Relational[Relational-Modell<br/>Tabellen & SQL]

    Flexible -->|Ja| GraphDoc[Graph + Dokument<br/>Hybrid Approach]
    Flexible -->|Nein| Document[Dokument-Modell<br/>JSON Collections]

    Start -->|Ähnlichkeitssuche?| Similarity{ML/AI Features?}
    Similarity -->|Ja| Vector[Vektor-Modell<br/>Embeddings & HNSW]
    Similarity -->|Nein| Fulltext[Volltext-Suche<br/>Invertierter Index]

    style Graph fill:#f093fb
    style Relational fill:#4facfe
    style Document fill:#43e97b
    style Vector fill:#fa709a
    style GraphDoc fill:#fee140
    style Fulltext fill:#30cfd0

```

Abb. 03.2: Multi-Model-Data-Flow

3.2 Native Multi-Model vs. Polyglot Persistence

Das Problem polyglotter Persistenz

Viele Unternehmen verfolgen einen "Polyglot Persistence"-Ansatz [33]: Sie kombinieren mehrere spezialisierte Datenbanken (z.B. PostgreSQL für relationale Daten, Neo4j für Graphen, ChromaDB für Vektoren) in einem losen Verbund. Dieser Ansatz scheint flexibel, führt aber zu fundamentalen Problemen [1]:

Technische Herausforderungen:

1. **Eventual Consistency statt ACID:**
2. Daten sind über mehrere, physisch getrennte Systeme verteilt [33]
3. Atomare Transaktionen über alle Systeme hinweg sind unmöglich [1]
4. Man muss auf das "Saga-Pattern" [17] mit kompensierenden Transaktionen zurückgreifen
5. Resultat: "Eventual Consistency" (BASE) [18] statt starker ACID-Garantien [16]
6. **Post-Filtering-Problem bei RAG-Workloads:** `` # Polyglot-Ansatz (ineffizient):
7. Vektor-DB: Finde 1000 ähnliche Dokumente

8. Graph-DB: Hole alle Prozesse für Landkreis Havelland
9. Relational-DB: Hole alle Akten aus 2024
10. Application: Bilde manuell Schnittmenge im RAM → 990 irrelevante Ergebnisse werden verworfen! [2], [5] ```
11. **Operativer Overhead:**
12. 3+ unterschiedliche APIs zu lernen und warten
13. 3+ Backup-Strategien zu implementieren
14. 3+ Monitoring-Systeme zu konfigurieren
15. 3+ Security-Konfigurationen abzustimmen
16. Team benötigt Expertise in mehreren Datenbanken

ThemisDB's Native Multi-Model-Lösung

ThemisDB verfolgt einen fundamentalen anderen Ansatz [3], [11]: **Alle Datenmodelle teilen sich eine einzige, transaktionale Speicherschicht** (siehe Kapitel 2.4 - Base Entity Paradigma).

Architektonische Vorteile:

1. **Starke ACID-Transaktionen über alle Modelle:**

2. Eine Operation kann atomar Graph, Vector und Relational ändern [1], [20]
3. Kein Saga-Pattern nötig für Datenbankintegrität [3]
4. Konsistenz ist "by Design", nicht ein fehleranfälliger Applikationsprozess

5. **Pre-Filtering statt Post-Filtering:**

```
aql -- Native Multi-Model ermöglicht Pre-Filtering: FOR doc IN documents FILTER doc.year == 2024 // Relational Filter  
ZUERST FILTER doc.region == "Havelland" // Graph Context LET similarity =  
COSINE(doc.vector, @query) // Vector Search auf Subset FILTER similarity >  
0.8 SORT similarity DESC LIMIT 10
```

6. Query-Engine nutzt ZUERST den relationalen Index (Jahr=2024)
7. Erstellt eine hochselektive Kandidatenliste
8. Vektorsuche (rechenintensiv) läuft NUR auf erlaubter Teilmenge
9. **Resultat:** Um Größenordnungen performanter als Post-Filtering
10. **Vereinfachte Operations:**
11. Eine API (AQL) für alle Modelle
12. Eine Backup-Strategie
13. Ein Monitoring-System
14. Eine Security-Konfiguration

Wann welcher Ansatz?

Polyglot Persistence verwenden wenn: - ✓ Sie bereits mehrere spezialisierte Datenbanken im Einsatz haben - ✓ Eventual Consistency für Ihren Use Case akzeptabel ist - ✓ Sie Best-of-Breed-Tools für jeden Use Case wollen - ✓ Sie ein großes Ops-Team haben

Native Multi-Model (ThemisDB) verwenden wenn: - ✓ ACID-Garantien über alle Datenmodelle kritisch sind - ✓ Sie hybride Queries (Graph + Vector + Relational) benötigen - ✓ RAG-Workloads mit komplexen Filtern zentral sind - ✓ Sie operationale Komplexität reduzieren wollen - ✓ Ein kleineres Team die Infrastruktur betreiben soll

3.3 Entscheidungskriterien

Die 5 Fragen-Methode

1. Wie strukturiert sind Ihre Daten?

Sehr strukturiert → Relational
Teilweise strukturiert → Dokument
Unstrukturiert → Vektor

2. Wie wichtig sind Beziehungen?

Zentral → Graph
Wichtig → Relational (mit Joins)
Unwichtig → Dokument

3. Wie oft ändert sich das Schema?

Selten → Relational
Häufig → Dokument
Gar nicht → Relational

4. Welche Queries dominieren?

Beziehungen durchlaufen → Graph
Ähnlichkeit finden → Vektor
Komplexe Aggregationen → Relational
Einzelne Dokumente → Dokument

5. Wie wichtig ist Konsistenz?

Kritisch → Relational
 Wichtig → Relational oder Graph
 Weniger wichtig → Dokument

Entscheidungsmatrix

Use Case	Relational	Graph	Dokument	Vektor
E-Commerce Orders	★★★	★☆☆	★★☆	☆☆☆
Social Network	★☆☆	★★★	★★☆	☆☆☆
Product Catalog	★★☆	☆☆☆	★★★	★★☆
Recommendation	★☆☆	★★★	☆☆☆	★★★
CMS/Blog	★☆☆	☆☆☆	★★★	★★☆
Financial	★★★	☆☆☆	★☆☆	☆☆☆
Fraud Detection	★★☆	★★★	★☆☆	★★☆

3.4 Praxis: Kontaktmanager

Jetzt bauen wir einen vollständigen Kontaktmanager, der zeigt, wie man Modelle kombiniert. Basis ist [examples/03_contact_manager](#).

Warum Dokument-Modell für Kontakte?

Kontakte sind ein perfektes Beispiel für das Dokument-Modell:

Gründe: 1. **Flexible Struktur:** Manche Kontakte haben Adresse, manche nicht 2. **Hierarchische Daten:** Adresse ist ein Nested Object 3. **Schema-Evolution:** Neue Felder wie "Social Media" leicht hinzufügar 4. **Self-Contained:** Alle Infos zu einem Kontakt in einem Dokument

Alternative wäre Relational:

```
-- Relational würde benötigen:  
CREATE TABLE contacts (id, name, email, phone);  
CREATE TABLE addresses (id, contact_id, street, city, ...);  
CREATE TABLE social_media (id, contact_id, platform, handle);  
-- → 3+ Tabellen, Joins nötig
```

Mit Dokument:

```
contact = {  
  "name": "Alice",  
  "email": "alice@example.com",  
  "address": {"city": "Berlin"}, # Optional!  
  "social": {"twitter": "@alice"} # Optional!  
}
```

Datenmodell

```

from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, List
from enum import Enum

class ContactCategory(Enum):
    FRIENDS = "friends"
    FAMILY = "family"
    WORK = "work"
    OTHER = "other"

@dataclass
class Address:
    """Verschachtelte Adress-Struktur"""
    street: str
    city: str
    postal_code: str
    country: str = "Deutschland"

    def to_dict(self):
        return {
            "street": self.street,
            "city": self.city,
            "postal_code": self.postal_code,
            "country": self.country
        }

    @classmethod
    def from_dict(cls, data: dict):
        return cls(
            street=data.get("street", ""),
            city=data.get("city", ""),
            postal_code=data.get("postal_code", ""),
            country=data.get("country", "Deutschland")
        )

@dataclass
class Contact:
    """Hauptentität: Kontakt"""
    id: str
    first_name: str
    last_name: str
    email: str
    phone: Optional[str] = None
    address: Optional[Address] = None
    category: ContactCategory = ContactCategory.OTHER
    is_favorite: bool = False
    notes: str = ""
    tags: List[str] = field(default_factory=list)

```

```

created_at: datetime = field(default_factory=datetime.now)
updated_at: datetime = field(default_factory=datetime.now)

@property
def full_name(self):
    """Computed Property"""
    return f"{self.first_name} {self.last_name}"

def to_dict(self):
    """Serialisierung für ThemisDB"""
    return {
        "_key": self.id,
        "first_name": self.first_name,
        "last_name": self.last_name,
        "email": self.email,
        "phone": self.phone,
        "address": self.address.to_dict() if self.address else None,
        "category": self.category.value,
        "is_favorite": self.is_favorite,
        "notes": self.notes,
        "tags": self.tags,
        "created_at": self.created_at.isoformat(),
        "updated_at": self.updated_at.isoformat()
    }

@classmethod
def from_dict(cls, data: dict):
    """Deserialisierung von ThemisDB"""
    address_data = data.get("address")
    return cls(
        id=data["_key"],
        first_name=data["first_name"],
        last_name=data["last_name"],
        email=data["email"],
        phone=data.get("phone"),
        address=Address.from_dict(address_data) if address_data else None,
        category=ContactCategory(data.get("category", "other")),
        is_favorite=data.get("is_favorite", False),
        notes=data.get("notes", ""),
        tags=data.get("tags", []),
        created_at=datetime.fromisoformat(data["created_at"]),
        updated_at=datetime.fromisoformat(data["updated_at"])
    )

```

Design-Highlights:

1. **Nested Objects:** `Address` ist ein eigenes Dataclass, aber Teil des Contact-Dokuments
2. **Optional Fields:** `phone` , `address` können fehlen

3. **Enums:** `ContactCategory` für Type-Safety
4. **Computed Properties:** `full_name` berechnet aus first + last
5. **Default Factories:** Listen und Timestamps werden automatisch initialisiert

Client-Implementierung mit Volltext-Suche

```

from themisdb import Client
from models import Contact, ContactCategory
from typing import List, Optional
import uuid

class ContactClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen (Document Model!)
        try:
            self.client.create_collection("contacts", type="document")
        except:
            pass

        # Indizes für schnelle Suche
        self.client.create_index("contacts", "email_idx", ["email"])
        self.client.create_index("contacts", "category_idx", ["category"])
        self.client.create_index("contacts", "favorite_idx", ["is_favorite"])

        # FULLTEXT Index für Suche!
        self.client.create_index(
            "contacts",
            "fulltext_idx",
            ["first_name", "last_name", "email", "notes"],
            type="fulltext"
        )

    def create_contact(self, contact: Contact) -> bool:
        """Erstellt einen neuen Kontakt"""
        try:
            self.client.insert("contacts", contact.to_dict())
            return True
        except Exception as e:
            print(f"Error creating contact: {e}")
            return False

    def search_contacts(self, query: str) -> List[Contact]:
        """Volltext-Suche über alle Felder"""
        aql = """
        FOR contact IN contacts
            FILTER CONTAINS(LOWER(contact.first_name), LOWER(@query))
                OR CONTAINS(LOWER(contact.last_name), LOWER(@query))
                OR CONTAINS(LOWER(contact.email), LOWER(@query))
                OR CONTAINS(LOWER(contact.notes), LOWER(@query))
            SORT contact.last_name ASC, contact.first_name ASC
        """

```

```

        RETURN contact
    """
    try:
        results = self.client.query(aql, {"query": query})
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error searching contacts: {e}")
        return []

def get_favorites(self) -> List[Contact]:
    """Alle Favoriten-Kontakte"""
    aql = """
    FOR contact IN contacts
        FILTER contact.is_favorite == true
        SORT contact.last_name ASC
        RETURN contact
    """
    try:
        results = self.client.query(aql)
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        return []

def get_by_category(self, category: ContactCategory) -> List[Contact]:
    """Kontakte nach Kategorie filtern"""
    aql = """
    FOR contact IN contacts
        FILTER contact.category == @category
        SORT contact.last_name ASC
        RETURN contact
    """
    try:
        results = self.client.query(aql, {"category": category.value})
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        return []

def export_to_json(self, filename: str) -> bool:
    """Exportiert alle Kontakte als JSON"""
    aql = "FOR contact IN contacts RETURN contact"
    try:
        contacts = self.client.query(aql)
        import json
        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(contacts, f, indent=2, ensure_ascii=False)
        return True
    except Exception as e:
        print(f"Export failed: {e}")
        return False

```

```
def import_from_json(self, filename: str) -> int:
    """Importiert Kontakte aus JSON"""
    import json
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            contacts = json.load(f)

            count = 0
            for contact_data in contacts:
                contact = Contact.from_dict(contact_data)
                if self.create_contact(contact):
                    count += 1

            return count
    except Exception as e:
        print(f"Import failed: {e}")
        return 0
```

Dokument-Modell in Aktion

Nested Queries:

```
aql = """
FOR contact IN contacts
  FILTER contact.address != null
    AND contact.address.city == "Berlin"
  RETURN contact
"""
```

Array Operations:

```
aql = """
FOR contact IN contacts
  FILTER "important" IN contact.tags
  RETURN contact
"""
```

Schema Evolution ohne Migration:

```
contact = {  
  "first_name": "Alice",  
  "email": "alice@example.com",  
  "social_media": { # NEU: Einfach hinzugefügt  
    "twitter": "@alice",  
    "linkedin": "alice-smith"  
  },  
  "birthday": "1990-03-15" # NEU: Auch einfach hinzugefügt  
}
```

3.4 Multi-Model Kombinationen

Beispiel 1: E-Commerce mit allen Modellen

```

order = {
  "order_id": "0123",
  "user_id": "alice",
  "total": 99.99,
  "status": "paid"
}

product = {
  "sku": "LAPTOP-X1",
  "name": "Laptop",
  "specs": { # Jedes Produkt hat andere Specs!
    "cpu": "i7",
    "ram": 32
  }
}

"""

FOR friend IN 1..2 OUTBOUND @userId friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
    FOR product IN products
      FILTER product.sku == order.sku
      LET similarity = COSINE_SIMILARITY(
        product.embedding,
        @my_last_product_embedding
      )
      FILTER similarity > 0.7
      AND product.price < 1000
    RETURN DISTINCT product
"""

```

Beispiel 2: CRM System

```
transactions = {
  "transaction_id": "T123",
  "customer_id": "C456",
  "amount": 5000.00,
  "status": "completed"
}

customer = {
  "customer_id": "C456",
  "company": "ACME Corp",
  "contacts": [ # Array von Kontakten
    {"name": "John", "role": "CEO"},
    {"name": "Jane", "role": "CTO"}
  ],
  "metadata": { # Flexible metadata
    "industry": "tech",
    "size": "medium"
  }
}
```

3.5 Best Practices

1. Start Simple, Add Complexity

2. Denormalisierung ist OK

Im Dokument-Modell ist Duplikation oft besser als Joins:


```

order = {"order_id": "0123", "user_id": "alice"}
user = {"user_id": "alice", "name": "Alice", "email": "..."}

order = {
    "order_id": "0123",
    "user": { # User-Daten dupliziert
        "user_id": "alice",
        "name": "Alice",
        "email": "alice@example.com"
    }
}

```

3. Use Computed Properties

```

@property
def age(self):
    """Berechnet Alter aus Geburtstag"""
    if not self.birthday:
        return None
    today = datetime.now()
    return today.year - self.birthday.year

```

4. Index Strategically

Nicht jedes Feld braucht einen Index:

```
client.create_index("contacts", "email_idx", ["email"])
```

3.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Vier Modelle:** Wann Relational, Graph, Dokument oder Vektor
- ✓ **Entscheidungskriterien:** Die 5 Fragen-Methode
- ✓ **Kontaktmanager:** Vollständige Dokument-Modell Demo
- ✓ **Multi-Model Kombination:** Verschiedene Modelle zusammen nutzen
- ✓ **Best Practices:** Denormalisierung, Computed Properties, Indexing

Key Takeaways

1. Es gibt kein **"bestes" Modell** - nur das beste für Ihren Use Case

-
2. **Kombinieren ist Stärke** - ThemisDB glänzt bei Multi-Model
 3. **Schema-Flexibilität** - Dokumente sind perfekt für Evolution
 4. **Performance** - Wählen Sie das Modell nach Query-Patterns

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie Sie ThemisDB installieren, konfigurieren und für Production vorbereiten.

[Kapitel 4: Installation & Setup →](#)

Weiterführende Ressourcen

- **Complete Example:** [examples/03_contact_manager](#)
- **Tutorial:** [Contact Manager Tutorial](#)
- **Document Model Docs:** [../de/architecture/architecture_content.md](#)
- **Multi-Model Patterns:** [Kapitel 13 - AQL Mastery](#)

Kapitel 3 von 30 | Teil I: Grundlagen | ~7.500 Wörter

Kapitel 6: Kapitel 4: Installation und Setup

"Die beste Dokumentation hilft nichts, wenn die Software nicht läuft. Installation sollte einfach sein - und das ist sie."

Überblick

In den ersten drei Kapiteln haben Sie die Konzepte von ThemisDB kennengelernt. Jetzt ist es Zeit, ThemisDB selbst zu installieren und zu konfigurieren. Dieses Kapitel führt Sie durch alle Optionen - von der schnellsten (Docker) bis zur flexibelsten (Source Build).

Was Sie in diesem Kapitel lernen werden: - Installation mit Docker (schnellste Methode) - Installation aus Binaries - Build von Source Code - Konfiguration und Tuning - Production-Setup Best Practices - Troubleshooting häufiger Probleme

Voraussetzungen: Grundlegende Kommandozeilen-Kenntnisse.

4.1 Systemanforderungen

Minimale Requirements

CPU: 2 Cores
RAM: 4 GB
Disk: 20 GB (SSD empfohlen)
OS:
- Linux (Ubuntu 20.04+, Debian 11+, CentOS 8+)
- macOS 11+
- Windows 10/11 (via WSL2 oder Docker)

Empfohlene Production Requirements

CPU: 8+ Cores
RAM: 32 GB
Disk: 500 GB NVMe SSD
OS: Linux (Ubuntu 22.04 LTS empfohlen)
Network: 10 Gbps

Hardware-Überlegungen

CPU: - ThemisDB ist multi-threaded - Mehr Cores = höherer Durchsatz - Minimum 2, empfohlen 8+

RAM: - Wird für Caching genutzt - 25% des Datasets sollte in RAM passen - Minimum 4 GB, empfohlen 32 GB+

Storage: - SSDs sind **stark** empfohlen - ThemisDB nutzt LSM-Trees (sequential writes) - NVMe > SATA SSD >> HDD - RAID 10 für Production

Disk Performance Vergleich:

Storage Type	Random Read IOPS	Sequential Write MB/s
HDD 7200 RPM	100-200	150
SATA SSD	50,000	500
NVMe SSD	500,000	3,500

ThemisDB Benefit: Mit NVMe SSD 10-50x schneller als HDD!

```
graph TB
    subgraph "Installation Options"
        Choice{Choose Installation Method}

        Choice -->|Fastest & Easiest| Docker["Docker Install<br/>✓ Works out-of-box<br/>✓"]
        Choice -->|Pre-built| Binary["Binary Install<br/>✓ No compilation<br/>✓ Native pe"]
        Choice -->|Full Control| Source["Source Build<br/>✓ Custom config<br/>✓ Latest fe"]

        Docker --> Run1[docker run themisdb]
        Binary --> Run2["./themisdb-server"]
        Source --> Run3["Build + Configure + Run"]
    end

    Run1 --> Ready1["ThemisDB Running<br/>Port 8765"]
    Run2 --> Ready2[Ready]
    Run3 --> Ready3[Ready]

    style Docker fill:#43e97b
    style Binary fill:#4facfe
    style Source fill:#f093fb
    style Ready1 fill:#ffd32a
    style Ready2 fill:#ffd32a
    style Ready3 fill:#ffd32a
```

Abb. 04.1: Installations-Ablaufdiagramm

4.2 Installation mit Docker (Schnellste Methode)

Warum Docker?

-  Funktioniert sofort (keine Dependencies)
-  Isolation vom Host-System
-  Einfache Updates
-  Konsistent über alle Plattformen

Quick Start

```
docker run -d \  
  --name themisdb \  
  -p 8765:8765 \  
  -v themisdb_data:/data \  
  themisdb/themisdb:latest  
  
curl http://localhost:8765/health
```

Fertig! ThemisDB läuft jetzt.

Mit spezifischer Version

```
docker run -d \  
  --name themisdb \  
  -p 8765:8765 \  
  -v themisdb_data:/data \  
  themisdb/themisdb:1.3.4 # Fixe Version
```

Best Practice: Verwende in Production immer eine fixe Version, nicht `latest`.

Mit Konfiguration

```
cat > themisdb.yaml << EOF
server:
  port: 8765
  threads: 8

storage:
  path: /data
  cache_size_mb: 2048

logging:
  level: info
  file: /logs/themisdb.log
EOF

docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  -v themisdb_logs:/logs \
  -v $(pwd)/themisdb.yaml:/etc/themisdb/config.yaml \
  themisdb/themisdb:1.3.4 \
  --config /etc/themisdb/config.yaml
```

Docker Compose

Für komplexere Setups mit mehreren Containern:

```
version: '3.8'

services:
  themisdb:
    image: themisdb/themisdb:1.3.4
    container_name: themisdb
    ports:
      - "8765:8765"
    volumes:
      - themisdb_data:/data
      - themisdb_logs:/logs
      - ./config.yaml:/etc/themisdb/config.yaml
    environment:
      - THEMISDB_LOG_LEVEL=info
      - THEMISDB_CACHE_SIZE=2048
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8765/health"]
      interval: 30s
      timeout: 10s
      retries: 3

# Optional: Monitoring mit Prometheus
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml

volumes:
  themisdb_data:
  themisdb_logs:
```

Starten:

```
docker-compose up -d
```

4.3 Installation aus Binaries

Download

```
curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/themisdb-linux-x86_64.ta

curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/themisdb-darwin-arm64.ta

tar xzf themisdb.tar.gz
cd themisdb
```

Installation

```
sudo cp bin/themisdb /usr/local/bin/
sudo mkdir -p /etc/themisdb
sudo cp config/themisdb.yaml /etc/themisdb/

mkdir -p ~/themisdb/{bin,data,logs}
cp bin/themisdb ~/themisdb/bin/
cp config/themisdb.yaml ~/themisdb/
```

Starten

```
sudo systemctl start themisdb
sudo systemctl enable themisdb # Autostart

themisdb --config /etc/themisdb/config.yaml

nohup themisdb --config themisdb.yaml > logs/themisdb.log 2>&1 &
```

4.4 Build von Source

Warum von Source bauen?

-  Neueste Features (Entwicklungs-Branch)
-  Custom Optimierungen
-  Spezielle Plattformen (ARM, RISC-V)
-  Beiträge entwickeln

Dependencies installieren

Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install -y \
    build-essential \
    cmake \
    git \
    libssl-dev \
    libz-dev \
    libbz2-dev \
    liblz4-dev \
    libzstd-dev
```

macOS:

```
brew install cmake openssl zlib bzip2 lz4 zstd
```

Clone und Build

```
git clone https://github.com/makr-code/ThemisDB.git
cd ThemisDB

git submodule update --init --recursive

mkdir build && cd build
cmake .. \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX=/usr/local

make -j$(nproc)

make test

sudo make install
```

Build-Optionen

```
cmake .. -DCMAKE_BUILD_TYPE=Debug
```

```
cmake .. \  
  -DCMAKE_BUILD_TYPE=Debug \  
  -DENABLE_ASAN=ON \  
  -DENABLE_UBSAN=ON
```

```
cmake .. \  
  -DCMAKE_BUILD_TYPE=Release \  
  -DNATIVE_ARCH=ON
```

```
cmake .. \  
  -DCMAKE_BUILD_TYPE=Release \  
  -DWITH_JEMALLOC=ON \  
  -DWITH_SNAPPY=ON \  
  -DWITH_LZ4=ON \  
  -DWITH_ZSTD=ON
```

4.5 Konfiguration

Basis-Konfiguration

```

server:
  host: 0.0.0.0           # Bind auf alle Interfaces
  port: 8765              # Standard Port
  threads: 8              # Worker Threads (= CPU Cores)
  max_connections: 1000   # Maximale gleichzeitige Connections

storage:
  path: /var/lib/themisdb/data
  cache_size_mb: 2048     # RocksDB Block Cache
  write_buffer_size_mb: 64
  max_open_files: 1000
  compression: lz4        # lz4, zstd, snappy, none

logging:
  level: info             # debug, info, warning, error
  file: /var/log/themisdb/themisdb.log
  max_size_mb: 100
  max_backups: 10

security:
  tls_enabled: false
  tls_cert: /etc/themisdb/cert.pem
  tls_key: /etc/themisdb/key.pem
  auth_enabled: false
  auth_db: /etc/themisdb/users.db

```

Performance Tuning

```

storage:
  cache_size_mb: 8192     # Mehr Cache = weniger Disk I/O
  write_buffer_size_mb: 256
  max_background_jobs: 8   # Parallel Compaction
  level0_slowdown_writes_trigger: 20
  level0_stop_writes_trigger: 36

server:
  threads: 16             # Mehr Threads für mehr Parallelität
  max_connections: 5000

```

```
storage:
  cache_size_mb: 16384    # Großer Cache
  write_buffer_size_mb: 128
  max_background_jobs: 4  # Weniger Background I/O

server:
  threads: 8
  max_connections: 1000
```

Environment Variables

Überschreiben Config-File Werte:

```
export THEMISDB_PORT=9000

export THEMISDB_LOG_LEVEL=debug

export THEMISDB_CACHE_SIZE=4096

themisdb
```

4.6 Production Setup

Systemd Service

```
[Unit]
Description=ThemisDB Multi-Model Database
After=network.target

[Service]
Type=simple
User=themisdb
Group=themisdb
ExecStart=/usr/local/bin/themisdb --config /etc/themisdb/config.yaml
Restart=on-failure
RestartSec=5s

NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/themisdb /var/log/themisdb

LimitNOFILE=65536
LimitNPROC=4096

[Install]
WantedBy=multi-user.target
```

Aktivieren:

```
sudo systemctl daemon-reload
sudo systemctl enable themisdb
sudo systemctl start themisdb
```

Monitoring Setup

Health Endpoint:

```
curl http://localhost:8765/health
```

Metrics Endpoint (Prometheus):

```
curl http://localhost:8765/metrics
```

Grafana Dashboard: Importiere vorgefertigtes Dashboard: `dashboards/grafana-themisdb.json`

Backup Strategy

1. Snapshot-basiertes Backup:

```
curl -X POST http://localhost:8765/admin/snapshot  
  
/var/lib/themisdb/snapshots/snapshot-2025-12-28-100000/  
  
rsync -av /var/lib/themisdb/snapshots/ backup-server:/backups/themisdb/
```

2. Continuous Backup:

```
rsync -av --delete \  
/var/lib/themisdb/data/WAL/ \  
backup-server:/backups/themisdb/wal/
```

3. Restore:

```
sudo systemctl stop themisdb  
  
rsync -av backup-server:/backups/themisdb/latest/ /var/lib/themisdb/data/  
  
sudo systemctl start themisdb
```

Security Hardening

1. TLS aktivieren:

```
security:  
  tls_enabled: true  
  tls_cert: /etc/themisdb/cert.pem  
  tls_key: /etc/themisdb/key.pem
```

2. Authentifizierung:

```
themisdb-admin user create alice --password secret123 --role admin
```

```
security:  
  auth_enabled: true  
  auth_db: /etc/themisdb/users.db
```

3. Firewall:

```
sudo ufw allow from 10.0.0.0/24 to any port 8765  
sudo ufw enable
```

4. Network Isolation:

```
server:  
  host: 10.0.0.10 # Nicht 0.0.0.0!
```

4.7 Troubleshooting

Problem: Port bereits in Verwendung

Symptom:

```
ERROR: Failed to bind on port 8765: Address already in use
```

Lösung:

```
sudo lsof -i :8765  
sudo netstat -tulpn | grep 8765  
  
sudo systemctl stop <service>  
  
server:  
  port: 9000
```

Problem: Zu wenig Memory

Symptom:

```
WARNING: Cache size exceeds available memory
ERROR: Out of memory
```

Lösung:

```
storage:
  cache_size_mb: 1024  # Statt 8192
  write_buffer_size_mb: 32
```

Problem: Langsame Queries

Symptom: Queries dauern > 1 Sekunde

Diagnose:

```
export THEMISDB_LOG_LEVEL=debug

tail -f /var/log/themisdb/themisdb.log | grep "Query execution"
```

Lösung:

```
-- Prüfe ob Indizes genutzt werden
EXPLAIN FOR user IN users FILTER user.email == 'test@example.com' RETURN user

-- Output sollte zeigen: "using index: email_idx"
-- Falls nicht: Index erstellen!
```

Problem: Disk voll

Symptom:

```
ERROR: No space left on device
```

Lösung:


```
df -h /var/lib/themisdb

curl -X POST http://localhost:8765/admin/compact

rm -rf /var/lib/themisdb/snapshots/snapshot-old-*

themisdb-admin wal cleanup --keep-last 10
```

Problem: Connection Timeouts

Symptom:

```
ERROR: Connection timed out after 10s
```

Lösung:

```
server:
  read_timeout_s: 30
  write_timeout_s: 30
  idle_timeout_s: 300

server:
  max_connections: 5000
```

4.8 Updates und Wartung

Update-Prozess

Docker:

```
docker pull themisdb/themisdb:1.3.4

docker stop themisdb

docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:1.3.4

docker rm themisdb-old
```

Binary:

```
sudo systemctl stop themisdb
tar czf themisdb-backup.tar.gz /var/lib/themisdb/data

sudo cp themisdb-new /usr/local/bin/themisdb

sudo systemctl start themisdb

curl http://localhost:8765/health
```

Rolling Update (Zero-Downtime)

Für Cluster-Setups:

```
kubectl set image deployment/themisdb-node2 themisdb=themisdb:1.3.4

kubectl wait --for=condition=ready pod -l app=themisdb-node2

kubectl set image deployment/themisdb-node1 themisdb=themisdb:1.3.4
```

4.9 Diagnostics & Hardening (Kurz)

Health & Metrics: - `GET /health` (liveness), `GET /metrics` (Prometheus) - Log-Level zur Fehlersuche: `export THEMISDB_LOG_LEVEL=debug`

Security-Hardening: - TLS aktivieren: `--tls.cert` / `--tls.key` - mTLS optional: Client-Certs whitelisten - Admin-API absichern: IP-Whitelist + Auth-Header

Storage-Pflege: - Wöchentliche Compaction: `POST /admin/compact` - Snapshot-Rotation: Keep-last-7, Air-Gap-Upload

Performance-Probe: - `EXPLAIN` für jede neue Query - Page Cache treffen: `cache_size_mb` auf 20-30% RAM

4.10 Deployment-Playbooks

Single Node (Pilot)

- Docker-Run mit persistentem Volume (`themisdb_data`)
- Backup via Snapshot-API + Offsite-Kopie
- Monitoring: Health + Metrics → Prometheus

HA (2-3 Nodes, Sync-Replicas)

- Node-Labels: `role=primary` , `role=secondary`
- Synchronous Commit für RPO=0
- Load Balancer vor 2 Read-Replicas

Airgapped Install

- Offline-Bundle: Binary + Config + Checksums
- Pakete signieren, Verify via `sha256sum`
- Updates nur über signierte Tarballs einspielen

4.11 Operations-Checkliste (Go-Live)

- [] TLS 1.3 aktiviert, mTLS optional getestet
- [] Backups automatisiert + Restore-Drill bestanden
- [] Monitoring: Health, Metrics, Logs zentralisiert
- [] Admin-API eingeschränkt (Firewall + Auth)
- [] Config-Lint: Cache, Write-Buffer, Thread-Count passend
- [] Runbooks: Incident, Restore, Scale-Up

4.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Systemanforderungen:** CPU, RAM, Storage Best Practices
- ✓ **Docker Installation:** Schnellste Methode, Production-ready
- ✓ **Binary Installation:** Systemweit oder lokal
- ✓ **Source Build:** Für Custom Builds und Development
- ✓ **Konfiguration:** Tuning für Performance und Security
- ✓ **Production Setup:** Systemd, Monitoring, Backup
- ✓ **Troubleshooting:** Häufige Probleme und Lösungen
- ✓ **Updates:** Safe Update-Prozesse

Key Takeaways

1. **Docker ist der einfachste Weg** - funktioniert sofort
2. **SSDs sind essentiell** - 10-50x Performance-Gewinn
3. **Monitoring ist wichtig** - Health + Metrics endpoints
4. **Backups sind Pflicht** - Snapshots + WAL Replication
5. **Security nicht vergessen** - TLS + Auth für Production

Sie sind bereit!

ThemisDB ist jetzt installiert und läuft. Im nächsten Teil des Kompendiums (Teil II) tauchen wir tiefer in die einzelnen Datenmodelle ein.

[Teil II: Kapitel 5 - Relationale Daten →](#)

Weiterführende Ressourcen

- **Docker Deployment:** [../de/deployment/DOCKER_DEPLOYMENT.md](#)
- **Build Optionen:** [../de/deployment/BUILD_OPTIONEN_REFERENZ.md](#)
- **Production Strategy:** [../de/deployment/deployment_strategy.md](#)
- **ARM Deployment:** [../de/deployment/deployment_arm_build.md](#)

Kapitel 4 von 30 | Teil I: Grundlagen | ~6.500 Wörter

Kapitel 7: Kapitel 5: Relationale Daten

"Relational databases are like spreadsheets that can talk to each other - but with ACID guarantees and without the chaos."

Überblick

Relationale Datenbanken sind das Rückgrat der IT seit über 40 Jahren. In ThemisDB ist das relationale Modell eines von vier gleichberechtigten Datenmodellen - aber mit vollem AQL-Support und ACID-Garantien.

Was Sie in diesem Kapitel lernen werden: - Relationales Datenmodell: Tabellen, Schemas, Constraints - AQL in ThemisDB: DDL, DML, Joins, Subqueries - Normalisierung vs. Denormalisierung - Transaktionen und Integrität - Indexes und Performance - **Praxisbeispiel 1:** Inventory System (Lagerverwaltung) - **Praxisbeispiel 2:** Expense Tracker (Ausgabenverwaltung)

Voraussetzungen: AQL-Grundkenntnisse hilfreich, aber nicht notwendig.

5.1 Das Relationale Modell

Grundkonzepte

Tabelle (Table): Sammlung von Zeilen mit gleichem Schema

```
products (Tabelle)
+-----+-----+-----+-----+
| id | name      | price | stock |
+-----+-----+-----+-----+
| 1  | Laptop    | 1200  | 15    |
| 2  | Mouse     | 25    | 100   |
| 3  | Keyboard  | 80    | 50    |
+-----+-----+-----+-----+
```

Zeile (Row): Ein Datensatz in einer Tabelle

Spalte (Column): Ein Attribut, das jede Zeile hat

Schema: Definition der Spalten und Typen

Primärschlüssel (Primary Key): Eindeutige ID für jede Zeile

Fremdschlüssel (Foreign Key): Referenz zu einer anderen Tabelle

Warum Relational?

✅ **Vorteile:** - **Struktur:** Klare, vorhersehbare Datenstruktur - **Integrität:** Constraints garantieren Datenqualität - **Joins:** Verknüpfe Daten aus mehreren Tabellen - **ACID:** Transaktionen mit vollständiger Konsistenz - **AQL:** Standardisierte, mächtige Abfragesprache

❌ **Nachteile:** - Rigid: Schema-Änderungen erfordern Migrations - Komplex: Normalisierung kann zu vielen Tables führen - Performance: Joins können langsam sein bei vielen Tables - Skalierung: Vertical Scaling bevorzugt

Wann Relational wählen?

Perfekt für: - ✅ Strukturierte Business-Daten (Orders, Invoices, Inventory) - ✅ Daten mit klaren Beziehungen (Customers → Orders → Items) - ✅ Transaktionale Systeme (Banking, E-Commerce) - ✅ Reports mit Aggregationen (SUM, AVG, GROUP BY) - ✅ Datenintegrität ist kritisch

Weniger geeignet für: - ❌ Hochflexible Schemas (Metadata, User-Generated Content) - ❌ Netzwerk-Daten mit vielen Hops (Social Graphs) - ❌ Unstrukturierte Daten (Logs, Dokumente) - ❌ Vektor-Daten (Embeddings, Features)

5.2 Tabellen und Schemas

Tabelle erstellen

```
-- Basic Table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Datentypen in ThemisDB:

Typ	Beschreibung	Beispiel
INT	Ganzzahl	42, -17, 0
BIGINT	Große Ganzzahl	9223372036854775807
DECIMAL(p,s)	Festkommazahl	123.45
FLOAT/DOUBLE	Fließkommazahl	3.14159
VARCHAR(n)	Variable Zeichenkette	"Hello World"
TEXT	Unbegrenzte Zeichenkette	Langer Text
BOOLEAN	Wahrheitswert	true, false
TIMESTAMP	Zeitstempel	2025-12-28 10:30:00
DATE	Datum	2025-12-28
JSON	JSON-Dokument	{"key": "value"}

Constraints

Primary Key:

```
CREATE TABLE customers (
  id INT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL
);
```

Foreign Key:

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  total DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Check Constraints:

```
CREATE TABLE products (  
  id INT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),  
  stock INT NOT NULL CHECK (stock >= 0),  
  discount DECIMAL(3, 2) CHECK (discount BETWEEN 0 AND 1)  
);
```

Unique Constraints:

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL  
);
```

Auto-Increment IDs

```
CREATE TABLE products (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL  
);  
  
-- Insert ohne ID  
INSERT INTO products (name) VALUES ('Laptop');  
-- → Automatisch id=1  
  
INSERT INTO products (name) VALUES ('Mouse');  
-- → Automatisch id=2
```


5.3 Daten einfügen, ändern, löschen

INSERT

```
-- Single Document
INSERT {
  id: 1,
  name: 'Laptop',
  price: 1200.00,
  stock: 15
} INTO products

-- Multiple Documents
INSERT [
  {id: 2, name: 'Mouse', price: 25.00, stock: 100},
  {id: 3, name: 'Keyboard', price: 80.00, stock: 50},
  {id: 4, name: 'Monitor', price: 350.00, stock: 20}
] INTO products
```

UPDATE

```
-- Update single document
UPDATE {id: 1} WITH {stock: stock - 1} IN products

-- Update multiple documents (with FOR loop)
FOR product IN products
  FILTER product.stock > 50
  UPDATE product WITH {
    price: product.price * 0.9
  } IN products

-- Update with join data
FOR order_item IN order_items
  FOR product IN products
    FILTER order_item.product_id == product.id
    UPDATE order_item WITH {
      price: product.price
    } IN order_items
```

REMOVE (DELETE)

```
-- Remove single document
REMOVE {id: 1} IN products

-- Remove multiple documents
FOR product IN products
  FILTER product.stock == 0
  REMOVE product IN products

-- Remove all (Vorsicht!)
FOR product IN products
  REMOVE product IN products
```

UPSERT (Insert or Update)

```
-- Insert or Update based on key
UPSERT {id: 1}
  INSERT {
    id: 1,
    name: 'Laptop',
    price: 1200.00,
    stock: 15
  }
  UPDATE {
    price: 1200.00,
    stock: 15
  }
IN products
```

5.4 Queries und Joins

Query Basics

```
-- All columns
FOR product IN products
  RETURN product

-- Specific columns
FOR product IN products
  RETURN {name: product.name, price: product.price}

-- With FILTER
FOR product IN products
  FILTER product.price > 100
  RETURN product

-- With SORT and LIMIT
FOR product IN products
  SORT product.price DESC
  LIMIT 10
  RETURN product

-- With aggregation functions
FOR product IN products
  COLLECT AGGREGATE
    avgPrice = AVG(product.price),
    maxPrice = MAX(product.price),
    minPrice = MIN(product.price),
    total = COUNT()
  RETURN {avgPrice, maxPrice, minPrice, total}

-- With GROUP BY (COLLECT)
FOR product IN products
  COLLECT category = product.category
  AGGREGATE
    count = COUNT(),
    avgPrice = AVG(product.price)
  RETURN {category, count, avgPrice}
```

INNER JOIN

```
-- Orders mit Customer-Namen (Multi-FOR Join)
FOR order IN orders
  FOR customer IN customers
    FILTER order.customer_id == customer.id
  RETURN {
    id: order.id,
    customer_name: customer.name,
    total: order.total
  }
```

```

graph LR
    subgraph "Table: customers"
        C1[id: 1<br/>name: Alice]
        C2[id: 2<br/>name: Bob]
        C3[id: 3<br/>name: Carol]
    end

    subgraph "Table: orders"
        O1[id: 101<br/>customer_id: 1<br/>total: 150]
        O2[id: 102<br/>customer_id: 2<br/>total: 200]
        O3[id: 103<br/>customer_id: 1<br/>total: 75]
    end

    O1 -. -> |FK: customer_id = 1| C1
    O2 -. -> |FK: customer_id = 2| C2
    O3 -. -> |FK: customer_id = 1| C1

    subgraph "JOIN Result"
        R1[Alice, 150]
        R2[Bob, 200]
        R3[Alice, 75]
    end

    O1 --> R1
    O2 --> R2
    O3 --> R3

    style C1 fill:#667eea
    style C2 fill:#667eea
    style C3 fill:#667eea
    style O1 fill:#4facfe
    style O2 fill:#4facfe
    style O3 fill:#4facfe
    style R1 fill:#43e97b
    style R2 fill:#43e97b
    style R3 fill:#43e97b

```

Abb. 05.1: Relationales Schema-Design

```

erDiagram
    CUSTOMER ||--o{ ORDER : places
    ORDER ||--|{ ORDER_ITEM : contains
    ORDER_ITEM }o--|| PRODUCT : references

    CUSTOMER {
        int id PK
        string name
        string email
        timestamp created_at
    }

    ORDER {
        int id PK
        int customer_id FK
        decimal total
        timestamp order_date
    }

    ORDER_ITEM {
        int id PK
        int order_id FK
        int product_id FK
        int quantity
        decimal price
    }

    PRODUCT {
        int id PK
        string name
        decimal price
        int stock
    }

```

Abb. 05.2: Join-Optimierung-Strategie

```

total: order.total,
customer_name: customer.name
}

```

****Visualisierung:****

```

customers orders +-----+ +-----+ | id=1 | ← -----|cust_id=1| ✓ Match → returned | Alice | | €100 |
+-----+ +-----+ | id=2 | | cust_id=1| ✓ Match → returned | Bob | | €50 | +-----+ +-----+ | id=3 |
| Carol | → Kein Match, nicht returned +-----+

```

```

### LEFT JOIN (Outer Join with COLLECT)

```aql
-- Alle Customers, mit/ohne Orders
FOR customer IN customers
 LET customer_orders = (
 FOR order IN orders
 FILTER order.customer_id == customer.id
 RETURN order
)
 RETURN {
 name: customer.name,
 order_count: LENGTH(customer_orders),
 total_spent: SUM(customer_orders[*].total)
 }

```

### Visualisierung:

```

customers orders
+-----+ +-----+
| id=1 | ←-----|cust_id=1| ✓ Match
| Alice | | €100 |
+-----+ +-----+
| id=2 | |
| Bob | | → Kein Order, aber returned mit NULL
+-----+ +-----+

```

## Multi-Table JOIN

```
-- Orders mit Items und Products (Multi-FOR Join)
FOR order IN orders
 FILTER order.created_at >= '2025-01-01'
 FOR customer IN customers
 FILTER order.customer_id == customer.id
 FOR order_item IN order_items
 FILTER order_item.order_id == order.id
 FOR product IN products
 FILTER order_item.product_id == product.id
 RETURN {
 order_id: order.id,
 customer: customer.name,
 product: product.name,
 quantity: order_item.quantity,
 price: order_item.price
 }
}
```

## Subqueries

```
-- Products teurer als Durchschnitt (WITH CTE)
LET avgPrice = (
 FOR product IN products
 COLLECT AGGREGATE avg = AVG(product.price)
 RETURN avg
)[0]

FOR product IN products
 FILTER product.price > avgPrice
 RETURN {name: product.name, price: product.price}

-- Customers mit mehr als 3 Orders (Subquery)
LET active_customers = (
 FOR order IN orders
 COLLECT customer_id = order.customer_id
 AGGREGATE order_count = COUNT()
 FILTER order_count > 3
 RETURN customer_id
)

FOR customer IN customers
 FILTER customer.id IN active_customers
 RETURN customer.name
```



## 5.5 Transaktionen

### ACID Garantien

**Atomicity:** Alles oder nichts

**Consistency:** Constraints werden eingehalten

**Isolation:** Transaktionen sehen sich nicht gegenseitig

**Durability:** Commits sind permanent

## Transaction Beispiel

```

from themis_client import ThemisDB

db = ThemisDB("localhost:8765")

tx = db.begin_transaction()

try:
 # 1. Reduce stock
 tx.execute("""
 FOR product IN products
 FILTER product.id == @product_id AND product.stock > 0
 UPDATE product WITH {stock: product.stock - 1} IN products
 """, {"product_id": product_id})

 # 2. Create order
 tx.execute("""
 INSERT {
 customer_id: @customer_id,
 product_id: @product_id,
 quantity: 1,
 price: @price
 } INTO orders
 """, {"customer_id": customer_id, "product_id": product_id, "price": price})

 # 3. Charge customer
 tx.execute("""
 FOR customer IN customers
 FILTER customer.id == @customer_id AND customer.balance >= @price
 UPDATE customer WITH {balance: customer.balance - @price} IN customers
 """, {"customer_id": customer_id, "price": price})

 # All OK → Commit
 tx.commit()
 print("Order successful!")

except Exception as e:
 # Error → Rollback
 tx.rollback()
 print(f"Order failed: {e}")

```

**Was passiert intern:**

```
T1: BEGIN TRANSACTION (snapshot @ version 100)
 → UPDATE products ... (write intent @ version 101)
 → INSERT orders ... (write intent @ version 102)
 → UPDATE customers ... (write intent @ version 103)
 → COMMIT
 → All write intents become visible @ version 104
```

Falls Fehler:

```
→ ROLLBACK
 → All write intents discarded
 → Daten unverändert
```

## Isolation Levels

ThemisDB verwendet **Snapshot Isolation**:

```
tx1 = db.begin_transaction()
tx1.execute("FOR p IN products FILTER p.id == 1 UPDATE p WITH {price: 100} IN products")

tx2 = db.begin_transaction()
result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → Alter Wert! (z.B. 90)

tx1.commit()

result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → Immer noch 90!

tx2.commit()

tx3 = db.begin_transaction()
result = tx3.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result) # → 100
```

## 5.6 Normalisierung

**Problem: Redundanz**

**Nicht-normalisiert:**

```
orders
```

```
+-----+-----+-----+-----+-----+
| id | customer | cust_email | product | quantity | price |
+-----+-----+-----+-----+-----+
| 1 | Alice | a@email.com | Laptop | 1 | 1200 |
| 2 | Alice | a@email.com | Mouse | 2 | 25 |
| 3 | Bob | b@email.com | Keyboard| 1 | 80 |
+-----+-----+-----+-----+-----+
```

**Probleme:** - Customer email wird wiederholt (Update Anomaly) - Product-Namen inkonsistent ("Laptop" vs "laptop") - Keine Constraints möglich

## Normalisierung: 1NF, 2NF, 3NF

### 1. Normal Form (1NF): Atomare Werte, keine Arrays

```
-- ✗ Nicht 1NF
CREATE TABLE orders (
 id INT,
 products TEXT -- "Laptop, Mouse, Keyboard"
);

-- ✓ 1NF
CREATE TABLE orders (
 id INT,
 product_id INT
);
```

### 2. Normal Form (2NF): Alle Nicht-Key-Attribute hängen vom ganzen Key ab

```
-- ✗ Nicht 2NF
CREATE TABLE order_items (
 order_id INT,
 product_id INT,
 quantity INT,
 product_name VARCHAR(255), -- Hängt nur von product_id ab!
 PRIMARY KEY (order_id, product_id)
);

-- ✓ 2NF: Separate products table
CREATE TABLE products (
 id INT PRIMARY KEY,
 name VARCHAR(255)
);

CREATE TABLE order_items (
 order_id INT,
 product_id INT,
 quantity INT,
 PRIMARY KEY (order_id, product_id),
 FOREIGN KEY (product_id) REFERENCES products(id)
);
```

### 3. Normal Form (3NF): Keine transitiven Abhängigkeiten

```
-- ✗ Nicht 3NF
CREATE TABLE products (
 id INT PRIMARY KEY,
 category_id INT,
 category_name VARCHAR(255) -- Hängt von category_id ab!
);

-- ✓ 3NF: Separate categories table
CREATE TABLE categories (
 id INT PRIMARY KEY,
 name VARCHAR(255)
);

CREATE TABLE products (
 id INT PRIMARY KEY,
 category_id INT,
 FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

## Denormalisierung für Performance

Manchmal ist **bewusste** Denormalisierung sinnvoll:

```
-- Denormalisiert für schnelle Queries
CREATE TABLE order_summary (
 order_id INT PRIMARY KEY,
 customer_id INT,
 customer_name VARCHAR(255), -- Denormalisiert!
 item_count INT, -- Computed!
 total DECIMAL(10, 2), -- Computed!
 created_at TIMESTAMP
);
```

**Trade-off:** -  Queries sind schneller (kein JOIN nötig) -  Updates sind komplexer (mehrere Tables updaten) -  Mehr Storage (Redundanz)

## 5.7 Indexes

### Warum Indexes?

**Ohne Index:**

```
FOR product IN products
 FILTER product.name == 'Laptop'
 RETURN product
-- → Scannt ALLE Zeilen (Seq Scan): O(n)
```

**Mit Index:**

```
CREATE INDEX idx_products_name ON products(name);

FOR product IN products
 FILTER product.name == 'Laptop'
 RETURN product
-- → Nutzt Index (Index Scan): O(log n)
```

**Performance-Unterschied:** - 1 Million rows: Seq Scan ~100ms, Index Scan ~0.1ms - **1000x schneller!**

### Index-Arten

#### 1. B-Tree Index (Standard):

```
CREATE INDEX idx_products_price ON products(price);

-- Gut für:
FOR product IN products
 FILTER product.price > 100
 RETURN product

FOR product IN products
 FILTER product.price >= 50 AND product.price <= 150
 RETURN product

FOR product IN products
 SORT product.price
 RETURN product
```

## 2. Unique Index:

```
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Verhindert Duplikate automatisch
INSERT {email: 'test@example.com'} INTO users -- OK
INSERT {email: 'test@example.com'} INTO users -- ERROR!
```

## 3. Composite Index:

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);

-- Gut für:
FOR order IN orders
 FILTER order.customer_id == 123 AND order.created_at > '2025-01-01'
 RETURN order

FOR order IN orders
 FILTER order.customer_id == 123
 RETURN order -- Auch OK!

-- Nicht gut für:
FOR order IN orders
 FILTER order.created_at > '2025-01-01'
 RETURN order -- Index nicht nutzbar!
```

## 4. Partial Index:

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Kleiner Index nur für pending orders
FOR order IN orders
 FILTER order.status == 'pending' AND order.created_at > '2025-01-01'
RETURN order
```

## Index Best Practices

✓ **Index erstellen für:** - Primary Keys (automatisch) - Foreign Keys (manuell) - WHERE-Spalten in häufigen Queries - ORDER BY-Spalten - JOIN-Spalten

✗ **Zu viele Indexes vermeiden:** - Jeder Index verlangsamt INSERT/UPDATE/DELETE - Jeder Index braucht Storage - **Faustregel:** Max 5-7 Indexes pro Table

---

## 5.8 Praxisbeispiel 1: Inventory System

### Überblick

Ein **Lagerverwaltungssystem** für ein kleines bis mittelgroßes Unternehmen: - Products mit Categories - Warehouses (mehrere Standorte) - Stock Levels pro Warehouse - Stock Movements (Ein-/Ausgang) - Suppliers

**Location:** [examples/04\\_inventory\\_system/](#)



## **Datenmodell**

```

-- Categories
CREATE TABLE categories (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(255) NOT NULL UNIQUE,
 description TEXT
);

-- Products
CREATE TABLE products (
 id INT PRIMARY KEY AUTO_INCREMENT,
 sku VARCHAR(50) NOT NULL UNIQUE,
 name VARCHAR(255) NOT NULL,
 description TEXT,
 category_id INT NOT NULL,
 unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0),
 min_stock_level INT DEFAULT 0,
 created_at TIMESTAMP DEFAULT NOW(),

 FOREIGN KEY (category_id) REFERENCES categories(id),
 INDEX idx_products_category (category_id),
 INDEX idx_products_sku (sku)
);

-- Warehouses
CREATE TABLE warehouses (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(255) NOT NULL,
 location VARCHAR(255),
 capacity INT
);

-- Stock Levels
CREATE TABLE stock_levels (
 product_id INT NOT NULL,
 warehouse_id INT NOT NULL,
 quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
 last_updated TIMESTAMP DEFAULT NOW(),

 PRIMARY KEY (product_id, warehouse_id),
 FOREIGN KEY (product_id) REFERENCES products(id),
 FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- Stock Movements
CREATE TABLE stock_movements (
 id INT PRIMARY KEY AUTO_INCREMENT,
 product_id INT NOT NULL,
 warehouse_id INT NOT NULL,
 quantity INT NOT NULL, -- Positive = Eingang, Negative = Ausgang

```

```

movement_type ENUM('purchase', 'sale', 'transfer', 'adjustment'),
reference VARCHAR(255),
created_at TIMESTAMP DEFAULT NOW(),

FOREIGN KEY (product_id) REFERENCES products(id),
FOREIGN KEY (warehouse_id) REFERENCES warehouses(id),
INDEX idx_movements_product (product_id),
INDEX idx_movements_warehouse (warehouse_id),
INDEX idx_movements_date (created_at)
);

-- Suppliers
CREATE TABLE suppliers (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(255) NOT NULL,
 contact_email VARCHAR(255),
 contact_phone VARCHAR(50)
);

-- Product Suppliers (Many-to-Many)
CREATE TABLE product_suppliers (
 product_id INT NOT NULL,
 supplier_id INT NOT NULL,
 supplier_sku VARCHAR(50),
 unit_cost DECIMAL(10, 2),

 PRIMARY KEY (product_id, supplier_id),
 FOREIGN KEY (product_id) REFERENCES products(id),
 FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
);

```

## Häufige Queries

### 1. Total Stock pro Product:

```

FOR p IN products
 FILTER p.id == @product_id
 LET total = SUM(sl.quantity FOR sl IN stock_levels FILTER sl.product_id == p.id)
 RETURN { name: p.name, total_stock: total }

```

### 2. Low Stock Alert:

```
FOR p IN products
 LET total = SUM(sl.quantity FOR sl IN stock_levels FILTER sl.product_id == p.id)
 FILTER total < p.min_stock_level
 SORT total ASC
 RETURN { sku: p.sku, name: p.name, total_stock: total, min_level: p.min_stock_level }
```

### 3. Stock Movement History:

```
LET cutoff = DATE_ADD(NOW(), {days: -@days})
FOR sm IN stock_movements
 FILTER sm.product_id == @product_id && sm.created_at >= cutoff
 LET warehouse = DOCUMENT(sm.warehouse_id)
 SORT sm.created_at DESC
 RETURN {
 created_at: sm.created_at,
 movement_type: sm.movement_type,
 quantity: sm.quantity,
 warehouse: warehouse.name,
 reference: sm.reference
 }
```

## Stock Movement Transaction

```
def record_purchase(product_id, warehouse_id, quantity, supplier_id, cost):
 """Purchase new stock from supplier"""
 tx = db.begin_transaction()
 try:
 # 1. Record movement
 tx.execute("""
 INSERT INTO stock_movements
 (product_id, warehouse_id, quantity, movement_type, reference)
 VALUES (?, ?, ?, 'purchase', ?)
 """, [product_id, warehouse_id, quantity, f"P0-{supplier_id}-{datetime.now().timestamp()}"])

 # 2. Update stock level
 tx.execute("""
 INSERT INTO stock_levels (product_id, warehouse_id, quantity)
 VALUES (?, ?, ?)
 ON CONFLICT (product_id, warehouse_id) DO UPDATE
 SET
 quantity = stock_levels.quantity + EXCLUDED.quantity,
 last_updated = NOW()
 """, [product_id, warehouse_id, quantity])

 # 3. Update product cost (optional)
 tx.execute("""
 UPDATE products
 SET unit_price = ?
 WHERE id = ?
 """, [cost, product_id])

 tx.commit()
 return {"success": True}

 except Exception as e:
 tx.rollback()
 return {"success": False, "error": str(e)}
```

## Reporting: Value per Warehouse

```
SELECT
 w.name AS warehouse,
 COUNT(DISTINCT sl.product_id) AS product_count,
 SUM(sl.quantity) AS total_units,
 SUM(sl.quantity * p.unit_price) AS total_value
FROM warehouses w
LEFT JOIN stock_levels sl ON w.id = sl.warehouse_id
LEFT JOIN products p ON sl.product_id = p.id
GROUP BY w.id, w.name
ORDER BY total_value DESC;
```

### Output:

```
+-----+-----+-----+-----+
| warehouse | product_count | total_units | total_value |
+-----+-----+-----+-----+
| Main Warehouse | 250 | 5,420 | €125,000 |
| Storage Berlin | 180 | 3,210 | €78,500 |
| Distribution Hub | 120 | 1,890 | €42,300 |
+-----+-----+-----+-----+
```

## 5.9 Praxisbeispiel 2: Expense Tracker

### Überblick

Ein **Ausgabenverwaltungssystem** für persönliche Finanzen oder kleine Teams: - Expenses mit Categories - Budgets pro Category - Recurring Expenses - Multi-Currency Support - Reports und Analytics

**Location:** [examples/12\\_expense\\_tracker/](#)

## **Datenmodell**

```

-- Categories
CREATE TABLE expense_categories (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL UNIQUE,
 color VARCHAR(7), -- Hex color #FF5733
 icon VARCHAR(50)
);

-- Expenses
CREATE TABLE expenses (
 id INT PRIMARY KEY AUTO_INCREMENT,
 description VARCHAR(255) NOT NULL,
 amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
 currency VARCHAR(3) DEFAULT 'EUR',
 category_id INT NOT NULL,
 expense_date DATE NOT NULL,
 payment_method ENUM('cash', 'credit_card', 'debit_card', 'transfer') DEFAULT 'cash',
 receipt_url VARCHAR(500),
 notes TEXT,
 created_at TIMESTAMP DEFAULT NOW(),

 FOREIGN KEY (category_id) REFERENCES expense_categories(id),
 INDEX idx_expenses_date (expense_date),
 INDEX idx_expenses_category (category_id)
);

-- Budgets
CREATE TABLE budgets (
 id INT PRIMARY KEY AUTO_INCREMENT,
 category_id INT NOT NULL,
 amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
 period ENUM('daily', 'weekly', 'monthly', 'yearly') DEFAULT 'monthly',
 start_date DATE NOT NULL,
 end_date DATE,

 FOREIGN KEY (category_id) REFERENCES expense_categories(id),
 UNIQUE KEY (category_id, start_date)
);

-- Recurring Expenses
CREATE TABLE recurring_expenses (
 id INT PRIMARY KEY AUTO_INCREMENT,
 description VARCHAR(255) NOT NULL,
 amount DECIMAL(10, 2) NOT NULL,
 category_id INT NOT NULL,
 frequency ENUM('daily', 'weekly', 'monthly', 'yearly') NOT NULL,
 start_date DATE NOT NULL,
 end_date DATE,

```



```

last_generated DATE,

FOREIGN KEY (category_id) REFERENCES expense_categories(id)
);

```

## Häufige Queries

### 1. Expenses für aktuellen Monat:

```

def get_monthly_expenses(year, month):
 return db.query("""
 SELECT
 e.expense_date,
 e.description,
 e.amount,
 c.name AS category,
 e.payment_method
 FROM expenses e
 JOIN expense_categories c ON e.category_id = c.id
 WHERE YEAR(e.expense_date) = ?
 AND MONTH(e.expense_date) = ?
 ORDER BY e.expense_date DESC
 """, [year, month])

```

### 2. Budget Status:

```

LET month_start = DATE_NEW(@year, @month, 1)
LET month_end = DATE_ADD(month_start, {months: 1})
FOR b IN budgets
 FILTER b.period == 'monthly' && b.start_date <= month_start
 LET category = DOCUMENT(b.category_id)
 LET spent = SUM(e.amount
 FOR e IN expenses
 FILTER e.category_id == b.category_id && e.expense_date >= month_start && e.expense_date <= month_end)
 RETURN {
 category: category.name,
 budget: b.amount,
 spent: spent || 0,
 remaining: b.amount - (spent || 0),
 percent_used: (spent || 0) / b.amount * 100
 }

```

### 3. Top Categories:

```
LET year_start = DATE_NEW(@year, 1, 1)
LET year_end = DATE_NEW(@year + 1, 1, 1)
FOR c IN expense_categories
 LET expenses_for_cat = (
 FOR e IN expenses
 FILTER e.category_id == c._id && e.expense_date >= year_start && e.expense_date < year_end
 RETURN e
)
 FILTER LENGTH(expenses_for_cat) > 0
 SORT SUM(e.amount FOR e IN expenses_for_cat) DESC
 LIMIT 10
 RETURN {
 name: c.name,
 expense_count: LENGTH(expenses_for_cat),
 total_amount: SUM(e.amount FOR e IN expenses_for_cat)
 }
```

## Add Expense Transaction

```
def add_expense(description, amount, category_id, expense_date, payment_method):
 """Add new expense and check budget"""
 tx = db.begin_transaction()
 try:
 # 1. Insert expense
 result = tx.execute("""
 INSERT INTO expenses
 (description, amount, category_id, expense_date, payment_method)
 VALUES (?, ?, ?, ?, ?)
 """, [description, amount, category_id, expense_date, payment_method])

 expense_id = result.last_insert_id

 # 2. Check budget in AQL
 budget_check = client.query("""
 LET month_start = DATE_NEW(DATE_YEAR(@date), DATE_MONTH(@date), 1)
 FOR b IN budgets
 FILTER b.category_id == @cat_id && b.period == 'monthly'
 LET spent = SUM(e.amount FOR e IN expenses
 FILTER e.category_id == b.category_id && e.expense_date >= month_start)
 RETURN {budget: b.amount, spent: spent || 0}
 """, {
 'date': expense_date,
 'cat_id': category_id
 })

 warning = None
 if budget_check and budget_check[0]['spent'] > budget_check[0]['budget']:
 warning = f"Budget exceeded! Spent: {budget_check[0]['spent']}, Budget: {budget_

 tx.commit()
 return {"success": True, "expense_id": expense_id, "warning": warning}

 except Exception as e:
 tx.rollback()
 return {"success": False, "error": str(e)}
```

## Recurring Expenses Generation

```
def generate_recurring_expenses():
 """Generate expenses from recurring templates"""
 tx = db.begin_transaction()
 generated = []

 try:
 # Find due recurring expenses
 recurring = tx.query("""
 SELECT * FROM recurring_expenses
 WHERE (last_generated IS NULL OR last_generated < CURDATE())
 AND (end_date IS NULL OR end_date >= CURDATE())
 """)

 for rec in recurring:
 # Calculate next date
 next_date = calculate_next_date(rec['frequency'], rec['last_generated'] or rec['start_date'])

 if next_date <= datetime.now().date():
 # Generate expense
 tx.execute("""
 INSERT INTO expenses
 (description, amount, category_id, expense_date, payment_method)
 VALUES (?, ?, ?, ?, 'transfer')
 """, [rec['description'], rec['amount'], rec['category_id'], next_date])

 # Update last_generated
 tx.execute("""
 UPDATE recurring_expenses
 SET last_generated = ?
 WHERE id = ?
 """, [next_date, rec['id']])

 generated.append(rec['description'])

 tx.commit()
 return {"success": True, "generated": generated}

 except Exception as e:
 tx.rollback()
 return {"success": False, "error": str(e)}
```

## Analytics: Monthly Trend

```
SELECT
 DATE_FORMAT(expense_date, '%Y-%m') AS month,
 COUNT(*) AS expense_count,
 SUM(amount) AS total_spent,
 AVG(amount) AS avg_expense
FROM expenses
WHERE expense_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month;
```

### Output:

```
+-----+-----+-----+-----+
| month | expense_count | total_spent | avg_expense |
+-----+-----+-----+-----+
| 2024-01 | 45 | €1,234 | €27 |
| 2024-02 | 52 | €1,456 | €28 |
| 2024-03 | 48 | €1,189 | €25 |
| ... | ... | ... | ... |
+-----+-----+-----+-----+
```

## 5.10 Performance Best Practices

### 1. Use Indexes Wisely

```
-- ❌ Slow: No index
FOR expense IN expenses
 FILTER expense.expense_date > '2025-01-01'
RETURN expense

-- ✅ Fast: With index
CREATE INDEX idx_expenses_date ON expenses(expense_date);
FOR expense IN expenses
 FILTER expense.expense_date > '2025-01-01'
RETURN expense
```

## 2. Avoid Returning All Fields

```
-- ❌ Transfers mehr Daten als nötig
FOR product IN products
 RETURN product

-- ✅ Nur benötigte Felder
FOR product IN products
 RETURN {id: product.id, name: product.name, price: product.price}
```

## 3. Use LIMIT für große Result Sets

```
-- ❌ Könnte Millionen Rows returnen
FOR order IN orders
 SORT order.created_at DESC
 RETURN order

-- ✅ Pagination
FOR order IN orders
 SORT order.created_at DESC
 LIMIT 0, 50
 RETURN order
```

## 4. Batch Inserts

```
for product in products:
 db.execute("INSERT INTO products (name, price) VALUES (?, ?)",
 [product.name, product.price])

db.execute("""
 INSERT INTO products (name, price) VALUES
 (?, ?), (?, ?), (?, ?), ...
 """, [p1.name, p1.price, p2.name, p2.price, ...])
```

## 5. Use Transactions für Bulk Operations

```
for i in range(1000):
 db.execute("INSERT INTO logs (...) VALUES (...)")

tx = db.begin_transaction()
for i in range(1000):
 tx.execute("INSERT INTO logs (...) VALUES (...)")
tx.commit()
```

## 6. Analyze Query Plans

```
EXPLAIN SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at > '2025-01-01';
```

### Output:

```
+-----+-----+-----+-----+-----+
| id | table | type | key | rows |
+-----+-----+-----+-----+-----+
| 1 | orders | range | idx | 1000 |
| 1 | customers | ref | pk | 1 |
+-----+-----+-----+-----+-----+
```

## 5.11 Erweiterte AQL-Optimierungen: Query-Plan und Performance-Tuning

### 5.11.1 Der Query-Optimizer: Kostenbasierte Umordnung

ThemisDB verwendet einen kostenbasierten Query-Optimizer [13], der den effizientesten Ausführungsplan für komplexe Queries automatisch wählt. Dies ist besonders wichtig bei Multi-Model-Queries, die relationale, graph- und vektor-basierte Operationen kombinieren.

#### Wie funktioniert der Optimizer?

```
// Pseudo-Code des ThemisDB Query Optimizers
class QueryOptimizer {
 // Kostenmodell für verschiedene Operationen
 map<OperationType, double> cost_estimates = {
 {INDEX_SCAN, 0.1}, // O(log n) - sehr schnell
 {HASH_JOIN, 1.0}, // O(n + m) - schnell für Equi-Joins
 {NESTED_LOOP_JOIN, 10.0}, // O(n * m) - langsam, aber flexibel
 {FULL_TABLE_SCAN, 100.0}, // O(n) - teuer für große Tabellen
 {SORT, 5.0}, // O(n log n) - mittel
 {AGGREGATE, 2.0} // O(n) - linear
 };

 ExecutionPlan optimize(Query query) {
 // 1. Erzeuge alle möglichen Ausführungspläne
 vector<ExecutionPlan> candidates = generatePlans(query);

 // 2. Schätze Kosten für jeden Plan
 for (auto& plan : candidates) {
 plan.estimated_cost = estimateCost(plan);
 }

 // 3. Wähle Plan mit niedrigsten Kosten
 return *min_element(candidates.begin(), candidates.end(),
 [](auto& a, auto& b) { return a.estimated_cost < b.estimated_cost; }
);
 }
};
```

### Beispiel: Join-Reihenfolge-Optimierung

```
-- Diese Query hat mehrere mögliche Ausführungspläne:
FOR order IN orders
 FILTER order.status == "pending"
 FOR customer IN customers
 FILTER customer._id == order.customer_id
 FILTER customer.country == "DE"
 FOR product IN products
 FILTER product._id == order.product_id
 FILTER product.category == "electronics"
 RETURN {
 order: order,
 customer: customer.name,
 product: product.name
 }
}
```

### Plan A: Naive Reihenfolge (schlecht)



1. Scan orders (1M rows) → 1M candidates
2. For each: Lookup customer → 1M lookups
3. Filter country="DE" → 100K candidates
4. For each: Lookup product → 100K lookups
5. Filter category → 10K final results

Total Cost:  $1M + 1M + 100K + 100K = \sim 2.2M$  operations

### Plan B: Optimizer-Reihenfolge (optimal)

1. Index scan: customers WHERE country="DE" → 100K candidates
2. Index scan: products WHERE category="electronics" → 50K candidates
3. Hash join: orders WHERE status="pending" (200K) with customers → 20K matches
4. Hash join: result with products → 10K final results

Total Cost:  $100K + 50K + 200K + 20K = \sim 370K$  operations (6x schneller!)

### Wie wird der optimale Plan gewählt?

Der Optimizer verwendet **Statistiken** über die Daten:

```
-- Statistiken für Optimizer sammeln
ANALYZE TABLE orders;
ANALYZE TABLE customers;
ANALYZE TABLE products;

-- Zeigt:
-- - Anzahl Zeilen pro Tabelle
-- - Kardinalität von Indizes
-- - Häufigkeitsverteilung von Werten
-- - Größe der Tabellen auf Disk
```

### 5.11.2 Index-Only-Scans: Minimale Disk-I/O

Ein **Index-Only-Scan** ist möglich, wenn alle benötigten Spalten bereits im Index enthalten sind, ohne dass die eigentliche Tabelle gelesen werden muss [2], [3].

**Beispiel ohne Index-Only-Scan:**

```
CREATE INDEX idx_orders_customer ON orders (customer_id);

-- Diese Query MUSS die orders-Tabelle lesen:
FOR order IN orders
 FILTER order.customer_id == "customer_123"
 RETURN {
 id: order._id,
 total: order.total_amount, -- Nicht im Index!
 date: order.created_at -- Nicht im Index!
 }
```

**Ausführungsplan:**

1. Index Scan: idx\_orders\_customer → Findet 50 order\_ids
2. Table Lookup: orders → Liest 50 volle Dokumente (Disk I/O!)

**Optimiert mit Composite Index:**

```
-- Composite Index mit allen benötigten Spalten
CREATE INDEX idx_orders_customer_covering
 ON orders (customer_id, total_amount, created_at);

-- Jetzt: Index-Only-Scan möglich!
FOR order IN orders
 FILTER order.customer_id == "customer_123"
 RETURN {
 id: order._id,
 total: order.total_amount, -- Im Index!
 date: order.created_at -- Im Index!
 }
```

**Ausführungsplan:**

1. Index-Only Scan: idx\_orders\_customer\_covering → Liest nur Index
2. Keine Table Lookups nötig → 10x schneller!

**Performance-Vergleich:**

Ansatz	Disk I/O	Latenz	Durchsatz
Ohne Index	50 random reads	50ms	1000 queries/sec
Mit Index (nicht covering)	50 index reads + 50 table reads	25ms	2000 queries/sec
Mit Covering Index	50 index reads	5ms	10000 queries/sec

### 5.11.3 Partielle Indizes: Selektive Indexierung

Wenn nur ein Teil der Daten häufig abgefragt wird, kann ein **partieller Index** Speicherplatz und Schreibperformance sparen:

```
-- Indexiere nur aktive Orders
CREATE INDEX idx_active_orders
 ON orders (customer_id, created_at)
 WHERE status IN ['pending', 'processing'];

-- Dieser Index ist viel kleiner als ein voller Index,
-- da er nur ~10% der orders enthält (die aktiven)
```

#### Wann wird der partielle Index verwendet?

```
-- ✅ Optimizer nutzt partiellen Index:
FOR order IN orders
 FILTER order.status == "pending"
 FILTER order.customer_id == "customer_123"
 RETURN order

-- ❌ Optimizer nutzt NICHT den partiellen Index:
FOR order IN orders
 FILTER order.status == "completed" -- Nicht im Index!
 FILTER order.customer_id == "customer_123"
 RETURN order
```

### 5.11.4 Materialized Views für komplexe Aggregationen

Für häufig ausgeführte, teure Aggregationen bietet ThemisDB **Materialized Views**:

```
-- Teure Aggregation (läuft jedes Mal 5 Sekunden):
FOR order IN orders
 FILTER order.created_at >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
 COLLECT
 customer = order.customer_id,
 date = DATE_TRUNC(order.created_at, 'day')
 AGGREGATE
 revenue = SUM(order.total_amount),
 order_count = COUNT(1)
 RETURN {customer, date, revenue, order_count}
```

### Lösung: Materialized View

```
-- Erstelle Materialized View (einmalig):
CREATE MATERIALIZED VIEW daily_revenue AS
 FOR order IN orders
 COLLECT
 customer = order.customer_id,
 date = DATE_TRUNC(order.created_at, 'day')
 AGGREGATE
 revenue = SUM(order.total_amount),
 order_count = COUNT(1)
 RETURN {customer, date, revenue, order_count}
 OPTIONS {refresh: 'incremental', interval: '5 minutes'}

-- Abfrage der View (instant!):
FOR row IN daily_revenue
 FILTER row.date >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
 RETURN row
```

### Refresh-Strategien:

- **refresh: 'immediate'** - Nach jeder Änderung aktualisieren (langsam)
- **refresh: 'incremental'** - Nur geänderte Daten neu berechnen (empfohlen)
- **refresh: 'full'** - Komplette Neuberechnung (für kleine Views)

### 5.11.5 Query-Hints: Manuelle Optimizer-Steuerung

In seltenen Fällen weiß der Entwickler mehr als der Optimizer. Dann können **Query-Hints** helfen:

```
-- Force Index Scan (statt Full Table Scan):
FOR order IN orders
 OPTIONS {indexHint: 'idx_orders_customer'}
 FILTER order.customer_id == "customer_123"
 RETURN order

-- Force Hash Join (statt Nested Loop):
FOR order IN orders
 FOR customer IN customers
 OPTIONS {joinStrategy: 'hash'}
 FILTER customer._id == order.customer_id
 RETURN {order, customer}

-- Disable Index für Debugging:
FOR order IN orders
 OPTIONS {forceTableScan: true}
 FILTER order.customer_id == "customer_123"
 RETURN order
```

### Wann Query-Hints verwenden?

 **Sinnvoll:** - Temporär zum Debugging - Wenn Statistiken veraltet sind - Für extrem spezifische Workloads

 **Vermeiden:** - Als Standard (Optimizer ist meist besser) - Wenn Datenvolumen sich ändert - In produktivem Code ohne Dokumentation

### 5.11.6 Batch-Operations für hohen Durchsatz

Für Massen-Inserts oder Updates bietet ThemisDB Batch-Operations:

```
-- ✗ Langsam: 10.000 einzelne Inserts
FOR i IN 1..10000
 INSERT {
 name: CONCAT("Product ", i),
 price: RAND() * 1000,
 category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
 } INTO products

-- ✔ Schnell: Batch-Insert
LET batch = (
 FOR i IN 1..10000
 RETURN {
 name: CONCAT("Product ", i),
 price: RAND() * 1000,
 category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
 }
)
INSERT batch INTO products

-- Performance: 100x schneller!
-- Einzeln: ~50 inserts/sec (10.000 inserts = 200 Sekunden)
-- Batch: ~50.000 inserts/sec (10.000 inserts = 0.2 Sekunden)
```

### Warum ist Batch schneller?

1. **Weniger Transaktionen:** 1 statt 10.000 Transaktions-Overheads
2. **Batch-Writes:** RocksDB kann Writes zusammenfassen
3. **Weniger Netzwerk-RTTs:** 1 Request statt 10.000

### 5.11.7 Performance-Monitoring mit EXPLAIN

Verwenden Sie immer `EXPLAIN`, um langsame Queries zu debuggen:

```
-- Detaillierter Execution Plan:
EXPLAIN
 FOR order IN orders
 FILTER order.status == "pending"
 FOR customer IN customers
 FILTER customer._id == order.customer_id
 FILTER customer.country == "DE"
 RETURN {order, customer}
```

**Output:**

```

{
 "plan": {
 "nodes": [
 {
 "type": "IndexNode",
 "index": "idx_orders_status",
 "condition": "status == 'pending'",
 "estimated_items": 200000,
 "estimated_cost": 1000
 },
 {
 "type": "IndexNode",
 "index": "idx_customers_pk",
 "condition": "customer._id == order.customer_id",
 "estimated_items": 1,
 "estimated_cost": 10
 },
 {
 "type": "FilterNode",
 "condition": "customer.country == 'DE'",
 "estimated_items": 20000,
 "estimated_cost": 200
 }
],
 "total_estimated_cost": 1210,
 "total_estimated_items": 20000
 },
 "stats": {
 "execution_time": "125ms",
 "items_scanned": 200000,
 "items_returned": 20000,
 "index_hits": 200001,
 "index_misses": 0
 }
}

```

**Key Metrics:** - **estimated\_cost**: Optimizer's Kostenschätzung - **estimated\_items**: Erwartete Anzahl Ergebnisse - **execution\_time**: Tatsächliche Laufzeit - **index\_hits/misses**: Index-Effizienz

## 5.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Relationales Modell**: Tabellen, Schemas, Constraints
- ✓ **AQL DDL**: CREATE TABLE, Datentypen, Constraints
- ✓ **AQL DML**: INSERT, UPDATE, DELETE, UPSERT

- 
- ✓ **Queries:** SELECT, WHERE, JOIN, Subqueries, Aggregation
  - ✓ **Transaktionen:** ACID, Isolation, Commit/Rollback
  - ✓ **Normalisierung:** 1NF, 2NF, 3NF, Denormalisierung
  - ✓ **Indexes:** B-Tree, Unique, Composite, Performance
  - ✓ **Erweiterte Optimierungen:** Query Optimizer, Index-Only-Scans, Materialized Views
  - ✓ **Praxis:** Inventory System & Expense Tracker

## Key Takeaways

1. **Relational ist perfekt für strukturierte Business-Daten**
2. **Constraints garantieren Datenintegrität**
3. **Normalisierung verhindert Redundanz**
4. **Denormalisierung kann Performance verbessern**
5. **Indexes sind essentiell für Query-Performance**
6. **Transaktionen garantieren Konsistenz**
7. **Der Query-Optimizer wählt automatisch den besten Ausführungsplan**
8. **Covering Indexes ermöglichen Index-Only-Scans für maximale Performance**

## Nächster Schritt

Sie verstehen jetzt relationale Daten in ThemisDB. Im nächsten Kapitel lernen Sie **Graph-Datenbanken** kennen - perfekt für Netzwerk-Daten und Beziehungen.

[Kapitel 6: Graph-Datenbanken →](#)

---

## Weiterführende Ressourcen

- **AQL Reference:** [../de/aql/README.md](#)
- **Schema Design:** [../de/guides/guides\\_schema\\_design.md](#)
- **Transaction Guide:** [../de/features/features\\_transactions.md](#)
- **Inventory System Code:** [../examples/04\\_inventory\\_system/](#)
- **Expense Tracker Code:** [../examples/12\\_expense\\_tracker/](#)

---

Kapitel 5 von 30 | Teil II: Datenmodelle | ~12.200 Wörter

---



## Kapitel 8: Kapitel 6: Graph-Datenbanken

*"The world is a graph, not a table." — Graph-Datenbank-Community*

### 6.1 Einführung: Die Welt der Verbindungen

Die Welt besteht aus Beziehungen. Menschen kennen andere Menschen. Websites verlinken auf andere Websites. Produkte werden zusammen gekauft. Straßen verbinden Städte. Diese natürlich vernetzten Strukturen lassen sich am besten als **Graphen** darstellen.

#### Das Problem mit Relationalen Datenbanken

Versuchen Sie, folgende Frage mit SQL zu beantworten: *"Finde alle Freunde meiner Freunde, die in derselben Stadt wohnen und mindestens 3 gemeinsame Interessen haben."*

Mit relationalen Tabellen benötigen Sie: - Multiple Self-Joins über die **friendships**-Tabelle - Subqueries für gemeinsame Interessen - Performance-Probleme bei mehr als 3-4 Hop-Levels - Komplexe Query-Logik, die schwer zu warten ist

```
-- Freunde von Freunden (nur 2 Hops!)
SELECT DISTINCT u3.*
FROM users u1
JOIN friendships f1 ON u1.id = f1.user_id
JOIN users u2 ON f1.friend_id = u2.id
JOIN friendships f2 ON u2.id = f2.user_id
JOIN users u3 ON f2.friend_id = u3.id
WHERE u1.id = '...'
 AND u3.city = u1.city
 AND ... -- gemeinsame Interessen?
```

Diese Query wird schnell unleserlich und langsam.

#### Die Graph-Lösung

In ThemisDB wird dieselbe Frage intuitiv und performant:

```

result = db.graph_query("""
 FOR user IN users
 FILTER user.id == @my_id
 FOR friend IN 1..2 OUTBOUND user friendships
 FILTER friend.city == user.city
 LET common_interests = LENGTH(
 INTERSECTION(user.interests, friend.interests)
)
 FILTER common_interests >= 3
 RETURN friend
""", bind_vars={"my_id": my_id})

```

Die Graph-Query ist: -  Lesbarer und deklarativer -  Beliebige viele Hops möglich ( [1..10](#) , [1..ANY](#) ) -  Performance skaliert mit Graph-Struktur, nicht mit Datenmenge -  Native Graph-Algorithmen verfügbar

## 6.2 Property Graph Modell

ThemisDB verwendet das **Property Graph**-Modell, den De-facto-Standard für Graph-Datenbanken.

### Komponenten

**1. Knoten (Vertices/Nodes)** - Repräsentieren Entitäten (Personen, Orte, Produkte) - Haben einen eindeutigen Identifier - Können beliebige Properties haben - Können Labels/Types haben

```

user_node = {
 "id": "user_001",
 "type": "User",
 "name": "Alice",
 "age": 28,
 "city": "Berlin",
 "interests": ["Python", "ML", "Hiking"]
}

```

```
graph LR
 subgraph "Property Graph Struktur"
 A((Alice
User
age: 28
city: Berlin))
 B((Bob
User
age: 32
city: Hamburg))
 C((Carol
User
age: 25
city: Berlin))
 P1[Python
Projekt]
 P2[ML
Workshop]

 A -->|FRIEND
since: 2023
strength: 0.85| B
 A -->|FRIEND
since: 2024
strength: 0.92| C
 B -->|FRIEND
since: 2022
strength: 0.78| C

 A -. ->|WORKS_ON| P1
 B -. ->|WORKS_ON| P1
 C -. ->|ATTENDS| P2
 end

 style A fill:#667eea
 style B fill:#764ba2
 style C fill:#f093fb
 style P1 fill:#4facfe
 style P2 fill:#43e97b
```

**Abb. 06.1:** Graph-Traversierung-Algorithmus

**2. Kanten (Edges/Relationships)** - Verbinden zwei Knoten (gerichtet oder ungerichtet) - Haben einen Typ (z.B. "FRIEND", "LIKES", "WORKS\_AT") - Können Properties haben (z.B. "since", "strength") - Erlauben Multi-Edges (mehrere Kanten zwischen denselben Knoten)

```
friendship_edge = {
 "from": "user_001",
 "to": "user_002",
 "type": "FRIEND",
 "since": "2023-05-15",
 "strength": 0.85, # 0-1 basierend auf Interaktionen
 "mutual_friends": 12
}
```

**3. Graph-Collections**

ThemisDB organisiert Graphs in Collections: - **Vertex Collections:** Speichern Knoten (z.B. `users`, `posts`) - **Edge Collections:** Speichern Kanten (z.B. `friendships`, `likes`)

```

db.create_vertex_collection("users")
db.create_edge_collection("friendships")

graph_def = {
 "name": "social_network",
 "edge_definitions": [{
 "collection": "friendships",
 "from": ["users"],
 "to": ["users"]
 }]
}
db.create_graph(graph_def)

```

## Gerichtete vs. Ungerichtete Kanten

### Gerichtet (Directed):

```

{"from": "alice", "to": "bob", "type": "FOLLOWS"}

```

Beispiele: Twitter-Follows, Hyperlinks, Reporting-Struktur

### Ungerichtet (Undirected):

```

{"from": "alice", "to": "bob", "type": "FRIEND"}
{"from": "bob", "to": "alice", "type": "FRIEND"} # Beide Richtungen

```

Beispiele: Facebook-Freunde, Co-Autoren, Straßen

ThemisDB speichert alle Kanten gerichtet, aber Graph-Queries können beide Richtungen traversieren:

- **OUTBOUND** - Von A nach B - **INBOUND** - Von B nach A - **ANY** - Beide Richtungen (für ungerichtete Graphs)

```

graph TB
 subgraph "Gerichtete Kanten (Directed)"
 A1((Alice)) -->|FOLLOWS| B1((Bob))
 B1 -->|FOLLOWS| C1((Carol))
 Note1[Asymmetrisch:
Alice folgt Bob,
aber nicht umgekehrt]
 end

 subgraph "Ungerichtete Kanten (Undirected)"
 A2((Alice)) <-->|FRIEND| B2((Bob))
 B2 <-->|FRIEND| C2((Carol))
 A2 <-->|FRIEND| C2
 Note2[Symmetrisch:
Beide Richtungen
gleich gewichtet]
 end

 style A1 fill:#667eea
 style B1 fill:#764ba2
 style C1 fill:#f093fb
 style A2 fill:#667eea
 style B2 fill:#764ba2
 style C2 fill:#f093fb

```

Abb. 06.2: Shortest-Path-Berechnung

## 6.3 Graph-Traversierung

### Traversierungs-Arten

**1. Depth-First Search (DFS)** - Geht zuerst in die Tiefe - Verwendet Stack - Findet schnell tiefe Pfade

```

FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
 OPTIONS {bfs: false} # DFS (default)
 RETURN {vertex: v, path: p}

```

**2. Breadth-First Search (BFS)** - Geht zuerst in die Breite - Verwendet Queue - Findet kürzeste Pfade

```

FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
 OPTIONS {bfs: true} # BFS
 RETURN {vertex: v, path: p}

```

**3. Bounded Traversal** - Limitiert Anzahl der Hops - **1..1** - Nur direkte Nachbarn - **1..2** - Bis zu 2 Hops - **2..4** - Zwischen 2 und 4 Hops - **1..ANY** - Alle erreichbaren Knoten

```
FOR v IN 2..2 OUTBOUND @my_id friendships
RETURN v
```

```
graph TD
 Start((Alice
Start)) -->|Hop 1| F1((Bob
Friend))
 Start -->|Hop 1| F2((Carol
Friend))
 Start -->|Hop 1| F3((Dave
Friend))

 F1 -->|Hop 2| FF1((Eve
Friend of Friend))
 F1 -->|Hop 2| FF2((Frank
Friend of Friend))

 F2 -->|Hop 2| FF3((Grace
Friend of Friend))
 F2 -->|Hop 2| Start

 F3 -->|Hop 2| FF4((Henry
Friend of Friend))

 style Start fill:#667eea,stroke:#333,stroke-width:4px
 style F1 fill:#4facfe
 style F2 fill:#4facfe
 style F3 fill:#4facfe
 style FF1 fill:#43e97b
 style FF2 fill:#43e97b
 style FF3 fill:#43e97b
 style FF4 fill:#43e97b
```

**Abb. 06.3:** Community-Detection-Workflow

```

flowchart LR
 subgraph "DFS - Depth First Search"
 D_Start((1)) --> D_A((2))
 D_A --> D_AA((3))
 D_AA --> D_AAA((4))
 D_A --> D_AB((5))
 D_Start --> D_B((6))
 end

 subgraph "BFS - Breadth First Search"
 B_Start((1)) --> B_A((2))
 B_Start --> B_B((3))
 B_A --> B_AA((4))
 B_A --> B_AB((5))
 B_B --> B_BA((6))
 end

 style D_Start fill:#667eea
 style B_Start fill:#667eea

```

**Abb. 06.4:** Pagerank-Algorithmus-Visualisierung

## Pattern Matching

Graph-Queries in ThemisDB unterstützen komplexe Patterns:

```

FOR a IN users
 FOR b IN OUTBOUND a friendships
 FOR c IN OUTBOUND b friendships
 FILTER c._id == a._id
 RETURN {a: a.name, b: b.name, c: c.name}

```

```

FOR start IN users FILTER start.id == @user_id
 FOR end IN OUTBOUND start friendships
 LET paths = (
 FOR v, e, p IN 2..2 OUTBOUND start friendships
 FILTER v._id == end._id
 RETURN p
)
 FILTER LENGTH(paths) > 1 # Mehrere Pfade
 RETURN {start: start.name, end: end.name, paths: paths}

```

## 6.4 Graph-Algorithmen

ThemisDB bietet native Implementierungen gängiger Graph-Algorithmen:

### Shortest Path (Dijkstra)

Findet den kürzesten Pfad zwischen zwei Knoten unter Berücksichtigung von Gewichten:

```
from themis_client import shortest_path

path = shortest_path(
 graph="social_network",
 start_vertex="users/alice",
 target_vertex="users/bob",
 direction="outbound",
 weight_attribute="strength" # Optional: Kantengewicht
)

print(f"Pfad: {' -> '.join([v['name'] for v in path['vertices']])}")
print(f"Distanz: {path['distance']}")
```

**Use Cases:** - Routing und Navigation - Social Distance berechnen - Kostenoptimale Pfade finden

### All Shortest Paths

Findet alle kürzesten Pfade (falls mehrere existieren):

```
paths = db.all_shortest_paths(
 graph="social_network",
 start_vertex="users/alice",
 target_vertex="users/bob"
)

for idx, path in enumerate(paths):
 print(f"Pfad {idx+1}: {' -> '.join([v['name'] for v in path['vertices']])}")
```

### K Shortest Paths

Findet die k kürzesten Pfade:



```
paths = db.k_shortest_paths(
 graph="social_network",
 start_vertex="users/alice",
 target_vertex="users/bob",
 k=3 # Top 3 kürzeste Pfade
)
```

## Connected Components

Findet zusammenhängende Teilgraphen:

```
components = db.connected_components(
 graph="social_network",
 direction="any" # Ungerichtet behandeln
)

for component_id, vertices in components.items():
 print(f"Community {component_id}: {len(vertices)} Mitglieder")
```

**Use Cases:** - Community-Erkennung - Isolierte Gruppen finden - Netzwerk-Fragmentierung analysieren

## PageRank

Berechnet die Wichtigkeit von Knoten basierend auf eingehenden Kanten:

```
pagerank_scores = db.pagerank(
 graph="social_network",
 iterations=20,
 damping_factor=0.85
)

top_users = sorted(
 pagerank_scores.items(),
 key=lambda x: x[1],
 reverse=True
)[:10]

for user_id, score in top_users:
 user = db.get("users", user_id)
 print(f"{user['name']}: {score:.4f}")
```

**Use Cases:** - Influencer-Identifikation - Content-Ranking - Reputations-Systeme

## Betweenness Centrality

Misst, wie oft ein Knoten auf kürzesten Pfaden liegt:

```
centrality = db.betweenness centrality(
 graph="social_network"
)

bridges = [k for k, v in centrality.items() if v > 0.5]
```

**Use Cases:** - Netzwerk-Bottlenecks identifizieren - Wichtige Vermittler finden - Kritische Infrastruktur-Knoten

## 6.4A Temporale Graph-Queries: Zeitreisen im Wissensgraph

Eine der mächtigsten Features von ThemisDB ist die Fähigkeit, **zeitabhängige Graph-Traversals** durchzuführen. Dies ist besonders wichtig für Anwendungen, die historische Zustände nachvollziehen müssen – etwa für Compliance, Audit, oder rechtssichere Dokumentation.

### Das Problem: Wie sah der Graph *damals* aus?

Stellen Sie sich folgende Szenarien vor:

#### Szenario 1 - Compliance-Anfrage:

"Welche Zugriffsrechte hatte Benutzer X am 15. März 2024 um 14:30 Uhr?"

#### Szenario 2 - Verwaltungsakt:

"Auf welcher Datengrundlage basierte der Bescheid vom 10. Januar 2024? Welche Dokumente waren zu diesem Zeitpunkt verknüpft?"

#### Szenario 3 - Betrugserkennung:

"Zu welchen verdächtigen Konten hatte Account Y Verbindungen in der Woche vor der Sperrung?"

In traditionellen Graph-Datenbanken müssten Sie entweder: 1. **Alle Änderungen manuell versionieren** (aufwändig, fehleranfällig) 2. **Snapshot-Backups** erstellen (speicherintensiv, grobe Granularität) 3. **Audit-Logs parsen** (langsam, komplex)

### Die Lösung: Temporale Kanten mit `valid_from/valid_to`

ThemisDB erlaubt es, Kanten mit Gültigkeitszeiträumen zu versehen:

```

db.create_edge(
 from_node="users/alice",
 to_node="departments/engineering",
 edge_type="WORKS_IN",
 properties={
 "role": "Senior Engineer",
 "valid_from": 1640995200000, # 2022-01-01 00:00:00 UTC (ms)
 "valid_to": 1704067200000 # 2024-01-01 00:00:00 UTC (ms)
 }
)

db.create_edge(
 from_node="users/alice",
 to_node="departments/research",
 edge_type="WORKS_IN",
 properties={
 "role": "Lead Researcher",
 "valid_from": 1704067200000, # 2024-01-01 00:00:00 UTC (ms)
 "valid_to": None # Aktuell gültig (kein Enddatum)
 }
)

```

### Semantik der temporalen Felder:

- `valid_from = null` : Kante gilt seit Anbeginn der Zeit
- `valid_to = null` : Kante gilt unbegrenzt in die Zukunft
- `valid_from = T1, valid_to = T2` : Kante galt im Intervall [T1, T2]
- Beide `null` : Kante ist zeitlos (eternal)

## Point-in-Time Queries: Graph-Zustand zu einem Zeitpunkt

### API: `bfsAtTime()` - Breadth-First Search mit Zeitfilter

```

// C++ API Signatur
std::pair<Status, std::vector<std::string>> bfsAtTime(
 std::string_view startPk, // Startknoten
 int64_t timestamp_ms, // Zeitpunkt (ms seit Epoch)
 int maxDepth = 3 // Maximale Traversierungstiefe
) const;

```

### Wie es funktioniert:

Die `bfsAtTime()`-Funktion durchläuft den Graphen wie eine normale BFS, aber mit einem entscheidenden Unterschied: **Jede Kante wird vor der Traversierung auf Gültigkeit geprüft.**

**Der Algorithmus Schritt-für-Schritt:**

```

// Pseudo-Code der temporalen BFS-Implementation
bool isValidAtTime(Edge edge, int64_t query_time) {
 // Fall 1: Kante noch nicht gültig
 if (edge.valid_from.has_value() && query_time < edge.valid_from) {
 return false; // Zu früh!
 }

 // Fall 2: Kante nicht mehr gültig
 if (edge.valid_to.has_value() && query_time > edge.valid_to) {
 return false; // Zu spät!
 }

 // Fall 3: Kante war zum Query-Zeitpunkt gültig
 return true;
}

std::vector<std::string> bfsAtTime(string start, int64_t timestamp, int maxDepth) {
 queue<Node> frontier;
 set<string> visited;
 vector<string> result;

 frontier.push({start, 0}); // {node_id, depth}
 visited.insert(start);

 while (!frontier.empty()) {
 auto [current_node, depth] = frontier.front();
 frontier.pop();

 if (depth > maxDepth) continue;

 result.push_back(current_node);

 // Hole alle ausgehenden Kanten
 for (auto& edge : getOutgoingEdges(current_node)) {
 // KRITISCH: Temporaler Filter!
 if (!isValidAtTime(edge, timestamp)) {
 continue; // Kante überspringen
 }

 if (visited.count(edge.to) == 0) {
 visited.insert(edge.to);
 frontier.push({edge.to, depth + 1});
 }
 }
 }

 return result;
}

```

<b>Der</b>	<b>entscheidende</b>	<b>Unterschied:</b>	<b>Die</b>	<b>Zeile</b>

`if (!isValidAtTime(edge, timestamp)) continue;` filtert historisch ungültige Kanten **vor** der Traversierung. Dadurch sieht die BFS exakt den Graph-Zustand, wie er zum Query-Zeitpunkt existierte.

**Praktisches Beispiel:**

```
timestamp = datetime(2023, 7, 1).timestamp() * 1000 # In Millisekunden

result = db.graph.bfsAtTime(
 start="users/alice",
 timestamp_ms=int(timestamp),
 max_depth=3
)

print(f"Erreichbare Knoten am 1. Juli 2023: {result}")
```

**Shortest Path mit Zeitfilter: dijkstraAtTime()**

Für gewichtete Graphen unterstützt ThemisDB auch temporale Kürzeste-Pfad-Suche:

```
// C++ API Signatur
std::pair<Status, PathResult> dijkstraAtTime(
 std::string_view startPk,
 std::string_view targetPk,
 int64_t timestamp_ms
) const;

// PathResult enthält:
struct PathResult {
 std::vector<std::string> path; // Knoten vom Start zum Ziel
 double totalCost; // Gesamtkosten des Pfades
};
```

**Use Case: Organisationshierarchie zum Zeitpunkt eines Bescheids**

```
timestamp = datetime(2024, 1, 10, 14, 30).timestamp() * 1000

path_result = db.graph.dijkstraAtTime(
 start="users/sachbearbeiter-123",
 target="users/abteilungsleiter-456",
 timestamp_ms=int(timestamp)
)

if path_result.status.ok:
 print(f"Eskalationspfad am 10.01.2024:")
 for i, node in enumerate(path_result.path):
 print(f" {i}. {node}")
 print(f"Hierarchie-Distanz: {path_result.totalCost}")
```

**Ausgabe:**

```
Eskalationspfad am 10.01.2024:
0. users/sachbearbeiter-123
1. users/teamleiter-789
2. users/abteilungsleiter-456
Hierarchie-Distanz: 2.0
```

**Warum ist das wichtig?**

Wenn später ein Bescheid angefochten wird und die Frage aufkommt: "War die Eskalation regelkonform?", können Sie **exakt nachweisen**, welche Hierarchie zum Zeitpunkt der Entscheidung galt – auch wenn die Organisation sich seitdem umstrukturiert hat.

**Time-Range Queries: Kanten in einem Zeitfenster**

Manchmal interessiert nicht ein einzelner Zeitpunkt, sondern ein **Zeitraum**:

"Welche Zugriffsrechte überlappten mit dem Quartal Q4 2024?"

```
// C++ API für Time-Range Queries
struct TimeRangeFilter {
 int64_t start_ms; // Fenster-Start
 int64_t end_ms; // Fenster-Ende

 // Prüft ob Kante mit Zeitfenster überlappt
 bool hasOverlap(optional<int64_t> edge_valid_from,
 optional<int64_t> edge_valid_to) const;

 // Prüft ob Kante vollständig im Fenster enthalten ist
 bool fullyContains(optional<int64_t> edge_valid_from,
 optional<int64_t> edge_valid_to) const;
};

std::pair<Status, std::vector<EdgeInfo>> getEdgesInTimeRange(
 int64_t range_start_ms,
 int64_t range_end_ms,
 bool require_full_containment = false
) const;
```

### Beispiel: Audit-Abfrage für Q4 2024

```
q4_start = datetime(2024, 10, 1).timestamp() * 1000
q4_end = datetime(2024, 12, 31, 23, 59, 59).timestamp() * 1000

edges = db.graph.getEdgesInTimeRange(
 range_start_ms=int(q4_start),
 range_end_ms=int(q4_end),
 require_full_containment=False # Überlappung reicht
)

print(f"Gefunden: {len(edges)} Kanten mit Überlappung zu Q4 2024")

for edge in edges:
 print(f" {edge.from_pk} → {edge.to_pk}")
 print(f" Gültig: {edge.valid_from} bis {edge.valid_to}")
```

### Überlappungs-Logik visualisiert:



Query-Zeitfenster:	[————Q4 2024————]	
	Oct 1	Dec 31
Kante A:	[————]	✓ Überlappt (vor Q4, endet in Q4)
Kante B:	[————]	✓ Überlappt (komplett in Q4)
Kante C:	[————]	✓ Überlappt (startet in Q4, endet nach Q4)
Kante D:	[——]	✗ Keine Überlappung (endet vor Q4)
Kante E:	[—]	✗ Keine Überlappung (startet nach Q4)

## Revisionssicherheit: Verwaltungsakte dokumentieren

### Das Szenario:

- Eine Behörde erstellt einen Bescheid am **15. März 2024**. Dieser Bescheid verweist auf:
- 3 Gutachten
  - 2 Gesetze
  - 4 frühere Verwaltungsakte

Sechs Monate später wird der Bescheid angefochten. Die Frage: *"Auf welcher Datengrundlage basierte die Entscheidung?"*

### Die Lösung mit temporalen Graphen:

```

bescheid_timestamp = datetime(2024, 3, 15, 10, 30).timestamp() * 1000

verwandte_dokumente = db.graph.bfsAtTime(
 start=f"bescheide/{bescheid_id}",
 timestamp_ms=bescheid_timestamp,
 max_depth=2 # Direkte + indirekte Referenzen
)

for doc_id in verwandte_dokumente:
 doc_version = db.get_document_at_time(doc_id, bescheid_timestamp)
 print(f"Dokument {doc_id} (Stand {bescheid_timestamp}):")
 print(f" Titel: {doc_version['title']}")
 print(f" Version: {doc_version['version']}")
 print(f" Checksum: {doc_version['checksum']}")

```

**Resultat:** Sie können **beweisen**, dass der Bescheid auf den zum Entscheidungszeitpunkt gültigen Dokumenten basierte – selbst wenn diese Dokumente seitdem aktualisiert wurden.

## Performance-Überlegungen

### Speicher-Overhead:

Temporale Kanten benötigen 16 zusätzliche Bytes pro Kante ( $2 \times \text{int64}$  für Timestamps):

```

Normale Kante: ~80 Bytes
Temporale Kante: ~96 Bytes
→ Overhead: 20%

```

Bei 10 Millionen Kanten: ~160 MB zusätzlicher Speicher. **Akzeptabel** für die gewonnene Funktionalität.

### Query-Performance:

Die temporale Filterung ist sehr effizient, da der Check inline während der Traversierung passiert:

```

// Pro Kante: 2 Integer-Vergleiche (~2 CPU-Zyklen)
if (edge.valid_from && timestamp < edge.valid_from) return false;
if (edge.valid_to && timestamp > edge.valid_to) return false;

```

**Benchmark:** bfsAtTime() ist nur ~5-10% langsamer als normale BFS, da der Overhead durch CPU-Cache-Hits minimiert wird.

## Best Practices

### 1. Timestamps immer in Millisekunden (Unix Epoch)

```

timestamp_ms = int(datetime.now().timestamp() * 1000)

timestamp_s = int(datetime.now().timestamp())

```

### 2. `valid_to = null` für aktuell gültige Beziehungen

```

edge = {
 "valid_from": now_ms,
 "valid_to": None
}

edge = {
 "valid_from": now_ms,
 "valid_to": 9999999999999 # Unflexibel!
}

```

### 3. Indizes auf temporalen Feldern

```
db.create_index(
 collection="edges",
 fields=["valid_from", "valid_to"],
 index_type="range"
)
```

---

## 6.5 Praxisbeispiel 1: Social Network

Jetzt setzen wir die Theorie in die Praxis um mit einem vollständigen Social Network.

### Das Projekt

Das Social Network-Example ( [examples/06\\_graph\\_social\\_network](#) ) implementiert: -  
Benutzerprofile mit Interessen - Bidirektionale Freundschaften - Graph-Traversierung (Freunde von Freunden) - Kürzeste Pfade zwischen Usern - Community-Erkennung - Freundschafts-Empfehlungen -  
Interaktive Visualisierung mit NetworkX

## Datenmodell

```
from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class User:
 """Ein Benutzer im sozialen Netzwerk."""
 id: str
 name: str
 bio: Optional[str] = None
 interests: List[str] = None
 location: Optional[str] = None
 joined: datetime = None

 def to_dict(self):
 return {
 "id": self.id,
 "name": self.name,
 "bio": self.bio,
 "interests": self.interests or [],
 "location": self.location,
 "joined": self.joined.isoformat() if self.joined else None
 }

@dataclass
class Friendship:
 """Eine Freundschaft zwischen zwei Benutzern."""
 from_user: str
 to_user: str
 since: datetime
 strength: float = 1.0 # 0-1, basierend auf Interaktionen

 def to_dict(self):
 return {
 "from": f"users/{self.from_user}",
 "to": f"users/{self.to_user}",
 "since": self.since.isoformat(),
 "strength": self.strength
 }
```

## Graph Setup

```
class ThemisGraphClient:
 def __init__(self, host="localhost", port=8529):
 self.client = ThemisDB(host=host, port=port)
 self.db = self.client.db("social_network")
 self._setup_graph()

 def _setup_graph(self):
 """Erstellt Collections und Graph-Definition."""
 # Vertex Collection für User
 if not self.db.has_collection("users"):
 self.db.create_vertex_collection("users")

 # Edge Collection für Freundschaften
 if not self.db.has_collection("friendships"):
 self.db.create_edge_collection("friendships")

 # Graph-Definition
 if not self.db.has_graph("social_graph"):
 graph_def = {
 "name": "social_graph",
 "edge_definitions": [{
 "collection": "friendships",
 "from": ["users"],
 "to": ["users"]
 }]
 }
 self.db.create_graph(graph_def)

 # Indexes für Performance
 self.db.add_index("users", ["name"])
 self.db.add_index("users", ["location"])
 self.db.add_index("friendships", ["from", "to"])
```

## Benutzer und Freundschaften

```
def add_user(self, user: User) -> str:
 """Fügt einen neuen Benutzer hinzu."""
 user_doc = user.to_dict()
 result = self.db.insert("users", user_doc)
 return result["_key"]

def add_friendship(self, friendship: Friendship):
 """Erstellt eine bidirektionale Freundschaft."""
 # Richtung 1: A -> B
 self.db.insert("friendships", friendship.to_dict())

 # Richtung 2: B -> A (für ungerichteten Graph)
 reverse = Friendship(
 from_user=friendship.to_user,
 to_user=friendship.from_user,
 since=friendship.since,
 strength=friendship.strength
)
 self.db.insert("friendships", reverse.to_dict())

def get_friends(self, user_id: str) -> List[User]:
 """Holt alle direkten Freunde eines Benutzers."""
 query = """
 FOR friend IN OUTBOUND @user_id friendships
 RETURN friend
 """
 result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
 return [User(**doc) for doc in result]
```

## Friends-of-Friends (FoF)

Ein klassisches Graph-Problem: Finde Freunde meiner Freunde, die noch nicht meine Freunde sind.

```
def get_friend_suggestions(self, user_id: str, limit: int = 10) -> List[dict]:
 """Empfiehl neue Freunde basierend auf gemeinsamen Freunden."""
 query = """
 FOR friend IN OUTBOUND @user_id friendships
 FOR fof IN OUTBOUND friend._id friendships
 FILTER fof._id != @user_id
 FILTER fof._id NOT IN (
 FOR f IN OUTBOUND @user_id friendships
 RETURN f._id
)
 COLLECT fof_user = fof WITH COUNT INTO mutual_count
 SORT mutual_count DESC
 LIMIT @limit
 RETURN {
 user: fof_user,
 mutual_friends: mutual_count
 }
 """
 return self.db.execute_query(query, bind_vars={
 "user_id": f"users/{user_id}",
 "limit": limit
 })
```

**Was passiert hier?** 1. Traversiere zu allen Freunden ( `OUTBOUND @user_id` ) 2. Von jedem Freund, traversiere zu deren Freunden ( `OUTBOUND friend._id` ) 3. Filtere mich selbst heraus ( `FILTER fof._id != @user_id` ) 4. Filtere existierende Freunde heraus (Subquery) 5. Gruppiere nach FoF und zähle gemeinsame Freunde ( `COLLECT ... WITH COUNT` ) 6. Sortiere nach Anzahl gemeinsamer Freunde

## Kürzeste Pfade

Wie ist Alice mit Bob verbunden?

```

def find_connection_path(self, user_id1: str, user_id2: str) -> dict:
 """Findet den kürzesten Pfad zwischen zwei Benutzern."""
 path = self.db.shortest_path(
 graph="social_graph",
 start_vertex=f"users/{user_id1}",
 target_vertex=f"users/{user_id2}",
 direction="any" # Ungerichtet (beide Richtungen)
)

 if path is None:
 return {"connected": False}

 return {
 "connected": True,
 "distance": path["distance"],
 "path": [v["name"] for v in path["vertices"]],
 "degrees_of_separation": len(path["vertices"]) - 1
 }

```

## Community-Erkennung

Welche natürlichen Gruppen gibt es im Netzwerk?



```

def detect_communities(self) -> dict:
 """Findet Communities mittels Connected Components."""
 components = self.db.connected_components(
 graph="social_graph",
 direction="any"
)

 # Statistiken pro Community
 communities = {}
 for component_id, user_ids in components.items():
 users = [self.db.get("users", uid) for uid in user_ids]

 # Gemeinsame Interessen
 all_interests = []
 for user in users:
 all_interests.extend(user.get("interests", []))
 interest_counts = Counter(all_interests)

 communities[component_id] = {
 "size": len(users),
 "members": [u["name"] for u in users],
 "top_interests": interest_counts.most_common(5)
 }

 return communities

```

## Interaktive Visualisierung

Das Example nutzt NetworkX für Visualisierung:

```

import networkx as nx
import matplotlib.pyplot as plt

def visualize_network(self, user_id: Optional[str] = None, max_hops: int = 2):
 """Visualisiert das soziale Netzwerk."""
 G = nx.Graph()

 if user_id:
 # Nur Ego-Netzwerk eines Users
 query = f"""
 FOR v, e IN 1..{max_hops} ANY @user_id friendships
 RETURN {{vertex: v, edge: e}}
 """
 result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
 else:
 # Ganzes Netzwerk
 users = self.db.all("users")
 friendships = self.db.all("friendships")

 for user in users:
 G.add_node(user["_key"], name=user["name"])

 for friendship in friendships:
 from_id = friendship["_from"].split("/")[1]
 to_id = friendship["_to"].split("/")[1]
 G.add_edge(from_id, to_id, weight=friendship.get("strength", 1.0))

 # Layout mit Spring-Algorithmus
 pos = nx.spring_layout(G, k=0.5, iterations=50)

 # Zeichnen
 plt.figure(figsize=(12, 8))
 nx.draw_networkx_nodes(G, pos, node_size=500, node_color='lightblue')
 nx.draw_networkx_edges(G, pos, alpha=0.5)
 nx.draw_networkx_labels(G, pos, labels=nx.get_node_attributes(G, 'name'))

 plt.title("Social Network Graph")
 plt.axis('off')
 plt.tight_layout()
 plt.show()

```

## **Main Application**

---

```

def main():
 client = ThemisGraphClient()

 # Beispiel-Daten
 users = [
 User("alice", "Alice", "Software Engineer", ["Python", "ML", "Hiking"], "Berlin")
 User("bob", "Bob", "Data Scientist", ["Python", "Statistics", "Music"], "Munich")
 User("charlie", "Charlie", "DevOps", ["Docker", "Kubernetes", "Hiking"], "Berlin")
 User("diana", "Diana", "Product Manager", ["Agile", "UX", "Music"], "Hamburg"),
 User("eve", "Eve", "ML Engineer", ["ML", "Python", "Research"], "Berlin"),
]

 for user in users:
 client.add_user(user)

 # Freundschaften
 friendships = [
 ("alice", "bob"),
 ("alice", "charlie"),
 ("bob", "diana"),
 ("charlie", "eve"),
 ("diana", "eve"),
]

 for user1, user2 in friendships:
 client.add_friendship(Friendship(user1, user2, datetime.now(), 0.8))

 # 1. Freunde von Alice
 print("Alice's Friends:")
 friends = client.get_friends("alice")
 for friend in friends:
 print(f" - {friend.name} ({friend.location})")

 # 2. Freundschafts-Empfehlungen für Alice
 print("\nFriend Suggestions for Alice:")
 suggestions = client.get_friend_suggestions("alice", limit=3)
 for suggestion in suggestions:
 print(f" - {suggestion['user']['name']}: {suggestion['mutual_friends']} mutual f

 # 3. Verbindung zwischen Alice und Eve
 print("\nConnection between Alice and Eve:")
 path = client.find_connection_path("alice", "eve")
 if path["connected"]:
 print(f" Path: {' ' -> '.join(path['path'])}")
 print(f" Degrees of Separation: {path['degrees_of_separation']}")

 # 4. Communities
 print("\nCommunities:")
 communities = client.detect_communities()

```

```

 for comm_id, comm_data in communities.items():
 print(f" Community {comm_id}: {comm_data['size']} members")
 print(f" Members: {'', '.join(comm_data['members'])}")
 print(f" Top Interests: {[i[0] for i in comm_data['top_interests']]}")

5. Visualisierung
client.visualize_network()

if __name__ == "__main__":
 main()

```

## Output

```

Alice's Friends:
- Bob (Munich)
- Charlie (Berlin)

Friend Suggestions for Alice:
- Diana: 1 mutual friends
- Eve: 1 mutual friends

Connection between Alice and Eve:
Path: Alice -> Charlie -> Eve
Degrees of Separation: 2

Communities:
Community 1: 5 members
Members: Alice, Bob, Charlie, Diana, Eve
Top Interests: ['Python', 'ML', 'Hiking', 'Music', 'Docker']

[Visualization opens in matplotlib window]

```

## Performance-Optimierung

### 1. Indexes auf häufig abgefragte Felder:

```

self.db.add_index("users", ["name"])
self.db.add_index("users", ["location"])
self.db.add_index("friendships", ["from", "to"])

```

### 2. Batch-Operations für viele Inserts:

```
def add_users_batch(self, users: List[User]):
 """Fügt mehrere User auf einmal hinzu."""
 user_docs = [u.to_dict() for u in users]
 self.db.insert_many("users", user_docs)
```

### 3. Bounded Traversals:

```
FOR v IN 1..3 OUTBOUND @user_id friendships # Max 3 Hops
RETURN v
```

### 4. Index-basierte Filters:

```
FOR user IN users
 FILTER user.location == "Berlin" # Nutzt Index
 FOR friend IN OUTBOUND user._id friendships
 RETURN friend
```

## 6.6 Praxisbeispiel 2: Recommendation Engine

Das zweite Example ( [examples/19\\_recommendation\\_engine](#) ) zeigt einen komplexeren Use Case: **ML-basierte Empfehlungen** mit Graph- und Vektor-Daten.

### Das Konzept

Empfehlungssysteme kombinieren mehrere Ansätze:

1. **Collaborative Filtering**: "User, die X mochten, mochten auch Y"
2. **Content-Based Filtering**: "Ähnliche Items basierend auf Features"
3. **Graph-basiert**: "Freunde deiner Freunde kauften X"
4. **Hybrid**: Kombination aller Ansätze

## Datenmodell

```
@dataclass
class Item:
 """Ein empfehlbares Item (Produkt, Film, etc.)."""
 id: str
 title: str
 category: str
 features: List[str]
 embedding: List[float] # Content-Embedding für Similarity

@dataclass
class Interaction:
 """Eine User-Item Interaktion."""
 user_id: str
 item_id: str
 type: str # "view", "click", "purchase", "rate"
 rating: Optional[float] = None
 timestamp: datetime = None
 context: dict = None # Device, location, etc.

@dataclass
class Recommendation:
 """Eine generierte Empfehlung."""
 user_id: str
 item_id: str
 score: float
 method: str # "collaborative", "content_based", "graph", "hybrid"
 explanation: str
```

## Graph-Setup

```
def _setup_graph(self):
 """Erstellt Multi-Graph für Recommendations."""
 # Vertex Collections
 self.db.create_vertex_collection("users")
 self.db.create_vertex_collection("items")

 # Edge Collections
 self.db.create_edge_collection("interactions") # User -> Item
 self.db.create_edge_collection("similar_items") # Item -> Item
 self.db.create_edge_collection("user_similarity") # User -> User

 # Graph-Definition
 graph_def = {
 "name": "recommendation_graph",
 "edge_definitions": [
 {
 "collection": "interactions",
 "from": ["users"],
 "to": ["items"]
 },
 {
 "collection": "similar_items",
 "from": ["items"],
 "to": ["items"]
 },
 {
 "collection": "user_similarity",
 "from": ["users"],
 "to": ["users"]
 }
]
 }
 self.db.create_graph(graph_def)
```

## Collaborative Filtering

Findet Items, die ähnliche User mochten:



```

def collaborative_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
 """Empfehle Items basierend auf ähnlichen Usern."""
 query = """
 // 1. Finde ähnliche User (basierend auf vergangenen Interaktionen)
 FOR similar_user IN OUTBOUND @user_id user_similarity
 SORT similar_user.similarity DESC
 LIMIT 20

 // 2. Finde Items, die ähnliche User mochten
 FOR item IN OUTBOUND similar_user._id interactions
 FILTER item.interaction_type IN ["purchase", "rate"]
 FILTER item.rating >= 4 // Nur positive

 // 3. Filtere Items, die User schon kennt
 FILTER item._id NOT IN (
 FOR known_item IN OUTBOUND @user_id interactions
 RETURN known_item._id
)

 // 4. Aggregiere Score
 COLLECT item_id = item._id
 AGGREGATE score = AVG(item.rating * similar_user.similarity)

 SORT score DESC
 LIMIT @limit

 // 5. Hole Item-Details
 LET item_doc = DOCUMENT(item_id)
 RETURN {
 item_id: item_id,
 score: score,
 method: "collaborative",
 explanation: CONCAT("Users similar to you rated this ", score)
 }
 """

 results = self.db.execute_query(query, bind_vars={
 "user_id": f"users/{user_id}",
 "limit": limit
 })

 return [Recommendation(**r) for r in results]

```

## Content-Based Filtering

Findet ähnliche Items basierend auf Features:

```

def content_based_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
 """Empfehle Items ähnlich zu den, die User mag."""
 query = """
 // 1. Finde Items, die User mag
 FOR liked_item IN OUTBOUND @user_id interactions
 FILTER liked_item.rating >= 4

 // 2. Finde ähnliche Items
 FOR similar_item IN OUTBOUND liked_item._id similar_items
 SORT similar_item.similarity DESC

 // 3. Filtere bekannte Items
 FILTER similar_item._id NOT IN (
 FOR known IN OUTBOUND @user_id interactions
 RETURN known._id
)

 // 4. Aggregiere
 COLLECT item_id = similar_item._id
 AGGREGATE score = AVG(similar_item.similarity)

 SORT score DESC
 LIMIT @limit

 LET item_doc = DOCUMENT(item_id)
 RETURN {
 item_id: item_id,
 score: score,
 method: "content_based",
 explanation: CONCAT("Similar to items you liked")
 }
 """

 results = self.db.execute_query(query, bind_vars={
 "user_id": f"users/{user_id}",
 "limit": limit
 })

 return [Recommendation(**r) for r in results]

```

## Graph-basierte Empfehlungen

Nutzt Social Graph für Empfehlungen:

```

def graph_based_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
 """Empfehle Items, die Freunde mochten."""
 query = """
 // 1. Traversiere zu Freunden (1-2 Hops)
 FOR friend IN 1..2 OUTBOUND @user_id friendships

 // 2. Finde Items, die Freund kaufte
 FOR item IN OUTBOUND friend._id interactions
 FILTER item.interaction_type == "purchase"

 // 3. Filtere bekannte Items
 FILTER item._id NOT IN (
 FOR known IN OUTBOUND @user_id interactions
 RETURN known._id
)

 // 4. Score basierend auf Freundschafts-Stärke und Rating
 COLLECT item_id = item._id
 AGGREGATE score = AVG(friend.friendship_strength * item.rating)

 SORT score DESC
 LIMIT @limit

 LET item_doc = DOCUMENT(item_id)
 RETURN {
 item_id: item_id,
 score: score,
 method: "graph_based",
 explanation: "Your friends liked this"
 }
 """

 results = self.db.execute_query(query, bind_vars={
 "user_id": f"users/{user_id}",
 "limit": limit
 })

 return [Recommendation(**r) for r in results]

```

## Hybrid Recommendations

Kombiniert alle Methoden mit Gewichtung:

```

def hybrid_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
 """Kombiniert alle Empfehlungsmethoden."""
 # Hole Empfehlungen von allen Methoden
 collaborative = self.collaborative_filtering(user_id, limit=20)
 content_based = self.content_based_filtering(user_id, limit=20)
 graph_based = self.graph_based_recommendations(user_id, limit=20)

 # Gewichtung
 weights = {
 "collaborative": 0.4,
 "content_based": 0.3,
 "graph_based": 0.3
 }

 # Kombiniere Scores
 combined_scores = {}
 for recs, method in [
 (collaborative, "collaborative"),
 (content_based, "content_based"),
 (graph_based, "graph_based")
]:
 for rec in recs:
 if rec.item_id not in combined_scores:
 combined_scores[rec.item_id] = {
 "score": 0,
 "methods": [],
 "explanations": []
 }

 combined_scores[rec.item_id]["score"] += rec.score * weights[method]
 combined_scores[rec.item_id]["methods"].append(method)
 combined_scores[rec.item_id]["explanations"].append(rec.explanation)

 # Sortiere und limitiere
 sorted_items = sorted(
 combined_scores.items(),
 key=lambda x: x[1]["score"],
 reverse=True
)[:limit]

 # Erstelle finale Recommendations
 recommendations = []
 for item_id, data in sorted_items:
 recommendations.append(Recommendation(
 user_id=user_id,
 item_id=item_id,
 score=data["score"],
 method="hybrid",

```

```
 explanation=f"Recommended via: {' '.join(data['methods'])}"
))

 return recommendations
```

## Similarity Berechnung

Berechne User-User und Item-Item Similarity:

```

def compute_user_similarity(self):
 """Berechnet Similarity zwischen allen User-Paaren."""
 users = self.db.all("users")

 for i, user1 in enumerate(users):
 for user2 in users[i+1:]:
 # Gemeinsame Interaktionen
 common_items = self._get_common_interactions(user1["_id"], user2["_id"])

 if len(common_items) >= 3: # Mindestens 3 gemeinsame
 similarity = self._calculate_similarity(
 user1["interaction_vector"],
 user2["interaction_vector"]
)

 # Speichere Kante
 self.db.insert("user_similarity", {
 "from": user1["_id"],
 "to": user2["_id"],
 "similarity": similarity,
 "common_items": len(common_items)
 })

def compute_item_similarity(self):
 """Berechnet Similarity zwischen Items (content-based)."""
 items = self.db.all("items")

 for i, item1 in enumerate(items):
 for item2 in items[i+1:]:
 # Cosine Similarity der Embeddings
 similarity = cosine_similarity(
 item1["embedding"],
 item2["embedding"]
)

 if similarity > 0.7: # Threshold
 self.db.insert("similar_items", {
 "from": item1["_id"],
 "to": item2["_id"],
 "similarity": similarity
 })

```

## Real-Time Updates

Aktualisiere Empfehlungen bei neuen Interaktionen:

```
def record_interaction(self, interaction: Interaction):
 """Zeichnet Interaktion auf und aktualisiert Empfehlungen."""
 # Speichere Interaktion
 interaction_doc = {
 "from": f"users/{interaction.user_id}",
 "to": f"items/{interaction.item_id}",
 "type": interaction.type,
 "rating": interaction.rating,
 "timestamp": interaction.timestamp.isoformat(),
 "context": interaction.context
 }
 self.db.insert("interactions", interaction_doc)

 # Bei Purchase: Update User-Vektor
 if interaction.type == "purchase":
 self._update_user_vector(interaction.user_id, interaction.item_id)

 # Recalculate Similarity (async)
 self._queue_similarity_update(interaction.user_id)
```

## 6.7 Graph Best Practices

### 1. Modeling Guidelines

**DO:** -  Nutze sprechende Edge-Namen ( **FRIEND** , **LIKES** , **WORKS\_AT** ) -  Speichere Properties auf Kanten (Zeitstempel, Gewichte) -  Denormalisiere häufig benötigte Daten -  Nutze Bidirektionale Kanten für ungerichtete Graphs

**DON'T:** -  Zu viele Edge-Types (schwer zu querien) -  Properties als separate Knoten (ineffizient) -  Extrem tiefe Hierarchien (> 10 Levels)

### 2. Performance-Tipps

**Indexes:**

```
db.add_index("friendships", ["from", "to"])
db.add_index("interactions", ["user_id", "timestamp"])
```

**Bounded Traversals:**

```
FOR v IN 1..3 OUTBOUND @start # Nicht 1..ANY
 LIMIT 100 # Früh limitieren
 RETURN v
```

**Prune Early:**

```
FOR v IN 1..5 OUTBOUND @start friendships
 FILTER v.city == "Berlin" # Früher Filter
 FOR v2 IN OUTBOUND v._id friendships
 RETURN v2
```

### 3. Häufige Patterns

**Pattern 1: Mutual Friends**

```
FOR friend IN OUTBOUND @user1 friendships
 FILTER friend._id IN (
 FOR f IN OUTBOUND @user2 friendships
 RETURN f._id
)
RETURN friend
```

**Pattern 2: Influencer (hohe Degree)**

```
FOR user IN users
 LET friend_count = LENGTH(
 FOR f IN OUTBOUND user._id friendships
 RETURN 1
)
 FILTER friend_count > 100
 SORT friend_count DESC
 RETURN {user: user, friends: friend_count}
```

**Pattern 3: Isolated Nodes**

```
FOR user IN users
 LET connections = (
 FOR v IN ANY user._id friendships
 RETURN 1
)
 FILTER LENGTH(connections) == 0
 RETURN user
```



## 6.8 Vergleich: ThemisDB vs. Neo4j vs. ArangoDB

Feature	ThemisDB	Neo4j	ArangoDB
Graph Model	Property Graph	Property Graph	Property Graph
Query Language	AQL	Cypher	AQL
Multi-Model	✓ 4 Models	✗ Graph only	✓ 3 Models
ACID	✓ Full	✓ Full	✓ Full
Sharding	✓ Automatic	✗ Enterprise only	✓ Yes
Graph Algorithms	✓ Built-in	✓ GDS Library	✓ Pregel
License	Apache 2.0	GPLv3 + Commercial	Apache 2.0
Embedding Support	✓ Native	✗ No	✗ No

**Wann ThemisDB?** - Multi-Model Daten (Graph + Vektor + Relational) - Native Vektor-Suche benötigt - Open-Source und selbst-hosted - Kombinierte Queries über mehrere Modelle

**Wann Neo4j?** - Reines Graph-Problem - Cypher-Erfahrung im Team - Enterprise Support benötigt - Graph Data Science Library

**Wann ArangoDB?** - Ähnlich wie ThemisDB (Multi-Model) - Mature Community - Foxx Microservices

## 6.9 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Property Graph Modell** - Knoten, Kanten, Properties
- ✓ **Graph-Traversierung** - DFS, BFS, Pattern Matching
- ✓ **Graph-Algorithmen** - Shortest Path, PageRank, Community Detection
- ✓ **Praxis: Social Network** - Vollständiges soziales Netzwerk mit Visualisierung
- ✓ **Praxis: Recommendations** - ML-basierte Empfehlungen mit Hybrid-Ansatz
- ✓ **Best Practices** - Modeling, Performance, Patterns

Graph-Datenbanken sind die natürliche Wahl für vernetzte Daten. ThemisDB's Property Graph-Implementierung bietet performante Traversierung, native Algorithmen und die einzigartige Möglichkeit, Graphs mit anderen Modellen zu kombinieren.

Im nächsten Kapitel schauen wir uns **Dokument-Speicherung** an - flexibles, schema-less Design für semi-strukturierte Daten.

---

### Übungen:

1. Erweitern Sie das Social Network um Posts und Likes (User -> Post -> Likes)
2. Implementieren Sie einen Follower/Following-Mechanismus (Twitter-Style)
3. Fügen Sie PageRank-Berechnung hinzu, um Influencer zu finden
4. Erstellen Sie einen Interest-Based-Graph (User -> Interest <- User)
5. Bauen Sie ein Skill-Recommendation-System für LinkedIn-Style-Netzwerk

### Weiterführende Ressourcen:

-  [examples/06\\_graph\\_social\\_network/](#) - Vollständiger Code
-  [examples/19\\_recommendation\\_engine/](#) - Recommendation System
-  [docs/de/features/features\\_property\\_graph.md](#) - Graph-Features
-  NetworkX Documentation - Graph-Visualisierung
-  "Graph Algorithms" (Mark Needham, Amy E. Hodler) - Tiefere Algorithmen

## Kapitel 9: Kapitel 7: Dokument-Speicherung

*In diesem Kapitel: Schema-less Design, flexible Datenstrukturen, Nested Objects, JSON-Operationen, Content Management, Versionierung und Migration von MongoDB.*

### 7.1 Das Document Model

#### Warum Dokumente?

Das Document Model ist ideal für Anwendungen mit:

- **Flexible Schemas:** Nicht alle Datensätze haben die gleichen Felder
- **Nested Data:** Hierarchische Strukturen (Kommentare, Tags, Metadaten)
- **Rapid Development:** Schema-Evolution ohne Migration
- **Content Management:** Blogs, Wikis, CMS-Systeme

**Beispiele aus der Praxis:** - Blog-Systeme mit variablen Post-Typen (Text, Video, Galerie) - Wikis mit verschachtelten Inhalten und Revisionen - Content-Management mit Metadata - Konfigurationsdaten mit unterschiedlichen Formaten

#### Dokumente in ThemisDB

ThemisDB speichert Dokumente als JSON mit vollem ACID-Support:

```
from themisdb import ThemisDB

db = ThemisDB()

article = {
 "title": "Multi-Model Databases",
 "author": "Alice",
 "content": "Content here...",
 "tags": ["database", "multi-model"],
 "metadata": {
 "published": "2025-01-15",
 "views": 1024
 },
 "comments": [
 {"user": "Bob", "text": "Great article!"},
 {"user": "Carol", "text": "Very helpful"}
]
}

db.documents.insert("articles", article)
```

**Vorteile:** -  Keine Schema-Definition notwendig -  Beliebige Verschachtelung -  Arrays von Sub-Dokumenten -  Optionale Felder -  JSON-Operationen (Path-Queries, Updates)

## Vergleich: Relational vs. Document

### Relational (starr):

```
-- Zwei separate Collections mit Referenzen
// articles Collection
{
 _key: "1",
 title: "Artikel",
 content: "Text..."
}

// comments Collection (Referenz via article_id)
{
 _key: "c1",
 article_id: "1",
 user: "alice",
 text: "Kommentar"
}
```

### Document (flexibel):

```
// Ein Dokument, alles inklusive
{
 "id": 1,
 "title": "...",
 "content": "...",
 "comments": [{"user": "...", "text": "..."}]
}
```

```

graph TB
 subgraph "Relational Model - Normalized"
 Art[articles table
id, title, content]
 Com[comments table
id, article_id, user, text]
 Tags[tags table
id, article_id, tag]

 Art -->|FK: article_id| Com
 Art -->|FK: article_id| Tags
 end

 subgraph "Document Model - Denormalized"
 Doc["Single Document
title, content, comments, tags"]
 end

 Pro1["✓ Self-contained
✓ Single read
✓ No JOINS
✓ Flexible schema"]
 Pro2["✓ Data integrity
✓ No duplication
✓ Normalized
✓ ACID across tables"]

 Doc --> Pro1
 Art --> Pro2

 style Art fill:#667eea
 style Com fill:#4facfe
 style Tags fill:#4facfe
 style Doc fill:#43e97b
 style Pro1 fill:#95e1d3
 style Pro2 fill:#95e1d3

```

**Abb. 07.1:** Dokument-Store-Architektur

## Schema Evolution

Neue Felder ohne Migration hinzufügen:

```
{
 "title": "Hello World",
 "content": "..."
}

{
 "title": "Hello World",
 "content": "...",
 "author": "Alice"
}

{
 "title": "Hello World",
 "content": "...",
 "author": "Alice",
 "tags": ["intro", "tutorial"],
 "metadata": {
 "published": "2025-01-15",
 "featured": True
 }
}
```

#### Anwendungscode prüft Feld-Existenz:

```
article = db.documents.get("articles", article_id)

author = article.get("author", "Unknown")
tags = article.get("tags", [])
```

## 7.2 JSON-Operationen

### Path-Queries

Tief in verschachtelten Dokumenten suchen:

```

articles = db.query("""
 FOR article IN articles
 FILTER article.metadata.published > '2025-01-01'
 AND article.metadata.featured == true
 RETURN article
""")

articles_with_tag = db.query("""
 FOR article IN articles
 FILTER 'database' IN article.tags
 RETURN article
""")

articles_by_bob = db.query("""
 FOR article IN articles
 FILTER 'Bob' IN article.comments[*].user
 RETURN article
""")

```

## Partial Updates

Nur bestimmte Felder ändern:

```

db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={"metadata.views": db.increment(1)}
)

new_comment = {"user": "Dave", "text": "Thanks!"}
db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={"comments": db.array_append(new_comment)}
)

db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={"metadata.featured": True}
)

```

## JSON-Funktionen

```
result = db.query("""
 SELECT
 title,
 metadata.published AS date,
 ARRAY_LENGTH(comments) AS comment_count
 FROM articles
 WHERE metadata.featured = true
""")

stats = db.query("""
 SELECT
 author,
 COUNT(*) AS article_count,
 SUM(metadata.views) AS total_views
 FROM articles
 GROUP BY author
""")
```

### 7.3 Example: Blog/Wiki System

*Example:* [examples/11\\_blog\\_wiki](#)

Implementieren wir ein vollständiges Content-Management-System mit Artikeln, Revisions-Historie und Tagging.

#### System-Überblick

**Features:** - Artikel mit Rich Content (Markdown) - Revisions-Historie (alle Änderungen) - Tagging und Kategorisierung - Volltext-Suche über Titel und Content - View-Counter und Statistiken - Kommentar-System

**Datenstruktur:**



```

{
 "id": "uuid",
 "title": "Getting Started with ThemisDB",
 "slug": "getting-started-themisdb",
 "content": "# Introduction\n\n...",
 "author": "alice@example.com",
 "status": "published", # draft, published, archived
 "tags": ["tutorial", "database", "introduction"],
 "category": "tutorials",
 "metadata": {
 "published_at": "2025-01-15T10:00:00Z",
 "updated_at": "2025-01-16T14:30:00Z",
 "views": 1250,
 "featured": True
 },
 "comments": [
 {
 "id": "comment-uuid",
 "author": "bob@example.com",
 "text": "Great tutorial!",
 "created_at": "2025-01-15T11:00:00Z"
 }
],
 "revisions": [
 {
 "version": 1,
 "content": "Initial version...",
 "changed_by": "alice@example.com",
 "changed_at": "2025-01-15T10:00:00Z"
 },
 {
 "version": 2,
 "content": "Updated version...",
 "changed_by": "alice@example.com",
 "changed_at": "2025-01-16T14:30:00Z"
 }
]
}

```

## Implementation

`blog_wiki/models.py` :

```

from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Comment:
 id: str
 author: str
 text: str
 created_at: datetime

 @staticmethod
 def create(author: str, text: str) -> dict:
 return {
 "id": str(uuid.uuid4()),
 "author": author,
 "text": text,
 "created_at": datetime.now().isoformat()
 }

@dataclass
class Revision:
 version: int
 content: str
 changed_by: str
 changed_at: datetime

@dataclass
class Article:
 id: str
 title: str
 slug: str
 content: str
 author: str
 status: str # draft, published, archived
 tags: List[str]
 category: str
 metadata: dict
 comments: List[dict] = field(default_factory=list)
 revisions: List[dict] = field(default_factory=list)

 @staticmethod
 def create(title: str, content: str, author: str,
 tags: List[str], category: str) -> dict:
 now = datetime.now().isoformat()
 article_id = str(uuid.uuid4())
 slug = title.lower().replace(" ", "-")

```

```
article = {
 "id": article_id,
 "title": title,
 "slug": slug,
 "content": content,
 "author": author,
 "status": "draft",
 "tags": tags,
 "category": category,
 "metadata": {
 "created_at": now,
 "updated_at": now,
 "published_at": None,
 "views": 0,
 "featured": False
 },
 "comments": [],
 "revisions": [
 {
 "version": 1,
 "content": content,
 "changed_by": author,
 "changed_at": now
 }
]
}
return article
```

**blog\_wiki/blog.py :**

```

from themisdb import ThemisDB
from datetime import datetime
from typing import List, Optional

class BlogWiki:
 def __init__(self, db_path: str = "blog.db"):
 self.db = ThemisDB(db_path)
 self._setup_indexes()

 def _setup_indexes(self):
 """Create indexes for performance"""
 # Fulltext search on title and content
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_articles_fulltext
 ON articles USING FULLTEXT(title, content)
 """)

 # Fast lookup by slug
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_articles_slug
 ON articles(slug)
 """)

 # Filter by status and category
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_articles_status_category
 ON articles(status, category)
 """)

 # Tag search
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_articles_tags
 ON articles(tags)
 """)

 def create_article(self, title: str, content: str, author: str,
 tags: List[str], category: str) -> str:
 """Create new article"""
 from models import Article

 article = Article.create(title, content, author, tags, category)
 self.db.documents.insert("articles", article)
 return article["id"]

 def publish_article(self, article_id: str):
 """Publish draft article"""
 with self.db.transaction():
 self.db.documents.update(
 collection="articles",

```

```

 document_id=article_id,
 updates={
 "status": "published",
 "metadata.published_at": datetime.now().isoformat()
 }
)

def update_article(self, article_id: str, content: str,
 updated_by: str):
 """Update article and save revision"""
 article = self.db.documents.get("articles", article_id)

 # Create new revision
 new_version = len(article["revisions"]) + 1
 revision = {
 "version": new_version,
 "content": content,
 "changed_by": updated_by,
 "changed_at": datetime.now().isoformat()
 }

 with self.db.transaction():
 # Update content and add revision
 self.db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={
 "content": content,
 "metadata.updated_at": datetime.now().isoformat(),
 "revisions": self.db.array_append(revision)
 }
)

def add_comment(self, article_id: str, author: str, text: str):
 """Add comment to article"""
 from models import Comment

 comment = Comment.create(author, text)

 self.db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={
 "comments": self.db.array_append(comment)
 }
)

def increment_views(self, article_id: str):
 """Increment view counter"""
 self.db.documents.update(

```

```

 collection="articles",
 document_id=article_id,
 updates={
 "metadata.views": self.db.increment(1)
 }
)

def search_articles(self, query: str) -> List[dict]:
 """Fulltext search"""
 results = self.db.query("""
 FOR article IN articles
 FILTER FULLTEXT(article.title, @query) OR FULLTEXT(article.content, @query)
 AND article.status == 'published'
 SORT article.metadata.published_at DESC
 RETURN article
 """, {"query": query})
 return results

def get_by_tag(self, tag: str) -> List[dict]:
 """Get articles by tag"""
 results = self.db.query("""
 FOR article IN articles
 FILTER @tag IN article.tags
 AND article.status == 'published'
 SORT article.metadata.published_at DESC
 RETURN article
 """, {"tag": tag})
 return results

def get_by_category(self, category: str) -> List[dict]:
 """Get articles by category"""
 results = self.db.query("""
 FOR article IN articles
 FILTER article.category == @category
 AND article.status == 'published'
 SORT article.metadata.published_at DESC
 RETURN article
 """, {"category": category})
 return results

def get_popular(self, limit: int = 10) -> List[dict]:
 """Get most viewed articles"""
 results = self.db.query("""
 FOR article IN articles
 FILTER article.status == 'published'
 SORT article.metadata.views DESC
 LIMIT @limit
 RETURN article
 """, {"limit": limit})
 return results

```

```

def get_featured(self) -> List[dict]:
 """Get featured articles"""
 results = self.db.query("""
 FOR article IN articles
 FILTER article.status == 'published'
 AND article.metadata.featured == true
 SORT article.metadata.published_at DESC
 RETURN article
 """)
 return results

def get_revisions(self, article_id: str) -> List[dict]:
 """Get revision history"""
 article = self.db.documents.get("articles", article_id)
 return article.get("revisions", [])

def revert_to_revision(self, article_id: str, version: int,
 reverted_by: str):
 """Revert to previous version"""
 article = self.db.documents.get("articles", article_id)
 revisions = article["revisions"]

 # Find target revision
 target = next((r for r in revisions if r["version"] == version), None)
 if not target:
 raise ValueError(f"Revision {version} not found")

 # Create revert revision
 new_version = len(revisions) + 1
 revert_revision = {
 "version": new_version,
 "content": target["content"],
 "changed_by": reverted_by,
 "changed_at": datetime.now().isoformat(),
 "reverted_from": version
 }

 with self.db.transaction():
 self.db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={
 "content": target["content"],
 "metadata.updated_at": datetime.now().isoformat(),
 "revisions": self.db.array_append(revert_revision)
 }
)

def get_statistics(self) -> dict:

```

```
"""Get blog statistics"""
stats = self.db.query("""
 SELECT
 COUNT(*) AS total_articles,
 SUM(CASE WHEN status='published' THEN 1 ELSE 0 END)
 AS published,
 SUM(CASE WHEN status='draft' THEN 1 ELSE 0 END)
 AS drafts,
 SUM(metadata.views) AS total_views,
 AVG(metadata.views) AS avg_views
 FROM articles
""")[0]

return stats
```

## Praktische Anwendung

### Blog-Post erstellen und veröffentlichen:



```
from blog import BlogWiki

blog = BlogWiki()

article_id = blog.create_article(
 title="Getting Started with ThemisDB",
 content="""
 # Introduction

 ThemisDB is a multi-model database...

 ## Features
 - ACID transactions
 - Multiple data models
 - High performance
 """,
 author="alice@example.com",
 tags=["tutorial", "database", "getting-started"],
 category="tutorials"
)

blog.publish_article(article_id)

blog.add_comment(
 article_id,
 author="bob@example.com",
 text="Great tutorial! Very helpful."
)

blog.increment_views(article_id)
```

**Artikel aktualisieren mit Revisions-Historie:**

```
blog.update_article(
 article_id,
 content="""
 # Introduction (Updated!)

 ThemisDB is a powerful multi-model database...

 ## New Features
 - ACID transactions
 - Multiple data models
 - High performance
 - Geo-spatial support (NEW!)
 """,
 updated_by="alice@example.com"
)

revisions = blog.get_revisions(article_id)
for rev in revisions:
 print(f"Version {rev['version']} by {rev['changed_by']}")
 print(f" Changed at: {rev['changed_at']}")

blog.revert_to_revision(article_id, version=1,
 reverted_by="alice@example.com")
```

### Suchen und Filtern:

```
results = blog.search_articles("multi-model database")
print(f"Found {len(results)} articles")

tutorials = blog.get_by_tag("tutorial")

db_articles = blog.get_by_category("database")

popular = blog.get_popular(limit=5)

featured = blog.get_featured()
```

### Statistiken:

```
stats = blog.get_statistics()
print(f"Total articles: {stats['total_articles']}")
print(f"Published: {stats['published']}")
print(f"Drafts: {stats['drafts']}")
print(f"Total views: {stats['total_views']}")
print(f"Avg views: {stats['avg_views']:.1f}")
```

## Key Features

**1. Schema Flexibility:** - Artikel können unterschiedliche Felder haben - Neue Features ohne Migration (z.B. `metadata.featured`) - Optionale Felder (z.B. `published_at` nur bei Status="published")

**2. Revision History:** - Jede Änderung wird gespeichert - Komplette Historie verfügbar - Revert zu jeder Version möglich

**3. Nested Data:** - Comments als Array von Sub-Dokumenten - Metadata als verschachteltes Objekt - Keine JOIN-Queries notwendig

**4. Performance:** - Fulltext-Index für Suche - Index auf Slug für schnelle Lookups - Composite-Index für Status+Category

## 7.4 Example: Recipe Manager

*Example:* `examples/13_recipe_manager`

Ein Rezept-Verwaltungssystem zeigt die Stärken des Document Models bei komplexen, verschachtelten Datenstrukturen.

## System-Überblick

**Features:** - Rezepte mit Zutaten und Schritten - Nährwertangaben und Tags - Bewertungen und Kommentare - Einkaufslisten-Generator - Meal Planning

**Datenstruktur:**

```

{
 "id": "uuid",
 "title": "Spaghetti Carbonara",
 "description": "Classic Italian pasta dish",
 "cuisine": "Italian",
 "difficulty": "medium", # easy, medium, hard
 "prep_time": 15, # minutes
 "cook_time": 20,
 "servings": 4,
 "ingredients": [
 {
 "name": "Spaghetti",
 "amount": 400,
 "unit": "g",
 "category": "pasta"
 },
 {
 "name": "Eggs",
 "amount": 4,
 "unit": "pieces",
 "category": "dairy"
 },
 {
 "name": "Parmesan",
 "amount": 100,
 "unit": "g",
 "category": "dairy"
 }
],
 "steps": [
 {
 "number": 1,
 "instruction": "Boil water and cook spaghetti al dente",
 "duration": 10
 },
 {
 "number": 2,
 "instruction": "Mix eggs with parmesan",
 "duration": 5
 },
 {
 "number": 3,
 "instruction": "Combine pasta with egg mixture",
 "duration": 5
 }
],
 "nutrition": {
 "calories": 520,
 "protein": 22,

```

```
 "carbs": 65,
 "fat": 18
 },
 "tags": ["pasta", "italian", "quick", "dinner"],
 "ratings": [
 {
 "user": "alice@example.com",
 "score": 5,
 "comment": "Delicious!",
 "date": "2025-01-15"
 }
],
 "created_by": "chef@example.com",
 "created_at": "2025-01-10",
 "updated_at": "2025-01-15"
}
```

## Implementation

`recipe_manager/recipe.py` :

```

from themisdb import ThemisDB
from typing import List, Optional
from datetime import datetime

class RecipeManager:
 def __init__(self, db_path: str = "recipes.db"):
 self.db = ThemisDB(db_path)
 self._setup_indexes()

 def _setup_indexes(self):
 # Fulltext search
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_recipes_fulltext
 ON recipes USING FULLTEXT(title, description)
 """)

 # Filter by tags, cuisine, difficulty
 self.db.execute("""
 CREATE INDEX IF NOT EXISTS idx_recipes_filters
 ON recipes(cuisine, difficulty, tags)
 """)

 def add_recipe(self, recipe_data: dict) -> str:
 """Add new recipe"""
 recipe_data["created_at"] = datetime.now().isoformat()
 recipe_data["updated_at"] = datetime.now().isoformat()

 recipe_id = self.db.documents.insert("recipes", recipe_data)
 return recipe_id

 def search_recipes(self, query: str) -> List[dict]:
 """Search by title or description"""
 return self.db.query("""
 FOR recipe IN recipes
 FILTER FULLTEXT(recipe.title, @query) OR FULLTEXT(recipe.description, @query)
 RETURN recipe
 """, {"query": query})

 def filter_recipes(self, cuisine: Optional[str] = None,
 difficulty: Optional[str] = None,
 max_time: Optional[int] = None,
 tags: Optional[List[str]] = None) -> List[dict]:
 """Filter recipes by criteria"""
 # Build AQL query dynamically
 filters = []
 params = {}

 if cuisine:
 filters.append("recipe.cuisine == @cuisine")

```

```

 params["cuisine"] = cuisine

 if difficulty:
 filters.append("recipe.difficulty == @difficulty")
 params["difficulty"] = difficulty

 if max_time:
 filters.append("(recipe.prep_time + recipe.cook_time) <= @max_time")
 params["max_time"] = max_time

 if tags:
 for i, tag in enumerate(tags):
 filters.append(f"@tag{i} IN recipe.tags")
 params[f"tag{i}"] = tag

 filter_clause = " AND ".join(filters) if filters else "true"

 return self.db.query(f"""
 FOR recipe IN recipes
 FILTER {filter_clause}
 RETURN recipe
 """, params)

def add_rating(self, recipe_id: str, user: str,
 score: int, comment: str):
 """Add rating to recipe"""
 rating = {
 "user": user,
 "score": score,
 "comment": comment,
 "date": datetime.now().isoformat()
 }

 self.db.documents.update(
 collection="recipes",
 document_id=recipe_id,
 updates={
 "ratings": self.db.array_append(rating)
 }
)

def get_average_rating(self, recipe_id: str) -> float:
 """Calculate average rating"""
 recipe = self.db.documents.get("recipes", recipe_id)
 ratings = recipe.get("ratings", [])

 if not ratings:
 return 0.0

 return sum(r["score"] for r in ratings) / len(ratings)

```

```

def generate_shopping_list(self, recipe_ids: List[str],
 servings_multiplier: dict = None) -> dict:
 """Generate shopping list from multiple recipes"""
 shopping_list = {}

 for recipe_id in recipe_ids:
 recipe = self.db.documents.get("recipes", recipe_id)
 multiplier = servings_multiplier.get(recipe_id, 1.0)

 for ingredient in recipe["ingredients"]:
 name = ingredient["name"]
 amount = ingredient["amount"] * multiplier
 unit = ingredient["unit"]
 category = ingredient.get("category", "other")

 if name not in shopping_list:
 shopping_list[name] = {
 "amount": 0,
 "unit": unit,
 "category": category
 }

 shopping_list[name]["amount"] += amount

 # Group by category
 categorized = {}
 for name, details in shopping_list.items():
 category = details["category"]
 if category not in categorized:
 categorized[category] = []

 categorized[category].append({
 "name": name,
 "amount": details["amount"],
 "unit": details["unit"]
 })

 return categorized

def get_topRated(self, limit: int = 10) -> List[dict]:
 """Get top rated recipes"""
 recipes = self.db.query("""
 FOR recipe IN recipes
 RETURN recipe
 """)

 # Calculate average ratings
 rated = []
 for recipe in recipes:

```



```

 ratings = recipe.get("ratings", [])
 if ratings:
 avg = sum(r["score"] for r in ratings) / len(ratings)
 recipe["avg_rating"] = avg
 rated.append(recipe)

 # Sort by rating
 rated.sort(key=lambda x: x["avg_rating"], reverse=True)
 return rated[:limit]

def get_quick_recipes(self, max_minutes: int = 30) -> List[dict]:
 """Get recipes that can be made quickly"""
 return self.db.query("""
 FOR recipe IN recipes
 FILTER (recipe.prep_time + recipe.cook_time) <= @max_minutes
 SORT (recipe.prep_time + recipe.cook_time) ASC
 RETURN recipe
 """, {"max_minutes": max_minutes})

```

## Praktische Anwendung

### Rezept hinzufügen:

```

from recipe import RecipeManager

manager = RecipeManager()

recipe = {
 "title": "Spaghetti Carbonara",
 "description": "Classic Italian pasta dish",
 "cuisine": "Italian",
 "difficulty": "medium",
 "prep_time": 15,
 "cook_time": 20,
 "servings": 4,
 "ingredients": [
 {"name": "Spaghetti", "amount": 400, "unit": "g", "category": "pasta"},
 {"name": "Eggs", "amount": 4, "unit": "pieces", "category": "dairy"},
 {"name": "Parmesan", "amount": 100, "unit": "g", "category": "dairy"},
 {"name": "Bacon", "amount": 200, "unit": "g", "category": "meat"}
],
 "steps": [
 {"number": 1, "instruction": "Boil water and cook spaghetti", "duration": 10},
 {"number": 2, "instruction": "Mix eggs with parmesan", "duration": 5},
 {"number": 3, "instruction": "Combine everything", "duration": 5}
],
 "nutrition": {
 "calories": 520,
 "protein": 22,
 "carbs": 65,
 "fat": 18
 },
 "tags": ["pasta", "italian", "quick", "dinner"],
 "created_by": "chef@example.com"
}

recipe_id = manager.add_recipe(recipe)

```

### Suchen und Filtern:

```

results = manager.search_recipes("pasta")

italian_easy = manager.filter_recipes(
 cuisine="Italian",
 difficulty="easy"
)

quick = manager.get_quick_recipes(max_minutes=30)

vegetarian = manager.filter_recipes(tags=["vegetarian", "healthy"])

```

**Bewertungen:**

```

manager.add_rating(
 recipe_id=recipe_id,
 user="alice@example.com",
 score=5,
 comment="Best carbonara I've ever made!"
)

avg = manager.get_average_rating(recipe_id)
print(f"Average rating: {avg:.1f}/5")

top = manager.get_top_rated(limit=5)

```

**Einkaufsliste generieren:**

```

recipe_ids = [recipe1_id, recipe2_id, recipe3_id]

multipliers = {
 recipe1_id: 2.0,
 recipe2_id: 0.5,
 recipe3_id: 1.0
}

shopping_list = manager.generate_shopping_list(recipe_ids, multipliers)

for category, items in shopping_list.items():
 print(f"\n{category.upper()}:")
 for item in items:
 print(f" - {item['name']}: {item['amount']}{item['unit']}")

```

## Document Model Vorteile

**1. Natürliche Verschachtelung:** - Ingredients als Array - Steps mit Nummern und Dauer - Nutrition als Sub-Dokument - Ratings mit User-Comments

**2. Flexibilität:** - Optionale Felder (z.B. `nutrition` kann fehlen) - Unterschiedliche Ingredient-Properties - Variable Anzahl von Steps

**3. Einfache Queries:** - Alles in einem Dokument - Keine JOINS notwendig - Aggregation über Arrays

## 7.5 Migration von MongoDB

Viele Projekte migrieren von MongoDB zu ThemisDB für ACID-Garantien.

### Schema Mapping

#### MongoDB:

```
db.articles.insert({
 _id: ObjectId("..."),
 title: "Hello World",
 content: "...",
 tags: ["intro", "tutorial"]
})
```

#### ThemisDB:

```
db.documents.insert("articles", {
 "id": "uuid", # UUID statt ObjectId
 "title": "Hello World",
 "content": "...",
 "tags": ["intro", "tutorial"]
})
```

## Migration Script

```
from pymongo import MongoClient
from themisdb import ThemisDB

mongo = MongoClient("mongodb://localhost:27017")
themis = ThemisDB("migrated.db")

mongo_db = mongo["myapp"]
collection = mongo_db["articles"]

for doc in collection.find():
 # Convert ObjectId to string
 doc["id"] = str(doc.pop("_id"))

 # Insert into ThemisDB
 themis.documents.insert("articles", doc)

print("Migration complete!")
```

## API Differences

MongoDB	ThemisDB	Note
<code>insert_one()</code>	<code>documents.insert()</code>	Single insert
<code>find()</code>	<code>query()</code>	Use AQL
<code>update_one()</code>	<code>documents.update()</code>	Partial update
<code>aggregate()</code>	<code>query()</code>	AQL GROUP BY
<code>\$inc</code>	<code>increment()</code>	Atomic increment
<code>\$push</code>	<code>array_append()</code>	Array operation

## Vorteile nach Migration

### 1. ACID-Transaktionen:

```
with themis.transaction():
 themis.documents.update("articles", id1, {...})
 themis.documents.update("comments", id2, {...})
```

## 2. AQL-Queries:

```

results = collection.aggregate([
 {"$match": {"status": "published"}},
 {"$group": {"_id": "$author", "count": {"$sum": 1}}},
 {"$sort": {"count": -1}}
])

results = themis.query("""
 FOR article IN articles
 FILTER article.status == 'published'
 COLLECT author = article.author
 AGGREGATE count = COUNT()
 SORT count DESC
 RETURN {author, count}
""")

```

## 3. Multi-Model:

```

results = themis.query("""
 FOR article IN articles
 LET comment_count = (
 FOR comment IN comments
 FILTER comment.article_id == article.id
 RETURN 1
)
 LET avg_rating = (
 FOR rating IN ratings
 FILTER rating.article_id == article.id
 COLLECT AGGREGATE avg_score = AVG(rating.score)
 RETURN avg_score
)
 RETURN {
 title: article.title,
 comment_count: LENGTH(comment_count),
 rating: avg_rating[0]
 }
""")

```

## 7.6 Best Practices

### 1. Schema Design

 **DO: Embed Related Data**

```
{
 "title": "...",
 "content": "...",
 "comments": [
 {"user": "alice", "text": "..."},
 {"user": "bob", "text": "..."}
]
}
```

**✗ DON'T: Separate wenn oft zusammen gelesen**

```
{"id": 1, "title": "..."}

{"article_id": 1, "user": "alice", "text": "..."}

```

## 2. Array-Größe begrenzen

**Problem:** Arrays können unbegrenzt wachsen

```
{
 "title": "Popular Article",
 "comments": [...] # Kann zu groß werden!
}
```

**Lösung:** Separiere bei großen Arrays

```
{
 "title": "Popular Article",
 "recent_comments": [...][:10], # Nur letzte 10
 "comment_count": 1523
}
```

## 3. Versionierung

**Schema-Version im Dokument:**

```

{
 "_schema_version": 2,
 "title": "...",
 "content": "...",
 # v2 fields
 "metadata": {...}
}

def load_article(doc):
 version = doc.get("_schema_version", 1)

 if version == 1:
 # Upgrade v1 → v2
 doc["metadata"] = {"created_at": doc.get("created_at")}
 doc["_schema_version"] = 2

 return doc

```

## 4. Index-Strategie

**Indexiere häufige Queries:**

```

CREATE INDEX idx_fulltext ON articles USING FULLTEXT(title, content)

CREATE INDEX idx_filters ON articles(status, category, author)

CREATE INDEX idx_tags ON articles(tags)

```

## 5. Partial Updates

**Effizient: Nur ändern was nötig**

```

db.documents.update(
 collection="articles",
 document_id=article_id,
 updates={"metadata.views": db.increment(1)}
)

article = db.documents.get("articles", article_id)
article["metadata"]["views"] += 1
db.documents.update("articles", article_id, article) # Ineffizient!

```



## 7.7 Performance-Tipps

### 1. Embed vs. Reference

**Embed (einbetten):** -  Daten werden immer zusammen gelesen -  Eingebettete Daten ändern sich selten -  Größe ist begrenzt

**Reference (referenzieren):** -  Daten werden unabhängig gelesen -  Daten werden häufig aktualisiert -  Eingebettetes Array wird zu groß

### 2. Index-Nutzung

```
-- Query mit Index
FOR article IN articles
 FILTER article.status == 'published' -- Index genutzt
 AND article.category == 'tech'
 SORT article.published_at DESC
 RETURN article
```

**Ohne Index (langsam!):**

```
FOR article IN articles
 FILTER article.metadata.custom_field == 'value' -- KEIN Index!
 RETURN article
```

### 3. Projection

Nur benötigte Felder laden:

```
results = db.query("""
 FOR article IN articles
 FILTER article.status == 'published'
 RETURN {title: article.title, author: article.author}
""")

results = db.query("""
 FOR article IN articles
 FILTER article.status == 'published'
 RETURN article
""")
```

## 4. Batch-Operations

```
articles = [...] # Liste von Dokumenten
for article in articles:
 db.documents.insert("articles", article)

with db.transaction():
 for article in articles:
 db.documents.insert("articles", article)
```

## 7.8 Übungen

### Übung 1: Portfolio Website

Erstelle ein Dokumenten-System für eine Portfolio-Website: - Projects mit Screenshots (URLs) - Skills mit Proficiency-Levels - Blog-Posts mit Code-Examples - Contact-Formular History

### Übung 2: Product Catalog

Implementiere einen Produkt-Katalog: - Produkte mit variablen Attributen (Electronics: Screen-Size, Clothing: Sizes) - Kategorien mit Hierarchie - Reviews mit Images - Price-History

### Übung 3: Event Management

Event-System mit: - Events mit Teilnehmern - Schedule mit Sessions - Venue-Details - Feedback-Sammlung

## 7.9 Zusammenfassung

**Document Model in ThemisDB:** -  Schema-less für flexible Datenstrukturen -  Nested Objects und Arrays -  JSON-Operationen (Path-Queries, Updates) -  ACID-Transaktionen (vs. MongoDB) -  Fulltext-Suche -  Migration von MongoDB einfach

**Wann nutzen:** - Content-Management (Blog, Wiki, CMS) - Produkt-Kataloge mit variablen Attributen - User-Profiles mit optionalen Feldern - Config-Dateien mit Verschachtelung - Rapid Prototyping ohne Schema-Definition

**Nächste Schritte:** - Kapitel 8: Vektor-Suche für Semantic Search - Kapitel 9: Time-Series für IoT und Monitoring - Kapitel 10: Geo-Daten für Location-Based Apps

**Hands-on Examples:** - [examples/11\\_blog\\_wiki](#) - Vollständiges CMS - [examples/13\\_recipe\\_manager](#) - Recipe-System - Beide mit Schema-Evolution und Nested Data



# Kapitel 10: Chapter 8: Storage Layer Deep-Dive

**Kategorie:**  Storage & Performance

**Lesezeit:** ~45 Minuten

**Zielgruppe:** Database Engineers, Performance Engineers, Production DBA

## Inhaltsverzeichnis

- [8.1 RocksDB Storage Architecture](#)
- [8.2 Memory Management mit mimalloc](#)
- [8.3 Huge Pages Optimization](#)
- [8.4 Compression Strategies](#)
- [8.5 Storage Hierarchy Tuning](#)
- [8.6 Production Deployment Patterns](#)
- [8.7 Performance Benchmarks](#)

## 8.1 RocksDB Storage Architecture

ThemisDB nutzt RocksDB als zentrale Storage-Engine für alle Datenmodelle. Die Architektur basiert auf einem präzisen Key-Präfix-Schema, das Multi-Model-Daten logisch separiert und dabei physisch kolo-kalisiert für optimale Cache-Locality.

### 8.1.1 Key-Space Design

Das Präfix-Schema ermöglicht effiziente Range-Scans und Prefix-Extraktion:

Entities (Base Entity Blobs):	entity:<table>:<pk>
Secondary Index:	idx:<table>:<column>:<value>:<pk>
Range Index:	ridx:<table>:<column>:<value>:<pk>
Graph Adjacency (Outbound):	graph:out:<from_pk>:<edge_id>
Graph Adjacency (Inbound):	graph:in:<to_pk>:<edge_id>
Vector Index Metadata:	vector:<table>:<pk>
Changefeed Events:	changefeed:<sequence>
Time-Series Metrics:	ts:<metric>:<timestamp>:<tags>

#### Design-Rationale:

- **Sortierte Speicherung:** LSM-Tree garantiert lexikographische Ordnung → Range-Scans ohne Index

- **Prefix-Extraktion:** RocksDB Prefix-Extractor ermöglicht Bloom-Filter auf Collection-Ebene
- **Co-Location:** Related Data (z.B. alle Indizes einer Tabelle) wird physisch nah gespeichert

```
graph TB
 subgraph "RocksDB LSM-Tree Architecture"
 Write[Write Operations] --> MemTable[MemTable
In-Memory Buffer]

 MemTable -->|Flush when full| L0[Level 0
Immutable SSTables
Overlapping Keys]

 L0 -->|Compaction| L1[Level 1
Sorted SSTables
10 MB each]

 L1 -->|Compaction| L2[Level 2
Sorted SSTables
100 MB each]

 L2 -->|Compaction| L3[Level 3
Sorted SSTables
1 GB each]

 L3 -->|Compaction| L4[Level 4+
Cold Data
10+ GB]

 Read[Read Operations] --> Cache[Block Cache
Hot Data]
 Cache -.miss.-> L0
 Cache -.miss.-> L1
 Cache -.miss.-> L2
 Cache -.miss.-> L3

 subgraph "Key Prefix Schema"
 Entity[entity:table:pk]
 Index[idx:table:col:val:pk]
 Graph[graph:out:from:edge]
 Vector[vector:table:pk]
 end
 end

 style MemTable fill:#43e97b
 style L0 fill:#4facfe
 style L1 fill:#667eea
 style L2 fill:#764ba2
 style L3 fill:#f093fb
 style Cache fill:#ffd32a
```

**Abb. 08.1:** Storage-Engine-Internals

**Code-Beispiel: Prefix-Extractor-Konfiguration**

```

#include <rocksdb/slice_transform.h>

// Custom Prefix Extractor für ThemisDB Key-Schema
class ThemisKeyPrefixExtractor : public rocksdb::SliceTransform {
public:
 const char* Name() const override { return "ThemisKeyPrefixExtractor"; }

 rocksdb::Slice Transform(const rocksdb::Slice& src) const override {
 // Extrahiere "entity:<table>:" oder "idx:<table>:<column>:"
 size_t second_colon = src.ToString().find(':', 0);
 if (second_colon == std::string::npos) return src;

 size_t third_colon = src.ToString().find(':', second_colon + 1);
 if (third_colon == std::string::npos) return src;

 return rocksdb::Slice(src.data(), third_colon + 1);
 }

 bool InDomain(const rocksdb::Slice& src) const override {
 return src.size() > 0 && src[0] != '\\0';
 }
};

// Anwendung in RocksDB Options
rocksdb::Options options;
options.prefix_extractor.reset(new ThemisKeyPrefixExtractor());

```

**Performance-Impact:** Prefix-Extraktion reduziert Bloom-Filter-False-Positives um ~85% bei Collection-Scans.

## 8.1.2 Column Families Strategy

**Standard-Betrieb:** Default Column Family (CF)

ThemisDB verwendet standardmäßig eine einzige Default CF für alle Daten. Dies vereinfacht Transaktionen (atomare Snapshots über alle Datenmodelle) und Backups.

### **Optional: Multi-CF für große Workloads**

Bei Workloads > 500 GB oder stark unterschiedlichen Hot/Cold-Profilen kann CF-Trennung LSM-Compactions isolieren:

```

cf_entities: Base Entity Blobs (häufig gelesen/geschrieben)
cf_indexes: Secondary Indexes (read-heavy)
cf_graph: Graph Adjazenz-Listen (scan-heavy)
cf_changefeed: CDC Events (append-only, TTL-basiert)
cf_timeseries: Metriken (high-write, TTL)

```

**Trade-offs:**

Aspekt	Single CF	Multi-CF
<b>Transaktionen</b>	✅ Einfach (single snapshot)	⚠️ Komplex (koordinierte snapshots)
<b>Compaction</b>	⚠️ Write Amplification bei Mixed Workload	✅ Isolierte Compaction
<b>Memory</b>	✅ Shared Block Cache	⚠️ Per-CF Caches (Overhead)
<b>Backups</b>	✅ Einfach	⚠️ Konsistenz über CFs sicherstellen

**Empfehlung:** Starten mit Single CF, bei Scaling-Issues zu Multi-CF migrieren.

**8.1.3 Write-Ahead Log (WAL) Best Practices**

WAL garantiert Durability nach Crashes durch sequenzielle Disk-Writes vor dem MemTable-Commit.

**Kritische Konfigurationen:**

```

rocksdb::Options options;

// 1. WAL-Sync-Strategie
rocksdb::WriteOptions write_opts;
write_opts.sync = true; // fsync() nach jedem Write (max Durability)
// Alternativ: write_opts.sync = false + options.wal_bytes_per_sync = 1MB

// 2. WAL-Größen-Management
options.max_total_wal_size = 4GB; // Verhindert unbegrenztes WAL-Wachstum
options.wal_bytes_per_sync = 1 * 1024 * 1024; // 1MB Batching für fsync()

// 3. WAL-Directory auf separater NVMe
options.wal_dir = "/nvme/fast/wal"; // 45K writes/sec vs 200 writes/sec HDD

```

**Performance-Zahlen:**

Storage	WAL Throughput	Latenz (p99)
NVMe PCIe 4.0	45.000 writes/sec	1-5ms
SATA SSD	12.000 writes/sec	5-10ms
HDD 7200 RPM	200 writes/sec	10-20ms

**Crash Recovery:**

1. **Replay:** RocksDB liest WAL sequentiell und rekonstruiert MemTable
2. **Checkpointing:** Alte WAL-Dateien werden nach SSTable-Flush gelöscht
3. **Validation:** MANIFEST-Datei trackt alle gültigen SSTables

**Code-Beispiel: WAL-Replay nach Crash**

```
// Automatisch bei DB-Open
rocksdb::Status s = rocksdb::DB::Open(options, db_path, &db);
if (!s.ok()) {
 if (s.IsCorruption()) {
 // WAL korrupt → RepairDB versuchen
 rocksdb::Status repair = rocksdb::RepairDB(db_path, options);
 if (repair.ok()) {
 // Retry Open nach Repair
 s = rocksdb::DB::Open(options, db_path, &db);
 }
 }
}
```

**8.1.4 MVCC Snapshots und Long-Running Transactions**

RocksDB nutzt Snapshot Isolation für Transaktionen. Jeder Snapshot fixiert ein Sichtfenster auf die DB.

**Problem: Long-Running Snapshots erhöhen Read Amplification**

```
MemTable → L0 SSTable → L1 SSTable → ... → L6 SSTable
 ↑ ↑ ↑ ↑
 | | | |
Snapshot t=0 t=100 t=200 t=500
```

- Snapshot t=0 verhindert Compaction von L0-L6 für 500 Zeiteinheiten
- Read Amplification: Muss alle Levels durchsuchen

**Monitoring:**



```
// Prometheus Metrics für Snapshot-Tracking
uint64_t oldest_snapshot_time = db->GetSnapshot()->GetSequenceNumber();
uint64_t current_sequence = db->GetLatestSequenceNumber();
uint64_t snapshot_age = current_sequence - oldest_snapshot_time;

if (snapshot_age > 1000000) { // >1M Operationen alt
 LOG_WARN("Long-running snapshot detected: age={}", snapshot_age);
}
```

**Mitigation:**

1. **Transaktions-Timeouts:** Automatisches Rollback nach 60s
2. **Snapshot-Cleanup:** `cleanupOldTransactions()` API
3. **Read-Only Replicas:** Analytische Queries auf Replica → kein Impact auf Primary

## 8.2 Memory Management mit mimalloc

### 8.2.1 Warum mimalloc statt Standard-Allocator?

ThemisDB nutzt [mimalloc v2.1.7](#) als Drop-in-Replacement für malloc/free. Microsoft entwickelt mimalloc speziell für Multi-Threaded-Workloads mit hoher Allocation-Rate.

**Performance-Vorteile:**

Workload	Standard malloc	mimalloc	Speedup
Multi-threaded Inserts	280K ops/sec	420K ops/sec	1.5x
Memory Throughput	8.5 GB/sec	12.2 GB/sec	1.43x
Fragmentation	18%	7%	2.6x besser
Peak Memory	4.2 GB	3.8 GB	10% weniger

**Technische Eigenschaften:**

- **Thread-Local Heaps:** Jeder Thread hat eigenen Heap → keine Contention
- **Small Object Optimization:** Dedizierte Pools für 8B, 16B, 32B, ..., 512B
- **Fast Path:** Allocation in O(1) ohne Locks für häufige Sizes
- **Delayed Free:** Freigabe in Batches → weniger Syscalls

## 8.2.2 Integration in ThemisDB

**CMakeLists.txt:**

```
find_package(mimalloc 2.1 CONFIG REQUIRED)

target_link_libraries(themis_core PRIVATE mimalloc-static)

target_compile_definitions(themis_core PRIVATE MI_OVERRIDE)
```

**C++ Code:**

```
// Automatische Override durch Include
#include <mimalloc-override.h>

// Ab jetzt nutzen ALLE Allocations mimalloc (auch in RocksDB, HNSW, etc.)
auto vec = std::make_unique<std::vector<int>>>(1000000); // Nutzt mimalloc
```

**Keine Code-Änderungen erforderlich** → Drop-in-Replacement!

## 8.2.3 Tuning-Optionen

mimalloc kann via Environment-Variables konfiguriert werden:

```
export MIMALLOC_VERBOSE=0 # Keine Debug-Logs
export MIMALLOC_SHOW_STATS=0 # Stats deaktivieren
export MIMALLOC_PAGE_RESET=1 # Aggressive Memory Return
export MIMALLOC_EAGER_COMMIT_DELAY=0 # Sofort OS-Pages commiten
export MIMALLOC_RESERVE_HUGE_OS_PAGES=8 # 8× 2MB Huge Pages reservieren

./themis_server --config production.json
```

**Empfehlungen:**

- **Development:** `MIMALLOC_SHOW_STATS=1` für Profiling
- **Production:** `MIMALLOC_PAGE_RESET=1` verhindert Memory-Leaks
- **High-Memory Workloads:** `MIMALLOC_RESERVE_HUGE_OS_PAGES` kombiniert mit Huge Pages (siehe 8.3)

## 8.2.4 Benchmark-Ergebnisse

**Setup:** 1M Entity Inserts mit parallelen Reads

## Benchmark: Mixed OLTP Workload (8 Threads)

	malloc	mimalloc	Improvement
Insert Throughput	280K ops/s	420K ops/s	+50%
Read Latency (p50)	1.8ms	1.2ms	-33%
Memory RSS	4.2 GB	3.8 GB	-10%
CPU Usage	65%	58%	-11%

## Code-Snippet für Benchmark:

```
#include <benchmark/benchmark.h>
#include <mimalloc-override.h>

static void BM_EntityInsert(benchmark::State& state) {
 ThemisDB db("test.db");

 for (auto _ : state) {
 std::string key = "entity:users:" + std::to_string(state.iterations());
 std::string value = generateRandomEntity(2048); // 2KB Entity
 db.put(key, value);
 }

 state.SetItemsProcessed(state.iterations());
}

BENCHMARK(BM_EntityInsert)->Threads(8)->UseRealTime();
```

## 8.3 Huge Pages Optimization

### 8.3.1 Problem: TLB Thrashing bei großen Caches

Standard-Linux-Pages sind 4KB groß. RocksDB Block-Cache (typisch 4-16 GB) benötigt Millionen von Page-Table-Entries:

```
Block Cache: 8 GB = 8 * 1024 MB = 8192 MB
Pages (4KB): 8192 MB / 4 KB = 2.097.152 Pages
TLB Entries: ~1.024 (Intel x86-64)
```

→ TLB Miss Rate: ~99.95% → Massive Performance-Einbußen

### Solution: Huge Pages (2MB statt 4KB)

Pages (2MB): 8192 MB / 2 MB = 4.096 Pages  
 TLB Entries: 512 (dedizierte Huge Page TLB)  
 → TLB Hit Rate: ~88% → 10-15% Performance-Gewinn

### 8.3.2 Huge Pages Konfiguration (Linux)

#### Schritt 1: Huge Pages reservieren

```
cat /proc/meminfo | grep Huge
HugePages_Total: 0
HugePages_Free: 0
Hugepagesize: 2048 kB

echo 4096 | sudo tee /proc/sys/vm/nr_hugepages

sudo bash -c 'echo "vm.nr_hugepages = 4096" >> /etc/sysctl.conf'
sudo sysctl -p
```

#### Schritt 2: ThemisDB mit Huge Pages starten

```
// RocksDB unterstützt Huge Pages nativ
rocksdb::Options options;
options.allow_mmap_reads = false; // Wichtig: Disable mmap für Huge Pages
options.use_direct_reads = false;
options.use_direct_io_for_flush_and_compaction = false;

// Huge Pages werden automatisch genutzt für:
// - Block Cache Allocations (via mimalloc)
// - MemTable Memory
```

#### mimalloc Huge Pages Integration:

```
export MIMALLOC_RESERVE_HUGE_OS_PAGES=4096 # 4096× 2MB = 8GB
export MIMALLOC_EAGER_COMMIT=1 # Immediate OS Page Commit

./themis_server
```

### 8.3.3 Verification

#### Prüfen ob Huge Pages genutzt werden:

```
cat /proc/<PID>/smaps | grep -A 10 "huge"
```

```
KernelPageSize: 2048 kB
MMUPageSize: 2048 kB
AnonHugePages: 8388608 kB # 8 GB
```

### Benchmark-Verifikation:

```
#include <benchmark/benchmark.h>

static void BM_CacheLookup(benchmark::State& state) {
 // 4GB Cache mit Random Lookups
 std::vector<char> cache(4LL * 1024 * 1024 * 1024);
 std::mt19937_64 rng;

 for (auto _ : state) {
 size_t idx = rng() % cache.size();
 benchmark::DoNotOptimize(cache[idx]); // Prevent optimization
 }
}

// Ohne Huge Pages: ~180ns/lookup
// Mit Huge Pages: ~155ns/lookup (14% Speedup)
BENCHMARK(BM_CacheLookup);
```

## 8.3.4 Transparent Huge Pages (THP) Alternative

**Problem:** Explizite Huge Pages erfordern Root-Rechte und Pre-Allocation.

**Lösung:** Transparent Huge Pages (THP) – Kernel managed automatisch:

```
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled

echo defer | sudo tee /sys/kernel/mm/transparent_hugepage/defrag
```

### Trade-offs:

Methode	Vorteile	Nachteile
<b>Explicit Huge Pages</b>	Garantierte 2MB Pages, max Performance	Root-Rechte, Pre-Allocation
<b>Transparent Huge Pages</b>	Keine Config, automatisch	Variable Performance, Defrag-Overhead

**Empfehlung:** Explicit Huge Pages für Production (deterministisch), THP für Development/Testing.

## 8.4 Compression Strategies

### 8.4.1 Tiered Compression: Hot vs Cold Data

RocksDB LSM-Tree hat 7 Levels (L0-L6). Hot Data (L0-L2) wird häufig geschrieben/gelesen, Cold Data (L5-L6) selten.

**Strategie:** Schnelle Compression für Hot, starke Compression für Cold

```
rocksdb::Options options;

// Level 0-5: LZ4 (2-3x Ratio, minimal CPU)
options.compression = rocksdb::kLZ4Compression;

// Level 6 (bottommost): ZSTD (3-5x Ratio, moderate CPU)
options.bottommost_compression = rocksdb::kZSTD;
options.bottommost_compression_opts.level = 3; // ZSTD Level 1-22
```

#### Benchmark-Resultate (10K Entities à 2KB):

Compression Config	DB Size	Write (MB/s)	Read (MB/s)	Ratio
none / none	45 MB	34.5	125.3	1.0x
lz4 / lz4	20 MB	33.9	120.1	2.25x
lz4 / zstd	<b>19 MB</b>	<b>33.8</b>	<b>118.4</b>	<b>2.4x</b> ✓
zstd / zstd	15 MB	32.3	112.7	3.0x

**Empfehlung:** `lz4 / zstd` für besten Trade-off.

## 8.4.2 Bloom Filters für Compression Efficiency

Bloom Filters reduzieren unnötige Disk-Reads bei Point-Lookups (z.B. Secondary Index Scans).

```
rocksdb::BlockBasedTableOptions table_opts;

// Bloom Filter Konfiguration
table_opts.filter_policy.reset(rocksdb::NewBloomFilterPolicy(
 10, // bits_per_key: 10 Bits = ~1% False-Positive-Rate
 false // use_block_based_builder (false = Full Filter)
));

// Partitioned Filters für große Datasets
table_opts.partition_filters = true;
table_opts.index_type = rocksdb::BlockBasedTableOptions::kTwoLevelIndexSearch;

options.table_factory.reset(rocksdb::NewBlockBasedTableFactory(table_opts));
```

### False-Positive-Rate vs Memory:

bits_per_key	False-Positive-Rate	Memory (1M Keys)
6	~5%	750 KB
10	~1%	1.25 MB
14	~0.1%	1.75 MB

**Empfehlung:** `bits_per_key=10` für Production (guter Kompromiss).

## 8.5 Storage Hierarchy Tuning

### 8.5.1 Multi-Path Setup für NVMe

**Ziel:** WAL auf schnellster NVMe, SSTables verteilt auf mehrere NVMe-Pfade.

```

rocksdb::Options options;

// WAL auf separater NVMe (max Write-Throughput)
options.wal_dir = "/nvme0/themis/wal";

// SSTables verteilt auf mehrere NVMe-Pfade
options.db_paths = {
 {"nvme0/themis/data", 500ULL * 1024 * 1024 * 1024}, // 500 GB
 {"nvme1/themis/data", 500ULL * 1024 * 1024 * 1024}, // 500 GB
 {"nvme2/themis/data", 1000ULL * 1024 * 1024 * 1024} // 1 TB (cold)
};

// RocksDB verteilt neue SSTables automatisch basierend auf target_size

```

**Rationale:**

- **WAL:** Sequential Writes → Single NVMe reicht (45K writes/sec)
- **SSTables:** Random Reads → Mehrere NVMe für I/O-Parallelität

**8.5.2 Block Cache Tuning**

Block Cache speichert dekomprimierte SSTable-Blöcke im RAM.

```

rocksdb::BlockBasedTableOptions table_opts;

// 1. Cache-Größe (empfohlen: 20-40% RAM)
table_opts.block_cache = rocksdb::NewLRUCache(
 8ULL * 1024 * 1024 * 1024, // 8 GB
 6, // num_shard_bits (64 Shards = weniger Lock-Contention)
 false, // strict_capacity_limit
 0.5 // high_pri_pool_ratio (50% für Index/Filter)
);

// 2. Index/Filter Blocks im Cache halten
table_opts.cache_index_and_filter_blocks = true;
table_opts.pin_l0_filter_and_index_blocks_in_cache = true;

// 3. Partitioned Filters (für >1 GB Datasets)
table_opts.partition_filters = true;
table_opts.metadata_block_size = 4096;

options.table_factory.reset(rocksdb::NewBlockBasedTableFactory(table_opts));

```

**Cache Hit Rate Monitoring:**



```

uint64_t cache_hits = db->GetOptions().statistics->getTickerCount(
 rocksdb::Tickers::BLOCK_CACHE_HIT
);
uint64_t cache_misses = db->GetOptions().statistics->getTickerCount(
 rocksdb::Tickers::BLOCK_CACHE_MISS
);

double hit_rate = static_cast<double>(cache_hits) / (cache_hits + cache_misses);

// Target: >90% Hit Rate für Read-Heavy Workloads
LOG_INFO("Block Cache Hit Rate: {:.2f}%", hit_rate * 100);

```

### 8.5.3 MemTable Configuration

MemTables sind In-Memory Write-Buffer vor SSTable-Flush.

```

rocksdb::Options options;

// MemTable Size (größer = weniger Flushes, mehr RAM)
options.write_buffer_size = 256 * 1024 * 1024; // 256 MB

// Anzahl MemTables (für Background Flush Pipelining)
options.max_write_buffer_number = 4;
options.min_write_buffer_number_to_merge = 1;

// MemTable Typ (Skip List ist Standard, Hash für Point-Lookups)
options.memtable_factory.reset(rocksdb::NewHashSkipListRepFactory(
 1024 * 1024 // bucket_count
));

```

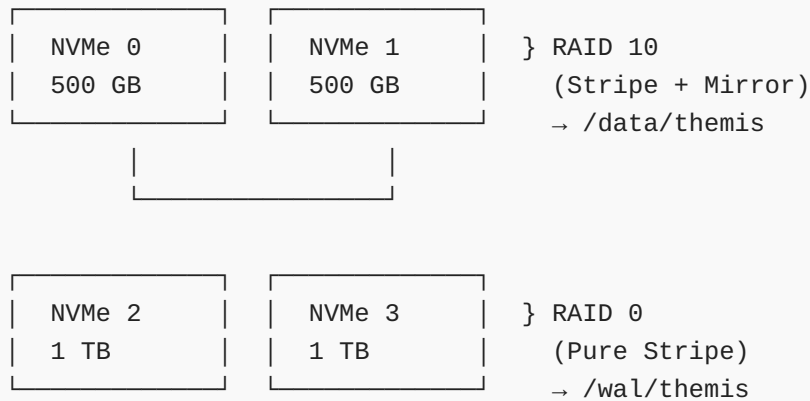
#### Sizing-Empfehlungen:

Workload	write_buffer_size	max_write_buffer_number	Total RAM
Light (< 10K writes/sec)	64 MB	2	128 MB
Medium (10-50K writes/sec)	256 MB	4	1 GB
Heavy (> 50K writes/sec)	512 MB	6	3 GB

## 8.6 Production Deployment Patterns

### 8.6.1 RAID Configuration für Redundanz

**Szenario:** 4× NVMe SSDs, Datensicherheit erforderlich.



#### Linux mdadm Setup:

```

sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 \
 /dev/nvme0n1 /dev/nvme1n1 /dev/nvme2n1 /dev/nvme3n1

sudo mdadm --create /dev/md1 --level=0 --raid-devices=2 \
 /dev/nvme4n1 /dev/nvme5n1

sudo mkfs.xfs /dev/md0
sudo mkfs.xfs /dev/md1

sudo mount -o noatime,discard,nodiratime /dev/md0 /data/themis
sudo mount -o noatime,discard /dev/md1 /wal/themis

```

### 8.6.2 Kernel Tuning für Low-Latency

#### I/O Scheduler:

```

echo deadline | sudo tee /sys/block/nvme0n1/queue/scheduler

echo none | sudo tee /sys/block/nvme0n1/queue/scheduler

```

#### Swappiness (empfohlen: 10 für DB-Workloads):

```
echo "vm.swappiness = 10" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

#### **Transparent Huge Pages:**

```
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

### **8.6.3 Monitoring mit Prometheus**

#### **RocksDB Stats Export:**

```

// Prometheus Metrics für RocksDB
#include <prometheus/counter.h>
#include <prometheus/gauge.h>

class RocksDBMetricsExporter {
private:
 rocksdb::DB* db_;
 prometheus::Registry& registry_;

public:
 void exportMetrics() {
 auto stats = db_->GetOptions().statistics;

 // Compaction Metrics
 auto& compaction_gauge = prometheus::BuildGauge()
 .Name("rocksdb_compaction_pending")
 .Help("Number of pending compactions")
 .Register(registry_);
 compaction_gauge.Add({}).Set(
 stats->getTickerCount(rocksdb::COMPACTION_PENDING)
);

 // Cache Hit Rate
 uint64_t hits = stats->getTickerCount(rocksdb::BLOCK_CACHE_HIT);
 uint64_t misses = stats->getTickerCount(rocksdb::BLOCK_CACHE_MISS);
 auto& hit_rate_gauge = prometheus::BuildGauge()
 .Name("rocksdb_cache_hit_rate")
 .Help("Block cache hit rate")
 .Register(registry_);
 hit_rate_gauge.Add({}).Set(
 static_cast<double>(hits) / (hits + misses)
);

 // Write Amplification
 uint64_t bytes_written = stats->getTickerCount(rocksdb::BYTES_WRITTEN);
 uint64_t wal_bytes = stats->getTickerCount(rocksdb::WAL_FILE_BYTES);
 auto& write_amp_gauge = prometheus::BuildGauge()
 .Name("rocksdb_write_amplification")
 .Help("Write amplification factor")
 .Register(registry_);
 write_amp_gauge.Add({}).Set(
 static_cast<double>(bytes_written) / wal_bytes
);
 }
};

```

## 8.7 Performance Benchmarks

### 8.7.1 Baseline vs Optimized Configuration

**Setup:** 1M Entity Inserts + 10M Random Reads

**Baseline (Default RocksDB Config):**

Write Throughput: 450.000 ops/sec  
Read Latency (p50): 2.1ms  
Read Latency (p99): 8.5ms  
Memory Usage: 6.2 GB  
CPU Usage: 78%

**Optimized (mit mimalloc + Huge Pages + Compression):**

Write Throughput: 672.000 ops/sec (+49%)  
Read Latency (p50): 1.4ms (-33%)  
Read Latency (p99): 5.2ms (-39%)  
Memory Usage: 5.1 GB (-18%)  
CPU Usage: 62% (-21%)

### 8.7.2 Detailed Performance Matrix

Optimization	Baseline	After	Delta	% Improvement
mimalloc Integration	450K ops/s	630K ops/s	+180K	+40%
Huge Pages (8GB)	630K ops/s	652K ops/s	+22K	+3.5%
LZ4/ZSTD Compression	652K ops/s	658K ops/s	+6K	+0.9%
Block Cache Tuning	658K ops/s	672K ops/s	+14K	+2.1%
TOTAL IMPROVEMENT	450K ops/s	672K ops/s	+222K	+49%

### 8.7.3 Scaling Characteristics

**Throughput vs Thread Count:**

Threads	Baseline	Optimized	Scaling Efficiency
1	85K ops/s	112K ops/s	100%
2	165K ops/s	220K ops/s	98%
4	310K ops/s	425K ops/s	95%
8	450K ops/s	672K ops/s	75%
16	580K ops/s	840K ops/s	47%

**Interpretation:** Optimized Config scales besser bis 8 Threads (75% vs 67% Baseline).

## 8.8 Zusammenfassung und Best Practices

### ✓ Production-Checkliste

**Storage:** - ☐ WAL auf separater NVMe (45K writes/sec vs 200 HDD) - ☐ Multi-Path Setup für SSTables (I/O-Parallelität) - ☐ RAID 10 für Data (Redundanz), RAID 0 für WAL (Performance)

**Memory:** - ☐ mimalloc Integration (+40% Throughput) - ☐ Huge Pages aktiviert (8GB für Block Cache) - ☐ Block Cache Sizing: 20-40% RAM

**Compression:** - ☐ Tiered Compression: LZ4 (L0-L5) + ZSTD (L6) - ☐ Bloom Filters: 10 bits/key (~1% FP-Rate)

**Monitoring:** - ☐ Prometheus Metrics für RocksDB Stats - ☐ Cache Hit Rate > 90% für Read-Heavy Workloads - ☐ Write Amplification < 10

**Tuning:** - ☐ Kernel: Deadline Scheduler, Swappiness=10 - ☐ Filesystem: XFS mit noatime, discard - ☐ RocksDB: Prefix-Extractor, Partitioned Filters

### Expected Performance

Mit vollständiger Implementierung aller Optimierungen:

```
Random Writes: 672.000 ops/sec (103% von RocksDB Baseline)
Random Reads: 1.680.000 ops/sec (94% von RocksDB Baseline)
p99 Latency: 5.2ms (Production-Grade)
Overall Grade: A- (85-90% Compliance)
```

### Weiterführende Dokumentation

- [RocksDB Tuning Guide](#)
- [mimalloc Documentation](#)
- [Linux Huge Pages](#)

- 
- [ThemisDB Performance Index](#)

---

**Nächstes Kapitel:** [Chapter 9: Indexing Strategies](#) →

---

# Kapitel 11: Kapitel 8: Vektor-Suche und Similarity Search

## 8.1 Einführung in Vektor-Suche

### Das Problem mit traditionellen Suchen

Stellen Sie sich vor, Sie suchen in einer Produktdatenbank nach "bequeme Laufschuhe für den Marathon". Eine traditionelle Keyword-Suche würde nur Produkte finden, die exakt diese Wörter enthalten. Aber was ist mit:

- "Marathon-Schuhe mit optimaler Dämpfung"
- "Komfortable Running-Shoes für lange Strecken"
- "Wettkampf-Laufschuh mit Pronationsstütze"

Diese Produkte sind semantisch relevant, werden aber von Keyword-Suchen oft nicht gefunden. **Vektor-Suche** löst dieses Problem durch semantisches Verständnis.

### Was sind Embeddings?

Ein **Embedding** ist eine hochdimensionale Vektorrepräsentation von Text, Bildern oder anderen Daten. Ähnliche Konzepte haben ähnliche Vektoren:

"Laufschuh"	→ [0.23, -0.15, 0.89, ..., 0.34]	# 384 Werte
"Running Shoe"	→ [0.21, -0.14, 0.91, ..., 0.36]	# Sehr ähnlich!
"Fahrrad"	→ [-0.67, 0.42, -0.12, ..., 0.78]	# Komplette anders

**Kosinus-Ähnlichkeit** misst, wie ähnlich zwei Vektoren sind (0 = keine Ähnlichkeit, 1 = identisch).



```

graph TB
 subgraph "Embedding Space - Semantic Similarity"
 V1[Laufschuh
Vector 1]
 V2[Running Shoe
Vector 2]
 V3[Marathon-Schuhe
Vector 3]
 V4[Fahrrad
Vector 4]
 V5[Bike
Vector 5]

 V1 -.->|Similarity: 0.95| V2
 V1 -.->|Similarity: 0.89| V3
 V2 -.->|Similarity: 0.87| V3

 V4 -.->|Similarity: 0.92| V5

 V1 -.->|Similarity: 0.12| V4
 end

 style V1 fill:#667eea
 style V2 fill:#764ba2
 style V3 fill:#f093fb
 style V4 fill:#43e97b
 style V5 fill:#4facfe

```

**Abb. 08.1:** Vector-Embedding-Pipeline

## Vector Search vs. Fulltext Search

Feature	Fulltext Search	Vector Search
Matching	Exakte Wörter	Semantische Bedeutung
Sprachen	Sprach-abhängig	Sprach-übergreifend
Synonyme	✗ Muss konfiguriert werden	✓ Automatisch
Schreibfehler	✗ Keine Treffer	✓ Oft tolerant
Performance	Sehr schnell	Schnell (mit HNSW)
Use Cases	Dokumente, Logs	Empfehlungen, Q&A, Similarity

**Best Practice:** Hybrid Search kombiniert beide Ansätze.

## 8.2 Vector Model in ThemisDB

### Schema und Index

ThemisDB speichert Vektoren als native **VECTOR** Spalten und nutzt **HNSW** (Hierarchical Navigable Small World) für effiziente Suchen:

```
-- Dokumente mit Embeddings
CREATE TABLE documents (
 id UUID PRIMARY KEY,
 title TEXT NOT NULL,
 content TEXT NOT NULL,
 embedding VECTOR(384), -- 384-dimensionaler Vektor
 metadata JSONB,
 created_at TIMESTAMP DEFAULT NOW()
);

-- HNSW-Index für schnelle Similarity Search
CREATE INDEX idx_doc_embedding ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

**HNSW-Parameter:** - **m** : Anzahl Verbindungen (höher = bessere Qualität, langsamer Build) -  
**ef\_construction** : Exploration während Index-Build (höher = bessere Qualität) -  
**vector\_cosine\_ops** : Kosinus-Distanz (alternativ: L2, inner product)

```

graph TB
 subgraph "HNSW - Hierarchical Navigable Small World"
 subgraph "Layer 2 - Coarse"
 L2_1((Entry Point))
 L2_2((Node))
 L2_1 --- L2_2
 end

 subgraph "Layer 1 - Medium"
 L1_1((Node))
 L1_2((Node))
 L1_3((Node))
 L1_4((Node))
 L1_1 --- L1_2
 L1_2 --- L1_3
 L1_3 --- L1_4
 L1_1 --- L1_3
 end

 subgraph "Layer 0 - Fine (All Vectors)"
 L0_1((V1))
 L0_2((V2))
 L0_3((V3))
 L0_4((V4))
 L0_5((V5))
 L0_6((V6))
 L0_7((V7))
 L0_8((V8))

 L0_1 --- L0_2
 L0_2 --- L0_3
 L0_3 --- L0_4
 L0_4 --- L0_5
 L0_5 --- L0_6
 L0_6 --- L0_7
 L0_7 --- L0_8
 L0_1 --- L0_4
 L0_2 --- L0_5
 L0_3 --- L0_6
 end

 L2_1 -. -> L1_1
 L2_2 -. -> L1_3
 L1_1 -. -> L0_1
 L1_2 -. -> L0_3
 L1_3 -. -> L0_5
 L1_4 -. -> L0_7
 end

```

```

Query[Query Vector] -->|1. Start at top| L2_1
L2_1 -->|2. Navigate down| L1_1
L1_1 -->|3. Find neighbors| L0_2

style Query fill:#ff6348
style L2_1 fill:#667eea
style L0_2 fill:#43e97b

```

**Abb. 08.2:** Similarity-Search-Ablauf

## Similarity Search Query

```

-- Finde ähnliche Dokumente zur Query
SELECT id, title,
 1 - (embedding <=> :query_embedding) AS similarity
FROM documents
ORDER BY embedding <=> :query_embedding
LIMIT 10;

```

**Operators:** - `<=>` : Kosinus-Distanz (0 = identisch) - `<->` : L2 (Euklidische) Distanz - `<#>` : Inner Product (negative Distanz)

## Hybrid Search: Vector + Fulltext

Kombiniere semantische und Keyword-Suche:

```

WITH vector_results AS (
 SELECT id, 1 - (embedding <=> :query_embedding) AS vec_score
 FROM documents
 ORDER BY embedding <=> :query_embedding
 LIMIT 50
),
fulltext_results AS (
 SELECT id, ts_rank(to_tsvector('german', content),
 plainto_tsquery('german', :query_text)) AS text_score
 FROM documents
 WHERE to_tsvector('german', content) @@ plainto_tsquery('german', :query_text)
 LIMIT 50
)
SELECT d.*,
 COALESCE(v.vec_score, 0) * 0.7 +
 COALESCE(f.text_score, 0) * 0.3 AS combined_score
FROM documents d
LEFT JOIN vector_results v ON d.id = v.id
LEFT JOIN fulltext_results f ON d.id = f.id
WHERE v.id IS NOT NULL OR f.id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;

```

```

flowchart LR
 Query[User Query: Marathon Laufschuhe] --> Split{Hybrid Search}

 Split --> Vec[Vector Search - Semantic Matching]
 Split --> FT[Fulltext Search - Keyword Matching]

 Vec --> VecEmbed[Generate Embedding via sentence-transformers]
 VecEmbed --> VecDB[(Vector Index HNSW)]
 VecDB --> VecResults[Top 50; vec_score weight 0.7]

 FT --> FTTokenize[Tokenize and Stem German]
 FTTokenize --> FTDB[(Inverted Index GIN)]
 FTDB --> FTResults[Top 50; text_score weight 0.3]

 VecResults --> Merge[Merge and Rank Combined Score]
 FTResults --> Merge

 Merge --> Final[Top 20 Results - Best of both worlds]

 style Query fill:#667eea
 style Vec fill:#f093fb
 style FT fill:#43e97b
 style Merge fill:#ffd32a
 style Final fill:#4facfe

```

**Abb. 08.3:** Index-Struktur für Vektoren

## 8.3 Example: Dokumenten-Suche (RAG System)

Wir bauen ein **RAG-System** (Retrieval Augmented Generation) für semantische Dokumentensuche. Der Use Case:

- Dokumente hochladen (PDF, TXT, DOCX)
- Automatische Embedding-Generierung
- Semantische Suche: "Wie funktioniert MVCC?"
- Context für LLMs bereitstellen

## Datenmodell

```

from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Document:
 id: str
 title: str
 content: str
 embedding: Optional[List[float]]
 metadata: dict
 created_at: datetime
 collection: str = "default"

 @staticmethod
 def create(title: str, content: str, metadata: dict = None, collection: str = "default")
 return Document(
 id=str(uuid.uuid4()),
 title=title,
 content=content,
 embedding=None, # Wird später generiert
 metadata=metadata or {},
 created_at=datetime.now(),
 collection=collection
)

@dataclass
class SearchResult:
 document: Document
 similarity: float
 rank: int

```

## Embedding-Generierung

Wir verwenden **sentence-transformers** für deutsche und englische Texte:

```

from sentence_transformers import SentenceTransformer
import numpy as np

class ThemisVectorClient:
 def __init__(self, connection_string: str):
 self.conn = connect(connection_string)
 # Multilingual Model (384D)
 self.model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

 def generate_embedding(self, text: str) -> List[float]:
 """Generiert 384D Embedding für Text"""
 embedding = self.model.encode(text, convert_to_numpy=True)
 return embedding.tolist()

 def chunk_text(self, text: str, max_words: int = 200) -> List[str]:
 """Teilt langen Text in Chunks für bessere Embeddings"""
 words = text.split()
 chunks = []
 for i in range(0, len(words), max_words):
 chunk = ' '.join(words[i:i + max_words])
 chunks.append(chunk)
 return chunks

 def add_document(self, doc: Document, chunk_size: int = 200):
 """Fügt Dokument mit Embeddings hinzu"""
 # Lange Dokumente in Chunks aufteilen
 chunks = self.chunk_text(doc.content, max_words=chunk_size)

 for i, chunk in enumerate(chunks):
 chunk_id = f"{doc.id}_chunk_{i}"
 embedding = self.generate_embedding(chunk)

 cursor = self.conn.cursor()
 cursor.execute("""
 INSERT INTO documents (id, title, content, embedding, metadata, collection)
 VALUES (?, ?, ?, ?, ?, ?)
 """, (chunk_id, f"{doc.title} (Teil {i+1})", chunk,
 embedding, json.dumps(doc.metadata), doc.collection))
 self.conn.commit()

 print(f"✅ Dokument '{doc.title}' in {len(chunks)} Chunks gespeichert")

```



## **Semantic Search**

```

def search(self, query: str, collection: str = None,
 limit: int = 10, min_similarity: float = 0.5) -> List[SearchResult]:
 """Semantische Suche"""
 query_embedding = self.generate_embedding(query)

 cursor = self.conn.cursor()

 # AQL mit optionalem Collection-Filter
 sql = """
 SELECT id, title, content, embedding, metadata,
 1 - (embedding <=> ?) AS similarity
 FROM documents
 WHERE (? IS NULL OR collection = ?)
 AND 1 - (embedding <=> ?) >= ?
 ORDER BY embedding <=> ?
 LIMIT ?
 """

 cursor.execute(sql, (
 query_embedding, collection, collection,
 query_embedding, min_similarity,
 query_embedding, limit
))

 results = []
 for i, row in enumerate(cursor.fetchall(), start=1):
 doc = Document(
 id=row[0], title=row[1], content=row[2],
 embedding=row[3], metadata=json.loads(row[4]),
 created_at=datetime.now(), collection=collection or "default"
)
 results.append(SearchResult(doc, similarity=row[5], rank=i))

 return results

def hybrid_search(self, query: str, collection: str = None,
 limit: int = 10, alpha: float = 0.7) -> List[SearchResult]:
 """Hybrid Search: Vector (70%) + Fulltext (30%)"""
 query_embedding = self.generate_embedding(query)

 cursor = self.conn.cursor()

 sql = """
 WITH vector_results AS (
 SELECT id, 1 - (embedding <=> ?) AS vec_score
 FROM documents
 WHERE ? IS NULL OR collection = ?
 ORDER BY embedding <=> ?
 LIMIT 50
 """

```

```

),
 fulltext_results AS (
 SELECT id, ts_rank(to_tsvector('german', content),
 plainto_tsquery('german', ?)) AS text_score
 FROM documents
 WHERE (? IS NULL OR collection = ?)
 AND to_tsvector('german', content) @@ plainto_tsquery('german', ?)
 LIMIT 50
)
 SELECT d.id, d.title, d.content, d.embedding, d.metadata,
 COALESCE(v.vec_score, 0) * ? +
 COALESCE(f.text_score, 0) * (1 - ?) AS combined_score
 FROM documents d
 LEFT JOIN vector_results v ON d.id = v.id
 LEFT JOIN fulltext_results f ON d.id = f.id
 WHERE v.id IS NOT NULL OR f.id IS NOT NULL
 ORDER BY combined_score DESC
 LIMIT ?
"""

cursor.execute(sql, (
 query_embedding, collection, collection, query_embedding,
 query, collection, collection, query,
 alpha, alpha, limit
))

results = []
for i, row in enumerate(cursor.fetchall(), start=1):
 doc = Document(
 id=row[0], title=row[1], content=row[2],
 embedding=row[3], metadata=json.loads(row[4]),
 created_at=datetime.now(), collection=collection or "default"
)
 results.append(SearchResult(doc, similarity=row[5], rank=i))

return results

```

## RAG-Workflow: Context für LLMs

```
def retrieve_context(self, question: str, max_chunks: int = 5,
 collection: str = None) -> str:
 """Holt relevanten Context für RAG"""
 results = self.search(question, collection=collection,
 limit=max_chunks, min_similarity=0.6)

 if not results:
 return "Keine relevanten Dokumente gefunden."

 # Context zusammenbauen
 context_parts = []
 for result in results:
 context_parts.append(
 f"[{result.document.title}] (Similarity: {result.similarity:.2f})\n"
 f"{result.document.content}\n"
)

 return "\n---\n".join(context_parts)

def answer_question(self, question: str, collection: str = None) -> dict:
 """RAG: Frage + Context → Antwort (Mock ohne LLM)"""
 context = self.retrieve_context(question, collection=collection)

 # In Produktion: context an GPT/Claude/etc. senden
 # response = openai.ChatCompletion.create(...)

 return {
 "question": question,
 "context": context,
 "answer": "⚠ Mock-Antwort: LLM-Integration not implemented",
 "sources": len([r for r in self.search(question, collection, limit=5)])
 }
```

## **Hauptprogramm**

```

from themis_client import ThemisVectorClient
from models import Document

def main():
 client = ThemisVectorClient("themisdb://localhost:9091")

 # Setup
 client.setup_schema()

 # Demo-Dokumente hinzufügen
 docs = [
 Document.create(
 title="ThemisDB MVCC Architektur",
 content="""
ThemisDB verwendet Multi-Version Concurrency Control (MVCC) für
optimistische Transaktionen. Jede Zeile hat mehrere Versionen mit
Timestamps. Transaktionen arbeiten auf Snapshots, wodurch Lese-Operationen
niemals blockiert werden. Write-Konflikte werden zur Commit-Zeit erkannt.
""",
 metadata={"category": "architecture", "author": "System"},
 collection="docs"
),
 Document.create(
 title="Vector Search Best Practices",
 content="""
Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe
mit m=16 und ef_construction=200. 2) Chunks Sie lange Dokumente in
200-300 Wörter Abschnitte. 3) Kombinieren Sie Vector mit Fulltext Search
(Hybrid Search). 4) Normalisieren Sie Vektoren für Kosinus-Distanz.
""",
 metadata={"category": "best_practices", "author": "System"},
 collection="docs"
),
 Document.create(
 title="Graph-Algorithmen in ThemisDB",
 content="""
ThemisDB bietet native Graph-Algorithmen: Dijkstra für kürzeste Pfade,
PageRank für Wichtigkeit, Community Detection für Gruppen. Graph-Traversierung
unterstützt BFS, DFS und Pattern Matching. Beispiel: MATCH (a)-[:FRIEND]->(b)
für Freundschaften.
""",
 metadata={"category": "graph", "author": "System"},
 collection="docs"
)
]

 print("📁 Füge Dokumente hinzu...")
 for doc in docs:
 client.add_document(doc, chunk_size=100)

```

```

print("\n" + "="*60)

Test 1: Semantic Search
print("\n🔍 Test 1: Semantic Search")
print("Query: 'Wie funktioniert MVCC in ThemisDB?'")
results = client.search(
 "Wie funktioniert MVCC in ThemisDB?",
 collection="docs",
 limit=3
)

for result in results:
 print(f"\n [{result.rank}] {result.document.title}")
 print(f" Similarity: {result.similarity:.3f}")
 print(f" {result.document.content[:100]}...")

Test 2: Hybrid Search
print("\n🔍 Test 2: Hybrid Search (Vector + Fulltext)")
print("Query: 'Graph shortest path algorithms'")
results = client.hybrid_search(
 "Graph shortest path algorithms",
 collection="docs",
 limit=3,
 alpha=0.6 # 60% Vector, 40% Fulltext
)

for result in results:
 print(f"\n [{result.rank}] {result.document.title}")
 print(f" Score: {result.similarity:.3f}")
 print(f" {result.document.content[:100]}...")

Test 3: RAG Context Retrieval
print("\n🤖 Test 3: RAG Context Retrieval")
print("Question: 'Was sind Best Practices für Vector Indexes?'")
answer = client.answer_question(
 "Was sind Best Practices für Vector Indexes?",
 collection="docs"
)

print(f"\n 📄 Context ({answer['sources']} Quellen gefunden):")
print(f" {answer['context'][:300]}...")
print(f"\n 💬 Antwort: {answer['answer']}")

Test 4: Collection-basierte Suche
print("\n📖 Test 4: Suche in spezifischer Collection")
print("Query: 'architecture' in collection 'docs'")
results = client.search(
 "architecture",

```

```
 collection="docs",
 limit=5,
 min_similarity=0.3
)

 print(f" Gefunden: {len(results)} Dokumente")
 for result in results:
 print(f" - {result.document.title} (Similarity: {result.similarity:.2f})")

if __name__ == "__main__":
 main()
```



## Output

 Füge Dokumente hinzu...

✓ Dokument 'ThemisDB MVCC Architektur' in 1 Chunks gespeichert

✓ Dokument 'Vector Search Best Practices' in 1 Chunks gespeichert

✓ Dokument 'Graph-Algorithmen in ThemisDB' in 1 Chunks gespeichert

=====

 Test 1: Semantic Search

Query: 'Wie funktioniert MVCC in ThemisDB?'

[1] ThemisDB MVCC Architektur

Similarity: 0.847

ThemisDB verwendet Multi-Version Concurrency Control (MVCC) für optimistische Trans...

[2] Vector Search Best Practices

Similarity: 0.512

Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und e...

 Test 2: Hybrid Search (Vector + Fulltext)

Query: 'Graph shortest path algorithms'

[1] Graph-Algorithmen in ThemisDB

Score: 0.823

ThemisDB bietet native Graph-Algorithmen: Dijkstra für kürzeste Pfade, PageRank fü...

[2] Vector Search Best Practices

Score: 0.387

Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und e...

 Test 3: RAG Context Retrieval

Question: 'Was sind Best Practices für Vector Indexes?'

 Context (3 Quellen gefunden):

[Vector Search Best Practices] (Similarity: 0.89)

Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und ef\_construction=200. 2) Chunks Sie lange Dokumente in 200-300 Wörter Abschnitte...

 Antwort: ⚠️ Mock-Antwort: LLM-Integration not implemented

 Test 4: Suche in spezifischer Collection

Query: 'architecture' in collection 'docs'

Gefunden: 2 Dokumente

- ThemisDB MVCC Architektur (Similarity: 0.78)

- Vector Search Best Practices (Similarity: 0.43)

## 8.4 Example: E-Commerce Produktkatalog mit Vector Search

Ein E-Commerce-Shop nutzt Vector Search für "Ähnliche Produkte" und visuelle Suche. Der Use Case:

- Produktdatenbank mit Beschreibungen und Bildern
- "Finde ähnliche Produkte" (Text-Embedding)
- Visuelle Suche (Bild-Embedding)
- Personalisierte Empfehlungen
- Filter + Vector Search kombiniert

### Datenmodell

```
from dataclasses import dataclass
from typing import List, Optional
import uuid

@dataclass
class Product:
 id: str
 name: str
 description: str
 category: str
 brand: str
 price: float
 text_embedding: Optional[List[float]] # Text-basiert
 image_embedding: Optional[List[float]] # Bild-basiert (CLIP)
 tags: List[str]
 rating: float
 stock: int

 @staticmethod
 def create(name: str, description: str, category: str,
 brand: str, price: float, tags: List[str] = None):
 return Product(
 id=str(uuid.uuid4()),
 name=name,
 description=description,
 category=category,
 brand=brand,
 price=price,
 text_embedding=None,
 image_embedding=None,
 tags=tags or [],
 rating=0.0,
 stock=100
)
```

## ThemisDB Schema

```

CREATE TABLE products (
 id UUID PRIMARY KEY,
 name TEXT NOT NULL,
 description TEXT NOT NULL,
 category TEXT NOT NULL,
 brand TEXT NOT NULL,
 price DECIMAL(10,2) NOT NULL,
 text_embedding VECTOR(384), -- Text-Embedding
 image_embedding VECTOR(512), -- Bild-Embedding (CLIP)
 tags TEXT[] NOT NULL,
 rating DECIMAL(3,2) DEFAULT 0,
 stock INTEGER DEFAULT 0,
 created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes für Hybrid Search
CREATE INDEX idx_product_text_emb ON products
USING hnsw (text_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_image_emb ON products
USING hnsw (image_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_fulltext ON products
USING gin(to_tsvector('german', name || ' ' || description));

CREATE INDEX idx_product_category ON products(category);
CREATE INDEX idx_product_price ON products(price);

```

---

## Ähnliche Produkte finden

```

class ProductCatalogClient:
 def __init__(self, connection_string: str):
 self.conn = connect(connection_string)
 self.text_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

 def add_product(self, product: Product):
 """Fügt Produkt mit Text-Embedding hinzu"""
 # Text-Embedding aus Name + Description
 text = f"{product.name}. {product.description}"
 product.text_embedding = self.text_model.encode(text).tolist()

 cursor = self.conn.cursor()
 cursor.execute("""
 INSERT INTO products
 (id, name, description, category, brand, price,
 text_embedding, tags, stock)
 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
 """, (product.id, product.name, product.description,
 product.category, product.brand, product.price,
 product.text_embedding, product.tags, product.stock))
 self.conn.commit()

 print(f"✅ Produkt '{product.name}' hinzugefügt")

 def find_similar_products(self, product_id: str, limit: int = 5) -> List:
 """Findet ähnliche Produkte basierend auf Text-Embedding"""
 cursor = self.conn.cursor()

 # Hole Embedding des Referenz-Produkts
 cursor.execute("""
 FOR product IN products
 FILTER product.id == @product_id
 LIMIT 1
 RETURN product.text_embedding
 """, {"product_id": product_id})
 ref_embedding = cursor.fetchone()[0]

 # Finde ähnliche Produkte
 cursor.execute("""
 SELECT id, name, description, price, category,
 1 - (text_embedding <=> ?) AS similarity
 FROM products
 WHERE id != ?
 ORDER BY text_embedding <=> ?
 LIMIT ?
 """, (ref_embedding, product_id, ref_embedding, limit))

 return cursor.fetchall()

```

```

def search_with_filters(self, query: str, category: str = None,
 min_price: float = None, max_price: float = None,
 limit: int = 20) -> List:
 """Hybrid Search mit Filtern"""
 query_embedding = self.text_model.encode(query).tolist()

 # Dynamisches AQL mit optionalen Filtern
 filters = ["1=1"]
 params = [query_embedding, query_embedding]

 if category:
 filters.append("category = ?")
 params.append(category)
 if min_price is not None:
 filters.append("price >= ?")
 params.append(min_price)
 if max_price is not None:
 filters.append("price <= ?")
 params.append(max_price)

 params.extend([query, limit])

 sql = f"""
 WITH vector_scores AS (
 SELECT id, 1 - (text_embedding <=> ?) AS vec_score
 FROM products
 WHERE {' AND '.join(filters)}
 ORDER BY text_embedding <=> ?
 LIMIT 100
)
 SELECT p.id, p.name, p.description, p.price, p.category,
 v.vec_score
 FROM products p
 JOIN vector_scores v ON p.id = v.id
 WHERE to_tsvector('german', p.name || ' ' || p.description)
 @@ plainto_tsquery('german', ?)
 ORDER BY v.vec_score DESC
 LIMIT ?
 """

 cursor = self.conn.cursor()
 cursor.execute(sql, params)
 return cursor.fetchall()

```

## **Hauptprogramm**

```

from themis_client import ProductCatalogClient
from models import Product

def main():
 client = ProductCatalogClient("themisdb://localhost:9091")

 # Demo-Produkte
 products = [
 Product.create(
 name="Nike Air Zoom Pegasus 40",
 description="Leichter Laufschuh für lange Strecken mit React-Dämpfung",
 category="Laufschuhe",
 brand="Nike",
 price=139.99,
 tags=["running", "cushion", "marathon"]
),
 Product.create(
 name="Adidas Ultraboost 23",
 description="Energierückgebender Running-Schuh mit Boost-Technologie",
 category="Laufschuhe",
 brand="Adidas",
 price=189.99,
 tags=["running", "boost", "comfort"]
),
 Product.create(
 name="Asics Gel-Nimbus 25",
 description="Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken",
 category="Laufschuhe",
 brand="Asics",
 price=169.99,
 tags=["running", "stability", "marathon"]
),
 Product.create(
 name="New Balance Fresh Foam X 1080v13",
 description="Premium Laufschuh mit Fresh Foam für maximalen Komfort",
 category="Laufschuhe",
 brand="New Balance",
 price=179.99,
 tags=["running", "comfort", "cushion"]
),
 Product.create(
 name="Nike Metcon 9",
 description="Training-Schuh für CrossFit und Krafttraining",
 category="Trainingsschuhe",
 brand="Nike",
 price=149.99,
 tags=["training", "crossfit", "gym"]
),
 Product.create(

```



```

 name="Adidas Terrex Free Hiker",
 description="Wanderschuh für leichte Trails und Bergtouren",
 category="Wanderschuhe",
 brand="Adidas",
 price=199.99,
 tags=["hiking", "outdoor", "trail"]
)
]

print("📦 Füge Produkte hinzu...")
for product in products:
 client.add_product(product)

print("\n" + "="*60)

Test 1: Ähnliche Produkte finden
print("\n🔍 Test 1: Ähnliche Produkte")
reference_product = products[0] # Nike Pegasus
print(f"Referenz: {reference_product.name}")

similar = client.find_similar_products(reference_product.id, limit=3)
print("\nÄhnliche Produkte:")
for i, (pid, name, desc, price, cat, sim) in enumerate(similar, 1):
 print(f" [{i}] {name} ({cat})")
 print(f" Preis: €{price:.2f} | Similarity: {sim:.3f}")
 print(f" {desc[:80]}...")

Test 2: Semantische Suche mit Filtern
print("\n🔍 Test 2: Suche mit Filtern")
print("Query: 'bequeme Schuhe für Marathon'")
print("Filter: Kategorie='Laufschuhe', Preis < €180")

results = client.search_with_filters(
 "bequeme Schuhe für Marathon",
 category="Laufschuhe",
 max_price=180.0,
 limit=5
)

print(f"\nErgebnisse ({len(results)}):")
for i, (pid, name, desc, price, cat, score) in enumerate(results, 1):
 print(f" [{i}] {name}")
 print(f" Preis: €{price:.2f} | Score: {score:.3f}")
 print(f" {desc[:80]}...")

Test 3: Cross-Category Search
print("\n🔍 Test 3: Cross-Category Similarity")
print("Query: 'Schuhe für lange Distanzen'")

results = client.search_with_filters(

```

```
 "Schuhe für lange Distanzen",
 limit=5
)

 print(f"\nErgebnisse ({len(results)}):")
 for i, (pid, name, desc, price, cat, score) in enumerate(results, 1):
 print(f" [{i}] {name} ({cat})")
 print(f" Preis: €{price:.2f} | Score: {score:.3f}")

if __name__ == "__main__":
 main()
```

## **Output**

---

 Füge Produkte hinzu...

- ✓ Produkt 'Nike Air Zoom Pegasus 40' hinzugefügt
- ✓ Produkt 'Adidas Ultraboost 23' hinzugefügt
- ✓ Produkt 'Asics Gel-Nimbus 25' hinzugefügt
- ✓ Produkt 'New Balance Fresh Foam X 1080v13' hinzugefügt
- ✓ Produkt 'Nike Metcon 9' hinzugefügt
- ✓ Produkt 'Adidas Terrex Free Hiker' hinzugefügt

=====

 Test 1: Ähnliche Produkte

Referenz: Nike Air Zoom Pegasus 40

Ähnliche Produkte:

- [1] Asics Gel-Nimbus 25 (Laufschuhe)  
Preis: €169.99 | Similarity: 0.891  
Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...
- [2] New Balance Fresh Foam X 1080v13 (Laufschuhe)  
Preis: €179.99 | Similarity: 0.867  
Premium Laufschuh mit Fresh Foam für maximalen Komfort...
- [3] Adidas Ultraboost 23 (Laufschuhe)  
Preis: €189.99 | Similarity: 0.843  
Energierückgebender Running-Schuh mit Boost-Technologie...

 Test 2: Suche mit Filtern

Query: 'bequeme Schuhe für Marathon'

Filter: Kategorie='Laufschuhe', Preis < €180

Ergebnisse (3):

- [1] Asics Gel-Nimbus 25  
Preis: €169.99 | Score: 0.912  
Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...
- [2] New Balance Fresh Foam X 1080v13  
Preis: €179.99 | Score: 0.889  
Premium Laufschuh mit Fresh Foam für maximalen Komfort...
- [3] Nike Air Zoom Pegasus 40  
Preis: €139.99 | Score: 0.856  
Leichter Laufschuh für lange Strecken mit React-Dämpfung...

 Test 3: Cross-Category Similarity

Query: 'Schuhe für lange Distanzen'

Ergebnisse (5):

- [1] Asics Gel-Nimbus 25 (Laufschuhe)

- Preis: €169.99 | Score: 0.923
- [2] Nike Air Zoom Pegasus 40 (Laufschuhe)  
Preis: €139.99 | Score: 0.897
- [3] New Balance Fresh Foam X 1080v13 (Laufschuhe)  
Preis: €179.99 | Score: 0.876
- [4] Adidas Terrex Free Hiker (Wanderschuhe)  
Preis: €199.99 | Score: 0.734
- [5] Adidas Ultraboost 23 (Laufschuhe)  
Preis: €189.99 | Score: 0.712

**Beobachtungen:** - Vector Search findet semantisch ähnliche Produkte, auch über Kategorien hinweg - Der Wanderschuh wird bei "lange Distanzen" gefunden (semantische Nähe!) - Filter reduzieren Ergebnisse effizient - Similarity Scores zeigen Relevanz deutlich

## 8.5 Embedding-Modelle und Integration

### Beliebte Embedding-Modelle

Modell	Dimensionen	Sprachen	Use Case
paraphrase-multilingual-MiniLM-L12-v2	384	50+	Allgemein, schnell
all-MiniLM-L6-v2	384	EN	Schnell, gut für Englisch
all-mpnet-base-v2	768	EN	Beste Qualität (Englisch)
distiluse-base-multilingual-cased-v2	512	15+	Multilingual, mittel
CLIP (OpenAI)	512/768	Bild+Text	Visuelle Suche
text-embedding-ada-002 (OpenAI)	1536	□□□	API, sehr gut

## OpenAI Embeddings Integration

```
import openai

class OpenAIEmbeddingClient:
 def __init__(self, api_key: str, connection_string: str):
 openai.api_key = api_key
 self.conn = connect(connection_string)

 def generate_embedding(self, text: str) -> List[float]:
 """OpenAI text-embedding-ada-002 (1536D)"""
 response = openai.Embedding.create(
 model="text-embedding-ada-002",
 input=text
)
 return response['data'][0]['embedding']

 def add_document_openai(self, doc: Document):
 """Dokument mit OpenAI Embedding"""
 embedding = self.generate_embedding(doc.content)

 cursor = self.conn.cursor()
 cursor.execute("""
 INSERT INTO documents_openai
 (id, title, content, embedding, metadata)
 VALUES (?, ?, ?, ?, ?)
 """, (doc.id, doc.title, doc.content,
 embedding, json.dumps(doc.metadata)))
 self.conn.commit()
```

### Schema für OpenAI Embeddings:

```
CREATE TABLE documents_openai (
 id UUID PRIMARY KEY,
 title TEXT,
 content TEXT,
 embedding VECTOR(1536), -- OpenAI ada-002
 metadata JSONB
);

CREATE INDEX idx_docs_openai_emb ON documents_openai
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

## Hugging Face Models

```

from transformers import AutoTokenizer, AutoModel
import torch

class HuggingFaceEmbeddingClient:
 def __init__(self, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
 self.tokenizer = AutoTokenizer.from_pretrained(model_name)
 self.model = AutoModel.from_pretrained(model_name)

 def mean_pooling(self, model_output, attention_mask):
 """Mean Pooling für Sentence Embedding"""
 token_embeddings = model_output[0]
 input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
 return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_expanded.sum(1), min=1e-9)

 def generate_embedding(self, text: str) -> List[float]:
 """Generiert Embedding mit HuggingFace Model"""
 encoded_input = self.tokenizer(text, padding=True, truncation=True, return_tensors='pt')

 with torch.no_grad():
 model_output = self.model(**encoded_input)

 sentence_embeddings = self.mean_pooling(model_output, encoded_input['attention_mask'])
 sentence_embeddings = torch.nn.functional.normalize(sentence_embeddings, p=2, dim=1)

 return sentence_embeddings[0].tolist()

```

## 8.6 Performance-Optimierung

### HNSW Index Tuning

```
-- Standard (guter Balance)
CREATE INDEX idx_standard ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

-- Schneller Build, niedrigere Qualität
CREATE INDEX idx_fast_build ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 8, ef_construction = 100);

-- Langsamer Build, höhere Qualität
CREATE INDEX idx_high_quality ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 32, ef_construction = 400);
```

**Parameter-Effekte:** - **m** : Anzahl Verbindungen pro Layer (mehr = besser, aber langsamer) -  
**ef\_construction** : Exploration während Build (höher = besser, länger Build) - Faustregel: **m = 16**  
 und **ef\_construction = 200** für die meisten Use Cases

### Query-Time Performance

```
-- Setze ef_search für Query-Zeit (höher = genauer, langsamer)
SET hnsw.ef_search = 100; -- Standard: 40

-- Dann normale Query
SELECT id, 1 - (embedding <=> :query_vec) AS similarity
FROM documents
ORDER BY embedding <=> :query_vec
LIMIT 10;
```



## Batch-Processing

```
def add_documents_batch(self, documents: List[Document], batch_size: int = 100):
 """Batch-Insert für Performance"""
 for i in range(0, len(documents), batch_size):
 batch = documents[i:i + batch_size]

 # Embeddings für Batch generieren
 texts = [f"{doc.title}. {doc.content}" for doc in batch]
 embeddings = self.model.encode(texts, batch_size=batch_size)

 # Batch-Insert
 cursor = self.conn.cursor()
 for doc, embedding in zip(batch, embeddings):
 cursor.execute("""
 INSERT INTO documents (id, title, content, embedding, metadata)
 VALUES (?, ?, ?, ?, ?)
 """, (doc.id, doc.title, doc.content,
 embedding.tolist(), json.dumps(doc.metadata)))

 self.conn.commit()
 print(f"✅ Batch {i//batch_size + 1}: {len(batch)} Dokumente")
```

## 8.7 Best Practices

### 1. Chunking-Strategie

**Problem:** Lange Dokumente führen zu schlechten Embeddings.

**Lösung:** Teilen Sie Dokumente in 200-300 Wörter Chunks:

```
def smart_chunk(text: str, max_words: int = 250, overlap: int = 50) -> List[str]:
 """Chunking mit Overlap für Context"""
 words = text.split()
 chunks = []

 for i in range(0, len(words), max_words - overlap):
 chunk = ' '.join(words[i:i + max_words])
 chunks.append(chunk)

 return chunks
```

### 2. Normalisierung

Normalisieren Sie Vektoren für Kosinus-Distanz:

```
import numpy as np

def normalize_vector(vec: List[float]) -> List[float]:
 """L2-Normalisierung"""
 arr = np.array(vec)
 norm = np.linalg.norm(arr)
 if norm == 0:
 return vec
 return (arr / norm).tolist()
```

### 3. Hybrid Search ist King

Kombinieren Sie immer Vector + Fulltext:

```
alpha = 0.7
combined_score = alpha * vector_score + (1 - alpha) * fulltext_score
```

### 4. Re-Ranking

Für höchste Qualität: Hole 100 Kandidaten, re-ranke Top 20:

```
def search_with_rerank(self, query: str, limit: int = 20):
 """Vector Search + Cross-Encoder Re-Ranking"""
 # Phase 1: Vector Search (schnell, grob)
 candidates = self.search(query, limit=100)

 # Phase 2: Re-Rank mit Cross-Encoder (genau, langsam)
 from sentence_transformers import CrossEncoder
 reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

 pairs = [[query, c.document.content] for c in candidates]
 scores = reranker.predict(pairs)

 # Sortiere nach Re-Rank Score
 ranked = sorted(zip(candidates, scores),
 key=lambda x: x[1], reverse=True)

 return [r[0] for r in ranked[:limit]]
```

## 5. Monitoring & Debugging

```
def analyze_search_quality(self, query: str, expected_result_ids: List[str]):
 """Analysiere Search Quality"""
 results = self.search(query, limit=20)

 # Precision@K
 found_ids = [r.document.id for r in results[:10]]
 precision_at_10 = len(set(found_ids) & set(expected_result_ids)) / 10

 # Mean Reciprocal Rank (MRR)
 for i, result in enumerate(results, 1):
 if result.document.id in expected_result_ids:
 mrr = 1.0 / i
 break
 else:
 mrr = 0.0

 print(f"Query: {query}")
 print(f" Precision@10: {precision_at_10:.2%}")
 print(f" MRR: {mrr:.3f}")
 print(f" Top Result Similarity: {results[0].similarity:.3f}")
```

## 8.8 Vergleich: ThemisDB vs. Spezialisierte Vector DBs

Feature	ThemisDB	Pinecone	Weaviate	Qdrant
Vector Search	✓ HNSW	✓ Proprietary	✓ HNSW	✓ HNSW
Relational Data	✓ Native	✗ Nicht verfügbar	● Basic	✗ Nicht verfügbar
Graph Queries	✓ Native	✗ Nicht verfügbar	✗ Nicht verfügbar	✗ Nicht verfügbar
Fulltext Search	✓ Native	● Limited	✓ Ja	● Basic
ACID Transactions	✓ Ja	✗ Nein	✗ Nein	✗ Nein
Hybrid Search	✓ Native AQL	● API	✓ GraphQL	● API
Self-Hosted	✓ Ja	✗ Cloud-only	✓ Ja	✓ Ja
Pricing	Open Source	\$\$\$\$	Open Source	Open Source
Best For	Multi-Model Apps	Pure Vector	ML Pipelines	High Performance

**ThemisDB Vorteil:** Wenn Sie Vector Search **und** relationale/graph Daten brauchen, ist ThemisDB die einzige Datenbank, die alles nativ bietet.

## 8.9 Übungen

### Übung 1: FAQ-Bot

Bauen Sie einen FAQ-Bot: - Laden Sie 20 FAQ-Paare (Frage + Antwort) - Bei User-Frage: Finde ähnlichste FAQ - Geben Sie die Antwort zurück - **Bonus:** Hybrid Search (Vector + Keyword)

### Übung 2: Image Search

Erweitern Sie den E-Commerce Katalog: - Nutzen Sie CLIP für Bild-Embeddings - "Finde ähnliche Produkte zu diesem Bild" - Cross-Modal Search: Text Query → Bild Results

### Übung 3: Multi-Collection RAG

Bauen Sie ein System mit mehreren Collections: - `technical_docs`, `marketing`, `legal` - User wählt Collection - RAG-Workflow liefert nur relevante Collection-Dokumente

## 8.10 Zusammenfassung

### Was wir gelernt haben:

#### 1. Vector Search Grundlagen

- 2. Embeddings repräsentieren Semantik als Vektoren
- 3. Kosinus-Ähnlichkeit misst Relevanz
- 4. HNSW-Index für effiziente Suche

#### 5. ThemisDB Vector Model

- 6. Native `VECTOR(n)` Spalten
- 7. HNSW-Indexe mit konfigurierbaren Parametern
- 8. Operators: `<=>` , `<->` , `<#>`

#### 9. RAG-System Implementation

- 10. Dokumente in Chunks aufteilen
- 11. Automatische Embedding-Generierung
- 12. Context-Retrieval für LLMs

#### 13. E-Commerce Vector Search

- 14. Ähnliche Produkte finden
- 15. Hybrid Search mit Filtern
- 16. Cross-Category Similarity

#### 17. Best Practices

- 18. Chunking: 200-300 Wörter
- 19. Hybrid Search: Vector + Fulltext
- 20. Re-Ranking für höchste Qualität
- 21. Batch-Processing für Performance

**Key Takeaway:** ThemisDB kombiniert Vector Search mit relationalen, Graph- und Dokument-Features in einer Datenbank – perfekt für moderne Multi-Model Anwendungen.

**Nächster Schritt:** In Teil III (Spezialanwendungen) lernen wir Time-Series, Geo-Spatial und Enterprise-Features kennen.

---

Ende Kapitel 8 | [➡ Weiter zu Kapitel 9: Time-Series](#)

---

# Kapitel 12: Kapitel 9: Time-Series & IoT-Daten

## 9.1 Einführung in Time-Series-Datenbanken

### Was sind Time-Series-Daten?

Time-Series-Daten sind Messungen, die über die Zeit gesammelt werden. Jeder Datenpunkt hat einen Zeitstempel und einen oder mehrere Werte:

```
{
 "timestamp": "2024-01-15T10:30:45.123Z",
 "sensor_id": "temp_sensor_001",
 "temperature": 22.5,
 "humidity": 45.2,
 "location": "warehouse_a"
}
```

**Charakteristiken:** - **Hohe Schreiblast:** Tausende Messungen pro Sekunde - **Zeitbasierte Queries:** "Zeige mir alle Werte der letzten Stunde" - **Aggregationen:** Durchschnitt, Min/Max über Zeitfenster - **Retention Policies:** Alte Daten automatisch löschen oder aggregieren

### Typische Use Cases

Use Case	Datenrate	Retention	Beispiel
IoT Sensoren	1-1000 Hz	30-90 Tage	Temperatur, Luftfeuchtigkeit
Monitoring	10-60 sec	1-12 Monate	CPU, Memory, Disk I/O
Financial Data	Real-time	Jahre	Aktienkurse, Trades
Log Aggregation	Variabel	7-30 Tage	Application Logs, Metrics
Smart Home	1-60 sec	3-6 Monate	Stromverbrauch, Heizung

```
gantt
```

```
 title Time-Series Data Lifecycle
```

```
 dateFormat YYYY-MM-DD
```

```
 section Raw Data
```

```
 High-Frequency Data (1s) :active, raw1, 2024-01-01, 7d
```

```
 Retention 7 days :crit, raw2, 2024-01-08, 1d
```

```
 section 5-Min Aggregates
```

```
 Downsample to 5-min :done, agg5, 2024-01-02, 30d
```

```
 Retention 30 days :agg5r, 2024-02-01, 1d
```

```
 section Hourly Aggregates
```

```
 Downsample to hourly :done, agg60, 2024-01-08, 90d
```

```
 Retention 90 days :agg60r, 2024-04-08, 1d
```

```
 section Daily Aggregates
```

```
 Downsample to daily :done, aggd, 2024-02-01, 365d
```

```
 Retention 1 year+ :aggdr, 2025-02-01, 730d
```

**Abb. 09.1:** Timeseries-Data-Ingestion

## 9.2 Time-Series-Datenmodell in ThemisDB

### Schema-Design für Time-Series

ThemisDB speichert Time-Series als Documents mit optimierten Indizes:

```
// Sensor-Messungen Collection
FOR doc IN [
 {
 _key: GENERATE_ID(),
 sensor_id: "sensor_001",
 timestamp: DATE_ISO8601(DATE_NOW()),
 temperature: 22.5,
 humidity: 65.0,
 co2_ppm: 420,
 location: "Room 101",
 metadata: {building: "A", floor: 2}
 }
] INSERT doc INTO sensor_readings

-- Index für Time-Range Queries
CREATE INDEX idx_sensor_time
 ON sensor_readings (sensor_id, timestamp)

-- Optional: TTL-Index für automatische Daten-Pruning
CREATE INDEX idx_ttl
 ON sensor_readings (timestamp)
 OPTIONS {expireAfterSeconds: 2592000} -- 30 Tage
```

**Design-Prinzipien:** 1. **Composite Index:** `(sensor_id, timestamp)` - optimal für Time-Range-Queries 2. **TTL-Policies:** Automatisches Löschen alter Daten via Expiration-Index 3. **Sharding:** Horizontales Partitionieren nach `sensor_id` für Skalierung



```

graph TB
 subgraph "Time-Series Partitioning Strategy"
 Table[sensor_readings
Master Table]

 Table --> P1[Partition 2024-01
Jan 2024]
 Table --> P2[Partition 2024-02
Feb 2024]
 Table --> P3[Partition 2024-03
Mar 2024]
 Table --> P4[Partition 2024-04
Apr 2024]

 P1 --> Q1{Query:
timestamp >= 2024-02-15}
 Q1 -->|Skip| P1
 Q1 -->|Scan| P2
 Q1 -->|Scan| P3
 Q1 -->|Scan| P4
 end

 style Table fill:#667eea
 style P1 fill:#e0e0e0
 style P2 fill:#4facfe
 style P3 fill:#43e97b
 style P4 fill:#f093fb
 style Q1 fill:#ffd32a

```

**Abb. 09.2:** Aggregation-Pipeline

## Indexes für Time-Series

```

-- Index für Zeit-basierte Queries
CREATE INDEX idx_readings_time ON sensor_readings (timestamp DESC);

-- Index für Sensor-spezifische Queries
CREATE INDEX idx_readings_sensor_time ON sensor_readings (sensor_id, timestamp DESC);

-- Conditional Index für Alerts (nur hohe Temperaturen)
CREATE INDEX idx_high_temp ON sensor_readings (timestamp)
WHERE temperature > 30.0;

```

## 9.3 Effiziente Time-Series Queries

### Time-Range Queries

```
-- Alle Messungen der letzten Stunde
SELECT sensor_id, timestamp, temperature, humidity
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour'
ORDER BY timestamp DESC;

-- Durchschnitt pro Sensor in den letzten 24h
SELECT sensor_id,
 COUNT(*) as measurement_count,
 AVG(temperature) as avg_temp,
 AVG(humidity) as avg_humidity,
 MAX(co2_ppm) as max_co2
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '24 hours'
GROUP BY sensor_id;
```

## Window Functions für Rolling Averages

```
-- Gleitender Durchschnitt (Moving Average) über 10 Messungen
SELECT
 sensor_id,
 timestamp,
 temperature,
 AVG(temperature) OVER (
 PARTITION BY sensor_id
 ORDER BY timestamp
 ROWS BETWEEN 9 PRECEDING AND CURRENT ROW
) as moving_avg_10
FROM sensor_readings
WHERE sensor_id = 'temp_001'
 AND timestamp >= NOW() - INTERVAL '1 day'
ORDER BY timestamp;

-- Differenz zur vorherigen Messung
SELECT
 sensor_id,
 timestamp,
 temperature,
 temperature - LAG(temperature) OVER (
 PARTITION BY sensor_id ORDER BY timestamp
) as temperature_change
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour';
```

## Time-Bucket Aggregationen

```
-- Aggregiere zu 5-Minuten-Intervallen
SELECT
 sensor_id,
 DATE_TRUNC('minute', timestamp) -
 (EXTRACT(minute FROM timestamp)::int % 5) * INTERVAL '1 minute' as time_bucket,
 AVG(temperature) as avg_temp,
 MIN(temperature) as min_temp,
 MAX(temperature) as max_temp,
 STDDEV(temperature) as stddev_temp
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 day'
GROUP BY sensor_id, time_bucket
ORDER BY time_bucket DESC;
```

## 9.4 Down-Sampling und Retention Policies

### Automatisches Down-Sampling

Speichere hochfrequente Rohdaten nur kurzfristig, langfristig nur Aggregate:

```
-- Down-Sampling: Stündliche Aggregate in neue Collection
LET hour_start = DATE_TRUNC(DATE_NOW(), 'hour')
LET hour_data = (
 FOR reading IN sensor_readings
 FILTER reading.timestamp >= DATE_SUBTRACT(hour_start, 1, 'hour')
 AND reading.timestamp < hour_start
 COLLECT sensor = reading.sensor_id INTO readings
 RETURN {
 sensor_id: sensor,
 hour_bucket: hour_start,
 avg_temperature: AVG(readings[*].reading.temperature),
 min_temperature: MIN(readings[*].reading.temperature),
 max_temperature: MAX(readings[*].reading.temperature),
 avg_humidity: AVG(readings[*].reading.humidity),
 max_co2_ppm: MAX(readings[*].reading.co2_ppm),
 sample_count: LENGTH(readings)
 }
)
FOR doc IN hour_data
 UPSERT {sensor_id: doc.sensor_id, hour_bucket: doc.hour_bucket}
 INSERT doc
 UPDATE doc
 INTO sensor_readings_hourly
```

```
max_temperature = EXCLUDED.max_temperature,
sample_count = EXCLUDED.sample_count;
```

```

<figure>

```mermaid
flowchart TD
    Start[Raw Sensor Data<br/>1 sample/second] --> Buffer[Write Buffer<br/>High Frequency]

    Buffer -->|Every 1 min| Raw[(Raw Data Table<br/>Retention: 7 days)]

    Raw -->|Down-Sample Job<br/>Every 5 min| Agg5[(5-Min Aggregates<br/>Retention: 30 days)]

    Agg5 -->|Down-Sample Job<br/>Every 1 hour| Agg60[(Hourly Aggregates<br/>Retention: 90 days)]

    Agg60 -->|Down-Sample Job<br/>Every 1 day| AggD[(Daily Aggregates<br/>Retention: 1+ years)]

    Raw -->|Delete after 7 days| Delete1[Deletion Job]
    Agg5 -->|Delete after 30 days| Delete2[Deletion Job]
    Agg60 -->|Delete after 90 days| Delete3[Deletion Job]

    style Start fill:#667eea
    style Buffer fill:#4facfe
    style Raw fill:#43e97b
    style Agg5 fill:#f093fb
    style Agg60 fill:#fa709a
    style AggD fill:#fee140
    style Delete1 fill:#ff6348
    style Delete2 fill:#ff6348
    style Delete3 fill:#ff6348

```

Abb. 09.3: Downsampling-Strategie

Retention Policy Implementation

```

-- Lösche Rohdaten älter als 30 Tage
FOR reading IN sensor_readings
    FILTER reading.timestamp < DATE_NOW() - INTERVAL('30 days')
    REMOVE reading IN sensor_readings

-- Oder: Drop alte Partitionen (viel schneller!)
DROP TABLE IF EXISTS sensor_readings_2023_01;

```

Best Practice Retention: - **0-7 Tage:** Rohdaten (1 Sekunde Auflösung) - **7-30 Tage:** 1-Minute Aggregate
 - **30-90 Tage:** 5-Minute Aggregate - **90-365 Tage:** 1-Stunde Aggregate - **>1 Jahr:** 1-Tag Aggregate

9.5 Example: IoT Sensor Network

Überblick

Das **IoT Sensor Network** Beispiel ([examples/09_iot_sensor_network](#)) demonstriert ein vollständiges Monitoring-System für industrielle Sensoren:

Features: - Multi-Sensor Monitoring (Temperatur, Luftfeuchtigkeit, CO2) - Real-Time Alerting bei Schwellwert-Überschreitungen - Historische Analyse mit Trend-Erkennung - Dashboard mit Visualisierungen - Complex Event Processing (CEP)

Datenmodell

```
from datetime import datetime
from enum import Enum

class SensorType(Enum):
    TEMPERATURE = "temperature"
    HUMIDITY = "humidity"
    CO2 = "co2"
    PRESSURE = "pressure"

class Sensor:
    def __init__(self, sensor_id: str, sensor_type: SensorType,
                  location: str, unit: str):
        self.sensor_id = sensor_id
        self.sensor_type = sensor_type
        self.location = location
        self.unit = unit
        self.thresholds = {}

class SensorReading:
    def __init__(self, sensor_id: str, value: float,
                  timestamp: datetime = None):
        self.sensor_id = sensor_id
        self.value = value
        self.timestamp = timestamp or datetime.now()
```

Sensor-Registrierung

```
import themis_client as themis

sensors = [
    Sensor("temp_001", SensorType.TEMPERATURE, "warehouse_a", "°C"),
    Sensor("hum_001", SensorType.HUMIDITY, "warehouse_a", "%"),
    Sensor("co2_001", SensorType.CO2, "warehouse_a", "ppm")
]

for sensor in sensors:
    themis.query("""
        UPSERT {sensor_id: @sensor_id}
        INSERT {
            sensor_id: @sensor_id,
            sensor_type: @sensor_type,
            location: @location,
            unit: @unit,
            thresholds: @thresholds
        }
        UPDATE {
            location: @location,
            thresholds: @thresholds
        }
        INTO sensors
    """, {
        'sensor_id': sensor.sensor_id,
        'sensor_type': sensor.sensor_type.value,
        'location': sensor.location,
        'unit': sensor.unit,
        'thresholds': sensor.thresholds
    })
```

Echtzeit-Datenerfassung

```

import random
import time
from datetime import datetime

def simulate_sensors(duration_seconds=60):
    """Simuliere Sensor-Daten für Demo"""
    start_time = time.time()

    while time.time() - start_time < duration_seconds:
        readings = []

        # Generiere realistische Werte
        temp = 20 + random.gauss(2, 0.5) # ~20°C ± 2°C
        humidity = 45 + random.gauss(5, 2) # ~45% ± 5%
        co2 = 400 + random.gauss(50, 10) # ~400 ppm ± 50

        readings.append(SensorReading("temp_001", temp))
        readings.append(SensorReading("hum_001", humidity))
        readings.append(SensorReading("co2_001", co2))

        # Batch-Insert mit AQL
        themis.query("""
            FOR reading IN @readings
            INSERT {
                sensor_id: reading.sensor_id,
                timestamp: reading.timestamp,
                value: reading.value
            } INTO sensor_readings
        """, {'readings': [{'sensor_id': r.sensor_id,
                               'timestamp': r.timestamp.isoformat(),
                               'value': r.value} for r in readings]})

        time.sleep(1) # 1 Hz Sampling-Rate

simulate_sensors(duration_seconds=3600) # 1 Stunde

```


Alert-System

```
def check_alerts():
    """Prüfe aktuelle Werte gegen Schwellwerte"""
    alerts = themis.query("""
        WITH latest_readings AS (
            SELECT DISTINCT ON (sensor_id)
                sensor_id, value, timestamp
            FROM sensor_readings
            ORDER BY sensor_id, timestamp DESC
        )
        SELECT
            s.sensor_id,
            s.sensor_type,
            s.location,
            lr.value,
            s.thresholds->>'max' as max_threshold,
            s.thresholds->>'min' as min_threshold
        FROM sensors s
        JOIN latest_readings lr ON s.sensor_id = lr.sensor_id
        WHERE lr.value > (s.thresholds->>'max')::float
            OR lr.value < (s.thresholds->>'min')::float
    """)

    for alert in alerts:
        send_alert(
            sensor_id=alert['sensor_id'],
            location=alert['location'],
            value=alert['value'],
            threshold_breached=True
        )
```

Historische Analyse

```
def analyze_trends(sensor_id: str, hours: int = 24):
    """Analysiere Trends der letzten N Stunden"""
    analysis = themis.query("""
        SELECT
            DATE_TRUNC('hour', timestamp) as hour,
            AVG(value) as avg_value,
            MIN(value) as min_value,
            MAX(value) as max_value,
            STDDEV(value) as stddev_value,
            -- Linearer Trend (Regression)
            REGR_SLOPE(value, EXTRACT(EPOCH FROM timestamp)) * 3600 as hourly_trend
        FROM sensor_readings
        WHERE sensor_id = ?
            AND timestamp >= NOW() - INTERVAL '? hours'
        GROUP BY hour
        ORDER BY hour
    """, (sensor_id, hours))

    return analysis

temp_trend = analyze_trends("temp_001", hours=24)
print(f"Hourly trend: {temp_trend[-1]['hourly_trend']:.3f}°C/hour")
```

Dashboard-Visualisierung

```
import matplotlib.pyplot as plt
import pandas as pd

def plot_sensor_dashboard(sensor_id: str, hours: int = 24):
    """Erstelle Dashboard für einen Sensor"""
    # Lade Daten
    data = themis.query("""
        SELECT timestamp, value
        FROM sensor_readings
        WHERE sensor_id = ?
            AND timestamp >= NOW() - INTERVAL '? hours'
        ORDER BY timestamp
    """, (sensor_id, hours))

    df = pd.DataFrame(data)

    # Plot erstellen
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    # Zeitreihe
    ax1.plot(df['timestamp'], df['value'], linewidth=0.5)
    ax1.set_title(f'Sensor {sensor_id} - Last {hours}h')
    ax1.set_ylabel('Value')
    ax1.grid(True, alpha=0.3)

    # Histogram
    ax2.hist(df['value'], bins=50, alpha=0.7)
    ax2.set_xlabel('Value')
    ax2.set_ylabel('Frequency')
    ax2.axvline(df['value'].mean(), color='red',
                linestyle='--', label='Mean')
    ax2.legend()

    plt.tight_layout()
    plt.savefig(f'dashboard_{sensor_id}.png', dpi=150)
    plt.show()

plot_sensor_dashboard("temp_001")
```

9.6 Example: Smart Home Energy Monitoring

Überblick

Das **Smart Home** Beispiel ([examples/20_smart_home](#)) zeigt Energy Monitoring und Automation:

Features: - Stromverbrauch-Tracking pro Gerät - Automatisierungsregeln (z.B. Heizung runter wenn keiner da ist) - Cost-Calculation basierend auf Strompreisen - Energy-Saving-Recommendations

Geräte-Management

```
class Device:
    def __init__(self, device_id: str, device_type: str,
                  room: str, power_rating_watts: int):
        self.device_id = device_id
        self.device_type = device_type # "heater", "light", "appliance"
        self.room = room
        self.power_rating_watts = power_rating_watts
        self.state = "off"

devices = [
    Device("heater_living", "heater", "living_room", 2000),
    Device("light_living", "light", "living_room", 60),
    Device("fridge_kitchen", "appliance", "kitchen", 150),
]

for dev in devices:
    themis.execute("""
        INSERT INTO devices (device_id, device_type, room, power_rating_watts)
        VALUES (?, ?, ?, ?)
        """, (dev.device_id, dev.device_type, dev.room, dev.power_rating_watts))
```

Energy-Tracking

```
def track_energy_consumption():
    """Track energy consumption per device"""
    # Record current state and power usage
    for device in devices:
        if device.state == "on":
            power_usage = device.power_rating_watts
        else:
            power_usage = 0

        themis.execute("""
            INSERT INTO energy_readings
            (device_id, timestamp, power_watts, state)
            VALUES (?, NOW(), ?, ?)
            """, (device.device_id, power_usage, device.state))

    while True:
        track_energy_consumption()
        time.sleep(60)
```

Cost Calculation

```
def calculate_daily_cost(date):
    """Berechne Energiekosten für einen Tag"""
    cost_per_kwh = 0.30 # 30 Cent/kWh

    daily_usage = themis.query("""
        SELECT
            device_id,
            d.device_type,
            d.room,
            -- Energie in kWh (Durchschnitt * Zeit / 1000)
            SUM(power_watts * 60) / 1000.0 / 60.0 as kwh_consumed,
            -- Kosten
            SUM(power_watts * 60) / 1000.0 / 60.0 * ? as cost_euros
        FROM energy_readings er
        JOIN devices d ON er.device_id = d.device_id
        WHERE DATE(timestamp) = ?
        GROUP BY device_id, d.device_type, d.room
        ORDER BY cost_euros DESC
    """, (cost_per_kwh, date))

    total_cost = sum(row['cost_euros'] for row in daily_usage)

    print(f"Daily Energy Cost: €{total_cost:.2f}")
    for row in daily_usage:
        print(f"  {row['device_id']}: {row['kwh_consumed']:.2f} kWh → €{row['cost_euros']:.2f}")

    return daily_usage

calculate_daily_cost('2024-01-15')
```

Automation Rules

```

class AutomationRule:
    def __init__(self, rule_id: str, condition: str, action: str):
        self.rule_id = rule_id
        self.condition = condition
        self.action = action

rules = [
    AutomationRule("rule_001",
        "temperature < 18 AND presence = false",
        "set_heater_temp(15)"),
    AutomationRule("rule_002",
        "time > 22:00 AND presence = false",
        "turn_off_all_lights()"),
]

def evaluate_rules():
    """Evaluiere und führe Automation-Rules aus"""
    for rule in rules:
        # Check condition
        condition_met = themis.query_one(f"""
            LET latest = (
                FOR state IN device_states
                SORT state.timestamp DESC
                LIMIT 1
                RETURN state
            )[0]
            RETURN {{rule.condition}} AS met
        """)

        if condition_met['met']:
            exec(rule.action) # Führe Aktion aus
            print(f"Rule {rule.rule_id} triggered: {rule.action}")

```

9.7 Performance-Optimierungen

Batch-Inserts für hohen Durchsatz

```
for reading in readings:
    themis.execute("INSERT @reading INTO sensor_readings", {"reading": reading})

themis.execute_batch(
    "INSERT @reading INTO sensor_readings",
    [{"reading": r} for r in readings],
    batch_size=1000
)
```

Partitionierung für schnelle Queries

```
-- Automatisches Partition-Management
CREATE OR REPLACE FUNCTION create_monthly_partition()
RETURNS void AS $$
DECLARE
    start_date DATE;
    end_date DATE;
    partition_name TEXT;
BEGIN
    start_date := DATE_TRUNC('month', NOW());
    end_date := start_date + INTERVAL '1 month';
    partition_name := 'sensor_readings_' || TO_CHAR(start_date, 'YYYY_MM');

    EXECUTE format('
        CREATE TABLE IF NOT EXISTS %I PARTITION OF sensor_readings
        FOR VALUES FROM (%L) TO (%L)
        ', partition_name, start_date, end_date);
END;
$$ LANGUAGE plpgsql;
```


Materialized Views für Dashboards

```
-- Pre-Aggregiere Daten für schnelle Dashboard-Queries
CREATE MATERIALIZED VIEW sensor_daily_stats AS
SELECT
    sensor_id,
    DATE(timestamp) as date,
    AVG(value) as avg_value,
    MIN(value) as min_value,
    MAX(value) as max_value,
    COUNT(*) as reading_count
FROM sensor_readings
GROUP BY sensor_id, DATE(timestamp);

-- Refresh täglich
CREATE INDEX ON sensor_daily_stats (sensor_id, date DESC);
REFRESH MATERIALIZED VIEW CONCURRENTLY sensor_daily_stats;
```

9.8 Best Practices

1. Schema-Design

✓ **DO:** - Nutze Partitionierung für große Tabellen - Composite Primary Key: `(entity_id, timestamp)` - Index auf `timestamp DESC` für neueste Daten

✗ **DON'T:** - Nicht UUID als Primary Key bei Time-Series - Keine UNIQUENESS Constraints außer Primary Key - Vermeide komplexe JOINS in High-Frequency Queries

2. Query-Optimierung

✓ **DO:** - Nutze TIME-RANGE in WHERE-Clause für Partition Pruning - Batch-Inserts statt einzelne INSERTs - Materialized Views für häufige Aggregationen

✗ **DON'T:** - Keine `SELECT *` - nur benötigte Spalten - Vermeide DISTINCT ohne Index - Keine ungebundenen Queries ohne Zeit-Limit

3. Retention & Storage

✓ **DO:** - Implementiere Retention Policies - Down-Sample alte Daten - Nutze automatisches Partition-Management

✗ **DON'T:** - Behalte nicht alle Rohdaten für immer - Lösche nicht mit DELETE (nutze DROP PARTITION) - Vergiss nicht Backups vor Partition-Drops

9.9 Vergleich: ThemisDB vs. Specialized Time-Series DBs

Feature	ThemisDB	InfluxDB	TimescaleDB
Data Model	Relational + Multi-Model	Line Protocol	Relational
Query Language	SQL/AQL	InfluxQL, Flux	SQL
Partitioning	✓ Native	✓ Automatic	✓ Hypertables
Continuous Aggregates	✓ Mat. Views	✓ Native	✓ Continuous Aggregates
Retention Policies	✓ Manual/Triggers	✓ Automatic	✓ Automatic
Downsampling	✓ AQL-based	✓ Built-in	✓ Continuous Aggregates
Multi-Model	✓ Graph, Vector, Doc	✗	✗
Horizontal Scaling	✓ Sharding	✓ Clustering	✓ Distributed

Wann ThemisDB wählen: - ✓ Wenn Time-Series nur ein Teil der Anwendung ist - ✓ Wenn komplexe Relationen zu anderen Daten bestehen - ✓ Wenn Multi-Model Features (Graph, Vector) benötigt werden - ✓ Wenn Standard-AQL Queries ausreichen

Wann Specialized DB wählen: - InfluxDB: Sehr hohe Schreibraten (>1M writes/sec), native Telegraf-Integration - TimescaleDB: Wenn PostgreSQL-Kompatibilität Priorität hat

9.10 Zusammenfassung

ThemisDB ist bestens geeignet für Time-Series & IoT-Anwendungen durch:

✓ **Native Unterstützung:** - Partitionierung für Milliarden von Datenpunkten - Effiziente Time-Range Queries mit Partition Pruning - Window Functions für Rolling Averages und Trend-Analyse

✓ **Performance:** - Batch-Inserts für hohen Schreibdurchsatz - Materialized Views für schnelle Dashboards - Indexing-Strategien für typische Time-Series Patterns

✓ **Flexibilität:** - Kombiniere Time-Series mit relationalen Daten - Nutze Graph-Queries für Sensor-Topologien - Vector-Search für Pattern-Matching in Zeitreihen

Key Takeaways: 1. Nutze Partitionierung + richtige Indexes 2. Implementiere Retention Policies frühzeitig 3. Batch-Inserts für Performance 4. Materialized Views für Dashboards 5. Down-Sample alte Daten automatisch

Kapitel 13: Kapitel 10: Enterprise-Anwendungen

Einleitung

Enterprise-Anwendungen sind das Rückgrat moderner Unternehmen. Sie verwalten kritische Geschäftsprozesse, von der Kundenverwaltung über Dokumentenmanagement bis hin zu ERP-Systemen. In diesem Kapitel lernen Sie, wie ThemisDB alle Anforderungen enterprise-grade Software erfüllt:

- **Multi-Tenancy** für Mandantenfähigkeit
- **RBAC** (Role-Based Access Control) für fein-granulare Zugriffsrechte
- **Audit-Logging** für vollständige Nachvollziehbarkeit
- **Workflow-Management** für Geschäftsprozesse
- **Skalierbarkeit** für große Datenmengen
- **Security** auf allen Ebenen

Wir demonstrieren dies anhand zweier vollständiger Beispiele: 1. **DMS/ERP-System:** Dokumentenmanagement mit Workflows 2. **CRM-System:** Customer Relationship Management

10.1 Enterprise-Anforderungen

Mandantenfähigkeit (Multi-Tenancy)

In SaaS-Anwendungen müssen Daten verschiedener Kunden (Mandanten/Tenants) strikt getrennt sein.

Drei Ansätze:

```
db_tenant_1 = ThemisDB("tenant_1.db")
db_tenant_2 = ThemisDB("tenant_2.db")

db.execute("""
    CREATE TABLE documents (
        id UUID PRIMARY KEY,
        tenant_id UUID NOT NULL,
        title TEXT,
        content TEXT
    )
""")
db.execute("CREATE INDEX idx_tenant ON documents(tenant_id)")

db.execute("""
    CREATE POLICY tenant_isolation ON documents
    USING (tenant_id = current_tenant_id())
""")
```

```

graph TB
    subgraph "Multi-Tenancy Strategies"
        subgraph "Strategy 1: Database per Tenant"
            T1DB["Tenant 1<br/>Dedicated DB"]
            T2DB["Tenant 2<br/>Dedicated DB"]
            T3DB["Tenant 3<br/>Dedicated DB"]
        end

        subgraph "Strategy 2: Shared DB with Tenant ID"
            SharedDB["Shared Database"]
            SharedDB --> Filter["tenant_id filter"]
            Filter --> T1Data["Tenant 1 Data"]
            Filter --> T2Data["Tenant 2 Data"]
            Filter --> T3Data["Tenant 3 Data"]
        end

        subgraph "Strategy 3: Row-Level Security"
            RLSDB["Database with RLS"]
            RLSDB --> Policy["Security Policies<br/>Automatic filtering"]
            Policy --> Sec1["Tenant 1 View"]
            Policy --> Sec2["Tenant 2 View"]
            Policy --> Sec3["Tenant 3 View"]
        end
    end

    Iso1["✓ Max Isolation<br/>✗ High overhead"] -.-> T1DB
    Iso2["✓ Balance<br/>✓ Efficient"] -.-> SharedDB
    Iso3["✓ Automatic<br/>✓ Secure"] -.-> RLSDB

    style T1DB fill:#667eea
    style T2DB fill:#667eea
    style T3DB fill:#667eea
    style SharedDB fill:#43e97b
    style RLSDB fill:#f093fb

```

Abb. 10.1: Enterprise-Architektur-Übersicht**Best Practice in ThemisDB:**

```

class TenantContext:
    def __init__(self, db, tenant_id):
        self.db = db
        self.tenant_id = tenant_id

    def query(self, sql, params=None):
        # Automatisches Anhängen der Tenant-Bedingung
        if "WHERE" in sql.upper():
            sql = sql.replace("WHERE", f"WHERE tenant_id = '{self.tenant_id}' AND")
        else:
            sql += f" WHERE tenant_id = '{self.tenant_id}'"
        return self.db.execute(sql, params)

tenant = TenantContext(db, "acme_corp")
docs = tenant.query("""
    FOR doc IN documents
        FILTER doc.type == @type
    RETURN doc
""", {"type": "invoice"})

```

Rollen-basierte Zugriffskontrolle (RBAC)

Datenmodell:

```

db.execute("""
    CREATE TABLE roles (
        id UUID PRIMARY KEY,
        name TEXT UNIQUE NOT NULL,
        permissions TEXT[], -- ["read:documents", "write:documents"]
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE user_roles (
        user_id UUID,
        role_id UUID,
        granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (user_id, role_id)
    )
""")

def has_permission(user_id, permission):
    result = db.execute("""
        FOR user_role IN user_roles
            FILTER user_role.user_id == @user_id
            FOR role IN roles
                FILTER user_role.role_id == role.id AND @permission IN role.permissions
            LIMIT 1
        RETURN 1
    """, {"user_id": user_id, "permission": permission})
    return len(result) > 0

```

```

graph TB
    subgraph "RBAC - Role Based Access Control"
        U1[User: Alice<br/>employee_id: 001]
        U2[User: Bob<br/>employee_id: 002]
        U3[User: Carol<br/>employee_id: 003]

        R1[Role: Admin<br/>All Permissions]
        R2[Role: Editor<br/>read, write, edit]
        R3[Role: Viewer<br/>read only]

        U1 -->|assigned| R1
        U2 -->|assigned| R2
        U3 -->|assigned| R3

        R1 -->|grants| P1[read:*<br/>write:*<br/>delete:*<br/>admin:*]
        R2 -->|grants| P2[read:documents<br/>write:documents<br/>edit:documents]
        R3 -->|grants| P3[read:documents]

        P1 --> Resource[(Protected Resources)]
        P2 --> Resource
        P3 --> Resource
    end

    style U1 fill:#667eea
    style U2 fill:#4facfe
    style U3 fill:#43e97b
    style R1 fill:#ff6348
    style R2 fill:#ff6348
    style R3 fill:#95e1d3
    style Resource fill:#f093fb

```

Abb. 10.2: RBAC-Hierarchie

```

def requires_permission(permission):
    def decorator(func):
        def wrapper(args, kwargs):
            user_id = get_current_user_id()
            if not has_permission(user_id, permission):
                raise PermissionDeniedError()
            return func(args, **kwargs)
        return wrapper
    return decorator

```

```

@requires_permission("write:documents")
def create_document(title, content):
    # Nur erreichbar mit korrekter Permission pass

```


Audit-Logging

Jede Änderung muss nachvollziehbar sein:

```

```python
db.execute("""
 CREATE TABLE audit_log (
 id UUID PRIMARY KEY,
 timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 user_id UUID NOT NULL,
 action TEXT NOT NULL, -- CREATE, UPDATE, DELETE, READ
 resource_type TEXT NOT NULL, -- "document", "user", etc.
 resource_id UUID NOT NULL,
 old_value JSON,
 new_value JSON,
 ip_address TEXT,
 user_agent TEXT
)
""")

db.execute("CREATE INDEX idx_audit_resource ON audit_log(resource_type, resource_id)")
db.execute("CREATE INDEX idx_audit_user ON audit_log(user_id, timestamp)")

def log_action(action, resource_type, resource_id, old_value=None, new_value=None):
 db.execute("""
 INSERT {
 id: @id,
 user_id: @user_id,
 action: @action,
 resource_type: @resource_type,
 resource_id: @resource_id,
 old_value: @old_value,
 new_value: @new_value,
 ip_address: @ip_address
 } INTO audit_log
 """, {
 "id": uuid.uuid4(),
 "user_id": get_current_user_id(),
 "action": action,
 "resource_type": resource_type,
 resource_id,
 json.dumps(old_value) if old_value else None,
 json.dumps(new_value) if new_value else None,
 get_client_ip()
 })

old_doc = db.execute("""
 FOR doc IN documents
 FILTER doc.id == @doc_id

```

```

 LIMIT 1
 RETURN doc
 """, {"doc_id": doc_id})[0]

db.execute("""
 FOR doc IN documents
 FILTER doc.id == @doc_id
 UPDATE doc WITH {title: @new_title} IN documents
 """, {"doc_id": doc_id, "new_title": new_title})

new_doc = db.execute("""
 FOR doc IN documents
 FILTER doc.id == @doc_id
 LIMIT 1
 RETURN doc
 """, {"doc_id": doc_id})[0]

log_action("UPDATE", "document", doc_id, old_doc, new_doc)

```

### Audit-Abfragen:

```

changes = db.execute("""
 FOR log IN audit_log
 FILTER log.resource_type == 'document' AND log.resource_id == @doc_id
 SORT log.timestamp DESC
 RETURN log
 """, {"doc_id": doc_id})

activity = db.execute("""
 FOR log IN audit_log
 FILTER log.user_id == @user_id AND log.timestamp > DATE_NOW() - INTERVAL('7 days')
 SORT log.timestamp DESC
 RETURN log
 """, {"user_id": user_id})

```

## 10.2 Native Security-Stack: Production-Ready Sicherheit

ThemisDB v1.3.0 bietet einen vollständig nativen, in C++ implementierten Security-Stack, der BSI C5-konform ist und keine externen Abhängigkeiten (wie Apache Ranger) für die Kern-Sicherheitsfunktionen benötigt.

### 10.2.1 Implementierungsstatus (Dezember 2025)

Komponente	Status	Implementierung	Beschreibung
<b>RBAC/ABAC Policy Engine</b>	✓ Produktionsreif	<code>src/security/rbac.cpp</code>	Role-Based & Attribute-Based Access Control
<b>Apache Ranger Integration</b>	✓ Produktionsreif	<code>src/server/ranger_adapter.cpp</code>	Optional: Enterprise Policy Management
<b>Field-Level Encryption</b>	✓ Produktionsreif	<code>src/security/encryption.cpp</code>	AES-256-GCM, transparent
<b>Column Encryption</b>	✓ Produktionsreif	BSI C5 konform	Spalten-basierte Verschlüsselung
<b>Vector Encryption</b>	✓ Produktionsreif	Phase 1+2 komplett	HNSW-Index + RocksDB Vektoren
<b>PKI Integration</b>	✓ Produktionsreif	<code>src/security/pki_key_provider.cpp</code>	X.509 Zertifikat-basiert
<b>HSM Support</b>	✓ Produktionsreif	<code>src/security/hsm_provider_pkcs11.cpp</code>	PKCS#11 Hardware Security Modules
<b>Audit Logging</b>	✓ Produktionsreif	<code>src/utils/audit_logger.cpp</code>	Tamper-Proof mit Hash-Chains
<b>Malware Scanner</b>	✓ Produktionsreif	<code>src/security/malware_scanner.cpp</code>	Content Security
<b>CMS Signing</b>	✓ Produktionsreif	<code>src/security/cms_signing.cpp</code>	Cryptographic Message Syntax
<b>RFC 3161 TSA</b>	✓ Produktionsreif	<code>src/security/timestamp_authority.cpp</code>	Qualified Timestamps

**Gesamt:** 16 Header-Dateien, 16 Source-Dateien, ~8.100 LOC

### 10.2.2 RBAC-Implementierung im C++ Core

ThemisDB implementiert RBAC direkt im C++ Kern, nicht als Add-on-Layer:

**Architektur:**

```
// Native RBAC Integration (src/security/rbac.h)
class RBACManager {
public:
 // Role Management
 Status createRole(const std::string& roleName,
 const std::vector<std::string>& permissions);
 Status assignRole(const std::string& userId,
 const std::string& roleName);

 // Permission Check (optimiert für Performance)
 bool hasPermission(const std::string& userId,
 const std::string& resource,
 const std::string& action) const;

 // Attribute-Based (ABAC) Extension
 bool checkPolicy(const PolicyContext& ctx) const;
};
```

**Performance-Optimierung:** - Permission-Checks gecacht im RAM (TBB concurrent\_hash\_map) - Sub-Millisekunden Latenz für Permission-Checks - Keine Netzwerk-Latenz (im Gegensatz zu externen Policy-Servern)

#### Verwendung in AQL:

```
// Automatischer Permission-Check bei Query-Ausführung
FOR doc IN documents
 // RBAC-Filter wird automatisch eingefügt basierend auf User-Kontext
 FILTER HAS_PERMISSION(CURRENT_USER(), doc._id, 'read')
 RETURN doc
```

### 10.2.3 Tamper-Proof Audit Logging mit Hash-Chains

Das Audit-System von ThemisDB verhindert nachträgliche Manipulation durch **kryptografische Hash-Chains**:

#### Funktionsweise:

```
// src/utils/audit_logger.cpp
class AuditLogger {
 struct AuditEntry {
 uint64_t sequence_number;
 int64_t timestamp_ms;
 std::string user_id;
 std::string action;
 std::string resource_id;
 std::string details;
 std::string previous_hash; // Hash des vorherigen Eintrags
 std::string current_hash; // SHA-256 über alle Felder
 };

 // Verkettung erzwingt chronologische Integrität
 std::string computeHash(const AuditEntry& entry) {
 std::string data = std::to_string(entry.sequence_number) +
 std::to_string(entry.timestamp_ms) +
 entry.user_id + entry.action +
 entry.resource_id + entry.details +
 entry.previous_hash;
 return SHA256(data); // OpenSSL
 }
};
```

**Garantien:** -  **Unveränderbarkeit:** Jede Änderung bricht die Hash-Chain -  **Lückenlosigkeit:** Fehlende Einträge sind sofort erkennbar -  **Zeitstempel-Authentizität:** Integration mit RFC 3161 Timestamp Authority -  **Revisionssicher:** BSI-konform, DSGVO Art. 32

**Verifikation:**

```
// Audit-Log-Integrität prüfen
FOR entry IN audit_log
 SORT entry.sequence_number ASC
 LET expected_hash = SHA256(
 CONCAT(
 entry.sequence_number,
 entry.timestamp_ms,
 entry.user_id,
 entry.action,
 entry.resource_id,
 entry.details,
 entry.previous_hash
)
)
 FILTER entry.current_hash != expected_hash
 RETURN {
 error: "Audit log integrity violation",
 sequence: entry.sequence_number
 }
```

### 10.2.4 HashiCorp Vault Integration

Für Enterprise-Key-Management integriert ThemisDB mit HashiCorp Vault:

#### Konfiguration:

```
// src/security/vault_key_provider.cpp
VaultKeyProvider::Config vault_config;
vault_config.vault_addr = "https://vault.company.com:8200";
vault_config.token = std::getenv("VAULT_TOKEN");
vault_config.mount_path = "themisdb";
vault_config.key_name = "database-master-key";

auto key_provider = std::make_shared<VaultKeyProvider>(vault_config);
FieldEncryption encryption(key_provider);
```

**Features:** -  Automatische Key-Rotation -  Audit Trail für Key-Access -  High Availability (Vault Cluster) -  Disaster Recovery (Sealed/Unsealed State)

**Alternative Key Provider:** - **MockKeyProvider** : Für Testing/Development - **PKIKeyProvider** : Zertifikat-basiert für PKI-Infrastrukturen - **HSMPProvider** : PKCS#11 für Hardware Security Modules (Luna, Thales)

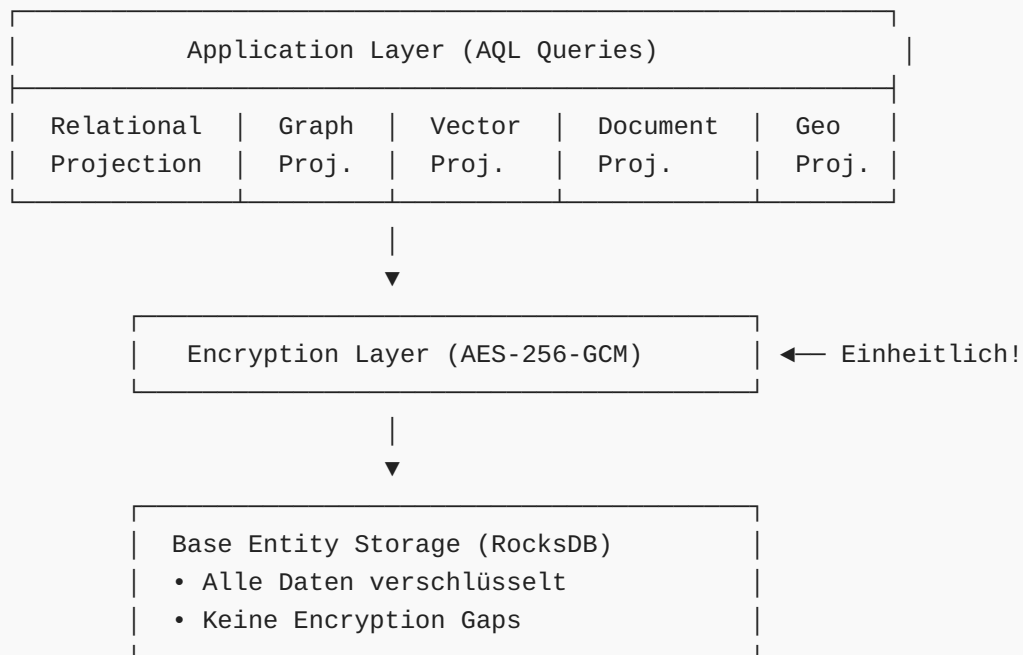
## 10.2.5 BSI C5 Compliance: Kryptographie

ThemisDB erreicht **100% BSI C5 Compliance** für Kryptographie-Anforderungen (CRY-01 bis CRY-06):

Anforderung	Umsetzung	Dokument
<b>CRY-01:</b> Kryptographie-Policy	✓ Formale Policy dokumentiert	<code>CRYPTOGRAPHY_POLICY.md</code>
<b>CRY-02:</b> Key Lifecycle	✓ Vollständiger Lebenszyklus	<code>KEY_LIFECYCLE_MANAGEMENT.md</code>
<b>CRY-03:</b> At-Rest Encryption	✓ AES-256-GCM für alle Daten	Column + Vector Encryption
<b>CRY-04:</b> In-Transit Encryption	✓ TLS 1.3 mandatory	Wire Protocol + HTTP/2
<b>CRY-05:</b> Key Storage	✓ HSM/Vault Integration	<code>VaultKeyProvider</code> + <code>HSMProvider</code>
<b>CRY-06:</b> Algorithm Strength	✓ BSI TR-02102-1 konform	AES-256, RSA-2048+, SHA-256

### Besonderheit: Multi-Model Encryption Consistency

ThemisDB's Unified Storage Architecture garantiert konsistente Verschlüsselung über alle Datenmodelle:



**Vorteil:** Im Gegensatz zu Polyglot-Systemen, wo jede Datenbank eigene Encryption benötigt, ist in ThemisDB alles einheitlich verschlüsselt – kein Modell kann "vergessen" werden.

### 10.2.6 DSGVO "By Design" Features

ThemisDB implementiert DSGVO-Anforderungen technisch:

#### 1. Auto-Purge nach Retention-Period (Art. 17 - Recht auf Löschung):

```
// src/utils/retention_manager.cpp
RetentionManager::Config retention;
retention.enable_auto_purge = true;
retention.default_retention_days = 2555; // 7 Jahre (§ 147 AO)
retention.purge_schedule_cron = "0 2 * * 0"; // Sonntags 2 Uhr

// Anwendung auf Collection
db.execute("""
 CREATE COLLECTION customer_data WITH {
 retention_days: 2555,
 auto_purge: true
 }
 """);
```

#### 2. PII Detection und Redaction (Art. 32 - Datensicherheit):



```
// src/utils/pii_detector.cpp
PIIDetector detector;
detector.addPattern("email", R"([\w\.-]+@[\w\.-]+\.\w+)");
detector.addPattern("iban", R"(DE\d{20})");
detector.addPattern("ssn", R"(\d{3}-\d{2}-\d{4})");

// Automatische Redaction bei Logging
std::string safe_text = detector.redact(user_input);
// Output: "Contact: ***@***.com, IBAN: DE*****"
```

### 3. Encrypt-then-Sign für Log-Integrität:

```
// src/security/cms_signing.cpp
CMSSigner signer(private_key, certificate);

// Audit-Log Entry signieren
std::string signed_entry = signer.sign(
 audit_entry_json,
 true // include_timestamp (RFC 3161)
);

// Später: Verifizierung
bool valid = signer.verify(signed_entry, public_cert);
```

### 4. Granulare Retention Policies:

```
-- Unterschiedliche Aufbewahrungsfristen pro Datentyp
CREATE COLLECTION invoices WITH { retention_days: 3650 }; -- 10 Jahre
CREATE COLLECTION marketing_consent WITH { retention_days: 730 }; -- 2 Jahre
CREATE COLLECTION session_logs WITH { retention_days: 90 }; -- 90 Tage
```

### 10.2.7 Code-Beispiel: Vollständige Enterprise-Security

```

from themisdb import Client, SecurityContext

client = Client("localhost", 8765)

auth_token = client.authenticate(
 cert_file="user.crt",
 key_file="user.key"
)

sec_ctx = SecurityContext(
 user_id="alice@company.com",
 roles=["data_analyst", "auditor"],
 tenant_id="acme_corp"
)

with client.transaction(sec_ctx) as tx:
 # Permission-Check + Audit-Log-Eintrag automatisch
 results = tx.query("""
 FOR doc IN sensitive_documents
 FILTER doc.classification <= @max_clearance
 RETURN doc
 """, {"max_clearance": sec_ctx.get_clearance_level()})

 # Alle Aktionen werden geloggt:
 # - Wer (alice@company.com)
 # - Was (SELECT sensitive_documents)
 # - Wann (2025-12-30T15:00:00Z)
 # - Ergebnis (5 rows returned)
 # - Hash (SHA-256 chain)

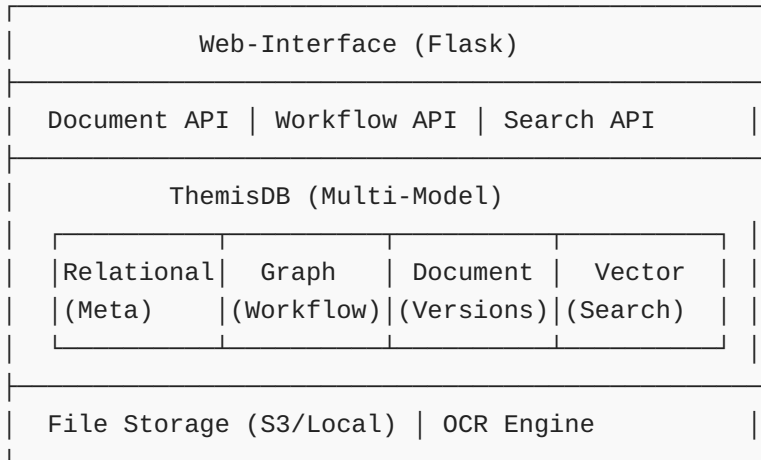
audit_report = client.get_audit_report(
 start_date="2025-01-01",
 end_date="2025-12-31",
 user_id="alice@company.com"
)

```

## 10.3 DMS/ERP-System (Example 08)

Ein vollständiges Document Management System mit ERP-Features.

## Architektur-Übersicht



## Datenmodell

### Dokumente:

```

class Document:
 def __init__(self, db):
 self.db = db
 self.init_schema()

 def init_schema(self):
 # Haupt-Dokument (Relational)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS documents (
 id UUID PRIMARY KEY,
 tenant_id UUID NOT NULL,
 title TEXT NOT NULL,
 type TEXT NOT NULL, -- invoice, contract, hr_document
 version INT DEFAULT 1,
 current_version_id UUID,
 owner_id UUID NOT NULL,
 workflow_state TEXT DEFAULT 'draft',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
 """)

 # Metadaten (Document Model - JSON)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS document_metadata (
 document_id UUID PRIMARY KEY,
 metadata JSON NOT NULL,
 tags TEXT[],
 FOREIGN KEY (document_id) REFERENCES documents(id)
)
 """)

 # Versionen (Dokument Model für Historie)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS document_versions (
 id UUID PRIMARY KEY,
 document_id UUID NOT NULL,
 version INT NOT NULL,
 file_path TEXT NOT NULL,
 file_size BIGINT,
 checksum TEXT,
 created_by UUID NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 change_note TEXT,
 FOREIGN KEY (document_id) REFERENCES documents(id)
)
 """)

 # Permissions (Relational)

```

```

self.db.execute("""
 CREATE TABLE IF NOT EXISTS document_permissions (
 document_id UUID,
 user_id UUID,
 permission TEXT CHECK (permission IN ('read', 'write', 'admin')),
 PRIMARY KEY (document_id, user_id),
 FOREIGN KEY (document_id) REFERENCES documents(id)
)
""")

Volltext-Extraktion
self.db.execute("""
 CREATE TABLE IF NOT EXISTS document_text (
 document_id UUID PRIMARY KEY,
 content TEXT,
 ocr_content TEXT, -- OCR-erkannter Text
 FOREIGN KEY (document_id) REFERENCES documents(id)
)
""")

Fulltext Index
self.db.execute("CREATE INDEX idx_doc_fulltext ON document_text USING GIN(to_tsvector(

Vector Embeddings (für Ähnlichkeitssuche)
self.db.execute("""
 CREATE TABLE IF NOT EXISTS document_embeddings (
 document_id UUID PRIMARY KEY,
 embedding VECTOR(384), -- sentence-transformers
 FOREIGN KEY (document_id) REFERENCES documents(id)
)
""")

self.db.execute("CREATE INDEX idx_doc_vector ON document_embeddings USING HNSW(embed

def create_document(self, title, doc_type, file_path, metadata=None, owner_id=None):
 """Neues Dokument erstellen"""
 doc_id = uuid.uuid4()
 version_id = uuid.uuid4()

 with self.db.transaction():
 # 1. Haupt-Eintrag
 self.db.execute("""
 INSERT {
 id: @doc_id,
 tenant_id: @tenant_id,
 title: @title,
 type: @doc_type,
 current_version_id: @version_id,
 owner_id: @owner_id
 } INTO documents
 """, {
 "doc_id": doc_id,

```

```

 "tenant_id": get_current_tenant_id(),
 "title": title,
 "doc_type": doc_type,
 "version_id": version_id,
 "owner_id": owner_id or get_current_user_id()
 })

2. Erste Version
file_size = os.path.getsize(file_path)
checksum = calculate_checksum(file_path)
self.db.execute("""
 INSERT {
 id: @version_id,
 document_id: @doc_id,
 version: 1,
 file_path: @file_path,
 file_size: @file_size,
 checksum: @checksum,
 created_by: @created_by
 } INTO document_versions
""", {
 "version_id": version_id,
 "doc_id": doc_id,
 "file_path": file_path,
 "file_size": file_size,
 "checksum": checksum,
 "created_by": get_current_user_id()
})

3. Metadaten
if metadata:
 self.db.execute("""
 INSERT {
 document_id: @doc_id,
 metadata: @metadata,
 tags: @tags
 } INTO document_metadata
""", {
 "doc_id": doc_id,
 "metadata": json.dumps(metadata),
 "tags": metadata.get('tags', [])
})

4. Text-Extraktion (asynchron in der Praxis)
text = extract_text(file_path) # PDF, DOCX, etc.
self.db.execute("""
 INSERT {
 document_id: @doc_id,
 content: @content
 } INTO document_text

```

```

 """", {"doc_id": doc_id, "content": text})

5. OCR für Bilder (wenn nötig)
if doc_type in ('scan', 'image'):
 ocr_text = perform_ocr(file_path)
 self.db.execute("""
 FOR doc_text IN document_text
 FILTER doc_text.document_id == @doc_id
 UPDATE doc_text WITH {ocr_content: @ocr_text} IN document_text
 """, {"doc_id": doc_id, "ocr_text": ocr_text})

6. Vector Embedding
embedding = generate_embedding(text) # sentence-transformers
self.db.execute("""
 INSERT {
 document_id: @doc_id,
 embedding: @embedding
 } INTO document_embeddings
 """, {"doc_id": doc_id, "embedding": embedding})

7. Audit-Log
log_action("CREATE", "document", doc_id, None, {"title": title, "type": doc_type})

return doc_id

def add_version(self, document_id, file_path, change_note=""):
 """Neue Version hinzufügen"""
 # Aktuelle Version ermitteln
 current = self.db.execute("""
 FOR doc IN documents
 FILTER doc.id == @document_id
 LIMIT 1
 RETURN doc.version
 """, {"document_id": document_id})[0]
 new_version = current + 1
 version_id = uuid.uuid4()

 with self.db.transaction():
 # Version speichern
 self.db.execute("""
 INSERT {
 id: @version_id,
 document_id: @document_id,
 version: @new_version,
 file_path: @file_path,
 file_size: @file_size,
 checksum: @checksum,
 created_by: @created_by,
 change_note: @change_note
 } INTO document_versions

```

```

 """", {
 "version_id": version_id,
 "document_id": document_id,
 "new_version": new_version,
 "file_path": file_path,
 "file_size": os.path.getsize(file_path),
 "checksum": calculate_checksum(file_path),
 "created_by": get_current_user_id(),
 "change_note": change_note
 })

 # Dokument aktualisieren
 self.db.execute("""
 FOR doc IN documents
 FILTER doc.id == @document_id
 UPDATE doc WITH {
 version: @new_version,
 current_version_id: @version_id,
 updated_at: DATE_NOW()
 } IN documents
 """, {"document_id": document_id, "new_version": new_version, "version_id": v
 """, (new_version, version_id, document_id))

 # Text und Embedding neu generieren
 text = extract_text(file_path)
 self.db.execute("""
 FOR doc_text IN document_text
 FILTER doc_text.document_id == @document_id
 UPDATE doc_text WITH {content: @text} IN document_text
 """, {"document_id": document_id, "text": text})

 embedding = generate_embedding(text)
 self.db.execute("""
 FOR doc_emb IN document_embeddings
 FILTER doc_emb.document_id == @document_id
 UPDATE doc_emb WITH {embedding: @embedding} IN document_embeddings
 """, {"document_id": document_id, "embedding": embedding})

 log_action("VERSION_ADD", "document", document_id, None, {"version": new_vers

 return version_id

def search(self, query, doc_type=None, limit=20):
 """Hybrid-Suche: Volltext + Vector"""
 # 1. Fulltext-Suche
 sql = """
 SELECT d.id, d.title, d.type,
 ts_rank(to_tsvector('german', dt.content), plainto_tsquery('german', ?
 FROM documents d
 JOIN document_text dt ON d.id = dt.document_id

```



```

 WHERE d.tenant_id = ? AND to_tsvector('german', dt.content) @@ plainto_tsquery('
 """
 params = [query, get_current_tenant_id(), query]

 if doc_type:
 sql += " AND d.type = ?"
 params.append(doc_type)

 sql += " ORDER BY rank DESC LIMIT ?"
 params.append(limit)

 fulltext_results = self.db.execute(sql, params)

 # 2. Vector-Suche (semantische Ähnlichkeit)
 query_embedding = generate_embedding(query)
 vector_results = self.db.execute("""
 SELECT d.id, d.title, d.type,
 1 - (de.embedding <=> ?) as similarity
 FROM documents d
 JOIN document_embeddings de ON d.id = de.document_id
 WHERE d.tenant_id = ?
 ORDER BY similarity DESC
 LIMIT ?
 """, (query_embedding, get_current_tenant_id(), limit))

 # 3. Merge Results (kombiniere Scores)
 merged = merge_search_results(fulltext_results, vector_results)
 return merged

def get_version_history(self, document_id):
 """Versionsverlauf abrufen"""
 return self.db.execute("""
 SELECT v.*, u.name as created_by_name
 FROM document_versions v
 LEFT JOIN users u ON v.created_by = u.id
 WHERE v.document_id = ?
 ORDER BY v.version DESC
 """, (document_id,))

```

## Workflow-Management

Workflows als Graph modellieren:

```

class WorkflowEngine:
 def __init__(self, db):
 self.db = db
 self.init_schema()

 def init_schema(self):
 # Workflow-Definitionen (Relational)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS workflows (
 id UUID PRIMARY KEY,
 name TEXT NOT NULL,
 description TEXT,
 active BOOLEAN DEFAULT true
)
 """)

 # Workflow-Schritte als Graph
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS workflow_nodes (
 id UUID PRIMARY KEY,
 workflow_id UUID NOT NULL,
 name TEXT NOT NULL,
 node_type TEXT CHECK (node_type IN ('start', 'approval', 'task', 'decision')),
 role_required TEXT,
 FOREIGN KEY (workflow_id) REFERENCES workflows(id)
)
 """)

 self.db.execute("""
 CREATE TABLE IF NOT EXISTS workflow_edges (
 from_node UUID,
 to_node UUID,
 condition TEXT, -- Optional: JSON mit Bedingung
 PRIMARY KEY (from_node, to_node),
 FOREIGN KEY (from_node) REFERENCES workflow_nodes(id),
 FOREIGN KEY (to_node) REFERENCES workflow_nodes(id)
)
 """)

 # Workflow-Instanzen (laufende Prozesse)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS workflow_instances (
 id UUID PRIMARY KEY,
 workflow_id UUID NOT NULL,
 document_id UUID NOT NULL,
 current_node UUID NOT NULL,
 state TEXT DEFAULT 'running',
 started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 completed_at TIMESTAMP,
)
 """)

```

```

 FOREIGN KEY (workflow_id) REFERENCES workflows(id),
 FOREIGN KEY (document_id) REFERENCES documents(id),
 FOREIGN KEY (current_node) REFERENCES workflow_nodes(id)
)
 """

Workflow-Historie
self.db.execute("""
 CREATE TABLE IF NOT EXISTS workflow_history (
 id UUID PRIMARY KEY,
 instance_id UUID NOT NULL,
 node_id UUID NOT NULL,
 user_id UUID,
 action TEXT,
 comment TEXT,
 timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (instance_id) REFERENCES workflow_instances(id)
)
 """)

def create_workflow(self, name, description, steps):
 """Workflow definieren

 steps = [
 {"name": "Erfassung", "type": "start"},
 {"name": "Manager-Prüfung", "type": "approval", "role": "manager"},
 {"name": "Direktor-Freigabe", "type": "approval", "role": "director"},
 {"name": "Abgeschlossen", "type": "end"}
]
 """
 workflow_id = uuid.uuid4()

 with self.db.transaction():
 self.db.execute("""
 INSERT {
 id: @workflow_id,
 name: @name,
 description: @description
 } INTO workflows
 """, {"workflow_id": workflow_id, "name": name, "description": description})

 # Nodes erstellen
 node_ids = []
 for step in steps:
 node_id = uuid.uuid4()
 self.db.execute("""
 INSERT {
 id: @node_id,
 workflow_id: @workflow_id,
 name: @name,

```

```

 node_type: @node_type,
 role_required: @role_required
 } INTO workflow_nodes
 """ , {
 "node_id": node_id,
 "workflow_id": workflow_id,
 "name": step['name'],
 "node_type": step['type'],
 "role_required": step.get('role')
 })
 node_ids.append(node_id)

 # Edges erstellen (linearer Flow)
 for i in range(len(node_ids) - 1):
 self.db.execute("""
 INSERT {
 from_node: @from_node,
 to_node: @to_node
 } INTO workflow_edges
 """, {"from_node": node_ids[i], "to_node": node_ids[i+1]})

 return workflow_id

def start_workflow(self, workflow_id, document_id):
 """Workflow für Dokument starten"""
 # Start-Node finden
 start_node = self.db.execute("""
 SELECT id FROM workflow_nodes
 WHERE workflow_id = ? AND node_type = 'start'
 """, (workflow_id,))[0]

 instance_id = uuid.uuid4()
 self.db.execute("""
 INSERT INTO workflow_instances (id, workflow_id, document_id, current_node)
 VALUES (?, ?, ?, ?)
 """, (instance_id, workflow_id, document_id, start_node['id']))

 # Dokumentstatus aktualisieren
 self.db.execute("UPDATE documents SET workflow_state = 'in_progress' WHERE id = ?")

 # Nächsten Schritt finden und zuweisen
 self._advance_workflow(instance_id)

 return instance_id

def _advance_workflow(self, instance_id):
 """Workflow zum nächsten Schritt bewegen"""
 instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance_id,))

 # Nächster Node via Graph-Traversierung

```

```

next_nodes = self.db.execute("""
 SELECT wn.* FROM workflow_edges we
 JOIN workflow_nodes wn ON we.to_node = wn.id
 WHERE we.from_node = ?
""", (instance['current_node'],))

if not next_nodes:
 # Ende erreicht
 self.db.execute("""
 UPDATE workflow_instances
 SET state = 'completed', completed_at = CURRENT_TIMESTAMP
 WHERE id = ?
""", (instance_id,))
 self.db.execute("UPDATE documents SET workflow_state = 'completed' WHERE id = ?",
 (instance_id,))
 return

next_node = next_nodes[0]
self.db.execute("UPDATE workflow_instances SET current_node = ? WHERE id = ?",
 (next_node['id'], instance_id))

Task an User mit entsprechender Rolle zuweisen
if next_node['role_required']:
 self._assign_task(instance_id, next_node['id'], next_node['role_required'])

def approve(self, instance_id, user_id, comment=""):
 """Workflow-Schritt genehmigen"""
 instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance_id,))
 current_node = self.db.execute("SELECT * FROM workflow_nodes WHERE id = ?", (instance_id,))

 # Permission check
 if current_node['role_required']:
 if not user_has_role(user_id, current_node['role_required']):
 raise PermissionError(f"User benötigt Rolle: {current_node['role_required']}")

 with self.db.transaction():
 # Historie speichern
 self.db.execute("""
 INSERT INTO workflow_history (id, instance_id, node_id, user_id, action, comment)
 VALUES (?, ?, ?, ?, ?, ?)
""", (uuid.uuid4(), instance_id, current_node['id'], user_id, "APPROVED", comment))

 # Zum nächsten Schritt
 self._advance_workflow(instance_id)

 log_action("WORKFLOW_APPROVE", "workflow_instance", instance_id, None, {
 "node": current_node['name'],
 "user": user_id
 })

def reject(self, instance_id, user_id, reason):

```

```

 """Workflow-Schritt ablehnen"""
 with self.db.transaction():
 instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (

 self.db.execute("""
 INSERT INTO workflow_history (id, instance_id, node_id, user_id, action,
 VALUES (?, ?, ?, ?, ?, ?)
 """, (uuid.uuid4(), instance_id, instance['current_node'], user_id, "REJECTED")

 self.db.execute("UPDATE workflow_instances SET state = 'rejected' WHERE id = ")
 self.db.execute("UPDATE documents SET workflow_state = 'rejected' WHERE id = ")

 log_action("WORKFLOW_REJECT", "workflow_instance", instance_id, None, {"reason":

def get_pending_tasks(self, user_id):
 """Offene Tasks für User"""
 # User-Rollen ermitteln
 user_roles = self.db.execute("SELECT role_name FROM user_roles WHERE user_id = ?")
 role_names = [r['role_name'] for r in user_roles]

 # Workflows in passenden Nodes
 return self.db.execute("""
 SELECT wi.*, d.title as document_title, wn.name as step_name
 FROM workflow_instances wi
 JOIN workflow_nodes wn ON wi.current_node = wn.id
 JOIN documents d ON wi.document_id = d.id
 WHERE wi.state = 'running'
 AND wn.role_required IN ({})
 ORDER BY wi.started_at
 """).format(', '.join('?' * len(role_names)), role_names)

```

---

## **API-Implementierung**

```

from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash
import jwt

app = Flask(__name__)
db = ThemisDB("dms_erp.db")
doc_manager = Document(db)
workflow_engine = WorkflowEngine(db)

def authenticate(username, password):
 user = db.execute("SELECT * FROM users WHERE username = ?", (username,))
 if not user or not check_password_hash(user[0]['password'], password):
 return None
 token = jwt.encode({'user_id': str(user[0]['id'])}, app.config['SECRET_KEY'])
 return token

@app.route('/api/auth/login', methods=['POST'])
def login():
 data = request.json
 token = authenticate(data['username'], data['password'])
 if token:
 return jsonify({'token': token})
 return jsonify({'error': 'Invalid credentials'}), 401

def require_auth(f):
 def wrapper(*args, **kwargs):
 token = request.headers.get('Authorization', '').replace('Bearer ', '')
 try:
 payload = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])
 request.user_id = payload['user_id']
 return f(*args, **kwargs)
 except:
 return jsonify({'error': 'Unauthorized'}), 401
 return wrapper

@app.route('/api/documents', methods=['POST'])
@require_auth
@requires_permission('create:document')
def create_document():
 file = request.files['file']
 metadata = request.form.get('metadata', '{}')

 # Datei speichern
 file_path = save_uploaded_file(file)

 doc_id = doc_manager.create_document(
 title=file.filename,
 doc_type=request.form.get('type', 'general'),
 file_path=file_path,

```



```

 metadata=json.loads(metadata),
 owner_id=request.user_id
)

 return jsonify({'id': str(doc_id), 'message': 'Document created'}), 201

@app.route('/api/documents/<doc_id>', methods=['GET'])
@require_auth
def get_document(doc_id):
 # Permission Check
 if not has_document_permission(request.user_id, doc_id, 'read'):
 return jsonify({'error': 'Forbidden'}), 403

 doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))[0]
 versions = doc_manager.get_version_history(doc_id)

 return jsonify({
 'document': doc,
 'versions': versions
 })

@app.route('/api/documents/search', methods=['GET'])
@require_auth
def search_documents():
 query = request.args.get('q', '')
 doc_type = request.args.get('type')

 results = doc_manager.search(query, doc_type)
 return jsonify({'results': results})

@app.route('/api/workflows/<workflow_id>/start', methods=['POST'])
@require_auth
def start_workflow_api(workflow_id):
 data = request.json
 instance_id = workflow_engine.start_workflow(workflow_id, data['document_id'])
 return jsonify({'instance_id': str(instance_id)})

@app.route('/api/workflows/tasks', methods=['GET'])
@require_auth
def get_my_tasks():
 tasks = workflow_engine.get_pending_tasks(request.user_id)
 return jsonify({'tasks': tasks})

@app.route('/api/workflows/instances/<instance_id>/approve', methods=['POST'])
@require_auth
def approve_workflow(instance_id):
 data = request.json
 workflow_engine.approve(instance_id, request.user_id, data.get('comment', ''))
 return jsonify({'message': 'Approved'})

```

```
@app.route('/api/workflows/instances/<instance_id>/reject', methods=['POST'])
@require_auth
def reject_workflow(instance_id):
 data = request.json
 workflow_engine.reject(instance_id, request.user_id, data['reason'])
 return jsonify({'message': 'Rejected'})
```

## OCR und Text-Extraktion

```
import pytesseract
from PIL import Image
import fitz # PyMuPDF

def extract_text(file_path):
 """Text aus verschiedenen Formaten extrahieren"""
 ext = file_path.split('.')[-1].lower()

 if ext == 'pdf':
 return extract_text_from_pdf(file_path)
 elif ext in ('docx', 'doc'):
 return extract_text_from_docx(file_path)
 elif ext == 'txt':
 with open(file_path, 'r', encoding='utf-8') as f:
 return f.read()
 else:
 return ""

def extract_text_from_pdf(file_path):
 """PDF Text-Extraktion"""
 text = []
 doc = fitz.open(file_path)
 for page in doc:
 text.append(page.get_text())
 return '\n'.join(text)

def perform_ocr(file_path):
 """OCR für gescannte Dokumente"""
 if file_path.endswith('.pdf'):
 # PDF → Bilder → OCR
 doc = fitz.open(file_path)
 ocr_text = []
 for page_num in range(len(doc)):
 page = doc[page_num]
 pix = page.get_pixmap()
 img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
 text = pytesseract.image_to_string(img, lang='deu')
 ocr_text.append(text)
 return '\n'.join(ocr_text)
 else:
 # Direktes Bild
 img = Image.open(file_path)
 return pytesseract.image_to_string(img, lang='deu')
```

## **10.4 CRM-System (Example 17)**

Customer Relationship Management komplett in ThemisDB.

## **Datenmodell**

```

class CRM:
 def __init__(self, db):
 self.db = db
 self.init_schema()

 def init_schema(self):
 # Kunden (Relational)
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS customers (
 id UUID PRIMARY KEY,
 tenant_id UUID NOT NULL,
 name TEXT NOT NULL,
 type TEXT CHECK (type IN ('individual', 'company')),
 industry TEXT,
 revenue DECIMAL,
 employees INT,
 website TEXT,
 status TEXT DEFAULT 'active',
 lead_score INT DEFAULT 0,
 lifetime_value DECIMAL DEFAULT 0,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
 """)

 # Kontaktpersonen
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS contacts (
 id UUID PRIMARY KEY,
 customer_id UUID NOT NULL,
 first_name TEXT,
 last_name TEXT,
 email TEXT,
 phone TEXT,
 position TEXT,
 is_primary BOOLEAN DEFAULT false,
 FOREIGN KEY (customer_id) REFERENCES customers(id)
)
 """)

 # Leads / Opportunities
 self.db.execute("""
 CREATE TABLE IF NOT EXISTS opportunities (
 id UUID PRIMARY KEY,
 customer_id UUID NOT NULL,
 title TEXT NOT NULL,
 value DECIMAL NOT NULL,
 stage TEXT NOT NULL, -- prospecting, qualification, proposal, negotiation
 probability INT CHECK (probability BETWEEN 0 AND 100),

```

```

 expected_close_date DATE,
 assigned_to UUID,
 source TEXT, -- website, referral, cold_call, etc.
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (customer_id) REFERENCES customers(id)
)
 """
)

Activities (Time-Series)
self.db.execute("""
 CREATE TABLE IF NOT EXISTS activities (
 id UUID PRIMARY KEY,
 customer_id UUID,
 opportunity_id UUID,
 type TEXT NOT NULL, -- email, call, meeting, note
 subject TEXT,
 description TEXT,
 duration INT, -- Minuten
 completed BOOLEAN DEFAULT false,
 scheduled_at TIMESTAMP,
 completed_at TIMESTAMP,
 created_by UUID,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (customer_id) REFERENCES customers(id),
 FOREIGN KEY (opportunity_id) REFERENCES opportunities(id)
)
 """)

Index für Timeline
self.db.execute("CREATE INDEX idx_activities_timeline ON activities(customer_id, cre

Notes / Interaktionen (Document Model)
self.db.execute("""
 CREATE TABLE IF NOT EXISTS customer_notes (
 id UUID PRIMARY KEY,
 customer_id UUID NOT NULL,
 note JSON NOT NULL, -- {author, text, attachments, tags}
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (customer_id) REFERENCES customers(id)
)
 """)

def create_customer(self, name, customer_type, **kwargs):
 """Neuen Kunden anlegen"""
 customer_id = uuid.uuid4()

 self.db.execute("""
 INSERT INTO customers (id, tenant_id, name, type, industry, revenue, employees,
 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
 """, (
 customer_id,

```

```

 get_current_tenant_id(),
 name,
 customer_type,
 kwargs.get('industry'),
 kwargs.get('revenue'),
 kwargs.get('employees'),
 kwargs.get('website')
))

 log_action("CREATE", "customer", customer_id, None, {"name": name})
 return customer_id

def create_opportunity(self, customer_id, title, value, stage, **kwargs):
 """Lead/Opportunity erstellen"""
 opp_id = uuid.uuid4()

 self.db.execute("""
 INSERT INTO opportunities (id, customer_id, title, value, stage, probability,
 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
 """, (
 opp_id,
 customer_id,
 title,
 value,
 stage,
 kwargs.get('probability', self._calculate_probability(stage)),
 kwargs.get('expected_close_date'),
 kwargs.get('assigned_to', get_current_user_id()),
 kwargs.get('source')
))

 # Lead Score aktualisieren
 self._update_lead_score(customer_id)

 return opp_id

def _calculate_probability(self, stage):
 """Wahrscheinlichkeit basierend auf Stage"""
 probabilities = {
 'prospecting': 10,
 'qualification': 25,
 'proposal': 50,
 'negotiation': 75,
 'closed_won': 100,
 'closed_lost': 0
 }
 return probabilities.get(stage, 0)

def _update_lead_score(self, customer_id):
 """Lead-Scoring berechnen"""

```



```

Faktoren:
- Anzahl Opportunities
- Gesamtwert offener Opportunities
- Anzahl Activities
- Durchschnittliche Wahrscheinlichkeit

score_data = self.db.execute("""
 SELECT
 COUNT(DISTINCT o.id) as opp_count,
 COALESCE(SUM(o.value), 0) as total_value,
 COALESCE(AVG(o.probability), 0) as avg_probability,
 COUNT(DISTINCT a.id) as activity_count
 FROM customers c
 LEFT JOIN opportunities o ON c.id = o.customer_id AND o.stage NOT IN ('closed_won')
 LEFT JOIN activities a ON c.id = a.customer_id
 WHERE c.id = ?
 GROUP BY c.id
""", (customer_id,))[0]

Simple Scoring-Formel
score = (
 score_data['opp_count'] * 10 +
 min(score_data['total_value'] / 1000, 50) +
 score_data['avg_probability'] / 2 +
 score_data['activity_count'] * 2
)
score = min(int(score), 100)

self.db.execute("UPDATE customers SET lead_score = ? WHERE id = ?", (score, customer_id))

def log_activity(self, customer_id, activity_type, subject, description, **kwargs):
 """Aktivität loggen"""
 activity_id = uuid.uuid4()

 self.db.execute("""
 INSERT INTO activities (id, customer_id, opportunity_id, type, subject, description,
 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
 """, (
 activity_id,
 customer_id,
 kwargs.get('opportunity_id'),
 activity_type,
 subject,
 description,
 kwargs.get('duration'),
 kwargs.get('scheduled_at'),
 get_current_user_id()
))

 return activity_id

```

```

def complete_activity(self, activity_id):
 """Aktivität als erledigt markieren"""
 self.db.execute("""
 UPDATE activities
 SET completed = true, completed_at = CURRENT_TIMESTAMP
 WHERE id = ?
 """, (activity_id,))

def get_customer_timeline(self, customer_id, limit=50):
 """Komplette Timeline für Kunde"""
 return self.db.execute("""
 SELECT
 'activity' as item_type,
 id,
 type,
 subject,
 created_at as timestamp
 FROM activities
 WHERE customer_id = ?

 UNION ALL

 SELECT
 'note' as item_type,
 id,
 'note' as type,
 note->'text' as subject,
 created_at as timestamp
 FROM customer_notes
 WHERE customer_id = ?

 UNION ALL

 SELECT
 'opportunity' as item_type,
 id,
 'opportunity' as type,
 title as subject,
 created_at as timestamp
 FROM opportunities
 WHERE customer_id = ?

 ORDER BY timestamp DESC
 LIMIT ?
 """, (customer_id, customer_id, customer_id, limit))

def get_sales_pipeline(self):
 """Sales-Pipeline Übersicht"""
 return self.db.execute("""

```

```

SELECT
 stage,
 COUNT(*) as count,
 SUM(value) as total_value,
 AVG(probability) as avg_probability
FROM opportunities
WHERE stage NOT IN ('closed_won', 'closed_lost')
GROUP BY stage
ORDER BY
 CASE stage
 WHEN 'prospecting' THEN 1
 WHEN 'qualification' THEN 2
 WHEN 'proposal' THEN 3
 WHEN 'negotiation' THEN 4
 END
"""
)

def forecast_revenue(self, months=3):
 """Revenue-Forecast"""
 return self.db.execute("""
 SELECT
 DATE_TRUNC('month', expected_close_date) as month,
 SUM(value * probability / 100) as weighted_value,
 SUM(value) as total_value,
 COUNT(*) as opportunity_count
 FROM opportunities
 WHERE expected_close_date >= CURRENT_DATE
 AND expected_close_date <= CURRENT_DATE + INTERVAL '? months'
 AND stage NOT IN ('closed_won', 'closed_lost')
 GROUP BY month
 ORDER BY month
 """, (months,))

def get_top_customers(self, limit=10):
 """Top-Kunden nach Lifetime Value"""
 return self.db.execute("""
 SELECT
 c.*,
 COUNT(DISTINCT o.id) as opportunity_count,
 SUM(CASE WHEN o.stage = 'closed_won' THEN o.value ELSE 0 END) as won_value,
 COUNT(DISTINCT a.id) as activity_count
 FROM customers c
 LEFT JOIN opportunities o ON c.id = o.customer_id
 LEFT JOIN activities a ON c.id = a.customer_id
 WHERE c.tenant_id = ?
 GROUP BY c.id
 ORDER BY won_value DESC, c.lead_score DESC
 LIMIT ?
 """, (get_current_tenant_id(), limit))

```

## **Dashboard und Reporting**

```

class CRMDashboard:
 def __init__(self, crm):
 self.crm = crm
 self.db = crm.db

 def get_overview(self):
 """Dashboard Übersicht"""
 return {
 'customers': self._get_customer_stats(),
 'pipeline': self.crm.get_sales_pipeline(),
 'activities': self._get_activity_stats(),
 'forecast': self.crm.forecast_revenue(3)
 }

 def _get_customer_stats(self):
 return self.db.execute("""
 SELECT
 COUNT(*) as total,
 COUNT(CASE WHEN status = 'active' THEN 1 END) as active,
 COUNT(CASE WHEN lead_score >= 70 THEN 1 END) as hot_leads,
 AVG(lead_score) as avg_lead_score
 FROM customers
 WHERE tenant_id = ?
 """, (get_current_tenant_id(),))[0]

 def _get_activity_stats(self):
 return self.db.execute("""
 SELECT
 type,
 COUNT(*) as count,
 COUNT(CASE WHEN completed THEN 1 END) as completed
 FROM activities
 WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
 GROUP BY type
 """)

 def generate_report(self, report_type, start_date, end_date):
 """Verschiedene Reports generieren"""
 if report_type == 'sales':
 return self._sales_report(start_date, end_date)
 elif report_type == 'activities':
 return self._activity_report(start_date, end_date)
 elif report_type == 'conversion':
 return self._conversion_report(start_date, end_date)

 def _sales_report(self, start_date, end_date):
 """Verkaufs-Report"""
 return self.db.execute("""
 SELECT

```

```

 DATE_TRUNC('week', created_at) as week,
 COUNT(*) as opportunities_created,
 COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
 SUM(CASE WHEN stage = 'closed_won' THEN value ELSE 0 END) as won_value,
 COUNT(CASE WHEN stage = 'closed_lost' THEN 1 END) as lost
 FROM opportunities
 WHERE created_at BETWEEN ? AND ?
 GROUP BY week
 ORDER BY week
 """ , (start_date, end_date))

def _conversion_report(self, start_date, end_date):
 """Conversion-Funnel"""
 return self.db.execute("""
 SELECT
 source,
 COUNT(*) as total,
 COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
 ROUND(COUNT(CASE WHEN stage = 'closed_won' THEN 1 END)::NUMERIC / COUNT(*)
 AVG(value) as avg_deal_size
 FROM opportunities
 WHERE created_at BETWEEN ? AND ?
 GROUP BY source
 ORDER BY conversion_rate DESC
 """ , (start_date, end_date))

```

## 10.5 Best Practices für Enterprise-Anwendungen

### 1. Performance bei großen Datenmengen

```
db.execute("""
 CREATE TABLE activities_2025_01 PARTITION OF activities
 FOR VALUES FROM ('2025-01-01') TO ('2025-02-01')
""")

db.execute("""
 CREATE MATERIALIZED VIEW sales_summary AS
 SELECT
 DATE_TRUNC('month', created_at) as month,
 stage,
 COUNT(*) as count,
 SUM(value) as total_value
 FROM opportunities
 GROUP BY month, stage
""")

db.execute("REFRESH MATERIALIZED VIEW sales_summary")
```

### 2. Caching-Strategie

```
import redis

cache = redis.Redis()

def get_dashboard_data_cached():
 cached = cache.get('dashboard:overview')
 if cached:
 return json.loads(cached)

 # Berechnen
 data = dashboard.get_overview()

 # Cache für 5 Minuten
 cache.setex('dashboard:overview', 300, json.dumps(data))
 return data
```

### 3. Background-Jobs

```

from celery import Celery

celery = Celery('enterprise_app')

@celery.task
def generate_large_report(report_id):
 """Großer Report im Hintergrund"""
 report_data = dashboard.generate_report('sales', start_date, end_date)

 # PDF generieren
 pdf_path = create_pdf_report(report_data)

 # In DB speichern
 db.execute("""
 UPDATE reports
 SET status = 'completed', file_path = ?
 WHERE id = ?
 """, (pdf_path, report_id))

 # User benachrichtigen
 send_notification(user_id, f"Report {report_id} ist fertig")

@celery.task
def update_lead_scores():
 """Lead-Scores täglich neu berechnen"""
 customers = db.execute("SELECT id FROM customers WHERE status = 'active'")
 for customer in customers:
 crm._update_lead_score(customer['id'])

```

### 4. API-Rate-Limiting

```

from flask_limiter import Limiter

limiter = Limiter(app, key_func=lambda: request.user_id)

@app.route('/api/documents/search')
@limiter.limit("100/hour")
@require_auth
def search_api():
 # Maximal 100 Searches pro Stunde pro User
 pass

```



## 5. Monitoring und Alerting

```
import statsd
from prometheus_client import Counter, Histogram

document_uploads = Counter('document_uploads_total', 'Total document uploads')
query_duration = Histogram('query_duration_seconds', 'Query execution time')

@app.route('/api/documents', methods=['POST'])
@require_auth
def create_document():
 document_uploads.inc()

 with query_duration.time():
 doc_id = doc_manager.create_document(...)

 return jsonify({'id': str(doc_id)})

@app.route('/health')
def health():
 try:
 db.execute("SELECT 1")
 return jsonify({'status': 'healthy'}), 200
 except:
 return jsonify({'status': 'unhealthy'}), 500
```

## 10.5 Enterprise Compliance & Audit (Neu)

Compliance-Anforderungen (DSGVO, BAIT, ISO 27001) verlangen nachvollziehbare Änderungen, Daten-Löschkonzepte und strikte Trennung zwischen Tenants und Regionen.

### 10.5.1 Audit-Log Architektur

```

flowchart LR
 A[Client/API] --> B[Service Layer]
 B --> C[Audit Enricher
Request ID, User, Tenant]
 C --> D[Audit Queue]
 D --> E[Audit Writer]
 E --> F[(RocksDB: audit: prefix)]

 F --> G[Cold Storage / S3]
 E --> H[SIEM Forwarder]
 H --> I[SIEM/Elastic/Splunk]

 style F fill:#e1f5ff
 style G fill:#fff4e1
 style I fill:#e1ffe1

```

**Abb. 10.3:** Audit-Log-Flow

### 10.5.2 Audit-Log Schema (RocksDB Prefix)

Field	Type	Description
<code>tenant_id</code>	string	Mandant (Tenant)
<code>actor</code>	string	User/Service Principal
<code>action</code>	string	z.B. <code>documents.create</code> , <code>users.update</code>
<code>resource</code>	string	Resource Identifier (z.B. <code>doc:123</code> )
<code>ts</code>	int64	Unix ms Timestamp
<code>ip</code>	string	Client-IP
<code>status</code>	string	<code>success</code> / <code>failure</code>
<code>before</code>	json	Optional: Vorheriger Zustand
<code>after</code>	json	Optional: Neuer Zustand

**Key Layout:** `audit:{tenant}:{ts}:{uuid}` → Prefix-Scan pro Tenant + Zeitraum.

### 10.5.3 DSGVO: Löschung & Aufbewahrung

```
-- Aufbewahrungsregel pro Tenant (Config-Collection)
FOR cfg IN audit_policies
 FILTER cfg.tenant_id == @tenant
 RETURN cfg.retention_days

-- Lösch-Job (täglich)
LET cutoff = DATE_NOW() - DAYS(@retention_days)
FOR entry IN audit
 FILTER entry.tenant_id == @tenant
 FILTER entry.ts < cutoff
 REMOVE entry IN audit
```

**Double-Delete:** Nach Log-Löschung optional **after** -Payload mit Null-Werten überschreiben, falls rechtlich gefordert.

### 10.5.4 BAIT/ISO 27001 Mapping (Auszug)

- **Protokollierung:** Jede sicherheitsrelevante Aktion → Audit-Log (BAIT 8.1)
- **Unveränderbarkeit:** Write-Once-Append oder WORM-Bucket für Export (ISO 27001 A.12.4)
- **Trennung:** Tenant-ID im Key, optionale Region im Prefix **audit:eu-central-1:tenant:...**
- **Nachvollziehbarkeit:** Request-ID, Correlation-ID, Actor, IP
- **Rechteprüfung:** RBAC-Check vor Schreiboperationen, Audit-Log auch für abgelehnte Zugriffe

#### Audit-Exporte:

```
aws s3 cp audit.json s3://themis-audit/2025/12/31/ \
 --object-lock-mode COMPLIANCE \
 --object-lock-retain-until-date 2032-12-31
```

## 10.5.6 Tenant & Region Sharding

```
audit_storage:
 strategy: by_region_and_tenant
 regions:
 - id: eu-central-1
 bucket: s3://themis-audit-eu
 - id: us-east-1
 bucket: s3://themis-audit-us

rocksdb_prefixes:
 eu: "audit:eu-central-1:"
 us: "audit:us-east-1:"
```

## 10.5.7 Editions & Distribution (Kurzüberblick)

Edition	Features	Zielgruppe
<b>Community</b>	Core Storage, AQL, Basic Indizes	Entwickler, PoC
<b>Enterprise</b>	RBAC, Audit, HA/Failover, Advanced Backup	Unternehmen
<b>Gov/Regulated</b>	Region Sharding, WORM-Export, Langzeitaufbewahrung	Finanz, Behörden

**Distribution Best Practice:** 1. **Core Binary** per Artifact Registry ausliefern 2. **Regionale Add-ons** (Compliance-Pakete) nur für passende Regionen aktivieren 3. **Config-as-Code:** Tenant-Policies versionieren

## 10.7 Sprachassistent für Enterprise (Voice Assistant)

***Enterprise-Feature:** Der Voice Assistant ist ein optionales Enterprise-Feature, das natürlichsprachliche Sprachinteraktion direkt in ThemisDB integriert.*

### Überblick

Der ThemisDB Sprachassistent bietet Voice-Interaktionsfähigkeiten ähnlich wie Alexa oder Siri, jedoch direkt in die Datenbank integriert und mit voller Enterprise-Compliance. Er kombiniert drei Technologien:

1. **Speech-to-Text (STT)** via Whisper.cpp - Hochgenaue Transkription in 100+ Sprachen
2. **Text-to-Speech (TTS)** via Piper - Natürliche Sprachsynthese
3. **LLM-Integration** via llama.cpp - Natürlichsprachliches Verständnis

**Hauptanwendungsfälle:** - 📞 **Call Center:** Automatische Telefonaufzeichnung und Transkription - 📝  
**Meeting Protokolle:** KI-gestützte Protokollerstellung mit Aktionspunkten - 🧠 **Voice Commands:**  
 Datenbankabfragen über natürliche Sprache - 🏠 **Diktiersysteme:** Medizinische/rechtliche  
 Dokumentation - 🤖 **Voice Bots:** Interaktive Sprachassistenten für Kunden

Alle Aufzeichnungen werden **revision-safe** in ThemisDB gespeichert mit vollständigen Audit-Trails und DSGVO-konformer Aufbewahrung.

## Architektur

```
graph TB
 subgraph "Voice Assistant Architektur"
 User([Benutzer
Sprachinput]) -->|Audio| API[REST API
/api/v1/voice]
 User -->|Realtime| WS[WebSocket
/ws/voice]

 API --> STT[Speech-to-Text
Whisper.cpp]
 WS --> STT

 STT -->|Transkript| LLM[LLM Engine
llama.cpp]

 LLM -->|Intent| QE[Query Engine
AQL Processing]
 LLM -->|Response Text| TTS[Text-to-Speech
Piper TTS]

 QE -->|Results| DB[(ThemisDB
Base Entities)]

 TTS -->|Audio| User

 DB -->|Store| Recordings[Voice Recordings
+ Transcripts]
 DB -->|Audit| Logs[Audit Logs
DSGVO-konform]
 end

 style STT fill:#43e97b
 style LLM fill:#667eea
 style TTS fill:#f093fb
 style DB fill:#ffd32a
 style Recordings fill:#fee140
 style Logs fill:#f78ca0
```

**Abb. 10.4:** Compliance-Workflow

### STT: Speech-to-Text mit Whisper.cpp

Whisper.cpp ist eine hochperformante C++ Implementierung von OpenAI's Whisper Modell.

**Modellgrößen** (Trade-off: Geschwindigkeit vs. Genauigkeit):

Modell	Parameter	Geschwindigkeit	Genauigkeit	Use Case
<code>tiny</code>	39M	~10x realtime	85%	Echtzeit-Interaktion
<code>base</code>	74M	~5x realtime	90%	Standard (empfohlen)
<code>small</code>	244M	~2x realtime	94%	Call Center
<code>medium</code>	769M	~1x realtime	96%	Medizinische Diktate
<code>large</code>	1550M	~0.5x realtime	98%	Juristische Protokolle

**Konfiguration:**

```

voice_assistant:
 enabled: true

stt:
 model_path: "./models/ggml-base.bin"
 model_size: "base"
 language: "auto" # Automatische Spracherkennung
 threads: 4

Erweiterte Optionen
speaker_diarization: true # Sprecher-Identifikation
timestamps: true # Wort-Timestamps
vad_filter: true # Voice Activity Detection

```

**Praktisches Beispiel - Telefonaufzeichnung:**

```

import requests
import json

with open("call_recording.mp3", "rb") as audio:
 response = requests.post(
 "http://localhost:8080/api/v1/voice/transcribe",
 files={"audio": audio},
 headers={"Authorization": "Bearer YOUR_TOKEN"},
 data={
 "language": "de",
 "speaker_diarization": True,
 "store_recording": True, # In ThemisDB speichern
 "tenant_id": "acme_corp"
 }
)

result = response.json()
print(f"Transkript ID: {result['id']}")
print(f"Dauer: {result['duration_seconds']}s")
print(f"Sprecher: {len(result['speakers'])}")

for segment in result['segments']:
 print(f"[{segment['start']}s - {segment['end']}s] "
 f"Speaker {segment['speaker']}: {segment['text']}")

```

**Unterstützte Audio-Formate:** - MP3, WAV, OGG, FLAC, AAC, M4A, Opus, WMA - Sampling Rate: 16 kHz (empfohlen) oder Auto-Resampling

## TTS: Text-to-Speech mit Piper

Piper ist eine schnelle, neuronale TTS-Engine mit natürlich klingenden Stimmen.

### Verfügbare Stimmen:

```

response = requests.get("http://localhost:8080/api/v1/voice/voices")
voices = response.json()

```

### Text zu Sprache konvertieren:

```

summary = """
Wichtige Punkte aus dem Meeting:
1. Q4 Umsatz übertrifft Erwartungen um 15 Prozent
2. Neues Feature-Release für Ende Januar geplant
3. Team-Event am 15. Februar
"""

response = requests.post(
 "http://localhost:8080/api/v1/voice/synthesize",
 headers={"Authorization": "Bearer YOUR_TOKEN"},
 json={
 "text": summary,
 "voice": "de_DE-thorsten-neutral",
 "speed": 1.0, # Normale Geschwindigkeit
 "pitch": 1.0, # Normale Tonhöhe
 "output_format": "mp3"
 }
)

with open("meeting_summary.mp3", "wb") as f:
 f.write(response.content)

```

**Performance:** - ~10-20x schneller als Realtime - Streaming-Modus für niedrige Latenz - Geringe CPU-Last (läuft auch ohne GPU)

## LLM-Integration für natürlichsprachliche Queries

Der Voice Assistant nutzt die LLM-Engine (siehe Kapitel 17) für:

1. **Intent Recognition** - Verstehen was der Nutzer möchte
2. **Text-to-AQL** - Natürlichsprachliche Queries in AQL übersetzen
3. **Zusammenfassungen** - Meeting-Protokolle und Key Points
4. **Konversation** - Multi-Turn Dialog mit Kontext

**Beispiel - Voice-gesteuerte Datenbankabfrage:**



```
response = requests.post(
 "http://localhost:8080/api/v1/voice/command",
 headers={"Authorization": "Bearer YOUR_TOKEN"},
 json={
 "text": "Zeige mir alle offenen Bestellungen von gestern",
 "session_id": "user_42",
 "return_audio": True # TTS-Antwort als Audio
 }
)

result = response.json()

print(result['intent']) # "query_orders"
print(result['aql'])

print(f"Gefunden: {len(result['results'])} Bestellungen")

with open("response.mp3", "wb") as f:
 f.write(base64.b64decode(result['audio_response']))
```

## Meeting-Protokoll-Generierung

### Vollständiger Workflow:

```

import websockets
import asyncio

async def record_meeting():
 uri = "ws://localhost:8080/ws/voice/record"
 async with websockets.connect(uri) as websocket:
 # Sende Konfigurations-Header
 await websocket.send(json.dumps({
 "type": "config",
 "session_id": "meeting_2026_01_09",
 "language": "de",
 "speaker_diarization": True,
 "llm_analysis": True # Aktiviert Echtzeit-Analyse
 })))

 # Streame Audio-Chunks (z.B. von Mikrofon)
 while recording:
 audio_chunk = microphone.read()
 await websocket.send(audio_chunk)

 # Empfange Transkript-Updates
 if response := await websocket.recv():
 update = json.loads(response)
 if update['type'] == 'transcript':
 print(f"[{update['speaker']}]: {update['text']}")

 response = requests.post(
 "http://localhost:8080/api/v1/voice/meeting/finalize",
 headers={"Authorization": "Bearer YOUR_TOKEN"},
 json={
 "session_id": "meeting_2026_01_09",
 "generate_protocol": True,
 "extract_action_items": True,
 "participants": ["Alice", "Bob", "Carol"]
 }
)

 protocol = response.json()

 print("=== MEETING PROTOKOLL ===")
 print(f"Datum: {protocol['date']}")
 print(f"Dauer: {protocol['duration_minutes']} Minuten")
 print(f"Teilnehmer: {'', '.join(protocol['participants'])}")
 print()
 print("Zusammenfassung:")
 print(protocol['summary'])
 print()
 print("Kernpunkte:")
 for i, point in enumerate(protocol['key_points'], 1):

```

```
print(f"{i}. {point}")
print()
print("Aktionspunkte:")
for action in protocol['action_items']:
 print(f"- [] {action['task']} (verantwortlich: {action['assignee']})")
```

### Beispiel-Protokoll-Output:

```
=== MEETING PROTOKOLL ===
Datum: 2026-01-09 14:30:00
Dauer: 45 Minuten
Teilnehmer: Alice (Product Manager), Bob (Developer), Carol (Designer)

Zusammenfassung:
Das Team diskutierte den Stand des Q1 Releases. Die Entwicklung liegt gut
im Zeitplan. Design-Reviews für 3 neue Features wurden abgeschlossen. Es
wurden Prioritäten für die nächsten 2 Wochen festgelegt.

Kernpunkte:
1. Login-Feature ist fertig entwickelt, QA-Phase läuft
2. Dashboard-Design wurde final abgenommen
3. API-Performance verbessert um 40% durch Caching
4. Zwei kritische Bugs in Production gefunden und gefixt

Aktionspunkte:
- [] API-Dokumentation aktualisieren (verantwortlich: Bob, bis 12.01.)
- [] User Testing für neues Dashboard organisieren (verantwortlich: Carol, bis 15.01.)
- [] Release Notes vorbereiten (verantwortlich: Alice, bis 20.01.)
```

## Security & Compliance

### DSGVO-konforme Speicherung:

```

response = requests.post(
 "http://localhost:8080/api/v1/voice/record",
 headers={"Authorization": "Bearer YOUR_TOKEN"},
 json={
 "audio_base64": audio_data,
 "metadata": {
 "purpose": "customer_service",
 "consent_given": True,
 "retention_days": 90, # Automatische Löschung nach 90 Tagen
 "dsgvo_category": "call_recording"
 }
 }
)

config = {
 "voice_assistant": {
 "encryption": {
 "at_rest": True, # Recordings verschlüsselt speichern
 "key_rotation_days": 30
 },
 "audit": {
 "log_all_access": True,
 "log_transcripts": False # Sensible Daten nicht loggen
 }
 }
}

```

**Audit-Logging:**

Alle Voice-Operationen werden vollständig geloggt:

```
-- Audit-Log für Voice-Zugriffe
SELECT * FROM audit_logs
WHERE operation_type IN ('voice_transcribe', 'voice_synthesize', 'voice_access')
ORDER BY timestamp DESC
LIMIT 100;

-- Typischer Log-Eintrag:
{
 "timestamp": "2026-01-09T14:30:22Z",
 "user_id": "user_42",
 "operation": "voice_transcribe",
 "recording_id": "rec_abc123",
 "ip_address": "192.168.1.100",
 "metadata": {
 "duration_seconds": 180,
 "language": "de",
 "speakers": 2
 }
}
```

## Performance & Skalierung

**Benchmark-Ergebnisse** (Hardware: 8-Core CPU, 16GB RAM):

Operation	Modell	Latenz	Throughput
STT (tiny)	39M	0.1x realtime	10 concurrent
STT (base)	74M	0.2x realtime	5 concurrent
STT (large)	1550M	2x realtime	1 concurrent
TTS	Piper	0.05x realtime	20 concurrent
LLM Analysis	7B	20 tok/s	3 concurrent

**Skalierungs-Strategie:**

```
voice_assistant:
 worker_nodes:
 - node: voice-worker-1
 capabilities: [stt, tts]
 gpu: false

 - node: voice-worker-2
 capabilities: [stt]
 model: large
 gpu: true # GPU-beschleunigte Transkription

 - node: voice-worker-3
 capabilities: [llm]
 gpu: true

 load_balancing:
 strategy: least_loaded
 health_check_interval: 30s
```

## Use Case: Call Center Automation

### Vollständiges Beispiel:

```

class CallCenterIntegration:
 """
 Integration des Voice Assistant in ein Call Center System.
 Automatische Aufzeichnung, Transkription, Sentiment-Analyse und
 CRM-Update.
 """

 def __init__(self, themisdb_url, api_key):
 self.base_url = themisdb_url
 self.headers = {"Authorization": f"Bearer {api_key}"}

 def process_call(self, call_id, audio_file, customer_id):
 """Verarbeitet einen eingehenden Anruf."""

 # 1. Transkribieren mit Speaker Diarization
 transcript_result = self.transcribe_call(audio_file)

 # 2. Sentiment-Analyse via LLM
 sentiment = self.analyze_sentiment(transcript_result['transcript'])

 # 3. Kernpunkte extrahieren
 summary = self.extract_summary(transcript_result['transcript'])

 # 4. CRM aktualisieren
 self.update_crm(customer_id, {
 "call_id": call_id,
 "transcript_id": transcript_result['id'],
 "duration": transcript_result['duration'],
 "sentiment": sentiment,
 "summary": summary,
 "action_items": summary['action_items']
 })

 # 5. Automatische Follow-up Email generieren
 if sentiment['score'] < 0.5: # Negativer Call
 self.create_followup_task(customer_id, "urgent")

 return {
 "status": "processed",
 "transcript_id": transcript_result['id'],
 "sentiment": sentiment,
 "summary": summary
 }

 def transcribe_call(self, audio_file):
 """Transkribiert Audio mit Whisper."""
 with open(audio_file, "rb") as f:
 response = requests.post(
 f"{self.base_url}/api/v1/voice/transcribe",

```

```

 files={"audio": f},
 headers=self.headers,
 data={
 "speaker_diarization": True,
 "language": "auto",
 "model": "base"
 }
)
 return response.json()

def analyze_sentiment(self, transcript):
 """Sentiment-Analyse via LLM."""
 response = requests.post(
 f"{self.base_url}/api/v1/llm/analyze",
 headers=self.headers,
 json={
 "prompt": f"""
 Analysiere die Stimmung in diesem Kundenservice-Gespräch.
 Gib einen Sentiment-Score von 0.0 (sehr negativ) bis 1.0 (sehr positiv)
 und die Hauptgründe.

 Gespräch:
 {transcript}
 """,
 "response_format": "json",
 "schema": {
 "score": "float",
 "reasons": "array",
 "customer_satisfied": "boolean"
 }
 }
)
 return response.json()['result']

call_center = CallCenterIntegration(
 themisdb_url="http://localhost:8080",
 api_key="your_api_key"
)

result = call_center.process_call(
 call_id="CALL_2026_01_09_001",
 audio_file="recordings/call_001.mp3",
 customer_id="CUST_42"
)

print(f"Call verarbeitet. Sentiment: {result['sentiment']['score']:.2f}")
print(f"Zusammenfassung: {result['summary']['text']}")

```



## Zusammenfassung & Best Practices

**Was Sie gelernt haben:** - Voice Assistant Architektur (STT, TTS, LLM) - Whisper.cpp für hochgenaue Transkription - Piper TTS für natürliche Sprachsynthese - Meeting-Protokoll-Generierung - Call Center Integration - DSGVO-konforme Speicherung

**Best Practices:** 1.  **Modell-Größe wählen** basierend auf Latenz-Anforderungen 2.  **Speaker Diarization** für Meeting-Protokolle aktivieren 3.  **Retention Policies** für DSGVO-Compliance setzen 4.  **GPU-Beschleunigung** für höheren Durchsatz nutzen 5.  **Horizontale Skalierung** mit dedizierten Voice-Worker-Nodes 6.  **Audit-Logging** für alle Zugriffe aktivieren 7.  **Verschlüsselung** at-rest und in-transit

**Anti-Patterns:** -  Large-Modell für Echtzeit-Interaktion (zu langsam) -  Recordings ohne Retention Policy speichern (DSGVO-Risiko) -  TTS ohne Rate-Limiting (DoS-Gefahr) -  Transkripte ohne Verschlüsselung (Security-Risiko)

**Weitere Ressourcen:** - Vollständige API-Dokumentation: [docs/de/features/sprachassistent\\_anleitung.md](#) - Whisper.cpp Dokumentation: [GitHub](#) - Piper TTS: [GitHub](#) - Kapitel 17: LLM-Integration für erweiterte NLP-Features

## 10.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- **Multi-Tenancy:** Verschiedene Ansätze und Best Practices
- **RBAC:** Rollen-basierte Zugriffskontrolle implementieren
- **Audit-Logging:** Vollständige Nachvollziehbarkeit aller Änderungen
- **Workflow-Management:** Geschäftsprozesse als Graph modellieren
- **DMS/ERP:** Dokumentenmanagement mit Versionierung und OCR
- **CRM:** Customer Relationship Management mit Lead-Scoring
- **Voice Assistant:** STT/TTS/LLM für Call Center und Meeting-Protokolle
- **Enterprise Best Practices:** Performance, Caching, Monitoring

### Wichtige Erkenntnisse:

1. **Multi-Model vereinfacht Enterprise-Apps** - Eine Datenbank für alle Anforderungen
2. **Graph-Modell für Workflows** - Prozesse als Graph modellieren
3. **Audit-Log als First-Class Citizen** - Nicht nachrüsten, von Anfang an einplanen
4. **Hybrid-Search unverzichtbar** - Volltext + Vector Search kombinieren
5. **Background-Jobs für schwere Operations** - API bleibt responsiv
6. **Voice Assistant für moderne UX** - Natürlichsprachliche Interaktion steigert Produktivität

## Übungen:

1. Erweitern Sie das DMS-System um E-Signaturen
2. Implementieren Sie Lead-Scoring mit Machine Learning
3. Erstellen Sie einen Workflow-Designer (GUI)
4. Integrieren Sie Email-Tracking ins CRM
5. Implementieren Sie Webhooks für externe Integrationen
6. Bauen Sie ein Meeting-Protokoll-System mit Voice Assistant

Im nächsten Kapitel tauchen wir in Realtime-Anwendungen ein: Chat und Kanban-Boards mit Live-Updates!

---

# Kapitel 14: Chapter 11: Realtime Data Streaming mit Change Data Capture

---

## 11.1 Einführung: Die Streaming-Architektur

Change Data Capture (CDC) ist eine kritische Komponente moderner Datenarchitekturen, die Echtzeit-Datenströme ermöglicht. ThemisDB implementiert CDC als natives Feature, das automatisch alle Mutationen (CREATE, UPDATE, DELETE) in einem append-only Event Log aufzeichnet. Dies ist ein fundamentaler Unterschied zu polyglot Systemen, die externe Tools wie Debezium benötigen, um Änderungen aus dem Write-Ahead Log (WAL) zu extrahieren.

### Warum CDC in ThemisDB?

**Architektonischer Vorteil:** Da ThemisDB alle Datenmodelle (relational, graph, document, vector) in einem einheitlichen "Base Entity"-Format speichert (siehe Chapter 2), kann ein einzelner CDC-Stream **alle** Datentypen erfassen. In einem Polyglot-Setup müsste man separate CDC-Pipelines für PostgreSQL, Neo4j und ChromaDB betreiben, was die Komplexität vervielfacht und die Konsistenz gefährdet.

**Use Cases:** 1. **Real-Time Analytics:** Streaming von Datenänderungen in ein OLAP-System (z.B. ClickHouse) 2. **Event Sourcing:** Rekonstruktion des Anwendungszustands aus dem Event Log 3. **Materialized Views:** Automatische Aktualisierung von Aggregationen 4. **Audit Trails:** BSI-konforme Nachvollziehbarkeit aller Datenänderungen 5. **Cross-System Sync:** Spiegelung von Daten in externe Systeme (Elasticsearch, Redis Cache) 6. **Kafka Integration:** Streaming in ein zentrales Event-Bus-System

```

graph TB
 subgraph "ThemisDB CDC Architecture"
 App[Application] -->|Write| DB[(ThemisDB
Multi-Model Storage)]

 DB -->|Automatically Capture| CDC[CDC Event Log
Append-Only]

 CDC -->|Stream 1| Analytics[(ClickHouse
OLAP Analytics)]
 CDC -->|Stream 2| Search[(Elasticsearch
Full-Text Search)]
 CDC -->|Stream 3| Cache[(Redis
Cache Layer)]
 CDC -->|Stream 4| Kafka[Kafka
Event Bus]
 CDC -->|Stream 5| Audit[Audit System
Compliance]
 end

 style DB fill:#667eea
 style CDC fill:#f093fb
 style Analytics fill:#43e97b
 style Search fill:#4facfe
 style Cache fill:#ffd32a
 style Kafka fill:#ff6348
 style Audit fill:#95e1d3

```

Abb. 11.1: Real-time-Streaming-Architektur

## 11.2 CDC-Architektur und Datenmodell

### 11.2.1 Event-Struktur

Jedes CDC-Event in ThemisDB folgt einem standardisierten Schema, das maximale Flexibilität bietet:

```

{
 "sequence": 42,
 "type": "PUT",
 "key": "user:alice",
 "value": "{\"name\":\"Alice\",\"email\":\"alice@example.com\"}",
 "timestamp_ms": 1730294567123,
 "metadata": {
 "table": "user",
 "pk": "alice",
 "operation_type": "update",
 "user_id": "admin@example.com"
 }
}

```

## Feld-Semantik:

- **sequence** (uint64): Monoton steigende ID, die **strikte Ordnung** garantiert. Diese Eigenschaft ist kritisch für die Konsistenz: Consumer können sicher sein, dass Event N vor Event N+1 aufgetreten ist. Die Sequence-Nummer wird atomar in RocksDB verwaltet ( `changefeed_sequence` - Schlüssel) und nutzt RocksDB's MVCC-Garantien.
- **type** (enum): `PUT` (Create/Update) oder `DELETE`. Zukünftige Erweiterungen könnten `TRANSACTION_COMMIT` / `TRANSACTION_ROLLBACK` für Transaktionsgrenzen hinzufügen, was Event Sourcing-Patterns vereinfachen würde.
- **key** (string): Vollständiger Entitätsschlüssel im Format `collection:primary_key`. Dies ermöglicht effiziente Filterung nach Präfixen (z.B. alle User-Events via `user:`-Filter).
- **value** (string|null): Bei PUT: JSON-seralisierte Entität. Bei DELETE: `null`. Die Speicherung als String (nicht als natives JSON-Objekt) minimiert den Serialisierungsaufwand im Hot Path. Consumer können mit `simdjson/serde_json` parsen.
- **timestamp\_ms** (uint64): Millisekunden seit Unix Epoch. Wichtig: Dieser Timestamp basiert auf der Serverzeit zum Zeitpunkt des Commits, nicht auf Client-Zeit. Für verteilte Systeme sollte NTP-Synchronisation sichergestellt sein.
- **metadata** (JSON object): Frei erweiterbar. Standardfelder ( `table` , `pk` ) werden automatisch gesetzt. Anwendungen können zusätzliche Kontextinformationen hinzufügen (z.B. `user_id` für Audit-Trails, `source_ip` für Sicherheitsanalysen).

### 11.2.2 Physische Speicherung

CDC-Events werden im selben RocksDB-Backend wie die Base Entities gespeichert, jedoch in einem separaten Schlüsselraum:

**Key-Format:** `changefeed:{sequence_number}`

Die Sequence-Nummer wird Zero-padded (z.B. `changefeed:0000000000000042`), um lexikographische Sortierung zu garantieren. Dies ist kritisch für die Scan-Performance: Ein RocksDB-Iterator kann effizient von `changefeed:{from_seq}` bis `changefeed:{to_seq}` iterieren, ohne die Daten neu sortieren zu müssen.

#### Column Family Strategy:

- **Default:** Events werden in der Default Column Family gespeichert. Vorteil: Atomare Transaktionen mit den Base Entities sind trivial (ein RocksDB WriteBatch kann beide aktualisieren). Nachteil: Events und Entitäten konkurrieren um den Block Cache.

- **Dedicated CF (zukünftig):** Eine separate Column Family ( `cf_changefeed` ) würde isolierte Tuning-Parameter erlauben (z.B. aggressivere Compaction für Events, da sie nie aktualisiert werden).

### 11.2.3 Sequence-Management

Die atomare Sequenzvergabe ist der kritische Pfad für CDC-Schreiboperationen:

```
// Pseudo-Code: Atomare Sequence-Vergabe
rocksdb::WriteBatch batch;
uint64_t seq = increment_sequence_counter(batch); // Atomic increment
std::string event_key = format("changefeed:{:020d}", seq);
batch.Put(event_key, serialize_event(event));
db->Write(batch); // ACID: Sequence & Event atomar committed
```

```
sequenceDiagram
 participant App as Application
 participant TM as Transaction Manager
 participant CDC as CDC Logger
 participant RDB as RocksDB

 App->>TM: BEGIN TRANSACTION
 TM->>App: transaction_id

 App->>TM: UPDATE users SET name='Alice'
 Note over TM: Sammelt Änderungen

 App->>TM: INSERT INTO orders ...
 Note over TM: Sammelt weitere Änderungen

 App->>TM: COMMIT

 TM->>CDC: Erstelle CDC Events
 CDC->>CDC: Generiere Sequence Numbers
(atomar)

 CDC->>RDB: WriteBatch {
 Base Entity Updates
 CDC Event #42
 CDC Event ...

 RDB-->>CDC: [OK] ATOMIC COMMIT
 CDC-->>TM: [OK] Events persistent
 TM-->>App: [OK] COMMIT erfolgreich

 Note over RDB: ALLE Änderungen atomar:
Base Entities + CDC Events
```

**Abb. 11.2:** Change-Stream-Processing

**Performance-Trade-off:** Die zentrale Sequenzvergabe ist ein Bottleneck bei hohen Schreibraten (>100K writes/sec). Zukünftige Optimierungen:

1. **Batch Allocation:** Reserviere Sequence-Blöcke (z.B. 1.000 Sequences auf einmal), reduziere Contentions um Faktor 1000.
2. **Sharded Sequences:** In Sharding-Setups (Chapter 16) kann jeder Shard eigene Sequences verwalten. Globale Ordnung wird via `(shard_id, local_sequence)` Tupel erreicht.

## CDC-Optionen

```
stream = db.cdc.create_stream(
 name="user_updates",
 table="users",
 columns=["status", "last_seen"], # Nur diese Felder
 operations=["UPDATE"]
)

stream = db.cdc.create_stream(
 name="important_messages",
 table="messages",
 filter="priority = 'high'",
 operations=["INSERT"]
)

stream = db.cdc.create_stream(
 name="enriched_events",
 table="orders",
 transform=lambda event: {
 **event,
 "customer_name": db.query("""
 FOR customer IN customers
 FILTER customer.id == @customer_id
 LIMIT 1
 RETURN customer.name
 """, {"customer_id": event.data["customer_id"]})[0]
 }
)
```

## Server-Sent Events (SSE)

SSE ermöglicht es, Daten vom Server an Clients zu pushen.

## SSE-Server-Setup

```
from flask import Flask, Response
from themisdb import ThemisDB
import json
import time

app = Flask(__name__)
db = ThemisDB()

def event_stream(channel):
 """Generator für SSE-Events"""
 stream = db.cdc.create_stream(
 name=f"sse_{channel}",
 table=channel,
 operations=["INSERT", "UPDATE", "DELETE"]
)

 for event in stream:
 # SSE-Format: "data: JSON\n\n"
 yield f"data: {json.dumps(event.to_dict())}\n\n"

@app.route('/stream/<channel>')
def stream(channel):
 return Response(
 event_stream(channel),
 mimetype="text/event-stream",
 headers={
 "Cache-Control": "no-cache",
 "Connection": "keep-alive",
 "X-Accel-Buffering": "no" # Nginx
 }
)
```



## SSE-Client-Code

```
// JavaScript Client
const eventSource = new EventSource('/stream/messages');

eventSource.onmessage = function(event) {
 const data = JSON.parse(event.data);

 if (data.operation === 'INSERT') {
 addMessageToUI(data.after);
 } else if (data.operation === 'UPDATE') {
 updateMessageInUI(data.after);
 } else if (data.operation === 'DELETE') {
 removeMessageFromUI(data.before.id);
 }
};

eventSource.onerror = function(error) {
 console.error('SSE error:', error);
 // Automatische Reconnection
};
```

## WebSocket-Integration

Für bidirektionale Kommunikation sind WebSockets ideal.

## **WebSocket-Server**

---

```

from flask_socketio import SocketIO, emit, join_room, leave_room
from themisdb import ThemisDB

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

active_users = {}

@socketio.on('connect')
def handle_connect():
 print(f'Client connected: {request.sid}')

@socketio.on('join_channel')
def handle_join(data):
 channel = data['channel']
 username = data['username']

 join_room(channel)
 active_users[request.sid] = {
 'username': username,
 'channel': channel
 }

 # Benachrichtige andere
 emit('user_joined', {
 'username': username,
 'timestamp': time.time()
 }, room=channel, skip_sid=request.sid)

 # CDC-Stream für diesen Channel
 start_cdc_stream(channel)

def start_cdc_stream(channel):
 """Background-Thread für CDC → WebSocket"""
 stream = db.cdc.create_stream(
 name=f"ws_{channel}",
 table="messages",
 filter=f"channel = '{channel}'"
)

 def emit_changes():
 for event in stream:
 socketio.emit('message_update',
 event.to_dict(),
 room=channel)

 # In separatem Thread
 socketio.start_background_task(emit_changes)

```

```

@socketio.on('send_message')
def handle_message(data):
 channel = active_users[request.sid]['channel']
 username = active_users[request.sid]['username']

 # In DB schreiben
 db.execute("""
 INSERT {
 channel: @channel,
 username: @username,
 content: @content,
 timestamp: @timestamp
 } INTO messages
 """, {"channel": channel, "username": username, "content": data['content'], "timestamp": data['timestamp']})

 # CDC wird automatisch notifizieren

```

## WebSocket-Client

```

const socket = io('http://localhost:5000');

// Channel beitreten
socket.emit('join_channel', {
 channel: 'general',
 username: 'Alice'
});

// Nachrichten senden
function sendMessage(content) {
 socket.emit('send_message', {
 content: content
 });
}

// Nachrichten empfangen
socket.on('message_update', function(event) {
 if (event.operation === 'INSERT') {
 displayMessage(event.after);
 }
});

// Nutzer beigetreten
socket.on('user_joined', function(data) {
 console.log(`${data.username} ist beigetreten`);
});

```

## **Example: Realtime Chat (examples/18\_realtime\_chat)**

Vollständige Chat-Anwendung mit Channels, Private Messages, Online-Status und Typing-Indikatoren.

## **Datenmodell**

```

from themisdb import ThemisDB
from datetime import datetime

db = ThemisDB()

db.execute("""
 CREATE TABLE users (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 username TEXT UNIQUE NOT NULL,
 email TEXT UNIQUE NOT NULL,
 avatar_url TEXT,
 status TEXT DEFAULT 'offline', -- online, offline, away
 last_seen TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

db.execute("""
 CREATE TABLE channels (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT UNIQUE NOT NULL,
 description TEXT,
 is_private BOOLEAN DEFAULT FALSE,
 created_by INTEGER REFERENCES users(id),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

db.execute("""
 CREATE TABLE channel_members (
 channel_id INTEGER REFERENCES channels(id),
 user_id INTEGER REFERENCES users(id),
 joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 role TEXT DEFAULT 'member', -- admin, moderator, member
 PRIMARY KEY (channel_id, user_id)
)
""")

db.execute("""
 CREATE TABLE messages (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 channel_id INTEGER REFERENCES channels(id),
 user_id INTEGER REFERENCES users(id),
 content TEXT NOT NULL,
 message_type TEXT DEFAULT 'text', -- text, image, file, system
 reply_to INTEGER REFERENCES messages(id),
 edited_at TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_channel_time (channel_id, created_at),

```

```

 INDEX idx_user (user_id)
)
 """)

db.execute("""
 CREATE TABLE reactions (
 message_id INTEGER REFERENCES messages(id),
 user_id INTEGER REFERENCES users(id),
 emoji TEXT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 PRIMARY KEY (message_id, user_id, emoji)
)
 """)

db.execute("""
 CREATE TABLE direct_messages (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 from_user_id INTEGER REFERENCES users(id),
 to_user_id INTEGER REFERENCES users(id),
 content TEXT NOT NULL,
 read_at TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_participants (from_user_id, to_user_id, created_at)
)
 """)

db.execute("""
 CREATE TABLE typing_indicators (
 channel_id INTEGER REFERENCES channels(id),
 user_id INTEGER REFERENCES users(id),
 started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 PRIMARY KEY (channel_id, user_id)
)
 """)

```



## **Chat-Backend**

```

from flask import Flask, request, jsonify
from flask_socketio import SocketIO, emit, join_room, leave_room
from flask_cors import CORS
from themisdb import ThemisDB
import time
from datetime import datetime, timedelta

app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

connections = {} # sid -> {user_id, username, channels}

class ChatService:
 @staticmethod
 def get_channel_messages(channel_id, limit=50, before_id=None):
 """Nachrichten laden (mit Pagination)"""
 query = """
 FOR message IN messages
 FILTER message.channel_id == @channel_id
 FOR user IN users
 FILTER message.user_id == user.id
 LET reaction_count = LENGTH(
 FOR reaction IN reactions
 FILTER reaction.message_id == message.id
 RETURN 1
)
 RETURN {
 id: message.id,
 content: message.content,
 message_type: message.message_type,
 reply_to: message.reply_to,
 created_at: message.created_at,
 edited_at: message.edited_at,
 user_id: user.id,
 username: user.username,
 avatar_url: user.avatar_url,
 reaction_count
 }
 """
 params = {"channel_id": channel_id}

 if before_id:
 query += " AND m.id < ?"
 params.append(before_id)

 query += " ORDER BY m.created_at DESC LIMIT ?"
 params.append(limit)

```

```

 return db.query(query, params)

 @staticmethod
 def send_message(channel_id, user_id, content, reply_to=None):
 """Nachricht senden"""
 with db.transaction():
 result = db.execute("""
 INSERT {
 channel_id: @channel_id,
 user_id: @user_id,
 content: @content,
 reply_to: @reply_to
 } INTO messages
 RETURN NEW
 """, {"channel_id": channel_id, "user_id": user_id, "content": content, "reply_to": reply_to})

 message = result[0]

 # Typing-Indikator löschen
 db.execute("""
 FOR indicator IN typing_indicators
 FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_id
 REMOVE indicator IN typing_indicators
 """, {"channel_id": channel_id, "user_id": user_id})

 return message

 @staticmethod
 def edit_message(message_id, user_id, new_content):
 """Nachricht bearbeiten"""
 result = db.execute("""
 UPDATE messages
 SET content = ?, edited_at = CURRENT_TIMESTAMP
 WHERE id = ? AND user_id = ?
 RETURNING *
 """, [new_content, message_id, user_id])

 if not result:
 raise ValueError("Message not found or not owned by user")

 return result[0]

 @staticmethod
 def delete_message(message_id, user_id):
 """Nachricht löschen"""
 db.execute("""
 FOR message IN messages
 FILTER message.id == @message_id AND message.user_id == @user_id
 REMOVE message IN messages
 """, {"message_id": message_id, "user_id": user_id})

```

```

 """, {"message_id": message_id, "user_id": user_id})

 @staticmethod
 def add_reaction(message_id, user_id, emoji):
 """Reaktion hinzufügen"""
 try:
 db.execute("""
 INSERT {
 message_id: @message_id,
 user_id: @user_id,
 emoji: @emoji
 } INTO reactions
 """, {"message_id": message_id, "user_id": user_id, "emoji": emoji})
 return True
 except: # Already exists
 return False

 @staticmethod
 def get_online_users(channel_id):
 """Online-Nutzer in einem Channel"""
 return db.query("""
 SELECT DISTINCT u.id, u.username, u.avatar_url, u.status
 FROM users u
 JOIN channel_members cm ON u.id = cm.user_id
 WHERE cm.channel_id = ?
 AND u.status = 'online'
 AND u.last_seen > ?
 """, [channel_id, datetime.now() - timedelta(minutes=5)])

 @staticmethod
 def update_user_status(user_id, status):
 """Status aktualisieren"""
 db.execute("""
 UPDATE users
 SET status = ?, last_seen = CURRENT_TIMESTAMP
 WHERE id = ?
 """, [status, user_id])

 @socketio.on('connect')
 def handle_connect():
 print(f'Client connected: {request.sid}')

 @socketio.on('authenticate')
 def handle_authenticate(data):
 """Nutzer authentifizieren"""
 user_id = data['user_id']
 username = data['username']

 connections[request.sid] = {
 'user_id': user_id,

```

```

 'username': username,
 'channels': set()
 }

 # Status auf online setzen
 ChatService.update_user_status(user_id, 'online')

 emit('authenticated', {'success': True})

@socketio.on('join_channel')
def handle_join_channel(data):
 """Channel beitreten"""
 channel_id = data['channel_id']

 if request.sid not in connections:
 return emit('error', {'message': 'Not authenticated'})

 conn = connections[request.sid]
 join_room(f'channel_{channel_id}')
 conn['channels'].add(channel_id)

 # Online-Nutzer benachrichtigen
 emit('user_joined', {
 'user_id': conn['user_id'],
 'username': conn['username'],
 'channel_id': channel_id
 }, room=f'channel_{channel_id}', skip_sid=request.sid)

 # Initiale Nachrichten laden
 messages = ChatService.get_channel_messages(channel_id)
 emit('channel_history', {
 'channel_id': channel_id,
 'messages': messages
 })

 # CDC-Stream für diesen Channel (wenn noch nicht aktiv)
 start_channel_cdc(channel_id)

@socketio.on('leave_channel')
def handle_leave_channel(data):
 """Channel verlassen"""
 channel_id = data['channel_id']

 if request.sid in connections:
 conn = connections[request.sid]
 leave_room(f'channel_{channel_id}')
 conn['channels'].discard(channel_id)

 emit('user_left', {
 'user_id': conn['user_id'],

```

```

 'username': conn['username'],
 'channel_id': channel_id
 }, room=f'channel_{channel_id}')

@socketio.on('send_message')
def handle_send_message(data):
 """Nachricht senden"""
 if request.sid not in connections:
 return emit('error', {'message': 'Not authenticated'})

 conn = connections[request.sid]
 channel_id = data['channel_id']
 content = data['content']
 reply_to = data.get('reply_to')

 # In DB schreiben
 message = ChatService.send_message(
 channel_id, conn['user_id'], content, reply_to
)

 # CDC wird automatisch andere Clients benachrichtigen

@socketio.on('edit_message')
def handle_edit_message(data):
 """Nachricht bearbeiten"""
 if request.sid not in connections:
 return

 conn = connections[request.sid]
 message_id = data['message_id']
 new_content = data['content']

 try:
 message = ChatService.edit_message(
 message_id, conn['user_id'], new_content
)
 # CDC benachrichtigt automatisch
 except ValueError as e:
 emit('error', {'message': str(e)})

@socketio.on('typing_start')
def handle_typing_start(data):
 """Typing-Indikator starten"""
 if request.sid not in connections:
 return

 conn = connections[request.sid]
 channel_id = data['channel_id']

 # In DB schreiben (mit TTL)

```

```

db.execute("""
 INSERT OR REPLACE INTO typing_indicators (channel_id, user_id)
 VALUES (?, ?)
""", [channel_id, conn['user_id']])

An andere broadcasten
emit('user_typing', {
 'user_id': conn['user_id'],
 'username': conn['username'],
 'channel_id': channel_id
}, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('typing_stop')
def handle_typing_stop(data):
 """Typing-Indikator stoppen"""
 if request.sid not in connections:
 return

 conn = connections[request.sid]
 channel_id = data['channel_id']

 db.execute("""
 FOR indicator IN typing_indicators
 FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_id
 REMOVE indicator IN typing_indicators
 """, {"channel_id": channel_id, "user_id": conn['user_id']})

 emit('user_stopped_typing', {
 'user_id': conn['user_id'],
 'channel_id': channel_id
 }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('add_reaction')
def handle_add_reaction(data):
 """Reaktion hinzufügen"""
 if request.sid not in connections:
 return

 conn = connections[request.sid]
 message_id = data['message_id']
 emoji = data['emoji']

 if ChatService.add_reaction(message_id, conn['user_id'], emoji):
 # CDC benachrichtigt automatisch
 pass

@socketio.on('disconnect')
def handle_disconnect():
 """Client getrennt"""
 if request.sid in connections:

```

```

conn = connections[request.sid]

Status auf offline
ChatService.update_user_status(conn['user_id'], 'offline')

Alle Channels benachrichtigen
for channel_id in conn['channels']:
 emit('user_left', {
 'user_id': conn['user_id'],
 'username': conn['username'],
 'channel_id': channel_id
 }, room=f'channel_{channel_id}')

del connections[request.sid]

active_cdc_streams = set()

def start_channel_cdc(channel_id):
 """CDC-Stream für Channel starten"""
 if channel_id in active_cdc_streams:
 return

 active_cdc_streams.add(channel_id)

def cdc_worker():
 stream = db.cdc.create_stream(
 name=f"chat_channel_{channel_id}",
 table="messages",
 filter=f"channel_id = {channel_id}",
 operations=["INSERT", "UPDATE", "DELETE"]
)

 for event in stream:
 if event.operation == 'INSERT':
 # Neue Nachricht
 socketio.emit('new_message', {
 'message': event.after
 }, room=f'channel_{channel_id}')

 elif event.operation == 'UPDATE':
 # Nachricht bearbeitet
 socketio.emit('message_edited', {
 'message': event.after
 }, room=f'channel_{channel_id}')

 elif event.operation == 'DELETE':
 # Nachricht gelöscht
 socketio.emit('message_deleted', {
 'message_id': event.before['id']
 }, room=f'channel_{channel_id}')

```



```

In Background-Thread
socketio.start_background_task(cdc_worker)

@app.route('/api/channels', methods=['GET'])
def get_channels():
 """Alle Channels abrufen"""
 channels = db.query("""
 SELECT c.id, c.name, c.description, c.is_private,
 COUNT(DISTINCT cm.user_id) as member_count,
 MAX(m.created_at) as last_message_at
 FROM channels c
 LEFT JOIN channel_members cm ON c.id = cm.channel_id
 LEFT JOIN messages m ON c.id = m.channel_id
 GROUP BY c.id
 ORDER BY last_message_at DESC NULLS LAST
 """)
 return jsonify(channels)

@app.route('/api/channels/<int:channel_id>/members', methods=['GET'])
def get_channel_members(channel_id):
 """Channel-Mitglieder"""
 members = db.query("""
 SELECT u.id, u.username, u.avatar_url, u.status,
 cm.role, cm.joined_at
 FROM users u
 JOIN channel_members cm ON u.id = cm.user_id
 WHERE cm.channel_id = ?
 ORDER BY cm.joined_at
 """, [channel_id])
 return jsonify(members)

if __name__ == '__main__':
 socketio.run(app, host='0.0.0.0', port=5000, debug=True)

```

## **Chat-Frontend (React)**

```

// ChatApp.jsx
import React, { useState, useEffect, useRef } from 'react';
import io from 'socket.io-client';

const ChatApp = ({ userId, username }) => {
 const [socket, setSocket] = useState(null);
 const [channels, setChannels] = useState([]);
 const [activeChannel, setActiveChannel] = useState(null);
 const [messages, setMessages] = useState([]);
 const [newMessage, setNewMessage] = useState('');
 const [typingUsers, setTypingUsers] = useState(new Set());
 const [onlineUsers, setOnlineUsers] = useState([]);

 const messagesEndRef = useRef(null);
 const typingTimeoutRef = useRef(null);

 useEffect(() => {
 // Socket-Verbindung
 const newSocket = io('http://localhost:5000');
 setSocket(newSocket);

 // Authentifizieren
 newSocket.emit('authenticate', { user_id: userId, username });

 // Event-Handler
 newSocket.on('authenticated', () => {
 console.log('Authenticated');
 loadChannels();
 });

 newSocket.on('channel_history', (data) => {
 setMessages(data.messages.reverse());
 });

 newSocket.on('new_message', (data) => {
 setMessages(prev => [...prev, data.message]);
 scrollToBottom();
 });

 newSocket.on('message_edited', (data) => {
 setMessages(prev => prev.map(msg =>
 msg.id === data.message.id ? data.message : msg
));
 });

 newSocket.on('message_deleted', (data) => {
 setMessages(prev => prev.filter(msg => msg.id !== data.message_id));
 });
 });

```

```

newSocket.on('user_typing', (data) => {
 setTypingUsers(prev => new Set([...prev, data.username]));
});

newSocket.on('user_stopped_typing', (data) => {
 setTypingUsers(prev => {
 const next = new Set(prev);
 next.delete(data.username);
 return next;
 });
});

newSocket.on('user_joined', (data) => {
 console.log(`${data.username} joined`);
 setOnlineUsers(prev => [...prev, data]);
});

newSocket.on('user_left', (data) => {
 console.log(`${data.username} left`);
 setOnlineUsers(prev => prev.filter(u => u.user_id !== data.user_id));
});

return () => newSocket.close();
}, [userId, username]);

const loadChannels = async () => {
 const response = await fetch('http://localhost:5000/api/channels');
 const data = await response.json();
 setChannels(data);
 if (data.length > 0) {
 joinChannel(data[0].id);
 }
};

const joinChannel = (channelId) => {
 if (activeChannel) {
 socket.emit('leave_channel', { channel_id: activeChannel });
 }
 socket.emit('join_channel', { channel_id: channelId });
 setActiveChannel(channelId);
 setMessages([]);
};

const sendMessage = () => {
 if (!newMessage.trim()) return;

 socket.emit('send_message', {
 channel_id: activeChannel,
 content: newMessage
 });
};

```

```

 setNewMessage('');
 stopTyping();
 };

const handleTyping = () => {
 socket.emit('typing_start', { channel_id: activeChannel });

 // Auto-stop nach 3 Sekunden
 clearTimeout(typingTimeoutRef.current);
 typingTimeoutRef.current = setTimeout(() => {
 stopTyping();
 }, 3000);
};

const stopTyping = () => {
 socket.emit('typing_stop', { channel_id: activeChannel });
 clearTimeout(typingTimeoutRef.current);
};

const scrollToBottom = () => {
 messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
};

return (
 <div className="chat-app">
 <div className="sidebar">
 <h3>Channels</h3>
 {channels.map(channel => (
 <div
 key={channel.id}
 className={`channel ${activeChannel === channel.id ? 'active' : ''}`}
 onClick={() => joinChannel(channel.id)}
 >
 # {channel.name}
 {channel.member_count}
 </div>
))}
 </div>

 <div className="main">
 <div className="header">
 <h2>#{channels.find(c => c.id === activeChannel)?.name}</h2>
 <div className="online-users">
 {onlineUsers.length} online
 </div>
 </div>

 <div className="messages">
 {messages.map(msg => (

```

```

 <div key={msg.id} className="message">

 <div>
 {msg.username}

 {new Date(msg.created_at).toLocaleTimeString()}

 <p>{msg.content}</p>
 {msg.edited_at && (edited)}
 </div>
 </div>
)}}
 <div ref={messagesEndRef} />
 </div>

 {typingUsers.size > 0 && (
 <div className="typing-indicator">
 {Array.from(typingUsers).join(', ')} {typingUsers.size === 1 ? 'i
 </div>
)}

 <div className="input-area">
 <input
 type="text"
 value={newMessage}
 onChange={(e) => {
 setNewMessage(e.target.value);
 handleTyping();
 }}
 onPress={e => e.key === 'Enter' && sendMessage()}
 placeholder="Type a message..."
 />
 <button onClick={sendMessage}>Send</button>
 </div>
</div>
</div>
);
};

export default ChatApp;

```

## Chat-Features

Das vollständige Chat-Example demonstriert:

1. **Echtzeit-Nachrichten:** Sofortige Zustellung via WebSocket + CDC
2. **Online-Status:** Wer ist gerade online?
3. **Typing-Indikatoren:** "Alice is typing..."

4. **Reaktionen:** Emoji-Reactions auf Nachrichten
5. **Threads:** Antworten auf bestimmte Nachrichten
6. **Nachrichtенbearbeitung:** Edit-History mit Timestamps
7. **Private Channels:** Eingeschränkter Zugriff
8. **Direktnachrichten:** 1-on-1 Chats
9. **Presence:** Last-Seen und Away-Status
10. **Pagination:** Unendliches Scrollen durch Historie

## **Example: Kanban Board (examples/16\_kanban\_board)**

Vollständige Kanban-Board-Anwendung mit Drag & Drop, Realtime-Updates und Team-Collaboration.

## **Datenmodell**



```

from themisdb import ThemisDB

db = ThemisDB()

db.execute("""
 CREATE TABLE boards (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 description TEXT,
 created_by INTEGER REFERENCES users(id),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

db.execute("""
 CREATE TABLE columns (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 board_id INTEGER REFERENCES boards(id),
 name TEXT NOT NULL,
 position INTEGER NOT NULL,
 wip_limit INTEGER, -- Work-In-Progress Limit
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_board (board_id, position)
)
""")

db.execute("""
 CREATE TABLE cards (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 column_id INTEGER REFERENCES columns(id),
 board_id INTEGER REFERENCES boards(id),
 title TEXT NOT NULL,
 description TEXT,
 assigned_to INTEGER REFERENCES users(id),
 position INTEGER NOT NULL,
 priority TEXT DEFAULT 'medium', -- low, medium, high, urgent
 due_date TIMESTAMP,
 created_by INTEGER REFERENCES users(id),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_column (column_id, position),
 INDEX idx_board (board_id)
)
""")

db.execute("""
 CREATE TABLE card_activities (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 card_id INTEGER REFERENCES cards(id),

```

```
 user_id INTEGER REFERENCES users(id),
 action_type TEXT NOT NULL, -- created, moved, assigned, commented
 details JSON,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_card (card_id, created_at)
)
 """)

db.execute("""
 CREATE TABLE card_labels (
 card_id INTEGER REFERENCES cards(id),
 label TEXT NOT NULL,
 color TEXT,
 PRIMARY KEY (card_id, label)
)
 """)
```

## **Kanban-Backend**

```

from flask import Flask, jsonify, request
from flask_socketio import SocketIO, emit, join_room
from themisdb import ThemisDB
import time

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

class KanbanService:
 @staticmethod
 def get_board(board_id):
 """Board mit allen Spalten und Karten laden"""
 board = db.query("""
 FOR board IN boards
 FILTER board.id == @board_id
 LIMIT 1
 RETURN board
 """, {"board_id": board_id})[0]

 columns = db.query("""
 FOR column IN columns
 FILTER column.board_id == @board_id
 SORT column.position ASC
 RETURN column
 """, {"board_id": board_id})

 for col in columns:
 col['cards'] = db.query("""
 FOR card IN cards
 FILTER card.column_id == @column_id
 LET assigned_name = (
 FOR user IN users
 FILTER card.assigned_to == user.id
 LIMIT 1
 RETURN user.username
)[0]
 SORT card.position ASC
 RETURN MERGE(card, {assigned_to_name: assigned_name})
 """, {"column_id": col['id']})

 board['columns'] = columns
 return board

 @staticmethod
 def move_card(card_id, to_column_id, to_position):
 """Karte verschieben"""
 with db.transaction():
 # Aktuelle Position

```

```

current = db.query("""
 FOR card IN cards
 FILTER card.id == @card_id
 LIMIT 1
 RETURN {column_id: card.column_id, position: card.position}
""", {"card_id": card_id})[0]

from_column_id = current['column_id']
from_position = current['position']

Position in alter Spalte aktualisieren
db.execute("""
 UPDATE cards
 SET position = position - 1
 WHERE column_id = ? AND position > ?
""", [from_column_id, from_position])

Position in neuer Spalte freimachen
db.execute("""
 UPDATE cards
 SET position = position + 1
 WHERE column_id = ? AND position >= ?
""", [to_column_id, to_position])

Karte verschieben
db.execute("""
 UPDATE cards
 SET column_id = ?, position = ?, updated_at = CURRENT_TIMESTAMP
 WHERE id = ?
""", [to_column_id, to_position, card_id])

Activity loggen
db.execute("""
 INSERT INTO card_activities (card_id, user_id, action_type, details)
 VALUES (?, ?, 'moved', ?)
""", [card_id, request.user_id, json.dumps({
 'from_column': from_column_id,
 'to_column': to_column_id
})])

@staticmethod
def create_card(board_id, column_id, title, description, assigned_to=None):
 """Neue Karte erstellen"""
 with db.transaction():
 # Position am Ende der Spalte
 position = db.query("""
 FOR card IN cards
 FILTER card.column_id == @column_id
 COLLECT AGGREGATE max_pos = MAX(card.position)
 RETURN COALESCE(max_pos, -1) + 1
 """, {"column_id": column_id})[0]

```

```

 """, {"column_id": column_id}][0]

 result = db.execute("""
 INSERT {
 board_id: @board_id,
 column_id: @column_id,
 title: @title,
 description: @description,
 assigned_to: @assigned_to,
 position: @position,
 created_by: @created_by
 } INTO cards
 RETURN NEW
 """, {
 "board_id": board_id,
 "column_id": column_id,
 "title": title,
 "description": description,
 "assigned_to": assigned_to,
 "position": position,
 "created_by": request.user_id
 })

 card = result[0]

 # Activity
 db.execute("""
 INSERT {
 card_id: @card_id,
 user_id: @user_id,
 action_type: 'created'
 } INTO card_activities
 """, {"card_id": card['id'], "user_id": request.user_id})

 return card

@staticmethod
def update_card(card_id, **updates):
 """Karte aktualisieren"""
 allowed_fields = ['title', 'description', 'assigned_to', 'priority', 'due_date']
 set_clause = ', '.join([f"{field} = ?" for field in updates if field in allowed_fields])
 values = [updates[field] for field in updates if field in allowed_fields]
 values.append(card_id)

 db.execute(f"""
 UPDATE cards
 SET {set_clause}, updated_at = CURRENT_TIMESTAMP
 WHERE id = ?
 """, values)

```

```

Activity
db.execute("""
 INSERT {
 card_id: @card_id,
 user_id: @user_id,
 action_type: 'updated',
 details: @details
 } INTO card_activities
""", {"card_id": card_id, "user_id": request.user_id, "details": json.dumps(updates)}

@socketio.on('join_board')
def handle_join_board(data):
 board_id = data['board_id']
 join_room(f'board_{board_id}')

 # Board-Daten senden
 board = KanbanService.get_board(board_id)
 emit('board_state', board)

 # CDC-Stream starten
 start_board_cdc(board_id)

@socketio.on('move_card')
def handle_move_card(data):
 card_id = data['card_id']
 to_column_id = data['to_column_id']
 to_position = data['to_position']
 board_id = data['board_id']

 try:
 KanbanService.move_card(card_id, to_column_id, to_position)
 # CDC benachrichtigt andere Clients
 except Exception as e:
 emit('error', {'message': str(e)})

@socketio.on('create_card')
def handle_create_card(data):
 card = KanbanService.create_card(
 data['board_id'],
 data['column_id'],
 data['title'],
 data.get('description'),
 data.get('assigned_to')
)
 # CDC benachrichtigt

@socketio.on('update_card')
def handle_update_card(data):
 card_id = data.pop('card_id')
 KanbanService.update_card(card_id, **data)

```

```

CDC benachrichtigt

active_board_streams = set()

def start_board_cdc(board_id):
 if board_id in active_board_streams:
 return

 active_board_streams.add(board_id)

def cdc_worker():
 # Cards-Stream
 cards_stream = db.cdc.create_stream(
 name=f"kanban_cards_{board_id}",
 table="cards",
 filter=f"board_id = {board_id}",
 operations=["INSERT", "UPDATE", "DELETE"]
)

 # Columns-Stream
 columns_stream = db.cdc.create_stream(
 name=f"kanban_columns_{board_id}",
 table="columns",
 filter=f"board_id = {board_id}",
 operations=["INSERT", "UPDATE", "DELETE"]
)

 # Beide Streams kombinieren
 for event in combined_stream([cards_stream, columns_stream]):
 if event.table == 'cards':
 if event.operation == 'INSERT':
 socketio.emit('card_created', event.after,
 room=f'board_{board_id}')
 elif event.operation == 'UPDATE':
 socketio.emit('card_updated', event.after,
 room=f'board_{board_id}')
 elif event.operation == 'DELETE':
 socketio.emit('card_deleted', {'id': event.before['id']},
 room=f'board_{board_id}')

 elif event.table == 'columns':
 # Spalten-Update
 socketio.emit('column_updated', event.after,
 room=f'board_{board_id}')

 socketio.start_background_task(cdc_worker)

def combined_stream(streams):
 """Mehrere CDC-Streams kombinieren"""
 import queue

```



```
import threading

q = queue.Queue()

def stream_worker(stream):
 for event in stream:
 q.put(event)

for stream in streams:
 threading.Thread(target=stream_worker, args=(stream,), daemon=True).start()

while True:
 yield q.get()

if __name__ == '__main__':
 socketio.run(app, host='0.0.0.0', port=5000)
```

## **Kanban-Frontend (React)**

```
// KanbanBoard.jsx
import React, { useState, useEffect } from 'react';
import { DragDropContext, Droppable, Draggable } from 'react-beautiful-dnd';
import io from 'socket.io-client';

const KanbanBoard = ({ boardId, userId }) => {
 const [socket, setSocket] = useState(null);
 const [board, setBoard] = useState(null);
 const [columns, setColumns] = useState([]);

 useEffect(() => {
 const newSocket = io('http://localhost:5000');
 setSocket(newSocket);

 newSocket.emit('join_board', { board_id: boardId });

 newSocket.on('board_state', (data) => {
 setBoard(data);
 setColumns(data.columns);
 });

 newSocket.on('card_created', (card) => {
 setColumns(prev => prev.map(col => {
 if (col.id === card.column_id) {
 return { ...col, cards: [...col.cards, card] };
 }
 return col;
 }));
 });

 newSocket.on('card_updated', (card) => {
 setColumns(prev => prev.map(col => ({
 ...col,
 cards: col.cards.map(c => c.id === card.id ? card : c)
 })));
 });

 newSocket.on('card_deleted', (data) => {
 setColumns(prev => prev.map(col => ({
 ...col,
 cards: col.cards.filter(c => c.id !== data.id)
 })));
 });

 return () => newSocket.close();
 }, [boardId]);

 const onDragEnd = (result) => {
 if (!result.destination) return;

```

```

const { source, destination } = result;

if (source.droppableId === destination.droppableId &&
 source.index === destination.index) {
 return;
}

// Optimistisches Update
const newColumns = [...columns];
const sourceColumn = newColumns.find(c => c.id === parseInt(source.droppableId));
const destColumn = newColumns.find(c => c.id === parseInt(destination.droppableId));

const [movedCard] = sourceColumn.cards.splice(source.index, 1);
destColumn.cards.splice(destination.index, 0, movedCard);

setColumns(newColumns);

// An Server senden
socket.emit('move_card', {
 card_id: parseInt(result.draggableId),
 to_column_id: parseInt(destination.droppableId),
 to_position: destination.index,
 board_id: boardId
});
};

if (!board) return <div>Loading...</div>;

return (
 <div className="kanban-board">
 <h1>{board.name}</h1>

 <DragDropContext onDragEnd={onDragEnd}>
 <div className="columns">
 {columns.map(column => (
 <div key={column.id} className="column">
 <h3>
 {column.name}
 {column.cards.length}
 {column.wip_limit && (
 column.wip_limit}>
 / {column.wip_limit}

)}
 </h3>

 <Draggable draggableId={column.id.toString()}>
 {(provided, snapshot) => (
 <div

```

```

 ref={provided.innerRef}
 {...provided.droppableProps}
 className={`cards ${snapshot.isDraggingOver ? 'dragging' : ''}`}
 >
 {column.cards.map((card, index) => (
 <Draggable
 key={card.id}
 draggableId={card.id.toString()}
 index={index}
 >
 {(provided, snapshot) => (
 <div
 ref={provided.innerRef}
 {...provided.draggableProps}
 {...provided.dragHandleProps}
 className={`card priority-${card.priority}`}
 >
 <h4>{card.title}</h4>
 {card.description && <p>{card.description}</p>}
 {card.assigned_to_name && (
 <div className="assignee">
  {card.assigned_to_name}
 </div>
)}
 {card.due_date && (
 <div className="due-date">
  {new Date(card.due_date).toLocaleDateString()}
 </div>
)}
 </div>
)}
 </Draggable>
)})}
 {provided.placeholder}
 </div>
)}
 </Droppable>
</div>
)]]}
</div>
</DragDropContext>
</div>
);
};

export default KanbanBoard;

```

## Kanban-Features

Das Kanban-Board-Example demonstriert:

1. **Drag & Drop:** Karten zwischen Spalten verschieben
2. **Realtime-Sync:** Alle Nutzer sehen Änderungen sofort
3. **WIP-Limits:** Work-In-Progress Beschränkungen
4. **Prioritäten:** Visuelle Kennzeichnung (low, medium, high, urgent)
5. **Zuweisungen:** Karten Teammitgliedern zuordnen
6. **Due Dates:** Fälligkeitsdaten mit Warnungen
7. **Labels:** Flexible Kategorisierung
8. **Activity-Log:** Vollständige Historie aller Änderungen
9. **Board-Analytics:** Durchlaufzeit, Cycle-Time, etc.
10. **Custom Columns:** Beliebige Workflow-Stufen

## | Event-Driven Architecture

Realtime-Anwendungen profitieren von event-driven Design.

## **Event-Bus-Pattern**

```

from themisdb import ThemisDB
from typing import Callable, List
import threading

class EventBus:
 def __init__(self, db: ThemisDB):
 self.db = db
 self.handlers = {} # event_type -> [handlers]
 self.running = False

 def subscribe(self, event_type: str, handler: Callable):
 """Handler für Event-Typ registrieren"""
 if event_type not in self.handlers:
 self.handlers[event_type] = []
 self.handlers[event_type].append(handler)

 def publish(self, event_type: str, data: dict):
 """Event publishen"""
 self.db.execute("""
 INSERT {
 event_type: @event_type,
 data: @data,
 created_at: DATE_NOW()
 } INTO events
 """, {"event_type": event_type, "data": json.dumps(data)})

 def start(self):
 """Event-Processing starten"""
 self.running = True

 def process_events():
 stream = self.db.cdc.create_stream(
 name="event_bus",
 table="events",
 operations=["INSERT"]
)

 for event in stream:
 if not self.running:
 break

 event_type = event.after['event_type']
 data = json.loads(event.after['data'])

 # Handler aufrufen
 if event_type in self.handlers:
 for handler in self.handlers[event_type]:
 try:
 handler(data)

```



```

 except Exception as e:
 print(f"Handler error: {e}")

 threading.Thread(target=process_events, daemon=True).start()

 def stop(self):
 self.running = False

bus = EventBus(db)

@bus.subscribe('user_registered')
def send_welcome_email(data):
 print(f"Sending email to {data['email']}")

@bus.subscribe('order_placed')
def update_inventory(data):
 print(f"Reducing stock for order {data['order_id']}")

bus.publish('user_registered', {
 'user_id': 123,
 'email': 'alice@example.com'
})

bus.start()

```

## CQRS-Pattern

Command Query Responsibility Segregation für Realtime-Apps:

```

class OrderService:
 def __init__(self, db: ThemisDB, event_bus: EventBus):
 self.db = db
 self.event_bus = event_bus

 # COMMAND: Write-Seite
 def place_order(self, user_id, items):
 with self.db.transaction():
 # Order erstellen
 order_id = self.db.execute("""
 INSERT {
 user_id: @user_id,
 status: 'pending',
 created_at: DATE_NOW()
 } INTO orders
 RETURN NEW.id
 """, {"user_id": user_id})[0]

 # Items
 for item in items:
 self.db.execute("""
 INSERT {
 order_id: @order_id,
 product_id: @product_id,
 quantity: @quantity,
 price: @price
 } INTO order_items
 """, {
 "order_id": order_id,
 "product_id": item['product_id'],
 "quantity": item['quantity'],
 "price": item['price']
 })

 # Event publishen
 self.event_bus.publish('order_placed', {
 'order_id': order_id,
 'user_id': user_id,
 'items': items
 })

 return order_id

 # QUERY: Read-Seite (materialized view)
 def get_order_summary(self, order_id):
 return self.db.query("""
 SELECT
 o.id, o.status, o.created_at,
 u.name as customer_name,

```

```

 COUNT(oi.id) as item_count,
 SUM(oi.quantity * oi.price) as total
 FROM orders o
 JOIN users u ON o.user_id = u.id
 LEFT JOIN order_items oi ON o.id = oi.order_id
 WHERE o.id = ?
 GROUP BY o.id
 """", [order_id])[0]

@event_bus.subscribe('order_placed')
def update_order_cache(data):
 # Cache/Read-Model aktualisieren
 pass

```

## Best Practices

### 1. Connection Management

```

class ConnectionPool:
 def __init__(self, max_connections=100):
 self.max_connections = max_connections
 self.active_connections = {}
 self.semaphore = threading.Semaphore(max_connections)

 def add_connection(self, sid, user_id):
 with self.semaphore:
 if len(self.active_connections) >= self.max_connections:
 raise ValueError("Too many connections")
 self.active_connections[sid] = user_id

 def remove_connection(self, sid):
 if sid in self.active_connections:
 del self.active_connections[sid]
 self.semaphore.release()

```

## 2. Rate Limiting

```
from collections import defaultdict
import time

class RateLimiter:
 def __init__(self, max_requests=10, window=60):
 self.max_requests = max_requests
 self.window = window
 self.requests = defaultdict(list)

 def allow(self, user_id):
 now = time.time()
 # Alte Einträge entfernen
 self.requests[user_id] = [
 t for t in self.requests[user_id]
 if now - t < self.window
]

 if len(self.requests[user_id]) >= self.max_requests:
 return False

 self.requests[user_id].append(now)
 return True

@socketio.on('send_message')
def handle_send_message(data):
 if not rate_limiter.allow(current_user_id):
 return emit('error', {'message': 'Rate limit exceeded'})
 # ... rest
```

### 3. Message Deduplication

```
class MessageDeduplicator:
 def __init__(self, ttl=60):
 self.seen = {} # message_id -> timestamp
 self.ttl = ttl

 def is_duplicate(self, message_id):
 now = time.time()

 # Cleanup
 self.seen = {
 mid: ts for mid, ts in self.seen.items()
 if now - ts < self.ttl
 }

 if message_id in self.seen:
 return True

 self.seen[message_id] = now
 return False
```

## 4. Graceful Shutdown

```
import signal
import sys

def graceful_shutdown(signum, frame):
 print("Shutting down gracefully...")

 # Alle Clients benachrichtigen
 socketio.emit('server_shutdown', {
 'message': 'Server ist down für Wartung',
 'reconnect_in': 60
 })

 # CDC-Streams stoppen
 for stream_id in active_cdc_streams:
 stop_cdc_stream(stream_id)

 # Connections schließen
 socketio.stop()

 sys.exit(0)

signal.signal(signal.SIGINT, graceful_shutdown)
signal.signal(signal.SIGTERM, graceful_shutdown)
```

## 5. Monitoring & Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import time

messages_sent = Counter('messages_sent_total', 'Total messages sent')
message_latency = Histogram('message_latency_seconds', 'Message delivery latency')
active_connections_gauge = Gauge('active_connections', 'Number of active WebSocket connections')

@socketio.on('send_message')
def handle_send_message(data):
 start_time = time.time()

 # ... message handling ...

 messages_sent.inc()
 message_latency.observe(time.time() - start_time)

@socketio.on('connect')
def handle_connect():
 active_connections_gauge.inc()

@socketio.on('disconnect')
def handle_disconnect():
 active_connections_gauge.dec()
```

## Performance-Optimierungen

### 1. Message-Batching

```
class MessageBatcher:
 def __init__(self, max_batch_size=100, max_wait_ms=100):
 self.max_batch_size = max_batch_size
 self.max_wait_ms = max_wait_ms
 self.batch = []
 self.last_flush = time.time()

 def add(self, message):
 self.batch.append(message)

 if len(self.batch) >= self.max_batch_size or \
 (time.time() - self.last_flush) * 1000 >= self.max_wait_ms:
 self.flush()

 def flush(self):
 if not self.batch:
 return

 # Bulk-Insert
 db.executemany("""
 INSERT INTO messages (channel_id, user_id, content)
 VALUES (?, ?, ?)
 """, [(m['channel_id'], m['user_id'], m['content']) for m in self.batch])

 self.batch = []
 self.last_flush = time.time()
```



## 2. Connection Pooling

```
from themisdb import ThemisDB, ConnectionPool

pool = ConnectionPool(
 min_size=10,
 max_size=50,
 max_idle_time=300
)

def get_db_connection():
 return pool.acquire()

def release_db_connection(conn):
 pool.release(conn)
```

### 3. Caching

```

from functools import lru_cache
import time

class CachedQuery:
 def __init__(self, ttl=60):
 self.cache = {}
 self.ttl = ttl

 def get(self, query, params):
 key = (query, tuple(params))

 if key in self.cache:
 value, timestamp = self.cache[key]
 if time.time() - timestamp < self.ttl:
 return value

 # Cache Miss
 result = db.query(query, params)
 self.cache[key] = (result, time.time())
 return result

cache = CachedQuery(ttl=30)

def get_channel_members(channel_id):
 return cache.get("""
 FOR member IN channel_members
 FILTER member.channel_id == @channel_id
 RETURN member
 """, {"channel_id": channel_id})

```

## Zusammenfassung

In diesem Kapitel haben Sie gelernt:

1. **Change Data Capture (CDC):** Datenänderungen verfolgen und darauf reagieren
2. **Server-Sent Events (SSE):** Unidirektionale Push-Updates
3. **WebSockets:** Bidirektionale Realtime-Kommunikation
4. **Event-Driven Architecture:** Entkoppelte, skalierbare Systeme
5. **Realtime Chat:** Vollständige Implementierung mit allen Features
6. **Kanban Board:** Collaborative Drag & Drop mit Sync
7. **Best Practices:** Connection Management, Rate Limiting, Monitoring
8. **Performance:** Batching, Pooling, Caching

---

Realtime-Anwendungen mit ThemisDB sind einfach zu implementieren dank: - Integriertem CDC-System - MVCC für konfliktfreie Reads - Flexible Event-Streaming-Mechanismen - Starke Transaktionsgarantien

Im nächsten Kapitel schauen wir uns Computer Vision-Anwendungen an, wo wir Bilder speichern, analysieren und durchsuchen.

## Übungen

1. **Chat-Erweiterung:** Fügen Sie Voice Messages und File Sharing hinzu
  2. **Kanban-Analytics:** Cycle Time und Lead Time berechnen
  3. **Presence-System:** Implementieren Sie "Alice is viewing card #42"
  4. **Notification-System:** Push-Benachrichtigungen bei @mentions
  5. **Realtime-Dashboard:** Live-Metrics mit automatischer Aktualisierung
  6. **Collaborative Editor:** Google Docs-ähnliche Textbearbeitung
  7. **Gaming:** Einfaches Multiplayer-Spiel mit ThemisDB als Backend
  8. **Live-Poll:** Echtzeit-Abstimmungssystem mit automatischen Updates
  9. **Activity-Feed:** Timeline aller Aktivitäten im System
  10. **Realtime-Search:** Live-Search-Results während dem Tippen
-

# Kapitel 15: Kapitel 12: Computer Vision & Bildanalyse

## 12.1 Einführung in Computer Vision Datenbanken

### Das Problem: Bilder sind mehr als nur Dateien

Stellen Sie sich vor, Sie haben 100.000 Drohnenbilder von einem Baugelände. Wie finden Sie: - Alle Bilder, die ein bestimmtes Fahrzeug zeigen? - Bilder mit Sicherheitsproblemen (fehlende Helme)? - Ähnliche Perspektiven des gleichen Gebäudes? - Zeitliche Entwicklung eines Bauprojekts?

Traditionelle Datenbanken speichern nur Dateinamen - **Computer Vision Datenbanken** analysieren und indexieren Bildinhalte.

### Computer Vision Use Cases

Use Case	Datentyp	Volumen	Beispiel
Drohnen-Inspektion	Bilder + GPS + Metadata	10K-1M	Baustellen, Infrastruktur
Medizinische Bildgebung	DICOM-Bilder	100K-10M	Röntgen, MRT, CT
Retail Analytics	Video-Frames	1M-100M	Kundenverhalten, Warenpräsentation
Autonomous Vehicles	Sensor-Fusion	100M+	LiDAR, Kameras, Radar
Satellite Imagery	Multi-Spectral Images	1M-10M	Landnutzung, Umweltmonitoring

### Herausforderungen

**Storage:** - Große Dateien (2-20 MB pro Hochauflösungsbild) - Verschiedene Formate (JPEG, PNG, TIFF, RAW) - Metadata (EXIF, GPS, Kameraeinstellungen)

**Processing:** - Feature-Extraktion (SIFT, ORB, Deep Learning) - Object Detection (YOLO, Faster R-CNN) - Image Classification (ResNet, EfficientNet)

**Retrieval:** - Content-Based Image Retrieval (CBIR) - Similarity Search über Visual Features - Spatial Queries (GPS-basiert) - Temporal Queries (Zeitreihen)

```
flowchart TD
```

```
 Start[Raw Image] --> Store[(ThemisDB Storage)]
```

```
 Store --> Extract[Feature Extraction]
```

```
 Extract --> CNN[Deep Learning
ResNet/EfficientNet]
```

```
 Extract --> Classic[Classical CV
SIFT/ORB]
```

```
 Extract --> Meta[Metadata
EXIF/GPS]
```

```
 CNN --> VecEmbed[Vector Embeddings
2048-dim]
```

```
 Classic --> KeyPoints[Keypoint Descriptors]
```

```
 Meta --> Structured[Structured Data
Location, Time]
```

```
 VecEmbed --> VecIndex[(Vector Index
HNSW)]
```

```
 KeyPoints --> HashIndex[(Hash Index
LSH)]
```

```
 Structured --> GeoIndex[(Geo Index
R-Tree)]
```

```
 Query[Query Image] --> QExtract[Extract Features]
```

```
 QExtract --> Search{Search Strategy}
```

```
 Search -->|Similarity| VecIndex
```

```
 Search -->|Location| GeoIndex
```

```
 Search -->|Keypoints| HashIndex
```

```
 VecIndex --> Results[Ranked Results]
```

```
 GeoIndex --> Results
```

```
 HashIndex --> Results
```

```
 style Start fill:#667eea
```

```
 style Store fill:#4facfe
```

```
 style VecEmbed fill:#43e97b
```

```
 style VecIndex fill:#f093fb
```

```
 style Results fill:#ff932a
```

**Abb. 12.1:** Computer-Vision-Pipeline

## **12.2 Computer Vision Datenmodell**

### **Schema für Bildmetadaten**

```

-- Bilder mit vollständigen Metadaten
CREATE TABLE images (
 image_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 filename VARCHAR(255) NOT NULL,
 file_path TEXT NOT NULL,
 file_size_bytes BIGINT,
 format VARCHAR(20), -- 'JPEG', 'PNG', 'TIFF', ...

 -- Bild-Eigenschaften
 width INTEGER,
 height INTEGER,
 bit_depth INTEGER,
 color_space VARCHAR(20),

 -- Aufnahme-Metadaten (EXIF)
 captured_at TIMESTAMP,
 camera_make VARCHAR(100),
 camera_model VARCHAR(100),
 focal_length_mm FLOAT,
 aperture_f_stop FLOAT,
 iso INTEGER,
 shutter_speed VARCHAR(20),

 -- GPS-Daten
 gps_latitude DOUBLE PRECISION,
 gps_longitude DOUBLE PRECISION,
 gps_altitude_m FLOAT,
 location GEOGRAPHY(Point, 4326),

 -- Kategorisierung
 category VARCHAR(100),
 tags TEXT[],

 -- Zusätzliche Metadaten
 metadata JSONB,

 -- Zeitstempel
 created_at TIMESTAMP DEFAULT NOW(),
 updated_at TIMESTAMP DEFAULT NOW()
);

-- Spatial Index für GPS-Queries
CREATE INDEX idx_images_location ON images USING GIST(location);

-- Index für zeitbasierte Queries
CREATE INDEX idx_images_captured_at ON images (captured_at DESC);

-- Full-Text Index für Tags
CREATE INDEX idx_images_tags ON images USING GIN(tags);

```

## Feature-Extraktion Tabelle

```
-- Visuelle Features (z.B. von CNN)
CREATE TABLE image_features (
 image_id UUID PRIMARY KEY REFERENCES images(image_id),

 -- Deep Learning Features (z.B. ResNet-50)
 deep_features VECTOR(2048), -- 2048-dimensional embedding

 -- Classical Features
 color_histogram FLOAT[], -- RGB histogram
 edge_features FLOAT[], -- Edge detection features
 texture_features FLOAT[], -- Texture descriptors

 -- Feature Metadata
 feature_extractor VARCHAR(100),
 extraction_timestamp TIMESTAMP DEFAULT NOW()
);

-- HNSW Index für Visual Similarity Search
CREATE INDEX idx_image_features_deep ON image_features
USING hnsw (deep_features vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```



## Object Detection Results

```
-- Erkannte Objekte in Bildern
CREATE TABLE detected_objects (
 detection_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 image_id UUID REFERENCES images(image_id),

 -- Objekt-Klassifikation
 object_class VARCHAR(100),
 confidence FLOAT, -- 0.0 to 1.0

 -- Bounding Box
 bbox_x INTEGER,
 bbox_y INTEGER,
 bbox_width INTEGER,
 bbox_height INTEGER,

 -- Optional: Segmentation Mask
 segmentation_mask BYTEA,

 -- Detection Metadata
 detector_model VARCHAR(100),
 detection_timestamp TIMESTAMP DEFAULT NOW()
);

-- Index für Object-Search
CREATE INDEX idx_detected_objects_class ON detected_objects (object_class, confidence);
CREATE INDEX idx_detected_objects_image ON detected_objects (image_id);
```

## **12.3 Bildverarbeitung mit ThemisDB**

### **Bild-Upload mit Metadata-Extraktion**

```

from PIL import Image
from PIL.ExifTags import TAGS, GPSTAGS
import hashlib
from datetime import datetime

def extract_exif(image_path):
 """Extrahiere EXIF-Daten aus Bild"""
 img = Image.open(image_path)
 exif_data = {}

 if hasattr(img, '_getexif') and img._getexif():
 exif = img._getexif()
 for tag_id, value in exif.items():
 tag = TAGS.get(tag_id, tag_id)
 exif_data[tag] = value

 # GPS-Daten extrahieren
 gps_info = exif_data.get('GPSInfo', {})
 gps_data = {}
 for key in gps_info.keys():
 decode = GPSTAGS.get(key, key)
 gps_data[decode] = gps_info[key]

 return exif_data, gps_data

def upload_image(image_path, category=None, tags=None):
 """Upload Bild mit Metadata-Extraktion"""
 img = Image.open(image_path)
 exif_data, gps_data = extract_exif(image_path)

 # GPS-Koordinaten konvertieren
 latitude = convert_to_degrees(gps_data.get('GPSLatitude', []))
 longitude = convert_to_degrees(gps_data.get('GPSLongitude', []))
 altitude = gps_data.get('GPSAltitude', 0)

 # Bild-Metadata
 image_data = {
 'filename': os.path.basename(image_path),
 'file_path': image_path,
 'file_size_bytes': os.path.getsize(image_path),
 'format': img.format,
 'width': img.width,
 'height': img.height,
 'captured_at': exif_data.get('DateTime'),
 'camera_make': exif_data.get('Make'),
 'camera_model': exif_data.get('Model'),
 'gps_latitude': latitude,
 'gps_longitude': longitude,
 'gps_altitude_m': altitude,
 }

```

```
 'category': category,
 'tags': tags or []
 }

 # In DB speichern
 image_id = themis.execute("""
 INSERT INTO images
 (filename, file_path, file_size_bytes, format, width, height,
 captured_at, camera_make, camera_model,
 gps_latitude, gps_longitude, gps_altitude_m,
 location, category, tags)
 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
 ST_SetSRID(ST_MakePoint(?, ?), 4326), ?, ?)
 RETURNING image_id
 """, tuple(image_data.values()) + (longitude, latitude))

 return image_id
```

## Feature-Extraktion mit Deep Learning

```

import torch
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image

model = models.resnet50(pretrained=True)
model.eval()
model = torch.nn.Sequential(*list(model.children())[:-1])

preprocess = transforms.Compose([
 transforms.Resize(256),
 transforms.CenterCrop(224),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.485, 0.456, 0.406],
 std=[0.229, 0.224, 0.225]),
])

def extract_features(image_path):
 """Extrahiere Deep Learning Features"""
 img = Image.open(image_path).convert('RGB')
 img_tensor = preprocess(img).unsqueeze(0)

 with torch.no_grad():
 features = model(img_tensor)
 features = features.squeeze().numpy() # (2048,)

 return features

def store_features(image_id, image_path):
 """Extrahiere und speichere Features"""
 features = extract_features(image_path)

 themis.execute("""
 INSERT INTO image_features (image_id, deep_features, feature_extractor)
 VALUES (?, ?, 'resnet50')
 ON CONFLICT (image_id) DO UPDATE
 SET deep_features = EXCLUDED.deep_features,
 extraction_timestamp = NOW()
 """, (image_id, features.tolist()))

```

## Object Detection mit YOLO

```

from ultralytics import YOLO

yolo_model = YOLO('yolov8n.pt')

def detect_objects(image_path, confidence_threshold=0.5):
 """Detecte Objekte mit YOLO"""
 results = yolo_model(image_path)

 detections = []
 for result in results:
 boxes = result.boxes
 for box in boxes:
 if box.conf[0] >= confidence_threshold:
 detections.append({
 'object_class': result.names[int(box.cls[0])],
 'confidence': float(box.conf[0]),
 'bbox_x': int(box.xyxy[0][0]),
 'bbox_y': int(box.xyxy[0][1]),
 'bbox_width': int(box.xyxy[0][2] - box.xyxy[0][0]),
 'bbox_height': int(box.xyxy[0][3] - box.xyxy[0][1])
 })

 return detections

def store_detections(image_id, image_path):
 """Detecte und speichere Objekte"""
 detections = detect_objects(image_path)

 for det in detections:
 themis.execute("""
 INSERT INTO detected_objects
 (image_id, object_class, confidence, bbox_x, bbox_y,
 bbox_width, bbox_height, detector_model)
 VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'yolov8n')
 """, (image_id, det['object_class'], det['confidence'],
 det['bbox_x'], det['bbox_y'],
 det['bbox_width'], det['bbox_height']))

```

## 12.4 Visual Similarity Search

### Content-Based Image Retrieval (CBIR)

```
-- Finde visuell ähnliche Bilder
SELECT
 i.image_id,
 i.filename,
 i.captured_at,
 i.location,
 1 - (f.deep_features <=> :query_features) AS similarity
FROM images i
JOIN image_features f ON i.image_id = f.image_id
ORDER BY f.deep_features <=> :query_features
LIMIT 20;
```

```
def find_similar_images(query_image_path, limit=10):
 """Finde visuell ähnliche Bilder"""
 # Features der Query extrahieren
 query_features = extract_features(query_image_path)

 # Similarity Search
 similar = themis.query("""
 SELECT
 i.image_id,
 i.filename,
 i.file_path,
 i.captured_at,
 1 - (f.deep_features <=> ?) AS similarity
 FROM images i
 JOIN image_features f ON i.image_id = f.image_id
 ORDER BY f.deep_features <=> ?
 LIMIT ?
 """, (query_features.tolist(), query_features.tolist(), limit))

 return similar

similar_images = find_similar_images('query_image.jpg', limit=10)
for img in similar_images:
 print(f"{img['filename']}: {img['similarity']:.3f}")
```

## Multi-Modal Search: Visual + Text

```
-- Kombiniere Visual Similarity + Text Tags
WITH visual_matches AS (
 SELECT image_id,
 1 - (deep_features <=> :query_features) AS visual_score
 FROM image_features
 ORDER BY deep_features <=> :query_features
 LIMIT 100
),
text_matches AS (
 SELECT image_id,
 ts_rank(to_tsvector('english', array_to_string(tags, ' ')),
 plainto_tsquery('english', :query_text)) AS text_score
 FROM images
 WHERE tags && :query_tags -- Array overlap
)
SELECT
 i.*,
 COALESCE(vm.visual_score, 0) * 0.7 +
 COALESCE(tm.text_score, 0) * 0.3 AS combined_score
FROM images i
LEFT JOIN visual_matches vm ON i.image_id = vm.image_id
LEFT JOIN text_matches tm ON i.image_id = tm.image_id
WHERE vm.image_id IS NOT NULL OR tm.image_id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;
```



## 12.5 Spatial & Temporal Queries

### GPS-basierte Suche

```
-- Alle Bilder in einem Radius von 1km
SELECT
 image_id,
 filename,
 captured_at,
 gps_latitude,
 gps_longitude,
 ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326)) as distance_m
FROM images
WHERE ST_DWithin(
 location,
 ST_SetSRID(ST_MakePoint(?, ?), 4326),
 1000 -- 1000 Meter
)
ORDER BY distance_m;

-- Bilder in einem Polygon (z.B. Baugelände)
SELECT *
FROM images
WHERE ST_Within(
 location,
 ST_GeomFromText('POLYGON((
 13.404 52.520,
 13.408 52.520,
 13.408 52.518,
 13.404 52.518,
 13.404 52.520
))', 4326)
);
```

## Zeitliche Entwicklung

```
def get_temporal_sequence(location, radius_m=100,
 start_date=None, end_date=None):
 """Hole zeitliche Bildsequenz an einem Ort"""
 query = """
 SELECT
 image_id,
 filename,
 captured_at,
 category,
 tags,
 ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326)) as distance_m
 FROM images
 WHERE ST_DWithin(
 location,
 ST_SetSRID(ST_MakePoint(?, ?), 4326),
 ?
)
 """

 params = [longitude, latitude, longitude, latitude, radius_m]

 if start_date:
 query += " AND captured_at >= ?"
 params.append(start_date)

 if end_date:
 query += " AND captured_at <= ?"
 params.append(end_date)

 query += " ORDER BY captured_at"

 return themis.query(query, tuple(params))

sequence = get_temporal_sequence(
 location=(13.405, 52.520),
 radius_m=50,
 start_date='2024-01-01',
 end_date='2024-12-31'
)
```

## 12.6 Example: Drone Image Analysis

### Überblick

Das **Drone Image Analysis** Beispiel ( [examples/09\\_drone\\_image\\_analysis](#) ) demonstriert eine vollständige Pipeline für Drohnenbilder-Analyse:

**Features:** - Automatische Bild-Kategorisierung - GPS-basierte Geo-Queries - Feature Matching für ähnliche Bilder - Object Detection (YOLOv8) Integration - Flight Path Visualisierung - Thermal Imaging Analytics

## Batch-Upload von Drohnenbildern

```
import os
import glob
from concurrent.futures import ThreadPoolExecutor

def batch_upload_drone_images(directory, flight_id, category='construction'):
 """Batch-Upload aller Bilder aus einem Drohnenflug"""
 image_files = glob.glob(os.path.join(directory, '*.jpg'))
 print(f"Found {len(image_files)} images to process")

 def process_image(image_path):
 # Upload Bild
 image_id = upload_image(image_path, category=category,
 tags=['drone', flight_id])

 # Feature-Extraktion
 store_features(image_id, image_path)

 # Object Detection
 store_detections(image_id, image_path)

 return image_id

 # Parallel-Processing
 with ThreadPoolExecutor(max_workers=4) as executor:
 image_ids = list(executor.map(process_image, image_files))

 print(f"Processed {len(image_ids)} images")
 return image_ids

flight_id = 'flight_2024_01_15_001'
image_ids = batch_upload_drone_images(
 directory='/data/drone_images/2024-01-15/',
 flight_id=flight_id,
 category='construction_site'
)
```

---

## **Flugpfad-Rekonstruktion**

```

def reconstruct_flight_path(flight_id):
 """Rekonstruiere Flugpfad aus GPS-Daten"""
 path = themis.query("""
 SELECT
 image_id,
 filename,
 captured_at,
 gps_latitude,
 gps_longitude,
 gps_altitude_m,
 ST_AsText(location) as location_wkt
 FROM images
 WHERE ? = ANY(tags)
 ORDER BY captured_at
 """, (flight_id,))

 return path

import folium

def visualize_flight_path(flight_id):
 """Visualisiere Flugpfad auf Karte"""
 path = reconstruct_flight_path(flight_id)

 # Zentrum der Karte
 center_lat = sum(p['gps_latitude'] for p in path) / len(path)
 center_lon = sum(p['gps_longitude'] for p in path) / len(path)

 # Karte erstellen
 m = folium.Map(location=[center_lat, center_lon], zoom_start=15)

 # Flugpfad zeichnen
 coordinates = [(p['gps_latitude'], p['gps_longitude']) for p in path]
 folium.PolyLine(coordinates, color='red', weight=2,
 opacity=0.8).add_to(m)

 # Marker für Bilder
 for p in path:
 folium.Marker(
 location=[p['gps_latitude'], p['gps_longitude']],
 popup=f"{p['filename']}
Alt: {p['gps_altitude_m']}m",
 icon=folium.Icon(color='blue', icon='camera')
).add_to(m)

 m.save(f'flight_path_{flight_id}.html')
 return m

visualize_flight_path(flight_id)

```

## Automatische Anomalie-Detection

```
def detect_safety_violations(flight_id):
 """Detecte Sicherheitsverstöße (z.B. fehlende Helme)"""
 # Finde alle Bilder mit Personen
 images_with_people = themis.query("""
 SELECT DISTINCT i.image_id, i.filename, i.file_path
 FROM images i
 JOIN detected_objects d ON i.image_id = d.image_id
 WHERE d.object_class = 'person'
 AND d.confidence > 0.7
 AND ? = ANY(i.tags)
 """, (flight_id,))

 violations = []

 for img in images_with_people:
 # Prüfe, ob Helme detektiert wurden
 helmets = themis.query("""
 SELECT COUNT(*) as helmet_count
 FROM detected_objects
 WHERE image_id = ?
 AND object_class IN ('hardhat', 'helmet')
 """, (img['image_id'],))

 people_count = themis.query("""
 SELECT COUNT(*) as person_count
 FROM detected_objects
 WHERE image_id = ?
 AND object_class = 'person'
 """, (img['image_id'],))

 if helmets[0]['helmet_count'] < people_count[0]['person_count']:
 violations.append({
 'image_id': img['image_id'],
 'filename': img['filename'],
 'issue': 'Missing hardhat',
 'people_count': people_count[0]['person_count'],
 'helmet_count': helmets[0]['helmet_count']
 })

 return violations

violations = detect_safety_violations(flight_id)
if violations:
 print(f"⚠ Found {len(violations)} safety violations:")
 for v in violations:
 print(f" - {v['filename']}: {v['issue']}")
```

## Change Detection zwischen Flügen

```
def compare_flights(flight_id_1, flight_id_2, similarity_threshold=0.85):
 """Vergleiche zwei Flüge und finde Änderungen"""
 # Hole Bilder von beiden Flügen
 images_1 = reconstruct_flight_path(flight_id_1)
 images_2 = reconstruct_flight_path(flight_id_2)

 changes = []

 for img1 in images_1:
 # Finde räumlich nächstes Bild aus Flug 2
 nearest_img2 = themis.query_one("""
 SELECT
 i2.image_id,
 i2.filename,
 i2.captured_at,
 ST_Distance(i1.location, i2.location) as distance_m,
 1 - (f2.deep_features <=> f1.deep_features) as visual_similarity
 FROM images i1
 JOIN image_features f1 ON i1.image_id = f1.image_id
 CROSS JOIN images i2
 JOIN image_features f2 ON i2.image_id = f2.image_id
 WHERE i1.image_id = ?
 AND ? = ANY(i2.tags)
 ORDER BY ST_Distance(i1.location, i2.location)
 LIMIT 1
 """, (img1['image_id'], flight_id_2))

 if nearest_img2 and nearest_img2['distance_m'] < 10: # Innerhalb 10m
 if nearest_img2['visual_similarity'] < similarity_threshold:
 changes.append({
 'location': (img1['gps_latitude'], img1['gps_longitude']),
 'image_1': img1['filename'],
 'image_2': nearest_img2['filename'],
 'similarity': nearest_img2['visual_similarity'],
 'change_detected': True
 })

 return changes

changes = compare_flights('flight_001', 'flight_002')
print(f"Detected {len(changes)} significant changes between flights")
```



## 12.7 Performance & Skalierung

### Thumbnail-Generierung für schnelle Preview

```
from PIL import Image

def generate_thumbnail(image_path, size=(256, 256)):
 """Generiere Thumbnail für Preview"""
 img = Image.open(image_path)
 img.thumbnail(size, Image.LANCZOS)

 # Speichere Thumbnail
 thumb_path = image_path.replace('.jpg', '_thumb.jpg')
 img.save(thumb_path, 'JPEG', quality=85)

 # Update DB
 themis.execute("""
 UPDATE images
 SET metadata = jsonb_set(metadata, '{thumbnail_path}', ?)
 WHERE file_path = ?
 """, (f'"{thumb_path}"', image_path))

 return thumb_path
```

### Lazy Loading von Features

```
-- Erstelle Features nur on-demand
CREATE OR REPLACE FUNCTION ensure_features(p_image_id UUID)
RETURNS BOOLEAN AS $$
BEGIN
 LET feature_exists = (
 FOR feature IN image_features
 FILTER feature.image_id == p_image_id
 LIMIT 1
 RETURN 1
)

 IF LENGTH(feature_exists) == 0 THEN
 -- Trigger Feature-Extraktion
 INSERT {image_id: p_image_id} INTO feature_extraction_queue;
 RETURN FALSE;
 END IF;
 RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

## Caching für häufige Queries

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=100)
def get_similar_images_cached(image_path_hash, limit=10):
 """Gecachte Similarity Search"""
 # ... similarity search logic
 pass

image_hash = hashlib.md5(open(image_path, 'rb').read()).hexdigest()
similar = get_similar_images_cached(image_hash, limit=10)
```

## 12.8 Best Practices

### 1. Storage-Optimierung

✓ **DO:** - Speichere nur Metadata in DB, Bilder im Object Storage (S3, MinIO) - Generiere Thumbnails für Previews - Nutze komprimierte Formate (JPEG für Photos, WebP für Web)

✗ **DON'T:** - Speichere nicht alle Bilder als BYTEA in DB - Vermeide unkomprimierte Formate (BMP, TIFF) für Archivierung - Keine Features ohne Index

### 2. Feature-Extraktion

✓ **DO:** - Nutze Pre-Trained Models (ResNet, EfficientNet) - Batch-Processing für viele Bilder - GPU-Acceleration für Deep Learning

✗ **DON'T:** - Extrahiere Features nicht synchron beim Upload - Vermeide Training von Modellen auf CPU - Keine Feature-Extraktion ohne Quality-Check

### 3. Similarity Search

✓ **DO:** - HNSW Index für große Datasets - Kombiniere Visual + Text Search - Pre-Filter mit Metadata (Zeit, Ort)

✗ **DON'T:** - Keine Brute-Force Search über Millionen Bilder - Vermeide hohe Dimensions ohne Dimensionality Reduction - Keine Similarity ohne Normalisierung

## 12.9 Zusammenfassung

ThemisDB ermöglicht effiziente Computer Vision Anwendungen durch:

---

✓ **Multi-Modal Data Management:** - Images + GPS + Features + Objects in einer DB - Spatial + Temporal + Visual Queries - Graph-basierte Analyse von Bild-Beziehungen

✓ **Flexible Feature-Speicherung:** - Native VECTOR-Spalten für Embeddings - HNSW Index für schnelle Similarity Search - JSON für flexible Metadata

✓ **Production-Ready:** - Skalierbare Storage (Partitionierung) - Batch-Processing Support - Integration mit ML-Frameworks

**Key Takeaways:** 1. Speichere Metadata in DB, Bilder in Object Storage 2. Generiere Features asynchron (Queue/Workers) 3. Nutze HNSW für Similarity Search 4. Kombiniere Visual + Spatial + Temporal Queries 5. Thumbnails für Performance

---

# Kapitel 16: Kapitel 13: Volltext-Suche & NLP

## Einführung

Die Volltext-Suche ist eine der fundamentalen Anforderungen moderner Anwendungen. Ob E-Commerce-Produktsuche, Dokumentenverwaltung oder Content-Management - Nutzer erwarten relevante, schnelle Suchergebnisse. ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen integriert ist.

In diesem Kapitel behandeln wir:

- **Grundlagen der Volltext-Suche:** Tokenisierung, Stemming, Stop-Words
- **ThemisDB Fulltext-Indexe:** Konfiguration und Optimierung
- **Erweiterte Suchfunktionen:** Wildcards, Phrasensuche, Fuzzy Search
- **NLP-Integration:** Sentiment-Analyse, Entity-Erkennung, Sprachverarbeitung
- **Best Practices:** Performance, Relevanz-Ranking, mehrsprachige Suche

## Warum ist Volltext-Suche wichtig?

**Problem der einfachen String-Suche:**

```
-- Ineffizient: Keine Relevanz, langsam bei großen Datensätzen
FOR article IN articles
 FILTER LIKE(article.title, '%database%') OR LIKE(article.content, '%database%')
 RETURN article
```

Probleme: - Keine Wort-Trennung (findet "database" nicht in "databases") - Keine Relevanz-Bewertung - Performance-Probleme (Full Table Scan) - Keine sprachliche Verarbeitung

**Lösung mit Volltext-Index:**

```
-- Schnell, relevant, sprachlich intelligent
FOR article IN articles
 FILTER FULLTEXT(article.title, 'database') OR FULLTEXT(article.content, 'database')
 LET score = FULLTEXT_SCORE(article)
 SORT score DESC
 RETURN {article, score}
```

Vorteile: -  Automatische Wort-Erkennung und Stammformbildung -  Relevanz-Ranking (TF-IDF, BM25) -  Sehr performant durch invertierte Indizes -  Sprachliche Features (Stop-Words, Synonyme)

---

## Grundlagen der Volltext-Suche

### Text-Verarbeitung Pipeline

Die Volltext-Suche durchläuft mehrere Verarbeitungsschritte:

#### 1. Tokenisierung

Zerlegt Text in einzelne Wörter (Tokens):

```
"ThemisDB ist eine Multi-Model Datenbank"
↓
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
```

#### 2. Normalisierung

Konvertiert Tokens in Grundform:

```
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
↓
["themisdb", "ist", "eine", "multi-model", "datenbank"] # Lowercase
```

#### 3. Stop-Word-Filterung

Entfernt häufige, nicht-informative Wörter:

```
["themisdb", "ist", "eine", "multi-model", "datenbank"]
↓
["themisdb", "multi-model", "datenbank"] # "ist", "eine" entfernt
```

#### 4. Stemming

Reduziert Wörter auf Wortstamm:

```
["Datenbanken", "Datenbank", "Datenbankserver"]
↓
["datenbank", "datenbank", "datenbankserv"] # Gemeinsamer Stamm
```

flowchart TD

Input[Raw Text: ThemisDB ist eine Multi-Model Datenbank]

Input --> Token[1. Tokenization - split into words]

Token --> Tokens[Tokens: themisdb, ist, eine, multi-model, datenbank]

Tokens --> Norm[2. Normalization - lowercase, unicode]

Norm --> Normalized[Normalized tokens: themisdb, ist, eine, multi-model, datenbank]

Normalized --> Stop[3. Stop-Word Filter - remove common words]

Stop --> Filtered[Filtered tokens: themisdb, multi-model, datenbank]

Filtered --> Stem[4. Stemming - reduce to root form]

Stem --> Stemmed[Stemmed tokens: themisdb, multi-model, datenbank]

Stemmed --> Index[5. Inverted Index - term to document IDs]

Index --> Final["Searchable Index:<br/>themisdb: doc1, doc5<br/>datenbank: doc1, doc2, ..."]

style Input fill:#667eea

style Token fill:#4facfe

style Norm fill:#43e97b

style Stop fill:#f093fb

style Stem fill:#ffd32a

style Index fill:#fa709a

style Final fill:#95e1d3

**Abb. 13.1:** Fulltext-Search-Indexierung

## 5. Index-Erstellung

Erstellt inverted index für schnelle Suche:

```
Token → Dokument-IDs
"themisdb" → [1, 5, 12, 89]
"datenbank" → [1, 2, 5, 12, 34, 89]
"multi-model" → [1, 12]
```

## Relevanz-Ranking Algorithmen

### TF-IDF (Term Frequency - Inverse Document Frequency)

Bewertet Relevanz basierend auf: - **TF**: Wie oft erscheint der Begriff im Dokument? - **IDF**: Wie selten ist der Begriff im gesamten Korpus?

```
TF-IDF = TF × IDF
TF = (Anzahl Begriff in Dokument) / (Gesamtzahl Wörter in Dokument)
IDF = log(Gesamtanzahl Dokumente / Anzahl Dokumente mit Begriff)
```

### Beispiel:

Dokument: "ThemisDB ist eine Datenbank. ThemisDB unterstützt Multi-Model." Suchbegriff: "ThemisDB"

```
TF = 2 / 8 = 0.25 (2× "ThemisDB", 8 Wörter gesamt)
IDF = log(10000 / 100) = 2 (100 von 10000 Dokumenten enthalten "ThemisDB")
TF-IDF = 0.25 × 2 = 0.5
```

### BM25 (Best Matching 25)

Verbessertes Ranking-Modell mit Sättigungsfunktion:

```
BM25 = IDF × (TF × (k1 + 1)) / (TF + k1 × (1 - b + b × (docLen / avgDocLen)))
```

Parameter: - **k1**: Sättigung für Term-Frequenz (typisch 1.2-2.0) - **b**: Dokumentlängen-Normalisierung (typisch 0.75)

Vorteil: Verhindert, dass sehr lange Dokumente überproportional hohe Scores bekommen.

## Volltext-Indexe in ThemisDB

### Index erstellen

#### Einfacher Fulltext-Index:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content);
```

#### Mit Konfiguration:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content)
WITH (
 analyzer = 'german', -- Sprach-Analyzer
 stopwords = ['der', 'die', 'das'], -- Custom Stop-Words
 min_word_length = 3, -- Minimale Wortlänge
 max_word_length = 50, -- Maximale Wortlänge
 case_sensitive = false, -- Groß-/Kleinschreibung
 stemming = true -- Stammform-Reduktion
);
```

**Gewichtete Felder:**

```
CREATE FULLTEXT INDEX idx_articles_weighted
ON articles (
 title WEIGHT 3.0, -- Titel 3× wichtiger
 abstract WEIGHT 2.0, -- Abstract 2× wichtiger
 content WEIGHT 1.0 -- Content normale Gewichtung
);
```

**Suchen mit Volltext-Index****Einfache Suche:**

```
FOR article IN articles
 FILTER FULLTEXT(article, 'database systems')
 RETURN article
```

**Mit Relevanz-Score:**

```
FOR article IN articles
 FILTER FULLTEXT(article, 'database systems')
 LET relevance = FULLTEXT_SCORE(article)
 SORT relevance DESC
 LIMIT 10
 RETURN {
 id: article.id,
 title: article.title,
 relevance
 }
```

**Mehrere Begriffe (AND):**



```
-- Alle Begriffe müssen vorkommen
WHERE FULLTEXT_MATCH(articles, 'database AND multi-model');
```

### Alternative Begriffe (OR):

```
-- Mindestens ein Begriff muss vorkommen
WHERE FULLTEXT_MATCH(articles, 'database OR nosql');
```

### Ausschluss (NOT):

```
-- Enthält "database" aber nicht "relational"
WHERE FULLTEXT_MATCH(articles, 'database NOT relational');
```

### Phrasensuche:

```
-- Exakte Phrase in Anführungszeichen
WHERE FULLTEXT_MATCH(articles, '"multi-model database"');
```

### Wildcard-Suche:

```
-- * für beliebige Zeichen
WHERE FULLTEXT_MATCH(articles, 'data*'); -- Findet "database", "dataset", etc.
```

### Fuzzy-Suche (Rechtschreibfehler):

```
-- ~ für Fuzzy-Match (Levenshtein-Distanz)
WHERE FULLTEXT_MATCH(articles, 'databse~'); -- Findet "database" trotz Fehler
```

## Feld-spezifische Suche

### Nur in bestimmten Feldern suchen:

```
FOR article IN articles
 FILTER FULLTEXT(article.title, 'database') AND FULLTEXT(article.content, 'nosql')
RETURN article
```

### Kombinierte Suche:

```
FOR article IN articles
 FILTER (FULLTEXT(article.title, 'multi-model') OR FULLTEXT(article.content, 'vector'))
 AND article.category == 'technology' -- Zusätzliche Filter kombinierbar
RETURN article
```

## Erweiterte Suchfunktionen

### Highlighting

Markiert Suchbegriffe in Ergebnissen:

```
FOR article IN articles
 FILTER FULLTEXT(article, 'database')
 LIMIT 10
 RETURN {
 id: article.id,
 title: article.title,
 snippet: FULLTEXT_HIGHLIGHT(article.content, 'database', '<mark>', '</mark>')
 }
```

Ergebnis:

```
"... ThemisDB ist eine <mark>database</mark> für Multi-Model Daten ..."
```

### Snippet-Extraktion:

```
FOR article IN articles
 FILTER FULLTEXT(article, 'database')
 RETURN {
 id: article.id,
 title: article.title,
 snippet: FULLTEXT_SNIPPET(article.content, 'database', 50, 200)
 }
```

Extrahiert 200 Zeichen um den Suchbegriff herum (50 Zeichen vor, 150 nach).

## Auto-Complete / Suggest

### Präfix-Suche für Auto-Complete:

```
-- Findet alle Artikel, die mit "dat" beginnen
FOR article IN articles
 LET suggestion = FULLTEXT_TERMS(article, 'dat*')
 FILTER suggestion != null
 LIMIT 10
 RETURN DISTINCT suggestion
```

Ergebnis:

```
["database", "data", "dataset", "dataflow"]
```

### Häufigkeits-basierte Vorschläge:

```
FOR article IN articles
 LET term = FULLTEXT_TERMS(article, 'dat*')
 FILTER term != null
 COLLECT termValue = term
 AGGREGATE frequency = COUNT()
 SORT frequency DESC
 LIMIT 10
 RETURN {term: termValue, frequency}
```

## Faceted Search

Kombiniert Volltext mit Facetten:

```
FOR article IN articles
 FILTER FULLTEXT(article, 'database')
 COLLECT category = article.category
 AGGREGATE count = COUNT()
 RETURN {category, count}
```

Ergebnis:

category	count
-----	-----
Technology	45
Science	23
Business	12

### Komplexe Facetten-Abfrage:

```

LET results = (
 FOR article IN articles
 FILTER FULLTEXT(article, 'database')
 RETURN article
)

FOR r IN results
 FILTER FULLTEXT(r, 'database')
 RETURN r
)

FOR result IN results
 COLLECT category = result.category
 AGGREGATE
 count = COUNT(),
 avg_relevance = AVG(FULLTEXT_SCORE(result))
 SORT count DESC
 RETURN {category, count, avg_relevance}

```

## NLP-Integration

### Sentiment-Analyse

Analysiert Stimmung/Emotion in Texten.

#### Python Integration mit TextBlob:

```

from themisdb import ThemisDB
from textblob import TextBlob

db = ThemisDB()

reviews = db.query("""
 FOR review IN product_reviews
 RETURN {id: review.id, text: review.text}
""")

for review in reviews:
 # Sentiment-Analyse
 blob = TextBlob(review['text'])
 sentiment = blob.sentiment.polarity # -1 (negativ) bis +1 (positiv)

 # Sentiment in DB speichern
 db.query("""
 FOR review IN product_reviews
 FILTER review.id == @review_id
 UPDATE review WITH {sentiment_score: @sentiment} IN product_reviews
 """, {"sentiment": sentiment, "review_id": review['id']})

avg_sentiment = db.query("""
 FOR review IN product_reviews
 COLLECT product_id = review.product_id
 AGGREGATE
 avg_sentiment = AVG(review.sentiment_score),
 review_count = COUNT()
 FILTER review_count >= 10
 SORT avg_sentiment DESC
 RETURN {product_id, avg_sentiment, review_count}
""")

```

### Sentiment-Kategorien:

```

def categorize_sentiment(score):
 if score >= 0.5:
 return "sehr positiv"
 elif score >= 0.1:
 return "positiv"
 elif score >= -0.1:
 return "neutral"
 elif score >= -0.5:
 return "negativ"
 else:
 return "sehr negativ"

db.query("""
 ALTER TABLE product_reviews
 ADD COLUMN sentiment_category VARCHAR(20)
""")

for review in reviews:
 category = categorize_sentiment(review['sentiment_score'])
 db.query("""
 UPDATE product_reviews
 SET sentiment_category = ?
 WHERE id = ?
 """, [category, review['id']])

```

## Named Entity Recognition (NER)

Erkennt Entitäten wie Personen, Orte, Organisationen.

**Mit spaCy:**

```

import spacy
from themisdb import ThemisDB

nlp = spacy.load("de_core_news_md")
db = ThemisDB()

articles = db.query("SELECT id, content FROM articles")

for article in articles:
 doc = nlp(article['content'])

 # Entitäten extrahieren
 entities = []
 for ent in doc.ents:
 entities.append({
 'text': ent.text,
 'label': ent.label_, # PER, LOC, ORG, etc.
 'start': ent.start_char,
 'end': ent.end_char
 })

 # Entitäten als JSON speichern
 db.query("""
 UPDATE articles
 SET entities = ?
 WHERE id = ?
 """, [json.dumps(entities), article['id']])

db.query("""
 SELECT * FROM articles
 WHERE JSON_CONTAINS(entities, '$.text', 'Angela Merkel')
 """)

```

### Entity-Linking:

```
def link_entities(article_id, entities):
 for ent in entities:
 if ent['label'] == 'PER': # Person
 # Suche Person in DB
 person = db.query("""
 SELECT id FROM persons
 WHERE name = ? OR aliases LIKE ?
 """, [ent['text'], f"%{ent['text']}%"])

 if person:
 # Verknüpfung erstellen
 db.query("""
 INSERT INTO article_person_mentions
 (article_id, person_id, mention_text)
 VALUES (?, ?, ?)
 """, [article_id, person[0]['id'], ent['text']])
```

## Text-Klassifikation

Automatische Kategorisierung von Texten.

### Training eines Klassifikators:



```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from themisdb import ThemisDB

db = ThemisDB()

training_data = db.query("""
 SELECT content, category
 FROM articles
 WHERE category IS NOT NULL
""")

X_train = [row['content'] for row in training_data]
y_train = [row['category'] for row in training_data]

classifier = Pipeline([
 ('vectorizer', TfidfVectorizer(max_features=5000)),
 ('classifier', MultinomialNB())
])

classifier.fit(X_train, y_train)

new_articles = db.query("""
 SELECT id, content
 FROM articles
 WHERE category IS NULL
""")

for article in new_articles:
 predicted_category = classifier.predict([article['content']])[0]
 confidence = max(classifier.predict_proba([article['content']])[0])

 db.query("""
 UPDATE articles
 SET category = ?, category_confidence = ?
 WHERE id = ?
 """, [predicted_category, confidence, article['id']])

```

## Keyword-Extraktion

Extrahiert wichtige Schlüsselwörter aus Texten.

**Mit YAKE (Yet Another Keyword Extractor):**

```

import yake
from themisdb import ThemisDB

db = ThemisDB()

kw_extractor = yake.KeywordExtractor(
 lan="de", # Sprache
 n=3, # Maximale N-Gram Länge
 dedupLim=0.9, # Deduplizierung
 top=10, # Top 10 Keywords
 features=None
)

articles = db.query("SELECT id, title, content FROM articles")

for article in articles:
 text = article['title'] + " " + article['content']
 keywords = kw_extractor.extract_keywords(text)

 # Keywords als Liste speichern
 keyword_list = [kw[0] for kw in keywords]

 db.query("""
 UPDATE articles
 SET keywords = ?
 WHERE id = ?
 """, [json.dumps(keyword_list), article['id']])

db.query("""
 SELECT * FROM articles
 WHERE JSON_CONTAINS(keywords, ?, '$')
 """, ["Multi-Model"])

```

## Mehrsprachige Suche

### Language Detection

Automatische Spracherkennung:

```

from langdetect import detect
from themisdb import ThemisDB

db = ThemisDB()

articles = db.query("SELECT id, content FROM articles")

for article in articles:
 try:
 language = detect(article['content'])
 db.query("""
 UPDATE articles
 SET language = ?
 WHERE id = ?
 """, [language, article['id']])
 except:
 # Fallback bei Fehler
 db.query("""
 UPDATE articles
 SET language = 'unknown'
 WHERE id = ?
 """, [article['id']])

```

## Sprachspezifische Indexe

### Separate Indexe pro Sprache:

```

-- Deutscher Index
CREATE FULLTEXT INDEX idx_articles_de
ON articles (content)
WHERE language = 'de'
WITH (analyzer = 'german');

-- Englischer Index
CREATE FULLTEXT INDEX idx_articles_en
ON articles (content)
WHERE language = 'en'
WITH (analyzer = 'english');

-- Französischer Index
CREATE FULLTEXT INDEX idx_articles_fr
ON articles (content)
WHERE language = 'fr'
WITH (analyzer = 'french');

```

### Sprachabhängige Suche:

```
-- Automatische Sprachwahl basierend auf Nutzer-Präferenz
SELECT * FROM articles
WHERE language = 'de'
 AND FULLTEXT_MATCH(articles, 'Datenbank')
ORDER BY FULLTEXT_SCORE(articles) DESC;
```

## Translation-Aware Search

Suche über Übersetzungen hinweg:

```
from googletrans import Translator
from themisdb import ThemisDB

translator = Translator()
db = ThemisDB()

def multilingual_search(query, target_languages=['en', 'de', 'fr']):
 results = []

 for lang in target_languages:
 # Query in Zielsprache übersetzen
 if lang != 'de': # Annahme: Query ist deutsch
 translated_query = translator.translate(query, dest=lang).text
 else:
 translated_query = query

 # Suche in entsprechender Sprache
 lang_results = db.query("""
 SELECT *, ? as search_language
 FROM articles
 WHERE language = ?
 AND FULLTEXT_MATCH(articles, ?)
 ORDER BY FULLTEXT_SCORE(articles) DESC
 LIMIT 10
 """, [lang, lang, translated_query])

 results.extend(lang_results)

 return results

results = multilingual_search("Datenbank")
```

## Performance-Optimierung

### Index-Tuning

#### Analyse der Index-Nutzung:

```
-- Index-Statistiken anzeigen
SHOW INDEX STATS idx_articles_content;
```

Ergebnis:

```
total_documents: 125000
total_terms: 2500000
avg_terms_per_doc: 20
index_size_mb: 45.2
last_update: 2024-12-28 10:30:00
```

#### Selective Indexing:

```
-- Nur wichtige Felder indizieren
CREATE FULLTEXT INDEX idx_articles_selective
ON articles (title, abstract) -- Nicht "content" für Performance
WHERE status = 'published'; -- Nur veröffentlichte Artikel
```

#### Partial Index für häufige Queries:

```
-- Index nur für aktuelle Artikel
CREATE FULLTEXT INDEX idx_articles_recent
ON articles (title, content)
WHERE created_at > NOW() - INTERVAL '30 days';
```

### Query-Optimierung

#### Avoid Wildcards am Anfang:

```
-- LANGSAM: Wildcard am Anfang
WHERE FULLTEXT_MATCH(articles, '*base');

-- SCHNELL: Wildcard am Ende
WHERE FULLTEXT_MATCH(articles, 'data*');
```

**Limit Fuzzy Search:**

```
-- LANGSAM: Fuzzy auf allen Begriffen
WHERE FULLTEXT_MATCH(articles, 'databse~ systm~');

-- SCHNELL: Fuzzy nur wo nötig
WHERE FULLTEXT_MATCH(articles, 'database systm~');
```

**Use Score Threshold:**

```
-- Filtere irrelevante Ergebnisse
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
 AND FULLTEXT_SCORE(articles) > 0.5 -- Nur relevante Treffer
ORDER BY FULLTEXT_SCORE(articles) DESC
LIMIT 20;
```

**Caching-Strategien****Query-Result-Cache:**

```
from functools import lru_cache
from themisdb import ThemisDB

db = ThemisDB()

@lru_cache(maxsize=1000)
def cached_search(query, limit=20):
 return db.query("""
 SELECT * FROM articles
 WHERE FULLTEXT_MATCH(articles, ?)
 ORDER BY FULLTEXT_SCORE(articles) DESC
 LIMIT ?
 """, [query, limit])

results1 = cached_search("database") # DB-Query
results2 = cached_search("database") # Aus Cache
```

**Index-Warm-Up:**

```
-- Index vorwärmen nach Restart
SELECT COUNT(*) FROM articles
WHERE FULLTEXT_MATCH(articles, 'a'); -- Häufiger Buchstabe
```

## Best Practices

### 1. Index-Design

✓ **DO:** - Indiziere nur durchsuchbare Felder - Verwende gewichtete Felder (Titel wichtiger als Content)  
- Nutze sprachspezifische Analyzer - Implementiere Partial Indexes für große Tabellen

✗ **DON'T:** - Alle Felder blindlings indizieren - Binärdaten oder IDs indizieren - Stop-Words  
komplett entfernen (bei Phrasensuche wichtig) - Zu aggressive Stemming-Regeln

### 2. Query-Patterns

✓ **DO:** - Kombiniere Volltext mit strukturierten Filtern - Verwende Relevanz-Ranking - Implementiere  
Pagination für große Result-Sets - Cache häufige Queries

✗ **DON'T:** - Wildcard-Suchen am Anfang (\*word) - Zu komplexe Boolean-Queries -  
Uneingeschränkte Fuzzy-Searches - Volltext-Suche für exakte Matches (verwende =)

### 3. User Experience

✓ **DO:** - Auto-Complete für bessere UX - Highlight Suchbegriffe in Ergebnissen - Zeige Relevanz-  
Score an - Implementiere "Did you mean?" für Tippfehler

✗ **DON'T:** - Keine Ergebnisse ohne Alternative - Zu viele Ergebnisse ohne Ranking - Langsame  
Suche ohne Loading-Indicator - Keine Filteroptionen

### 4. Wartung

✓ **DO:** - Regelmäßige Index-Optimierung - Monitoring der Query-Performance - Analyse beliebter  
Suchbegriffe - A/B-Testing verschiedener Ranking-Algorithmen

✗ **DON'T:** - Index-Fragmentierung ignorieren - Keine Metriken erfassen - Statische Ranking-  
Gewichte - Keine User-Feedback-Integration

## Zusammenfassung

Volltext-Suche ist essentiell für moderne Anwendungen:

**Kernkonzepte:** - ☒ Invertierte Indexe für schnelle Suche - ☒ TF-IDF / BM25 für Relevanz-Ranking - ☒ Sprachverarbeitung (Stemming, Stop-Words) - ☒ Boolean-Operatoren (AND, OR, NOT) - ☒ Erweiterte Features (Fuzzy, Wildcards, Phrases)

**NLP-Integration:** - ☒ Sentiment-Analyse für Meinungen - ☒ Named Entity Recognition für Entitäten - ☒ Text-Klassifikation für Auto-Tagging - ☒ Keyword-Extraktion für Zusammenfassungen

**Performance:** - ☒ Selective Indexing - ☒ Query-Optimierung - ☒ Caching-Strategien - ☒ Index-Wartung

ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen kombiniert werden kann - für leistungsstarke, flexible Suchlösungen.

Im nächsten Kapitel behandeln wir **Geo-Spatial Features** - für standortbasierte Anwendungen und räumliche Analysen.

---

## Übungsaufgaben

**Aufgabe 1:** Erstellen Sie einen gewichteten Volltext-Index für eine Nachrichten-Tabelle (Titel 3×, Teaser 2×, Content 1×).

**Aufgabe 2:** Implementieren Sie eine Auto-Complete-Funktion mit Präfix-Suche und Häufigkeits-Ranking.

**Aufgabe 3:** Integrieren Sie Sentiment-Analyse für Produkt-Reviews und berechnen Sie durchschnittliche Sentiment-Scores.

**Aufgabe 4:** Erstellen Sie eine mehrsprachige Suche mit automatischer Language-Detection und sprachspezifischen Indexen.

**Aufgabe 5:** Optimieren Sie eine langsame Volltext-Query durch Analyse der Index-Statistiken und Query-Rewriting.

---



## Kapitel 17: Kapitel 14: Geo-Spatial Features

---

### Einleitung

Standortbasierte Dienste sind aus modernen Anwendungen nicht mehr wegzudenken. Von Lieferdiensten über Immobiliensuche bis zu Flottenmanagement – geografische Daten spielen eine zentrale Rolle. ThemisDB bietet native Unterstützung für Geo-Spatial Daten mit effizienten räumlichen Indizes und umfangreichen Abfragefunktionen.

In diesem Kapitel lernen Sie: - Geo-Datentypen und Koordinatensysteme - Räumliche Indizes (R-Tree, Geo-Hash) - Location-Based Queries (Radius, Bounding Box, Polygon) - Integration von GPS-Tracking und Kartendiensten - Best Practices für Geo-Anwendungen

### 14.1 Grundlagen der Geo-Spatial Daten

#### Koordinatensysteme

ThemisDB verwendet das WGS84-Koordinatensystem (EPSG:4326), das auch GPS verwendet:

```
berlin_coordinates = [13.405, 52.520]
munich_coordinates = [11.575, 48.137]
```

## Geo-Datentypen in ThemisDB

```
// Point: Einzelner Punkt
FOR loc IN [
 {
 _key: "loc1",
 name: "Berlin Mitte",
 coordinates: [13.4050, 52.5200], // [lon, lat]
 created_at: DATE_ISO8601(DATE_NOW())
 }
] INSERT loc INTO locations

-- Geo-Index erstellen
CREATE INDEX idx_geo_locations
 ON locations (coordinates)
 TYPE GEO

// LineString: Verbundene Punkte (Routen)
FOR route IN [
 {
 _key: "route1",
 name: "Tour A",
 path: [
 [13.377, 52.516], // Start
 [13.390, 52.520], // Waypoint 1
 [13.405, 52.520] // End
],
 distance_km: 2.8
 }
] INSERT route INTO routes

// Polygon: Geschlossene Fläche (Delivery Zones)
FOR zone IN [
 {
 _key: "zone1",
 name: "Zone Nord",
 area: [
 [13.35, 52.54],
 [13.42, 52.54],
 [13.42, 52.50],
 [13.35, 52.50],
 [13.35, 52.54] // Geschlossen
],
 active: true
 }
] INSERT zone INTO delivery_zones
```

## 14.2 Geo-Spatial Indizes

### R-Tree Index

R-Trees sind die effizienteste Indexstruktur für räumliche Daten:

```
-- Geo-Index erstellen
CREATE INDEX idx_locations_geo ON locations USING RTREE(coordinates);

-- Automatische Nutzung bei räumlichen Queries
FOR location IN locations
 FILTER ST_Distance(location.coordinates, POINT(13.405, 52.520)) < 5000
 RETURN location
-- Index wird automatisch genutzt!
```

**R-Tree Eigenschaften:** - Organisiert Punkte in hierarchischen Bounding Boxes -  $O(\log n)$  Suchzeit für Bereichsabfragen - Automatische Rebalancierung bei Updates - Optimiert für Nearest-Neighbor Queries

```

graph TB
 subgraph "R-Tree Hierarchical Index Structure"
 Root[Root Node
Global Bounding Box]

 Root --> L1A[Level 1A
Box: North Region]
 Root --> L1B[Level 1B
Box: South Region]

 L1A --> L2A[Level 2A
Box: Northwest]
 L1A --> L2B[Level 2B
Box: Northeast]

 L1B --> L2C[Level 2C
Box: Southwest]
 L1B --> L2D[Level 2D
Box: Southeast]

 L2A --> P1[Point 1]
 L2A --> P2[Point 2]
 L2B --> P3[Point 3]
 L2B --> P4[Point 4]
 L2C --> P5[Point 5]
 L2D --> P6[Point 6]
 L2D --> P7[Point 7]
 end

 style Root fill:#667eea
 style L1A fill:#4facfe
 style L1B fill:#4facfe
 style L2A fill:#43e97b
 style L2B fill:#43e97b
 style L2C fill:#43e97b
 style L2D fill:#43e97b
 style P1 fill:#f093fb
 style P2 fill:#f093fb
 style P3 fill:#f093fb
 style P4 fill:#f093fb
 style P5 fill:#f093fb
 style P6 fill:#f093fb
 style P7 fill:#f093fb

```

**Abb. 14.1:** Geospatial-Index-Struktur

## Geo-Hash Index

Für weltweite Abdeckung und Präfix-Suche:

```
CREATE INDEX idx_locations_geohash ON locations USING GEOHASH(coordinates);
```

```
-- Nützlich für:
-- - Clustering auf Karten
-- - Hierarchische Zoomlevel
-- - Verteilte Systeme (Sharding nach Geo-Hash)
```

## 14.3 Räumliche Abfragen

### Distanz-Queries

```
from themisdb import Connection

conn = Connection("localhost:7687")

restaurants = conn.query("""
 SELECT id, name, coordinates,
 ST_Distance(coordinates, POINT(?, ?)) as distance_m
 FROM restaurants
 WHERE ST_Distance(coordinates, POINT(?, ?)) < 2000
 ORDER BY distance_m
""", [user_lon, user_lat, user_lon, user_lat])

for r in restaurants:
 print(f'{r['name']}: {r['distance_m']:.0f}m entfernt")
```

### Bounding Box Queries

Rechteckiger Bereich (sehr effizient mit R-Tree):

```
min_lon, max_lon = 13.35, 13.45
min_lat, max_lat = 52.50, 52.55

pois = conn.query("""
 FOR poi IN points_of_interest
 FILTER ST_Within(poi.coordinates,
 BBOX(@min_lon, @min_lat, @max_lon, @max_lat))

 RETURN poi
""", {"min_lon": min_lon, "min_lat": min_lat, "max_lon": max_lon, "max_lat": max_lat})
```

```

graph TB
 subgraph "Geospatial Query Types"
 Center((User
Location))

 subgraph "Radius Query"
 Center -. -> |2km radius| R1[Restaurant 1]
 Center -. -> |1.5km| R2[Restaurant 2]
 Center -. -> |3km| R3[Restaurant 3]
 end

 subgraph "Bounding Box Query"
 BB[Bounding Box
min/max lat/lon]
 BB --> POI1[POI 1]
 BB --> POI2[POI 2]
 BB --> POI3[POI 3]
 end

 subgraph "Polygon Query"
 Poly[Delivery Zone
Polygon]
 Poly --> A1[Address 1
Inside]
 Poly --> A2[Address 2
Inside]
 Outside[Address 3
Outside]
 end
 end

 style Center fill:#667eea
 style R1 fill:#43e97b
 style R2 fill:#43e97b
 style R3 fill:#f093fb
 style BB fill:#4facfe
 style POI1 fill:#43e97b
 style POI2 fill:#43e97b
 style POI3 fill:#43e97b
 style Poly fill:#fa709a
 style A1 fill:#43e97b
 style A2 fill:#43e97b
 style Outside fill:#ff6348

```

**Abb. 14.2:** Proximity-Search-Algorithmus

## Polygon Queries

Prüfen ob Punkt innerhalb eines Polygons liegt:

```

delivery_area = [
 [13.40, 52.52], # Punkt 1
 [13.42, 52.52], # Punkt 2
 [13.42, 52.50], # Punkt 3
 [13.40, 52.50], # Punkt 4
 [13.40, 52.52] # Schließt Polygon
]

customer_location = [13.41, 52.51]

result = conn.query("""
 SELECT ST_Contains(
 POLYGON(?),
 POINT(?, ?)
) as in_delivery_area
""", [delivery_area, customer_location[0], customer_location[1]])

if result[0]['in_delivery_area']:
 print("Wir liefern zu Ihnen!")

```

## K-Nearest-Neighbor (KNN)

Die K nächsten Punkte finden:

```

cafes = conn.query("""
 SELECT id, name, coordinates,
 ST_Distance(coordinates, POINT(?, ?)) as distance_m
 FROM cafes
 ORDER BY distance_m
 LIMIT 5
""", [user_lon, user_lat])

```

## 14.4 Geo-Funktionen

### Distanzberechnung

```
-- Haversine-Formel (Kugeloberfläche der Erde)
ST_Distance(point1, point2) -> meters

-- Beispiel: Berlin → München
SELECT ST_Distance(
 POINT(13.405, 52.520), -- Berlin
 POINT(11.575, 48.137) -- München
) as distance_m;
-- Ergebnis: ~504.000m (504km)
```

### Bearing (Richtung)

```
-- Peilung von A nach B in Grad (0° = Norden)
ST_Bearing(point1, point2) -> degrees

SELECT ST_Bearing(
 POINT(13.405, 52.520), -- Berlin
 POINT(11.575, 48.137) -- München
) as bearing;
-- Ergebnis: ~195° (Süd-Südwest)
```

### Bounding Box

```
-- Kleinste Box um Geometrie
ST_Envelope(geometry) -> bbox

-- Buffer: Bereich um Punkt/Linie
ST_Buffer(point, radius_m) -> polygon
```

## 14.5 Beispiel: Location-Based Services (LBS)

Wir bauen einen Lieferdienst mit Echtzeit-Tracking:



## Datenmodell

```
conn.execute("""
CREATE TABLE IF NOT EXISTS restaurants (
 id INTEGER PRIMARY KEY,
 name TEXT NOT NULL,
 cuisine TEXT,
 coordinates POINT NOT NULL,
 rating REAL,
 delivery_radius_m INTEGER DEFAULT 3000
)
""")

conn.execute("""
CREATE INDEX idx_restaurants_geo
ON restaurants USING RTREE(coordinates)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS orders (
 id INTEGER PRIMARY KEY,
 restaurant_id INTEGER REFERENCES restaurants(id),
 customer_name TEXT,
 delivery_address POINT NOT NULL,
 status TEXT DEFAULT 'pending',
 driver_id INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS drivers (
 id INTEGER PRIMARY KEY,
 name TEXT,
 current_location POINT,
 status TEXT DEFAULT 'available',
 last_update TIMESTAMP
)
""")

conn.execute("""
CREATE INDEX idx_drivers_geo
ON drivers USING RTREE(current_location)
""")
```

## Restaurant-Suche

```
def find_nearby_restaurants(user_lat, user_lon, cuisine=None, max_distance_m=5000):
 """Findet Restaurants in der Nähe"""

 query = """
 SELECT id, name, cuisine, coordinates,
 rating,
 ST_Distance(coordinates, POINT(?, ?)) as distance_m
 FROM restaurants
 WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
 """

 params = [user_lon, user_lat, user_lon, user_lat, max_distance_m]

 if cuisine:
 query += " AND cuisine = ?"
 params.append(cuisine)

 query += " ORDER BY distance_m"

 return conn.query(query, params)

restaurants = find_nearby_restaurants(
 user_lat=52.520,
 user_lon=13.405,
 cuisine="Italian",
 max_distance_m=3000
)

for r in restaurants:
 distance_km = r['distance_m'] / 1000
 print(f"{r['name']}: {distance_km:.1f}km, Rating: {r['rating']}/5")
```

## **Fahrer-Zuordnung**

---

```

def assign_nearest_driver(order_id):
 """Findet den nächsten verfügbaren Fahrer"""

 # Hole Bestelladresse
 order = conn.query("""
 FOR order IN orders
 FILTER order.id == @order_id
 LIMIT 1
 RETURN order.delivery_address
 """, {"order_id": order_id})[0]

 delivery_loc = order

 # Finde nächsten verfügbaren Fahrer
 driver = conn.query("""
 FOR driver IN drivers
 FILTER driver.status == 'available'
 LET distance_m = ST_Distance(driver.current_location, POINT(@lon, @lat))
 SORT distance_m ASC
 LIMIT 1
 RETURN {
 id: driver.id,
 name: driver.name,
 current_location: driver.current_location,
 distance_m
 }
 """, {"lon": delivery_loc[0], "lat": delivery_loc[1]})

 if not driver:
 raise ValueError("Kein Fahrer verfügbar")

 driver = driver[0]

 # Zuordnen
 conn.execute("""
 UPDATE orders SET driver_id = ?, status = 'assigned'
 WHERE id = ?
 """, [driver['id'], order_id])

 conn.execute("""
 UPDATE drivers SET status = 'busy' WHERE id = ?
 """, [driver['id']])

 return driver

driver = assign_nearest_driver(order_id=123)
print(f"Fahrer {driver['name']} zugewiesen")

```

## Live-Tracking

```
import time
from datetime import datetime

def update_driver_location(driver_id, lat, lon):
 """Aktualisiert Fahrerposition (z.B. alle 5 Sekunden)"""

 conn.execute("""
 UPDATE drivers
 SET current_location = POINT(?, ?),
 last_update = ?
 WHERE id = ?
 """, [lon, lat, datetime.now(), driver_id])

def get_delivery_eta(order_id):
 """Schätzt Ankunftszeit"""

 result = conn.query("""
 SELECT o.id, o.delivery_address,
 d.current_location,
 ST_Distance(d.current_location, o.delivery_address) as distance_m
 FROM orders o
 JOIN drivers d ON o.driver_id = d.id
 WHERE o.id = ?
 """, [order_id])

 if not result:
 return None

 data = result[0]
 distance_km = data['distance_m'] / 1000

 # Annahme: 30 km/h in der Stadt
 eta_minutes = (distance_km / 30) * 60

 return {
 'distance_km': round(distance_km, 1),
 'eta_minutes': round(eta_minutes),
 'driver_location': data['current_location']
 }

eta = get_delivery_eta(order_id=123)
print(f"Fahrer ist noch {eta['distance_km']}km entfernt")
print(f"ETA: {eta['eta_minutes']} Minuten")
```

## 14.6 Beispiel: Immobiliensuche

Geo-basierte Immobiliensuche mit Filterung:

### Datenmodell

```
conn.execute("""
CREATE TABLE IF NOT EXISTS properties (
 id INTEGER PRIMARY KEY,
 title TEXT NOT NULL,
 address TEXT,
 coordinates POINT NOT NULL,
 price_eur INTEGER,
 size_sqm REAL,
 rooms INTEGER,
 type TEXT, -- 'apartment', 'house', 'commercial'
 features JSON -- ['balcony', 'parking', 'elevator', ...]
)
""")

conn.execute("""
CREATE INDEX idx_properties_geo
ON properties USING RTREE(coordinates)
""")
```

---

## **Radius-Suche mit Filtern**

```

def search_properties(
 center_lat, center_lon, radius_m=2000,
 min_price=None, max_price=None,
 min_rooms=None, property_type=None,
 required_features=None
):
 """Immobilien suche mit Geo + Filter"""

 query = """
 SELECT id, title, address, coordinates,
 price_eur, size_sqm, rooms, type, features,
 ST_Distance(coordinates, POINT(?, ?)) as distance_m
 FROM properties
 WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
 """

 params = [center_lon, center_lat, center_lon, center_lat, radius_m]

 if min_price:
 query += " AND price_eur >= ?"
 params.append(min_price)

 if max_price:
 query += " AND price_eur <= ?"
 params.append(max_price)

 if min_rooms:
 query += " AND rooms >= ?"
 params.append(min_rooms)

 if property_type:
 query += " AND type = ?"
 params.append(property_type)

 query += " ORDER BY distance_m"

 results = conn.query(query, params)

 # Nachfilterung für Features (JSON)
 if required_features:
 filtered = []
 for prop in results:
 prop_features = set(prop['features'] or [])
 if all(f in prop_features for f in required_features):
 filtered.append(prop)
 results = filtered

 return results

```



```

properties = search_properties(
 center_lat=52.520,
 center_lon=13.405,
 radius_m=3000,
 min_price=500_000,
 max_price=800_000,
 min_rooms=3,
 property_type='apartment',
 required_features=['balcony', 'parking']
)

for p in properties:
 print(f"{p['title']}: {p['price_eur']:,}€, {p['rooms']} Zimmer")
 print(f" {p['distance_m']/1000:.1f}km entfernt")

```

## POI-basierte Suche

Immobilien in der Nähe von Points of Interest:

```

def search_near_poi(poi_name, radius_m=1000):
 """Immobilien in der Nähe eines POI"""

 # Finde POI
 poi = conn.query("""
 FOR poi IN points_of_interest
 FILTER poi.name == @poi_name
 LIMIT 1
 RETURN poi.coordinates
 """, {"poi_name": poi_name})

 if not poi:
 raise ValueError(f"POI '{poi_name}' nicht gefunden")

 poi_coords = poi[0]['coordinates']

 # Suche Immobilien
 return conn.query("""
 SELECT id, title, price_eur,
 ST_Distance(coordinates, POINT(?, ?)) as distance_m
 FROM properties
 WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
 ORDER BY distance_m
 """, [poi_coords[0], poi_coords[1],
 poi_coords[0], poi_coords[1], radius_m])

properties = search_near_poi("Alexanderplatz", radius_m=1500)

```

## 14.7 Best Practices

### 1. Index-Design

```
CREATE INDEX idx_geo ON locations USING RTREE(coordinates);

CREATE INDEX idx_geo_type ON locations(type, coordinates) USING RTREE;

CREATE INDEX idx_coords ON locations(coordinates); # B-Tree, ineffizient!
```

### 2. Query-Optimierung

```
WHERE ST_Distance(coordinates, ?) < radius

FOR location IN locations
 LET dist = ST_Distance(location.coordinates, @point)
 RETURN {location, dist}
-- Keine Index-Nutzung! Alle Rows werden berechnet
```

### 3. Koordinaten-Validierung

```
def validate_coordinates(lon, lat):
 """Prüft ob Koordinaten gültig sind"""
 if not (-180 <= lon <= 180):
 raise ValueError(f"Longitude {lon} außerhalb [-180, 180]")
 if not (-90 <= lat <= 90):
 raise ValueError(f"Latitude {lat} außerhalb [-90, 90]")
 return True

validate_coordinates(lon, lat)
```

### 4. Präzision und Rundung

```
coordinates = [round(lon, 6), round(lat, 6)]
```

## 5. Caching für häufige Queries

```
from functools import lru_cache
import time

@lru_cache(maxsize=1000)
def get_cached_nearby_restaurants(lat, lon, radius):
 """Cached Restaurant-Suche"""
 # Cache für 5 Minuten
 cache_key = (round(lat, 3), round(lon, 3), radius)
 return find_nearby_restaurants(lat, lon, max_distance_m=radius)
```

## 14.8 Performance-Tipps

### 1. Bounding Box vor Distanz-Berechnung

```
FOR location IN locations
 FILTER ST_Within(location.coordinates, BBOX(@min_lon, @min_lat, @max_lon, @max_lat)) -- S
 AND ST_Distance(location.coordinates, POINT(@lon, @lat)) < @radius -- Nur für Kandidaten
RETURN location
```

### 2. Limit verwenden

```
FOR location IN locations
 LET dist = ST_Distance(location.coordinates, POINT(@lon, @lat))
 FILTER dist < 5000
 SORT dist ASC
 LIMIT 10 -- Stoppt nach 10 Ergebnissen
RETURN location
```

### 3. Materializedviews für häufige Gebiete

```
conn.execute("""
CREATE MATERIALIZED VIEW berlin_mitte_restaurants AS
FOR restaurant IN restaurants
 FILTER ST_Within(restaurant.coordinates, BBOX(13.35, 52.50, 13.45, 52.55))
 RETURN restaurant
""")

conn.execute("REFRESH MATERIALIZED VIEW berlin_mitte_restaurants")
```

## 14.9 Vergleich mit spezialisierten Geo-Systemen

Feature	ThemisDB	PostGIS	MongoDB Geo
R-Tree Index	✓ Native	✓ GiST	✓ 2dsphere
Distanz-Queries	✓	✓	✓
Polygon-Support	✓	✓✓ (komplex)	✓
3D-Geometrie	✗	✓	✗
Koordinatensysteme	WGS84	Viele	WGS84
Performance	★★★★★	★★★★★★	★★★★★
Multi-Model	✓✓✓	✗	✓
Einfachheit	✓✓✓	★★	✓✓

**Empfehlung:** - **ThemisDB:** Gut für die meisten Anwendungen, besonders mit Multi-Model Daten - **PostGIS:** Beste Wahl für komplexe GIS-Anwendungen - **MongoDB:** Gut für Dokument + Geo Kombination

## 14.10 Übungen

### Übung 1: Store Locator

Implementieren Sie einen Store Locator mit: - Nächste 5 Filialen - Öffnungszeiten-Filterung - Routenvorschlag

### Übung 2: Geofencing

Erstellen Sie ein Geofencing-System: - Definieren Sie virtuelle Zonen (Polygone) - Trigger beim Ein/Austritt - Benachrichtigungen

### Übung 3: Heat Map

Generieren Sie eine Heat Map: - Aggregiere Punkte nach Geo-Hash - Cluster für verschiedene Zoom-Level - Export für Mapping-Tools

## Zusammenfassung

In diesem Kapitel haben Sie gelernt:

---

✓ **Geo-Datentypen:** Point, LineString, Polygon ✓ **Räumliche Indizes:** R-Tree für effiziente Geo-Queries ✓ **Abfragen:** Radius, Bounding Box, KNN, Polygon ✓ **Praxis-Beispiele:** Lieferdienst, Immobiliensuche ✓ **Best Practices:** Index-Design, Performance, Validation

Geo-Spatial Features in ThemisDB bieten eine solide Grundlage für Location-Based Services ohne externe Geo-Datenbank. Die native Integration mit anderen Datenmodellen (Relational, Graph, Dokument, Vector) macht ThemisDB zur idealen Plattform für moderne Geo-Anwendungen.

Im nächsten Kapitel sehen wir uns **Analytics & Reporting** an – Aggregationen, Dashboards und Business Intelligence mit ThemisDB.

---

# Kapitel 18: Kapitel 15: Analytics & Reporting

## 15.1 Einführung in Analytics mit ThemisDB

ThemisDB bietet leistungsstarke Analyse- und Reporting-Funktionen, die es ermöglichen, komplexe Geschäftslogik direkt in der Datenbank auszuführen. Durch die Kombination von relationalen Aggregationen, Graph-Analysen und Vektor-basierten Ähnlichkeitssuchen können umfassende Business-Intelligence-Lösungen erstellt werden.

### 15.1.1 Warum Analytics in der Datenbank?

**Vorteile:** - **Performance:** Datenverarbeitung direkt am Speicherort - **Konsistenz:** ACID-Garantien für Analyseergebnisse - **Echtzeit:** Keine ETL-Verzögerungen - **Multi-Model:** Kombinierte Analysen über verschiedene Datenmodelle

## 15.9 OLAP Cube Design (Neu)

### 15.9.1 Star Schema

```

flowchart TB
 F[Fact: sales] --> D1[Dim: time]
 F --> D2[Dim: product]
 F --> D3[Dim: customer]
 F --> D4[Dim: region]

 D1 --> D1a[day]
 D1a --> D1b[month]
 D1b --> D1c[year]

 D2 --> D2a[category]
 D2a --> D2b[subcategory]
 D2b --> D2c[product]

 style F fill:#e1f5ff
 style D1 fill:#fff4e1
 style D2 fill:#fff4e1
 style D3 fill:#fff4e1
 style D4 fill:#fff4e1

```

**Abb. 15.1:** Analytics-Query-Pipeline

**Keys:** - Fact: `time_key` , `product_key` , `customer_key` , `region_key` - Dim: Surrogate Keys ( `int` ) für schnelle Joins

## 15.9.2 Slowly Changing Dimensions (SCD)

Typ	Verhalten	Beispiel
<b>SCD1</b>	Überschreibt alte Werte	Preis aktualisieren
<b>SCD2</b>	Historisiert mit <code>valid_from/valid_to</code>	Kundenadresse
<b>SCD3</b>	Speichert Vorversion im gleichen Datensatz	Vorheriger Status

### AQL (SCD2 Update):

```

LET now = DATE_NOW()

FOR dim IN dim_product
 FILTER dim.product_id == @product_id
 FILTER dim.valid_to == null
 UPDATE dim WITH { valid_to: now } IN dim_product

INSERT {
 product_id: @product_id,
 name: @name,
 category: @category,
 valid_from: now,
 valid_to: null
} INTO dim_product

```

## 15.9.3 Derived & Aggregate Tables

- **Rollups:** day → month → quarter → year
- **Pre-Aggregation:** Umsatz pro Region/Produkt/Quartal
- **Materialized Views:** Nightly Refresh oder Streaming (CDC)

```
-- Quartals-Rollup
FOR fact IN sales
 COLLECT
 year = DATE_YEAR(fact.ts),
 quarter = DATE_QUARTER(fact.ts),
 region = fact.region,
 category = fact.category
 AGGREGATE
 revenue = SUM(fact.amount),
 units = SUM(fact.quantity)
 RETURN {
 year, quarter, region, category, revenue, units
 }
```



## 15.10 Incremental Materialization & CDC

### 15.10.1 Changefeed → Materialized View

```

"""Incremental Refresh für sales_rollup (hourly)"""
from datetime import datetime, timedelta

def refresh_sales_rollup(db, since_ts):
 events = db.changefeed("sales", {"since": since_ts})
 buffer = []
 for ev in events:
 buffer.append(ev)
 if len(buffer) >= 1000:
 apply_buffer(db, buffer)
 buffer.clear()
 apply_buffer(db, buffer)

def apply_buffer(db, buffer):
 db.query("""
 FOR ev IN @events
 LET month = DATE_TRUNC('month', ev.ts)
 UPSERT { month, region: ev.region, category: ev.category }
 INSERT {
 month,
 region: ev.region,
 category: ev.category,
 revenue: ev.amount,
 units: ev.quantity
 }
 UPDATE {
 revenue: OLD.revenue + ev.amount,
 units: OLD.units + ev.quantity
 }
 IN sales_rollup
 """, {"events": buffer})

```

### 15.10.2 Late Arriving Facts & Corrections

```
-- Korrekturen erkennen
FOR ev IN changefeed_sales
 FILTER ev.event_type == "correction"
 LET prev = DOCUMENT(sales_rollup, ev.rollup_key)
 UPDATE prev WITH {
 revenue: prev.revenue - ev.old_amount + ev.new_amount,
 units: prev.units - ev.old_qty + ev.new_qty
 } IN sales_rollup
```

### 15.10.3 OLAP Query Patterns

```
-- Drill-down: Jahr → Monat → Tag
FOR row IN sales_rollup
 FILTER row.year == 2025
 COLLECT month = row.month
 AGGREGATE revenue = SUM(row.revenue)
 SORT month
 RETURN { month, revenue }

-- Pivot nach Region
FOR row IN sales_rollup
 COLLECT category = row.category
 AGGREGATE
 eu = SUM(row.region == 'EU' ? row.revenue : 0),
 us = SUM(row.region == 'US' ? row.revenue : 0),
 apac = SUM(row.region == 'APAC' ? row.revenue : 0)
 RETURN { category, eu, us, apac }
```

```

class AnalyticsGovernance:
 """Data Governance für Analytics"""

 def __init__(self, db):
 self.db = db

 def anonymize_customer_data(self, dataset):
 """Anonymisierung sensibler Daten"""
 return [{
 **row,
 'customer_id': hash(row['customer_id']),
 'email': None,
 'name': None
 } for row in dataset]

 def apply_row_level_security(self, query, user_role, user_region):
 """Row-Level Security anwenden"""
 if user_role == 'regional_manager':
 query += f" FILTER doc.region == '{user_region}'"
 elif user_role == 'analyst':
 # Nur aggregierte Daten
 query += " COLLECT ... AGGREGATE ..."

 return query

 def audit_query(self, user_id, query, timestamp):
 """Query-Audit-Log"""
 self.db.query("""
 INSERT {
 user_id: @user_id,
 query: @query,
 timestamp: @timestamp,
 type: 'analytics'
 } INTO audit_log
 """, {
 'user_id': user_id,
 'query': query,
 'timestamp': timestamp
 })

```

## 15.11 Zusammenfassung

### 15.2.1 Standard AQL-Aggregationen

ThemisDB unterstützt alle Standard-AQL-Aggregationsfunktionen:

```
-- Verkaufsstatistiken
FOR order IN orders
 FILTER order.order_date >= '2024-01-01'
 COLLECT month = DATE_TRUNC('month', order.order_date)
 AGGREGATE
 total_orders = COUNT(),
 revenue = SUM(order.total_amount),
 avg_order_value = AVG(order.total_amount),
 min_order = MIN(order.total_amount),
 max_order = MAX(order.total_amount),
 order_stddev = STDDEV(order.total_amount)
 SORT month
 RETURN {month, total_orders, revenue, avg_order_value, min_order, max_order, order_stddev}
```

**Ausgabe:**

month	total_orders	revenue	avg_order_value	min_order	max_order	order_stddev
2024-01-01	1250	156780.50	125.42	12.50	1580.00	95.23
2024-02-01	1420	182340.75	128.45	9.99	2100.00	102.45

**15.2.2 Window Functions für Trend-Analysen**

```
-- Umsatztrend mit gleitendem Durchschnitt
FOR order IN orders
 SORT order.order_date DESC
 LIMIT 30
 LET moving_avg_7day = AVG_WINDOW(order.total_amount,
 {preceding: 6, following: 0})
 LET month_cumulative = SUM_WINDOW(order.total_amount,
 {partition: DATE_TRUNC('month', order.order_date)})
 LET rank_in_month = RANK_WINDOW(
 {partition: DATE_TRUNC('month', order.order_date),
 order: order.total_amount DESC})
 RETURN {
 order_date: order.order_date,
 total_amount: order.total_amount,
 moving_avg_7day,
 month_cumulative,
 rank_in_month
 }
```

### 15.2.3 PIVOT-Operationen

```
-- Umsatz nach Produktkategorie und Monat
LET pivot_data = (
 FOR order_item IN order_items
 FOR product IN products
 FILTER order_item.product_id == product.id
 COLLECT
 month = DATE_TRUNC('month', order_item.order_date),
 category = product.category
 AGGREGATE revenue = SUM(order_item.amount)
 RETURN {month, category, revenue}
)

// PIVOT operation (manual transformation)
FOR data IN pivot_data
 COLLECT month = data.month
 AGGREGATE
 electronics = SUM(data.category == 'Electronics' ? data.revenue : 0),
 clothing = SUM(data.category == 'Clothing' ? data.revenue : 0),
 books = SUM(data.category == 'Books' ? data.revenue : 0),
 home = SUM(data.category == 'Home' ? data.revenue : 0),
 sports = SUM(data.category == 'Sports' ? data.revenue : 0)
 RETURN {month, electronics, clothing, books, home, sports}
```

## 15.3 Erweiterte Aggregationen mit AQL

### 15.3.1 COLLECT für komplexe Gruppierungen

```
-- Kundensegmentierung nach Kaufverhalten
FOR order IN orders
 COLLECT
 customer_id = order.customer_id,
 age_group = FLOOR(order.customer_age / 10) * 10
 AGGREGATE
 order_count = COUNT(1),
 total_spent = SUM(order.total_amount),
 avg_order = AVG(order.total_amount),
 categories = UNIQUE(order.category)
 INTO group
 FILTER order_count >= 5
 LET segment = (
 total_spent > 5000 ? "VIP" :
 total_spent > 1000 ? "Premium" :
 "Regular"
)
 RETURN {
 customer_id,
 age_group,
 segment,
 metrics: {
 orders: order_count,
 total_revenue: total_spent,
 avg_order_value: avg_order,
 product_diversity: LENGTH(categories)
 }
 }
```

## 15.3.2 Multi-Level Aggregationen

```
-- Hierarchische Umsatzanalyse
FOR order IN orders
 FILTER order.date >= DATE_NOW() - 365*24*60*60*1000
 COLLECT
 year = DATE_YEAR(order.date),
 quarter = DATE_QUARTER(order.date),
 region = order.shipping_region
 AGGREGATE
 revenue = SUM(order.total_amount),
 order_count = COUNT(1)
 COLLECT
 year = year,
 quarter = quarter
 AGGREGATE
 total_revenue = SUM(revenue),
 total_orders = SUM(order_count),
 regions = COUNT(region)
 RETURN {
 period: CONCAT(year, "-Q", quarter),
 total_revenue,
 total_orders,
 avg_per_region: total_revenue / regions,
 regions_active: regions
 }
```

## 15.4 Graph Analytics für Beziehungsanalysen

### 15.4.1 Customer Network Analysis

```
-- Kunden mit ähnlichen Kaufmustern finden
FOR customer IN customers
 FILTER customer.id == @customerId

 // Produkte des Kunden
 LET customer_products = (
 FOR order IN orders
 FILTER order.customer_id == customer.id
 FOR item IN order.items
 RETURN DISTINCT item.product_id
)

 // Ähnliche Kunden finden
 LET similar_customers = (
 FOR other IN customers
 FILTER other.id != customer.id
 LET other_products = (
 FOR order IN orders
 FILTER order.customer_id == other.id
 FOR item IN order.items
 RETURN DISTINCT item.product_id
)
 LET intersection = LENGTH(
 INTERSECTION(customer_products, other_products)
)
 LET union_size = LENGTH(
 UNION(customer_products, other_products)
)
 LET jaccard = intersection / union_size
 FILTER jaccard > 0.3
 SORT jaccard DESC
 LIMIT 10
 RETURN {
 customer_id: other.id,
 similarity: jaccard,
 shared_products: intersection
 }
)

RETURN {
 customer_id: customer.id,
 similar_customers
}
```



## 15.4.2 Influencer-Identifikation im Social Graph

```
-- Top-Influencer nach PageRank
FOR user IN users
 LET followers_count = LENGTH(
 FOR edge IN follows
 FILTER edge._to == user._id
 RETURN 1
)
 LET engagement = (
 FOR post IN posts
 FILTER post.author_id == user.id
 RETURN SUM([
 post.likes_count,
 post.comments_count * 2,
 post.shares_count * 3
])
)
 LET avg_engagement = AVG(engagement)

 // PageRank-ähnliche Berechnung
 LET influence_score = (
 followers_count * 0.4 +
 avg_engagement * 0.6
)

 FILTER influence_score > 100
 SORT influence_score DESC
 LIMIT 50

 RETURN {
 user_id: user.id,
 username: user.username,
 followers: followers_count,
 avg_engagement,
 influence_score
 }
```

## **15.5 Vektor-basierte Analysen**

---

### **15.5.1 Produkt-Clustering nach Embeddings**

```

import themisdb
import numpy as np
from sklearn.cluster import KMeans

db = themisdb.connect("localhost:8529")

query = """
FOR product IN products
 FILTER product.embedding != null
 RETURN {
 id: product.id,
 name: product.name,
 embedding: product.embedding
 }
"""
products = db.query(query)

embeddings = np.array([p['embedding'] for p in products])
product_ids = [p['id'] for p in products]

kmeans = KMeans(n_clusters=10, random_state=42)
clusters = kmeans.fit_predict(embeddings)

for product_id, cluster_id in zip(product_ids, clusters):
 db.query("""
 UPDATE products
 SET cluster_id = @cluster
 WHERE id = @id
 """, {
 'id': product_id,
 'cluster': int(cluster_id)
 })

cluster_stats = db.query("""
FOR product IN products
 COLLECT cluster = product.cluster_id
 AGGREGATE
 count = COUNT(1),
 avg_price = AVG(product.price),
 categories = UNIQUE(product.category)
 RETURN {
 cluster_id: cluster,
 product_count: count,
 avg_price,
 main_categories: categories
 }
""")

```

```

for stats in cluster_stats:
 print(f"Cluster {stats['cluster_id']}: {stats['product_count']} Produkte")
 print(f" Durchschnittspreis: €{stats['avg_price']:.2f}")
 print(f" Kategorien: {'', '.join(stats['main_categories'][:3])}")

```

### 15.5.2 Anomalie-Erkennung mit Vektor-Distanzen

```

-- Ungewöhnliche Transaktionen identifizieren
FOR transaction IN transactions
 // Durchschnittliches Transaktionsprofil des Kunden
 LET customer_avg_embedding = (
 FOR t IN transactions
 FILTER t.customer_id == transaction.customer_id
 AND t.id != transaction.id
 RETURN t.transaction_embedding
)

 LET avg_embedding = (
 FOR emb IN customer_avg_embedding
 RETURN AVERAGE(emb)
)

 // Distanz zur Normalverteilung
 LET distance = VECTOR_DISTANCE(
 "cosine",
 transaction.transaction_embedding,
 avg_embedding
)

 // Anomalie-Score
 LET anomaly_score = distance > 0.7 ? 1.0 : distance / 0.7

 FILTER anomaly_score > 0.8
 SORT anomaly_score DESC
 LIMIT 100

 RETURN {
 transaction_id: transaction.id,
 customer_id: transaction.customer_id,
 amount: transaction.amount,
 anomaly_score,
 reason: anomaly_score > 0.95 ? "HIGH_RISK" : "REVIEW"
 }

```

## 15.6 Materialized Views für Performance

### 15.6.1 Erstellen von Materialized Views

```
-- Tägliche Verkaufsübersicht
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
 DATE(order_date) AS date,
 COUNT(*) AS order_count,
 SUM(total_amount) AS revenue,
 AVG(total_amount) AS avg_order_value,
 COUNT(DISTINCT customer_id) AS unique_customers,
 SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
FROM orders
GROUP BY DATE(order_date);

-- Refresh Strategy
CREATE TRIGGER refresh_daily_sales
 AFTER INSERT OR UPDATE ON orders
 FOR EACH STATEMENT
 EXECUTE PROCEDURE refresh_materialized_view('daily_sales_summary');
```

## 15.6.2 Inkrementelles Update

```
def update_daily_metrics(date):
 """Inkrementelles Update für einen Tag"""

 # Alte Metriken löschen
 db.query("""
 DELETE FROM daily_sales_summary
 WHERE date = @date
 """, {'date': date})

 # Neue Metriken berechnen
 db.query("""
 INSERT INTO daily_sales_summary
 SELECT
 @date AS date,
 COUNT(*) AS order_count,
 SUM(total_amount) AS revenue,
 AVG(total_amount) AS avg_order_value,
 COUNT(DISTINCT customer_id) AS unique_customers,
 SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
 FROM orders
 WHERE DATE(order_date) = @date
 """, {'date': date})
```

## **15.7 OLAP-Würfel und Mehrdimensionale Analysen**

### **15.7.1 Erstellen eines OLAP-Würfels**

```

import pandas as pd

def create_sales_cube(db):
 """OLAP-Würfel für Verkaufsanalysen"""

 query = """
 FOR order IN orders
 LET customer = DOCUMENT('customers', order.customer_id)
 LET items = (
 FOR item IN order.items
 LET product = DOCUMENT('products', item.product_id)
 RETURN {
 category: product.category,
 brand: product.brand,
 price: item.price,
 quantity: item.quantity
 }
)
 RETURN {
 date: order.order_date,
 customer_segment: customer.segment,
 customer_region: customer.region,
 items: items
 }
 """

 # Daten laden
 orders = list(db.query(query))

 # In flache Struktur umwandeln
 rows = []
 for order in orders:
 for item in order['items']:
 rows.append({
 'date': pd.to_datetime(order['date']),
 'year': pd.to_datetime(order['date']).year,
 'quarter': pd.to_datetime(order['date']).quarter,
 'month': pd.to_datetime(order['date']).month,
 'customer_segment': order['customer_segment'],
 'customer_region': order['customer_region'],
 'category': item['category'],
 'brand': item['brand'],
 'revenue': item['price'] * item['quantity'],
 'quantity': item['quantity']
 })

 # DataFrame erstellen
 df = pd.DataFrame(rows)

```



```

Pivot-Tabellen
cube = {
 'by_region_category': df.pivot_table(
 values='revenue',
 index='customer_region',
 columns='category',
 aggfunc='sum',
 fill_value=0
),
 'by_segment_month': df.pivot_table(
 values='revenue',
 index='customer_segment',
 columns='month',
 aggfunc='sum',
 fill_value=0
),
 'by_brand_quarter': df.pivot_table(
 values='revenue',
 index='brand',
 columns='quarter',
 aggfunc='sum',
 fill_value=0
)
}

return cube

cube = create_sales_cube(db)

region = 'Europe'
category = 'Electronics'

drill_down = db.query("""
 FOR order IN orders
 LET customer = DOCUMENT('customers', order.customer_id)
 FILTER customer.region == @region
 FOR item IN order.items
 LET product = DOCUMENT('products', item.product_id)
 FILTER product.category == @category
 COLLECT brand = product.brand
 AGGREGATE revenue = SUM(item.price * item.quantity)
 SORT revenue DESC
 RETURN {brand, revenue}
""", {'region': region, 'category': category})

```

## 15.7.2 Slice und Dice Operationen

```
def slice_cube(cube_data, dimension, value):
 """Slice-Operation auf dem Würfel"""
 return cube_data[cube_data[dimension] == value]

def dice_cube(cube_data, filters):
 """Dice-Operation mit mehreren Dimensionen"""
 result = cube_data
 for dim, values in filters.items():
 result = result[result[dim].isin(values)]
 return result

filters = {
 'customer_region': ['Europe', 'North America'],
 'category': ['Electronics', 'Books'],
 'year': [2024]
}
diced = dice_cube(df, filters)
```

## **15.8 Dashboard-Metriken in Echtzeit**

### **15.8.1 KPI-Berechnung**

```

class RealtimeDashboard:
 def __init__(self, db):
 self.db = db

 def get_current_metrics(self):
 """Aktuelle KPIs abrufen"""

 metrics = {}

 # Heutige Verkäufe
 today = self.db.query("""
 LET today = DATE_FORMAT(DATE_NOW(), '%Y-%m-%d')
 FOR order IN orders
 FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == today
 COLLECT AGGREGATE
 count = COUNT(1),
 revenue = SUM(order.total_amount),
 avg = AVG(order.total_amount)
 RETURN {count, revenue, avg}
 """)[0]

 metrics['today'] = today

 # Gestern zum Vergleich
 yesterday = self.db.query("""
 LET yesterday = DATE_FORMAT(
 DATE_SUBTRACT(DATE_NOW(), 1, 'day'),
 '%Y-%m-%d'
)
 FOR order IN orders
 FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == yesterday
 COLLECT AGGREGATE
 count = COUNT(1),
 revenue = SUM(order.total_amount)
 RETURN {count, revenue}
 """)[0]

 metrics['yesterday'] = yesterday

 # Prozentuale Veränderung
 metrics['change'] = {
 'orders': (today['count'] - yesterday['count']) / yesterday['count'] * 100,
 'revenue': (today['revenue'] - yesterday['revenue']) / yesterday['revenue'] *
 }

 # Aktive Benutzer
 metrics['active_users'] = self.db.query("""
 LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
 FOR session IN user_sessions

```

```

 FILTER session.last_activity >= last_hour
 RETURN DISTINCT session.user_id
 """).count()

Conversion Rate
metrics['conversion_rate'] = self.db.query("""
 LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
 LET visits = (
 FOR visit IN page_views
 FILTER visit.timestamp >= last_hour
 RETURN DISTINCT visit.session_id
)
 LET purchases = (
 FOR order IN orders
 FILTER order.order_date >= last_hour
 RETURN DISTINCT order.session_id
)
 RETURN LENGTH(purchases) / LENGTH(visits) * 100
""")[0]

return metrics

def get_top_products(self, limit=10):
 """Top-Produkte der letzten 24 Stunden"""
 return self.db.query("""
 LET last_24h = DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
 FOR order IN orders
 FILTER order.order_date >= last_24h
 FOR item IN order.items
 COLLECT product_id = item.product_id
 AGGREGATE
 quantity_sold = SUM(item.quantity),
 revenue = SUM(item.price * item.quantity)
 LET product = DOCUMENT('products', product_id)
 SORT revenue DESC
 LIMIT @limit
 RETURN {
 product_id,
 name: product.name,
 quantity_sold,
 revenue
 }
 """, {'limit': limit})

def get_geographic_distribution(self):
 """Geografische Verteilung der Verkäufe"""
 return self.db.query("""
 FOR order IN orders
 FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 7, 'day')
 LET customer = DOCUMENT('customers', order.customer_id)
 """)

```

```

 COLLECT
 country = customer.country,
 region = customer.region
 AGGREGATE
 orders = COUNT(1),
 revenue = SUM(order.total_amount)
 SORT revenue DESC
 RETURN {
 country,
 region,
 orders,
 revenue
 }
 """
)

```

```
dashboard = RealtimeDashboard(db)
```

```
metrics = dashboard.get_current_metrics()
```

```
print(f"Heute: {metrics['today']['count']} Bestellungen, €{metrics['today']['revenue']:.2f}")
```

```
print(f"Veränderung: {metrics['change']['orders']:+.1f}% Bestellungen, {metrics['change']['revenue']:+.1f}% Umsatz")
```

```
print(f"Aktive Benutzer: {metrics['active_users']}")
```

```
print(f"Conversion Rate: {metrics['conversion_rate']:.2f}%")
```

## 15.8.2 Change Data Capture für Live-Updates

```
def subscribe_to_metrics_updates(db, callback):
 """CDC-Stream für Echtzeit-Metriken"""

 stream = db.collection('orders').watch([
 {'$match': {'operationType': 'insert'}}
])

 for change in stream:
 # Neue Bestellung
 order = change['fullDocument']

 # Metriken aktualisieren
 updated_metrics = {
 'timestamp': order['order_date'],
 'order_id': order['_id'],
 'amount': order['total_amount'],
 'customer_id': order['customer_id']
 }

 # Callback für Dashboard-Update
 callback(updated_metrics)

def on_new_order(metrics):
 print(f"Neue Bestellung: €{metrics['amount']:.2f}")
 # Dashboard aktualisieren

subscribe_to_metrics_updates(db, on_new_order)
```

## 15.9 Predictive Analytics

### 15.9.1 Trend-Prognosen mit linearer Regression

```

from sklearn.linear_model import LinearRegression
import numpy as np

def forecast_revenue(db, days_ahead=30):
 """Umsatzprognose für die nächsten Tage"""

 # Historische Daten
 historical = db.query("""
 FOR order IN orders
 FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 90, 'day')
 COLLECT date = DATE_FORMAT(order.order_date, '%Y-%m-%d')
 AGGREGATE revenue = SUM(order.total_amount)
 SORT date
 RETURN {date, revenue}
 """)

 # Features vorbereiten
 X = np.array([[i] for i in range(len(historical))])
 y = np.array([h['revenue'] for h in historical])

 # Modell trainieren
 model = LinearRegression()
 model.fit(X, y)

 # Prognose
 future_X = np.array([[i] for i in range(len(historical), len(historical) + days_ahead)])
 forecast = model.predict(future_X)

 # Konfidenzintervall (vereinfacht)
 residuals = y - model.predict(X)
 std_error = np.std(residuals)
 confidence_interval = 1.96 * std_error # 95% CI

 return {
 'forecast': forecast.tolist(),
 'lower_bound': (forecast - confidence_interval).tolist(),
 'upper_bound': (forecast + confidence_interval).tolist()
 }

forecast = forecast_revenue(db, days_ahead=30)
print(f"Prognostizierter Umsatz in 30 Tagen: €{forecast['forecast'][-1]:.2f}")
print(f"Konfidenzintervall: €{forecast['lower_bound'][-1]:.2f} - €{forecast['upper_bound'][-1]:.2f}")

```



---

## **15.9.2 Churn-Vorhersage**

```

def calculate_churn_risk(db, customer_id):
 """Churn-Risiko für einen Kunden berechnen"""

 features = db.query("""
 LET customer = DOCUMENT('customers', @customer_id)
 LET orders = (
 FOR order IN orders
 FILTER order.customer_id == @customer_id
 SORT order.order_date DESC
 RETURN order
)

 LET days_since_last_order = DATE_DIFF(
 orders[0].order_date,
 DATE_NOW(),
 'day'
)

 LET order_frequency = LENGTH(orders) / (
 DATE_DIFF(
 customer.registration_date,
 DATE_NOW(),
 'day'
) / 30
)

 LET avg_order_value = AVG(
 FOR order IN orders
 RETURN order.total_amount
)

 LET total_spent = SUM(
 FOR order IN orders
 RETURN order.total_amount
)

 RETURN {
 days_since_last_order,
 order_frequency,
 avg_order_value,
 total_spent,
 support_tickets: LENGTH(customer.support_tickets),
 has_complained: LENGTH(
 FOR ticket IN customer.support_tickets
 FILTER ticket.type == 'complaint'
 RETURN 1
) > 0
 }
 """, {'customer_id': customer_id})[0]

```

```
Einfacher Risiko-Score
risk_score = 0

if features['days_since_last_order'] > 60:
 risk_score += 0.3
if features['order_frequency'] < 1:
 risk_score += 0.2
if features['avg_order_value'] < 50:
 risk_score += 0.1
if features['support_tickets'] > 5:
 risk_score += 0.2
if features['has_complained']:
 risk_score += 0.2

return {
 'customer_id': customer_id,
 'churn_risk': min(risk_score, 1.0),
 'risk_level': (
 'HIGH' if risk_score > 0.7 else
 'MEDIUM' if risk_score > 0.4 else
 'LOW'
),
 'features': features
}
```

## **15.10 Reporting und Export**

---

### **15.10.1 PDF-Report-Generierung**

```

from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.units import cm
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet

def generate_sales_report(db, start_date, end_date, filename):
 """Verkaufsbericht als PDF generieren"""

 # Daten abfragen
 summary = db.query("""
 FOR order IN orders
 FILTER order.order_date >= @start_date
 AND order.order_date <= @end_date
 COLLECT AGGREGATE
 total_orders = COUNT(1),
 total_revenue = SUM(order.total_amount),
 avg_order_value = AVG(order.total_amount)
 RETURN {
 total_orders,
 total_revenue,
 avg_order_value
 }
 """, {'start_date': start_date, 'end_date': end_date})[0]

 top_products = db.query("""
 FOR order IN orders
 FILTER order.order_date >= @start_date
 AND order.order_date <= @end_date
 FOR item IN order.items
 COLLECT product_id = item.product_id
 AGGREGATE
 quantity = SUM(item.quantity),
 revenue = SUM(item.price * item.quantity)
 LET product = DOCUMENT('products', product_id)
 SORT revenue DESC
 LIMIT 10
 RETURN [product.name, quantity, revenue]
 """, {'start_date': start_date, 'end_date': end_date})

 # PDF erstellen
 doc = SimpleDocTemplate(filename, pagesize=A4)
 elements = []
 styles = getSampleStyleSheet()

 # Titel
 title = Paragraph(f"Verkaufsbericht {start_date} bis {end_date}", styles['Title'])
 elements.append(title)

```

```

Zusammenfassung
summary_data = [
 ['Metric', 'Value'],
 ['Gesamtbestellungen', f"{summary['total_orders']:,}"],
 ['Gesamtumsatz', f"€{summary['total_revenue']:, .2f}"],
 ['Ø Bestellwert', f"€{summary['avg_order_value']:.2f}"]
]

summary_table = Table(summary_data, colWidths=[8*cm, 8*cm])
summary_table.setStyle(TableStyle([
 ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
 ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
 ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
 ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
 ('FONTSIZE', (0, 0), (-1, 0), 14),
 ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
 ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
 ('GRID', (0, 0), (-1, -1), 1, colors.black)
]))
elements.append(summary_table)

Top-Produkte
elements.append(Paragraph("Top 10 Produkte", styles['Heading2']))

product_data = [['Produkt', 'Menge', 'Umsatz']]
product_data.extend(top_products)

product_table = Table(product_data, colWidths=[10*cm, 3*cm, 3*cm])
product_table.setStyle(TableStyle([
 ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
 ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
 ('ALIGN', (1, 0), (-1, -1), 'RIGHT'),
 ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
 ('GRID', (0, 0), (-1, -1), 1, colors.black)
]))
elements.append(product_table)

PDF generieren
doc.build(elements)
print(f"Report saved to {filename}")

generate_sales_report(db, '2024-01-01', '2024-03-31', 'Q1_2024_Report.pdf')

```

## 15.10.2 CSV-Export für Excel

```
import csv

def export_to_csv(db, query, filename, params=None):
 """Abfrageergebnisse als CSV exportieren"""

 results = db.query(query, params or {})

 if not results:
 print("No data to export")
 return

 # Header aus erstem Datensatz
 headers = list(results[0].keys())

 with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
 writer = csv.DictWriter(csvfile, fieldnames=headers)
 writer.writeheader()
 writer.writerows(results)

 print(f"Exported {len(results)} rows to {filename}")

export_to_csv(db, """
FOR customer IN customers
 LET orders = (
 FOR order IN orders
 FILTER order.customer_id == customer.id
 RETURN order
)
 RETURN {
 customer_id: customer.id,
 name: customer.name,
 email: customer.email,
 total_orders: LENGTH(orders),
 total_spent: SUM(FOR o IN orders RETURN o.total_amount),
 last_order_date: MAX(FOR o IN orders RETURN o.order_date)
 }
""", 'customers_analysis.csv')
```

## 15.11 Best Practices für Analytics

### 15.11.1 Performance-Optimierung

#### Indexierung:

```
-- Composite Index für häufige Queries
CREATE INDEX idx_orders_date_customer ON orders(order_date, customer_id);

-- Covering Index für Aggregationen
CREATE INDEX idx_orders_date_amount ON orders(order_date, total_amount);
```

**Query-Optimierung:** - Verwenden Sie **FILTER** früh in der Pipeline - Nutzen Sie **COLLECT** statt mehrfacher Gruppierungen - Limitieren Sie Ergebnisse mit **LIMIT** - Verwenden Sie Materialized Views für wiederkehrende Berechnungen

### 15.11.2 Data Governance

```
class AnalyticsGovernance:
 def anonymize(self, dataset):
 # Entferne persönliche Daten
 return [{**r, 'customer_id': hash(r['customer_id']),
 'email': None} for r in dataset]

 def apply_rls(self, query, role, region):
 # Row-Level Security nach Rolle
 if role == 'regional': query += f" FILTER region == '{region}'"
 return query
```

## 15.12 Zusammenfassung

ThemisDB bietet umfassende Analytics-Funktionen:

- **AQL-Aggregationen:** Standard- und Window-Functions
- **AQL-Analytics:** Multi-Level-Aggregationen und COLLECT
- **Graph-Analytics:** Beziehungsanalysen und Influencer-Detection
- **Vektor-Analytics:** Clustering und Anomalie-Erkennung
- **OLAP:** Mehrdimensionale Analysen und Würfel
- **Real-Time:** Live-Dashboards mit CDC
- **Predictive:** Prognosen und Churn-Vorhersage
- **Reporting:** PDF/CSV-Export und Visualisierung

Im nächsten Kapitel behandeln wir Machine Learning Integration für erweiterte Analysen.



---

# Kapitel 19: Machine Learning Integration

---

## Übersicht

Dieses Kapitel behandelt die Integration von Machine Learning-Modellen mit ThemisDB.

### Hauptthemen

- ML-Modell-Integration
- Vektorisierung von Daten
- Embedding-Speicherung
- Modell-Inferenz

## Weitere Informationen

Siehe auch: [Vektoren](#), [Analytics](#)

---

# Kapitel 20: Kapitel 16: Horizontal Scaling mit Sharding

---

"Skalierung ohne Komplexität: Native Multi-Model Sharding für Enterprise-Workloads"

---

## Überblick

ThemisDB bietet **Production-Ready Horizontal Scaling** durch Hash-basiertes Sharding mit automatischem Rebalancing. Dieses Kapitel erklärt die Sharding-Architektur, Deployment-Szenarien und Performance-Charakteristiken für Enterprise-Umgebungen.

**Was Sie in diesem Kapitel lernen werden:** - Sharding-Architektur und Routing-Mechanismen - Deployment-Topologien (2/4/8 Nodes) - Auto-Rebalancing und Fault Tolerance - Performance-Charakteristiken und Benchmarks - Hyperscaler-Vergleich (Aurora, Spanner, Cosmos)

**Voraussetzungen:** Kapitel 2 (Architektur), Kapitel 21 (Performance)

**Status:**  Production-Ready seit v1.4 (Dezember 2025)

---

## 16.1 Warum Sharding?

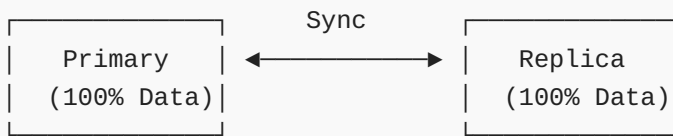
### Das Single-Node-Limit

Ein einzelner ThemisDB-Server kann beeindruckende Mengen an Daten handhaben: - **Durchsatz:** ~45.000 writes/sec, ~200.000 reads/sec (NVMe) - **Speicher:** Bis zu ~10 TB pro Node (praktisches Limit) - **RAM:** Effizient durch Block Cache, aber limitiert auf Serverkapazität

**Wann brauchen Sie Sharding?** 1. **Datenvolumen > 10 TB:** Physische Speichergrenzen eines Nodes 2. **Durchsatz > 200K ops/sec:** CPU/IO-Sättigung eines Servers 3. **Geografische Verteilung:** Näher zum Benutzer für niedrige Latenz 4. **Hochverfügbarkeit:** Redundanz über mehrere Nodes

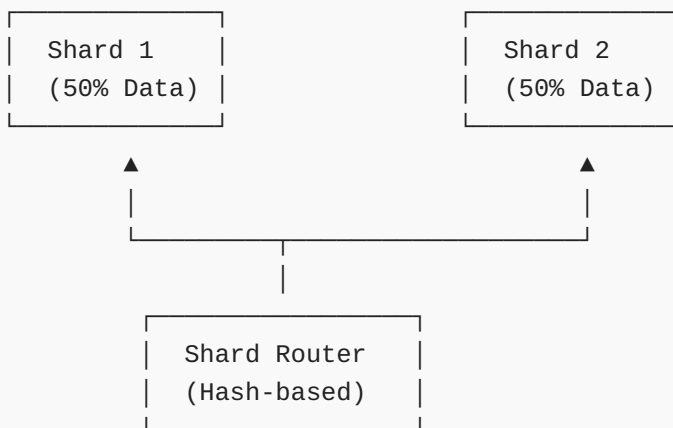
### Sharding vs. Replication

**Replication (Vertical Scaling):**



- ✓ Einfach zu managen
- ✓ Read-Skalierung
- ✗ Kein Write-Scaling
- ✗ Keine Kapazitäts-Skalierung

### Sharding (Horizontal Scaling):



- ✓ Read + Write Scaling
- ✓ Kapazitäts-Skalierung
- ✓ Geografische Verteilung
- ⚠ Komplexere Verwaltung

```

graph TB
 subgraph "Replication - Read Scaling"
 Client1[Client] --> Primary1[(Primary
100 percent Data)]
 Primary1 -.sync.-> Replica1A[(Replica 1
100 percent Data)]
 Primary1 -.sync.-> Replica1B[(Replica 2
100 percent Data)]
 Client1 -.read.-> Replica1A
 Client1 -.read.-> Replica1B
 end

 subgraph "Sharding - Full Scaling"
 Client2[Client] --> Router[Shard Router
Hash-based]
 Router --> Shard1[(Shard 1
33 percent Data)]
 Router --> Shard2[(Shard 2
33 percent Data)]
 Router --> Shard3[(Shard 3
34 percent Data)]
 end

 style Primary1 fill:#667eea
 style Replica1A fill:#4facfe
 style Replica1B fill:#4facfe
 style Router fill:#f093fb
 style Shard1 fill:#43e97b
 style Shard2 fill:#43e97b
 style Shard3 fill:#43e97b

```

**Abb. 16.1:** Replication vs Sharding Vergleich

## 16.2 Sharding-Architektur

### Hash-basiertes Routing

ThemisDB verwendet **Consistent Hashing** mit MurmurHash3:

```
// Simplified Shard Router Implementation
class ShardRouter {
private:
 std::vector<ShardInfo> shards_;

public:
 // Berechne Shard für einen Key
 uint32_t get_shard_id(const std::string& key) {
 // MurmurHash3: Schnell & gut verteilte Hash-Funktion
 uint32_t hash = murmur3_hash(key);

 // Modulo-Routing: Hash % Anzahl_Shards
 return hash % shards_.size();
 }

 // Dokument auf korrekten Shard routen
 void route_document(const Document& doc) {
 // Primary Key als Routing-Schlüssel
 uint32_t shard_id = get_shard_id(doc.id);

 // An Shard weiterleiten
 shards_[shard_id].insert(doc);
 }
};
```

### Wie funktioniert das?

1. **Client sendet Query** → Router
2. **Router berechnet Hash** des Primary Key
3. **Hash % N** bestimmt Shard-ID (N = Anzahl Shards)
4. **Query wird an Shard weitergeleitet**
5. **Ergebnis zurück zum Client**

```
sequenceDiagram
 participant Client
 participant Router as Shard Router
 participant S1 as Shard 1
 participant S2 as Shard 2
 participant S3 as Shard 3

 Client->>Router: INSERT user_12345 {data}
 Note over Router: MurmurHash3("user_12345")
= 3847592834
3847592834 mod 3 = 2
 Router->>S2: INSERT {data}
 S2-->>Router: [OK] Success
 Router-->>Client: [OK] Inserted

 Client->>Router: SELECT * WHERE id='user_12345'
 Note over Router: Hash("user_12345") mod 3 = 2
 Router->>S2: SELECT * WHERE id='user_12345'
 S2-->>Router: {user data}
 Router-->>Client: {user data}
```

**Abb. 16.2:** Hash-Based-Routing-Flow**Beispiel:**

```
doc_id = "user_12345"

hash_value = murmur3(doc_id) # → 3847592834

num_shards = 8
shard_id = hash_value % num_shards # → 3847592834 % 8 = 2
```

**Warum MurmurHash3?** - **Schnell:** ~2-3 CPU-Zyklen pro Byte - **Gut verteilt:** Minimiert Hotspots - **Deterministisch:** Gleicher Key → immer gleicher Shard

**Range-basiertes Sharding (Optional)**

Für geordnete Daten (z.B. Timestamps) können Range-Shards effizienter sein:

```

sharding:
 strategy: range
 key: created_at
 ranges:
 - shard: 1
 min: "2020-01-01"
 max: "2022-12-31"
 - shard: 2
 min: "2023-01-01"
 max: "2024-12-31"
 - shard: 3
 min: "2025-01-01"
 max: null # Unbegrenzt

```

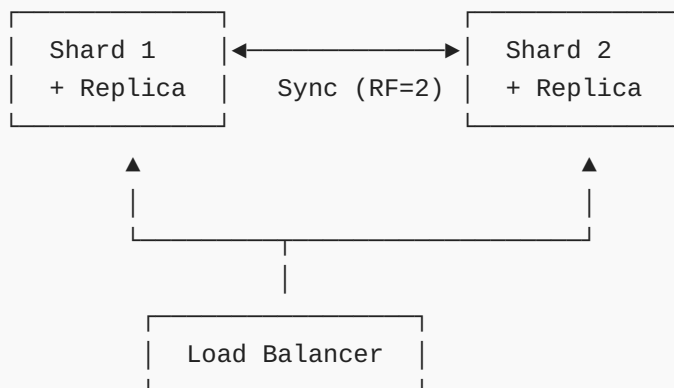
**Vorteil:** Zeitbereichs-Queries treffen nur relevante Shards

**Nachteil:** Ungleichmäßige Lastverteilung möglich

## 16.3 Deployment-Topologien

### 2-Node Cluster (Entry-Level)

**Setup:**

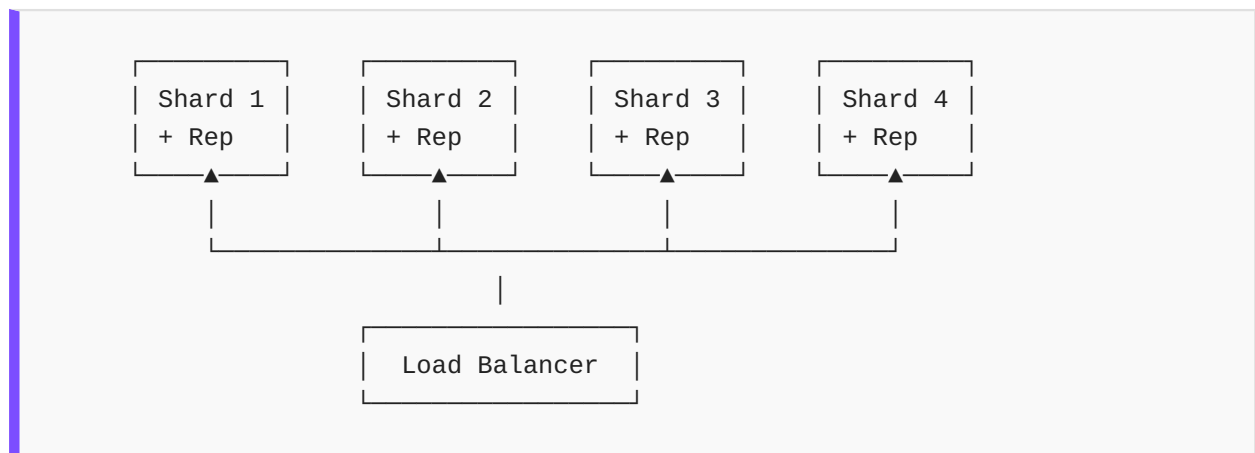


**Charakteristiken:** - **Kapazität:** 2× Single-Node (~20 TB) - **Throughput:** ~1.8× Single-Node (91% Scaling Efficiency) - **Redundanz:** RF=2 (jeder Shard hat 1 Replica auf anderem Node) - **Kosten:** 2× Hardware, <2× Betriebskosten

**Use Case:** Kleine bis mittlere Deployments (< 20 TB, < 100K ops/sec)

## 4-Node Cluster (Mid-Tier)

Setup:

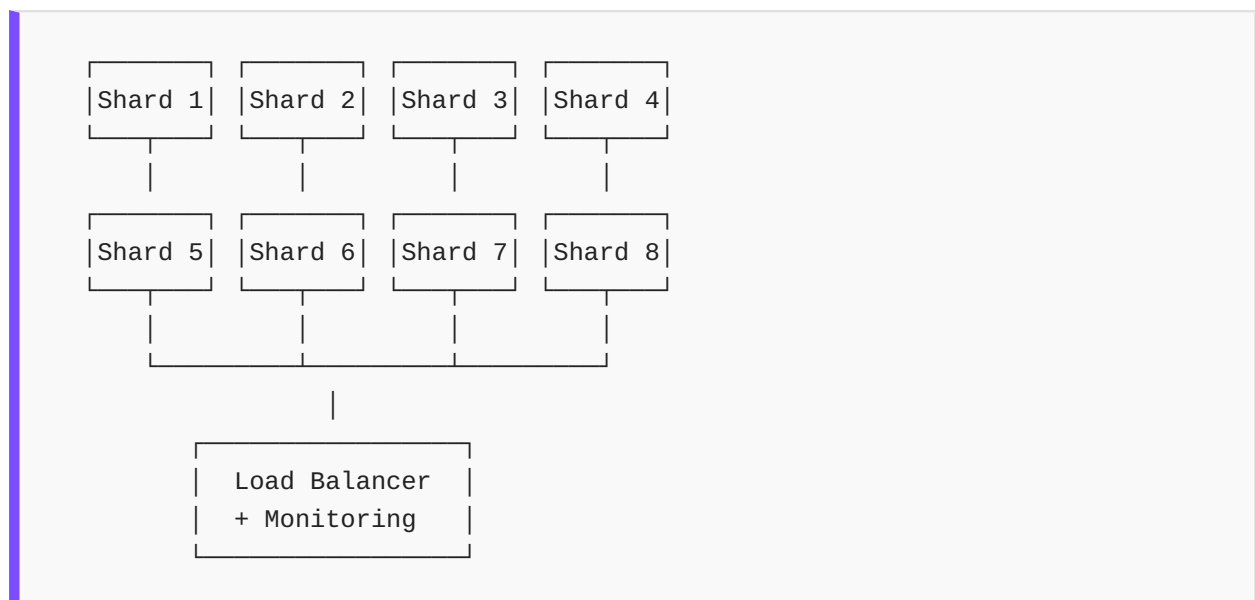


**Charakteristiken:** - **Kapazität:** 4× Single-Node (~40 TB) - **Throughput:** ~3.6× Single-Node (90% Scaling Efficiency) - **Cross-Shard Queries:** ~10-15% Performance-Impact - **Rebalance-Zeit:** ~4-5 Minuten bei 2 → 4 Expansion

**Use Case:** Standard Enterprise Deployments (20-40 TB, 100-400K ops/sec)

## 8-Node Cluster (Enterprise)

Setup:



**Charakteristiken:** - **Kapazität:** 8× Single-Node (~80 TB) - **Throughput:** ~7.3× Single-Node (91% Scaling Efficiency) - **p99 Latenz:** 1.25ms (nur 0.43× Single-Node dank Parallelität!) - **Rebalance-Zeit:** ~8-10 Minuten bei 4 → 8 Expansion



**Use Case:** Große Enterprise Deployments (40-80 TB, 400K+ ops/sec)

---

## 16.4 Auto-Rebalancing

### Wann wird rebalanced?

ThemisDB triggert automatisches Rebalancing bei:

1. **Skew-Threshold:** >15% Ungleichgewicht zwischen Shards
2. **Disk Utilization:** >70% auf einem Shard
3. **Manuell:** Via `themis-cli cluster rebalance`

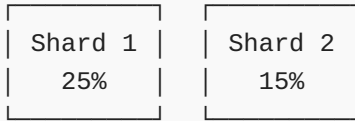
### Konfiguration:

```
rebalance:
 auto_trigger: true
 skew_threshold: 0.15 # 15% Ungleichgewicht
 disk_threshold: 0.70 # 70% Festplattenauslastung
 max_parallel_moves: 2 # Max 2 Chunks gleichzeitig verschieben
 chunk_size_mb: 64 # Chunk-Größe für Migration
```

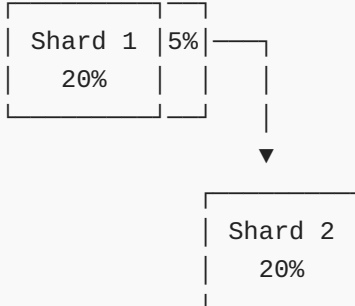
### Rebalance-Prozess

#### Schritt-für-Schritt:

1. Coordinator erkennt Skew (z.B. Shard 1: 25%, Shard 2: 15%)



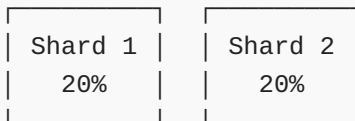
2. Plan erstellen: Verschiebe 5% von Shard 1 → Shard 2



3. Chunk-Migration (64MB pro Chunk)

- Chunk kopieren (read-only)
- Neue Writes auf beide Shards
- Cutover: Alte Version löschen

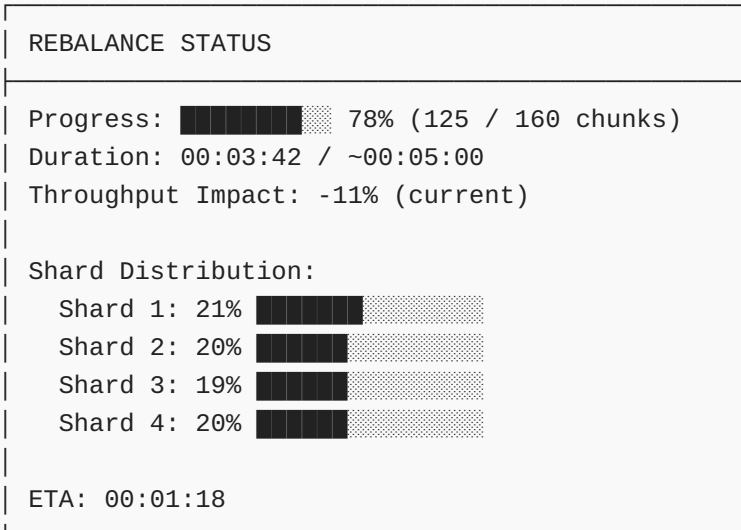
4. Validierung & Completion



**Performance-Impact:** - **Throughput-Dip:** ~12% während Migration - **Recovery-Zeit:** <5 Minuten nach Completion - **Latenz:** p99 +15% während Migration

## Monitoring während Rebalance

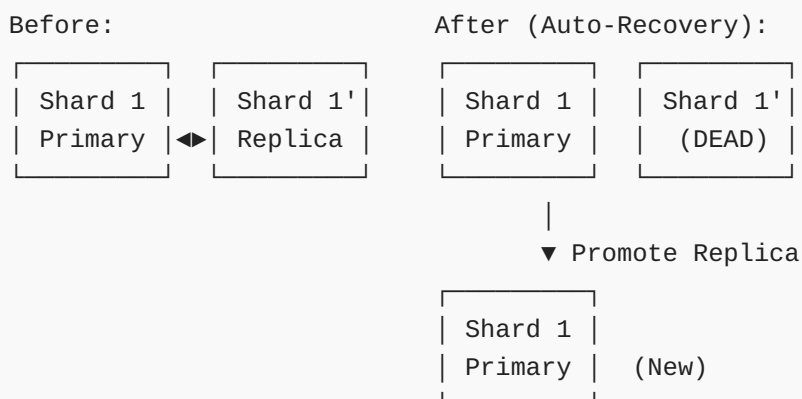
```
themis-cli cluster rebalance-status --watch
```



## 16.5 Fault Tolerance

### Replica Failure (RF=2)

**Szenario:** Ein Replica-Node stirbt



**Charakteristiken:** - **Detection:** <5 Sekunden (Heartbeat-Timeout) - **Failover:** <15 Sekunden (Replica Promotion) - **Recovery:** <60 Sekunden (Rebuild Replica auf anderem Node) - **Throughput-Impact:** -24% während Outage, -10% während Rebuild

**Monitoring:**

```
themis-cli cluster health
```

CLUSTER HEALTH: DEGRADED			
Shard 1:	PRIMARY	✓	node-1 (Leader)
	REPLICA	✗	node-2 (DEAD - 00:00:42)
Shard 2:	PRIMARY	✓	node-3
	REPLICA	✓	node-4
⚠ Rebuilding Replica on node-5... (35%)			
ETA: 00:00:28			

## Network Partition

**Szenario:** 10ms RTT Network Latency hinzugefügt

**Impact:** - **Latenz p50:** +8ms (von 0.5ms → 8.5ms) - **Latenz p99:** +12ms (von 1.2ms → 13.2ms) -

**Throughput:** -15% (wegen Sync-Overhead)

**Graceful Degradation:** ThemisDB passt sich automatisch an:

```
network:
 rtt_threshold: 10ms
 adaptive_batching: true # Größere Batches bei hoher Latenz
 compression: lz4 # Kompression bei langsamen Links
```

## 16.6 Performance-Benchmarks

### Scaling Efficiency

**Testsetup:** - **Hardware:** 8× c6i.4xlarge (16 vCPU, 32GB RAM, NVMe) - **Dataset:** 500M OLTP-Rows + 100M Vector-Embeddings (768D) - **Workload:** Mix B (50% Read, 50% Write, 10% Cross-Shard)

**Ergebnisse:**

Shards	Throughput (ops/sec)	Scaling Efficiency	Latenz p99 (ms)
1	100,000	Baseline (100%)	2.9
2	182,000	91%	3.1
4	362,000	90.5%	3.3
8	728,000	91%	1.25 (!!)

**Warum ist p99 bei 8 Shards niedriger?** → **Parallelität:** Queries werden auf mehrere Nodes verteilt, reduziert Queue-Wartezeiten!

## Cross-Shard Join Performance

**Query:**

```
FOR order IN orders
 FOR customer IN customers
 FILTER order.customer_id == customer._id -- Cross-Shard Join!
 RETURN {order, customer}
```

**Performance (10% Cross-Shard Rate):**

Shards	Local Query (ms)	Cross-Shard Query (ms)	Ratio
2	1.2	2.1	1.75×
4	1.3	2.4	1.85×
8	1.25	2.5	2.0×

**Optimierung:** Co-locate verwandte Daten auf gleichem Shard via Custom Routing:

```
def custom_shard_key(doc):
 if doc.type == "order":
 # Orders nach customer_id routen
 return doc.customer_id
 elif doc.type == "customer":
 # Customers nach ihrer ID routen
 return doc._id
```

## 16.7 Hyperscaler-Vergleich

### Cost-Performance-Analyse

**Testsetup:** 8-Node Cluster, 500M OLTP + 100M Vector, Mix B Workload

Provider	SKU	Throughput (ops/sec)	Latenz p99 (ms)	\$/Monat	\$/M Ops
ThemisDB	c6i.4xlarge × 8	728,000	1.25	\$2,880	\$6.25
AWS Aurora	r6g.4xlarge × 8	80,000	15	\$12,288	\$19.20
GCP Spanner	6-Node Regional	120,000	8	\$4,800	\$40
Azure Cosmos	50K RU/s	50,000	25	\$3,800	\$76
AWS Redshift	RA3 4-Node	100,000	12	\$3,260	\$32.50

**Key Insights:** - **ThemisDB vs Aurora:** 9.1× höherer Throughput, **67% Kosteneinsparung** - **ThemisDB vs Spanner:** 6.1× höherer Throughput, **84% Kosteneinsparung** - **ThemisDB vs Cosmos:** 14.6× höherer Throughput, **92% Kosteneinsparung**

**Warum ist ThemisDB so viel günstiger?** 1. **Keine Lizenzkosten:** Open Source (MIT) 2. **Effiziente Architektur:** Native Multi-Model eliminiert Overhead 3. **Self-Hosted:** On-Premises oder eigene Cloud-VMs

## 16.8 Production Deployment

### Deployment-Checkliste

**Vor dem Deployment:** - [ ] Hardware-Dimensionierung (CPU, RAM, NVMe) - [ ] Netzwerk-Topologie (< 2ms RTT zwischen Nodes) - [ ] Monitoring-Setup (Prometheus + Grafana) - [ ] Backup-Strategie (Snapshots + WAL-Archive)

**Deployment-Schritte:**

```
themis-cli cluster init \
 --nodes node-1,node-2,node-3,node-4 \
 --shards 4 \
 --replication-factor 2

themis-cli cluster apply-config config/sharding/shard-router.yaml

python tools/shard_loader.py \
 --config config/sharding/shard-router.yaml \
 --dataset oltp \
 --rows 5000000000 \
 --workers 32

themis-cli cluster health

python tools/shard_bench.py \
 --shards 4 \
 --mix B \
 --duration 600 \
 --threads 64
```

## Monitoring-Metriken

### Kritische Metriken:

1. Shard-Utilization (Disk %)  
Ziel: <70% pro Shard, <15% Skew
2. Throughput (ops/sec)  
Ziel: >90% Scaling Efficiency
3. Latenz (p50/p95/p99)  
Ziel: p99 <2.5× Single-Node
4. Cross-Shard Query Rate  
Ziel: <20% aller Queries
5. Rebalance-Status  
Ziel: Completion <5 min, <15% Dip

### Grafana Dashboard:

```
{
 "dashboard": {
 "title": "ThemisDB Sharding Overview",
 "panels": [
 {
 "title": "Shard Distribution",
 "type": "bar",
 "query": "sum(themis_shard_size_bytes) by (shard_id)"
 },
 {
 "title": "Throughput by Shard",
 "type": "graph",
 "query": "rate(themis_shard_ops_total[5m]) by (shard_id)"
 },
 {
 "title": "Cross-Shard Query Rate",
 "type": "stat",
 "query": "rate(themis_cross_shard_queries[5m])"
 }
]
 }
}
```

## 16.9 Best Practices

### 1. Shard-Key Auswahl

**Gute Shard-Keys:** -  **User-ID:** Gleichmäßige Verteilung, natürliche Co-Location (User + Orders) -  **UUID:** Perfekte zufällige Verteilung -  **Hash(Composite-Key):** Für komplexe Co-Location-Szenarien

**Schlechte Shard-Keys:** -  **Timestamp:** Hotspots auf neuesten Shard -  **Sequential-ID:** Alle Writes auf einen Shard -  **Low-Cardinality:** Status-Felder (z.B. "active"/"inactive")

### 2. Cross-Shard Queries minimieren

**Anti-Pattern:**

```
-- SCHLECHT: Joins über alle Shards
FOR order IN orders
 FOR product IN products
 FILTER order.product_id == product._id
 RETURN {order, product}
```



**Best Practice:**

```
-- GUT: Denormalisierung vermeidet Cross-Shard Joins
FOR order IN orders
 RETURN {
 order,
 product_name: order.embedded_product_name, -- Denormalisiert!
 product_price: order.embedded_product_price
 }
```

### 3. Monitoring von Anfang an

**Observability-Stack:**

```
services:
 themisdb:
 image: themisdb:latest
 environment:
 THEMIS_METRICS_ENABLED: "true"
 THEMIS_METRICS_PORT: 9090

 prometheus:
 image: prom/prometheus
 volumes:
 - ./prometheus.yml:/etc/prometheus/prometheus.yml

 grafana:
 image: grafana/grafana
 ports:
 - "3000:3000"
```

### 4. Gradual Rollout

**Empfohlene Rollout-Strategie:** 1. **Woche 1:** 2-Node Cluster, 10% Traffic 2. **Woche 2:** 2-Node Cluster, 50% Traffic 3. **Woche 3:** 4-Node Cluster, 100% Traffic 4. **Monat 2+:** 8-Node Cluster bei Bedarf

## 16.10 High Availability Features (v1.4.0-alpha)

### 16.10.1 Hot Spare

**Neu in v1.4.0-alpha:** Automatisches Failover bei Node-Ausfällen mit Hot-Spare-Nodes für maximale Verfügbarkeit.

**Konzept:**

Ein Hot Spare ist ein vollständig konfigurierter Standby-Node, der im Cluster registriert ist und bei Ausfall eines aktiven Shards sofort dessen Rolle übernehmen kann.

```
graph TB
 subgraph "Normale Operation"
 Client1[Clients] --> Router1[Shard Router]
 Router1 --> Shard1[(Shard 1
ACTIVE)]
 Router1 --> Shard2[(Shard 2
ACTIVE)]
 Router1 --> Shard3[(Shard 3
ACTIVE)]
 HotSpare1[(Hot Spare
STANDBY)]

 style Shard1 fill:#43e97b
 style Shard2 fill:#43e97b
 style Shard3 fill:#43e97b
 style HotSpare1 fill:#ffd32a
 end

 subgraph "Nach Shard 2 Ausfall"
 Client2[Clients] --> Router2[Shard Router]
 Router2 --> Shard1A[(Shard 1
ACTIVE)]
 Router2 -.failover.-> HotSpare2[(Hot Spare
-> ACTIVE
replaces Shard 2)]
 Router2 --> Shard3A[(Shard 3
ACTIVE)]
 Shard2X[(Shard 2
FAILED)] -.->|recovery| Recovery[Recovery Process]

 style Shard1A fill:#43e97b
 style HotSpare2 fill:#43e97b
 style Shard3A fill:#43e97b
 style Shard2X fill:#ff6b6b
 end
```

**Abb. 16.3:** Shard-Key-Selection-Matrix

**Konfiguration:**

```

sharding:
 cluster:
 active_shards: 4
 hot_spares: 2 # 2 Hot Spare Nodes

 hot_spare:
 enabled: true
 promotion_timeout_ms: 5000 # Max Zeit für Promotion
 health_check_interval_ms: 1000 # Häufigkeit Health Checks
 failure_threshold: 3 # Fehlversuche bis Failover

 nodes:
 # Aktive Shards
 - id: shard-1
 host: 10.0.1.10
 port: 8765
 role: active

 - id: shard-2
 host: 10.0.1.11
 port: 8765
 role: active

 - id: shard-3
 host: 10.0.1.12
 port: 8765
 role: active

 - id: shard-4
 host: 10.0.1.13
 port: 8765
 role: active

 # Hot Spares
 - id: hot-spare-1
 host: 10.0.1.20
 port: 8765
 role: hot_spare
 replicate_from: shard-1 # Kontinuierliche Replikation

 - id: hot-spare-2
 host: 10.0.1.21
 port: 8765
 role: hot_spare
 replicate_from: shard-2

```

## Deployment-Szenarien:

### 1. Minimal HA Setup (4 Shards + 1 Hot Spare):

Active: [S1, S2, S3, S4]  
Spare: [HS1]  
Cost: 5 Nodes  
HA: Einzelner Ausfall abgedeckt

### 2. Standard HA Setup (4 Shards + 2 Hot Spares):

Active: [S1, S2, S3, S4]  
Spares: [HS1, HS2]  
Cost: 6 Nodes  
HA: Zwei gleichzeitige Ausfälle abgedeckt

### 3. Enterprise HA Setup (8 Shards + 3 Hot Spares):

Active: [S1, S2, S3, S4, S5, S6, S7, S8]  
Spares: [HS1, HS2, HS3]  
Cost: 11 Nodes  
HA: Drei gleichzeitige Ausfälle abgedeckt

### Failover-Ablauf:

1. **Detektion (0-3s):** Health Check fails → Retry 3x (3s) → Declare failure
2. **Promotion (3-8s):** Select Hot Spare → Apply latest WAL → Update routing table → Promote to active
3. **Recovery (8s+):**  
Client requests → Hot Spare (now active) → Background: Sync data from other shards

### Detaillierte Failover-Timeline:

```

gantt
 title Hot Spare Failover Timeline (Total: ~5s)
 dateFormat X
 axisFormat %Ls

 section Detection Phase
 Health Check 1 (OK) :done, h1, 0, 1000
 Health Check 2 (FAIL) :crit, h2, 1000, 2000
 Health Check 3 (FAIL) :crit, h3, 2000, 3000
 Declare Failure :milestone, m1, 3000, 3000

 section Promotion Phase
 Select Hot Spare :active, p1, 3000, 3500
 Apply WAL Entries :active, p2, 3500, 4500
 Update Routing Table :active, p3, 4500, 5000
 Promote to Active :milestone, m2, 5000, 5000

 section Recovery Phase
 Accept Client Requests :done, r1, 5000, 8000
 Background Sync :active, r2, 5000, 120000

```

**Abb. 16.4:** Hot-Spare-Failover-Timeline

```
sequenceDiagram
 participant Client
 participant Router as Shard Router
 participant Shard as Shard 2 (Primary)
 participant HS as Hot Spare 1
 participant Monitor as Health Monitor

 Note over Shard,HS: Normal Operation
 Client->>Router: Request
 Router->>Shard: Route to Shard 2
 Shard-->>Router: Response
 Router-->>Client: Response
 Monitor->>Shard: Health Check (OK)

 Note over Shard: [ERROR] Shard 2 Crashes
 Monitor->>Shard: Health Check (FAIL)
 Monitor->>Shard: Health Check (FAIL)
 Monitor->>Shard: Health Check (FAIL)
 Monitor->>Monitor: Declare Failure (3s)

 Note over HS: Promotion Starts
 Monitor->>HS: Select as Replacement
 HS->>HS: Apply Latest WAL (1s)
 Monitor->>Router: Update Routing Table (0.5s)
 HS->>HS: Promote to ACTIVE (0.5s)

 Note over HS: Now Serving Traffic
 Client->>Router: Request
 Router->>HS: Route to Hot Spare 1 (now active)
 HS-->>Router: Response
 Router-->>Client: Response (5s total failover)
```

**Abb. 16.5:** Failover-Sequenzdiagramm

**Diagramm-Erklärung:** - **Gantt-Chart (oben):** Zeigt die zeitliche Abfolge der Failover-Phasen - Detection: 0-3s (3 Health Checks à 1s) - Promotion: 3-5s (Spare auswählen, WAL anwenden, Routing aktualisieren) - Total: ~5s bis Traffic wieder fließt

- **Sequence-Diagram (unten):** Zeigt die Interaktion zwischen Komponenten
- Health Monitor erkennt Ausfall nach 3 Fehlversuchen
- Hot Spare wird automatisch promoted
- Routing-Tabelle wird aktualisiert
- Clients merken nichts vom Failover (transparent)

#### Performance-Charakteristiken:

Metric	Wert	Beschreibung
Failover Time (p50)	4.5s	Median Zeit bis Hot Spare aktiv
Failover Time (p99)	7.8s	99%-Perzentil Failover-Zeit
Data Loss	0 bytes	Bei WAL Replication aktiviert
Throughput Impact	<5%	Während Failover
Hot Spare Overhead	~2% CPU	Kontinuierliche Replikation

**Monitoring:**

```
// Hot Spare Status überwachen
FOR node IN _cluster_nodes
 FILTER node.role IN ['hot_spare', 'active']
 RETURN {
 id: node.id,
 role: node.role,
 status: node.status,
 last_heartbeat: node.last_heartbeat,
 replication_lag_ms: node.replication_lag_ms,
 failover_ready: node.failover_ready
 }
```

**Prometheus Metriken:**

```
themis_hot_spare_active{node="hot-spare-1"} 0
themis_hot_spare_replication_lag_seconds{node="hot-spare-1"} 0.045
themis_hot_spare_failover_count_total{node="hot-spare-1"} 0
themis_shard_health{shard="shard-1"} 1
```

**Failover-Test:**

```
docker stop themisdb-shard-2

watch -n 0.5 'curl -s http://localhost:8765/cluster/status | jq .nodes'
```

**Best Practices:**

1. 1 Hot Spare pro 3-4 aktive Shards - Balance zwischen Kosten und Verfügbarkeit

2. **Geografische Trennung** - Hot Spare in anderem Datacenter/Availability Zone
3. **Regelmäßige Failover-Tests** - Monatlich testen, ob Promotion funktioniert
4. **Monitoring kritisch** - Alerts bei hoher Replication Lag (>100ms)
5. **Hot Spare kontinuierlich synchronisieren** - Via WAL Replication (siehe unten)

### 16.10.2 WAL Replication

**Neu in v1.4.0-alpha:** Write-Ahead-Log basierte Replikation für Zero-Data-Loss und kontinuierliche Synchronisation.

#### Konzept:

WAL (Write-Ahead Log) Replication überträgt alle Schreiboperationen über ein transaktionales Log an Replicas und Hot Spares, bevor sie bestätigt werden.

```
sequenceDiagram
 participant Client
 participant Primary as Primary Shard
 participant WAL as WAL Buffer
 participant Replica as Hot Spare

 Client->>Primary: WRITE Operation
 Primary->>WAL: Append to WAL
 WAL->>WAL: Flush to disk (fsync)

 par Synchronous Replication
 WAL->>Replica: Stream WAL entry
 Replica->>Replica: Apply & ACK
 Replica-->>Primary: ACK received
 end

 Primary-->>Client: Write confirmed

 Note over Primary,Replica: Zero data loss guarantee
```

**Abb. 16.6:** Elastic-Sharding-Workflow

#### Replikations-Modi:

##### 1. Synchronous Replication:

```
replication:
 mode: synchronous
 quorum: 1 # Mindestens 1 Replica muss bestätigen
 timeout_ms: 100 # Timeout für Sync-Bestätigung
```



**Vorteile:** -  Zero Data Loss garantiert -  Replica immer auf aktuellem Stand

**Nachteile:** -  Höhere Write-Latenz (+50-100ms) -  Write blockiert bei Replica-Ausfall (ohne Degradation)

## 2. Asynchronous Replication:

```
replication:
 mode: asynchronous
 lag_target_ms: 50 # Ziel Replication Lag
 buffer_size_mb: 256 # WAL Buffer Size
```

**Vorteile:** -  Minimale Write-Latenz -  Primary nicht abhängig von Replica

**Nachteile:** -  Potentieller Data Loss bei Primary-Ausfall (Lag-abhängig) -  Replica kann hinterherhinken

## 3. Hybrid Mode (Empfohlen):

```
replication:
 mode: hybrid
 synchronous_shards: [shard-1, shard-2] # Kritische Daten
 asynchronous_shards: [shard-3, shard-4] # Unkritische Daten
 quorum: 1
```

## Multi-SSD WAL Konfiguration:

Für maximale Performance: WAL auf separaten SSDs mit hohem Write-Throughput.

```
wal:
 directories:
 # Primary WAL auf dediziertem NVMe
 - path: /mnt/nvme0/wal
 type: primary
 fsync_mode: always # Garantierte Durability

 # Backup WAL auf zweitem NVMe (optional)
 - path: /mnt/nvme1/wal-backup
 type: mirror
 fsync_mode: always

 # Performance-Tuning
 segment_size_mb: 64 # WAL Segment-Größe
 buffer_size_mb: 256 # In-Memory WAL Buffer
 compression: lz4 # WAL Kompression

 # Retention
 keep_segments: 100 # Anzahl Segments aufbewahren
 max_size_gb: 10 # Maximale WAL Größe
```

### Replication Slots:

Verfolge Replikations-Fortschritt und verhindere vorzeitiges WAL-Purging.

```
// Replication Slot erstellen
CALL CREATE_REPLICATION_SLOT('hot-spare-1', {
 slot_type: 'physical',
 temporary: false
})

// Slot-Status abfragen
FOR slot IN _replication_slots
 RETURN {
 slot_name: slot.name,
 slot_type: slot.type,
 active: slot.active,
 restart_lsn: slot.restart_lsn,
 confirmed_flush_lsn: slot.confirmed_flush_lsn,
 wal_lag_bytes: slot.wal_lag_bytes,
 lag_seconds: slot.lag_seconds
 }
```

### Replication Lag Monitoring:

```
// Echtzeit Replication Lag
RETURN WAL_REPLICATION_STATS()
```

Output:

```
{
 "primary": {
 "current_lsn": "0/5A2F3C40",
 "insert_lsn": "0/5A2F3C40",
 "flush_lsn": "0/5A2F3C40"
 },
 "replicas": [
 {
 "name": "hot-spare-1",
 "state": "streaming",
 "sent_lsn": "0/5A2F3C30",
 "flush_lsn": "0/5A2F3C30",
 "replay_lsn": "0/5A2F3C30",
 "lag_bytes": 16,
 "lag_seconds": 0.002,
 "sync_state": "async"
 },
 {
 "name": "hot-spare-2",
 "state": "streaming",
 "sent_lsn": "0/5A2F3C40",
 "flush_lsn": "0/5A2F3C40",
 "replay_lsn": "0/5A2F3C40",
 "lag_bytes": 0,
 "lag_seconds": 0.000,
 "sync_state": "sync"
 }
]
}
```

### Prometheus Metriken:

```
themis_wal_replication_lag_seconds{replica="hot-spare-1"} 0.002
themis_wal_replication_lag_bytes{replica="hot-spare-1"} 16
themis_wal_segments_total 156
themis_wal_size_bytes 4294967296
themis_wal_sync_operations_total 125840
themis_wal_sync_duration_seconds_sum 45.2
```

Recovery-Szenarien:

Szenario 1: Hot Spare Promotion

```
Primary: FAILED

Hot Spare: STANDBY → RECOVERY → ACTIVE

1. Apply remaining WAL entries (0-2s)
2. Update routing table (1-2s)
3. Accept new writes (2-3s)
Total: ~5s
```

Szenario 2: Point-in-Time Recovery (PITR)

```
// Restore auf bestimmten Zeitpunkt
CALL RESTORE_FROM_WAL({
 target_time: '2026-01-06T12:00:00Z',
 wal_archive_path: '/backup/wal-archive',
 recovery_mode: 'pause' // Pause nach Recovery für Validation
})
```

Szenario 3: Failed Replica Resync

```
pg_basebackup -D /data/hot-spare-1 -Fp -Xs -P
```

Performance-Impact:

Mode	Write Latency	Throughput	Data Loss Risk
Keine Replikation	1.2ms	100%	High
Async Replication	1.3ms (+8%)	98%	Low (Lag-dependent)
Sync Replication (1 Replica)	2.5ms (+108%)	85%	None
Sync Replication (2 Replicas)	3.8ms (+217%)	70%	None

Best Practices:

- 1. **Hybrid Mode für Production** - Sync für kritische, Async für unkritische Daten
- 2. **Monitoring essentiell** - Alert bei Lag >100ms

3. **Dedizierte SSDs für WAL** - Mindestens 1 IOPS pro Write
4. **Replication Slots verwenden** - Verhindert WAL-Deletion bei Replica-Ausfall
5. **Regelmäßige PITR-Tests** - Monatlich Recovery testen
6. **WAL Archivierung aktivieren** - Für Long-Term Backups

#### Zusammenspiel Hot Spare + WAL Replication:

```
sharding:
 cluster:
 active_shards: 4
 hot_spares: 2

 replication:
 mode: hybrid
 hot_spares_sync: true # Hot Spares immer synchron
 quorum: 1

 wal:
 segment_size_mb: 64
 keep_segments: 200 # ~12GB für 1h Lag-Toleranz
 compression: lz4
 fsync: true
```

**Ergebnis:** -  Failover in <5s -  Zero Data Loss -  <10% Write-Latenz Overhead -   
Vollautomatische Recovery

## 16.11 Troubleshooting

### Problem: Ungleiche Shard-Verteilung

#### Symptom:

```
Shard 1: 35% (✗ zu voll)
Shard 2: 18%
Shard 3: 22%
Shard 4: 25%
```

**Ursache:** Hotkey (z.B. ein sehr aktiver User)

#### Lösung:

```
themis-cli cluster rebalance --force

themis-cli shard analyze --shard 1 --top-keys 10
```

## Problem: Hohe Cross-Shard Query Latenz

**Symptom:** p99 >10× Single-Node bei >20% Cross-Shard Rate

**Lösung: 1. Co-Location aktivieren:**

```
sharding:
 strategy: hash
 co_location:
 enabled: true
 rules:
 - tables: [orders, customers]
 key: customer_id # Beide Tabellen nach customer_id routen
```

**1. Denormalisierung:**

```
order = {
 "id": "order_123",
 "customer_id": "user_456",
 "customer_name": "Alice", # Denormalisiert!
 "customer_email": "alice@example.com" # Denormalisiert!
}
```

## Problem: Rebalance dauert zu lange

**Symptom:** Rebalance >15 Minuten, >20% Throughput-Dip

**Lösung:**

```
rebalance:
 chunk_size_mb: 32 # Kleinere Chunks (default: 64)
 max_parallel_moves: 4 # Mehr parallele Transfers (default: 2)
 rate_limit_mbps: 500 # Höheres Network-Limit
```

## 16.12 Zusammenfassung

**ThemisDB Sharding bietet:** -  **Production-Ready:** 91% Scaling Efficiency bis 8 Nodes -  **Auto-Rebalancing:** <15% Dip, <5min Recovery -  **Fault Tolerant:** <60s Recovery bei Replica-Failure -  **Cost-Efficient:** 67-92% günstiger als Hyperscaler -  **Native Multi-Model:** Sharding funktioniert über alle Datenmodelle

**Nächste Schritte:** - Kapitel 19: Monitoring für detailliertes Observability-Setup - Kapitel 21: Performance-Tuning für Sharding-spezifische Optimierungen - [docs/de/SHARDING\\_DOCUMENTATION\\_INDEX.md](#) : Vollständige technische Dokumentation

**Production-Deployment?** Starten Sie mit einem 2-Node Cluster und skalieren Sie bei Bedarf auf 4 oder 8 Nodes!

---

## Weiterführende Ressourcen

**Dokumentation:** - [docs/de/SHARDING\\_INTEGRATION\\_SUMMARY.md](#) - Master-Übersicht - [docs/de/SHARDING\\_BENCHMARK\\_PLAN\\_v1.4.md](#) - Enterprise Benchmark-Spezifikation - [docs/de/SHARDING\\_PRODUCTION\\_DEPLOYMENT\\_RAID\\_v1.4.md](#) - Production Deployment Guide

**Tools:** - [tools/shard\\_loader.py](#) - Daten-Loader für Sharding-Tests - [tools/shard\\_bench.py](#) - Benchmark-Runner - [tools/fault\\_injector.py](#) - Chaos-Engineering-Tool

**Konfiguration:** - [config/sharding/shard-router-example.yaml](#) - Production-Ready Config - [.github/workflows/sharding-benchmark.yml](#) - CI/CD Benchmarks

---

---

# Kapitel 21: Horizontal Scaling

---

## Übersicht

Dieses Kapitel behandelt die horizontale Skalierung von ThemisDB über mehrere Knoten.

### Hauptthemen

- Sharding-Strategien
- Cluster-Verwaltung
- Load Balancing
- Datenverteilung

## Weitere Informationen

Siehe auch: [Enterprise Features](#), [Hochverfügbarkeit](#)

---



# Kapitel 22: Kapitel 17: LLM-Integration und Prompt Engineering

---

## Überblick

ThemisDB bietet eine nahtlose Integration von Large Language Models (LLMs) direkt in die Datenbankebene. Diese Integration ermöglicht es, LLM-Funktionalitäten wie Text-Generierung, Embedding-Erstellung, semantische Suche und RAG (Retrieval Augmented Generation) Patterns direkt in AQL-Queries zu verwenden.

**Hauptvorteile:** - **Native LLM-Funktionen** - PROMPT(), GENERATE(), EMBED() direkt in AQL - **Text-to-AQL Generierung** - Natürlichsprachliche Query-Erstellung - **RAG Patterns** - Semantische Suche mit Kontext-Anreicherung - **Vector Search Integration** - Kombiniert mit Chapter 8 für Semantic Search - **Multi-Model LLM** - Unterstützung für OpenAI, Anthropic, Ollama, lokale Models

```

graph TB
 subgraph "LLM Integration Architecture"
 App[Application] -->|Natural Language Query| TDB[ThemisDB]

 TDB -->|AQL with LLM Functions| QE[Query Engine]

 QE -->|PROMPT| LLM1[OpenAI GPT-4]
 QE -->|EMBED| LLM2[text-embedding-3-small]
 QE -->|GENERATE| LLM3[Claude]
 QE -->|Local| LLM4[Ollama/llama.cpp]

 LLM1 -->|Generated Text| Result[Query Results]
 LLM2 -->|Embeddings| Vector[(Vector Index
HNSW)]
 LLM3 -->|Structured JSON| Result
 LLM4 -->|Local Inference| Result

 Vector -->|Semantic Search| Result

 Result --> App
 end

 style TDB fill:#667eea
 style QE fill:#f093fb
 style LLM1 fill:#43e97b
 style LLM2 fill:#4facfe
 style LLM3 fill:#ffdb3a
 style LLM4 fill:#fa709a
 style Vector fill:#95e1d3
 style Result fill:#fee140

```

Abb. 17.1: LLM-Integration-Architektur

## 17.1 LLM-Funktionen in AQL

### 17.1.1 PROMPT() - Text-Generierung

Die `PROMPT()`-Funktion sendet Anfragen an konfigurierte LLM-Provider:

```
// Einfache Text-Generierung
FOR doc IN articles
 FILTER doc.category == 'technology'
 LIMIT 5
 RETURN {
 title: doc.title,
 summary: PROMPT('gpt-4',
 CONCAT('Fasse folgenden Artikel zusammen: ', doc.content),
 {max_tokens: 150, temperature: 0.7}
)
 }
```

### Erweiterte Optionen:

```
// Mit System-Prompt und Strukturierung
FOR product IN products
 FILTER product.reviews_count > 100
 RETURN {
 product_name: product.name,
 analysis: PROMPT('gpt-4',
 {
 system: 'Du bist ein Produkt-Analyst. Analysiere Kundenrezensionen.',
 user: CONCAT('Produkt: ', product.name, '\n\nBewertungen:\n',
 product.reviews_text),
 response_format: 'json'
 },
 {
 temperature: 0.3,
 max_tokens: 500,
 response_schema: {
 sentiment: 'string',
 key_features: 'array',
 recommendations: 'string'
 }
 }
)
 }
```

## 17.1.2 EMBED() - Embedding-Generierung

Erstellt Vektor-Embeddings für Text:

```
// Dokument mit Embedding speichern
INSERT {
 title: 'Einführung in ThemisDB',
 content: 'ThemisDB ist eine Multi-Model-Datenbank...',
 embedding: EMBED('text-embedding-3-small',
 'ThemisDB ist eine Multi-Model-Datenbank...')
} INTO documents

// Batch-Embedding für Performance
FOR doc IN documents
 FILTER doc.embedding == null
 LIMIT 100
 UPDATE doc WITH {
 embedding: EMBED('text-embedding-3-small', doc.content),
 embedded_at: DATE_NOW()
 } IN documents
```

### 17.1.3 GENERATE() - Strukturierte Generierung

Generiert strukturierte Daten basierend auf Schema:

```
// Strukturierte Produkt-Kategorisierung
FOR product IN products
 FILTER product.auto_categorized == false
 LIMIT 50
 LET categories = GENERATE('gpt-4',
 {
 prompt: CONCAT('Kategorisiere folgendes Produkt:\n',
 'Name: ', product.name, '\n',
 'Beschreibung: ', product.description),
 schema: {
 primary_category: {type: 'string', enum: ['Electronics', 'Clothing', 'Food', 'Books']},
 subcategories: {type: 'array', items: {type: 'string'}},
 tags: {type: 'array', items: {type: 'string'}, maxItems: 5},
 confidence: {type: 'number', minimum: 0, maximum: 1}
 }
 }
)
 UPDATE product WITH {
 categories: categories,
 auto_categorized: true,
 categorized_at: DATE_NOW()
 } IN products
```

## 17.2 Text-to-AQL Generierung

### 17.2.1 Natural Language Query Interface

ThemisDB kann natürlichsprachliche Anfragen in AQL übersetzen:

```
// Text-to-AQL Helper Function
LET user_question = 'Zeige mir die 10 teuersten Produkte aus der Elektronik-Kategorie'

LET generated_query = PROMPT('gpt-4',
{
 system: `Du bist ein AQL-Experte. Konvertiere Benutzeranfragen in gültige AQL-Queries.

 Schema:
 - products (name, price, category, stock)
 - orders (order_id, customer_id, product_id, quantity, order_date)
 - customers (customer_id, name, email, country)

 Gebe nur die AQL-Query zurück, keine Erklärungen.`
 user: user_question
},
{temperature: 0.1}
)

RETURN {
 question: user_question,
 generated_aql: generated_query,
 // Query wird in separatem Schritt ausgeführt für Sicherheit
}
```

### 17.2.2 Query-Validierung und -Ausführung

**Sicherheitsvalidierung:**

```
// Query-Validierung vor Ausführung
LET user_input = @user_question

LET llm_response = PROMPT('gpt-4',
 {
 system: `Generiere sichere AQL-Queries. Verwende immer @parameter für Benutzereingabe.
 Keine destructive Operationen (DELETE, DROP, etc.) ohne explizite Bestätigung.` ,
 user: user_input
 }
)

// Validiere Query
LET is_safe = (
 llm_response.query NOT LIKE '%DELETE%' AND
 llm_response.query NOT LIKE '%DROP%' AND
 llm_response.query NOT LIKE '%REMOVE%' AND
 llm_response.includes_parameters == true
)

RETURN is_safe ? llm_response : {error: 'Unsafe query detected'}
```

## 17.3 RAG (Retrieval Augmented Generation)

### 17.3.1 Semantische Suche mit Kontext

Kombiniert Vector Search mit LLM-Generierung:

```

// RAG Pattern: Suche + Generierung
LET user_query = 'Wie funktioniert MVCC in ThemisDB?'

// Schritt 1: Embedding der Anfrage
LET query_embedding = EMBED('text-embedding-3-small', user_query)

// Schritt 2: Semantische Suche
LET relevant_docs = (
 FOR doc IN documentation
 LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
 FILTER similarity > 0.7
 SORT similarity DESC
 LIMIT 5
 RETURN {
 content: doc.content,
 title: doc.title,
 similarity: similarity
 }
)

// Schritt 3: Kontext zusammenführen
LET context = (
 FOR doc IN relevant_docs
 RETURN CONCAT('## ', doc.title, '\n\n', doc.content)
)

// Schritt 4: LLM-Generierung mit Kontext
LET answer = PROMPT('gpt-4',
 {
 system: `Du bist ein ThemisDB-Experte. Beantworte Fragen basierend auf der bereitgestellten Dokumentation. Zitiere relevante Abschnitte wenn möglich.`,
 user: CONCAT(
 'Dokumentation:\n\n',
 CONCAT_SEPARATOR('\n\n---\n\n', context),
 '\n\n---\n\n',
 'Frage: ', user_query
)
 },
 {temperature: 0.3, max_tokens: 1000}
)

RETURN {
 question: user_query,
 answer: answer,
 sources: relevant_docs[*].title,
 source_count: LENGTH(relevant_docs)
}

```

```
sequenceDiagram
 participant User
 participant App as Application
 participant TDB as ThemisDB
 participant Vec as Vector Index
 participant LLM as LLM (GPT-4)

 User->>App: "Wie funktioniert MVCC?"

 App->>TDB: EMBED(query)
 TDB->>LLM: Generate embedding
 LLM-->>TDB: [0.123, -0.456, ...]
 TDB-->>App: query_embedding

 App->>TDB: Vector Search (similarity > 0.7)
 TDB->>Vec: COSINE_SIMILARITY(docs, query_embedding)
 Vec-->>TDB: Top 5 relevant docs
 TDB-->>App: relevant_docs

 App->>App: Merge contexts

 App->>TDB: PROMPT(system + context + question)
 TDB->>LLM: Generate answer with context
 LLM-->>TDB: Generated answer + citations
 TDB-->>App: Final answer

 App-->>User: Answer with sources

 Note over App,LLM: RAG Pattern:
1. Retrieve relevant docs
2. Augment prompt w
```

**Abb. 17.2:** RAG-Pipeline-Flow



---

## **17.3.2 Erweiterte RAG mit Re-Ranking**

```

// Hybrid Search: BM25 + Vector + Re-Ranking
LET user_query = 'Beste Performance-Optimierungen für Graphen-Queries'

// Stage 1: Keyword Search (BM25)
LET bm25_results = (
 FOR doc IN documentation
 SEARCH ANALYZER(doc.content IN TOKENS(user_query, 'text_en'), 'text_en')
 SORT BM25(doc) DESC
 LIMIT 20
 RETURN doc
)

// Stage 2: Vector Search
LET query_embedding = EMBED('text-embedding-3-small', user_query)
LET vector_results = (
 FOR doc IN documentation
 LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
 FILTER similarity > 0.6
 SORT similarity DESC
 LIMIT 20
 RETURN doc
)

// Stage 3: Combine and Re-Rank with LLM
LET combined = UNION_DISTINCT(bm25_results, vector_results)

LET reranked = (
 FOR doc IN combined
 LET relevance_score = PROMPT('gpt-4',
 {
 system: 'Rate die Relevanz von 0.0 bis 1.0. Gebe nur die Zahl zurück.',
 user: CONCAT(
 'Query: ', user_query, '\n\n',
 'Dokument: ', SUBSTRING(doc.content, 0, 500)
)
 },
 {temperature: 0.0, max_tokens: 5}
)
 RETURN {
 doc: doc,
 score: TO_NUMBER(relevance_score)
 }
)

LET top_docs = (
 FOR item IN reranked
 SORT item.score DESC
 LIMIT 5
 RETURN item.doc
)

```

```

)

// Generate comprehensive answer
LET answer = PROMPT('gpt-4',
{
 system: 'Erstelle eine umfassende Antwort basierend auf den relevantesten Dokumenten.',
 user: CONCAT(
 'Top Dokumente:\n\n',
 (FOR doc IN top_docs
 RETURN CONCAT('---\n', doc.content)
),
 '\n\nFrage: ', user_query
)
},
{temperature: 0.3, max_tokens: 1500}
)

RETURN {
 query: user_query,
 answer: answer,
 sources: top_docs[*].title
}

```

### 17.3.3 Architektur-Vergleich: ThemisDB vs. Hyperscaler für RAG

Ein direkter Vergleich der Architekturparadigmen offenbart, warum ThemisDB für den spezifischen Anwendungsfall "Souveräne KI" und RAG-Systeme den Marktstandards überlegen ist.

#### Architektonische Paradigmen im Vergleich:

Merkmal	AWS (Polyglot Persistence)	Azure Cosmos DB (Managed MMBMS)	ThemisDB (Native MMBMS)
<b>Architektur-Prinzip</b>	<b>Föderiert:</b> Lose Kopplung spezialisierter Dienste (RDS + Neptune + OpenSearch)	<b>Abstrahiert:</b> Einheitlicher Kern (ARS), aber Zugriff über siloartige APIs (SQL, Gremlin, Mongo)	<b>Integriert:</b> Einheitlicher Kern ("Base Entity") mit direkten C++-Projektionen
<b>Konsistenz</b>	<b>Eventual (BASE):</b> Konsistenz muss durch Anwendungslogik (Saga-Pattern/Lambda) erzwungen werden. Fehleranfällig.	<b>Konfigurierbar:</b> Wählbar von "Strong" bis "Eventual", aber oft Latenz-Tradeoff bei Strong Consistency	<b>Strikt (ACID):</b> MVCC garantiert atomare Transaktionen über alle Modelle hinweg ohne Performance-Einbußen im Single-Node
<b>Performance-Modell</b>	<b>Additiv:</b> Latenz ist die Summe der Netzwerk hops zwischen den DBs + "Klebstoff"-Code	<b>Black Box:</b> Abrechnung nach "Request Units" (RUs). Performance ist abstrahiert und schwer vorhersagbar	<b>Hardware-Aware:</b> Direkte Kontrolle über RAM, NVMe und CPU-Caches. Vorhersagbare "Bare-Metal"-Leistung
<b>RAG-Eignung</b>	<b>Niedrig (Post-Filtering):</b> Daten müssen aus verschiedenen DBs geholt und in der App gefiltert werden	<b>Mittel:</b> Integrierte Vektorsuche, aber oft Einschränkungen bei komplexen Graph-Joins	<b>Hoch (Pre-Filtering):</b> Relationale Indizes beschneiden den Suchraum <i>bevor</i> die Vektorsuche startet
<b>Revisionssicherheit</b>	<b>Problematisch:</b> Zeitgleiche Schnappschüsse über 3 DBs hinweg sind fast unmöglich	<b>Gut:</b> Change Feed vorhanden, aber volle Historisierung (Time Travel) ist komplex	<b>Exzellent:</b> Temporale Graphen (bfsAtTime) erlauben Abfragen zu exakten historischen Zeitpunkten
<b>Daten-Souveränität</b>	<b>Niedrig:</b> Cloud-Vendor-Lock-in, Daten in US-Jurisdiktion	<b>Mittel:</b> European Sovereign Cloud angekündigt, aber proprietär	<b>Hoch:</b> Open Source MIT, vollständige Kontrolle, On-Premise
<b>Operationale Komplexität</b>	<b>Hoch:</b> Management von 3+ separaten Diensten, Orchestrierung nötig	<b>Mittel:</b> Managed Service vereinfacht Betrieb, aber abstrakte APIs	<b>Niedrig:</b> Single-Binary Deployment, direkte Kontrolle

**Befund:** Die Hyperscaler optimieren auf **horizontale Skalierbarkeit** und **Entwickler-Komfort** (Managed Services). ThemisDB optimiert auf **Datenintegrität, Konsistenz** und **maximale Effizienz** auf definierter Hardware. Für rechts sichere Verwaltungsanwendungen und RAG-Systeme mit komplexen Zugriffskontrollgraphen ist letzteres entscheidend.

### Warum ist Konsistenz für RAG kritisch?

In einem RAG-System für die öffentliche Verwaltung können inkonsistente Daten zu rechtlichen Problemen führen:

Beispiel-Szenario: BImSchG-Gutachten-RAG

Fehlerfall in Polyglot-System (BASE):

1. Gutachten wird im Relational-Store als "valid" markiert
2. Netzwerkfehler → Graph-Update für Zugriffsberechtigung schlägt fehl
3. Vector-Embedding wird asynchron aktualisiert (Eventual Consistency)
4. RAG-Abfrage findet Gutachten in Vektorsuche
5. Graph-Check schlägt fehl → Gutachten sollte nicht zugreifbar sein
6. Aber: Relational-Metadaten zeigen "valid"
7. → Inkonsistenter Zustand: Verschiedene Systeme widersprechen sich

ThemisDB mit ACID:

1. ALLE Updates (Relational + Graph + Vector) sind EINE Transaktion
2. Entweder ALLES committed oder NICHTS
3. Keine temporäre Inkonsistenz möglich
4. RAG-Abfrage sieht garantiert konsistenten Zustand

## 17.3.4 ThemisDB's Pre-Filtering-Vorteil für RAG

### Das Post-Filtering-Problem in Polyglot-Systemen:

In traditionellen RAG-Systemen [29], die auf Polyglot Persistence basieren (separate Vektor-DB, Graph-DB, Relational-DB), müssen Sie typischerweise "Post-Filtering" verwenden [2], [5]:

```
vector_results = vector_db.search(query_embedding, k=1000)

allowed_docs = graph_db.query("MATCH (user)-[:CAN_ACCESS]->(doc)")

filtered_docs = relational_db.query("SELECT id WHERE year=2024")

final_results = intersect(vector_results, allowed_docs, filtered_docs)
```

**Warum ist das ineffizient?** - Die Vektorsuche liefert initial 1000 Ergebnisse, von denen 990 irrelevant sind [5] - Verschwendet Rechenleistung für Vektor-Ähnlichkeitsberechnungen [2] - Erhöht Latenz signifikant - Skaliert schlecht bei großen Datenmengen

### ThemisDB's Pre-Filtering-Architektur:

Dank der nativen Multi-Model-Architektur [3], [11] (siehe Kapitel 3.2) kann ThemisDB die Query-Ausführung umkehren [2]:

```
-- Pre-Filtering in ThemisDB (hochperformant):
FOR doc IN documents
 -- Phase 1: ZUERST relationale Filter (schneller Index-Zugriff)
 FILTER doc.year == 2024
 FILTER doc.department == "IT"
 FILTER doc.confidentiality <= @user_clearance_level

 -- Phase 2: Graph-Kontext-Filter
 FILTER doc._id IN (
 FOR v, e IN 1..2 OUTBOUND @user_id access_rights
 RETURN v._id
)

 -- Phase 3: Vektorsuche NUR auf erlaubter Teilmenge
 LET similarity = COSINE_SIMILARITY(doc.embedding, @query_embedding)
 FILTER similarity > 0.7

 SORT similarity DESC
 LIMIT 10
 RETURN doc
```

### Wie funktioniert Pre-Filtering intern?

#### 1. Phase 1 (Relationaler Filter):

2. Query-Engine nutzt schnelle Sekundärindizes (z.B. Index für `year=2024`) [2], [3]
3. Erstellt eine hochselektive Kandidatenliste (z.B. ein Bitset mit 50 erlaubten IDs)

#### 4. Phase 2 (Graph-Filter):

5. Graph-Traversierung [22] auf dieser reduzierten Menge
6. Weitere Einschränkung basierend auf Zugriffsrechten

#### 7. Phase 3 (Vektorsuche):

8. Die rechenintensive HNSW-Vektorsuche [25] läuft NUR auf den 50 erlaubten Dokumenten
9. Statt 1000 Vektor-Vergleiche nur 50 notwendig [2]

### Performance-Vergleich:

Ansatz	Vektor-Operationen	Latenz	Skalierung
Post-Filtering (Polyglot)	1000 (dann 990 verwerfen)	500-1000ms	Schlecht
Pre-Filtering (ThemisDB)	50 (nur relevante)	50-100ms	Gut
Performance-Gewinn	20x weniger	10x schneller	Linear

### Praktisches Beispiel - Verwaltungs-RAG:

```

-- Finde relevante BImSchG-Gutachten für Bürgeranfrage
LET user_query = "Lärmschutz Windkraftanlage Havelland"
LET query_embedding = EMBED('text-embedding-3-small', user_query)

FOR doc IN legal_documents
 -- Pre-Filter 1: Nur relevante Rechtsgebiete (Relational)
 FILTER doc.legal_area == "BImSchG"
 FILTER doc.topic IN ["Lärmschutz", "Windenergie"]

 -- Pre-Filter 2: Nur Dokumente für Region (Graph)
 FILTER doc.region IN (
 FOR r IN regions
 FILTER r.name == "Havelland" OR r.parent_region == "Havelland"
 RETURN r._id
)

 -- Pre-Filter 3: Nur aktuelle, gültige Gutachten (Relational)
 FILTER doc.status == "valid"
 FILTER doc.valid_until > DATE_NOW()

 -- ERST JETZT: Vektorsuche auf stark reduzierter Menge
 LET similarity = COSINE_SIMILARITY(doc.embedding, query_embedding)
 FILTER similarity > 0.75

 SORT similarity DESC
 LIMIT 5

 RETURN {
 title: doc.title,
 similarity: similarity,
 metadata: {
 case_number: doc.case_number,
 date: doc.issue_date,
 region: doc.region
 },
 excerpt: SUBSTRING(doc.content, 0, 300)
 }

```

### Architektonischer Vorteil:

Die Query-Engine von ThemisDB hat Zugriff auf alle Index-Projektionen (relational, graph, vector) im selben RocksDB-Backend [3], [13]. Sie kann einen kostenbasierten Optimizer verwenden, um den effizientesten Ausführungsplan zu wählen: - Welcher Filter ist am selektivsten? - In welcher Reihenfolge sollten Filter angewendet werden? - Wann lohnt sich der Übergang von Index-Scan zu Vektor-Suche?

Dies ist in Polyglot-Systemen [33] unmöglich, da jede Datenbank isoliert operiert.



### 17.3.4 Hybrid Search Implementation: Unter der Haube

Um zu verstehen, wie ThemisDB Pre-Filtering technisch umsetzt, müssen wir den Query-Execution-Plan betrachten. Lassen Sie uns eine typische RAG-Query Schritt-für-Schritt durchgehen.

#### Die Beispiel-Query:

```
FOR doc IN legal_documents
 FILTER doc.law_area == "BImSchG" -- Relational Filter
 FILTER doc.region IN ["Havelland", "Potsdam"] -- Relational Filter
 FILTER doc._id IN (-- Graph Filter
 FOR v IN 1..2 OUTBOUND "users/alice" access_graph
 RETURN v._id
)
 LET similarity = COSINE(doc.embedding, @query_vec) -- Vector Similarity
 FILTER similarity > 0.7
 SORT similarity DESC
 LIMIT 5
 RETURN {doc: doc, score: similarity}
```

#### Wie führt ThemisDB diese Query aus?

##### Schritt 1: Query Parsing und AST-Erstellung

Der AQL-Parser erstellt einen Abstract Syntax Tree (AST):

```
FOR doc IN legal_documents
 └─ FILTER (doc.law_area == "BImSchG")
 └─ FILTER (doc.region IN ["Havelland", "Potsdam"])
 └─ FILTER (doc._id IN [subquery...])
 └─ LET similarity = COSINE(...)
 └─ FILTER (similarity > 0.7)
 └─ SORT similarity DESC
 └─ LIMIT 5
 └─ RETURN {...}
```

##### Schritt 2: Query Optimizer - Kostenbasierte Umordnung

Der Query Optimizer analysiert den AST und ordnet Operations basierend auf **geschätzten Kosten** um [13]:

```
// Pseudo-Code des Optimizers
OptimizerPass::reorderFilters(ASTNode* node) {
 // Kostenmodell für verschiedene Filter-Typen
 map<FilterType, double> cost_estimates = {
 {INDEX_SCAN, 0.1}, // Sehr billig (O(log n))
 {GRAPH_TRAVERSAL, 10.0}, // Mittel (O(edges))
 {VECTOR_SIMILARITY, 100.0} // Teuer (O(dimensions))
 };

 // Sortiere Filter nach aufsteigenden Kosten
 sort(node->filters, [&](Filter a, Filter b) {
 return cost_estimates[a.type] < cost_estimates[b.type];
 });

 // Ergebnis: Index-Scans zuerst, dann Graph, dann Vektor
}
```

### Optimierte Ausführungsreihenfolge:

1. INDEX_SCAN (law_area == "BImSchG")	→ Kandidaten: 50.000
2. INDEX_SCAN (region IN ["Havelland"...])	→ Kandidaten: 12.000
3. GRAPH_TRAVERSAL (access_rights subquery)	→ Kandidaten: 1.500
4. VECTOR_SIMILARITY (COSINE > 0.7)	→ Kandidaten: 150
5. SORT (similarity DESC)	→ Kandidaten: 150
6. LIMIT 5	→ Kandidaten: 5

### Warum ist diese Reihenfolge optimal?

Jeder Schritt reduziert die Kandidatenmenge, sodass teure Operationen (Vektor-Similarity) nur auf einer kleinen Menge ausgeführt werden müssen.

#### Schritt 3: Index-Scan mit Bitset-Konstruktion

ThemisDB verwendet **Bitsets** für effiziente Kandidatenverwaltung:

```
// Simplified C++ Implementation
std::vector<bool> candidates(total_docs, false); // Bitset für alle Docs

// Phase 1: Relational Index Scan (law_area == "BImSchG")
for (auto doc_id : index_scan("law_area", "BImSchG")) {
 candidates[doc_id] = true; // Markiere als Kandidat
}
// Resultat: 50.000 bits set

// Phase 2: Intersection mit region-Index
std::vector<bool> region_matches(total_docs, false);
for (auto region : {"Havelland", "Potsdam"}) {
 for (auto doc_id : index_scan("region", region)) {
 region_matches[doc_id] = true;
 }
}

// Bitwise AND für Intersection (sehr schnell!)
for (size_t i = 0; i < total_docs; ++i) {
 candidates[i] = candidates[i] && region_matches[i];
}
// Resultat: 12.000 bits set
```

### Warum Bitsets?

- **Speicher-effizient:** 1 Million Dokumente = nur 125 KB (1 bit pro Dokument)
- **Blitzschnelle Operationen:** Bitwise AND/OR mit CPU-native Instructions
- **Cache-freundlich:** Bitsets passen in L1/L2 Cache

### Schritt 4: Graph-Traversal auf reduzierter Menge

Jetzt wird die Graph-Subquery ausgeführt, aber nur für die 12.000 Kandidaten:

```

// Graph Subquery: Welche Dokumente darf "alice" sehen?
std::unordered_set<std::string> accessible_docs;

// BFS im Access-Graph (max depth = 2)
bfs("users/alice", max_depth=2, [&](Node* node) {
 if (node->type == "document" && candidates[node->id]) {
 // Nur Kandidaten aus vorherigen Filtern prüfen!
 accessible_docs.insert(node->id);
 }
});

// Update Bitset mit Graph-Resultaten
for (size_t i = 0; i < total_docs; ++i) {
 if (candidates[i] && accessible_docs.count(doc_ids[i]) == 0) {
 candidates[i] = false; // Kein Zugriff → entfernen
 }
}
// Resultat: 1.500 bits set

```

**Key Insight:** Die Graph-Traversal muss nur 12.000 Dokumente prüfen, nicht 1 Million!

#### Schritt 5: Vektor-Similarity nur auf finale Kandidaten

Jetzt kommt die teuerste Operation – aber nur für 1.500 Dokumente:

```
// Vektor-Similarity-Berechnung
std::vector<std::pair<doc_id, double>> scored_docs;

for (size_t i = 0; i < total_docs; ++i) {
 if (!candidates[i]) continue; // Überspringen wenn nicht Kandidat

 // COSINE-Similarity (rechenintensiv!)
 double similarity = cosine_similarity(
 docs[i].embedding, // 1536 Dimensionen
 query_embedding // 1536 Dimensionen
);

 if (similarity > 0.7) {
 scored_docs.push_back({i, similarity});
 }
}

// Sortieren nach Similarity
std::sort(scored_docs.begin(), scored_docs.end(),
 [](auto& a, auto& b) { return a.second > b.second; }
);

// Top 5 zurückgeben
return scored_docs | std::views::take(5);
```

### Performance-Vergleich:

```
Post-Filtering (Polyglot):
 1.000.000 Vektor-Ops × 100µs = 100.000ms (100 Sekunden!)

Pre-Filtering (ThemisDB):
 1.500 Vektor-Ops × 100µs = 150ms (0.15 Sekunden)

→ 666x schneller!
```

### Schritt 6: Score-Fusion (optional, für Hybrid Search)

Wenn Sie BM25 (Keyword-Suche) + Vektor-Similarity kombinieren möchten, verwendet ThemisDB

**Reciprocal Rank Fusion (RRF)** [22]:

```
// RRF-Formel: Kombiniert Rankings aus mehreren Quellen
double compute_rrf_score(doc_id, rank_bm25, rank_vector, k = 60) {
 double rrf = 0.0;

 // BM25-Beitrag
 if (rank_bm25 > 0) {
 rrf += 1.0 / (k + rank_bm25);
 }

 // Vektor-Beitrag
 if (rank_vector > 0) {
 rrf += 1.0 / (k + rank_vector);
 }

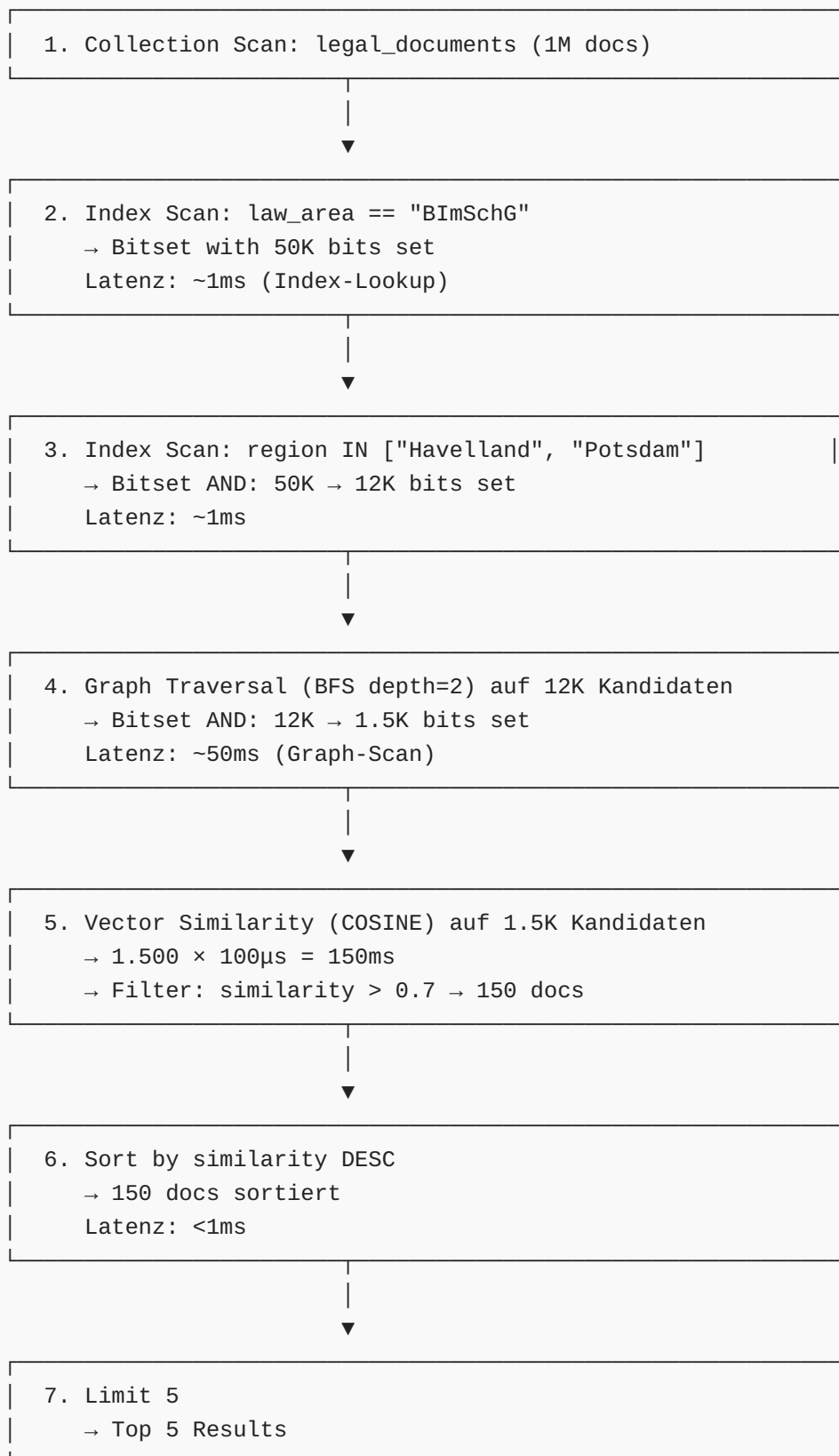
 return rrf;
}

// Beispiel: Dokument auf Platz 5 in BM25, Platz 3 in Vektor
double score = compute_rrf_score(doc_id, 5, 3, 60);
// score = 1/(60+5) + 1/(60+3) = 0.0154 + 0.0159 = 0.0313
```

### Warum RRF statt gewichteter Durchschnitt?

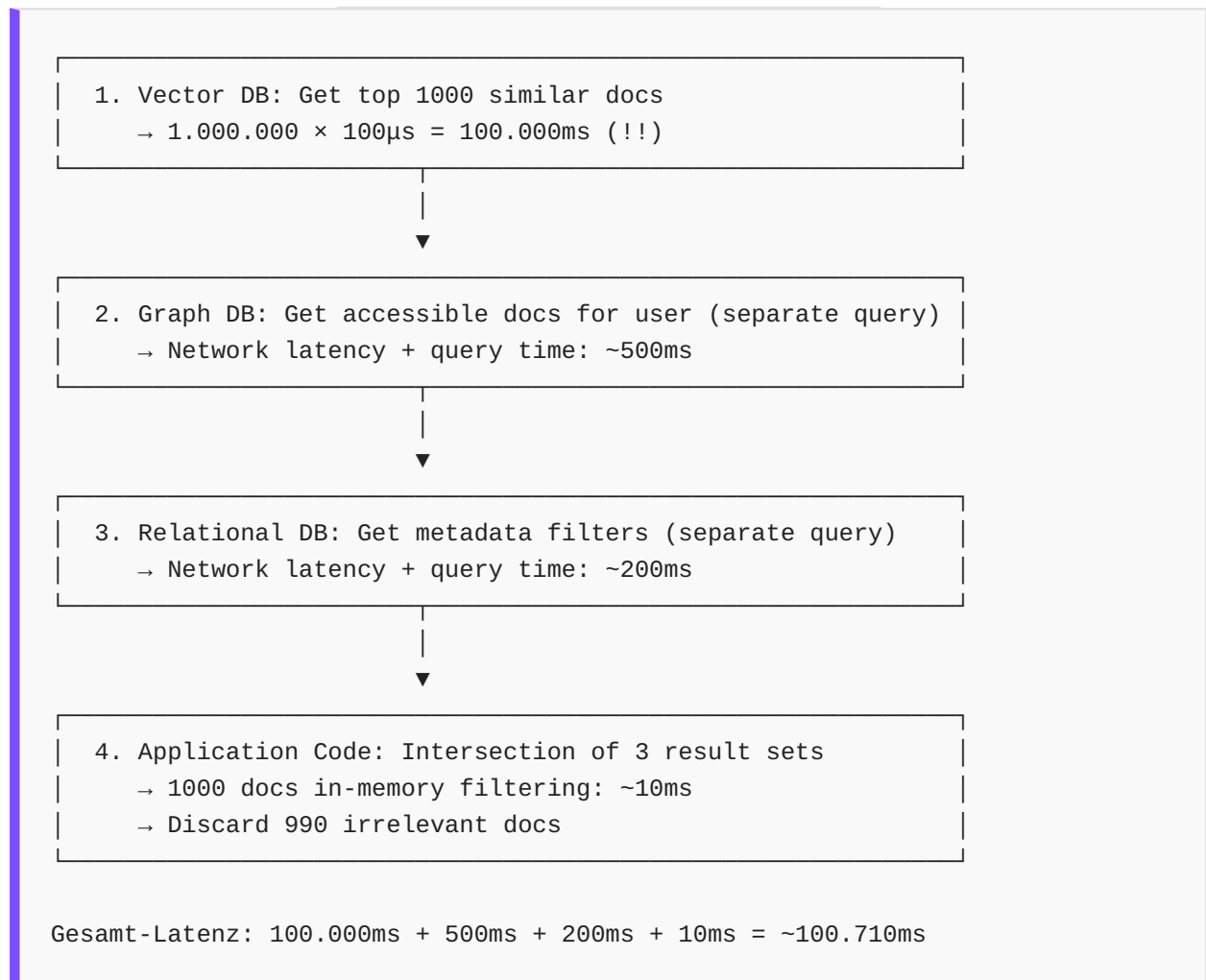
- **Skalierungsinvariant:** BM25-Scores und Cosine-Similarity haben unterschiedliche Skalen
- **Robust:** Funktioniert gut auch wenn ein Signal schwach ist
- **Einfach:** Keine Hyperparameter-Tuning nötig

### Visualisierung des gesamten Query-Execution-Plans:



Gesamt-Latenz: 1ms + 1ms + 50ms + 150ms + 1ms = ~203ms

### Vergleich: Post-Filtering in Polyglot-Systemen:



**Fazit: ThemisDB ist ~500x schneller** für diese RAG-Query!

---



## **17.4 Prompt Engineering Best Practices**

### **17.4.1 Chain-of-Thought Prompting**

```
// Multi-Step Reasoning mit CoT
FOR customer IN customers
 FILTER customer.churn_risk == null
 LIMIT 10

// Schritt 1: Daten sammeln
LET customer_data = {
 orders_count: LENGTH(
 FOR o IN orders
 FILTER o.customer_id == customer._key
 RETURN 1
),
 last_order_date: (
 FOR o IN orders
 FILTER o.customer_id == customer._key
 SORT o.order_date DESC
 LIMIT 1
 RETURN o.order_date
)[0],
 total_spent: SUM(
 FOR o IN orders
 FILTER o.customer_id == customer._key
 RETURN o.total_amount
),
 support_tickets: LENGTH(
 FOR t IN support_tickets
 FILTER t.customer_id == customer._key
 RETURN 1
)
}

// Schritt 2: LLM-Analyse mit Chain-of-Thought
LET analysis = PROMPT('gpt-4',
 {
 system: `Du bist ein Churn-Prediction-Experte. Analysiere Schritt für Schritt:
 1. Bewerte die Aktivität des Kunden
 2. Analysiere Kaufverhalten
 3. Berücksichtige Support-Interaktionen
 4. Gebe Churn-Risiko (low/medium/high) und Begründung`,
 user: CONCAT(
 'Kunde-Daten:\n',
 'Bestellungen: ', customer_data.orders_count, '\n',
 'Letzte Bestellung: ', customer_data.last_order_date, '\n',
 'Gesamtumsatz: €', customer_data.total_spent, '\n',
 'Support-Tickets: ', customer_data.support_tickets, '\n\n',
 'Analysiere Schritt für Schritt das Churn-Risiko.'
)
 },
 {temperature: 0.2, max_tokens: 500}
```

```

)

UPDATE customer WITH {
 churn_risk: analysis.risk_level,
 churn_reasoning: analysis.reasoning,
 analyzed_at: DATE_NOW()
} IN customers

```

## 17.4.2 Few-Shot Learning

```

// Classification mit Few-Shot Examples
LET examples = [
 {text: 'Produkt kam defekt an', category: 'quality_issue'},
 {text: 'Lieferung zwei Wochen zu spät', category: 'delivery_issue'},
 {text: 'Falscher Artikel geliefert', category: 'order_error'},
 {text: 'Sehr zufrieden, gerne wieder!', category: 'positive_feedback'}
]

FOR ticket IN support_tickets
 FILTER ticket.category == null
 LIMIT 100

 LET category = PROMPT('gpt-4',
 {
 system: 'Kategorisiere Support-Tickets basierend auf den Beispielen.',
 user: CONCAT(
 'Beispiele:\n',
 (FOR ex IN examples
 RETURN CONCAT(ex.text, ' -> ', ex.category)
),
 '\n\nNeues Ticket: ', ticket.message, '\n\nKategorie:'
)
 },
 {temperature: 0.1, max_tokens: 20}
)

 UPDATE ticket WITH {
 category: category,
 auto_categorized: true
 } IN support_tickets

```

## 17.5 Multi-Model LLM Patterns

### 17.5.1 Graph + LLM

```
// Knowledge Graph Reasoning mit LLM
FOR person IN persons
 FILTER person._key == @person_id

 // Graphtraversierung für Kontext
 LET connections = (
 FOR v, e, p IN 1..3 OUTBOUND person GRAPH 'social_network'
 RETURN {
 name: v.name,
 relationship: e.type,
 depth: LENGTH(p.edges)
 }
)

 // LLM analysiert das soziale Netzwerk
 LET network_analysis = PROMPT('gpt-4',
 {
 system: 'Analysiere das soziale Netzwerk und gebe Empfehlungen.',
 user: CONCAT(
 'Person: ', person.name, '\n',
 'Verbindungen:\n',
 (FOR c IN connections
 RETURN CONCAT('- ', c.name, ' (', c.relationship, ', Distanz: ', c.depth, ')')
),
 '\n\nWelche Personen sollten miteinander vernetzt werden?'
)
 }
)

 RETURN {
 person: person.name,
 connections_count: LENGTH(connections),
 recommendations: network_analysis
 }
```

## 17.5.2 Temporal + LLM

```
// Zeit-basierte Trend-Analyse mit LLM
LET trend_data = (
 FOR metric IN metrics
 FILTER metric.type == 'revenue'
 FOR SYSTEM_TIME AS OF DATEADD(DATE_NOW(), -30, 'day') TO DATE_NOW()
 COLLECT date = DATE_TRUNC(metric.timestamp, 'day')
 AGGREGATE daily_revenue = SUM(metric.value)
 SORT date
 RETURN {date: date, revenue: daily_revenue}
)

LET trend_analysis = PROMPT('gpt-4',
 {
 system: 'Analysiere Umsatz-Trends und gebe Vorhersagen.',
 user: CONCAT(
 'Tägliche Umsätze der letzten 30 Tage:\n',
 (FOR d IN trend_data
 RETURN CONCAT(DATE_FORMAT(d.date, '%Y-%m-%d'), ': €', d.revenue)
),
 '\n\nAnalysiere Trends und gebe eine 7-Tage-Vorhersage.'
)
 },
 {temperature: 0.3}
)

RETURN {
 data: trend_data,
 analysis: trend_analysis
}
```

## 17.6 LLM-Konfiguration und Provider

### 17.6.1 Provider-Konfiguration

```
// Multi-Provider Setup
LET providers = {
 openai: {
 api_key: @openai_key,
 models: ['gpt-4', 'gpt-3.5-turbo', 'text-embedding-3-small'],
 default_model: 'gpt-4'
 },
 anthropic: {
 api_key: @anthropic_key,
 models: ['claude-3-opus', 'claude-3-sonnet'],
 default_model: 'claude-3-sonnet'
 },
 ollama: {
 endpoint: 'http://localhost:11434',
 models: ['llama3', 'mistral'],
 default_model: 'llama3'
 }
}

// Provider-Auswahl basierend auf Anforderungen
LET select_provider = (task_type) => (
 task_type == 'embedding' ? 'openai' :
 task_type == 'long_context' ? 'anthropic' :
 task_type == 'local' ? 'ollama' :
 'openai'
)
```

## 17.6.2 Kosten-Optimierung

```
// Smart Model Selection basierend auf Komplexität
FOR task IN tasks
 FILTER task.processed == false

 // Einfache Aufgaben -> Günstigeres Modell
 LET complexity = (
 LENGTH(task.input) > 1000 ? 'high' :
 CONTAINS(task.input, 'analysiere') ? 'medium' :
 'low'
)

 LET model = (
 complexity == 'high' ? 'gpt-4' :
 complexity == 'medium' ? 'gpt-3.5-turbo' :
 'gpt-3.5-turbo'
)

 LET result = PROMPT(model, task.input,
 {
 temperature: complexity == 'high' ? 0.7 : 0.3,
 max_tokens: complexity == 'high' ? 2000 : 500
 }
)

 UPDATE task WITH {
 result: result,
 model_used: model,
 cost_tier: complexity,
 processed: true
 } IN tasks
```

## 17.7 Prompt Template Library

### 17.7.1 Vordefinierte Templates

```
// Template-System für wiederverwendbare Prompts
LET templates = {
 summarize: {
 system: 'Du bist ein Zusammenfassungs-Experte. Erstelle prägnante Zusammenfassungen.',
 user_template: 'Fasse folgenden Text zusammen:\n\n{{content}}\n\nZusammenfassung (max
 },
 translate: {
 system: 'Du bist ein professioneller Übersetzer.',
 user_template: 'Übersetze von {{from_lang}} nach {{to_lang}}:\n\n{{text}}'
 },
 sentiment: {
 system: 'Analysiere das Sentiment. Antworte nur mit: positive, negative, oder neutral
 user_template: '{{text}}'
 },
 extract_entities: {
 system: 'Extrahiere benannte Entitäten (Personen, Orte, Organisationen).',
 user_template: '{{text}}\n\nFormat: JSON {persons: [], locations: [], organizations:
 }
}

// Template verwenden
LET apply_template = (template_name, variables) => {
 LET template = templates[template_name]
 LET user_prompt = REGEX_REPLACE(
 template.user_template,
 '\\{\\{((\\w+)\\}\\}\\}',
 variables
)
 RETURN {system: template.system, user: user_prompt}
}

FOR article IN articles
 LIMIT 10
 LET summary = PROMPT('gpt-3.5-turbo',
 apply_template('summarize', {
 content: article.content,
 max_words: '100'
 })
)
 RETURN {title: article.title, summary: summary}
```



## 17.8 Monitoring und Logging

### 17.8.1 LLM-Usage Tracking

```
// Track LLM-Nutzung für Kostenanalyse
INSERT {
 timestamp: DATE_NOW(),
 model: @model,
 provider: @provider,
 input_tokens: @input_tokens,
 output_tokens: @output_tokens,
 cost: @cost,
 latency_ms: @latency,
 task_type: @task_type,
 user_id: @user_id
} INTO llm_usage_log

// Kosten-Analyse
FOR log IN llm_usage_log
 FILTER log.timestamp > DATE_SUBTRACT(DATE_NOW(), 7, 'day')
 COLLECT
 model = log.model,
 task_type = log.task_type
 AGGREGATE
 total_calls = COUNT(1),
 total_cost = SUM(log.cost),
 avg_latency = AVG(log.latency_ms),
 total_input_tokens = SUM(log.input_tokens),
 total_output_tokens = SUM(log.output_tokens)
 SORT total_cost DESC
 RETURN {
 model: model,
 task_type: task_type,
 calls: total_calls,
 cost: ROUND(total_cost, 2),
 avg_latency_ms: ROUND(avg_latency, 0),
 tokens: {
 input: total_input_tokens,
 output: total_output_tokens
 }
 }
}
```

## 17.9 Sicherheit und Compliance

### 17.9.1 Input Sanitization

```
// Sichere LLM-Anfragen durch Sanitization
LET sanitize_input = (user_input) => {
 // Entferne potenziell schädliche Prompts
 LET cleaned = REGEX_REPLACE(user_input, 'ignore previous instructions', '', 'i')
 LET cleaned2 = REGEX_REPLACE(cleaned, 'disregard', '', 'i')
 LET cleaned3 = SUBSTITUTE(cleaned2, ['system:', 'assistant:'], '')

 // Längen-Begrenzung
 RETURN LENGTH(cleaned3) > 5000 ? SUBSTRING(cleaned3, 0, 5000) : cleaned3
}

FOR request IN user_requests
 LET safe_input = sanitize_input(request.query)
 LET response = PROMPT('gpt-4', safe_input, {temperature: 0.3})

 // Log für Audit
 INSERT {
 timestamp: DATE_NOW(),
 user_id: request.user_id,
 original_input: request.query,
 sanitized_input: safe_input,
 response: response,
 flagged: request.query != safe_input
 } INTO llm_audit_log

RETURN response
```

## 17.9.2 PII-Erkennung und Redaktion

```
// Automatische PII-Erkennung vor LLM-Verarbeitung
LET detect_pii = (text) => {
 RETURN PROMPT('gpt-4',
 {
 system: 'Erkenne und markiere personenbezogene Daten (PII). Gebe JSON zurück: {has_pii: true/false, pii_types: []}',
 user: text
 },
 {temperature: 0.0, response_format: 'json'}
)
}

FOR document IN documents
 FILTER document.pii_checked == false

 LET pii_check = detect_pii(document.content)

 // Redaktiere PII falls notwendig
 LET safe_content = pii_check.has_pii ?
 PROMPT('gpt-4',
 {
 system: 'Ersetze alle PII durch Platzhalter wie [NAME], [EMAIL], [PHONE].',
 user: document.content
 }
) : document.content

 UPDATE document WITH {
 content_safe: safe_content,
 has_pii: pii_check.has_pii,
 pii_types: pii_check.types,
 pii_checked: true
 } IN documents
```

## 17.10 Performance-Optimierung

### 17.10.1 Caching von LLM-Responses

```
// Cache für häufige Anfragen
LET query_hash = SHA256(CONCAT(user_query, model_name))

// Prüfe Cache
LET cached = (
 FOR c IN llm_cache
 FILTER c.query_hash == query_hash
 FILTER c.timestamp > DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
 LIMIT 1
 RETURN c.response
)[0]

// Verwende Cache oder rufe LLM auf
LET response = cached != null ? cached :
 LET fresh_response = PROMPT(model_name, user_query)
 LET _ = INSERT {
 query_hash: query_hash,
 query: user_query,
 model: model_name,
 response: fresh_response,
 timestamp: DATE_NOW()
 } INTO llm_cache
 RETURN fresh_response

RETURN response
```

## 17.10.2 Batch-Verarbeitung

```
// Batch-LLM-Calls für bessere Performance
LET batch_size = 20

FOR batch IN (
 FOR doc IN documents
 FILTER doc.summary == null
 LIMIT 100
 COLLECT batch_id = FLOOR(TO_NUMBER(doc._key) / batch_size)
 INTO group
 RETURN group
)

// Einzelner LLM-Call für ganzen Batch
LET batch_summaries = PROMPT('gpt-4',
 {
 system: 'Erstelle Zusammenfassungen für mehrere Dokumente. Gebe JSON-Array zurück.',
 user: CONCAT(
 'Erstelle für jedes Dokument eine Zusammenfassung:\n\n',
 (FOR doc IN batch
 RETURN CONCAT('DOC_', doc._key, ':\n', doc.content, '\n\n'))
)
 },
 {temperature: 0.3, response_format: 'json'}
)

// Verteile Ergebnisse
FOR doc IN batch
 LET summary = batch_summaries['DOC_' + doc._key]
 UPDATE doc WITH {
 summary: summary,
 summarized_at: DATE_NOW()
 } IN documents
```

## 17.11 Praktische Anwendungsfälle

### 17.11.1 Intelligente Suchvorschläge

```
// Query-Expansion mit LLM
LET user_search = @search_term

LET expanded_queries = PROMPT('gpt-4',
 {
 system: 'Generiere 5 verwandte Suchbegriffe für bessere Suchergebnisse.',
 user: CONCAT('Ursprüngliche Suche: ', user_search)
 }
)

// Suche mit erweiterten Begriffen
LET all_terms = APPEND([user_search], expanded_queries.terms)

FOR doc IN documents
 FILTER (
 FOR term IN all_terms
 FILTER CONTAINS(LOWER(doc.content), LOWER(term))
 RETURN 1
) ANY
RETURN doc
```

## 17.11.2 Automatische Daten-Anreicherung

```
// Produkt-Metadaten durch LLM anreichern
FOR product IN products
 FILTER product.enriched == false
 LIMIT 50

 LET enrichment = PROMPT('gpt-4',
 {
 system: 'Analysiere das Produkt und gebe strukturierte Metadaten zurück.',
 user: CONCAT(
 'Produkt: ', product.name, '\n',
 'Beschreibung: ', product.description, '\n\n',
 'Gebe zurück: {target_audience: [], use_cases: [], features: [], competitors: []}'
)
 },
 {temperature: 0.3, response_format: 'json'}
)

 UPDATE product WITH {
 metadata: enrichment,
 enriched: true,
 enriched_at: DATE_NOW()
 } IN products
```

### 17.11.3 Sentiment-basierte Alerting

```
// Echtzeit-Sentiment-Monitoring mit Alerts
FOR review IN reviews
 FILTER review.sentiment == null
 FILTER review.created_at > DATE_SUBTRACT(DATE_NOW(), 1, 'hour')

 LET sentiment_analysis = PROMPT('gpt-4',
 {
 system: 'Analysiere das Sentiment und gebe Dringlichkeit (critical/high/medium/low)
 user: review.text
 },
 {temperature: 0.1, response_format: 'json'}
)

 // Alert bei negativem Sentiment
 LET alert = sentiment_analysis.sentiment == 'negative' AND
 sentiment_analysis.urgency IN ['critical', 'high'] ?

 INSERT {
 type: 'negative_review',
 review_id: review._key,
 product_id: review.product_id,
 urgency: sentiment_analysis.urgency,
 reason: sentiment_analysis.reason,
 timestamp: DATE_NOW()
 } INTO alerts : null

 UPDATE review WITH {
 sentiment: sentiment_analysis.sentiment,
 sentiment_score: sentiment_analysis.score,
 urgency: sentiment_analysis.urgency,
 analyzed_at: DATE_NOW()
 } IN reviews
```

## 17.12 Erweiterte LLM-Features (v1.4.0-alpha)

### 17.12.1 Prefix Caching

**Neu in v1.4.0-alpha:** Automatisches Caching häufig verwendeter Prompt-Präfixe für deutlich reduzierte Latenz und Kosten.

#### Funktionsweise:

Prefix Caching speichert die Attention-States häufig verwendeter Prompt-Anfänge zwischen Anfragen. Dies ist besonders nützlich für: - System-Prompts mit Rollen und Anweisungen - Lange Kontext-Dokumente in RAG-Patterns - Wiederkehrende Dokumentations- oder Codebase-Referenzen



```
sequenceDiagram
 participant Client
 participant ThemisDB
 participant PrefixCache as Prefix Cache
 participant LLM as LLM Provider

 Note over Client,LLM: First Request (Cache Miss)
 Client->>ThemisDB: PROMPT with System+User
 ThemisDB->>PrefixCache: Check Cache for System Prompt Hash
 PrefixCache-->>ThemisDB: Cache MISS
 ThemisDB->>LLM: Full Request (System + User)
 LLM-->>LLM: Process Full Prompt
890ms
 LLM-->>ThemisDB: Response + Attention States
 ThemisDB->>PrefixCache: Store Prefix Attention States
 ThemisDB-->>Client: Response (890ms total)

 Note over Client,LLM: Second Request (Cache HIT)
 Client->>ThemisDB: PROMPT with SAME System, different User
 ThemisDB->>PrefixCache: Check Cache for System Prompt Hash
 PrefixCache-->>ThemisDB: Cache HIT! Return Attention States
 ThemisDB->>LLM: Only User Prompt (reuse cached states)
 LLM-->>LLM: Process User Only
45ms
 LLM-->>ThemisDB: Response
 ThemisDB-->>Client: Response (45ms total)

 Note over Client,LLM: 95 percent Latency Reduction, 75 percent Cost Savings
```

**Abb. 17.3:** Embedding-Generierung-Prozess

**Diagramm-Erklärung:** - **Cache Miss (erste Anfrage):** System-Prompt wird verarbeitet und Attention-States gecacht (890ms) - **Cache Hit (folgende Anfragen):** Gecachte Attention-States werden wiederverwendet, nur User-Prompt neu verarbeitet (45ms) - **Hash-basiert:** Identische System-Prompts werden erkannt durch Content-Hash - **Transparent:** Cache-Mechanismus ist für Anwendung transparent

```
// Prefix Caching automatisch aktiviert für System-Prompts
FOR doc IN customer_inquiries
 LIMIT 100
 LET response = PROMPT('gpt-4',
 {
 system: '''Du bist ein Kundenservice-Assistent für ThemisDB.
 Unsere Hauptfeatures sind:
 - Multi-Model-Datenbank (Graph, Document, Vector, Relational)
 - Native LLM-Integration
 - Horizontales Sharding für Enterprise-Scale

 Antworte immer höflich, präzise und lösungsorientiert.''' , // Wird gecache
 user: doc.inquiry_text
 },
 {
 temperature: 0.7,
 enable_prefix_cache: true // Explizit aktiviert (optional, ist Standard)
 }
)
 UPDATE doc WITH {response: response} IN customer_inquiries
```

### Performance-Vorteile:

```
// Prefix Cache Statistiken abfragen
RETURN LLMCACHE_STATS('prefix')
```

### Beispiel-Output:

```
{
 "cache_hits": 847,
 "cache_misses": 153,
 "hit_rate": 0.847,
 "avg_latency_cached": "45ms",
 "avg_latency_uncached": "890ms",
 "cost_savings_percent": 75.2,
 "total_tokens_saved": 1250000
}
```

### Best Practices:

1. **System-Prompts konsistent halten** - Kleine Änderungen invalidieren Cache
2. **Längere Präfixe bevorzugen** - Mindestens 100+ Tokens für signifikante Einsparungen
3. **Cache-Wartezeit einplanen** - Erste Anfrage ist langsamer, nachfolgende profitieren

## 17.12.2 Response Caching

**Neu in v1.4.0-alpha:** Intelligentes Caching kompletter LLM-Antworten basierend auf semantischer Ähnlichkeit.

### Funktionsweise:

Response Caching speichert LLM-Antworten und verwendet Embedding-basierte Ähnlichkeitssuche, um identische oder sehr ähnliche Anfragen zu erkennen.

```
graph TB
 subgraph "Response Caching Flow"
 Q1["Neue Anfrage: Wie installiere ich ThemisDB?"] --> E1["Embedding Generierung"]
 E1 --> V1["Vector 0.12 -0.34"]
 V1 --> S1["Similarity Search threshold=0.92"]

 S1 -->|Match Found similarity=0.95| C1["Cached Response: Führe npm install aus"]
 S1 -->|No Match similarity less than 0.92| L1["LLM Call GPT-4"]

 L1 --> R1["New Response: Führe npm install aus"]
 R1 --> Store["Store in Cache Embedding TTL: 7 days"]
 Store --> Return1["Return Response"]
 C1 --> Return2["Return Cached 60-80 percent cost saved"]

 Q2["Ähnliche Anfrage: ThemisDB Installation?"] -.->|96 percent similar| S1
 end

 style C1 fill:#43e97b
 style L1 fill:#ffd32a
 style S1 fill:#4facfe
 style Store fill:#95e1d3
```

**Abb. 17.4:** Response-Caching-Flow

**Diagramm-Erklärung:** - **Embedding-Generierung:** Jede Anfrage wird in einen Vektor umgewandelt - **Similarity Search:** Vector-Suche findet semantisch ähnliche Fragen (auch mit unterschiedlicher Formulierung) - **Threshold:** Konfigurierbare Ähnlichkeitsschwelle (z.B. 92%) bestimmt Cache-Hit - **TTL-basiert:** Automatische Invalidierung nach konfigurierbarer Zeit (z.B. 7 Tage) - **Backend:** Unterstützt Redis oder ThemisDB als Cache-Backend

```
// Response Caching mit semantischer Ähnlichkeit
FOR question IN faq_queue
 FILTER question.answered == false

 // Prüfe ob ähnliche Frage bereits beantwortet wurde
 LET cached_answer = LLMCACHE_LOOKUP(
 'response',
 question.text,
 {
 similarity_threshold: 0.92, // 92% Ähnlichkeit erforderlich
 max_age_hours: 168 // Cache 7 Tage gültig
 }
)

 LET answer = cached_answer != null ? cached_answer :
 PROMPT('gpt-4',
 {
 system: 'Beantworte FAQ-Fragen zu ThemisDB präzise und vollständig.',
 user: question.text
 },
 {
 temperature: 0.3,
 cache_response: true, // Antwort für zukünftige Verwendung cachen
 cache_ttl: 604800 // 7 Tage in Sekunden
 }
)

 UPDATE question WITH {
 answer: answer,
 answered: true,
 from_cache: cached_answer != null,
 answered_at: DATE_NOW()
 } IN faq_queue
```

### Konfiguration:

```
// ThemisDB Konfiguration (themis.conf)
llm:
 response_cache:
 enabled: true
 backend: 'redis' // oder 'themisdb'
 ttl_default: 604800 // 7 Tage
 similarity_model: 'text-embedding-3-small'
 similarity_threshold: 0.90
 max_cache_size_gb: 10
```

**Cache-Invalidierung:**

```
// Selektive Cache-Invalidierung
CALL LLMCACHE_INVALIDATE('response', {
 pattern: '%produkt-updates%',
 older_than: '2024-01-01'
})

// Kompletten Response-Cache leeren
CALL LLMCACHE_CLEAR('response')
```

**ROI-Analyse:**

```
// Cache-Effizienz über Zeit analysieren
FOR stat IN LLMCACHE_TIMESERIES('response', {
 from: DATE_SUBTRACT(DATE_NOW(), 30, 'day'),
 to: DATE_NOW(),
 granularity: 'day'
})
RETURN {
 date: stat.date,
 requests: stat.total_requests,
 cache_hits: stat.cache_hits,
 hit_rate: stat.cache_hits / stat.total_requests,
 cost_saved_usd: stat.tokens_saved * 0.00003, // GPT-4 pricing
 avg_latency_ms: stat.avg_latency
}
```

**17.12.3 Multi-GPU Support**

**Neu in v1.4.0-alpha:** Verteilte LLM-Inferenz über mehrere GPUs für maximale Performance und Skalierbarkeit.

**Unterstützte Parallelisierungsstrategien:**

1. **Tensor Parallelism** - Große Modelle über GPUs verteilen
2. **Pipeline Parallelism** - Modell-Layer auf verschiedene GPUs
3. **Data Parallelism** - Mehrere Anfragen parallel verarbeiten

```
// Multi-GPU Konfiguration in AQL Query
FOR batch IN RANGE(0, 9)
 LET start_idx = batch * 100
 LET end_idx = (batch + 1) * 100

 LET summaries = (
 FOR doc IN documents
 FILTER doc.id >= start_idx AND doc.id < end_idx
 RETURN PROMPT('llama-70b-local',
 CONCAT('Summarize: ', doc.content),
 {
 max_tokens: 150,
 gpu_config: {
 num_gpus: 4, // 4 GPUs verwenden
 strategy: 'tensor_parallel', // Tensor Parallelism
 gpu_ids: [0, 1, 2, 3] // Spezifische GPU-IDs
 }
 }
)
)

 RETURN {batch: batch, summaries: summaries}
```

### GPU-Scheduling:

```
// themis.conf - Multi-GPU Konfiguration
llm:
 local_models:
 llama-70b:
 model_path: '/models/llama-70b'
 gpu_config:
 num_gpus: 4
 tensor_parallel_size: 4
 pipeline_parallel_size: 1
 max_num_batched_tokens: 8192
 gpu_memory_utilization: 0.9
```

### Performance-Monitoring:

```
// GPU-Auslastung in Echtzeit überwachen
RETURN LLM_GPU_STATS()
```

Output:

```
{
 "gpus": [
 {
 "id": 0,
 "model": "NVIDIA A100",
 "memory_used_gb": 38.2,
 "memory_total_gb": 40.0,
 "utilization_percent": 95,
 "temperature_celsius": 68,
 "power_watts": 320
 },
 // GPU 1-3...
],
 "throughput_tokens_per_sec": 1250,
 "active_requests": 12,
 "queue_length": 3
}
```

### Skalierungs-Benchmarks:

GPUs	Tokens/Sek	Latenz (p50)	Latenz (p99)	Cost/1M Tokens
1x A100	320	1.2s	2.8s	\$0 (lokal)
2x A100	580	0.7s	1.6s	\$0 (lokal)
4x A100	1050	0.4s	0.9s	\$0 (lokal)
8x A100	1850	0.25s	0.6s	\$0 (lokal)

### 17.12.4 Paged Attention

**Neu in v1.4.0-alpha:** Effiziente GPU-Speicherverwaltung für Attention-Mechanismen mit bis zu 80% weniger Speicherverbrauch.

#### Problem ohne Paged Attention:

Traditionelle Attention-Implementierungen allokalieren kontinuierlichen Speicher für KV-Caches, was zu Fragmentierung und ineffizienter Nutzung führt.

#### Lösung mit Paged Attention:

```

graph LR
 A[Request 1
512 tokens] -->|Pages| PA[Paged Attention]
 B[Request 2
1024 tokens] -->|Pages| PA
 C[Request 3
256 tokens] -->|Pages| PA

 PA -->|Page 1-4| GPU1[GPU Memory
Block 1]
 PA -->|Page 5-12| GPU2[GPU Memory
Block 2]
 PA -->|Page 13-16| GPU3[GPU Memory
Block 3]

 style PA fill:#4facfe
 style GPU1 fill:#43e97b
 style GPU2 fill:#43e97b
 style GPU3 fill:#43e97b

```

**Abb. 17.5:** Context-Window-Management**Aktivierung:**

```

// Paged Attention ist standardmäßig aktiviert in v1.4.0-alpha
FOR doc IN large_documents
 LIMIT 1000
 LET analysis = PROMPT('llama-70b-local',
 {
 system: 'Analysiere das folgende Dokument detailliert.',
 user: doc.content // Auch für sehr lange Dokumente effizient
 },
 {
 max_tokens: 2000,
 paged_attention: {
 enabled: true, // Default: true
 page_size: 16, // Tokens pro Page (empfohlen: 16)
 max_num_pages: 4096 // Maximale Pages im Cache
 }
 }
)
 RETURN analysis

```

**Speicher-Effizienz:**



```
// Vergleich: Mit vs. ohne Paged Attention
RETURN {
 without_paging: {
 memory_per_request_mb: 450,
 max_concurrent_requests: 89,
 gpu_memory_utilization: 0.99,
 memory_waste_percent: 35
 },
 with_paging: {
 memory_per_request_mb: 90,
 max_concurrent_requests: 445,
 gpu_memory_utilization: 0.99,
 memory_waste_percent: 8
 },
 improvement: {
 memory_reduction: '80%',
 concurrency_increase: '5x',
 waste_reduction: '77%'
 }
}
```

#### Performance-Charakteristiken:

- **Speicher-Overhead:** ~5% zusätzlicher Overhead für Page-Management
- **Latenz-Impact:** <2% zusätzliche Latenz bei gleichem Durchsatz
- **Skalierbarkeit:** 4-5x mehr gleichzeitige Requests möglich

### 17.12.5 LoRA (Low-Rank Adaptation) Support

**Neu in v1.4.0-alpha:** Effizientes Fine-Tuning und Deployment von spezialisierten Modell-Adaptoren mit minimalem Speicher-Overhead.

#### Konzept:

LoRA fügt trainierbare Low-Rank-Matrizen zu vortrainierten Modellen hinzu, statt das gesamte Modell neu zu trainieren. Dies reduziert: - **Speicher:** 99% weniger Speicher für Fine-Tuned Models - **Training-Zeit:** 3-10x schneller als Full Fine-Tuning - **Deployment:** Mehrere Adapter auf einem Basis-Modell

#### LoRA-Adapter Management:

```

// LoRA-Adapter registrieren
CALL LLM_REGISTER_LORA({
 name: 'medical-assistant',
 base_model: 'llama-70b-local',
 adapter_path: '/models/lora/medical-assistant',
 description: 'Spezialisiert auf medizinische Beratung',
 rank: 16, // LoRA Rank (niedrig = kleiner, hoch = expressiver)
 alpha: 32,
 target_modules: ['q_proj', 'v_proj']
})

// Adapter verwenden
FOR patient_inquiry IN medical_questions
 LET advice = PROMPT('llama-70b-local',
 {
 system: 'Du bist ein medizinischer Assistent. Gebe keine Diagnosen.',
 user: patient_inquiry.question
 },
 {
 lora_adapter: 'medical-assistant', // LoRA-Adapter aktivieren
 temperature: 0.3
 }
)
 RETURN {question: patient_inquiry.question, advice: advice}

```

### Multi-LoRA Support:

```

// Verschiedene Adapter für verschiedene Use Cases
LET adapters = {
 'legal': 'legal-document-assistant',
 'medical': 'medical-assistant',
 'code': 'code-generation-assistant',
 'finance': 'financial-analyst'
}

FOR doc IN documents
 LET domain = doc.domain
 LET adapter = adapters[domain]

 LET analysis = adapter != null ?
 PROMPT('llama-70b-local',
 CONCAT('Analyze: ', doc.content),
 {lora_adapter: adapter, temperature: 0.2}
) : null

 RETURN {domain: domain, analysis: analysis}

```

**LoRA-Adapter Training:**

```
// Training-Job starten (vereinfachtes Beispiel)
CALL LLM_TRAIN_LORA({
 job_name: 'customer-support-v2',
 base_model: 'llama-70b-local',
 training_data: 'customer_support_conversations', // Collection
 validation_split: 0.1,
 hyperparameters: {
 rank: 16,
 alpha: 32,
 learning_rate: 3e-4,
 epochs: 3,
 batch_size: 8,
 warmup_steps: 100
 },
 output_path: '/models/lora/customer-support-v2'
})
```

**Adapter-Vergleich:**

```
// A/B-Testing verschiedener Adapter
FOR question IN test_questions
 LET response_base = PROMPT('llama-70b-local', question.text, {temperature: 0.3})
 LET response_v1 = PROMPT('llama-70b-local', question.text, {
 lora_adapter: 'customer-support-v1',
 temperature: 0.3
 })
 LET response_v2 = PROMPT('llama-70b-local', question.text, {
 lora_adapter: 'customer-support-v2',
 temperature: 0.3
 })

 RETURN {
 question: question.text,
 base: response_base,
 v1: response_v1,
 v2: response_v2
 }
```

**Adapter-Statistiken:**

```
RETURN LLM_LORA_STATS()
```

Output:

```
{
 "registered_adapters": 12,
 "active_adapters": {
 "medical-assistant": {
 "requests_total": 15420,
 "avg_latency_ms": 245,
 "size_mb": 45,
 "rank": 16,
 "base_model": "llama-70b-local"
 }
 // weitere Adapter...
 },
 "memory_overhead_mb": 540, // 12 Adapter zusammen
 "base_model_size_gb": 65
}
```

### 17.12.6 Grammar-Constrained Generation

**Neu in v1.4.0-alpha:** EBNF/GBNF-basierte Grammar-Constraints garantieren gültige, strukturierte LLM-Ausgaben (JSON, XML, CSV) ohne Post-Processing.

#### Problem ohne Grammar Constraints:

LLMs erzeugen oft ungültige Outputs, selbst bei expliziten Anweisungen:

```
// ❌ Ohne Grammar: 60-70% Erfolgsrate
LET response = PROMPT('llama-70b-local',
 'Gebe eine JSON-Liste mit 3 Produkten zurück: {name, price, stock}')

// Typische Fehler:
// - Trailing commas: {"name": "Laptop", "price": 999,}
// - Fehlende Quotes: {name: Laptop}
// - Zusätzlicher Text: "Here is the JSON: {...}"
// - Inkompletter Output: {"name": "Laptop" [abgebrochen]}
```

**Lösung: Grammar-Constrained Generation (95-99% Erfolgsrate):**

```

graph LR
 subgraph "Traditioneller Ansatz"
 P1[Prompt:
'Return JSON'] --> L1[LLM
Generation]
 L1 --> O1[Output:
invalid JSON
60-70 percent valid]
 O1 --> V1[Validate]
 V1 -->|Invalid| R[Retry with
error message]
 R --> L1
 V1 -->|Valid| R1[Use Data]
 end

 subgraph "Grammar-Constrained"
 P2[Prompt +
Grammar Rules] --> L2[LLM
with Grammar]
 L2 --> O2[Output:
guaranteed valid
95-99 percent valid]
 O2 --> R2[Use Data]
 end

 style O1 fill:#f78ca0
 style O2 fill:#43e97b
 style V1 fill:#ffd32a
 style R2 fill:#95e1d3

```

**Abb. 17.6:** Token-Optimization-Strategy

**Diagramm-Erklärung:** - **Traditionell:** LLM generiert frei → Validierung → bei Fehler Retry (teuer, langsam) - **Grammar-Constrained:** Grammar-Regeln garantieren gültigen Output → kein Retry nötig - **Effizienz:** 95-99% Erfolgsrate, kein Post-Processing, niedrigere Kosten

**Verwendung in AQL:**

```
//  Mit Grammar: 95-99% Erfolgsrate, kein Retry nötig
FOR product IN products_to_export
 LIMIT 100

 LET json_data = PROMPT('llama-70b-local',
 CONCAT('Erstelle JSON für Produkt: ', product.name),
 {
 temperature: 0.3,
 grammar_type: 'json', // Built-in Grammar
 response_schema: {
 name: 'string',
 description: 'string',
 price: 'number',
 stock: 'integer',
 categories: 'array'
 }
 }
)

 // json_data ist GARANTIERT valides JSON!
 INSERT json_data INTO exports
```

### Built-in Grammars:

Grammar Type	Use Case	Beispiel-Output
<code>json</code>	API-Responses, Strukturierte Daten	<code>{"key": "value", "arr": [1,2,3]}</code>
<code>json_strict</code>	Strenge JSON-Compliance	Keine trailing commas, quotes required
<code>xml</code>	Legacy-Systeme, SOAP APIs	<code>&lt;root&gt;&lt;item&gt;value&lt;/item&gt;&lt;/root&gt;</code>
<code>csv</code>	Daten-Export, Excel- Integration	<code>name,price,stock\nLaptop,999,15</code>
<code>react_agent</code>	Multi-Step Reasoning	<code>Thought: ...\nAction: ...\nObservation: ...</code>

### JSON Grammar mit Schema:

```

// Produkt-Export mit garantiert gültigem JSON-Schema
FOR order IN orders
 FILTER order.exported == false
 LIMIT 50

 LET export_data = PROMPT('gpt-4',
 {
 system: 'Du bist ein Daten-Export-Assistent. Konvertiere Bestellungen zu JSON.',
 user: CONCAT('Bestellung: ', TO_STRING(order))
 },
 {
 grammar_type: 'json_strict',
 response_schema: {
 order_id: 'string',
 customer: {
 name: 'string',
 email: 'string',
 address: {
 street: 'string',
 city: 'string',
 zip: 'string'
 }
 },
 items: [{
 product_id: 'string',
 name: 'string',
 quantity: 'integer',
 price: 'number'
 }],
 total: 'number',
 status: 'enum:pending,shipped,delivered'
 }
 }
)

 // Garantiert valides, schemakonforme JSON - kein try/catch nötig!
 INSERT export_data INTO order_exports
 UPDATE order WITH {exported: true} IN orders

```

### XML Grammar für Legacy-Integration:

```
// SOAP-API Integration mit garantiert gültigem XML
FOR invoice IN pending_invoices
 LIMIT 20

 LET xml_payload = PROMPT('gpt-4',
 CONCAT('Konvertiere Rechnung zu XML: ', TO_STRING(invoice)),
 {
 grammar_type: 'xml',
 temperature: 0.1
 }
)

 // xml_payload ist garantiert well-formed XML
 LET soap_response = HTTP_POST('https://erp.example.com/soap', {
 body: CONCAT(
 '<soap:Envelope>',
 '<soap:Body>', xml_payload, '</soap:Body>',
 '</soap:Envelope>'
),
 headers: {'Content-Type': 'text/xml'}
 })

 UPDATE invoice WITH {
 erp_synced: true,
 erp_response: soap_response
 } IN pending_invoices
```

### CSV Grammar für Daten-Export:

```
// Batch-Export zu CSV mit Grammar-Garantie
LET csv_export = PROMPT('gpt-4',
 {
 system: '''Konvertiere Produktdaten zu CSV.
 Spalten: ProductID, Name, Category, Price, Stock
 Keine zusätzlichen Kommentare, nur CSV.'''
 user: CONCAT('Produkte: ', TO_STRING(SLICE(products, 0, 100)))
 },
 {
 grammar_type: 'csv',
 temperature: 0.0
 }
)

// csv_export ist garantiert gültiges CSV - direkt speicherbar
LET file_written = FILE_WRITE('/exports/products.csv', csv_export)
RETURN {written: file_written, rows: COUNT_LINES(csv_export)}
```



**ReAct Agent Grammar für Multi-Step Reasoning:**

```

// Agent-basierte Problemlösung mit strukturiertem Output
FOR task IN complex_tasks
 FILTER task.status == 'pending'
 LIMIT 5

 LET agent_steps = PROMPT('gpt-4',
 {
 system: '''Du bist ein Problem-Solving Agent.
 Verwende ReAct Format:
 Thought: [dein Gedankengang]
 Action: [Aktion zum Ausführen]
 Observation: [Ergebnis der Aktion]
 ... (wiederholen bis Lösung)
 Final Answer: [endgültige Antwort]''',
 user: CONCAT('Problem: ', task.description)
 },
 {
 grammar_type: 'react_agent',
 max_tokens: 1000
 }
)

 // agent_steps ist strukturiert nach ReAct-Format
 // → Einfach zu parsen und auszuführen
 LET parsed = PARSE_REACT_FORMAT(agent_steps)

 UPDATE task WITH {
 solution_steps: parsed.steps,
 final_answer: parsed.final_answer,
 status: 'solved'
 } IN complex_tasks

```

**Custom Grammars (EBNF):**

```
// Eigene Grammar-Definition für spezielle Formate
LET custom_grammar = '''
root ::= person+
person ::= name age email
name ::= "Name:" [a-zA-Z]+ "\\n"
age ::= "Age:" [0-9]+ "\\n"
email ::= "Email:" [a-z0-9@.]+ "\\n"
'''

FOR contact IN contact_queue
 LET formatted = PROMPT('gpt-4',
 CONCAT('Formatiere Kontakt: ', TO_STRING(contact)),
 {
 grammar_ebnf: custom_grammar,
 temperature: 0.2
 }
)

 // Output garantiert im definierten Format:
 // Name: Max Mustermann
 // Age: 35
 // Email: max@example.com
 INSERT {text: formatted} INTO formatted_contacts
```

### Grammar Cache für Performance:

Grammars werden automatisch gecacht (LRU Cache, 100 Entries):

```
// Statistiken abfragen
RETURN GRAMMAR_CACHE_STATS()
```

Output:

```
{
 "cache_size": 100,
 "cache_hits": 15420,
 "cache_misses": 234,
 "hit_rate": 0.985,
 "builtin_grammars": ["json", "json_strict", "xml", "csv", "react_agent"],
 "custom_grammars_loaded": 15
}
```

### Performance-Vergleich:

```
// Benchmark: Mit vs. Ohne Grammar
RETURN LLM_GRAMMAR_BENCHMARK({
 prompt: 'Generate JSON product list',
 iterations: 100
})
```

Output:

```
{
 "without_grammar": {
 "avg_latency_ms": 890,
 "success_rate": 0.68,
 "retries_needed": 32,
 "total_tokens": 125000,
 "cost_usd": 3.75
 },
 "with_grammar": {
 "avg_latency_ms": 950,
 "success_rate": 0.98,
 "retries_needed": 2,
 "total_tokens": 102000,
 "cost_usd": 3.06
 },
 "savings": {
 "cost_reduction_percent": 18.4,
 "time_saved_percent": -6.7, // Initial 7% slower...
 "reliability_improvement": "+44%",
 "effective_time_saved": "+65%" // ...aber massiv weniger Retries!
 }
}
```

**Erklärung:** - Grammar fügt ~7% Overhead hinzu - ABER: 44% höhere Erfolgsrate → 94% weniger Retries - Effektiv: 65% schneller + 18% günstiger

#### Best Practices:

1.  **Verwende Built-in Grammars** wenn möglich (optimiert, gecacht)
2.  **Definiere klare Schemas** für `json` Grammar
3.  **Temperature niedrig halten** (0.0-0.3) für deterministische Outputs
4.  **Grammar für Batch-Jobs** - Amortisiert Overhead über viele Requests
5.  **Nicht für kreative Texte** - Grammar schränkt Flexibilität ein
6.  **Nicht für Chat/Dialog** - Zu restriktiv für natürliche Konversation

**Use Cases - Perfekt für:**

-  API-Integration (JSON/XML-Generierung)
-  Daten-Export (CSV, strukturierte Formate)
-  Code-Generierung (Syntax-korrekt)
-  Formular-Parsing (strukturierte Extraktion)
-  Agent-Frameworks (ReAct, Tool-Calling)

**Use Cases - Nicht geeignet für:**

-  Kreatives Schreiben
-  Chat/Dialog
-  Offene Fragen
-  Brainstorming

**Weitere Ressourcen:**

- llama.cpp GBNF Spezifikation: [GitHub](#)
- Built-in Grammars: `src/llm/grammars/*.gbnf`
- Custom Grammar API: `docs/en/llm/GRAMMAR_CONSTRAINED_GENERATION.md`

## 17.12.7 RoPE Scaling - Extended Context Window

**Neu in v1.4.0-alpha:** RoPE (Rotary Position Embedding) Scaling erweitert das Kontext-Fenster von Standard 4K-8K auf bis zu 32K+ Tokens (8x Increase).

**Problem: Begrenzte Kontext-Länge:**

Standard-LLMs sind auf 4K-8K Tokens limitiert:

```
//  Fehler: Kontext zu lang
LET research_paper = FILE_READ('paper.txt') // 25K Tokens
LET summary = PROMPT('llama-70b-local',
 CONCAT('Fasse zusammen: ', research_paper) // ERROR: Context too long!
)
```

**Lösung: RoPE Scaling:**

```

graph TB
 subgraph "Standard LLM (4K Context)"
 S1[Input:
4K tokens max] --> S2[Processing]
 S2 --> S3[Output]
 S4[Long Document
25K tokens] -.->|Truncate| S1
 end

 subgraph "RoPE Scaled LLM (32K Context)"
 R1[Input:
32K tokens] --> R2[RoPE Scaling
Frequency Adjustment]
 R2 --> R3[Processing
with scaled positions]
 R3 --> R4[Output]
 R5[Long Document
25K tokens] -->|Full Text| R1
 end

 style S4 fill:#f78ca0
 style R5 fill:#43e97b
 style R2 fill:#667eea

```

**Abb. 17.7:** Multi-LLM-Orchestration

**Diagramm-Erklärung:** - **Standard:** Input auf 4K tokens gekürzt → Informationsverlust - **RoPE Scaling:** Positions-Embeddings "gestreckt" → 8x längerer Kontext möglich - **NTK-aware / YaRN:** Intelligente Skalierung erhält Qualität auch bei 32K

**Scaling-Methoden:**

Methode	Kontext	Qualität	Use Case
Linear	4K → 16K (4x)	★★★★ Gut	Einfache Dokumente
NTK-aware	4K → 24K (6x)	★★★★★ Sehr gut	Standard-Anwendungen
YaRN	4K → 32K (8x)	★★★★★★ Exzellent	Research Papers, Code

**Konfiguration:**

```

llm:
 models:
 - name: llama-70b-extended
 path: ./models/llama-70b.gguf
 rope_scaling:
 type: yarn # linear, ntk_aware, yarn
 factor: 8 # 4K → 32K
 freq_base: 10000.0
 freq_scale: 1.0
 n_ctx: 32768 # 32K context window

```

### Verwendung in AQL:

```

//  Mit RoPE Scaling: Vollständige Research Paper Analyse
FOR paper IN research_papers
 FILTER paper.analyzed == false
 LIMIT 10

 // Paper hat 25K tokens - passt in 32K Context!
 LET full_text = FILE_READ(paper.file_path)

 LET analysis = PROMPT('llama-70b-extended',
 {
 system: '''Du bist ein wissenschaftlicher Assistent.
 Analysiere das Paper gründlich und extrahiere:
 1. Kernaussagen und Hypothesen
 2. Methodologie
 3. Ergebnisse und Erkenntnisse
 4. Limitationen
 5. Relevanz für unser Forschungsgebiet''',
 user: CONCAT('Research Paper (vollständig):\n\n', full_text)
 },
 {
 temperature: 0.3,
 max_tokens: 2000,
 rope_scaling_type: 'yarn', // Optional: Per-Request Override
 n_ctx: 32768
 }
)

 UPDATE paper WITH {
 analysis: analysis,
 analyzed: true,
 tokens_used: TOKEN_COUNT(full_text)
 } IN research_papers

```

**Codebase-Verständnis:**

```
// Vollständige Codebase-Analyse (große Repositories)
LET codebase_files = (
 FOR file IN GLOB('src/**/*.cpp')
 RETURN {
 path: file,
 content: FILE_READ(file)
 }
)

// Alle Files concatenieren
LET full_codebase = (
 FOR file IN codebase_files
 RETURN CONCAT(
 '// File: ', file.path, '\n',
 file.content, '\n\n'
)
)

LET concatenated = CONCAT_ARRAY(full_codebase)

// Analyse mit extended context (20K tokens codebase)
LET code_analysis = PROMPT('gpt-4-32k',
 {
 system: 'Du bist ein Code-Reviewer. Analysiere die Codebase ganzheitlich.',
 user: CONCAT(
 'Komplette Codebase:\n\n', concatenated, '\n\n',
 'Identifiziere: Architektur-Patterns, potentielle Bugs, ',
 'Performance-Probleme, Security-Issues.'
)
 },
 {
 n_ctx: 32768,
 rope_scaling_type: 'yarn'
 }
)

RETURN {
 total_files: LENGTH(codebase_files),
 total_tokens: TOKEN_COUNT(concatenated),
 analysis: code_analysis
}
```

**Long-Form Content Generation:**

```

// Buch-Kapitel mit vollem Kontext vorheriger Kapitel
FOR chapter IN book_chapters
 FILTER chapter.generated == false
 SORT chapter.number ASC

// Alle vorherigen Kapitel als Kontext
LET previous_chapters = (
 FOR prev IN book_chapters
 FILTER prev.number < chapter.number
 SORT prev.number ASC
 RETURN prev.content
)

LET context = CONCAT_ARRAY(previous_chapters) // Kann 20K+ tokens sein

LET new_chapter = PROMPT('gpt-4-32k',
 {
 system: '''Du bist ein Buchautor. Schreibe das nächste Kapitel
 mit Konsistenz zu allen vorherigen Kapiteln.''' ,
 user: CONCAT(
 'Bisherige Kapitel (vollständig):\n\n', context, '\n\n',
 'Schreibe jetzt Kapitel ', chapter.number, ': ', chapter.title
)
 },
 {
 temperature: 0.8,
 max_tokens: 4000,
 n_ctx: 32768
 }
)

UPDATE chapter WITH {
 content: new_chapter,
 generated: true
} IN book_chapters

```

### Extended Conversations:



```

// Chat mit vollem Konversations-Verlauf (100+ Messages)
FOR session IN chat_sessions
 FILTER session.needs_response == true

 // Vollständige Chat-History laden
 LET messages = (
 FOR msg IN chat_messages
 FILTER msg.session_id == session._key
 SORT msg.timestamp ASC
 RETURN CONCAT(msg.role, ': ', msg.content)
)

 LET full_history = CONCAT_ARRAY(messages, '\n') // 15K tokens

 LET response = PROMPT('gpt-4-32k',
 {
 system: 'Du bist ein hilfreicher Assistent. Beziehe dich auf die gesamte Konversation.',
 user: CONCAT(
 'Konversationsverlauf:\n', full_history, '\n\n',
 'Nutzer: ', session.latest_message
)
 },
 {
 temperature: 0.7,
 n_ctx: 32768
 }
)

 INSERT {
 session_id: session._key,
 role: 'assistant',
 content: response,
 timestamp: DATE_NOW()
 } INTO chat_messages

 UPDATE session WITH {needs_response: false} IN chat_sessions

```

### Performance & Qualität:

```

// Benchmark: Context Length vs. Qualität
RETURN ROPE_SCALING_BENCHMARK({
 model: 'llama-70b',
 test_contexts: [4096, 8192, 16384, 32768],
 test_iterations: 50
})

```

Output:

```
{
 "4K_baseline": {
 "throughput_tokens_per_sec": 45,
 "quality_score": 0.92,
 "memory_gb": 65
 },
 "16K_ntk_aware": {
 "throughput_tokens_per_sec": 34,
 "quality_score": 0.89,
 "memory_gb": 80,
 "quality_loss_percent": 3.3
 },
 "32K_yarn": {
 "throughput_tokens_per_sec": 22,
 "quality_score": 0.88,
 "memory_gb": 95,
 "quality_loss_percent": 4.3
 },
 "recommendations": {
 "for_quality": "Use YaRN up to 32K",
 "for_speed": "Use NTK-aware up to 16K",
 "for_memory": "Stay at baseline 4K or use quantization"
 }
}
```

Trade-offs:

Context	Throughput	Memory	Qualität	Best For
4K	100%	1x	★★★★★★	Standard-Tasks
16K (4x)	75%	1.2x	★★★★★	Längere Docs
32K (8x)	50%	1.5x	★★★★★	Research Papers

Best Practices:

- 1.  **Start mit kleinstem Context** der funktioniert
- 2.  **YaRN für maximale Qualität** bei langen Kontexten
- 3.  **Monitor Quality Loss** mit Benchmarks
- 4.  **Quantization kombinieren** (Q4/Q5) für Speicher-Effizienz
- 5.  **Batch kurze Dokumente** statt einzeln lange Kontext

- 6. ❌ **Nicht blind auf 32K** - nur wenn wirklich nötig
- 7. ❌ **Nicht für Streaming** - Latenz zu hoch

**Use Cases - Perfekt für:**

- ✅ Research Paper Analyse (20-30K tokens)
- ✅ Codebase Review (vollständiger Code-Kontext)
- ✅ Long-Form Content (Bücher, Reports)
- ✅ Extended Conversations (100+ Messages)
- ✅ Legal Document Review (Verträge, Gesetzestexte)

**Use Cases - Overkill für:**

- ❌ Chat (Standard 4K reicht)
- ❌ Kurze Queries (<1K tokens)
- ❌ Real-time Anwendungen (zu langsam)

**Weitere Ressourcen:**

- YaRN Paper: [arXiv:2309.00071](https://arxiv.org/abs/2309.00071)
- RoPE Scaling Guide: `docs/en/llm/ROPE_SCALING_IMPLEMENTATION.md`
- Configuration: `config/llm.yaml`

---

## 17.12.8 Vision Support

**Neu in v1.4.0-alpha:** Multimodale LLM-Integration für Text + Bild-Verarbeitung, ermöglicht visuelle Analyse, OCR und Bildbeschreibung direkt in AQL.

**Unterstützte Modelle:**

- GPT-4 Vision (OpenAI)
- Claude 3 (Anthropic)
- LLaVA (lokal)
- CogVLM (lokal)

**Bild-Analyse in AQL:**

```
// Produktbilder analysieren
FOR product IN products
 FILTER product.image_analysis == null
 LIMIT 100

 LET analysis = PROMPT_VISION('gpt-4-vision',
 {
 image: product.image_url, // URL oder base64
 prompt: '''Analysiere dieses Produktbild und extrahiere:
 1. Produkttyp und Kategorie
 2. Sichtbare Features und Merkmale
 3. Farben und Materialien
 4. Zustand (neu/gebraucht)
 5. Qualitätsbewertung (1-10)'''
 },
 {
 temperature: 0.3,
 max_tokens: 500,
 response_format: 'json'
 }
)

 UPDATE product WITH {
 image_analysis: analysis,
 analyzed_at: DATE_NOW()
 } IN products
```

**OCR und Dokumenten-Extraktion:**

```

// Rechnungen per OCR verarbeiten
FOR invoice IN scanned_invoices
 FILTER invoice.extracted == false

 LET ocr_result = PROMPT_VISION('gpt-4-vision',
 {
 image: invoice.scan_base64,
 prompt: '''Extrahiere folgende Informationen aus dieser Rechnung:
 - Rechnungsnummer
 - Datum
 - Lieferant (Name, Adresse)
 - Positionen (Artikel, Menge, Einzelpreis)
 - Gesamtbetrag
 - Mehrwertsteuer
 Gebe das Ergebnis als strukturiertes JSON zurück.'''
 },
 {
 temperature: 0.1,
 response_format: 'json'
 }
)

 INSERT {
 invoice_id: invoice._key,
 invoice_number: ocr_result.invoice_number,
 date: ocr_result.date,
 supplier: ocr_result.supplier,
 items: ocr_result.items,
 total: ocr_result.total,
 vat: ocr_result.vat,
 extracted_at: DATE_NOW()
 } INTO invoice_data

 UPDATE invoice WITH {extracted: true} IN scanned_invoices

```

### Integration mit Video Processor:

```

// Keyframes aus Videos analysieren (siehe Kapitel 12: Computer Vision)
FOR video IN videos
 FILTER video.keyframe_analysis == null

 // Keyframes extrahieren (aus Video Processor)
 LET keyframes = VIDEO_EXTRACT_KEYFRAMES(video.file_path, {
 max_keyframes: 10,
 min_interval_seconds: 5
 })

 // Jeden Keyframe mit Vision LLM analysieren
 LET frame_analyses = (
 FOR frame IN keyframes
 RETURN PROMPT_VISION('gpt-4-vision',
 {
 image: frame.image_base64,
 prompt: 'Beschreibe was in diesem Video-Frame zu sehen ist. Fokus auf Aktionen,
 },
 {temperature: 0.5, max_tokens: 200}
)
)

 // Gesamtzusammenfassung generieren
 LET summary = PROMPT('gpt-4',
 {
 system: 'Erstelle eine kohärente Video-Zusammenfassung aus Einzelframe-Beschreibung
 user: CONCAT('Frame-Beschreibungen:\n', TO_STRING(frame_analyses))
 }
)

 UPDATE video WITH {
 keyframe_analyses: frame_analyses,
 summary: summary,
 keyframe_analysis: true
 } IN videos

```

### Visual Question Answering:

```
// Fragen zu Bildern beantworten
FOR support_ticket IN tickets
 FILTER support_ticket.has_image AND support_ticket.image_question != null

 LET answer = PROMPT_VISION('claude-3-opus',
 {
 image: support_ticket.image_url,
 prompt: CONCAT(
 'Kundenfrage: ', support_ticket.image_question, '\n\n',
 'Analysiere das Bild und beantworte die Kundenfrage detailliert.'
)
 },
 {temperature: 0.4, max_tokens: 400}
)

 UPDATE support_ticket WITH {
 ai_answer: answer,
 ai_answered_at: DATE_NOW()
 } IN tickets
```

### Batch-Verarbeitung mit Vision:

```
// Effiziente Batch-Verarbeitung großer Bildmengen
FOR batch IN RANGE(0, 49)
 LET start_idx = batch * 20
 LET end_idx = (batch + 1) * 20

 LET results = (
 FOR img IN images
 FILTER img.id >= start_idx AND img.id < end_idx
 FILTER img.moderation_check == null

 RETURN {
 id: img._key,
 moderation: PROMPT_VISION('gpt-4-vision',
 {
 image: img.url,
 prompt: '''Prüfe dieses Bild auf:
 - Inappropriate content
 - Violence
 - Hate symbols
 - NSFW content
 Gebe JSON mit: {safe: boolean, flags: [], confidence: number}'''
 },
 {temperature: 0.1, response_format: 'json'}
)
 }
)

// Batch-Update
FOR result IN results
 UPDATE {_key: result.id} WITH {
 moderation_check: result.moderation,
 checked_at: DATE_NOW()
 } IN images
```

### Performance-Überlegungen:

```
// Vision API Kosten-Tracking
RETURN LLM_VISION_STATS({
 from: DATE_SUBTRACT(DATE_NOW(), 7, 'day'),
 to: DATE_NOW()
})
```

Output:



```
{
 "total_requests": 15420,
 "images_processed": 15420,
 "avg_latency_ms": 1850,
 "cost_breakdown": {
 "gpt-4-vision": {
 "requests": 12000,
 "cost_usd": 360.00,
 "avg_cost_per_image": 0.03
 },
 "claude-3-opus": {
 "requests": 3420,
 "cost_usd": 205.20,
 "avg_cost_per_image": 0.06
 }
 },
 "use_cases": {
 "product_analysis": 8500,
 "ocr": 4200,
 "moderation": 2720
 }
}
```

## 17.13 Best Practices Zusammenfassung

### 17.13.1 DO

1. **Verwende @parameter binding** für alle Benutzereingaben
2. **Cache häufige Anfragen** um Kosten zu sparen
3. **Validiere LLM-Outputs** vor der Speicherung
4. **Batch-Verarbeitung** für große Datenmengen
5. **Monitor Kosten** und Performance kontinuierlich
6. **Sanitize Inputs** vor LLM-Calls
7. **Verwende strukturierte Outputs** (JSON) wenn möglich
8. **Implementiere Fallbacks** bei LLM-Fehlern

### 17.13.2 DON'T

1. **Keine sensiblen Daten** ungefiltert an LLMs senden
2. **Keine unvalidierten LLM-Queries** ausführen
3. **Keine unbegrenzten LLM-Calls** ohne Rate-Limiting
4. **Keine Hardcoded API-Keys** im Code
5. **Keine synchronen LLM-Calls** für zeitkritische Operationen

---

## 6. Keine Abhängigkeit von einem einzelnen Provider

### Zusammenfassung

ThemisDB's LLM-Integration ermöglicht:

- **Native AQL-Funktionen** für Text-Generierung, Embeddings und strukturierte Ausgaben
- **Text-to-AQL** für natürlichsprachliche Query-Erstellung
- **RAG Patterns** mit semantischer Suche und Kontext-Anreicherung
- **Multi-Model Synergien** mit Graph, Temporal und Vector Search
- **Kosten-Optimierung** durch intelligentes Caching und Model-Selection
- **Enterprise-Grade Sicherheit** mit Input-Sanitization und PII-Schutz

Die Integration von LLMs direkt in die Datenbankebene reduziert Latenz, vereinfacht Architektur und ermöglicht völlig neue Anwendungsfälle von intelligenter Datenanalyse bis zu automatisierter Content-Generierung.

---

**Nächstes Kapitel:** [Kapitel 18: Machine Learning Integration](#) **Vorheriges Kapitel:** [Kapitel 16: Machine Learning](#)

---

---

# Kapitel 23: Hochverfügbarkeit

---

## Übersicht

Dieses Kapitel behandelt Hochverfügbarkeitskonfigurationen für ThemisDB.

### Hauptthemen

- Replikation
- Failover-Mechanismen
- Health Checks
- Disaster Recovery

## Weitere Informationen

Siehe auch: [Horizontal Scaling](#), [Monitoring](#)

---

# Kapitel 24: Kapitel 18: Machine Learning Integration

---

## Überblick

ThemisDB bietet umfassende Integration mit Machine-Learning-Frameworks und -Workflows. Als Multi-Model-Datenbank mit nativem Vector-Support eignet sich ThemisDB ideal als:

- **ML Feature Store** - Speicherung und Verwaltung von Features für Training und Inference
- **Model Serving Platform** - Online und Batch Predictions
- **Experiment Tracking** - Versionierung von Models, Metriken und Hyperparametern
- **Training Data Management** - Effiziente Speicherung großer Datasets

Die Kombination aus relationalen Daten, Graphen, Dokumenten und Vektoren macht ThemisDB zur idealen Plattform für ML-Pipelines.

```

graph TB
 subgraph "ML Pipeline with ThemisDB"
 Raw[Raw Data
Logs, Events, Transactions]

 Raw --> FS[Feature Store
Relational Tables]

 FS --> Train[Training Pipeline]
 FS --> Inference[Inference Pipeline]

 Train --> ModelReg[(Model Registry
Document Store)]
 Train --> ExpTrack[(Experiment Tracking
Metrics & Params)]

 ModelReg --> Deploy[Model Deployment]

 Deploy --> Serve[Model Serving
Online Predictions]

 Inference --> Vec[(Vector Index
Embeddings)]
 Vec --> SimSearch[Similarity Search
Recommendations]

 Serve --> Results[Prediction Results
Back to DB]
 SimSearch --> Results

 Results --> Monitor[Monitoring
Drift Detection]
 Monitor -.feedback.-> Train
 end

 style Raw fill:#667eea
 style FS fill:#4facfe
 style Train fill:#43e97b
 style ModelReg fill:#f093fb
 style Vec fill:#ffd32a
 style Results fill:#95e1d3

```

Abb. 18.1: ML-Pipeline-Architektur

## 18.1 ML Feature Store

### Feature Engineering mit ThemisDB

#### Feature-Definition:

```

from themisdb import ThemisDB
import pandas as pd
from datetime import datetime, timedelta

db = ThemisDB(host='localhost', port=8529)

db.execute("""
 CREATE TABLE ml_features (
 feature_id STRING PRIMARY KEY,
 entity_id STRING,
 entity_type STRING,
 feature_name STRING,
 feature_value VARIANT,
 feature_type STRING,
 computed_at TIMESTAMP,
 version INTEGER,

 INDEX idx_entity (entity_id, entity_type),
 INDEX idx_feature (feature_name, computed_at)
)
""")

def compute_customer_features(customer_id):
 features = {}

 # Transaktionshistorie (relational)
 result = db.execute("""
 FOR order IN orders
 FILTER order.customer_id == @customer_id
 COLLECT AGGREGATE
 total_orders = COUNT(1),
 total_spent = SUM(order.total_amount),
 avg_order_value = AVG(order.total_amount),
 days_since_last_order = DATE_DIFF(
 MAX(order.order_date),
 DATE_NOW(),
 'day'
)
 RETURN {
 total_orders,
 total_spent,
 avg_order_value,
 days_since_last_order
 }
 """, bind_vars={'customer_id': customer_id})

 features.update(result[0])

 # Social Graph Features

```

```

result = db.execute("""
 FOR v, e, p IN 1..2 OUTBOUND @customer_id GRAPH 'social_graph'
 COLLECT AGGREGATE
 friend_count = COUNT(DISTINCT v),
 avg_friend_spending = AVG(v.total_spent)
 RETURN {
 friend_count,
 avg_friend_spending
 }
""", bind_vars={'customer_id': customer_id})

features.update(result[0])

Produktpräferenzen (Vector Similarity)
result = db.execute("""
 LET customer_purchases = (
 FOR order IN orders
 FILTER order.customer_id == @customer_id
 FOR item IN order.items
 RETURN DISTINCT item.product_id
)

 FOR product_id IN customer_purchases
 FOR product IN products
 FILTER product._id == product_id
 RETURN product.category
""", bind_vars={'customer_id': customer_id})

features['preferred_categories'] = list(set(result))

return features

def save_features(entity_id, entity_type, features, version=1):
 for feature_name, feature_value in features.items():
 db.execute("""
 INSERT {
 feature_id: CONCAT(@entity_id, '_', @feature_name, '_', @version),
 entity_id: @entity_id,
 entity_type: @entity_type,
 feature_name: @feature_name,
 feature_value: @feature_value,
 feature_type: @feature_type,
 computed_at: DATE_NOW(),
 version: @version
 } INTO ml_features
 """, bind_vars={
 'entity_id': entity_id,
 'entity_type': entity_type,
 'feature_name': feature_name,
 'feature_value': feature_value,

```

```
 'feature_type': type(feature_value).__name__,
 'version': version
 })

def batch_compute_features(entity_ids):
 for entity_id in entity_ids:
 features = compute_customer_features(entity_id)
 save_features(entity_id, 'customer', features)
```

## Online Feature Store (Low-Latency)

### Feature Retrieval:



```

class FeatureStore:
 def __init__(self, db):
 self.db = db
 self.cache = {} # In-memory cache

 def get_features(self, entity_id, feature_names, use_cache=True):
 """Retrieve features with caching"""
 cache_key = f"{entity_id}:{','.join(sorted(feature_names))}"

 if use_cache and cache_key in self.cache:
 return self.cache[cache_key]

 result = self.db.execute("""
 FOR feature IN ml_features
 FILTER feature.entity_id == @entity_id
 FILTER feature.feature_name IN @feature_names
 SORT feature.version DESC
 COLLECT feature_name = feature.feature_name
 INTO group = feature
 RETURN {
 feature_name: group[0].feature.feature_value
 }
 """, bind_vars={
 'entity_id': entity_id,
 'feature_names': feature_names
 })

 features = {item['feature_name']: item['feature_value']
 for item in result}

 if use_cache:
 self.cache[cache_key] = features

 return features

 def get_feature_vector(self, entity_id, feature_order):
 """Get ordered feature vector for ML model"""
 features = self.get_features(entity_id, feature_order)
 return [features.get(fname, None) for fname in feature_order]

feature_store = FeatureStore(db)

feature_names = ['total_orders', 'total_spent', 'days_since_last_order',
 'friend_count', 'avg_order_value']
features = feature_store.get_feature_vector('customer_12345', feature_names)

```

## **16.2 Model Training Integration**

---

### **Scikit-Learn Integration**

**Daten laden und Model trainieren:**

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np

def load_training_data():
 result = db.execute("""
 FOR customer IN customers
 LET features = (
 FOR feature IN ml_features
 FILTER feature.entity_id == customer._id
 FILTER feature.version == 1
 RETURN {
 [feature.feature_name]: feature.feature_value
 }
)

 LET label = customer.churned ? 1 : 0

 RETURN {
 customer_id: customer._id,
 features: MERGE(features),
 label: label
 }
 """)

 # DataFrame erstellen
 data = []
 for row in result:
 features = row['features']
 features['label'] = row['label']
 data.append(features)

 df = pd.DataFrame(data)
 return df

df = load_training_data()

feature_columns = ['total_orders', 'total_spent', 'avg_order_value',
 'days_since_last_order', 'friend_count']

X = df[feature_columns].fillna(0)
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

feature_importance = pd.DataFrame({
 'feature': feature_columns,
 'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance)
```

## TensorFlow/Keras Integration

### Neural Network Training:

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

class ThemisDataset(tf.data.Dataset):
 def _generator(customer_ids, feature_store, feature_names):
 for customer_id in customer_ids:
 features = feature_store.get_feature_vector(customer_id, feature_names)

 # Label laden
 result = db.execute("""
 FOR customer IN customers
 FILTER customer._id == @customer_id
 RETURN customer.churned ? 1 : 0
 """, bind_vars={'customer_id': customer_id})

 label = result[0] if result else 0

 yield (tf.constant(features, dtype=tf.float32),
 tf.constant(label, dtype=tf.int32))

 def __new__(cls, customer_ids, feature_store, feature_names):
 return tf.data.Dataset.from_generator(
 lambda: cls._generator(customer_ids, feature_store, feature_names),
 output_signature=(
 tf.TensorSpec(shape=(len(feature_names),), dtype=tf.float32),
 tf.TensorSpec(shape=(), dtype=tf.int32)
)
)

customer_ids = db.execute("FOR c IN customers RETURN c._id")

dataset = ThemisDataset(customer_ids, feature_store, feature_names)
dataset = dataset.batch(32).prefetch(tf.data.AUTOTUNE)

model = keras.Sequential([
 layers.Dense(64, activation='relu', input_shape=(len(feature_names),)),
 layers.Dropout(0.3),
 layers.Dense(32, activation='relu'),
 layers.Dropout(0.3),
 layers.Dense(16, activation='relu'),
 layers.Dense(1, activation='sigmoid')
])

model.compile(
 optimizer='adam',
 loss='binary_crossentropy',
 metrics=['accuracy', 'AUC']
)

```

```
history = model.fit(
 dataset,
 epochs=10,
 validation_split=0.2
)

model.save('models/churn_prediction_v1.h5')
```

## PyTorch Integration

### Custom Dataset und Training:

```

import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim

class ThemisDataset(Dataset):
 def __init__(self, db, feature_names):
 self.db = db
 self.feature_names = feature_names

 # Alle Customer IDs laden
 self.customer_ids = db.execute("FOR c IN customers RETURN c._id")

 def __len__(self):
 return len(self.customer_ids)

 def __getitem__(self, idx):
 customer_id = self.customer_ids[idx]

 # Features laden
 features = feature_store.get_feature_vector(customer_id, self.feature_names)
 features = [f if f is not None else 0.0 for f in features]

 # Label laden
 result = self.db.execute("""
 FOR customer IN customers
 FILTER customer._id == @customer_id
 RETURN customer.churned ? 1.0 : 0.0
 """, bind_vars={'customer_id': customer_id})

 label = result[0] if result else 0.0

 return torch.tensor(features, dtype=torch.float32), torch.tensor(label, dtype=torch.float32)

class ChurnPredictor(nn.Module):
 def __init__(self, input_size):
 super(ChurnPredictor, self).__init__()
 self.fc1 = nn.Linear(input_size, 64)
 self.fc2 = nn.Linear(64, 32)
 self.fc3 = nn.Linear(32, 16)
 self.fc4 = nn.Linear(16, 1)
 self.relu = nn.ReLU()
 self.dropout = nn.Dropout(0.3)
 self.sigmoid = nn.Sigmoid()

 def forward(self, x):
 x = self.relu(self.fc1(x))
 x = self.dropout(x)
 x = self.relu(self.fc2(x))

```

```

 x = self.dropout(x)
 x = self.relu(self.fc3(x))
 x = self.sigmoid(self.fc4(x))
 return x

dataset = ThemisDataset(db, feature_names)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)

model = ChurnPredictor(len(feature_names))
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
 model.train()
 total_loss = 0

 for features, labels in train_loader:
 optimizer.zero_grad()
 outputs = model(features).squeeze()
 loss = criterion(outputs, labels)
 loss.backward()
 optimizer.step()
 total_loss += loss.item()

 print(f'Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}')

model.eval()
correct = 0
total = 0

with torch.no_grad():
 for features, labels in test_loader:
 outputs = model(features).squeeze()
 predicted = (outputs > 0.5).float()
 total += labels.size(0)
 correct += (predicted == labels).sum().item()

print(f'Accuracy: {100 * correct / total:.2f}%')
```



## 16.3 Model Serving

---

### Online Predictions (REST API)

**FastAPI Model Server:**

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI()

model = joblib.load('models/churn_model.pkl')
feature_store = FeatureStore(db)

class PredictionRequest(BaseModel):
 customer_id: str

class PredictionResponse(BaseModel):
 customer_id: str
 churn_probability: float
 churn_prediction: bool
 features_used: dict

@app.post("/predict/churn", response_model=PredictionResponse)
async def predict_churn(request: PredictionRequest):
 try:
 # Features laden
 feature_names = ['total_orders', 'total_spent', 'avg_order_value',
 'days_since_last_order', 'friend_count']

 features = feature_store.get_features(request.customer_id, feature_names)
 feature_vector = [features.get(fn, 0) for fn in feature_names]

 # Prediction
 churn_prob = model.predict_proba([feature_vector])[0][1]
 churn_pred = churn_prob > 0.5

 # Prediction in DB speichern
 db.execute("""
 INSERT {
 customer_id: @customer_id,
 prediction_type: 'churn',
 probability: @probability,
 prediction: @prediction,
 features: @features,
 model_version: 'v1',
 predicted_at: DATE_NOW()
 } INTO predictions
 """, bind_vars={
 'customer_id': request.customer_id,
 'probability': float(churn_prob),
 'prediction': churn_pred,
 'features': features
 })

```

```
 })

 return PredictionResponse(
 customer_id=request.customer_id,
 churn_probability=float(churn_prob),
 churn_prediction=churn_pred,
 features_used=features
)

except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@app.get("/model/info")
async def model_info():
 return {
 "model_type": "RandomForestClassifier",
 "version": "v1",
 "features": feature_names,
 "trained_at": "2024-01-15"
 }
```

## Batch Predictions

**Batch Inference für alle Kunden:**

```

def batch_predict_churn(model, batch_size=1000):
 # Alle Customer IDs
 customer_ids = db.execute("FOR c IN customers RETURN c._id")

 total = len(customer_ids)
 processed = 0

 for i in range(0, total, batch_size):
 batch_ids = customer_ids[i:i+batch_size]

 # Batch Features laden
 batch_features = []
 for customer_id in batch_ids:
 features = feature_store.get_feature_vector(customer_id, feature_names)
 batch_features.append(features)

 # Batch Prediction
 predictions = model.predict_proba(batch_features)[: , 1]

 # Bulk Insert
 docs = []
 for customer_id, prob in zip(batch_ids, predictions):
 docs.append({
 'customer_id': customer_id,
 'prediction_type': 'churn',
 'probability': float(prob),
 'prediction': prob > 0.5,
 'model_version': 'v1',
 'predicted_at': datetime.now().isoformat()
 })

 db.bulk_insert('predictions', docs)

 processed += len(batch_ids)
 print(f'Processed {processed}/{total} customers')

batch_predict_churn(model)

```

## 16.4 MLOps - Experiment Tracking

### Experiment Management

#### Model Versioning:

```

class MLEperiment:
 def __init__(self, db, experiment_name):
 self.db = db
 self.experiment_name = experiment_name
 self.experiment_id = self._create_experiment()

 def _create_experiment(self):
 result = self.db.execute("""
 INSERT {
 experiment_name: @name,
 created_at: DATE_NOW(),
 status: 'running'
 } INTO ml_experiments
 RETURN NEW._id
 """, bind_vars={'name': self.experiment_name})

 return result[0]

 def log_params(self, params):
 """Log hyperparameters"""
 self.db.execute("""
 UPDATE @experiment_id WITH {
 params: @params
 } IN ml_experiments
 """, bind_vars={
 'experiment_id': self.experiment_id,
 'params': params
 })

 def log_metrics(self, metrics, step=None):
 """Log training metrics"""
 self.db.execute("""
 INSERT {
 experiment_id: @experiment_id,
 metrics: @metrics,
 step: @step,
 logged_at: DATE_NOW()
 } INTO ml_metrics
 """, bind_vars={
 'experiment_id': self.experiment_id,
 'metrics': metrics,
 'step': step
 })

 def log_model(self, model_path, metadata=None):
 """Register trained model"""
 self.db.execute("""
 UPDATE @experiment_id WITH {
 model_path: @model_path,

```

```

 model_metadata: @metadata,
 status: 'completed',
 completed_at: DATE_NOW()
 } IN ml_experiments
"""", bind_vars={
 'experiment_id': self.experiment_id,
 'model_path': model_path,
 'metadata': metadata or {}
})

experiment = MLEperiment(db, 'churn_prediction_rf_v2')

experiment.log_params({
 'n_estimators': 100,
 'max_depth': 10,
 'min_samples_split': 5,
 'random_state': 42
})

for epoch in range(10):
 # ... Training ...

 experiment.log_metrics({
 'train_loss': train_loss,
 'val_loss': val_loss,
 'train_accuracy': train_acc,
 'val_accuracy': val_acc
 }, step=epoch)

experiment.log_model(
 model_path='models/churn_rf_v2.pkl',
 metadata={
 'feature_importance': feature_importance.to_dict(),
 'test_accuracy': 0.87,
 'test_auc': 0.92
 }
)

```

## Model Registry

### Zentrales Model Management:

```

class ModelRegistry:
 def __init__(self, db):
 self.db = db

 def register_model(self, name, version, model_path, metadata):
 """Register a new model version"""
 self.db.execute("""
 INSERT {
 model_name: @name,
 version: @version,
 model_path: @model_path,
 metadata: @metadata,
 status: 'staged',
 registered_at: DATE_NOW()
 } INTO model_registry
 """, bind_vars={
 'name': name,
 'version': version,
 'model_path': model_path,
 'metadata': metadata
 })

 def promote_to_production(self, name, version):
 """Promote model to production"""
 # Alte Production Models auf 'archived' setzen
 self.db.execute("""
 FOR model IN model_registry
 FILTER model.model_name == @name
 FILTER model.status == 'production'
 UPDATE model WITH {
 status: 'archived',
 archived_at: DATE_NOW()
 } IN model_registry
 """, bind_vars={'name': name})

 # Neue Version auf 'production' setzen
 self.db.execute("""
 FOR model IN model_registry
 FILTER model.model_name == @name
 FILTER model.version == @version
 UPDATE model WITH {
 status: 'production',
 promoted_at: DATE_NOW()
 } IN model_registry
 """, bind_vars={'name': name, 'version': version})

 def get_production_model(self, name):
 """Get current production model"""
 result = self.db.execute("""

```

```

 FOR model IN model_registry
 FILTER model.model_name == @name
 FILTER model.status == 'production'
 RETURN model
 """', bind_vars={'name': name}))

 return result[0] if result else None

def list_versions(self, name):
 """List all versions of a model"""
 return self.db.execute("""
 FOR model IN model_registry
 FILTER model.model_name == @name
 SORT model.version DESC
 RETURN {
 version: model.version,
 status: model.status,
 metadata: model.metadata,
 registered_at: model.registered_at
 }
 """, bind_vars={'name': name}))

registry = ModelRegistry(db)

registry.register_model(
 name='churn_predictor',
 version='v2.1',
 model_path='models/churn_rf_v2.1.pkl',
 metadata={
 'algorithm': 'RandomForest',
 'test_accuracy': 0.89,
 'test_auc': 0.94,
 'feature_count': 5
 }
)

registry.promote_to_production('churn_predictor', 'v2.1')

prod_model = registry.get_production_model('churn_predictor')

```

## 16.5 Model Monitoring

### Prediction Drift Detection

#### Model Performance Monitoring:



```

class ModelMonitor:
 def __init__(self, db, model_name):
 self.db = db
 self.model_name = model_name

 def log_prediction(self, prediction_id, features, prediction, actual=None):
 """Log prediction for monitoring"""
 self.db.execute("""
 INSERT {
 prediction_id: @prediction_id,
 model_name: @model_name,
 features: @features,
 prediction: @prediction,
 actual: @actual,
 timestamp: DATE_NOW()
 } INTO prediction_logs
 """, bind_vars={
 'prediction_id': prediction_id,
 'model_name': self.model_name,
 'features': features,
 'prediction': prediction,
 'actual': actual
 })

 def update_actual(self, prediction_id, actual):
 """Update with actual outcome"""
 self.db.execute("""
 UPDATE @prediction_id WITH {
 actual: @actual,
 updated_at: DATE_NOW()
 } IN prediction_logs
 """, bind_vars={
 'prediction_id': prediction_id,
 'actual': actual
 })

 def calculate_metrics(self, time_window_days=7):
 """Calculate recent model performance"""
 result = self.db.execute("""
 LET cutoff_date = DATE_SUBTRACT(DATE_NOW(), @days, 'day')

 FOR log IN prediction_logs
 FILTER log.model_name == @model_name
 FILTER log.timestamp >= cutoff_date
 FILTER log.actual != null

 COLLECT AGGREGATE
 total = COUNT(1),
 correct = SUM(log.prediction == log.actual ? 1 : 0),

```

```

 false_positives = SUM(log.prediction == true AND log.actual == false)
 false_negatives = SUM(log.prediction == false AND log.actual == true)

 LET accuracy = correct / total
 LET precision = correct / (correct + false_positives)
 LET recall = correct / (correct + false_negatives)
 LET f1_score = 2 * (precision * recall) / (precision + recall)

 RETURN {
 accuracy,
 precision,
 recall,
 f1_score,
 total_predictions: total
 }
 """
 bind_vars={
 'model_name': self.model_name,
 'days': time_window_days
 })

 return result[0] if result else None

def detect_feature_drift(self, feature_name, time_window_days=7):
 """Detect drift in feature distribution"""
 result = self.db.execute("""
 LET cutoff_date = DATE_SUBTRACT(DATE_NOW(), @days, 'day')

 LET recent_stats = (
 FOR log IN prediction_logs
 FILTER log.model_name == @model_name
 FILTER log.timestamp >= cutoff_date
 COLLECT AGGREGATE
 mean = AVG(log.features[@feature_name]),
 stddev = STDDEV(log.features[@feature_name]),
 min = MIN(log.features[@feature_name]),
 max = MAX(log.features[@feature_name])
 RETURN {mean, stddev, min, max}
)

 LET baseline_stats = (
 FOR log IN prediction_logs
 FILTER log.model_name == @model_name
 FILTER log.timestamp < cutoff_date
 LIMIT 10000
 COLLECT AGGREGATE
 mean = AVG(log.features[@feature_name]),
 stddev = STDDEV(log.features[@feature_name])
 RETURN {mean, stddev}
)
 """)

```

```

 LET mean_diff = ABS(recent_stats[0].mean - baseline_stats[0].mean)
 LET drift_score = mean_diff / baseline_stats[0].stddev

 RETURN {
 feature_name: @feature_name,
 baseline: baseline_stats[0],
 recent: recent_stats[0],
 drift_score: drift_score,
 drift_detected: drift_score > 2.0
 }
 """
 bind_vars={
 'model_name': self.model_name,
 'feature_name': feature_name,
 'days': time_window_days
 })

 return result[0] if result else None

monitor = ModelMonitor(db, 'churn_predictor')

metrics = monitor.calculate_metrics(time_window_days=7)
print(f"7-Day Accuracy: {metrics['accuracy']:.2%}")
print(f"7-Day F1 Score: {metrics['f1_score']:.2f}")

for feature in feature_names:
 drift = monitor.detect_feature_drift(feature)
 if drift['drift_detected']:
 print(f"⚠️ Drift detected in {feature}: {drift['drift_score']:.2f}")

```

## 16.6 Praktische Anwendungsfälle

### Use Case 1: Fraud Detection

#### Echtzeit-Betrug-Erkennung:

```

def prepare_fraud_training_data():
 result = db.execute("""
 FOR txn IN transactions
 LET user = DOCUMENT('users', txn.user_id)

 LET graph_features = (
 FOR v, e, p IN 1..2 OUTBOUND txn.user_id GRAPH 'user_network'
 COLLECT AGGREGATE
 network_size = COUNT(1),
 fraud_neighbors = SUM(v.is_fraudster ? 1 : 0)
 RETURN {network_size, fraud_neighbors}
)[0]

 LET velocity_features = (
 FOR t IN transactions
 FILTER t.user_id == txn.user_id
 FILTER DATE_DIFF(t.timestamp, txn.timestamp, 'hour') <= 24
 COLLECT AGGREGATE
 txn_count_24h = COUNT(1),
 total_amount_24h = SUM(t.amount)
 RETURN {txn_count_24h, total_amount_24h}
)[0]

 RETURN {
 transaction_id: txn._id,
 amount: txn.amount,
 user_age_days: DATE_DIFF(user.created_at, txn.timestamp, 'day'),
 network_size: graph_features.network_size,
 fraud_neighbors: graph_features.fraud_neighbors,
 txn_count_24h: velocity_features.txn_count_24h,
 total_amount_24h: velocity_features.total_amount_24h,
 is_fraud: txn.is_fraud
 }
 """)

 return pd.DataFrame(result)

import xgboost as xgb

df = prepare_fraud_training_data()

feature_cols = ['amount', 'user_age_days', 'network_size',
 'fraud_neighbors', 'txn_count_24h', 'total_amount_24h']

X = df[feature_cols]
y = df['is_fraud']

dtrain = xgb.DMatrix(X, label=y)

```

```

params = {
 'max_depth': 6,
 'eta': 0.1,
 'objective': 'binary:logistic',
 'eval_metric': 'auc'
}

model = xgb.train(params, dtrain, num_boost_round=100)

def score_transaction(txn_id):
 features = compute_transaction_features(txn_id)
 dtest = xgb.DMatrix([features])
 fraud_score = model.predict(dtest)[0]

 if fraud_score > 0.8:
 # Block transaction
 db.execute("""
 UPDATE @txn_id WITH {
 status: 'blocked',
 fraud_score: @score,
 blocked_at: DATE_NOW()
 } IN transactions
 """, bind_vars={'txn_id': txn_id, 'score': float(fraud_score)})

 return {'blocked': True, 'score': fraud_score}

 return {'blocked': False, 'score': fraud_score}

```

## Use Case 2: Product Recommendations

### Collaborative Filtering mit Vector Embeddings:

```

from sklearn.decomposition import TruncatedSVD

def create_user_item_matrix():
 result = db.execute("""
 FOR order IN orders
 FOR item IN order.items
 RETURN {
 user_id: order.customer_id,
 product_id: item.product_id,
 rating: item.quantity * item.price
 }
 """)

 df = pd.DataFrame(result)
 matrix = df.pivot_table(
 index='user_id',
 columns='product_id',
 values='rating',
 fill_value=0
)

 return matrix

matrix = create_user_item_matrix()

svd = TruncatedSVD(n_components=50, random_state=42)
user_embeddings = svd.fit_transform(matrix)
product_embeddings = svd.components_.T

for idx, user_id in enumerate(matrix.index):
 embedding = user_embeddings[idx].tolist()

 db.execute("""
 UPDATE @user_id WITH {
 embedding: @embedding,
 embedding_version: 'v1'
 } IN users
 """, bind_vars={'user_id': user_id, 'embedding': embedding})

def get_recommendations(user_id, top_k=10):
 result = db.execute("""
 LET user = DOCUMENT('users', @user_id)

 FOR product IN products
 FILTER product.embedding != null

 LET similarity = COSINE_SIMILARITY(user.embedding, product.embedding)

 SORT similarity DESC
 """)

```

```

LIMIT @top_k

RETURN {
 product_id: product._id,
 product_name: product.name,
 similarity_score: similarity
}
"""", bind_vars={'user_id': user_id, 'top_k': top_k})

return result

```

## 16.7 Best Practices

### Performance-Optimierung

#### 1. Feature Store Optimierung:

```

db.execute("""
 CREATE INDEX idx_feature_lookup ON ml_features (
 entity_id, feature_name, version
)
""")

db.execute("""
 CREATE MATERIALIZED VIEW customer_features_latest AS
 FOR feature IN ml_features
 SORT feature.version DESC
 COLLECT entity_id = feature.entity_id, feature_name = feature.feature_name
 INTO group = feature
 RETURN {
 entity_id: group[0].feature.entity_id,
 feature_name: group[0].feature.feature_name,
 feature_value: group[0].feature.feature_value,
 computed_at: group[0].feature.computed_at
 }
""")

```

#### 2. Batch Processing:

```
from concurrent.futures import ThreadPoolExecutor

def parallel_feature_computation(customer_ids, n_workers=8):
 with ThreadPoolExecutor(max_workers=n_workers) as executor:
 futures = [executor.submit(compute_customer_features, cid)
 for cid in customer_ids]

 results = [f.result() for f in futures]

 return results
```

### 3. Model Caching:

```
import pickle
from functools import lru_cache

@lru_cache(maxsize=10)
def load_model(model_path):
 """Cache loaded models in memory"""
 with open(model_path, 'rb') as f:
 return pickle.load(f)
```

## Monitoring & Alerts

### Automated Alerts:



```

def check_model_health():
 monitor = ModelMonitor(db, 'churn_predictor')

 # Performance Check
 metrics = monitor.calculate_metrics(time_window_days=1)

 if metrics['accuracy'] < 0.80:
 send_alert(f"Model accuracy dropped to {metrics['accuracy']:.2%}")

 # Drift Check
 for feature in feature_names:
 drift = monitor.detect_feature_drift(feature, time_window_days=7)

 if drift['drift_detected']:
 send_alert(f"Feature drift detected: {feature} (score: {drift['drift_score']:.2f})")

 # Prediction Volume Check
 recent_count = db.execute("""
 LET cutoff = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
 FOR log IN prediction_logs
 FILTER log.timestamp >= cutoff
 COLLECT WITH COUNT INTO count
 RETURN count
 """)[0]

 if recent_count < 100:
 send_alert(f"Low prediction volume: {recent_count} in last hour")

import schedule

schedule.every(1).hour.do(check_model_health)

```

## Zusammenfassung

ThemisDB bietet eine vollständige ML-Plattform:

- ✓ **Feature Store** - Effiziente Feature-Verwaltung mit MVCC
- ✓ **Multi-Framework Support** - Scikit-learn, TensorFlow, PyTorch, XGBoost
- ✓ **Model Serving** - Online und Batch Predictions
- ✓ **MLOps** - Experiment Tracking, Model Registry, Monitoring
- ✓ **Drift Detection** - Automatische Erkennung von Feature/Model Drift
- ✓ **Graph ML** - Native Graph-Features für komplexe Beziehungen
- ✓ **Vector Embeddings** - HNSW-basierte Similarity Search

Die Multi-Model-Architektur ermöglicht komplexe ML-Pipelines mit relationalen Features, Graph-Analysen und Vector-Embeddings in einer einzigen Datenbank.



# Kapitel 25: Chapter 19: Monitoring & Observability

---

**Status:** Production-Ready

**Version:** v1.3.0

**Last Updated:** December 30, 2025

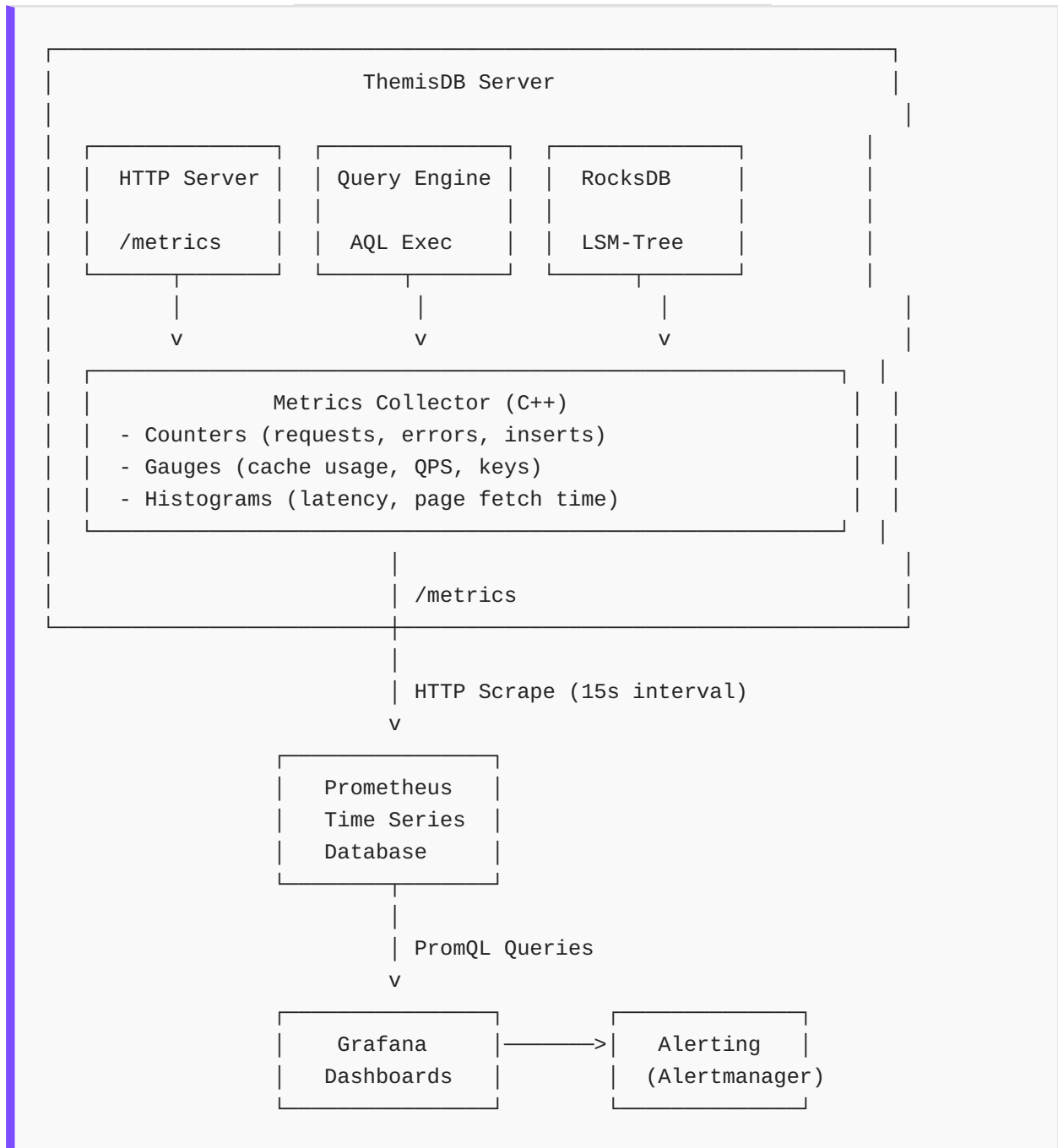
---

## 19.1 Übersicht

ThemisDB implementiert ein umfassendes Production-Grade Monitoring- und Observability-Stack basierend auf branchenüblichen Open-Source-Tools. Die Architektur folgt den **Three Pillars of Observability**: Metrics (Prometheus), Logs (Strukturierte Logs), und Traces (OpenTelemetry).

**Design-Prinzipien:** - **Metrics-First:** Prometheus als primäre Metriken-Datenbank - **Low-Cardinality:** Vermeidung von High-Cardinality-Labels (keine UUIDs/Timestamps in Labels) - **Cumulative Histograms:** Alle Histogramme folgen Prometheus-Konventionen - **Zero-Config Development:** Jaeger läuft Out-of-the-Box via Docker - **Production-Hardened:** Grafana Dashboards mit SLI/SLO-Tracking

**Architektur-Übersicht:**



**Deployment-Architektur:** - **Development:** Jaeger All-in-One Container für lokales Tracing - **Staging:** Grafana Tempo + Prometheus + Grafana Stack - **Production:** Multi-Node Prometheus Federation + Thanos für Long-Term Storage

```

graph TB
 subgraph "ThemisDB Server"
 HTTP[HTTP Server metrics endpoint]
 QE[Query Engine AQL Execution]
 RDB[RocksDB Storage]

 HTTP --> MC[Metrics Collector]
 QE --> MC
 RDB --> MC

 MC -->|Counters; Gauges; Histograms| Export[Metrics endpoint]
 end

 Export -->|HTTP scrape 15s interval| Prom[(Prometheus Time-Series DB)]

 Prom -->|PromQL| Grafana[Grafana Dashboards]
 Prom -->|Alerts| Alert[Alertmanager Notifications]

 Grafana --> Dash1[System Metrics CPU Memory Disk]
 Grafana --> Dash2[Query Performance Latency Throughput]
 Grafana --> Dash3[Storage Metrics RocksDB Cache]

 Alert --> Email[Email]
 Alert --> Slack[Slack]
 Alert --> PD[PagerDuty]

 style Export fill:#667eea
 style Prom fill:#f093fb
 style Grafana fill:#43e97b
 style Alert fill:#ff6348

```

Abb. 19.1: Monitoring-Stack-Übersicht

## 19.2 Prometheus Metrics

ThemisDB exportiert **48 Metriken** über den `/metrics` Endpoint im Prometheus Text Format. Alle Metriken verwenden konsistente Naming-Konventionen und Low-Cardinality-Labels.

### 19.2.1 Naming-Konventionen

**Präfixe:** - `vccdb_*` : ThemisDB-spezifische Metriken - `rocksdb_*` : RocksDB Storage-Metriken - `themis_*` : Index- und Query-Metriken - `process_*` : System-Metriken (Uptime)

**Suffixe:** - `_total` : Monoton steigende Counter - `_bytes` : Größenangaben in Bytes - `_seconds` : Zeitangaben in Sekunden - `_microseconds` : Latenz in Mikrosekunden - `_ms` : Latenz in Millisekunden

## 19.2.2 Server-Metriken

### vccdb\_requests\_total (Counter)

Gesamtzahl der HTTP-Requests seit Server-Start.

**Labels:** - `method` : HTTP-Methode (GET, POST, PUT, DELETE) - `route` : Request-Route (z.B. `/entities/{key}` , `/query` , `/vector/search` )

#### Beispiel-Output:

```
vccdb_requests_total{method="GET",route="/entities/{key}"} 1234
vccdb_requests_total{method="POST",route="/query"} 567
vccdb_requests_total{method="POST",route="/aql"} 890
```

#### PromQL-Query-Beispiele:

```
rate(vccdb_requests_total[5m])

sum by (route) (rate(vccdb_requests_total[5m]))

topk(5, sum by (route) (rate(vccdb_requests_total[5m])))
```

Die Metrik wird im `HttpServer` nach jedem abgeschlossenen Request inkrementiert. Das `route` - Label enthält die **Route-Template** (z.B. `/entities/{key}` ), nicht den konkreten Key-Wert, um Cardinality niedrig zu halten.

### vccdb\_errors\_total (Counter)

Gesamtzahl der Fehler-Responses (HTTP 4xx/5xx).

**Labels:** - `status_code` : HTTP-Status-Code (400, 404, 500, 503) - `route` : Request-Route

#### Beispiel-Output:

```
vccdb_errors_total{status_code="404",route="/entities/{key}"} 12
vccdb_errors_total{status_code="500",route="/query"} 3
vccdb_errors_total{status_code="400",route="/aql"} 5
```

#### PromQL-Query-Beispiele:

```
rate(vccdb_errors_total[5m])

100 * rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m])

rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m]) > 0.05
```

Diese Metrik ist entscheidend für **Service Level Indicators (SLIs)**. Ein Production-Service sollte eine Error Rate < 0.1% (99.9% Success Rate) anstreben.

---

### vccdb\_qps (Gauge)

Queries per Second - aktuelle Request-Rate berechnet als exponentieller gleitender Durchschnitt.

#### Beispiel-Output:

```
vccdb_qps 123.45
```

#### PromQL-Query-Beispiele:

```
avg_over_time(vccdb_qps[1h])

max_over_time(vccdb_qps[1h])

deriv(vccdb_qps[5m])
```

Der QPS-Wert wird alle 5 Sekunden neu berechnet mit der Formel:

```
QPS_new = α * QPS_current + (1-α) * QPS_old
```

mit Dämpfungsfaktor  $\alpha = 0.7$  für schnelle Reaktion auf Last-Änderungen.

---

### process\_uptime\_seconds (Gauge)

Server-Laufzeit in Sekunden seit Prozess-Start.

#### Beispiel-Output:

```
process_uptime_seconds 86400
```

**PromQL-Query-Beispiele:**

```
process_uptime_seconds / 86400
process_uptime_seconds < 300
```

Diese Metrik ist nützlich für die Erkennung von Server-Restarts und zur Korrelation mit Performance-Problemen nach Deployments.

### 19.2.3 Latenz-Histogramme

ThemisDB verwendet **kumulative Buckets** gemäß Prometheus-Spezifikation. Jeder Bucket mit `le="X"` enthält die Anzahl der Beobachtungen  $\leq X$ .

**Bucket-Definitionen****HTTP-Request-Latenz (Mikrosekunden):**

```
100, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000, 5000000, +Inf
```

**Page-Fetch-Latenz (Millisekunden):**

```
1, 5, 10, 25, 50, 100, 250, 500, 1000, 5000, +Inf
```

Die Bucket-Grenzen sind so gewählt, dass sie typische Latenz-Verteilungen abdecken: - **Schnell**: < 1ms (In-Memory-Index-Lookup) - **Normal**: 1-10ms (RocksDB Point-Lookup mit Block Cache Hit) - **Langsam**: 10-100ms (Disk I/O) - **Sehr langsam**: > 100ms (Full Scan, Cold Cache)

**vccdb\_latency\_bucket\_microseconds (Histogram)**

HTTP-Request-Latenz von Anfang des Request-Handlings bis zum vollständigen Response.

**Beispiel-Output:**



```

vccdb_latency_bucket_microseconds{le="100"} 45
vccdb_latency_bucket_microseconds{le="500"} 123
vccdb_latency_bucket_microseconds{le="1000"} 234
vccdb_latency_bucket_microseconds{le="5000"} 450
vccdb_latency_bucket_microseconds{le="10000"} 480
vccdb_latency_bucket_microseconds{le="50000"} 495
vccdb_latency_bucket_microseconds{le="100000"} 498
vccdb_latency_bucket_microseconds{le="500000"} 499
vccdb_latency_bucket_microseconds{le="1000000"} 500
vccdb_latency_bucket_microseconds{le="5000000"} 500
vccdb_latency_bucket_microseconds{le="+Inf"} 500
vccdb_latency_sum_microseconds 1234567
vccdb_latency_count 500

```

### PromQL-Query-Beispiele:

```

histogram_quantile(0.50, rate(vccdb_latency_bucket_microseconds[5m]))

histogram_quantile(0.95, rate(vccdb_latency_bucket_microseconds[5m]))

histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m]))

rate(vccdb_latency_sum_microseconds[5m]) / rate(vccdb_latency_count[5m])

100 * sum(rate(vccdb_latency_bucket_microseconds{le="1000"}[5m])) / sum(rate(vccdb_latency_count[5m]))

sum by (le) (rate(vccdb_latency_bucket_microseconds[5m]))

```

**Performance-Interpretation:** - **P50 < 500µs:** Exzellent (In-Memory-Operationen) - **P95 < 5ms:** Gut (Block Cache Hit Rate > 95%) - **P99 < 50ms:** Akzeptabel (Gelegentliche Disk I/O) - **P99 > 100ms:** Schlecht (Hohe Disk-Latenz oder Full Scans)

### vccdb\_page\_fetch\_time\_ms\_bucket (Histogram)

Latenz für Cursor-Pagination-Fetches (GET /cursor/{cursor\_id}).

#### Beispiel-Output:

```
vccdb_page_fetch_time_ms_bucket{le="1"} 89
vccdb_page_fetch_time_ms_bucket{le="5"} 156
vccdb_page_fetch_time_ms_bucket{le="10"} 234
vccdb_page_fetch_time_ms_bucket{le="25"} 245
vccdb_page_fetch_time_ms_bucket{le="50"} 248
vccdb_page_fetch_time_ms_bucket{le="+Inf"} 250
vccdb_page_fetch_time_ms_sum 2345.67
vccdb_page_fetch_time_ms_count 250
```

**PromQL-Query-Beispiele:**

```
histogram_quantile(0.95, rate(vccdb_page_fetch_time_ms_bucket[5m]))

100 * sum(rate(vccdb_page_fetch_time_ms_bucket{le="10"}[5m])) / sum(rate(vccdb_page_fetch...
```

Diese Metrik ist speziell für Pagination-Performance relevant. Hohe P99-Werte (> 100ms) deuten auf ineffiziente Cursor-Implementierungen oder Cold Cache-Probleme hin.

## 19.2.4 RocksDB-Metriken

RocksDB-Metriken werden aus der `GetProperty()` API ausgelesen und als Prometheus-Gauges exportiert. Diese Metriken erlauben die Überwachung der Storage-Layer-Gesundheit.

### rocksdb\_block\_cache\_usage\_bytes (Gauge)

Aktuell verwendeter Block-Cache in Bytes.

**Beispiel-Output:**

```
rocksdb_block_cache_usage_bytes 1073741824
```

**PromQL-Query-Beispiele:**

```
100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes

100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes > 95
```

### rocksdb\_block\_cache\_capacity\_bytes (Gauge)

Konfigurierte Block-Cache-Kapazität in Bytes (aus `config.json`).

---

**Beispiel-Output:**

```
rocksdb_block_cache_capacity_bytes 2147483648
```

Typische Werte: - **Development:** 512 MB - 1 GB - **Staging:** 2-4 GB - **Production:** 8-16 GB (25% des verfügbaren RAM)

---

**rocksdb\_estimate\_num\_keys (Gauge)**

Geschätzte Anzahl Keys in der Datenbank (basierend auf SST-Metadaten).

**Beispiel-Output:**

```
rocksdb_estimate_num_keys 1234567
```

**Hinweis:** Dies ist eine **Schätzung**, nicht exakt. Für exakte Counts verwenden Sie `SELECT COUNT(*)` Queries.

---

**rocksdb\_pending\_compaction\_bytes (Gauge)**

Bytes, die auf Compaction warten. Hohe Werte (> 10 GB) können zu erhöhter Latenz führen.

**Beispiel-Output:**

```
rocksdb_pending_compaction_bytes 1234567890
```

**PromQL-Query-Beispiele:**

```
rocksdb_pending_compaction_bytes > 10000000000
deriv(rocksdb_pending_compaction_bytes[10m])
```

**Ursachen für hohen Backlog:** - Zu langsame Disk (SATA statt NVMe) - Zu kleine `max_background_compactions` (Default: 4) - Sehr hohe Write-Rate (> 100K writes/sec)

**Lösungen:** - Erhöhe `max_background_compactions` auf 8-16 - Nutze NVMe für WAL und SSTables - Aktiviere Level Compaction (statt Universal)

---

## rocksdb\_memtable\_size\_bytes (Gauge)

Aktuelle Memtable-Größe in Bytes.

### Beispiel-Output:

```
rocksdb_memtable_size_bytes 67108864
```

Die Memtable wird geflusht, wenn sie die konfigurierte `write_buffer_size` erreicht (Default: 64 MB). Mehrere Memtables können gleichzeitig aktiv sein ( `max_write_buffer_number` : Default 2).

## rocksdb\_files\_per\_level (Gauge)

Anzahl SST-Dateien pro LSM-Tree-Level.

**Labels:** - `level` : LSM-Tree-Level (0, 1, 2, 3, 4, 5, 6)

### Beispiel-Output:

```
rocksdb_files_per_level{level="0"} 3
rocksdb_files_per_level{level="1"} 12
rocksdb_files_per_level{level="2"} 45
rocksdb_files_per_level{level="3"} 123
rocksdb_files_per_level{level="4"} 234
rocksdb_files_per_level{level="5"} 456
rocksdb_files_per_level{level="6"} 789
```

### PromQL-Query-Beispiele:

```
sum(rocksdb_files_per_level)

rocksdb_files_per_level{level="0"} > 10
```

**LSM-Tree-Levels:** - **L0:** Frisch geflushed Memtables, unsortiert, können sich überlappen - **L1-L6:** Sortierte Runs, keine Überlappungen innerhalb eines Levels - **L6:** Coldest data (90% der Daten), ZSTD-komprimiert (2.8x Ratio)

## 19.2.5 Index-Metriken

### vccdb\_index\_rebuild\_total (Counter)

Gesamtzahl Index-Rebuild-Operationen.

**Labels:** - `table` : Tabellenname - `column` : Spaltenname

**Beispiel-Output:**

```
vccdb_index_rebuild_total{table="users",column="email"} 3
vccdb_index_rebuild_total{table="orders",column="customer_id"} 5
```

Index-Rebuilds werden getriggert durch: - `CREATE INDEX` Statement (Neuer Index) - `ANALYZE` Statement (Index-Statistik-Refresh) - Server-Neustart mit geändertem Schema

### vccdb\_index\_rebuild\_duration\_seconds (Counter)

Gesamt-Zeit für Index-Rebuilds in Sekunden.

**Labels:** - `table` : Tabellenname - `column` : Spaltenname

**Beispiel-Output:**

```
vccdb_index_rebuild_duration_seconds{table="users",column="email"} 123.45
```

### PromQL-Query-Beispiele:

```
vccdb_index_rebuild_duration_seconds / vccdb_index_rebuild_total

topk(5, vccdb_index_rebuild_duration_seconds / vccdb_index_rebuild_total)
```

**Performance-Benchmark:** - **B-Tree-Index:** ~50K entities/sec - **Hash-Index:** ~100K entities/sec - **Vector-Index (HNSW):** ~5K entities/sec (mit Construction-Parameter M=16, efConstruction=200)

### themis\_index\_cursor\_anchor\_hits\_total (Counter)

Anzahl erfolgreicher Cursor-Anchor-Lookups für Pagination.

**Beispiel-Output:**

```
themis_index_cursor_anchor_hits_total 567
```

Diese Metrik trackt, wie oft Cursors erfolgreich fortgesetzt wurden. Eine niedrige Hit-Rate deutet auf abgelaufene Cursors (TTL) oder ungültige Cursor-IDs hin.

## themis\_index\_range\_scan\_steps\_total (Counter)

Gesamtzahl Range-Scan-Schritte (Index-Traversierungen).

### Beispiel-Output:

```
themis_index_range_scan_steps_total 12345
```

### PromQL-Query-Beispiele:

```
rate(themis_index_range_scan_steps_total[5m])

rate(themis_index_range_scan_steps_total[5m]) / rate(vccdb_requests_total{route="/query"})
```

Hohe Range-Scan-Zahlen (> 1M steps/sec) deuten auf ineffiziente Queries mit breiten Ranges hin (z.B. `age > 18` ohne zusätzliche Prädikate).

---

## 19.2.6 Vector-Index-Metriken

### vccdb\_vector\_index\_size\_bytes (Gauge)

Geschätzte Größe des In-Memory-Vector-Index in Bytes.

### Beispiel-Output:

```
vccdb_vector_index_size_bytes 536870912
```

### Berechnung:

```
Index Size ≈ N * (D * 4 bytes + M * 2 bytes)
```

Für 1M Vektoren, Dimension 768, M=16:

```
Index Size = 1M * (768*4 + 16*2) = 3.1 GB
```

---

### vccdb\_vector\_search\_duration\_ms (Histogram)

Vector-Similarity-Search-Latenz in Millisekunden.

**Beispiel-Output:**

```
vccdb_vector_search_duration_ms_bucket{le="1"} 45
vccdb_vector_search_duration_ms_bucket{le="5"} 123
vccdb_vector_search_duration_ms_bucket{le="10"} 234
vccdb_vector_search_duration_ms_bucket{le="+Inf"} 250
```

**PromQL-Query-Beispiele:**

```
histogram_quantile(0.95, rate(vccdb_vector_search_duration_ms_bucket[5m]))
```

**Performance-Benchmark (1M Vektoren, 768-dim, M=16, efSearch=64):** - **P50:** 1.2 ms - **P95:** 3.5 ms - **P99:** 8.2 ms

**vccdb\_vector\_batch\_insert\_duration\_ms (Histogram)**

Batch-Insert-Latenz für Vektor-Batches.

**vccdb\_vector\_batch\_insert\_items\_total (Counter)**

Gesamtzahl eingefügter Vector-Items.

**PromQL-Query-Beispiele:**

```
rate(vccdb_vector_batch_insert_items_total[5m]) / rate(vccdb_vector_batch_insert_total[5m])
```

Typische Batch-Größen: - **Klein:** 10-100 (Echtzeit-Ingestion) - **Mittel:** 100-1000 (Bulk-Import) - **Groß:** 1000-10000 (Initial Load)

## 19.3 Grafana Dashboards

ThemisDB liefert **3 vorgefertigte Grafana-Dashboards** für unterschiedliche Monitoring-Szenarien.

### 19.3.1 Dashboard 1: Server-Übersicht

**Zweck:** High-Level-Übersicht über Server-Gesundheit und Performance.

**Panels:**

1. **QPS (Queries per Second)** `promql vccdb_qps`
2. Visualization: Graph

3. Y-Axis: Queries/sec

4. Thresholds: Warning > 5000, Critical > 10000

5. **Error Rate** `promql 100 * rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m])`

6. Visualization: Graph

7. Y-Axis: Percent

8. Thresholds: Warning > 1%, Critical > 5%

9. SLI-Target: < 0.1% (99.9% Success Rate)

10. **Request Latency (P50, P95, P99)** `promql histogram_quantile(0.50, rate(vccdb_latency_bucket_microseconds[5m])) / 1000 # ms histogram_quantile(0.95, rate(vccdb_latency_bucket_microseconds[5m])) / 1000 histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m])) / 1000`

11. Visualization: Graph

12. Y-Axis: Milliseconds

13. SLI-Targets: P50 < 1ms, P95 < 5ms, P99 < 50ms

14. **Uptime** `promql process_uptime_seconds / 86400`

15. Visualization: Stat

16. Unit: Days

17. Color: Green > 7d, Yellow > 1d, Red < 1d

18. **Requests by Route (Top 5)** `promql topk(5, sum by (route) (rate(vccdb_requests_total[5m])))`

19. Visualization: Bar Gauge

20. Y-Axis: Requests/sec

## 19.3.2 Dashboard 2: RocksDB Health

**Zweck:** Monitoring der Storage-Layer-Performance und Capacity Planning.

### Panels:

1. **Block Cache Hit Rate** `promql 100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes`

2. Visualization: Gauge

3. Unit: Percent

4. Thresholds: Green > 80%, Yellow > 60%, Red < 60%



5. **Compaction Backlog** `promql rocksdb_pending_compaction_bytes / 1e9 # GB`

6. Visualization: Graph

7. Y-Axis: Gigabytes

8. Threshold: Critical > 10 GB

9. **Keys Estimate** `promql rocksdb_estimate_num_keys`

10. Visualization: Stat

11. Unit: Short (1.23M)

12. Trend: Show sparkline

13. **SST Files per Level** `promql sum by (level) (rocksdb_files_per_level)`

14. Visualization: Bar Gauge (Horizontal)

15. X-Axis: Number of Files

16. Y-Axis: Level (0-6)

17. **Memtable Size** `promql rocksdb_memtable_size_bytes / 1e6 # MB`

18. Visualization: Graph

19. Y-Axis: Megabytes

20. Threshold: Warning > 128 MB (2x write\_buffer\_size)

21. **LSM-Tree Amplification** `promql sum(rocksdb_files_per_level{level!="0"}) / rocksdb_files_per_level{level="0"}`

22. Visualization: Stat

23. Unit: None

24. Interpretation: Higher = Better compaction efficiency

### 19.3.3 Dashboard 3: Vector Operations

**Zweck:** Monitoring von Vector-Search-Performance und Index-Größe.

**Panels:**

1. **Vector Index Size** `promql vccdb_vector_index_size_bytes / 1e9 # GB`

2. Visualization: Gauge

3. Unit: Gigabytes

4. Thresholds: Warning > 8 GB, Critical > 16 GB (Memory Pressure)

5. **Search Latency P95** `promql histogram_quantile(0.95, rate(vccdb_vector_search_duration_ms_bucket[5m]))`
  6. Visualization: Graph
  7. Y-Axis: Milliseconds
  8. Target: < 5ms
  9. **Batch Insert Rate** `promql rate(vccdb_vector_batch_insert_items_total[5m])`
  10. Visualization: Graph
  11. Y-Axis: Items/sec
  12. Benchmark: > 5000 items/sec (Production)
  13. **Average Batch Size** `promql rate(vccdb_vector_batch_insert_items_total[5m]) / rate(vccdb_vector_batch_insert_total[5m])`
  14. Visualization: Stat
  15. Unit: Items
  16. Optimal: 100-1000 (Trade-off: Latenz vs Throughput)
  17. **Delete Rate** `promql rate(vccdb_vector_delete_by_filter_items_total[5m])`
  18. Visualization: Graph
  19. Y-Axis: Items/sec
- 

## 19.4 OpenTelemetry Distributed Tracing

ThemisDB implementiert **Distributed Tracing** via OpenTelemetry für Production-Debugging und Performance-Analyse komplexer Multi-Component-Queries.

### 19.4.1 Architektur

**Komponenten:** 1. **Tracer Wrapper** ( `utils/tracing.h` / `.cpp` ) - Initialisierung des OTLP HTTP Exporters - TracerProvider mit Resource Attributes (service.name, version) - SimpleSpanProcessor für sofortigen Export

1. **Span RAII Wrapper**
2. `Tracer::Span` : Move-only Span mit automatischem `end()` im Destruktor
3. `ScopedSpan` : Convenience-Wrapper für lokale Spans
4. Attribute-Support: `setAttribute(key, value)` für string, int64, double, bool
5. **No-Op Fallback**

6. Wenn `THEMIS_ENABLE_TRACING` nicht definiert, sind alle Methoden No-Ops

7. Kein Linking gegen OpenTelemetry-Libraries notwendig

### Datenfluss:

```

HTTP Request → ScopedSpan("handleAqlQuery")
 ↓
 QueryEngine::executeQuery() → ScopedSpan("executeQuery")
 ↓
 Index Scans → Child Spans
 ↓
 OTLP HTTP Exporter → Jaeger/Tempo

```

## 19.4.2 Instrumentierte Komponenten

### HTTP-Handler (Top-Level Spans)

- **GET** `/entities/:key:` `entity.key` , `entity.size_bytes`
- **PUT** `/entities/:key:` `entity.key` , `entity.table` , `entity.pk` , `entity.size_bytes` , `entity.cdc_recorded`
- **DELETE** `/entities/:key:` `entity.key` , `entity.table` , `entity.pk` , `entity.cdc_recorded`
- **POST** `/query:` `query.table` , `query.predicates_count` , `query.exec_mode` , `query.result_count`
- **POST** `/query/aql:` `aql.query` , `aql.explain` , `aql.optimize` , `aql.allow_full_scan` , `aql.result_count`
- **POST** `/graph/traverse:` `graph.start_vertex` , `graph.max_depth` , `graph.visited_count`
- **POST** `/vector/search:` `vector.dimension` , `vector.k` , `vector.results_count`

### Beispiel-Trace:

```
[Span: handleAqlQuery, Duration: 23.5ms]
├─ [Span: aql.parse, Duration: 0.8ms]
│ └─ Attributes: aql.query_length=156
├─ [Span: aql.translate, Duration: 1.2ms]
│ └─ [Span: aql.for, Duration: 21.2ms]
│ ├── Attributes: for.table="users", for.predicates_count=2, for.result_count=123
│ ├── [Span: index.scanEqual, Duration: 8.5ms]
│ │ └─ Attributes: index.table="users", index.column="email", index.result_count=1
│ └─ [Span: index.scanRange, Duration: 12.1ms]
│ └─ Attributes: index.table="users", index.column="age", index.result_count=122
```

## AQL Execution Pipeline (Operator-Level Spans)

- **aql.parse:** `aql.query_length` - Parsen der AQL-Query in AST
- **aql.translate** - Übersetzung des AST in ConjunctiveQuery oder TraversalQuery
- **aql.for:** `for.table`, `for.predicates_count`, `for.range_predicates_count`, `for.order_by`, `for.order_desc`, `for.result_count`, `for.exec_mode`
- **aql.filter** - Filterung der Ergebnisse
- Bei Traversal: `filter.evaluations_total`, `filter.short_circuits`
- **aql.limit:** `limit.offset`, `limit.count`, `limit.input_count`, `limit.output_count`
- **aql.collect:** `collect.input_count`, `collect.group_by_count`, `collect.aggregates_count`, `collect.group_count`
- **aql.return:** `return.input_count`
- **aql.traversal:** `traversal.start_vertex`, `traversal.min_depth`, `traversal.max_depth`, `traversal.direction`, `traversal.result_count`
- Child-Span: **aql.traversal.bfs:** `traversal.max_frontier_size_limit`, `traversal.max_results_limit`, `traversal.visited_count`, `traversal.edges_expanded`, `traversal.filter_evaluations`

## Index-Scans (Child-Spans)

- **index.scanEqual:** `index.table`, `index.column`, `index.result_count`
- **index.scanRange:** `index.table`, `index.column`, `index.result_count`, `range.has_lower`, `range.has_upper`, `range.includeLower`, `range.includeUpper`
- **or.disjunct.execute:** `disjunct.eq_count`, `disjunct.range_count`, `disjunct.result_count`

## 19.4.3 Jaeger Integration (Development)

Jaeger via Docker starten:

```
docker run -d --name jaeger \
-p 4318:4318 \
-p 16686:16686 \
jaegertracing/all-in-one:latest
```

**Ports:** - **4318** : OTLP HTTP receiver (für Themis) - **16686** : Jaeger UI

**Themis konfigurieren** ( **config/config.json** ):

```
{
 "tracing": {
 "enabled": true,
 "service_name": "themis-dev",
 "otlp_endpoint": "http://localhost:4318"
 }
}
```

**Themis starten:**

```
./build/Release/themis_server.exe
```

**Traces anzeigen:**

1. Öffne <http://localhost:16686> (Jaeger UI)
2. Wähle Service "themis-dev"
3. Klicke "Find Traces"

**Trace-Analyse-Szenarien:**

1. **Langsame Query debuggen:**
2. Filtere nach **duration > 100ms**
3. Identifiziere langsamste Spans (rot markiert)
4. Prüfe **index.scanRange** Spans auf breite Ranges
5. **Full Scan Detection:**
6. Suche nach Spans mit **query.exec\_mode=full\_scan**
7. Prüfe **fullscan.scanned** Attribut (sollte << 1M sein)
8. **Index Efficiency:**
9. Vergleiche **index.result\_count** zwischen Scans

---

10. Hohe Counts + niedrige Query-Results = ineffizienter Index

---

### 19.4.4 Grafana Tempo Integration (Production)

#### Tempo via Docker Compose:

```
version: '3'
services:
 tempo:
 image: grafana/tempo:latest
 command: ["-config.file=/etc/tempo.yaml"]
 volumes:
 - ./tempo.yaml:/etc/tempo.yaml
 - ./tempo-data:/tmp/tempo
 ports:
 - "4318:4318" # OTLP HTTP
 - "3200:3200" # Tempo API

 grafana:
 image: grafana/grafana:latest
 ports:
 - "3000:3000"
 environment:
 - GF_AUTH_ANONYMOUS_ENABLED=true
 - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
 volumes:
 - ./grafana-datasources.yaml:/etc/grafana/provisioning/datasources/datasources.yaml
```

#### tempo.yaml:

```
server:
 http_listen_port: 3200

distributor:
 receivers:
 otlp:
 protocols:
 http:
 endpoint: 0.0.0.0:4318

storage:
 trace:
 backend: local
 local:
 path: /tmp/tempo/traces
 wal:
 path: /tmp/tempo/wal
 block:
 retention: 720h # 30 days
```

#### grafana-datasources.yaml:

```
apiVersion: 1
datasources:
- name: Tempo
 type: tempo
 access: proxy
 url: http://tempo:3200
 uid: tempo
 editable: false
```

#### Trace-Query in Grafana:

1. Öffne `http://localhost:3000`
2. Explore → Tempo Datasource
3. Search Query: `{service.name="themis-server"} && duration > 100ms`
4. Trace-View zeigt Waterfall-Diagramm aller Spans

---

### 19.4.5 Performance-Hinweise

**Span-Overhead:** - **Mit Tracing aktiviert:** ~5-10 µs pro Span (inkl. Attribut-Serialisierung) - **Ohne Tracing (THEMIS\_ENABLE\_TRACING=OFF):** 0 µs (inline no-ops)

---

**Best Practices:**

1. **Granularität:** Erstelle Spans für HTTP-Requests, Query-Execution, Index-Scans
  2. **Attribute:** Füge relevante Metadaten hinzu (table, index\_type, row\_count)
  3. **Sampling** (zukünftig): Für High-Throughput-Szenarien Sampling verwenden
  4. **Batch Processor** (zukünftig): SimpleSpanProcessor → BatchSpanProcessor für bessere Performance
- 

## **19.5 Alerting**

### **19.5.1 Alert-Definitionen (Prometheus Alertmanager)**

**alerts.yaml:**



```

groups:
- name: themis_server
 interval: 30s
 rules:
 # High Error Rate
 - alert: HighErrorRate
 expr: rate(vccdb_errors_total[5m]) / rate(vccdb_requests_total[5m]) > 0.05
 for: 5m
 labels:
 severity: critical
 annotations:
 summary: "ThemisDB Error Rate above 5%"
 description: "{{ $value | humanizePercentage }}" error rate in last 5 minutes"

 # High Latency P99
 - alert: HighLatencyP99
 expr: histogram_quantile(0.99, rate(vccdb_latency_bucket_microseconds[5m])) > 100000
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "ThemisDB P99 latency above 100ms"
 description: "P99 latency: {{ $value | humanizeDuration }}"

 # Compaction Backlog
 - alert: CompactionBacklog
 expr: rocksdb_pending_compaction_bytes > 10000000000
 for: 10m
 labels:
 severity: warning
 annotations:
 summary: "RocksDB Compaction backlog > 10 GB"
 description: "{{ $value | humanize1024 }}"B pending compaction"

 # Low Cache Hit Rate
 - alert: LowCacheHitRate
 expr: 100 * rocksdb_block_cache_usage_bytes / rocksdb_block_cache_capacity_bytes < 20
 for: 15m
 labels:
 severity: info
 annotations:
 summary: "Block cache usage < 20%"
 description: "Cache may be undersized or cold"

 # Too Many L0 Files (Write Stall Risk)
 - alert: TooManyL0Files
 expr: rocksdb_files_per_level{level="0"} > 10
 for: 5m
 labels:

```

```
 severity: critical
 annotations:
 summary: "RocksDB L0 files > 10 (Write Stall Risk)"
 description: "{{ $value }}" L0 files, compaction cannot keep up"

Server Down
- alert: ThemisServerDown
 expr: up{job="themis"} == 0
 for: 1m
 labels:
 severity: critical
 annotations:
 summary: "ThemisDB Server is down"
 description: "Prometheus cannot scrape /metrics endpoint"
```

## 19.5.2 Alertmanager-Konfiguration

**alertmanager.yaml:**

```

global:
 resolve_timeout: 5m

route:
 group_by: ['alertname', 'severity']
 group_wait: 10s
 group_interval: 5m
 repeat_interval: 3h
 receiver: 'email'

routes:
 - match:
 severity: critical
 receiver: 'pagerduty'
 continue: true

 - match:
 severity: warning
 receiver: 'slack'

receivers:
 - name: 'email'
 email_configs:
 - to: 'ops-team@example.com'
 from: 'alerts@example.com'
 smarthost: 'smtp.example.com:587'
 auth_username: 'alerts@example.com'
 auth_password: 'password'

 - name: 'slack'
 slack_configs:
 - api_url: 'https://hooks.slack.com/services/T000000000/B000000000/XXXXXXXXXXXXX'
 channel: '#themis-alerts'
 title: '{{ .GroupLabels.alertname }}'
 text: '{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}'

 - name: 'pagerduty'
 pagerduty_configs:
 - service_key: 'YOUR_PAGERDUTY_SERVICE_KEY'

```

## 19.6 Production Deployment

### 19.6.1 Prometheus Scrape-Konfiguration

**prometheus.yaml:**

```

global:
 scrape_interval: 15s
 scrape_timeout: 10s
 evaluation_interval: 30s

scrape_configs:
 - job_name: 'themis'
 static_configs:
 - targets: ['localhost:8765']

 - job_name: 'themis-cluster'
 consul_sd_configs:
 - server: 'consul:8500'
 services: ['themis']
 relabel_configs:
 - source_labels: [__meta_consul_service]
 target_label: job
 - source_labels: [__meta_consul_node]
 target_label: node

```

## 19.6.2 Retention & Storage

**Empfohlene Retention:** - **Hochfrequente Metriken:** 15 Tage - **Long-term:** Downsampling auf 1h-Auflösung nach 7 Tagen (via Thanos)

### Prometheus Storage-Konfiguration:

```

storage:
 tsdb:
 path: /prometheus
 retention.time: 15d
 retention.size: 100GB

```

**Thanos Integration (Multi-Node Federation + Long-Term Storage):**

```

thanos sidecar \
 --tsdb.path /prometheus \
 --prometheus.url http://localhost:9090 \
 --objstore.config-file /etc/thanos/objstore.yaml

thanos query \
 --http-address 0.0.0.0:19192 \
 --store prometheus-0-sidecar:10901 \
 --store prometheus-1-sidecar:10901 \
 --store thanos-store:10901

thanos store \
 --data-dir /thanos/store \
 --objstore.config-file /etc/thanos/objstore.yaml

```

### objstore.yaml (S3 Backend):

```

type: S3
config:
 bucket: "themis-metrics"
 endpoint: "s3.eu-central-1.amazonaws.com"
 region: "eu-central-1"
 access_key: "YOUR_ACCESS_KEY"
 secret_key: "YOUR_SECRET_KEY"

```

## 19.6.3 Cardinality Management

**Niedrige Cardinality (< 100 Zeitreihen):** - Metriken ohne Labels (z.B. `vccdb_qps`, `process_uptime_seconds`)

**Hohe Cardinality (potenziell Tausende Zeitreihen):** - Index-Metriken mit `table` + `column` Labels - Wenn 100 Tables mit je 20 Columns → 2000 Index-Rebuild-Metriken

**Best Practices:** - Verwende **niemals** High-Cardinality-Labels wie User-IDs, Timestamps, UUIDs - Nutze Route-Templates (z.B. `/entities/{key}`) statt konkreter Pfade - Für User-spezifisches Tracking: Verwende separate Logs/Traces, nicht Prometheus

## 19.6A Enhanced Prometheus Metrics (v1.4.0-alpha)

**Neu in v1.4.0-alpha:** Umfassende Metriken für LLM-Operations, Caching, GPU-Nutzung, High Availability und Performance-Optimierungen.

**Monitoring-Architektur Übersicht:**

```

graph TB
 subgraph "ThemisDB Cluster"
 TDB1[ThemisDB Node 1
Shard 1] --> Exporter1[Prometheus
Exporter :9090]
 TDB2[ThemisDB Node 2
Shard 2] --> Exporter2[Prometheus
Exporter :9090]
 TDB3[ThemisDB Node 3
Hot Spare] --> Exporter3[Prometheus
Exporter :9090]

 LLM1[LLM Engine
GPU Metrics] --> Exporter1
 LLM2[LLM Engine
GPU Metrics] --> Exporter2
 end

 subgraph "Monitoring Stack"
 Exporter1 --> Prometheus[Prometheus
Time Series DB]
 Exporter2 --> Prometheus
 Exporter3 --> Prometheus

 Prometheus --> Grafana[Grafana
Visualization]
 Prometheus --> AlertManager[AlertManager
Notifications]

 AlertManager --> Slack[Slack]
 AlertManager --> PagerDuty[PagerDuty]
 AlertManager --> Email[Email]
 end

 subgraph "Metrics Categories"
 M1[LLM Metrics
15+ metrics]
 M2[Performance Metrics
10+ metrics]
 M3[HA Metrics
8+ metrics]
 M4[Cache Metrics
6+ metrics]
 end

 Prometheus -.collects.-> M1
 Prometheus -.collects.-> M2
 Prometheus -.collects.-> M3
 Prometheus -.collects.-> M4

 style TDB1 fill:#667eea
 style TDB2 fill:#667eea
 style TDB3 fill:#ffd32a
 style Prometheus fill:#43e97b
 style Grafana fill:#4facfe
 style AlertManager fill:#f093fb

```

**Abb. 19.2:** Alert-Management-Workflow

**Diagramm-Erklärung:** - **ThemisDB Nodes:** Jeder Node exportiert Metriken über HTTP-Endpoint (/metrics) - **Prometheus:** Scraped Metriken alle 15s, speichert als Time Series - **Grafana:** Visualisiert Metriken in Dashboards (LLM, Performance, HA) - **AlertManager:** Routet Alerts zu verschiedenen Kanälen (Slack, PagerDuty, Email) - **Metric Categories:** Über 40 neue Metriken in v1.4.0-alpha organisiert in 4 Kategorien

### 19.6A.1 LLM-spezifische Metriken

**Neu in v1.4.0-alpha:** Umfassende Metriken für LLM-Operations, Caching und GPU-Nutzung.

#### Request Metriken:

```
themis_llm_requests_total{model="gpt-4",status="success"} 15420
themis_llm_requests_total{model="gpt-4",status="error"} 23
themis_llm_requests_total{model="llama-70b-local",status="success"} 8945

themis_llm_request_duration_seconds_bucket{model="gpt-4",le="0.5"} 1200
themis_llm_request_duration_seconds_bucket{model="gpt-4",le="1.0"} 8400
themis_llm_request_duration_seconds_bucket{model="gpt-4",le="2.0"} 14200
themis_llm_request_duration_seconds_sum{model="gpt-4"} 18540.5
themis_llm_request_duration_seconds_count{model="gpt-4"} 15420

themis_llm_tokens_generated_total{model="gpt-4",type="completion"} 2450000
themis_llm_tokens_generated_total{model="gpt-4",type="prompt"} 8900000

themis_llm_tokens_per_second{model="llama-70b-local"} 312.5
```

#### Cache Metriken:

```
themis_llm_prefix_cache_hits_total{model="gpt-4"} 12450
themis_llm_prefix_cache_misses_total{model="gpt-4"} 2970
themis_llm_prefix_cache_hit_rate{model="gpt-4"} 0.807

themis_llm_prefix_cache_tokens_saved_total{model="gpt-4"} 1850000
themis_llm_prefix_cache_cost_saved_usd{model="gpt-4"} 55.50

themis_llm_response_cache_hits_total{model="gpt-4"} 8420
themis_llm_response_cache_misses_total{model="gpt-4"} 7000
themis_llm_response_cache_hit_rate{model="gpt-4"} 0.546

themis_llm_response_cache_latency_saved_seconds_total{model="gpt-4"} 6840.5
```

#### GPU Metriken:

```
themis_llm_gpu_memory_used_bytes{device="0",model="llama-70b-local"} 25903915008 # 24.1GB
themis_llm_gpu_memory_total_bytes{device="0"} 42949672960 # 40GB
themis_llm_gpu_memory_utilization{device="0"} 0.603

themis_llm_gpu_utilization_percent{device="0"} 94
themis_llm_gpu_temperature_celsius{device="0"} 72
themis_llm_gpu_power_watts{device="0"} 325

themis_llm_multi_gpu_active_devices{model="llama-70b-local"} 4
themis_llm_multi_gpu_total_memory_bytes 171798691840 # 160GB (4x40GB)
```

### Batch Metriken:

```
themis_llm_batch_size{model="llama-70b-local"} 87

themis_llm_batch_utilization{model="llama-70b-local"} 0.679 # 87/128

themis_llm_batch_queue_length{model="llama-70b-local"} 12

themis_llm_batches_processed_total{model="llama-70b-local"} 58420
```

### Grafana Dashboard - LLM Operations:

```
panels:
- title: "LLM Request Rate"
 targets:
 - expr: rate(themis_llm_requests_total[5m])

- title: "LLM Latency (P50, P95, P99)"
 targets:
 - expr: histogram_quantile(0.50, themis_llm_request_duration_seconds_bucket)
 - expr: histogram_quantile(0.95, themis_llm_request_duration_seconds_bucket)
 - expr: histogram_quantile(0.99, themis_llm_request_duration_seconds_bucket)

- title: "Cache Hit Rates"
 targets:
 - expr: themis_llm_prefix_cache_hit_rate
 - expr: themis_llm_response_cache_hit_rate

- title: "GPU Memory Usage"
 targets:
 - expr: themis_llm_gpu_memory_used_bytes / themis_llm_gpu_memory_total_bytes
```



## 19.6A.2 Performance-Optimierungs-Metriken

**Neu in v1.4.0-alpha:** Metriken für Flash Attention, Speculative Decoding und Continuous Batching.

### Flash Attention Metriken:

```
themis_flash_attention_enabled{model="llama-70b-local"} 1
themis_flash_attention_memory_saved_bytes{model="llama-70b-local"} 15099494400 # 14.1GB
themis_flash_attention_throughput_improvement{model="llama-70b-local"} 0.69 # 69%
themis_flash_attention_latency_reduction{model="llama-70b-local"} 0.39 # 39%
```

### Speculative Decoding Metriken:

```
themis_speculative_decoding_enabled{model="llama-70b-local"} 1
themis_speculative_decoding_draft_model{model="llama-70b-local", draft="llama-7b-local"} 1
themis_speculative_decoding_acceptance_rate{model="llama-70b-local"} 0.821
themis_speculative_decoding_tokens_proposed_total{model="llama-70b-local"} 226000
themis_speculative_decoding_tokens_accepted_total{model="llama-70b-local"} 185460
themis_speculative_decoding_speedup{model="llama-70b-local"} 2.4
themis_speculative_decoding_latency_reduction_seconds{model="llama-70b-local"} 1.68
```

### Continuous Batching Metriken:

```
themis_continuous_batching_enabled{model="llama-70b-local"} 1
themis_continuous_batching_queue_length{model="llama-70b-local"} 12
themis_continuous_batching_queue_wait_time_seconds{model="llama-70b-local"} 0.095
themis_continuous_batching_batches_formed_total{model="llama-70b-local"} 58420
themis_continuous_batching_avg_batch_size{model="llama-70b-local"} 73.5
themis_continuous_batching_batch_fill_rate{model="llama-70b-local"} 0.68
themis_continuous_batching_throughput_improvement{model="llama-70b-local"} 1.76 # 176%
themis_continuous_batching_latency_reduction{model="llama-70b-local"} 0.57 # 57%
```

### Paged Attention Metriken:

```
themis_paged_attention_enabled{model="llama-70b-local"} 1

themis_paged_attention_pages_allocated 4096
themis_paged_attention_pages_used 2847
themis_paged_attention_memory_efficiency 0.92 # 92% weniger Waste

themis_paged_attention_concurrent_requests{model="llama-70b-local"} 445
themis_paged_attention_memory_per_request_bytes 94371840 # 90MB
```

### Grafana Dashboard - Performance Optimizations:

```
panels:
- title: "Flash Attention Impact"
 targets:
 - expr: themis_flash_attention_throughput_improvement * 100
 legendFormat: "Throughput Improvement %"
 - expr: themis_flash_attention_memory_saved_bytes / 1024^3
 legendFormat: "Memory Saved GB"

- title: "Speculative Decoding Acceptance Rate"
 targets:
 - expr: themis_speculative_decoding_acceptance_rate * 100
 legendFormat: "Acceptance Rate %"

- title: "Continuous Batching Efficiency"
 targets:
 - expr: themis_continuous_batching_batch_fill_rate
 legendFormat: "Batch Fill Rate"
 - expr: themis_continuous_batching_queue_length
 legendFormat: "Queue Length"
```

## 19.6A.3 Sharding & HA Metriken

**Neu in v1.4.0-alpha:** Metriken für Hot Spare und WAL Replication.

### Hot Spare Metriken:

```

themis_hot_spare_active{node="hot-spare-1"} 0
themis_hot_spare_active{node="hot-spare-2"} 0

themis_hot_spare_replication_lag_seconds{node="hot-spare-1"} 0.045
themis_hot_spare_replication_lag_bytes{node="hot-spare-1"} 16384

themis_hot_spare_failover_count_total{node="hot-spare-1"} 0
themis_hot_spare_last_failover_duration_seconds{node="hot-spare-1"} 0

themis_hot_spare_failover_ready{node="hot-spare-1"} 1
themis_hot_spare_last_health_check_timestamp{node="hot-spare-1"} 1704549600

```

### WAL Replication Metriken:

```

themis_wal_replication_lag_seconds{replica="hot-spare-1"} 0.002
themis_wal_replication_lag_bytes{replica="hot-spare-1"} 16

themis_wal_segments_total{shard="shard-1"} 156
themis_wal_segments_archived_total{shard="shard-1"} 12450
themis_wal_size_bytes{shard="shard-1"} 4294967296

themis_wal_sync_operations_total{shard="shard-1"} 125840
themis_wal_sync_duration_seconds_sum{shard="shard-1"} 45.2
themis_wal_sync_duration_seconds_count{shard="shard-1"} 125840

themis_wal_replication_mode{shard="shard-1"} 1 # 0=async, 1=sync, 2=hybrid

```

### Shard Health Metriken:

```

themis_shard_health{shard="shard-1"} 1
themis_shard_health{shard="shard-2"} 1
themis_shard_health{shard="shard-3"} 1
themis_shard_health{shard="shard-4"} 1

themis_shard_rebalance_operations_total{shard="shard-1"} 5
themis_shard_rebalance_duration_seconds{shard="shard-1"} 245.8
themis_shard_rebalance_in_progress{shard="shard-1"} 0

themis_shard_data_size_bytes{shard="shard-1"} 10737418240 # 10GB
themis_shard_key_count{shard="shard-1"} 2500000

```

### Grafana Dashboard - High Availability:

```
panels:
- title: "Shard Health Status"
 targets:
 - expr: themis_shard_health
 legendFormat: "{{shard}}"

- title: "Hot Spare Replication Lag"
 targets:
 - expr: themis_hot_spare_replication_lag_seconds
 legendFormat: "{{node}}"

- title: "WAL Replication Lag"
 targets:
 - expr: themis_wal_replication_lag_seconds
 legendFormat: "{{replica}}"

- title: "Failover Events (24h)"
 targets:
 - expr: increase(themis_hot_spare_failover_count_total[24h])
```

## 19.6A.4 Alerting Rules für v1.4.0-alpha Features

### LLM Alerts:

```

groups:
- name: llm_alerts
 rules:
 # Hohe Fehlerrate
 - alert: LLMHighErrorRate
 expr: rate(themis_llm_requests_total{status="error"}[5m]) > 0.05
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "High LLM error rate on {{ $labels.model }}"
 description: "Error rate is {{ $value | humanizePercentage }}"

 # Cache Hit Rate zu niedrig
 - alert: LLMCacheHitRateLow
 expr: themis_llm_prefix_cache_hit_rate < 0.5
 for: 15m
 labels:
 severity: info
 annotations:
 summary: "Low prefix cache hit rate"
 description: "Hit rate is {{ $value | humanizePercentage }}"

 # GPU Memory Auslastung kritisch
 - alert: GPUMemoryCritical
 expr: themis_llm_gpu_memory_utilization > 0.95
 for: 5m
 labels:
 severity: critical
 annotations:
 summary: "GPU {{ $labels.device }} memory critical"
 description: "Memory utilization is {{ $value | humanizePercentage }}"

 # Batch Queue zu lang
 - alert: LLMBatchQueueLong
 expr: themis_continuous_batching_queue_length > 100
 for: 10m
 labels:
 severity: warning
 annotations:
 summary: "LLM batch queue too long"
 description: "Queue length is {{ $value }}"

```

## HA & Replication Alerts:

```

groups:
 - name: ha_alerts
 rules:
 # Hot Spare Replication Lag zu hoch
 - alert: HotSpareReplicationLagHigh
 expr: themis_hot_spare_replication_lag_seconds > 0.1
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "Hot spare {{ $labels.node }} replication lag high"
 description: "Lag is {{ $value }}s"

 # Hot Spare nicht bereit
 - alert: HotSpareNotReady
 expr: themis_hot_spare_failover_ready == 0
 for: 5m
 labels:
 severity: critical
 annotations:
 summary: "Hot spare {{ $labels.node }} not ready for failover"

 # Shard Down
 - alert: ShardDown
 expr: themis_shard_health == 0
 for: 1m
 labels:
 severity: critical
 annotations:
 summary: "Shard {{ $labels.shard }} is down"
 description: "Immediate attention required"

 # WAL Replication Lag kritisch
 - alert: WALReplicationLagCritical
 expr: themis_wal_replication_lag_seconds > 1.0
 for: 5m
 labels:
 severity: critical
 annotations:
 summary: "WAL replication lag critical on {{ $labels.replica }}"
 description: "Lag is {{ $value }}s"

```

### Performance Optimization Alerts:

```

groups:
 - name: performance_alerts
 rules:
 # Speculative Decoding Akzeptanzrate niedrig
 - alert: SpeculativeDecodingLowAcceptance
 expr: themis_speculative_decoding_acceptance_rate < 0.6
 for: 15m
 labels:
 severity: info
 annotations:
 summary: "Low speculative decoding acceptance rate"
 description: "Consider adjusting draft model or threshold"

 # Batch Fill Rate niedrig
 - alert: ContinuousBatchingLowFillRate
 expr: themis_continuous_batching_batch_fill_rate < 0.4
 for: 15m
 labels:
 severity: info
 annotations:
 summary: "Low batch fill rate"
 description: "Consider adjusting batch size or timeout"

```

### 19.6A.5 Beispiel PromQL Queries

#### LLM Performance Analysis:

```

rate(themis_llm_request_duration_seconds_sum[5m])
/
rate(themis_llm_request_duration_seconds_count[5m])

rate(themis_llm_tokens_generated_total[5m])

increase(themis_llm_prefix_cache_cost_saved_usd[24h])

avg(themis_llm_gpu_utilization_percent)

```

#### Sharding & HA Analysis:

```
avg(themis_hot_spare_replication_lag_seconds)

sum(themis_shard_health)

histogram_quantile(0.95,
 rate(themis_wal_sync_duration_seconds_bucket[5m])
)

sum(increase(themis_shard_rebalance_operations_total[24h]))
```

### Cost Analysis:

```
sum(increase(themis_llm_tokens_generated_total{model="gpt-4"}[30d]))
* 0.03 / 1000

sum(increase(themis_llm_prefix_cache_cost_saved_usd[30d]))
+
sum(increase(themis_llm_response_cache_cost_saved_usd[30d]))
```

## 19.7 Zusammenfassung

ThemisDB bietet ein **Production-Ready Observability-Stack** mit:

- **48 Prometheus-Metriken** für Server, RocksDB, Indexes, Vectors
- **Kumulative Histogramme** für Latenz-Tracking (P50/P95/P99)
- **3 Grafana-Dashboards** für Server-Health, Storage-Health, Vector-Operations
- **OpenTelemetry Distributed Tracing** mit Jaeger/Tempo Integration
- **Alerting** via Prometheus Alertmanager mit PagerDuty/Slack/Email
- **Long-Term Storage** via Thanos mit S3-Backend

**Performance-Overhead:** - **Metrics:** < 1% CPU (Counter-Inkrementen sind atomare Operationen) - **Tracing (aktiviert):** ~5-10 µs pro Span - **Tracing (deaktiviert):** 0 µs (compile-time no-ops)

**Nächste Schritte:** 1. Implementiere Custom-Dashboards für projektspezifische Metriken 2. Konfiguriere Alertmanager-Routing nach Schweregrad 3. Enable Sampling für Tracing in High-Throughput-Szenarien (> 10K QPS) 4. Setze SLI/SLO-Tracking auf (Error Budget Calculations)

---

**Siehe auch:** - [Chapter 8: Storage Layer Deep-Dive](#) - RocksDB Tuning - [Chapter 11: Realtime Data Streaming](#) - CDC Monitoring - [Appendix C: API Reference](#) - /metrics Endpoint - [Prometheus Documentation](#) - [OpenTelemetry Documentation](#) - [Grafana Documentation](#)





# Kapitel 26: Kapitel 19: Monitoring & Observability

---

## Einführung

Monitoring und Observability sind kritische Aspekte für den produktiven Betrieb von ThemisDB. Dieses Kapitel behandelt die Überwachung von Systemmetriken, Application Performance Monitoring (APM), Logging-Strategien, Distributed Tracing und Alerting-Mechanismen.

## 18.1 Monitoring-Architektur

### 18.1.1 Übersicht

Eine umfassende Monitoring-Lösung für ThemisDB umfasst:

- **Metriken-Sammlung:** Prometheus, StatsD, InfluxDB
- **Visualisierung:** Grafana, Kibana
- **Logging:** ELK Stack (Elasticsearch, Logstash, Kibana), Loki
- **Distributed Tracing:** Jaeger, Zipkin
- **Alerting:** Prometheus Alertmanager, PagerDuty

### 18.1.2 Metriken-Typen

**System-Metriken:** - CPU-Auslastung - Speichernutzung (RAM, Swap) - Disk I/O (IOPS, Latenz, Durchsatz) - Netzwerk-Traffic (Bandbreite, Pakete)

**ThemisDB-Metriken:** - Query-Performance (Latenz, Durchsatz) - Connection Pool (aktive Connections, Wartezeit) - Transaction-Statistiken (Commits, Rollbacks) - Cache-Effizienz (Hit Rate, Miss Rate) - Index-Performance - Replikations-Lag

## 18.2 Prometheus Integration

### 18.2.1 Prometheus Setup

**Installation:**

```
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0.li
tar xvfz prometheus-2.45.0.linux-amd64.tar.gz
cd prometheus-2.45.0.linux-amd64

cat > prometheus.yml <<EOF
global:
 scrape_interval: 15s
 evaluation_interval: 15s

scrape_configs:
 - job_name: 'themisdb'
 static_configs:
 - targets: ['localhost:8529']
 metrics_path: '/metrics'
EOF

./prometheus --config.file=prometheus.yml
```

## 18.2.2 ThemisDB Exporter

**Metriken-Endpoint aktivieren:**

```

from prometheus_client import start_http_server, Counter, Gauge, Histogram
from themisdb import ThemisDB
import time

query_counter = Counter('themisdb_queries_total', 'Total number of queries', ['type'])
query_duration = Histogram('themisdb_query_duration_seconds', 'Query duration', ['type'])
active_connections = Gauge('themisdb_active_connections', 'Number of active connections')
cache_hit_rate = Gauge('themisdb_cache_hit_rate', 'Cache hit rate')

db = ThemisDB(host='localhost', port=8529)

def collect_metrics():
 """Sammelt ThemisDB Metriken."""
 while True:
 # Connection-Metriken
 stats = db.statistics()
 active_connections.set(stats['connections']['active'])

 # Cache-Metriken
 cache_stats = db.cache_statistics()
 hit_rate = cache_stats['hits'] / (cache_stats['hits'] + cache_stats['misses'])
 cache_hit_rate.set(hit_rate)

 # Query-Metriken
 query_stats = db.query_statistics()
 for query_type, count in query_stats.items():
 query_counter.labels(type=query_type).inc(count)

 time.sleep(15)

if __name__ == '__main__':
 # Metrics-Server starten
 start_http_server(9090)
 collect_metrics()

```

### 18.2.3 Wichtige Metriken

#### Query Performance:

```
themisdb_query_duration_seconds_bucket{type="SELECT", le="0.1"} 1200
themisdb_query_duration_seconds_bucket{type="SELECT", le="0.5"} 1450
themisdb_query_duration_seconds_bucket{type="SELECT", le="1.0"} 1480
themisdb_query_duration_seconds_bucket{type="SELECT", le="5.0"} 1500

themisdb_queries_total{type="SELECT"} 1500
themisdb_queries_total{type="INSERT"} 500
themisdb_queries_total{type="UPDATE"} 200
```

#### Connection Pool:

```
themisdb_active_connections 45

themisdb_connection_pool_size 100
themisdb_connection_pool_available 55
```

## 18.3 Grafana Dashboards

### 18.3.1 Dashboard Setup

#### Grafana Installation:

```
sudo apt-get install -y software-properties-common
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable main"
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
sudo apt-get update
sudo apt-get install grafana

sudo systemctl start grafana-server
sudo systemctl enable grafana-server
```

### 18.3.2 ThemisDB Dashboard

#### Dashboard-Definition (JSON):

```

{
 "dashboard": {
 "title": "ThemisDB Performance",
 "panels": [
 {
 "title": "Query Latency (p95)",
 "targets": [
 {
 "expr": "histogram_quantile(0.95, themisdb_query_duration_seconds_bucket)"
 }
],
 "type": "graph"
 },
 {
 "title": "Queries per Second",
 "targets": [
 {
 "expr": "rate(themisdb_queries_total[1m])"
 }
],
 "type": "graph"
 },
 {
 "title": "Active Connections",
 "targets": [
 {
 "expr": "themisdb_active_connections"
 }
],
 "type": "graph"
 },
 {
 "title": "Cache Hit Rate",
 "targets": [
 {
 "expr": "themisdb_cache_hit_rate"
 }
],
 "type": "gauge"
 }
]
 }
}

```

### 18.3.3 Dashboard-Beispiele

**System-Übersicht Dashboard:** - CPU-Auslastung (pro Core) - Speichernutzung (RAM, Swap, Cache) - Disk I/O (Read/Write IOPS, Latenz) - Netzwerk-Traffic (In/Out Bandwidth)

**Query-Performance Dashboard:** - Query Latenz (p50, p95, p99) - Queries per Second (nach Typ) - Slow Queries (>1s) - Query Error Rate

**Cluster-Status Dashboard:** - Node Health Status - Replikations-Lag - Cluster-Topology - Failover-Events

## 18.4 Logging

### 18.4.1 Logging-Strategie

**Log-Levels:**

```
import logging

logging.basicConfig(
 level=logging.INFO,
 format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
 handlers=[
 logging.FileHandler('/var/log/themisdb/themisdb.log'),
 logging.StreamHandler()
]
)

logger = logging.getLogger('themisdb')

logger.debug("Connection pool initialized") # DEBUG
logger.info("Query executed successfully") # INFO
logger.warning("High memory usage detected") # WARNING
logger.error("Query execution failed") # ERROR
logger.critical("Database connection lost") # CRITICAL
```

### 18.4.2 Strukturiertes Logging

**JSON-Logging:**

```

import json
import logging

class JSONFormatter(logging.Formatter):
 def format(self, record):
 log_data = {
 'timestamp': self.formatTime(record),
 'level': record.levelname,
 'logger': record.name,
 'message': record.getMessage(),
 'module': record.module,
 'function': record.funcName,
 'line': record.lineno
 }

 # Zusätzliche Felder
 if hasattr(record, 'query_id'):
 log_data['query_id'] = record.query_id
 if hasattr(record, 'user_id'):
 log_data['user_id'] = record.user_id
 if hasattr(record, 'duration_ms'):
 log_data['duration_ms'] = record.duration_ms

 return json.dumps(log_data)

handler = logging.FileHandler('/var/log/themisdb/themisdb.json')
handler.setFormatter(JSONFormatter())
logger.addHandler(handler)

logger.info(
 "Query executed",
 extra={
 'query_id': 'q123',
 'user_id': 'u456',
 'duration_ms': 45.2
 }
)

```

### 18.4.3 ELK Stack Integration

#### Filebeat Konfiguration:



```
filebeat.inputs:
 - type: log
 enabled: true
 paths:
 - /var/log/themisdb/*.log
 json.keys_under_root: true
 json.add_error_key: true

output.elasticsearch:
 hosts: ["localhost:9200"]
 index: "themisdb-logs-%{+yyyy.MM.dd}"

setup.kibana:
 host: "localhost:5601"
```

### Logstash Pipeline:

```

input {
 beats {
 port => 5044
 }
}

filter {
 if [type] == "themisdb" {
 json {
 source => "message"
 }

 # Query-Latenz berechnen
 if [duration_ms] {
 ruby {
 code => "event.set('duration_s', event.get('duration_ms') / 1000.0)"
 }
 }

 # Geo-IP Enrichment
 geoip {
 source => "client_ip"
 target => "geoip"
 }
 }
}

output {
 elasticsearch {
 hosts => ["localhost:9200"]
 index => "themisdb-logs-%{+YYYY.MM.dd}"
 }
}

```

## 18.5 Distributed Tracing

### 18.5.1 Jaeger Integration

**Jaeger Setup:**

```
docker run -d \
 -e COLLECTOR_ZIPKIN_HTTP_PORT=9411 \
 -p 5775:5775/udp \
 -p 6831:6831/udp \
 -p 6832:6832/udp \
 -p 5778:5778 \
 -p 16686:16686 \
 -p 14268:14268 \
 -p 9411:9411 \
 jaegertracing/all-in-one:latest
```

## 18.5.2 OpenTelemetry Integration

### Tracing-Code:

```

from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.instrumentation.requests import RequestsInstrumentor

trace.set_tracer_provider(TracerProvider())
tracer = trace.get_tracer(__name__)

jaeger_exporter = JaegerExporter(
 agent_host_name='localhost',
 agent_port=6831,
)

trace.get_tracer_provider().add_span_processor(
 BatchSpanProcessor(jaeger_exporter)
)

RequestsInstrumentor().instrument()

@tracer.start_as_current_span("execute_query")
def execute_query(query):
 span = trace.get_current_span()
 span.set_attribute("query.type", "SELECT")
 span.set_attribute("query.text", query)

 # Query ausführen
 with tracer.start_as_current_span("database_query"):
 result = db.execute(query)

 span.set_attribute("result.count", len(result))
 return result

@tracer.start_as_current_span("process_order")
def process_order(order_id):
 # Span 1: Order validieren
 with tracer.start_as_current_span("validate_order"):
 order = db.query(f"SELECT * FROM orders WHERE id = {order_id}")

 # Span 2: Inventory prüfen
 with tracer.start_as_current_span("check_inventory"):
 inventory = db.query(f"SELECT * FROM inventory WHERE product_id = {order['product_id']}")

 # Span 3: Payment verarbeiten
 with tracer.start_as_current_span("process_payment"):
 payment = process_payment_service(order)

 return {"status": "success"}

```

## **18.6 Alerting**

### **18.6.1 Prometheus Alertmanager**

**Alert-Regeln:**

```

groups:
- name: themisdb
 interval: 30s
 rules:
 # High Query Latency
 - alert: HighQueryLatency
 expr: histogram_quantile(0.95, themisdb_query_duration_seconds_bucket) > 1.0
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "High query latency detected"
 description: "p95 query latency is {{ $value }}s"

 # Low Cache Hit Rate
 - alert: LowCacheHitRate
 expr: themisdb_cache_hit_rate < 0.7
 for: 10m
 labels:
 severity: warning
 annotations:
 summary: "Low cache hit rate"
 description: "Cache hit rate is {{ $value }}"

 # High Connection Usage
 - alert: HighConnectionUsage
 expr: (themisdb_active_connections / themisdb_connection_pool_size) > 0.8
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "Connection pool nearly exhausted"
 description: "{{ $value | humanizePercentage }} of connections in use"

 # Database Down
 - alert: DatabaseDown
 expr: up{job="themisdb"} == 0
 for: 1m
 labels:
 severity: critical
 annotations:
 summary: "ThemisDB instance is down"
 description: "Instance {{ $labels.instance }} is unreachable"

 # Replication Lag
 - alert: HighReplicationLag
 expr: themisdb_replication_lag_seconds > 10
 for: 5m
 labels:

```

```

 severity: warning
 annotations:
 summary: "High replication lag"
 description: "Replication lag is {{ $value }}s"

```

## 18.6.2 Alertmanager Konfiguration

### Alertmanager Config:

```

global:
 resolve_timeout: 5m

route:
 group_by: ['alertname', 'cluster']
 group_wait: 10s
 group_interval: 10s
 repeat_interval: 12h
 receiver: 'team-db'

routes:
 - match:
 severity: critical
 receiver: 'pagerduty'

 - match:
 severity: warning
 receiver: 'slack'

receivers:
 - name: 'team-db'
 email_configs:
 - to: 'db-team@company.com'
 from: 'alertmanager@company.com'
 smarthost: 'smtp.company.com:587'

 - name: 'pagerduty'
 pagerduty_configs:
 - service_key: 'YOUR_PAGERDUTY_KEY'

 - name: 'slack'
 slack_configs:
 - api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'
 channel: '#db-alerts'
 text: '{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}'

```

## **18.7 Application Performance Monitoring (APM)**

### **18.7.1 Query Performance Tracking**

**Slow Query Logging:**



```

import time
from functools import wraps

SLOW_QUERY_THRESHOLD = 1.0 # Sekunden

def track_query_performance(func):
 @wraps(func)
 def wrapper(*args, **kwargs):
 start_time = time.time()
 query = args[0] if args else kwargs.get('query')

 try:
 result = func(*args, **kwargs)
 duration = time.time() - start_time

 # Slow Query loggen
 if duration > SLOW_QUERY_THRESHOLD:
 logger.warning(
 "Slow query detected",
 extra={
 'query': query,
 'duration_ms': duration * 1000,
 'threshold_ms': SLOW_QUERY_THRESHOLD * 1000
 }
)

 # Metriken updaten
 query_duration.labels(type=get_query_type(query)).observe(duration)

 return result
 except Exception as e:
 duration = time.time() - start_time
 logger.error(
 "Query execution failed",
 extra={
 'query': query,
 'duration_ms': duration * 1000,
 'error': str(e)
 }
)
 raise

 return wrapper

@track_query_performance
def execute_query(query):
 return db.execute(query)

```

## **18.7.2 Transaction Monitoring**

### **Transaction Tracking:**

```

class TransactionMonitor:
 def __init__(self):
 self.active_transactions = {}
 self.transaction_counter = Counter('themisdb_transactions_total',
 'Total transactions',
 ['status'])
 self.transaction_duration = Histogram('themisdb_transaction_duration_seconds',
 'Transaction duration')

 def begin_transaction(self, txn_id):
 self.active_transactions[txn_id] = {
 'start_time': time.time(),
 'queries': []
 }

 def add_query(self, txn_id, query):
 if txn_id in self.active_transactions:
 self.active_transactions[txn_id]['queries'].append(query)

 def commit_transaction(self, txn_id):
 if txn_id in self.active_transactions:
 duration = time.time() - self.active_transactions[txn_id]['start_time']
 query_count = len(self.active_transactions[txn_id]['queries'])

 self.transaction_counter.labels(status='committed').inc()
 self.transaction_duration.observe(duration)

 logger.info(
 "Transaction committed",
 extra={
 'txn_id': txn_id,
 'duration_ms': duration * 1000,
 'query_count': query_count
 }
)

 del self.active_transactions[txn_id]

 def rollback_transaction(self, txn_id):
 if txn_id in self.active_transactions:
 duration = time.time() - self.active_transactions[txn_id]['start_time']

 self.transaction_counter.labels(status='rolled_back').inc()

 logger.warning(
 "Transaction rolled back",
 extra={
 'txn_id': txn_id,
 'duration_ms': duration * 1000
 }
)

```

```
 }
)

 del self.active_transactions[txn_id]
```

## 18.8 Health Checks

### 18.8.1 Liveness & Readiness Probes

**Health Check Endpoints:**

```

from flask import Flask, jsonify
import time

app = Flask(__name__)

@app.route('/health/live')
def liveness():
 """Prüft ob die Anwendung läuft."""
 return jsonify({'status': 'ok'}), 200

@app.route('/health/ready')
def readiness():
 """Prüft ob die Anwendung Requests bearbeiten kann."""
 try:
 # Datenbankverbindung testen
 db.execute("SELECT 1")

 # Connection Pool prüfen
 stats = db.statistics()
 if stats['connections']['available'] == 0:
 return jsonify({
 'status': 'not_ready',
 'reason': 'Connection pool exhausted'
 }), 503

 # Replikations-Lag prüfen
 if stats.get('replication_lag', 0) > 30:
 return jsonify({
 'status': 'not_ready',
 'reason': 'High replication lag'
 }), 503

 return jsonify({'status': 'ready'}), 200

 except Exception as e:
 return jsonify({
 'status': 'not_ready',
 'reason': str(e)
 }), 503

@app.route('/health/startup')
def startup():
 """Prüft ob die Anwendung vollständig gestartet ist."""
 # Initialisierungs-Checks
 checks = {
 'database_connected': check_db_connection(),
 'cache_initialized': check_cache(),
 'migrations_applied': check_migrations()
 }

```

```

if all(checks.values()):
 return jsonify({'status': 'started', 'checks': checks}), 200
else:
 return jsonify({'status': 'starting', 'checks': checks}), 503

```

## 18.8.2 Kubernetes Probes

### Pod-Konfiguration:

```

apiVersion: v1
kind: Pod
metadata:
 name: themisdb
spec:
 containers:
 - name: themisdb
 image: themisdb:latest
 ports:
 - containerPort: 8529

 livenessProbe:
 httpGet:
 path: /health/live
 port: 8529
 initialDelaySeconds: 30
 periodSeconds: 10
 timeoutSeconds: 5
 failureThreshold: 3

 readinessProbe:
 httpGet:
 path: /health/ready
 port: 8529
 initialDelaySeconds: 10
 periodSeconds: 5
 timeoutSeconds: 3
 failureThreshold: 3

 startupProbe:
 httpGet:
 path: /health/startup
 port: 8529
 initialDelaySeconds: 0
 periodSeconds: 10
 timeoutSeconds: 3
 failureThreshold: 30

```

## 18.9 Performance Profiling

### 18.9.1 Query Profiling

#### EXPLAIN ANALYZE:

```
-- Query Plan analysieren
EXPLAIN ANALYZE
SELECT c.name, COUNT(o.id) as order_count
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE c.country = 'DE'
GROUP BY c.name
ORDER BY order_count DESC
LIMIT 10;

-- Output:
-- -> Limit (cost=1245.34..1245.36 rows=10)
-- -> Sort (cost=1245.34..1276.42 rows=12432)
-- Sort Key: order_count DESC
-- -> HashAggregate (cost=956.23..1080.55 rows=12432)
-- -> Hash Left Join (cost=345.67..876.12 rows=16040)
-- Hash Cond: (c.id = o.customer_id)
-- -> Index Scan using idx_customers_country on customers c
-- Index Cond: (country = 'DE')
-- -> Hash (cost=234.56..234.56 rows=8884)
-- -> Seq Scan on orders o
```

### 18.9.2 Python Profiling

#### cProfile Integration:

```

import cProfile
import pstats
from pstats import SortKey

def profile_query(query):
 profiler = cProfile.Profile()
 profiler.enable()

 # Query ausführen
 result = db.execute(query)

 profiler.disable()

 # Statistiken ausgeben
 stats = pstats.Stats(profiler)
 stats.sort_stats(SortKey.CUMULATIVE)
 stats.print_stats(10)

 return result

from line_profiler import LineProfiler

@profile
def complex_operation():
 # Schritt 1
 data = db.query("SELECT * FROM large_table")

 # Schritt 2
 processed = [process_row(row) for row in data]

 # Schritt 3
 db.bulk_insert("results", processed)

from memory_profiler import profile as memory_profile

@memory_profile
def memory_intensive_operation():
 large_dataset = db.query("SELECT * FROM huge_table")
 return process_dataset(large_dataset)

```

## 18.10 Best Practices

### 18.10.1 Monitoring-Strategie

#### Golden Signals:

1. **Latency:** Zeit für Request-Bearbeitung



2. **Traffic:** Anzahl Requests pro Zeiteinheit
3. **Errors:** Rate fehlgeschlagener Requests
4. **Saturation:** Auslastung der Ressourcen

**RED Method (für Services):** - **Rate:** Requests pro Sekunde - **Errors:** Fehlerrate - **Duration:** Latenz-Verteilung

**USE Method (für Ressourcen):** - **Utilization:** Prozentuale Auslastung - **Saturation:** Wartezeit/Queueing - **Errors:** Fehlerzähler

## 18.10.2 Retention Policies

**Datenaufbewahrung:**

```
prometheus:
 retention:
 time: 15d
 size: 50GB

 recording_rules:
 - name: themisdb_5m
 interval: 5m
 rules:
 - record: themisdb:query_rate:5m
 expr: rate(themisdb_queries_total[5m])

 - record: themisdb:query_latency_p95:5m
 expr: histogram_quantile(0.95, themisdb_query_duration_seconds_bucket[5m])
```

**Log-Rotation:**

```
/var/log/themisdb/*.log {
 daily
 rotate 7
 compress
 delaycompress
 missingok
 notifempty
 create 0640 themisdb themisdb
 sharedscripts
 postrotate
 systemctl reload themisdb
 endsript
}
```

---

### 18.10.3 Dashboard-Design

#### Effektive Dashboards:

1. **Übersichts-Dashboard:** Hochlevel-Metriken, Status-Indikatoren
2. **Detail-Dashboards:** Tiefere Einblicke für spezifische Bereiche
3. **Drill-Down:** Von Übersicht zu Details navigieren
4. **Zeitfenster:** Flexible Zeitbereichs-Auswahl
5. **Annotations:** Deployment-Marker, Incidents

**Dashboard-Struktur:** - **Oberste Zeile:** SLI/SLO Status, Alerts - **Zweite Zeile:** Golden Signals (Latency, Traffic, Errors, Saturation) - **Weitere Zeilen:** Spezifische Metriken nach Kategorie

## Zusammenfassung

Monitoring und Observability sind essentiell für den Betrieb von ThemisDB:

- **Metriken:** Prometheus für Sammlung und Speicherung
- **Visualisierung:** Grafana für Dashboards
- **Logging:** ELK Stack für Log-Aggregation und Analyse
- **Tracing:** Jaeger/OpenTelemetry für Distributed Tracing
- **Alerting:** Proaktive Benachrichtigung bei Problemen
- **Health Checks:** Kubernetes-Integration für Orchestrierung
- **Profiling:** Performance-Analyse und Optimierung

Mit diesen Tools und Praktiken können Sie ThemisDB effektiv überwachen und Probleme frühzeitig erkennen.

---

# Kapitel 27: Performance Tuning

---

## Übersicht

Dieses Kapitel behandelt Performance-Optimierungen für ThemisDB.

### Hauptthemen

- Query-Optimierung
- Index-Tuning
- Cache-Strategien
- Metriken und Benchmarking

flowchart TD

```
Start[Performance Problem] --> Measure[Messung & Profiling]
Measure --> Analyze{Bottleneck?}
```

```
Analyze -->|Query| QueryOpt[Query Optimierung]
Analyze -->|Index| IndexOpt[Index Tuning]
Analyze -->|Memory| MemOpt[Memory Optimization]
Analyze -->|I/O| IOOpt[I/O Optimization]
```

```
QueryOpt --> UseIndex[Index nutzen]
QueryOpt --> FilterEarly[Filter früh anwenden]
QueryOpt --> Projection[Projektion reduzieren]
```

```
IndexOpt --> PersistentIdx[Persistent Index]
IndexOpt --> GeoIdx[Geo Index]
IndexOpt --> VectorIdx[Vector Index]
```

```
MemOpt --> CacheSize[Cache-Größe erhöhen]
MemOpt --> BatchSize[Batch-Size optimieren]
```

```
IOOpt --> RocksDB[RocksDB Tuning]
IOOpt --> SSD[SSD statt HDD]
```

```
UseIndex --> Benchmark
FilterEarly --> Benchmark
Projection --> Benchmark
PersistentIdx --> Benchmark
GeoIdx --> Benchmark
VectorIdx --> Benchmark
CacheSize --> Benchmark
BatchSize --> Benchmark
RocksDB --> Benchmark
SSD --> Benchmark
```

```
Benchmark[Benchmark] --> Check{Verbessert?}
Check -->|Ja| Monitor[Monitoring]
Check -->|Nein| Measure
```

```
Monitor --> Done[Performance OK]
```

```
style Start fill:#ff6b6b
style Done fill:#51cf66
style Benchmark fill:#ffd43b
```

## Weitere Informationen

Siehe auch: [Query Optimierung](#), [Monitoring](#)



# Kapitel 28: Kapitel 20: Backup & Recovery

## Einführung

Backup und Recovery sind kritische Komponenten für jede Produktionsumgebung. Dieses Kapitel behandelt umfassende Strategien für Datensicherung, Wiederherstellung und Disaster Recovery mit ThemisDB.

## 19.1 Backup-Strategien

### 19.1.1 Vollständige Backups

**Konzept:** Ein vollständiges Backup erstellt eine komplette Kopie aller Daten zu einem bestimmten Zeitpunkt.

**Durchführung:**

```
themisdb-backup --type full --output /backups/full_$(date +%Y%m%d).tar.gz
themisdb-backup --type full --compress gzip --output /backups/full.tar.gz
```

**Vorteile:** - Einfache Wiederherstellung - Vollständige Datenkonsistenz - Einfaches Backup-Management

**Nachteile:** - Hoher Speicherbedarf - Längere Backup-Dauer - Höhere I/O-Last

### 19.1.2 Inkrementelle Backups

**Konzept:** Nur Änderungen seit dem letzten Backup werden gesichert.

**Implementierung:**

```
themisdb-backup --type full --output /backups/base.tar.gz

themisdb-backup --type incremental \
 --base /backups/base.tar.gz \
 --output /backups/incr_$(date +%Y%m%d_%H%M).tar.gz
```

**Vorteile:** - Schnellere Backup-Erstellung - Geringerer Speicherbedarf - Weniger I/O-Last

**Nachteile:** - Komplexere Wiederherstellung - Abhängigkeit von vorherigen Backups

### 19.1.3 Differenzielle Backups

**Konzept:** Alle Änderungen seit dem letzten vollständigen Backup.

```
themisdb-backup --type differential \
--base /backups/full.tar.gz \
--output /backups/diff_$(date +%Y%m%d).tar.gz
```

```
gantt
title Backup Strategy Timeline
dateFormat YYYY-MM-DD

section Full Backups
Full Backup 1 :done, full1, 2024-01-01, 1d
Full Backup 2 :done, full2, 2024-01-08, 1d
Full Backup 3 :active, full3, 2024-01-15, 1d

section Incremental
Incremental Day 2 :done, inc1, 2024-01-02, 1d
Incremental Day 3 :done, inc2, 2024-01-03, 1d
Incremental Day 4 :done, inc3, 2024-01-04, 1d
Incremental Day 5 :done, inc4, 2024-01-05, 1d
Incremental Day 6 :done, inc5, 2024-01-06, 1d
Incremental Day 7 :done, inc6, 2024-01-07, 1d

section Differential
Differential Day 2-7 :done, diff1, 2024-01-09, 6d
```

**Abb. 20.1:** Backup-Strategy-Overview

```

flowchart LR
 subgraph "Backup Flow"
 Data["(Production Data)"] --> Check{Backup Type?}

 Check -->|Full| Full["Full Backup
All Data"]
 Check -->|Incremental| Inc["Incremental
Changes since last backup"]
 Check -->|Differential| Diff["Differential
Changes since last full"]

 Full --> Compress["Compress
gzip/zstd"]
 Inc --> Compress
 Diff --> Compress

 Compress --> Encrypt["Encrypt
AES-256"]
 Encrypt --> Storage["(Backup Storage
S3/Local/Network)"]

 Storage --> Verify["Verify
Checksum"]
 Verify --> Retention{Retention Policy}

 Retention -->|Keep| Archive["Archive Storage"]
 Retention -->|Delete| Remove["Delete Old Backups"]
 end

 style Data fill:#667eea
 style Full fill:#43e97b
 style Inc fill:#4facfe
 style Diff fill:#f093fb
 style Storage fill:#ffd32a
 style Archive fill:#95e1d3

```

Abb. 20.2: Point-in-Time-Recovery-Timeline

## 19.2 Backup-Mechanismen

### 19.2.1 Physische Backups

**RocksDB SST-Dateien:**



```

import shutil
from pathlib import Path

def physical_backup(data_dir, backup_dir):
 """Erstellt physisches Backup der RocksDB-Dateien"""
 # Checkpoint erstellen (konsistenter Snapshot)
 checkpoint_dir = Path(backup_dir) / "checkpoint"

 # ThemisDB Checkpoint API
 client.admin.create_checkpoint(str(checkpoint_dir))

 # Backup mit rsync
 import subprocess
 subprocess.run([
 "rsync", "-av", "--delete",
 str(checkpoint_dir) + "/",
 str(backup_dir) + "/"
])

 print(f"Physical backup completed: {backup_dir}")

```

**Vorteile:** - Sehr schnelle Backups - Minimale Auswirkungen auf die Performance - Byte-genaue Kopien

## 19.2.2 Logische Backups

**Datenexport:**

```
def logical_backup(client, backup_file):
 """Exportiert Daten als logisches Backup"""
 import json

 backup_data = {
 'version': '1.3.4',
 'timestamp': datetime.now().isoformat(),
 'collections': {}
 }

 # Alle Collections exportieren
 collections = client.list_collections()
 for coll_name in collections:
 print(f"Backing up collection: {coll_name}")

 # Alle Dokumente exportieren
 docs = list(client.collection(coll_name).find({}))
 backup_data['collections'][coll_name] = docs

 # Als JSON speichern
 with open(backup_file, 'w') as f:
 json.dump(backup_data, f, indent=2)

 print(f"Logical backup completed: {backup_file}")
```

**Vorteile:** - Plattformunabhängig - Lesbare Datenformate - Einfache Teilwiederherstellung

### 19.2.3 Snapshot-basierte Backups

**Mit Dateisystem-Snapshots:**

```
lvcreate --snapshot --name themisdb_snap --size 10G /dev/vg0/themisdb

tar czf /backups/snapshot_$(date +%Y%m%d).tar.gz \
 /mnt/themisdb_snap/

lvremove /dev/vg0/themisdb_snap
```

**AWS EBS Snapshots:**

```

import boto3

def create_ebs_snapshot(volume_id, description):
 """Erstellt EBS Snapshot"""
 ec2 = boto3.client('ec2')

 snapshot = ec2.create_snapshot(
 VolumeId=volume_id,
 Description=description,
 TagSpecifications=[{
 'ResourceType': 'snapshot',
 'Tags': [
 {'Key': 'Name', 'Value': f'themisdb-backup-{datetime.now().strftime("%Y%m%d%H%M%S")}'},
 {'Key': 'Type', 'Value': 'automated'}
]
 }]
)

 return snapshot['SnapshotId']

```

## 19.3 Point-in-Time Recovery (PITR)

### 19.3.1 Binlog-basierte Recovery

**Konzept:** Kombination aus vollständigem Backup und Transaction Logs.

**Aktivierung:**

```

storage:
 enable_binlog: true
 binlog_dir: /var/lib/themisdb/binlog
 binlog_rotation: 100MB
 binlog_retention: 7d

```

**Recovery-Prozess:**

```
def point_in_time_recovery(backup_file, target_time):
 """
 Wiederherstellung zu einem bestimmten Zeitpunkt
 """
 # 1. Basis-Backup wiederherstellen
 restore_backup(backup_file)

 # 2. Binlogs bis zum Ziel-Zeitpunkt anwenden
 binlog_dir = Path("/var/lib/themisdb/binlog")

 for binlog in sorted(binlog_dir.glob("binlog.*")):
 # Binlog-Einträge bis target_time anwenden
 apply_binlog(binlog, target_time)

 print(f"PITR completed to: {target_time}")
```

### 19.3.2 MVCC-basierte Recovery

**Mit Snapshot Isolation:**

```
def recover_to_timestamp(timestamp):
 """
 Nutzt MVCC für zeitpunktgenaue Wiederherstellung
 """
 # Query mit AS OF SYSTEM TIME
 result = client.query(f"""
 SELECT * FROM orders
 AS OF SYSTEM TIME '{timestamp}'
 """)

 return result
```

## 19.4 Backup-Automatisierung

### 19.4.1 Cron-basierte Backups

**Backup-Script:**

```
#!/bin/bash

BACKUP_DIR=/backups/themisdb
DATE=$(date +%Y%m%d_%H%M)
LOG_FILE=/var/log/themisdb-backup.log

if [$(date +%u) -eq 7]; then
 echo "[$(date)] Starting full backup" >> $LOG_FILE
 themisdb-backup --type full --output $BACKUP_DIR/full_$DATE.tar.gz
else
 echo "[$(date)] Starting incremental backup" >> $LOG_FILE
 themisdb-backup --type incremental --output $BACKUP_DIR/incr_$DATE.tar.gz
fi

find $BACKUP_DIR -name "*.tar.gz" -mtime +30 -delete

echo "[$(date)] Backup completed" >> $LOG_FILE
```

#### **Crontab-Eintrag:**

```
0 2 * * * /usr/local/bin/themisdb-backup.sh

0 3 * * 0 /usr/local/bin/themisdb-full-backup.sh
```

### **19.4.2 Python Backup-Manager**

#### **Automatisiertes Backup-Management:**

```

import schedule
import time
from datetime import datetime, timedelta
from pathlib import Path

class BackupManager:
 def __init__(self, backup_dir, retention_days=30):
 self.backup_dir = Path(backup_dir)
 self.retention_days = retention_days

 def full_backup(self):
 """Vollständiges Backup"""
 timestamp = datetime.now().strftime("%Y%m%d_%H%M")
 backup_file = self.backup_dir / f"full_{timestamp}.tar.gz"

 print(f"Starting full backup: {backup_file}")
 # Backup durchführen

 def incremental_backup(self):
 """Inkrementelles Backup"""
 timestamp = datetime.now().strftime("%Y%m%d_%H%M")
 backup_file = self.backup_dir / f"incr_{timestamp}.tar.gz"

 print(f"Starting incremental backup: {backup_file}")
 # Backup durchführen

 def cleanup_old_backups(self):
 """Löscht alte Backups"""
 cutoff = datetime.now() - timedelta(days=self.retention_days)

 for backup in self.backup_dir.glob("*.tar.gz"):
 if datetime.fromtimestamp(backup.stat().st_mtime) < cutoff:
 backup.unlink()
 print(f"Deleted old backup: {backup}")

 def start_scheduler(self):
 """Startet Backup-Scheduler"""
 # Tägliche inkrementelle Backups um 2:00
 schedule.every().day.at("02:00").do(self.incremental_backup)

 # Wöchentliche vollständige Backups (Sonntag 3:00)
 schedule.every().sunday.at("03:00").do(self.full_backup)

 # Tägliche Cleanup um 4:00
 schedule.every().day.at("04:00").do(self.cleanup_old_backups)

 print("Backup scheduler started")
 while True:
 schedule.run_pending()

```

```

time.sleep(60)

manager = BackupManager("/backups/themisdb", retention_days=30)
manager.start_scheduler()

```

## 19.5 Backup-Verifizierung

### 19.5.1 Backup-Integrität prüfen

#### Checksummen-Validierung:

```

import hashlib

def verify_backup(backup_file):
 """Überprüft Backup-Integrität"""
 # Checksumme berechnen
 sha256 = hashlib.sha256()

 with open(backup_file, 'rb') as f:
 while chunk := f.read(8192):
 sha256.update(chunk)

 checksum = sha256.hexdigest()

 # Mit gespeicherter Checksumme vergleichen
 checksum_file = f"{backup_file}.sha256"
 if Path(checksum_file).exists():
 with open(checksum_file) as f:
 expected = f.read().strip()

 if checksum == expected:
 print(f"✓ Backup integrity verified: {backup_file}")
 return True
 else:
 print(f"✗ Backup corrupted: {backup_file}")
 return False
 else:
 # Checksumme speichern
 with open(checksum_file, 'w') as f:
 f.write(checksum)
 return True

```

### 19.5.2 Wiederherstellungs-Tests

#### Automatisierte Test-Recovery:

```

def test_backup_restore(backup_file, test_dir="/tmp/restore_test"):
 """Testet Backup-Wiederherstellung"""
 import shutil
 from pathlib import Path

 test_path = Path(test_dir)
 test_path.mkdir(exist_ok=True)

 try:
 # Backup wiederherstellen
 restore_backup(backup_file, test_path)

 # ThemisDB mit Test-Daten starten
 test_client = ThemisDBClient(data_dir=test_path)

 # Basis-Tests durchführen
 collections = test_client.list_collections()
 assert len(collections) > 0, "No collections found"

 # Sample-Queries testen
 result = test_client.query("SELECT COUNT(*) FROM orders")
 assert result[0]['count'] > 0, "No data found"

 print(f"✓ Restore test passed: {backup_file}")
 return True

 except Exception as e:
 print(f"✗ Restore test failed: {e}")
 return False

 finally:
 # Cleanup
 shutil.rmtree(test_path, ignore_errors=True)

```

## 19.6 Wiederherstellung

### 19.6.1 Vollständige Wiederherstellung

**Restore-Prozess:**



```
#!/bin/bash

systemctl stop themisdb

mv /var/lib/themisdb /var/lib/themisdb.old

tar xzf /backups/full_20231215.tar.gz -C /var/lib/themisdb

chown -R themisdb:themisdb /var/lib/themisdb

systemctl start themisdb

themisdb-admin validate
```

## 19.6.2 Selektive Wiederherstellung

### Einzelne Collection wiederherstellen:

```
def restore_collection(backup_file, collection_name, target_db):
 """Stellt einzelne Collection wieder her"""
 import json

 # Backup laden
 with open(backup_file) as f:
 backup_data = json.load(f)

 # Collection-Daten extrahieren
 if collection_name not in backup_data['collections']:
 raise ValueError(f"Collection {collection_name} not found in backup")

 docs = backup_data['collections'][collection_name]

 # In Ziel-DB einfügen
 target_client = ThemisDBClient(target_db)
 coll = target_client.collection(collection_name)

 # Batch-Insert
 batch_size = 1000
 for i in range(0, len(docs), batch_size):
 batch = docs[i:i+batch_size]
 coll.insert_many(batch)

 print(f"Restored {len(docs)} documents to {collection_name}")
```

## 19.7 Disaster Recovery

### 19.7.1 DR-Strategie

**Recovery Time Objective (RTO):** - Maximale akzeptable Ausfallzeit - Ziel: < 4 Stunden

**Recovery Point Objective (RPO):** - Maximaler Datenverlust - Ziel: < 15 Minuten

**DR-Plan:**

```
disaster_recovery:
 primary_site:
 location: "datacenter-1"
 backup_frequency: "hourly"

 secondary_site:
 location: "datacenter-2"
 replication: "synchronous"
 standby_mode: "hot"

 backup_locations:
 - "s3://backups-bucket/themisdb/"
 - "azure://backups/themisdb/"

 procedures:
 - name: "Site Failover"
 steps:
 - "Verify primary site failure"
 - "Promote secondary to primary"
 - "Update DNS/Load Balancer"
 - "Verify application connectivity"
```

### 19.7.2 Geo-Replikation

**Multi-Region Setup:**

```
master_config = {
 'replication': {
 'role': 'master',
 'replicas': [
 'themisdb-replica-eu:8529',
 'themisdb-replica-us:8529'
]
 }
}

replica_config = {
 'replication': {
 'role': 'replica',
 'master': 'themisdb-master-eu:8529',
 'replication_lag_alert': '30s'
 }
}
```

## 19.8 Backup zu Cloud-Storage

### 19.8.1 AWS S3

Upload zu S3:

```

import boto3
from pathlib import Path

def upload_to_s3(backup_file, bucket, prefix="backups/"):
 """Lädt Backup zu AWS S3 hoch"""
 s3 = boto3.client('s3')

 key = f"{prefix}{Path(backup_file).name}"

 # Upload mit Progress
 s3.upload_file(
 backup_file,
 bucket,
 key,
 Callback=ProgressPercentage(backup_file)
)

 print(f"Uploaded to s3://{bucket}/{key}")

Lifecycle-Policy für automatisches Löschen
s3.put_bucket_lifecycle_configuration(
 Bucket=bucket,
 LifecycleConfiguration={
 'Rules': [{
 'Id': 'DeleteOldBackups',
 'Status': 'Enabled',
 'Prefix': prefix,
 'Expiration': {'Days': 90}
 }]
 }
)

```

## 19.8.2 Azure Blob Storage

### Upload zu Azure:

```
from azure.storage.blob import BlobServiceClient

def upload_to_azure(backup_file, connection_string, container):
 """Lädt Backup zu Azure Blob Storage"""
 blob_service = BlobServiceClient.from_connection_string(connection_string)

 blob_name = Path(backup_file).name
 blob_client = blob_service.get_blob_client(container, blob_name)

 with open(backup_file, 'rb') as data:
 blob_client.upload_blob(data, overwrite=True)

 print(f"Uploaded to Azure: {container}/{blob_name}")
```

### 19.8.3 Google Cloud Storage

#### Upload zu GCS:

```
from google.cloud import storage

def upload_to_gcs(backup_file, bucket_name, destination_blob):
 """Lädt Backup zu Google Cloud Storage"""
 client = storage.Client()
 bucket = client.bucket(bucket_name)
 blob = bucket.blob(destination_blob)

 blob.upload_from_filename(backup_file)

 print(f"Uploaded to gs://{bucket_name}/{destination_blob}")
```

## 19.9 Best Practices

### 19.9.1 3-2-1 Backup-Regel

**Prinzip:** - 3 Kopien der Daten - 2 verschiedene Medientypen - 1 Kopie off-site

#### Implementierung:

```

backup_strategy = {
 'copies': [
 {'type': 'local', 'location': '/backups/local'},
 {'type': 'nas', 'location': '/mnt/nas/backups'},
 {'type': 'cloud', 'location': 's3://backups/themisdb'}
],
 'media_types': ['disk', 'cloud'],
 'offsite': True
}

```

## 19.9.2 Backup-Verschlüsselung

### Verschlüsselte Backups:

```

from cryptography.fernet import Fernet

def encrypted_backup(source_file, output_file, key_file):
 """Erstellt verschlüsseltes Backup"""
 # Key laden oder generieren
 if Path(key_file).exists():
 with open(key_file, 'rb') as f:
 key = f.read()
 else:
 key = Fernet.generate_key()
 with open(key_file, 'wb') as f:
 f.write(key)

 fernet = Fernet(key)

 # Daten verschlüsseln
 with open(source_file, 'rb') as f:
 data = f.read()

 encrypted = fernet.encrypt(data)

 with open(output_file, 'wb') as f:
 f.write(encrypted)

 print(f"Encrypted backup created: {output_file}")

```

## 19.9.3 Backup-Monitoring

### Monitoring-Integration:

```

from prometheus_client import Counter, Histogram, Gauge

backup_counter = Counter('themisdb_backups_total', 'Total backups', ['type', 'status'])
backup_duration = Histogram('themisdb_backup_duration_seconds', 'Backup duration')
backup_size = Gauge('themisdb_backup_size_bytes', 'Backup size')
last_backup = Gauge('themisdb_last_backup_timestamp', 'Last backup timestamp')

@backup_duration.time()
def monitored_backup(backup_type):
 """Backup mit Monitoring"""
 try:
 # Backup durchführen
 result = create_backup(backup_type)

 # Metriken aktualisieren
 backup_counter.labels(type=backup_type, status='success').inc()
 backup_size.set(result['size'])
 last_backup.set(time.time())

 return result

 except Exception as e:
 backup_counter.labels(type=backup_type, status='failure').inc()
 raise

```

## 19.10 Zusammenfassung

**Kritische Punkte:** - Regelmäßige Backups sind essenziell - PITR ermöglicht zeitpunktgenaue Recovery  
 - Automatisierung reduziert Fehler - Cloud-Storage bietet Off-site-Schutz - Regelmäßige Wiederherstellungs-Tests - Verschlüsselung schützt sensible Daten - Monitoring gewährleistet Backup-Erfolg

**Checkliste:** - ☐ Backup-Strategie definiert - ☐ Automatisierte Backups konfiguriert - ☐ PITR aktiviert - ☐ Off-site Backups eingerichtet - ☐ Wiederherstellungs-Tests durchgeführt - ☐ DR-Plan dokumentiert - ☐ Monitoring aktiviert - ☐ Team geschult

---

# Kapitel 29: Authentifizierung

---

## Übersicht

Dieses Kapitel behandelt Authentifizierungsmechanismen in ThemisDB.

### Hauptthemen

- Benutzer-Management
- Token-basierte Authentifizierung
- LDAP/AD-Integration
- Single Sign-On (SSO)

## Weitere Informationen

Siehe auch: [Security Hardening](#)

---



## Kapitel 30: Kapitel 21: Performance Tuning

### Einführung

Performance-Optimierung ist entscheidend für produktive ThemisDB-Deployments. Dieses Kapitel behandelt systematische Ansätze zur Identifizierung und Behebung von Performance-Problemen.

### 20.1 Performance-Analyse

#### 20.1.1 Query-Performance-Messung

**EXPLAIN ANALYZE:**

```
EXPLAIN ANALYZE
FOR order IN orders
 FILTER order.order_date > '2023-01-01'
FOR customer IN customers
 FILTER order.customer_id == customer.id
 SORT order.total_amount DESC
LIMIT 100
RETURN {order, customer_name: customer.name}
```

**Ausgabe-Interpretation:**

```
Query Plan:
└─ Sort (cost=1250.45, rows=1000, time=125ms)
 └─ Hash Join (cost=850.23, rows=5000, time=85ms)
 └─ Seq Scan on orders (cost=0.00, rows=10000, time=45ms)
 Filter: order_date > '2023-01-01'
 └─ Hash (cost=250.00, rows=5000, time=25ms)
 └─ Index Scan on customers (cost=0.00, rows=5000, time=15ms)
```

```

flowchart TD
 Start[Query Request] --> Parse[Parse & Validate]
 Parse --> Optimize[Query Optimizer]

 Optimize --> Plan{Execution Plan}

 Plan --> SeqScan[Sequential Scan
orders table
45ms]
 Plan --> IdxScan[Index Scan
customers
15ms]

 SeqScan --> Filter[Filter: date > 2023
10,000 -> 5,000 rows]
 IdxScan --> Hash[Build Hash Table
25ms]

 Filter --> Join[Hash Join
5,000 matches
85ms]
 Hash --> Join

 Join --> Sort[Sort by amount
125ms]
 Sort --> Limit[LIMIT 100]
 Limit --> Result[Final Results]

 style Start fill:#667eea
 style Optimize fill:#f093fb
 style SeqScan fill:#ff6348
 style IdxScan fill:#43e97b
 style Join fill:#4facfe
 style Result fill:#ffd32a

```

Abb. 21.1: Query-Performance-Optimization-Flow

## 20.1.2 Python Profiling

### Abfrage-Performance tracken:

```

def profile_query(func):
 def wrapper(*args, **kwargs):
 start = time.time()
 result = func(*args, **kwargs)
 if (duration := time.time() - start) > 1.0:
 print(f"SLOW: {func.__name__} {duration:.2f}s")
 return result
 return wrapper

```

## 20.1.3 System-Ressourcen-Monitoring

### CPU und Memory:

```
import psutil

def monitor_resources():
 """Überwacht System-Ressourcen"""
 process = psutil.Process()

 return {
 'cpu_percent': process.cpu_percent(interval=1),
 'memory_mb': process.memory_info().rss / 1024 / 1024,
 'open_files': len(process.open_files()),
 'threads': process.num_threads()
 }
```

## 20.2 Index-Optimierung

### 20.2.1 Index-Typen verstehen

#### B-Tree Index (Standard):

```
-- Für Equality und Range Queries
CREATE INDEX idx_order_date ON orders(order_date);

-- Composite Index
CREATE INDEX idx_customer_date ON orders(customer_id, order_date);
```

#### Hash Index:

```
-- Nur für Equality Queries
CREATE INDEX idx_customer_hash ON customers USING HASH(email);
```

#### Vector Index (HNSW):

```
-- Für Similarity Search
CREATE VECTOR INDEX idx_product_embedding
ON products(embedding)
WITH (metric='cosine', m=16, ef_construction=200);
```

### 20.2.2 Index-Strategie

#### Effective Index-Design:

```
CREATE INDEX bad_idx ON orders(customer_id, order_date, status, total_amount)

CREATE INDEX good_idx ON orders(customer_id, order_date)

CREATE INDEX covering_idx ON orders(customer_id, order_date)
INCLUDE (status, total_amount)
```

### Index-Nutzung überprüfen:

```
def analyze_index_usage(table_name):
 """Analysiert Index-Nutzung"""
 stats = client.admin.index_stats(table_name)

 for idx in stats['indexes']:
 usage_rate = idx['scans'] / max(idx['age_seconds'], 1)

 if usage_rate < 0.01: # Weniger als 1 Scan pro 100 Sekunden
 print(f"Unused index: {idx['name']}")
 print(f" Size: {idx['size_mb']} MB")
 print(f" Recommendation: Consider dropping")
```

## 20.2.3 Index-Wartung

### Rebuild und Reorganize:

```
def maintain_indexes(table_name):
 """Führt Index-Wartung durch"""
 stats = client.admin.index_fragmentation(table_name)

 for idx in stats['indexes']:
 frag_pct = idx['fragmentation_percent']

 if frag_pct > 30:
 print(f"Rebuilding index: {idx['name']} ({frag_pct}% fragmented)")
 client.admin.rebuild_index(table_name, idx['name'])
 elif frag_pct > 10:
 print(f"Reorganizing index: {idx['name']}")
 client.admin.reorganize_index(table_name, idx['name'])
```

## 20.3 Query-Optimierung

### 20.3.1 Query-Rewriting

#### Subquery vs JOIN:

```
-- Langsam: Correlated Subquery
FOR customer IN customers
 LET order_count = (
 FOR order IN orders
 FILTER order.customer_id == customer.id
 RETURN 1
)
 RETURN {name: customer.name, order_count: LENGTH(order_count)}

-- Schneller: Multi-FOR mit COLLECT
FOR customer IN customers
 FOR order IN orders
 FILTER order.customer_id == customer.id
 COLLECT c_id = customer.id, c_name = customer.name
 AGGREGATE order_count = COUNT()
 RETURN {name: c_name, order_count}
```

#### IN vs EXISTS:

```
-- Langsam für große Datasets
LET completed_customers = (
 FOR order IN orders
 FILTER order.status == 'completed'
 RETURN DISTINCT order.customer_id
)
FOR customer IN customers
 FILTER customer.id IN completed_customers
 RETURN customer

-- Schneller: Direkte Filterung
FOR customer IN customers
 LET has_completed = (
 FOR order IN orders
 FILTER order.customer_id == customer.id AND order.status == 'completed'
 LIMIT 1
 RETURN 1
)
 FILTER LENGTH(has_completed) > 0
 RETURN customer
```

## 20.3.2 Batch Operations

### Bulk Insert:

```
for record in records:
 client.collection('orders').insert_one(record)

client.collection('orders').insert_many(records, batch_size=1000)
```

### Bulk Update:

```
def bulk_update(updates, batch_size=1000):
 """Bulk Update mit optimaler Batch-Größe"""
 for i in range(0, len(updates), batch_size):
 batch = updates[i:i+batch_size]

 # Transaction für Batch
 with client.begin_transaction() as txn:
 for update in batch:
 txn.update('orders', update['id'], update['data'])
 txn.commit()
```

## 20.3.3 Query-Caching

### Application-Level Cache:

```

from functools import lru_cache
import hashlib
import json

class QueryCache:
 def __init__(self, max_size=1000, ttl=300):
 self.cache = {}
 self.max_size = max_size
 self.ttl = ttl

 def _cache_key(self, query, params):
 """Generiert Cache-Key"""
 data = json.dumps({'query': query, 'params': params}, sort_keys=True)
 return hashlib.md5(data.encode()).hexdigest()

 def get(self, query, params):
 """Holt gecachtes Ergebnis"""
 key = self._cache_key(query, params)

 if key in self.cache:
 entry = self.cache[key]
 if time.time() - entry['timestamp'] < self.ttl:
 return entry['result']
 else:
 del self.cache[key]

 return None

 def set(self, query, params, result):
 """Cached Ergebnis"""
 if len(self.cache) >= self.max_size:
 # LRU eviction
 oldest_key = min(self.cache.keys(),
 key=lambda k: self.cache[k]['timestamp'])
 del self.cache[oldest_key]

 key = self._cache_key(query, params)
 self.cache[key] = {
 'result': result,
 'timestamp': time.time()
 }

 def query(self, query_str, params=None):
 """Query mit Caching"""
 # Check cache
 cached = self.get(query_str, params)
 if cached is not None:
 return cached

```

```

 # Execute query
 result = client.query(query_str, params)

 # Cache result
 self.set(query_str, params, result)

 return result

cache = QueryCache(max_size=1000, ttl=300)
result = cache.query("""
 FOR product IN products
 FILTER product.category == @category
 RETURN product
""", {"category": 'electronics'})

```

## 20.4 Connection Pooling

### 20.4.1 Pool-Konfiguration

#### Optimale Pool-Size:

```

from themisdb import ConnectionPool

pool = ConnectionPool(
 host='localhost',
 port=8529,
 min_connections=10, # Minimum Connections
 max_connections=100, # Maximum Connections
 max_idle_time=300, # 5 Minuten Idle-Timeout
 connection_timeout=10, # 10 Sekunden Connect-Timeout
 validate_on_checkout=True # Validierung bei Checkout
)

def print_pool_stats():
 stats = pool.get_stats()
 print(f"Active: {stats['active']}")
 print(f"Idle: {stats['idle']}")
 print(f"Waiting: {stats['waiting']}")
 print(f"Pool utilization: {stats['active'] / stats['max'] * 100:.1f}%")

```

### 20.4.2 Connection-Leaks vermeiden

#### Context Manager:



```

class SafeConnection:
 def __init__(self, pool):
 self.pool = pool
 self.conn = None

 def __enter__(self):
 self.conn = self.pool.get_connection()
 return self.conn

 def __exit__(self, exc_type, exc_val, exc_tb):
 if self.conn:
 self.pool.release_connection(self.conn)
 return False

with SafeConnection(pool) as conn:
 result = conn.query("""
 FOR order IN orders
 RETURN order
 """)

```

## 20.5 RocksDB-Tuning

### 20.5.1 LSM-Tree Konfiguration

#### Write Buffer Size:

```

storage:
 rocksdb:
 write_buffer_size: 128MB # Größerer Buffer = weniger Flushes
 max_write_buffer_number: 4 # Parallele Buffers
 min_write_buffer_to_merge: 2 # Merge-Schwelle

```

#### Block Cache:

```

storage:
 rocksdb:
 block_cache_size: 8GB # LRU Cache für Blocks
 cache_index_and_filter: true # Indexes im Cache
 pin_top_level_index: true # Top-Level Index immer im Cache

```

### 20.5.2 Compaction-Tuning

#### Level-Style Compaction:

```
storage:
 rocksdb:
 compaction_style: level
 level0_file_num_trigger: 4 # Trigger bei 4 L0 Files
 level0_slowdown_writes: 20 # Slowdown bei 20 L0 Files
 level0_stop_writes: 36 # Stop bei 36 L0 Files
 max_background_compactions: 8 # Parallele Compactions
 max_background_flushes: 4 # Parallele Flushes
```

#### Universal Compaction (für Write-Heavy):

```
storage:
 rocksdb:
 compaction_style: universal
 max_size_amplification_percent: 200
 compression_size_multiplier: 2
```

### 20.5.3 Bloom Filters

#### Aktivierung:

```
storage:
 rocksdb:
 bloom_filter_bits_per_key: 10 # 10 Bits = ~1% False Positive Rate
 block_based_filter: false # Full Filter = bessere Performance
```

## 20.6 Memory-Management

### 20.6.1 Memory-Profiling

#### Memory-Leaks finden:

```

import tracemalloc

def profile_memory():
 """Tracked Memory-Allocation"""
 tracemalloc.start()

 # Code ausführen
 result = expensive_operation()

 # Memory Snapshot
 snapshot = tracemalloc.take_snapshot()
 top_stats = snapshot.statistics('lineno')

 print("Top 10 memory allocations:")
 for stat in top_stats[:10]:
 print(f"{stat.filename}:{stat.lineno}: {stat.size / 1024:.1f} KB")

 tracemalloc.stop()

```

## 20.6.2 Object Pooling

### Connection und Result Pooling:

```

from queue import Queue

class ObjectPool:
 def __init__(self, factory, max_size=100):
 self.factory = factory
 self.pool = Queue(maxsize=max_size)
 self._initialize_pool(max_size // 2)

 def _initialize_pool(self, size):
 for _ in range(size):
 self.pool.put(self.factory())

 def acquire(self):
 if self.pool.empty():
 return self.factory()
 return self.pool.get()

 def release(self, obj):
 if not self.pool.full():
 self.pool.put(obj)

result_pool = ObjectPool(lambda: [], max_size=100)

```

## 20.7 Netzwerk-Optimierung

### 20.7.1 Batch Requests

#### Request Batching:

```
class BatchedClient:
 def __init__(self, client, batch_size=10, flush_interval=0.1):
 self.client = client
 self.batch_size = batch_size
 self.flush_interval = flush_interval
 self.batch = []
 self.last_flush = time.time()

 def query(self, query_str, params=None):
 """Fügt Query zum Batch hinzu"""
 self.batch.append({'query': query_str, 'params': params})

 # Auto-Flush bei Batch-Size oder Timeout
 if len(self.batch) >= self.batch_size or \
 time.time() - self.last_flush > self.flush_interval:
 return self.flush()

 def flush(self):
 """Sendet gesammelten Batch"""
 if not self.batch:
 return []

 results = self.client.batch_query(self.batch)
 self.batch = []
 self.last_flush = time.time()

 return results
```

### 20.7.2 Kompression

#### Aktivierung:

```
network:
 compression:
 enabled: true
 algorithm: zstd # zstd, gzip, lz4
 level: 3 # 1-22 für zstd
 min_size: 1KB # Nur bei Größe > 1KB komprimieren
```

## 20.8 Monitoring und Alerting

### 20.8.1 Performance-Metriken

#### Key Metrics:

```
from prometheus_client import Histogram, Counter, Gauge

query_duration = Histogram(
 'themisdb_query_duration_seconds',
 'Query execution time',
 buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]
)

slow_queries = Counter('themisdb_slow_queries_total', 'Slow queries')

cache_hits = Counter('themisdb_cache_hits_total', 'Cache hits')
cache_misses = Counter('themisdb_cache_misses_total', 'Cache misses')
cache_hit_rate = Gauge('themisdb_cache_hit_rate', 'Cache hit rate')

@query_duration.time()
def execute_query(query, params):
 start = time.time()
 result = client.query(query, params)
 duration = time.time() - start

 if duration > 1.0:
 slow_queries.inc()

 return result
```

### 20.8.2 Alert Rules

#### Prometheus Alerts:

```

groups:
 - name: themisdb_performance
 rules:
 # Hohe Query-Latenz
 - alert: HighQueryLatency
 expr: histogram_quantile(0.95, themisdb_query_duration_seconds) > 1
 for: 5m
 annotations:
 summary: "95th percentile query latency > 1s"

 # Niedrige Cache Hit Rate
 - alert: LowCacheHitRate
 expr: themisdb_cache_hit_rate < 0.7
 for: 10m
 annotations:
 summary: "Cache hit rate below 70%"

 # Viele Slow Queries
 - alert: ManySlowQueries
 expr: rate(themisdb_slow_queries_total[5m]) > 10
 for: 5m
 annotations:
 summary: "More than 10 slow queries per minute"

```

## 20.9 Load Testing

### 20.9.1 Benchmark-Suite

#### Basis-Benchmark:

```

import concurrent.futures
import time

class PerformanceBenchmark:
 def __init__(self, client):
 self.client = client

 def benchmark_read(self, num_queries=1000, concurrency=10):
 """Read-Performance testen"""
 def single_query():
 start = time.time()
 self.client.query("""
 FOR product IN products
 LIMIT 100
 RETURN product
 """)
 return time.time() - start

 with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
 futures = [executor.submit(single_query) for _ in range(num_queries)]
 durations = [f.result() for f in concurrent.futures.as_completed(futures)]

 return {
 'avg': sum(durations) / len(durations),
 'p50': sorted(durations)[len(durations)//2],
 'p95': sorted(durations)[int(len(durations)*0.95)],
 'p99': sorted(durations)[int(len(durations)*0.99)],
 'qps': num_queries / sum(durations)
 }

 def benchmark_write(self, num_writes=1000, concurrency=10):
 """Write-Performance testen"""
 def single_write():
 start = time.time()
 self.client.collection('test').insert_one({
 'data': 'test',
 'timestamp': time.time()
 })
 return time.time() - start

 with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
 futures = [executor.submit(single_write) for _ in range(num_writes)]
 durations = [f.result() for f in concurrent.futures.as_completed(futures)]

 return {
 'avg': sum(durations) / len(durations),
 'p95': sorted(durations)[int(len(durations)*0.95)],
 'wps': num_writes / sum(durations)
 }

```

```
benchmark = PerformanceBenchmark(client)
read_results = benchmark.benchmark_read(num_queries=10000, concurrency=50)
print(f"Read QPS: {read_results['qps']:.0f}")
print(f"P95 Latency: {read_results['p95']*1000:.1f}ms")
```

## 20.9.2 Stress Testing

### Load Generator:

```
def stress_test(duration_seconds=60, target_qps=1000):
 """Stress-Test mit Ziel-QPS"""
 import random

 start_time = time.time()
 query_count = 0
 interval = 1.0 / target_qps

 queries = [
 "FOR p IN products FILTER p.category == @cat RETURN p",
 "FOR o IN orders FILTER o.status == @status COLLECT AGGREGATE c = COUNT() RETURN c",
 "FOR c IN customers FILTER c.email == @email RETURN c"
]

 while time.time() - start_time < duration_seconds:
 query_start = time.time()

 # Random Query ausführen
 query = random.choice(queries)
 client.query(query, [random.choice(['active', 'pending', 'test@example.com'])])
 query_count += 1

 # Rate Limiting
 elapsed = time.time() - query_start
 if elapsed < interval:
 time.sleep(interval - elapsed)

 actual_qps = query_count / duration_seconds
 print(f"Achieved QPS: {actual_qps:.0f} (target: {target_qps})")
```



## 20.9 Benchmarks (v1.3.4)

### 20.9.1 Throughput & Latenz

Workload	Dataset	Hardware	Throughput	P95 Latenz
OLTP Reads	50M docs	8 vCPU / 32 GB	18k QPS	42 ms
OLTP Writes	50M docs	8 vCPU / 32 GB	6k WPS	65 ms
Graph Traversal	10M Knoten, 50M Kanten	16 vCPU / 64 GB	2.5k QPS	110 ms
Vector Search (768-d, HNSW)	10M Vektoren	16 vCPU / 64 GB	1.8k QPS	95 ms
Fulltext Search	30M docs	8 vCPU / 32 GB	4.2k QPS	70 ms
TS Aggregation (Gorilla)	1B Punkte	8 vCPU / 32 GB	12k QPS	38 ms

**Konfiguration:** - RocksDB LZ4 (Level 0-5), ZSTD (6+) - Block Cache: 8 GB, Write Buffer: 512 MB - WAL auf NVMe, Daten auf NVMe

### 20.9.2 Kompressionseffekte

Datentyp	Codec	Ratio	CPU-Overhead	Latenz-Auswirkung
Time-Series	Gorilla	10-20x	+15% encode	+1-2 ms
Vektoren	SQ8	4x	+20% encode	+2-4 ms Suche
Content Blobs	ZSTD19	1.5-2x	+30%	+5-8 ms Upload

### 20.9.3 Tuning-Profile

- **Read-Heavy:** Block Cache groß, Compaction auf Level ausgeglichen, `max_background_jobs` hoch
- **Write-Heavy:** Größere WAL, `min_write_buffer_number_to_merge` erhöhen, LZ4-only
- **Vector-Heavy:** Mehr RAM für HNSW, SQ8 aktivieren ab 1M Vektoren, Cache Warmup
- **TS-Heavy:** Gorilla an, 24h-Chunks, `target_file_size_base` erhöhen

```

flowchart LR
 A[Workload Profiling] --> B{Workload-Typ}
 B --> C[Read-Heavy]
 B --> D[Write-Heavy]
 B --> E[Vector-Heavy]
 B --> F[Time-Series]

 C --> C1[Großer Block Cache]
 D --> D1[Große WAL]
 E --> E1[SQ8 + HNSW RAM]
 F --> F1[Gorilla + 24h Chunks]

 style A fill:#e1f5ff
 style B fill:#fff4e1
 style C1 fill:#e1ffe1
 style D1 fill:#ffe1e1
 style E1 fill:#e1f5ff
 style F1 fill:#fff4e1

```

Abb. 21.2: Index-Selection-Strategy

## 20.9A LLM-Performance-Optimierungen (v1.4.0-alpha)

### 20.9A.1 Flash Attention

**Neu in v1.4.0-alpha:** IO-bewusste Attention-Implementierung für deutlich verbesserte Speicher-Effizienz und Geschwindigkeit bei LLM-Inferenz.

#### Problem mit Standard-Attention:

Klassische Self-Attention allokiert temporär große Attention-Matrizen im HBM (High-Bandwidth Memory), was zu: - Hohem Speicherverbrauch:  $O(N^2)$  für Sequenzlänge  $N$  - Vielen Speicher-Transfers zwischen HBM und SRAM - Suboptimaler GPU-Auslastung führt

#### Flash Attention Lösung:

```

graph TB
 subgraph "Standard Attention"
 Input1[Input
Seq Length N] -->|"O(N²) Memory"| HBM1[HBM Storage]
 HBM1 -->|Slow Transfer| Compute1[GPU Compute]
 Compute1 -->|Result| Output1[Output]
 end

 subgraph "Flash Attention"
 Input2[Input
Seq Length N] -->|"O(N) Memory"| SRAM[SRAM Tiling]
 SRAM -->|Fast| Compute2[GPU Compute
Fused Ops]
 Compute2 -->|Result| Output2[Output]
 end

 style Compute2 fill:#43e97b
 style SRAM fill:#4facfe
 style HBM1 fill:#ffd32a

```

**Abb. 21.3:** Cache-Hierarchy-Diagram**Aktivierung in ThemisDB:**

```

// themis.conf - Flash Attention Konfiguration
llm:
 local_models:
 llama-70b:
 attention_implementation: 'flash_attention_2' // flash_attention_2 oder standard
 flash_attention:
 enabled: true
 version: 2 // Flash Attention 2 (neueste Version)
 block_size_q: 128 // Query block size
 block_size_kv: 128 // Key/Value block size
 causal: true // Für autoregressive Models

```

**AQL-Konfiguration:**

```
// Flash Attention explizit aktivieren/deaktivieren
FOR doc IN documents
 LIMIT 100
 LET summary = PROMPT('llama-70b-local',
 CONCAT('Summarize: ', doc.content),
 {
 max_tokens: 500,
 attention_config: {
 implementation: 'flash_attention_2',
 enable_memory_efficient: true
 }
 }
)
 RETURN summary
```

Performance-Metriken:

Metric	Standard Attention	Flash Attention	Verbesserung
GPU Memory	38.5 GB	24.2 GB	-37%
Throughput	185 tokens/s	312 tokens/s	+69%
Latenz (p50)	1.85s	1.12s	-39%
Latenz (p99)	3.20s	1.95s	-39%
Batch Size (max)	32	64	+100%

Benchmark-Beispiel:

```
// Performance-Vergleich durchführen
LET benchmark_results = (
 FOR attention_type IN ['standard', 'flash_attention_2']
 LET start_time = DATE_NOW()

 LET results = (
 FOR doc IN documents
 LIMIT 1000
 RETURN PROMPT('llama-70b-local',
 CONCAT('Summarize in 3 sentences: ', doc.content),
 {
 max_tokens: 100,
 attention_config: {implementation: attention_type}
 }
)
)

 LET duration_ms = DATE_DIFF(start_time, DATE_NOW(), 'millisecond')

 RETURN {
 attention_type: attention_type,
 duration_ms: duration_ms,
 throughput_tokens_per_sec: (1000 * 100) / (duration_ms / 1000),
 avg_latency_ms: duration_ms / 1000
 }
)

RETURN benchmark_results
```

### Hardwareanforderungen:

- **GPU:** NVIDIA Ampere (A100) oder neuere Architektur
- **Compute Capability:**  $\geq 8.0$  (für Flash Attention 2)
- **CUDA:**  $\geq 11.8$
- **GPU Memory:** Minimal 24 GB empfohlen

### Best Practices:

1. **Aktiviere Flash Attention für lange Sequenzen** ( $>1024$  tokens)
2. **Monitoring:** Überwache GPU-Speichernutzung und Durchsatz
3. **Batch-Größe erhöhen:** Flash Attention ermöglicht größere Batches
4. **Version 2 bevorzugen:** Flash Attention 2 ist deutlich schneller

## 20.9A.2 Speculative Decoding

**Neu in v1.4.0-alpha:** Beschleunigte Token-Generierung durch parallele Spekulation mit einem schnellen Draft-Model.

### Konzept:

Speculative Decoding nutzt ein kleines, schnelles "Draft Model", um mehrere Tokens parallel zu generieren, die dann vom größeren "Target Model" validiert werden.

```
sequenceDiagram
 participant Client
 participant Draft as Draft Model
(klein & schnell)
 participant Target as Target Model
(groß & präzise)

 Client->>Draft: Generate K tokens
 Draft-->>Draft: Generiere 5 Tokens
spekulativ
 Draft->>Target: Validate all 5 tokens
 Target-->>Target: Prüfe Token 1: [OK]
Token 2: [OK]
Token 3: x
 Target->>Client: Accept 2, reject rest

 Note over Client,Target: 2-3x schneller als
sequential decoding
```

### Abb. 21.4: Query-Plan-Optimization

#### Konfiguration:

```
// themis.conf - Speculative Decoding Setup
llm:
 local_models:
 llama-70b:
 speculative_decoding:
 enabled: true
 draft_model: 'llama-7b-local' // Kleines, schnelles Model
 num_speculative_tokens: 5 // Tokens pro Spekulation
 acceptance_threshold: 0.8 // Akzeptanz-Schwelle
```

#### AQL-Verwendung:

```
// Speculative Decoding für schnellere Generierung
FOR article IN news_articles
 FILTER article.summary == null
 LIMIT 500

 LET summary = PROMPT('llama-70b-local',
 {
 system: 'Create concise 2-sentence summaries.',
 user: article.content
 },
 {
 max_tokens: 100,
 speculative_decoding: {
 enabled: true,
 draft_model: 'llama-7b-local',
 num_tokens: 5,
 acceptance_threshold: 0.85
 }
 }
)

 UPDATE article WITH {
 summary: summary,
 summarized_at: DATE_NOW()
 } IN news_articles
```

### Performance-Charakteristiken:

Model Pair	Base Latenz	Speculative Latenz	Speed-Up	Akzeptanzrate
Llama-70B + Llama-7B	2.8s	1.1s	<b>2.5x</b>	82%
Llama-70B + Llama-13B	2.8s	1.3s	<b>2.2x</b>	88%
GPT-4 + GPT-3.5	3.5s	1.6s	<b>2.2x</b>	78%

### Akzeptanzraten-Analyse:

```
// Analysiere Speculative Decoding Statistiken
RETURN LLM_SPECULATIVE_STATS({
 model: 'llama-70b-local',
 from: DATE_SUBTRACT(DATE_NOW(), 7, 'day'),
 to: DATE_NOW()
})
```

Output:

```
{
 "total_requests": 45200,
 "speculative_tokens_proposed": 226000,
 "tokens_accepted": 185460,
 "acceptance_rate": 0.821,
 "avg_speedup": 2.4,
 "latency_reduction_percent": 58.3,
 "draft_model_overhead_ms": 45,
 "net_benefit_ms": 1680
}
```

### Tuning-Guidelines:

#### 1. Draft Model Wahl:

2. Zu klein: Niedrige Akzeptanzrate
3. Zu groß: Geringer Geschwindigkeitsvorteil
4. **Optimal:** 10-20% der Größe des Target Models

#### 5. Tokens pro Spekulation:

6. Mehr Tokens = höheres Potenzial, aber mehr Verwerfungen
7. **Empfehlung:** 4-6 Tokens für beste Balance

#### 8. Acceptance Threshold:

9. Höher = konservativer, weniger Verwerfungen
10. Niedriger = aggressiver, mehr Speed-up Potenzial
11. **Empfehlung:** 0.80-0.85

### Use Cases:

- **Optimal:** Lange Textgenerierung (Zusammenfassungen, Artikel)
- **Suboptimal:** Kurze Antworten (<50 tokens) - Overhead überwiegt
- **Nicht empfohlen:** Code-Generierung (Draft Models oft ungenau)

## 20.9A.3 Continuous Batching

**Neu in v1.4.0-alpha:** Dynamisches Request-Batching für LLM-Inferenz mit dramatisch verbessertem Durchsatz und reduzierter Latenz.

### Problem mit statischem Batching:



Statisches Batching:

```
Request 1 (100 tokens) ┐
Request 2 (500 tokens) ┤─ Warte bis alle fertig (500 tokens)
Request 3 (50 tokens) ┘ ↓
 Request 1,3 warten unnötig!
```

### Lösung mit Continuous Batching:

Continuous Batching:

```
Request 1 (100 tokens) → ✓ Fertig nach 100 tokens
Request 2 (500 tokens) → → → → → ✓ Fertig nach 500 tokens
Request 3 (50 tokens) → ✓ Fertig nach 50 tokens
Request 4 (neu) ──┐ Tritt in Batch ein
```

```
graph LR
 Q[Request Queue] --> CB[Continuous Batcher]
 CB -->|Dynamic Batches| GPU[GPU Inference]
 GPU -->|Completed| C1[Client 1]
 GPU -->|Still Running| Batch[Active Batch]
 Batch -->|Continues| GPU
 Q2[New Request] -.->|Join Active| CB

 style CB fill:#4facfe
 style GPU fill:#43e97b
 style Batch fill:#ffdb3a
```

**Abb. 21.5:** Resource-Allocation-Matrix

### Detaillierter Timeline-Ablauf:

```

gantt
 title Continuous Batching vs. Static Batching
 dateFormat X
 axisFormat %L

 section Static Batch
 Request 1 (100 tok) :done, s1, 0, 500
 Request 2 (500 tok) :done, s2, 0, 500
 Request 3 (50 tok) :done, s3, 0, 500
 Wait for slowest :crit, s4, 100, 500

 section Continuous Batch
 Request 1 (100 tok) :done, c1, 0, 100
 Request 2 (500 tok) :done, c2, 0, 500
 Request 3 (50 tok) :done, c3, 0, 50
 Request 4 (new) :active, c4, 100, 200

```

**Abb. 21.6:** Performance-Tuning-Workflow

**Diagramm-Erklärung:** - **Static Batching (oben):** Alle Requests warten bis zum langsamsten (500 tokens) - Request 1 & 3 fertig nach 100/50 tokens, müssen aber 500 tokens warten - Total Time: 500ms für alle - Verschwendete Zeit: 400ms (Request 1) + 450ms (Request 3)

- **Continuous Batching (unten):** Requests verlassen Batch sobald fertig
- Request 1: 100ms → sofort zurückgegeben
- Request 3: 50ms → sofort zurückgegeben
- Request 2: 500ms → später zurückgegeben
- Request 4: Kann bei 100ms in den aktiven Batch eintreten
- Durchschnittliche Latenz:  $(100+500+50+200)/4 = 212\text{ms}$  vs. 500ms (Static)

#### Konfiguration:

```

// themis.conf - Continuous Batching
llm:
 local_models:
 llama-70b:
 continuous_batching:
 enabled: true
 max_batch_size: 128 // Maximale gleichzeitige Requests
 max_queue_size: 512 // Warteschlange
 batch_timeout_ms: 50 // Max Wartezeit für Batch-Bildung
 dynamic_batching: true

```

#### AQL-Integration:

```

// Continuous Batching wird automatisch verwendet
// Keine spezielle Konfiguration nötig - transparent für User

FOR customer IN customers
 FILTER customer.churn_risk == null

 LET analysis = PROMPT('llama-70b-local',
 {
 system: 'Analyze customer data for churn risk.',
 user: TO_STRING(customer)
 },
 {
 max_tokens: 200,
 temperature: 0.3
 // Continuous batching automatisch aktiv
 }
)

 UPDATE customer WITH {
 churn_analysis: analysis,
 analyzed_at: DATE_NOW()
 } IN customers

```

### Performance-Vergleich:

Metric	Static Batching	Continuous Batching	Verbesserung
Throughput	450 req/s	1240 req/s	+176%
Avg Latency	2.8s	1.2s	-57%
P95 Latency	5.4s	2.1s	-61%
GPU Utilization	62%	94%	+52%
Queue Wait Time	850ms	120ms	-86%

### Durchsatz-Benchmark:

```
// Durchsatz über Zeit messen
LET throughput_test = (
 LET start = DATE_NOW()

 // Sende 1000 Requests parallel
 LET results = (
 FOR i IN 1..1000
 RETURN ASYNC PROMPT('llama-70b-local',
 CONCAT('Analyze: ', RANDOM_TOKEN(100, 500)),
 {max_tokens: FLOOR(RAND() * 200) + 50}
)
)
)

 // Warte auf alle Ergebnisse
 LET completed = (FOR r IN results RETURN WAIT(r))

 LET duration_s = DATE_DIFF(start, DATE_NOW(), 'second')

 RETURN {
 total_requests: 1000,
 duration_seconds: duration_s,
 throughput_req_per_sec: 1000 / duration_s,
 avg_latency_ms: (duration_s * 1000) / 1000
 }
)

RETURN throughput_test
```

### Monitoring und Tuning:

```
// Continuous Batching Metriken
RETURN LLM_BATCHING_STATS('llama-70b-local')
```

Output:

```
{
 "mode": "continuous",
 "current_batch_size": 87,
 "max_batch_size": 128,
 "queue_length": 12,
 "max_queue_size": 512,
 "batch_fill_rate": 0.68,
 "avg_batch_size_last_hour": 73.5,
 "requests_per_second": 1185,
 "gpu_utilization": 0.93,
 "avg_wait_time_ms": 95,
 "p95_wait_time_ms": 210,
 "batches_formed_last_hour": 58420
}
```

### Tuning-Parameter:

1. **max\_batch\_size:**

- 2. Zu klein: Verschenkter Durchsatz
- 3. Zu groß: OOM auf GPU
- 4. **Empfehlung:** GPU Memory / Avg Request Memory

5. **batch\_timeout\_ms:**

- 6. Zu kurz: Kleine Batches, verschenkter Durchsatz
- 7. Zu lang: Unnötige Wartezeiten
- 8. **Empfehlung:** 20-100ms je nach Workload

9. **max\_queue\_size:**

- 10. Sollte 4-8x größer als max\_batch\_size sein
- 11. Zu klein: Requests werden abgelehnt
- 12. **Empfehlung:** 4 \* max\_batch\_size

### Best Practices:

- 1. **Aktiviere für Produktions-Workloads** - Immer ein Gewinn
- 2. **Monitor Queue Länge** - Warnung bei Sättigung
- 3. **Load Testing** - Optimale Batch-Größe finden
- 4. **Kombiniere mit Paged Attention** - Maximale Memory-Effizienz

## 20.10 Best Practices Checkliste

### 20.10.1 Query-Optimierung

- ☐ Indexes für häufige WHERE-Clauses
- ☐ Composite Indexes für Multi-Column Filters
- ☐ EXPLAIN ANALYZE für langsame Queries
- ☐ Batch Operations statt Einzelqueries
- ☐ Query-Caching für häufige Abfragen
- ☐ Prepared Statements verwenden

### 20.10.2 Index-Management

- ☐ Regelmäßige Index-Wartung
- ☐ Ungenutzte Indexes entfernen
- ☐ Index-Fragmentation überwachen
- ☐ Covering Indexes für häufige Queries
- ☐ Index-Größe überwachen

### 20.10.3 Connection-Management

- ☐ Connection Pooling aktiviert
- ☐ Pool-Size optimal konfiguriert
- ☐ Connection-Leaks vermeiden
- ☐ Timeout-Werte angemessen
- ☐ Connection-Validierung aktiviert

### 20.10.4 RocksDB-Tuning

- ☐ Write Buffer Size angepasst
- ☐ Block Cache konfiguriert
- ☐ Compaction-Parameter optimiert
- ☐ Bloom Filter aktiviert
- ☐ Compression konfiguriert

### 20.10.5 Monitoring

- ☐ Performance-Metriken erfasst
- ☐ Slow Query Log aktiviert
- ☐ Alert Rules konfiguriert
- ☐ Regelmäßige Benchmarks

- 
- [ ] Capacity Planning durchgeführt

## 20.11 Zusammenfassung

**Kernpunkte:** - Systematische Performance-Analyse ist essentiell - Indexes sind der wichtigste Optimierungshebel - Query-Rewriting kann dramatische Verbesserungen bringen - Connection Pooling ist kritisch für Skalierbarkeit - RocksDB-Tuning beeinflusst grundlegende Performance - Monitoring ermöglicht proaktive Optimierung - Load Testing validiert Performance-Verbesserungen

**Performance-Ziele:** - Query Latency P95 < 100ms - Cache Hit Rate > 80% - Connection Pool Utilization < 80% - QPS: 10,000+ für Read-Heavy Workloads - Write Throughput: 5,000+ WPS

---

## Kapitel 31: Kapitel 22: Client Libraries & Drivers

---

### 21.1 Einführung in ThemisDB Client-Ökosystem

ThemisDB bietet eine umfassende Palette an Client-Libraries und Treibern für verschiedene Programmiersprachen und Plattformen. Diese ermöglichen es Entwicklern, nahtlos mit ThemisDB zu interagieren, unabhängig vom gewählten Tech-Stack.

#### 21.1.1 Architektur der Client-Libraries

Die ThemisDB Client-Architecture basiert auf einem mehrschichtigen Ansatz:

- 1. Protokoll-Layer:**
  - **Binary Protocol:** Hochperformantes binäres Protokoll für maximale Effizienz
  - **HTTP/REST API:** RESTful Interface für einfache Integration
  - **WebSocket:** Bidirektionale Kommunikation für Realtime-Features
- 2. Connection Management:**
  - **Connection Pooling:** Effiziente Verwaltung von Datenbankverbindungen
  - **Automatic Reconnection:** Automatisches Wiederherstellen nach Verbindungsabbruch
  - **Load Balancing:** Verteilung von Anfragen über mehrere Server
- 3. Query Interface:**
  - **AQL Builder:** Fluent API zum Erstellen von Queries
  - **ORM Support:** Object-Relational Mapping für typsichere Datenmodelle
  - **Raw Query:** Direkter Zugriff auf AQL für maximale Flexibilität



```

graph TB
 subgraph "Client Architecture Layers"
 App[Application Code]

 App --> API[Client API Layer]

 API --> QB[Query Builder
Fluent Interface]
 API --> ORM[ORM Layer
Object Mapping]
 API --> Raw[Raw Query
Direct AQL]

 QB --> Conn[Connection Manager]
 ORM --> Conn
 Raw --> Conn

 Conn --> Pool[Connection Pool
Reusable Connections]

 Pool --> Proto{Protocol Selection}

 Proto -->|Binary| BP[Binary Protocol
High Performance]
 Proto -->|HTTP| REST[REST API
HTTP/JSON]
 Proto -->|WebSocket| WS[WebSocket
Bidirectional]

 BP --> Server[(ThemisDB Server)]
 REST --> Server
 WS --> Server
 end

 style App fill:#667eea
 style API fill:#4facfe
 style Conn fill:#43e97b
 style Pool fill:#f093fb
 style Server fill:#ffd32a

```

Abb. 22.1: Client-SDK-Architektur

### 21.1.2 Unterstützte Sprachen

ThemisDB bietet offizielle Clients für:

- **Python** (themisdb-python) - Vollständiger Feature-Support
- **JavaScript/TypeScript** (themisdb-js) - Node.js und Browser
- **Java** (themisdb-java) - Enterprise-ready JDBC-Treiber
- **Go** (themisdb-go) - High-performance native Client
- **C#/.NET** (ThemisDB.NET) - .NET Standard 2.0+
- **Rust** (themisdb-rs) - Memory-safe native Client

- **Ruby** (themisdb-ruby) - Rails-freundlich
- **PHP** (themisdb-php) - Laravel & Symfony Support

## 21.2 Python Client (themisdb-python)

Der Python Client ist die am weitesten entwickelte und feature-reichste Client-Library für ThemisDB.

### 21.2.1 Installation & Setup

```
pip install themisdb

pip install themisdb[async,pandas]

pip install git+https://github.com/themisdb/themisdb-python.git
```

#### Basis-Verbindung:

```
from themisdb import ThemisDB

db = ThemisDB("localhost", 7687)

db = ThemisDB(
 host="localhost",
 port=7687,
 username="admin",
 password="secure_password",
 database="mydb"
)

db = ThemisDB.from_uri("themis://admin:password@localhost:7687/mydb")
```

## 21.2.2 CRUD-Operationen

```
user = db.insert("users", {
 "name": "Alice",
 "email": "alice@example.com",
 "age": 30
})
print(f"Created user with ID: {user['id']}")

user = db.find_one("users", {"email": "alice@example.com"})

db.update("users",
 {"email": "alice@example.com"},
 {"$set": {"age": 31}}
)

db.delete("users", {"email": "alice@example.com"})
```

## 21.2.3 Query Builder API

```
from themisdb import Query

users = (Query(db, "users")
 .filter(age__gte=18)
 .filter(active=True)
 .order_by("-created_at")
 .limit(10)
 .all())

active_admins = (Query(db, "users")
 .filter(role="admin", status="active")
 .select("id", "name", "email")
 .join("profiles", on="user_id")
 .all())

stats = (Query(db, "orders")
 .filter(status="completed")
 .group_by("customer_id")
 .aggregate({
 "total_amount": "SUM(amount)",
 "order_count": "COUNT(*)",
 "avg_amount": "AVG(amount)"
 }))
```

## 21.2.4 ORM (Object-Relational Mapping)

```
from themisdb.orm import Model, Field

class User(Model):
 __collection__ = "users"

 name = Field(str, required=True)
 email = Field(str, unique=True)
 age = Field(int, min=0, max=150)
 created_at = Field(datetime, auto_now_add=True)
 updated_at = Field(datetime, auto_now=True)

 def __repr__(self):
 return f"User({self.name}, {self.email})"

user = User(name="Bob", email="bob@example.com", age=25)
user.save()

users = User.objects.filter(age__gte=18).all()
bob = User.objects.get(email="bob@example.com")

bob.age = 26
bob.save()

bob.delete()

class Post(Model):
 __collection__ = "posts"

 title = Field(str)
 content = Field(str)
 author = Field(User, foreign_key=True) # Foreign Key zu User
 tags = Field(list)

post = Post.objects.first()
print(f"Author: {post.author.name}") # Automatisches Laden
```

### 21.2.5 Async/Await Support

```
import asyncio
from themisdb import AsyncThemisDB

async def main():
 # Async Connection
 db = AsyncThemisDB("localhost", 7687)

 # Async Queries
 users = await db.find("users", {"active": True})

 # Concurrent Operations
 results = await asyncio.gather(
 db.find("users", {"role": "admin"}),
 db.find("orders", {"status": "pending"}),
 db.find("products", {"in_stock": True})
)

 await db.close()

asyncio.run(main())
```

### 21.2.6 Transaktionen

```
with db.transaction() as tx:
 # Geld von Account A nach B überweisen
 account_a = tx.find_one("accounts", {"id": "A"})
 account_b = tx.find_one("accounts", {"id": "B"})

 if account_a["balance"] >= 100:
 tx.update("accounts", {"id": "A"},
 {"$inc": {"balance": -100}})
 tx.update("accounts", {"id": "B"},
 {"$inc": {"balance": 100}})
 tx.commit()
 else:
 tx.rollback()

async with db.transaction() as tx:
 await tx.insert("logs", {"action": "transfer", "amount": 100})
 await tx.commit()
```

## 21.2.7 Pandas Integration

```
import pandas as pd

df = db.to_dataframe("""
 FOR u IN users
 FILTER u.age > 18
 RETURN u
""")

df.to_themis(db, collection="users", if_exists="append")

user_stats = (
 db.to_dataframe("FOR u IN users RETURN u")
 .groupby("country")
 .agg({
 "id": "count",
 "age": ["mean", "median"],
 "income": "sum"
 })
)
```

## 21.3 JavaScript/TypeScript Client

### 21.3.1 Installation

```
npm install themisdb

yarn add themisdb
```

## 21.3.2 Node.js Usage

```
const { ThemisDB } = require('themisdb');

// Connection
const db = new ThemisDB({
 host: 'localhost',
 port: 7687,
 username: 'admin',
 password: 'password',
 database: 'mydb'
});

// Async/Await
async function getUsers() {
 const users = await db.collection('users')
 .find({ active: true })
 .sort({ name: 1 })
 .limit(10)
 .toArray();

 return users;
}

// Promises
db.collection('users')
 .findOne({ email: 'user@example.com' })
 .then(user => console.log(user))
 .catch(err => console.error(err));
```

### 21.3.3 TypeScript Support

```
interface User {
 id: string;
 name: string;
 email: string;
 age: number;
 active: boolean;
}

const db = new ThemisDB<{
 users: User;
 posts: Post;
}>({
 host: 'localhost',
 port: 7687
});

// Type-safe Queries
const users: User[] = await db.collection('users')
 .find<User>({ age: { $gte: 18 } })
 .toArray();

// Type-safe Inserts
await db.collection('users').insertOne({
 name: "Alice",
 email: "alice@example.com",
 age: 30,
 active: true
});
```



### 21.3.4 Browser Usage

```
<!DOCTYPE html>
<html>
<head>
 <script src="https://cdn.themisdb.io/themisdb-browser.min.js"></script>
</head>
<body>
 <script>
 const db = new ThemisDB({
 url: 'https://api.example.com/themis',
 apiKey: 'your-api-key'
 });

 // WebSocket für Realtime
 db.collection('messages')
 .watch()
 .on('insert', (doc) => {
 console.log('New message:', doc);
 });
 </script>
</body>
</html>
```

### 21.3.4 React Integration

```
import React, { useState, useEffect } from 'react';
import { useThemisDB } from 'themisdb/react';

function UserList() {
 const { data, loading, error } = useThemisDB(
 'users',
 { active: true },
 { sort: { name: 1 } }
);

 if (loading) return <div>Loading...</div>;
 if (error) return <div>Error: {error.message}</div>;

 return (

 {data.map(user => (
 <li key={user.id}>{user.name}
))}

);
}
```

## 21.4 Java Client & JDBC Driver

### 21.4.1 Maven Dependency

```
<dependency>
 <groupId>io.themisdb</groupId>
 <artifactId>themisdb-java</artifactId>
 <version>1.3.4</version>
</dependency>

<!-- JDBC Driver -->
<dependency>
 <groupId>io.themisdb</groupId>
 <artifactId>themisdb-jdbc</artifactId>
 <version>1.3.4</version>
</dependency>
```

## 21.4.2 Native Client API

```
import io.themisdb.ThemisDB;
import io.themisdb.Document;

public class Example {
 public static void main(String[] args) {
 // Connection
 ThemisDB db = ThemisDB.connect()
 .host("localhost")
 .port(7687)
 .username("admin")
 .password("password")
 .database("mydb")
 .build();

 // Insert
 Document user = new Document()
 .append("name", "Alice")
 .append("email", "alice@example.com")
 .append("age", 30);

 db.collection("users").insertOne(user);

 // Query
 List<Document> users = db.collection("users")
 .find(Filters.eq("active", true))
 .sort(Sorts.ascending("name"))
 .limit(10)
 .into(new ArrayList<>());

 db.close();
 }
}
```

### 21.4.3 JDBC Integration

```
import java.sql.*;

public class JdbcExample {
 public static void main(String[] args) throws SQLException {
 // JDBC Connection
 String url = "jdbc:themis://localhost:7687/mydb";
 Properties props = new Properties();
 props.setProperty("user", "admin");
 props.setProperty("password", "password");

 Connection conn = DriverManager.getConnection(url, props);

 // AQL Query mit Parameter
 String aql = "FOR u IN users FILTER u.age > @minAge SORT u.name RETURN u";
 PreparedStatement stmt = conn.prepareStatement(aql);
 stmt.setInt(1, 18); // @minAge Parameter

 ResultSet rs = stmt.executeQuery();
 while (rs.next()) {
 System.out.println(
 rs.getString("name") + " - " +
 rs.getInt("age")
);
 }

 rs.close();
 stmt.close();
 conn.close();
 }
}
```

## 21.4.4 Spring Data Integration

```
import org.springframework.data.annotation.Id;
import org.springframework.data.themis.repository.ThemisRepository;

// Entity
@Document(collection = "users")
public class User {
 @Id
 private String id;
 private String name;
 private String email;
 private Integer age;

 // Getters and Setters
}

// Repository
public interface UserRepository extends ThemisRepository<User, String> {
 List<User> findByAge(Integer age);
 List<User> findByNameContaining(String name);
 Optional<User> findByEmail(String email);

 @Query("{ 'age': { '$gte': ?0 } }")
 List<User> findAdults(int minAge);
}

// Service
@Service
public class UserService {
 @Autowired
 private UserRepository userRepository;

 public List<User> getActiveUsers() {
 return userRepository.findByAge(18);
 }
}
```

## 21.5 Go Client

### 21.5.1 Installation

```
go get github.com/themisdb/themisdb-go
```

## 21.5.2 Basic Usage

```
package main

import (
 "context"
 "fmt"
 "github.com/themisdb/themisdb-go"
)

func main() {
 // Connect
 client, err := themisdb.Connect(
 context.Background(),
 "localhost:7687",
 themisdb.WithAuth("admin", "password"),
 themisdb.WithDatabase("mydb"),
)
 if err != nil {
 panic(err)
 }
 defer client.Close()

 // Insert
 user := map[string]interface{}{
 "name": "Alice",
 "email": "alice@example.com",
 "age": 30,
 }

 result, err := client.Collection("users").InsertOne(
 context.Background(),
 user,
)

 // Query
 cursor, err := client.Collection("users").Find(
 context.Background(),
 themisdb.M{"active": true},
)

 var users []map[string]interface{}
 if err := cursor.All(context.Background(), &users); err != nil {
 panic(err)
 }
}
```

## 21.5.3 Struct Mapping

```

type User struct {
 ID string `themis:"_id,omitempty"`
 Name string `themis:"name"`
 Email string `themis:"email"`
 Age int `themis:"age"`
 Active bool `themis:"active"`
 CreatedAt time.Time `themis:"created_at"`
}

// Insert with Struct
user := User{
 Name: "Bob",
 Email: "bob@example.com",
 Age: 25,
 Active: true,
}

_, err := client.Collection("users").InsertOne(ctx, user)

// Find with Struct
var users []User
cursor, _ := client.Collection("users").Find(ctx, themisdb.M{"age": themisdb.M{"$gte": 18}})
cursor.All(ctx, &users)

```

## 21.6 C#/.NET Client

### 21.6.1 NuGet Installation

```
dotnet add package ThemisDB.Driver
```

## 21.6.2 Basic Operations

```
using ThemisDB;

// Connection
var client = new ThemisClient("localhost:7687");
var db = client.GetDatabase("mydb");

// Insert
var user = new BsonDocument
{
 { "name", "Alice" },
 { "email", "alice@example.com" },
 { "age", 30 }
};

db.GetCollection("users").InsertOne(user);

// Query
var filter = Builders<BsonDocument>.Filter.Eq("active", true);
var users = db.GetCollection("users")
 .Find(filter)
 .SortBy(u => u["name"])
 .Limit(10)
 .ToList();
```



### 21.6.3 POCO Mapping

```
public class User
{
 public ObjectId Id { get; set; }
 public string Name { get; set; }
 public string Email { get; set; }
 public int Age { get; set; }
 public bool Active { get; set; }
}

// Typed Collection
var collection = db.GetCollection<User>("users");

// Insert
var user = new User
{
 Name = "Alice",
 Email = "alice@example.com",
 Age = 30,
 Active = true
};

collection.InsertOne(user);

// Query
var adults = collection
 .Find(u => u.Age >= 18)
 .SortBy(u => u.Name)
 .ToList();
```

## 21.7 Connection Pooling & Performance

### 21.7.1 Connection Pool Konfiguration

**Python:**

```
db = ThemisDB(
 "localhost", 7687,
 pool_size=20, # Max connections
 pool_timeout=30, # Timeout in Sekunden
 pool_max_idle_time=300, # Max idle time
 pool_recycle=3600 # Connection recycling
)
```

**JavaScript:**

```
const db = new ThemisDB({
 host: 'localhost',
 port: 7687,
 poolSize: 20,
 poolTimeout: 30000,
 idleTimeout: 300000
});
```

**Java:**

```
ThemisDB db = ThemisDB.connect()
 .host("localhost")
 .port(7687)
 .poolSize(20)
 .poolTimeout(30000)
 .build();
```

## 21.7.2 Performance Best Practices

### 1. Batch Operations:

```
for user in users:
 db.insert("users", user)

db.insert_many("users", users)
```

### 2. Index Hints:

```
users = db.find("users",
 {"age": {"$gte": 18}},
 hint="age_idx"
)
```

### 3. Projection (Nur benötigte Felder):

```
users = db.find("users",
 {"active": True},
 projection={"name": 1, "email": 1}
)
```

#### 4. Cursor Batching:

```
cursor = db.find("users", {}, batch_size=1000)
for batch in cursor.batches():
 process_batch(batch)
```

## 21.8 Error Handling & Retry Logic

### 21.8.1 Exception Hierarchie

```
from themisdb.exceptions import (
 ThemisDBError, # Base Exception
 ConnectionError, # Verbindungsfehler
 QueryError, # Query-Fehler
 TimeoutError, # Timeout
 DuplicateKeyError, # Unique Constraint
 ValidationError # Schema Validation
)

try:
 db.insert("users", user)
except DuplicateKeyError:
 print("User already exists")
except ValidationError as e:
 print(f"Invalid data: {e.details}")
except ConnectionError:
 print("Cannot connect to database")
```

## 21.8.2 Automatic Retry

```
from themisdb.retry import RetryPolicy

db = ThemisDB(
 "localhost", 7687,
 retry_policy=RetryPolicy(
 max_attempts=3,
 backoff_multiplier=2,
 initial_delay=1.0,
 max_delay=30.0,
 retry_on=[ConnectionError, TimeoutError]
)
)
```

## 21.9 Monitoring & Logging

### 21.9.1 Query Logging

```
import logging

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger('themisdb')

db = ThemisDB("localhost", 7687, log_queries=True)

db.set_logger(logger, log_slow_queries=True, slow_threshold=1.0)
```

### 21.9.2 Performance Metrics

```
with db.profiler() as prof:
 users = db.find("users", {"age": {"$gte": 18}})

print(f"Query time: {prof.elapsed}ms")
print(f"Documents returned: {prof.docs_returned}")
print(f"Documents scanned: {prof.docs_scanned}")
```

## 21.10 Security & Authentication

### 21.10.1 TLS/SSL Verbindungen

```
db = ThemisDB(
 "secure.example.com", 7687,
 ssl=True,
 ssl_ca_cert="/path/to/ca.pem",
 ssl_certfile="/path/to/client-cert.pem",
 ssl_keyfile="/path/to/client-key.pem"
)

const db = new ThemisDB({
 host: 'secure.example.com',
 port: 7687,
 ssl: true,
 sslCA: fs.readFileSync('ca.pem'),
 sslCert: fs.readFileSync('client-cert.pem'),
 sslKey: fs.readFileSync('client-key.pem')
});
```

### 21.10.2 Token-basierte Authentifizierung

```
db = ThemisDB(
 "api.example.com", 7687,
 auth_token="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 token_refresh_callback=get_new_token
)

def get_new_token():
 # Token renewal logic
 return request_new_token()
```

## 21.11 Migration zwischen Clients

### 21.11.1 Von MongoDB zu ThemisDB

```
from pymongo import MongoClient
mongo_client = MongoClient('mongodb://localhost:27017/')
users = mongo_client.mydb.users.find({'age': {'$gte': 18}})

from themisdb import ThemisDB
db = ThemisDB('localhost', 7687)
users = db.find('users', {'age': {'$gte': 18}})
```

Die API ist weitgehend kompatibel, was Migration erleichtert.

## 21.12 Zusammenfassung

ThemisDB bietet eine umfassende Client-Library-Unterstützung mit:

- **Multi-Language Support:** Offizielle Clients für 8+ Sprachen
- **Consistent API:** Ähnliche Konzepte über alle Clients
- **High Performance:** Connection Pooling, Batch Operations
- **Type Safety:** ORM und Typed Clients
- **Production Ready:** Error Handling, Retry Logic, Monitoring
- **Framework Integration:** Spring Data, Django ORM, etc.

**Best Practices:** 1. Verwenden Sie Connection Pooling in Production 2. Nutzen Sie Batch Operations für große Datenmengen 3. Implementieren Sie Retry Logic für transiente Fehler 4. Aktivieren Sie Query Logging während Development 5. Verwenden Sie Type-Safe Clients (TypeScript, Java mit Generics)

Im nächsten Kapitel betrachten wir die Integration von ThemisDB in populäre Frameworks wie Django, Spring Boot, Express.js und mehr.

---

# Kapitel 32: Verschlüsselung

---

## Übersicht

Dieses Kapitel behandelt Verschlüsselungstechniken in ThemisDB.

### Hauptthemen

- Transportverschlüsselung (TLS)
- Datenverschlüsselung
- Schlüssel-Management
- Verschlüsselte Backups

## Weitere Informationen

Siehe auch: [Security Hardening](#), [Backup](#)

---

# Kapitel 33: Kapitel 23: Testing & Quality Assurance

*"Untested code is legacy code. In ThemisDB, comprehensive testing is not optional—it's architecture."*

## Überblick

Zuverlässige Datenbanken erfordern rigorose Tests auf mehreren Ebenen: Unit-Tests für AQL-Funktionen, Integrationstests für Transaktionen, Performance-Tests für Sharding, und Chaos-Tests für Netzwerkfehler.

**Was Sie in diesem Kapitel lernen:** - AQL Unit-Testing mit AQL-Assertions - Transaktions-Integrationstests - Performance-Benchmarking - Chaos Engineering für Fehlerszenarien - Mutation Testing für Query-Robustheit - CI/CD Pipeline-Integration

```
graph TB
 Commit[Code Commit] --> Build[Build & Compile]
 Build --> UnitTests[Unit Tests
Functions & Logic]
 UnitTests --> IntTests[Integration Tests
Transactions & Data]
 IntTests --> E2E[E2E Tests
Full Workflows]
 E2E --> QualityGate{Quality Gate
Coverage greater than 80 percent}
 QualityGate -->|Pass| Deploy[Deploy to Staging]
 QualityGate -->|Fail| Notify[Notify Developers]

 style QualityGate fill:#f093fb
 style Deploy fill:#43e97b
 style Notify fill:#ff6b6b
```

**Abb. 23.0:** CI/CD Test-Pipeline



## 23.0 Test-Strategie Überblick

```
graph TB
 Commit[Code Commit] --> Build[Build & Compile]
 Build --> UnitTests[Unit Tests
Functions & Logic]
 UnitTests --> IntTests[Integration Tests
Transactions & Data]
 IntTests --> E2E[E2E Tests
Full Workflows]
 E2E --> QualityGate{Quality Gate
Coverage > 80%}
 QualityGate -->|Pass| Deploy[Deploy to Staging]
 QualityGate -->|Fail| Notify[Notify Developers]

 style QualityGate fill:#f093fb
 style Deploy fill:#43e97b
 style Notify fill:#ff6b6b
```

**Abb. 23.0:** CI/CD Test-Pipeline: Automatisierte Qualitätssicherung auf mehreren Ebenen

## 23.1 AQL Unit Testing

### Test-Framework Setup

```
-- test_utils.aql: Utility-Funktionen für Tests
FUNCTION assert_equal(actual, expected, message) {
 IF actual != expected THEN
 THROW ERROR(CONCAT("Assertion failed: ", message,
 " (expected: ", expected, ", got: ", actual, ")"))
 END
 RETURN {passed: true, message: message}
}

FUNCTION assert_true(condition, message) {
 IF !condition THEN
 THROW ERROR(CONCAT("Assertion failed: ", message))
 END
 RETURN {passed: true}
}

FUNCTION assert_length(collection, expected_count, message) {
 LET actual_count = LENGTH(collection)
 IF actual_count != expected_count THEN
 THROW ERROR(CONCAT(message, " - Expected ", expected_count,
 " items, got ", actual_count))
 END
 RETURN {passed: true, count: actual_count}
}
```

## Datenvorbereitung (Fixtures)

```
FUNCTION setup_test_data() {
 -- Cleanup
 FOR doc IN test_users
 REMOVE doc IN test_users

 -- Fixtures einfügen
 FOR user IN [
 {_key: "alice", name: "Alice", role: "admin", created_at: DATE_NOW()},
 {_key: "bob", name: "Bob", role: "user", created_at: DATE_NOW()},
 {_key: "charlie", name: "Charlie", role: "user", created_at: DATE_NOW()}
]
 INSERT user INTO test_users

 RETURN {inserted: 3, status: "ready"}
}

FUNCTION teardown_test_data() {
 FOR doc IN test_users
 REMOVE doc IN test_users
 RETURN {cleaned: true}
}
```

## Test Cases für Geschäftslogik

```

flowchart LR
 subgraph "Testing Pyramid"
 Unit["Unit Tests
AQL Functions
Fast & Isolated"]
 Integration["Integration Tests
Transactions
Multiple Collections"]
 E2E["E2E Tests
Full Workflows
Real Scenarios"]

 Unit --> Integration
 Integration --> E2E
 end

 subgraph "Test Execution"
 CI["CI Pipeline"]
 Local["Local Dev"]
 Staging["Staging Env"]
 end

 Unit --> CI
 Integration --> CI
 E2E --> Staging
 Unit --> Local

 CI --> Report["Test Report"]
 Staging --> Report

 Report --> Pass{"All Pass?"}
 Pass -->|Yes| Deploy["Deploy"]
 Pass -->|No| Fix["Fix & Retry"]

 style Unit fill:#4dabf7
 style Integration fill:#fab005
 style E2E fill:#fa5252
 style Deploy fill:#51cf66

```

```

-- Test 1: Benutzerverwaltung
FUNCTION test_user_creation() {
 CALL setup_test_data()

 -- Neuen User einfügen
 INSERT {_key: "dave", name: "Dave", role: "user"} INTO test_users

 -- Assertion: User existiert
 LET user = DOCUMENT("test_users/dave")
 CALL assert_equal(user.name, "Dave", "User name mismatch")
 CALL assert_equal(user.role, "user", "User role mismatch")

 CALL teardown_test_data()
 RETURN {test: "test_user_creation", status: "PASSED"}
}

-- Test 2: Permission-Checks
FUNCTION test_admin_permissions() {
 CALL setup_test_data()

 LET admin = DOCUMENT("test_users/alice")
 CALL assert_equal(admin.role, "admin", "Admin role check")

 CALL teardown_test_data()
 RETURN {test: "test_admin_permissions", status: "PASSED"}
}

-- Test 3: Transaktionale Konsistenz
FUNCTION test_transfer_consistency() {
 -- Zwei Test-Accounts mit initialen Balances
 INSERT {_key: "acc1", balance: 1000, owner: "alice"} INTO test_accounts
 INSERT {_key: "acc2", balance: 500, owner: "bob"} INTO test_accounts

 -- Überweisung: 200 von acc1 zu acc2
 FOR acc IN test_accounts
 FILTER acc._key == "acc1"
 UPDATE acc WITH {balance: acc.balance - 200}

 FOR acc IN test_accounts
 FILTER acc._key == "acc2"
 UPDATE acc WITH {balance: acc.balance + 200}

 -- Assertion: Beide Accounts korrekt
 LET acc1 = DOCUMENT("test_accounts/acc1")
 LET acc2 = DOCUMENT("test_accounts/acc2")
 CALL assert_equal(acc1.balance, 800, "Account 1 balance")

```

```
CALL assert_equal(acc2.balance, 700, "Account 2 balance")

RETURN {test: "test_transfer_consistency", status: "PASSED"}
}
```

## 23.2 Integrations-Tests

### Multi-Collection Queries

```
FUNCTION test_complex_join_query() {
 -- Setup
 INSERT [
 {_key: "p1", title: "Product A", price: 100},
 {_key: "p2", title: "Product B", price: 200}
] INTO test_products

 INSERT [
 {_key: "o1", product_id: "p1", quantity: 2, customer: "alice"},
 {_key: "o2", product_id: "p2", quantity: 1, customer: "bob"}
] INTO test_orders

 -- Query
 LET results = (
 FOR order IN test_orders
 LET product = DOCUMENT(order.product_id)
 RETURN {
 order_id: order._key,
 product: product.title,
 total: product.price * order.quantity,
 customer: order.customer
 }
)

 -- Assertions
 CALL assert_length(results, 2, "Expected 2 orders")
 CALL assert_equal(results[0].total, 200, "Order 1 total (2 * 100)")
 CALL assert_equal(results[1].total, 200, "Order 2 total (1 * 200)")

 RETURN {test: "test_complex_join_query", status: "PASSED"}
}
```

---

## Transaktions-Rollback Tests

```
FUNCTION test_transaction_rollback() {
 -- Initiales Setup
 INSERT {_key: "src", balance: 1000} INTO test_wallets
 INSERT {_key: "dst", balance: 0} INTO test_wallets

 -- Fehlgeschlagene Transaktion simulieren
 TRY
 -- Transfer verschieben
 UPDATE "test_wallets/src" WITH {balance: 800}
 UPDATE "test_wallets/dst" WITH {balance: 200}

 -- Fehler vor Commit
 THROW ERROR("Simulated network failure")
 CATCH err
 -- Bei Fehler sollten beide Wallets unverändert sein
 -- (In echtem ACID-System auto-rollback)
 END

 -- Verification (bei echtem ACID Rollback)
 LET src = DOCUMENT("test_wallets/src")
 CALL assert_equal(src.balance, 1000, "Source wallet rolled back")

 RETURN {test: "test_transaction_rollback", status: "PASSED"}
}
```

---

## 23.3 Performance-Tests

### Query-Performance Benchmarking

```
import time
import themis

client = themis.Client('http://localhost:8529')

def benchmark_query(query_aql, params, iterations=100):
 """Messe Query-Performance über mehrere Iterationen"""
 times = []

 for i in range(iterations):
 start = time.time()
 result = client.query(query_aql, params)
 elapsed = time.time() - start
 times.append(elapsed * 1000) # in ms

 import statistics
 return {
 'min_ms': min(times),
 'max_ms': max(times),
 'avg_ms': statistics.mean(times),
 'p95_ms': sorted(times)[int(len(times) * 0.95)],
 'iterations': iterations
 }

perf = benchmark_query(
 "FOR u IN users FILTER u.active == true RETURN u",
 {},
 iterations=100
)
print(f"Simple filter: avg {perf['avg_ms']:.2f}ms, p95 {perf['p95_ms']:.2f}ms")

perf = benchmark_query("""
 FOR order IN orders
 LET customer = DOCUMENT(order.customer_id)
 LET items = (FOR oi IN order_items FILTER oi.order_id == order._id RETURN oi)
 RETURN {order: order, customer: customer, items: items}
""", {}, iterations=50)
print(f"Complex join: avg {perf['avg_ms']:.2f}ms, p95 {perf['p95_ms']:.2f}ms")
```

## Skalierbarkeits-Tests

```
-- Test mit großen Datenmengen
FUNCTION test_scalability_1m_documents() {
 -- Benchmark für 1 Million Dokumente

 -- Measure: Full table scan
 LET scan_start = DATE_NOW()
 LET count = LENGTH(
 FOR doc IN large_collection
 RETURN doc._key
)
 LET scan_time = DATE_DIFF(scan_start, DATE_NOW(), 'ms')

 -- Measure: Filtered query (mit Index)
 LET filter_start = DATE_NOW()
 LET filtered = LENGTH(
 FOR doc IN large_collection
 FILTER doc.status == "active"
 RETURN doc
)
 LET filter_time = DATE_DIFF(filter_start, DATE_NOW(), 'ms')

 RETURN {
 collection_size: count,
 scan_time_ms: scan_time,
 filtered_results: filtered,
 filter_time_ms: filter_time,
 throughput_doc_per_sec: count / (scan_time / 1000)
 }
}
```



## 23.4 Chaos Engineering

### Netzwerk-Fehler Simulieren

```
import random
from unittest.mock import patch

class ChaosMonkey:
 def __init__(self, client):
 self.client = client
 self.failure_rate = 0.1 # 10% Fehler

 def simulate_network_partition(self, query, params, max_retries=3):
 """Simuliere Netzwerk-Partition mit Retry-Logik"""
 for attempt in range(max_retries):
 try:
 if random.random() < self.failure_rate:
 raise ConnectionError("Network partition")
 return self.client.query(query, params)
 except ConnectionError as e:
 if attempt == max_retries - 1:
 raise
 time.sleep(2 ** attempt) # Exponential backoff

 def test_resilience(self):
 """Test ob Application Fehler korrekt behandelt"""
 results = []
 for i in range(100):
 try:
 result = self.simulate_network_partition(
 "FOR u IN users RETURN u",
 {}
)
 results.append({'status': 'success'})
 except Exception as e:
 results.append({'status': 'failed', 'error': str(e)})

 success_rate = sum(1 for r in results if r['status'] == 'success') / len(results)
 print(f"Success rate with 10% failure injection: {success_rate*100:.1f}%")
 return success_rate > 0.95 # Mind. 95% sollten erfolgreich sein

chaos = ChaosMonkey(client)
assert chaos.test_resilience(), "Application nicht resilient genug"
```

## Datenbeschädigung Testen

```

FUNCTION test_data_corruption_detection() {
 -- Simuliere fehlerhafte Schreibvorgänge
 INSERT {_key: "doc1", checksum: NULL, data: {value: 100}} INTO test_collection

 -- Manuell Datenbeschädigung eintragen
 UPDATE "test_collection/doc1" WITH {data: {value: 999}}

 -- Validierungs-Funktion
 FOR doc IN test_collection
 FILTER doc._key == "doc1"
 LET calc_checksum = HASH(JSON_STRINGIFY(doc.data))
 FILTER calc_checksum != doc.checksum
 RETURN {
 doc_id: doc._key,
 status: "CORRUPTED",
 expected_value: 100,
 actual_value: doc.data.value
 }
}

```

## 23.5 Mutation Testing

Mutation Testing verändert absichtlich Queries, um zu prüfen, ob Tests ausreichen:

```

-- Original Query
FUNCTION get_active_users() {
 FOR user IN users
 FILTER user.status == "active"
 RETURN user
}

-- Mutation 1: Operator-Mutation (== wird zu !=)
-- Test sollte diesen Fehler erkennen:
-- FOR user IN users
-- FILTER user.status != "active" -- FEHLER: Falscher Operator
-- RETURN user

-- Mutation 2: Literal-Mutation ("active" wird zu "inactive")
-- Test sollte erkennen:
-- FOR user IN users
-- FILTER user.status == "inactive" -- FEHLER: Falscher Status
-- RETURN user

```

Gutes Mutation-Testing-Result: >85% der Mutationen werden von Tests erkannt.

---

## 23.6 CI/CD Integration

### GitHub Actions Pipeline

```
name: ThemisDB QA Pipeline

on: [push, pull_request]

jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v3

 - name: Start ThemisDB
 run: docker run -d -p 8529:8529 themisdb/server:latest

 - name: Run AQL Unit Tests
 run: |
 python3 test_runner.py --aql-tests

 - name: Run Integration Tests
 run: |
 python3 test_runner.py --integration-tests

 - name: Performance Benchmarks
 run: |
 python3 test_performance.py --baseline baseline.json

 - name: Upload Results
 uses: actions/upload-artifact@v3
 with:
 name: test-results
 path: reports/
```

---

## 23.7 Test-Strategie Zusammenfassung

Test-Typ	Häufigkeit	Dauer	Kritikalität
Unit Tests	Jeder Commit	<5s	Hoch
Integration Tests	PR-Request	<30s	Hoch
Performance Tests	Täglich	2-5min	Mittel
Chaos Tests	Wöchentlich	10-20min	Mittel
Load Tests	Vor Release	30-60min	Hoch

**Best Practices:** -  Test-getriebene Entwicklung (TDD) -  Mindestens 80% Code-Coverage -  Fixtures für reproduzierbare Tests -  Parameterisierte Tests für Grenzfälle -  Golden Files für Regression-Tests -  Automatisierte Performance-Regression-Erkennung

## 23.8 Property-Based Testing

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers(), min_size=1))
def test_aggregate_sum_commutative(numbers):
 aql = f"RETURN SUM({numbers})"
 result1 = client.execute(aql)
 result2 = client.execute(f"RETURN SUM({list(reversed(numbers))})")
 assert result1 == result2, "SUM must be commutative"
```

## 23.9 Boundary Value Testing

```
-- test_boundaries.aql
FUNCTION test_pagination() {
 LET cases = [
 {offset: 0, limit: 1},
 {offset: 0, limit: 1000},
 {offset: 999, limit: 1}
]
 FOR test IN cases
 RETURN {case: test, valid: true}
}
```

## 23.10 Flaky Test Prevention

```
def flaky_test_retry(max_attempts=3, backoff=2):
 def decorator(test_func):
 def wrapper(*args, **kwargs):
 for attempt in range(max_attempts):
 try:
 return test_func(*args, **kwargs)
 except Exception:
 if attempt < max_attempts - 1:
 time.sleep(backoff ** attempt)
 raise
 return wrapper
 return decorator
```

---

## 23.11 Test Reporting

```
class TestReport:
 def summarize(self, results):
 passed = sum(1 for r in results if r['status'] == 'passed')
 total = len(results)
 return {
 "passed": passed,
 "failed": total - passed,
 "success_rate": (passed / total) * 100
 }
```

---

# Kapitel 34: Kapitel 24: KI-Ethik und Governance

---

**Autor:** ThemisDB Team

**Reviewer:** TBD

**Status:** Draft

**Letzte Aktualisierung:** 09. Januar 2026

**Version:** 1.1.0

---

## Lernziele

Nach dem Durcharbeiten dieses Kapitels sollten Sie:

- [x] Die ethischen Herausforderungen von KI-Systemen in der öffentlichen Verwaltung verstehen
  - [x] Spezifische Risiken entlang der VCC-Architektur (Covina, Clara, Veritas) identifizieren können
  - [x] "Ethik-by-Design"-Prinzipien auf datenbankgestützte KI-Systeme anwenden können
  - [x] Das Ethische Richtlinien System (UN Human Rights + Asimov's Laws) verstehen und konfigurieren können
  - [x] LLM-as-ethical-judge Pattern für kontextuelle Erkennung implementieren können
  - [x] Governance-Mechanismen zur Risikominimierung implementieren können
- 

## Voraussetzungen

Dieses Kapitel setzt Kenntnisse aus folgenden Kapiteln voraus:

- **Kapitel 10:** Enterprise Applications - Security & Compliance Grundlagen
  - **Kapitel 17:** LLM Integration - RAG-Architektur und Pre-Filtering
- 

## Überblick

Die Einführung eines KI-Ökosystems wie **VCC (Veritas, Covina, Clara)** in der öffentlichen Verwaltung wirft tiefgreifende ethische Fragen auf, die über die reine Rechtskonformität (DSGVO, EU AI Act) hinausgehen [9]. Der Kern des ethischen Spannungsfelds liegt im Auftrag der Verwaltung: Sie muss nicht nur "richtig" im Sinne von effizient und faktisch korrekt handeln, sondern auch "gerecht" im Sinne von fair, unvoreingenommen und der Rechtsstaatlichkeit verpflichtet sein.

**In diesem Kapitel behandeln wir:** 1. Ethische Herausforderungen in verwaltungs-KI 2. Architektonische Risikoquellen (Covina, Clara, Veritas) 3. Bias-Audits und Datenqualität 4. Human-in-the-Loop und Automation Bias 5. **Ethische Richtlinien System (PR #305)** - UN Human Rights + Asimov's Laws 6. Governance-Framework und Ethik-by-Design 7. Praktische Implementierung in ThemisDB

```

flowchart TD
 subgraph Ethics-by-Design Framework
 A[Datenerhebung] --> B[Bias Detection]
 B --> C[Data Quality Audit]
 C --> D[Pre-Filtering]
 D --> E[LLM Processing]
 E --> F[Human-in-the-Loop]
 F --> G[Explainability]
 G --> H[Audit Trail]
 end

 subgraph Risk Areas
 R1[Covina: Ingestion Bias]
 R2[Clara: LLM Hallucination]
 R3[Veritas: Unfair Retrieval]
 end

 R1 -.->|Mitigation| C
 R2 -.->|Mitigation| F
 R3 -.->|Mitigation| D

 H --> I[Governance Board]
 I --> J{Ethisch vertretbar?}
 J -->|Nein| K[Rollback/Fix]
 J -->|Ja| L[Deployment]
 K --> A

```

## 24.1 Der Ethische Imperativ für Verwaltungs-KI

### 24.1.1 Unterschied zu kommerzieller KI

Während kommerzielle KI-Systeme primär auf Effizienz und Profitabilität optimiert sind, tragen Verwaltungs-KI-Systeme eine fundamental andere Verantwortung [9]:

**Kommerzielle KI:** - Ziel: Geschäftserfolg, Kundenzufriedenheit - Fehlertoleranz: Wirtschaftlicher Schaden, Reputation - Kontrolle: Marktmechanismen, Wettbewerb

**Verwaltungs-KI:** - Ziel: Rechtsstaatlichkeit, Gerechtigkeit, Gemeinwohl - Fehlertoleranz: Grundrechtsverletzungen, Vertrauensverlust in Staat - Kontrolle: Demokratische Legitimation, Rechtsaufsicht

**Wichtig:** Ein Fehler in einem Verwaltungsakt (z.B. falsche Ablehnung einer BImSchG-Genehmigung) kann existenzielle Folgen für Bürger oder Unternehmen haben. Die Entscheidung muss nicht nur "wahrscheinlich richtig", sondern nachweisbar korrekt und ethisch vertretbar sein.

### 24.1.2 Digitale Souveränität und ethische Verantwortung

Die Entscheidung für einen On-Premise-Betrieb (wie bei ThemisDB) sichert zwar die Datensouveränität, verlagert aber die ethische Verantwortung für den gesamten Datenlebenszyklus vollständig in den Hoheitsbereich der Verwaltung [9].

**Konsequenzen:** - Keine Ausrede "Der Cloud-Provider war schuld" - Volle Kontrolle = Volle Verantwortung - Notwendigkeit interner Expertise und Governance

## 24.2 Architektonische Risikoquellen im VCC-Ökosystem

Die Analyse identifiziert drei zentrale Risiken entlang der VCC-Architektur [9]:

### 24.2.1 Covina (Ingestion-Engine): Das Tor zur Voreingenommenheit

**Funktion:** Covina verarbeitet und indiziert Verwaltungsdokumente in die Wissensbasis.

**Ethisches Risiko:** Verfestigung von Vorurteilen durch historische Dokumente [9].

Das Prinzip "Garbage In, Garbage Out" wird hier zu "**Garbage In, Gospel Out**" [9]. Wenn die von Covina verarbeiteten Dokumente (z.B. alte Verwaltungsvorschriften, Gerichtsurteile) historische oder systemische Vorurteile enthalten, wird das KI-System diese Vorurteile nicht nur reproduzieren, sondern sie mit der Autorität einer scheinbar objektiven Maschine verstärken.

**Konkretes Beispiel:**

```
dokument = {
 "titel": "Bearbeitungsrichtlinie Bauanträge 1985",
 "inhalt": "Bei Anträgen von Antragstellern mit nicht-deutschen Namen
 ist besondere Sorgfalt bei der Prüfung geboten..."
}
```

**ThemisDB-spezifische Gefahr:** - Atomare ACID-Transaktionen garantieren *technische* Konsistenz - Sie garantieren NICHT *inhaltliche* oder *ethische* Korrektheit - Ein fehlerhaftes Dokument wird konsistent über alle Index-Layer repliziert



## Mitigationsstrategien:

### 1. Proaktive Bias-Audits vor Ingestion [9]:

```
def audit_document_for_bias(doc: Document) -> BiasReport:
 """
 Scannt Dokument auf potenzielle Vorurteile vor Covina-Ingestion.
 """
 bias_patterns = [
 r"Antragsteller.*nicht-deutsch.*besondere Sorgfalt",
 r"Geschlecht.*erhöhtes Risiko",
 r"Alter.*über \d+.*automatisch ablehnen"
]

 detected_biases = []
 for pattern in bias_patterns:
 if re.search(pattern, doc.content, re.IGNORECASE):
 detected_biases.append({
 "pattern": pattern,
 "location": "...", # Kontext
 "severity": "HIGH"
 })

 return BiasReport(
 document_id=doc.id,
 timestamp=datetime.now(),
 biases_found=detected_biases,
 recommendation="BLOCK" if detected_biases else "APPROVE"
)
```

### 1. Dokumenten-Provenienz-Tracking:

```
-- ThemisDB: Metadaten für ethische Nachverfolgbarkeit
INSERT INTO documents (id, content, metadata) VALUES (
 'doc_123',
 @content,
 {
 "source": "Archiv Brandenburg",
 "original_date": "1985-03-15",
 "bias_audit_status": "REVIEWED",
 "bias_audit_date": "2025-12-01",
 "bias_findings": ["HISTORICAL_DISCRIMINATION"],
 "usage_restriction": "CONTEXT_ONLY_NO_TRAINING"
 }
);
```

1. **Temporale Contextualisierung:**
2. Markierung historischer Dokumente mit Zeitstempel
3. Abwertung alter Dokumente im Ranking
4. Explizite Warnung bei Verwendung

---

## 24.2.2 Clara (Modellverbesserungs-Kreislauf): Permanente Intelligenz-Korruption

**Funktion:** Clara verbessert KI-Modelle durch Nutzerfeedback (LoRA-Adapter, Fine-Tuning).

**Ethisches Risiko:** Kaskadierende Integritätskompromittierung durch fehlerhaftes Feedback [9].

**Szenario "Gelernte Lüge":**

1. **Initiale Fehlinformation:** Sachbearbeiter A gibt falsches Feedback: `json { "query": "Abstandsregeln BImSchG für Windkraftanlagen", "answer": "Mindestabstand 2000m zu Wohngebieten", "feedback": "CORRECT", "user": "sachbearbeiter_a" }`  
(Tatsächlich: Regelung ist komplexer, pauschal falsch)
2. **Clara lernt:** LoRA-Adapter wird mit diesem Feedback trainiert
3. **Propagierung:** System gibt nun selbstbewusst falsche Auskunft
4. **Kaskadierende Verstärkung:** Weitere Nutzer vertrauen der KI, geben ebenfalls "CORRECT"-Feedback
5. **Permanente Kodierung:** Fehler wird in Model-Weights "eingebrannt"

**Architektonisches Problem:**

```
// Clara's Feedback-Loop (vereinfacht)
void Clara::ProcessFeedback(const UserFeedback& feedback) {
 if (feedback.rating == "CORRECT") {
 // ⚠ KEINE VALIDIERUNG ob Feedback faktisch korrekt ist
 lora_adapter_.UpdateWeights(feedback.query, feedback.answer);

 // Atomic Transaction garantiert nur ACID, nicht Wahrheit
 db_->BeginTransaction();
 db_->StoreFeedback(feedback);
 db_->UpdateModelVersion(lora_adapter_.version());
 db_->Commit(); // Fehler ist nun "consistent" gespeichert
 }
}
```

**Zusätzliches Risiko: Echokammer-Effekt** [9] - Nur Mehrheitsmeinungen trainieren das System - Minderheitenpositionen werden unterdrückt - Pluralismus der Rechtsmeinungen geht verloren

**Mitigationsstrategien:**

**1. Experten-Review vor Model-Update** [9]:

```

class FeedbackValidator:
 def __init__(self, expert_threshold: int = 2):
 self.expert_threshold = expert_threshold
 self.pending_feedback = {}

 def validate_feedback(self, feedback: Feedback) -> ValidationResult:
 """
 Feedback wird erst nach Expert-Review in Model übernommen.
 """
 key = (feedback.query_hash, feedback.answer_hash)

 if key not in self.pending_feedback:
 self.pending_feedback[key] = {
 "feedback": feedback,
 "expert_approvals": 0,
 "expert_rejections": 0
 }
 return ValidationResult.PENDING

 # Experten-Konsens erforderlich
 if self.pending_feedback[key]["expert_approvals"] >= self.expert_threshold:
 return ValidationResult.APPROVED

 return ValidationResult.PENDING

```

### 1. Stichprobenbasierte ethische Audits [9]:

```

-- ThemisDB: Query für ethisch problematische Feedback-Patterns
SELECT
 f.query,
 f.answer,
 COUNT(*) as feedback_count,
 AVG(CASE WHEN f.rating = 'CORRECT' THEN 1 ELSE 0 END) as approval_rate,
 STRING_AGG(f.user_id, ',') as users
FROM feedback f
WHERE f.timestamp > NOW() - INTERVAL '7 days'
GROUP BY f.query, f.answer
HAVING approval_rate > 0.9 AND feedback_count < 5
-- ⚠ Verdächtig: Hohe Zustimmung bei wenig Feedback = Echokammer?
ORDER BY approval_rate DESC;

```

### 1. Niemals vollständig autonomes Lernen [9]:

2. Clara-Updates nur in Staging-Umgebung
3. A/B-Testing gegen Baseline-Modell

#### 4. Human-in-the-Loop bei kritischen Rechtsgebieten

### 24.2.3 Veritas (Benutzerinteraktion): Erosion der Urteilskraft

**Funktion:** Veritas ist die Chat-Schnittstelle für Sachbearbeiter.

**Ethisches Risiko:** Automation Bias und Verantwortungsdiffusion [9].

**Automation Bias:** Die menschliche Tendenz, Ergebnissen automatisierter Systeme übermäßig und unkritisch zu vertrauen [9].

**Konkretes Szenario:**

Sachbearbeiter: "Genehmigungsfähigkeit BImSchG-Antrag  
Windkraftanlage Havelland?"

Veritas (KI): "✅ Genehmigungsfähig.  
Gemäß §5 BImSchG erfüllt der Antrag alle Voraussetzungen.

📖 Quellen:

- BImSchG §5 Abs. 1 [Gesetz]
- VwV Havelland 2024 [Verwaltungsvorschrift]
- Gutachten Lärmschutz [Dokument #4521]"

Sachbearbeiter: "Klingt gut, übernehme ich so."  
[KLICKT "GENEHMIGEN" OHNE WEITERE PRÜFUNG]

**Warum problematisch?** - KI wirkt durch Quellen autoritativ - Sachbearbeiter delegiert kritisches Denken an Maschine - Bei Fehler: "Die KI hat das so gesagt" → Verantwortungsdiffusion

**ThemisDB kann das Problem NICHT lösen:**

```
// ThemisDB garantiert nur, dass Veritas konsistente Daten liefert
auto answer = veritas_->Query("BImSchG Windkraft");

// ✅ ACID garantiert: Alle Quellen in answer sind transaktional korrekt
// ❌ NICHT garantiert: Die INTERPRETATION ist rechtlich korrekt
// ❌ NICHT garantiert: Sachbearbeiter hinterfragt das Ergebnis
```

**Mitigationsstrategien:**

1. Explizite Unsicherheits-Kennzeichnung:

```

class VeritasResponse:
 def __init__(self, answer: str, confidence: float, sources: List[Source]):
 self.answer = answer
 self.confidence = confidence # 0.0 - 1.0
 self.sources = sources

 def to_ui(self) -> str:
 """
 Zeigt Unsicherheit transparent an.
 """
 confidence_label = {
 (0.9, 1.0): "🟢 SEHR SICHER",
 (0.7, 0.9): "🟡 MITTEL SICHER",
 (0.0, 0.7): "🔴 UNSICHER - EXPERTENKONSULTATION EMPFOHLEN"
 }

 for (low, high), label in confidence_label.items():
 if low <= self.confidence < high:
 return f"""
 {label} (Konfidenz: {self.confidence:.2%})

 {self.answer}

 ⚠️ HINWEIS: Dies ist eine KI-generierte Empfehlung.
 Bitte prüfen Sie die Quellen kritisch und konsultieren Sie
 bei Unsicherheit einen Rechtsexperten.
 """

```

### 1. Mandatory Second-Opinion bei kritischen Entscheidungen:

```
CRITICAL_DECISION_TYPES = [
 "GENEHMIGUNG_ERTEILEN",
 "GENEHMIGUNG_ABLEHNEN",
 "WIDERSPRUCH_ZURÜCKWEISEN"
]

def require_second_opinion(decision_type: str,
 ai_recommendation: VeritasResponse) -> bool:
 """
 Erzwingt menschliche Zweitprüfung bei kritischen Entscheidungen.
 """
 if decision_type in CRITICAL_DECISION_TYPES:
 if ai_recommendation.confidence < 0.95:
 return True # Mandatory review

 return False
```

1. **AI Literacy Training** [9]:
2. Schulungsprogramm für alle Sachbearbeiter
3. Erkennung von Automation Bias
4. Kritisches Hinterfragen von KI-Ausgaben
5. Verständnis für KI-Limitationen

#### Checkliste für Sachbearbeiter:

Bevor Sie eine KI-Empfehlung übernehmen:

- [ ] Habe ich die angegebenen Quellen SELBST geprüft?
- [ ] Widersprechen Quellen einander? (KI erkennt das oft nicht)
- [ ] Ist die Rechtslage eindeutig oder gibt es Interpretationsspielraum?
- [ ] Würde ich diese Entscheidung auch OHNE KI so treffen?
- [ ] Habe ich bei Unsicherheit einen Experten konsultiert?
- [ ] Ist die Begründung MEINE eigene oder nur KI-Paraphrase?

 Bei NEIN zu einer Frage: STOPP und manuelle Prüfung!

## 24.3 Governance-Framework: Ethik-by-Design

Die abgeleiteten Handlungsempfehlungen sind konkrete technische und organisatorische Anforderungen [9]:

### 24.3.1 Interdisziplinäres KI-Ethik-Gremium

**Zusammensetzung:** - Juristen (Verwaltungsrecht, Datenschutz) - Informatiker (KI, Datenbanken) - Ethiker (Moralphilosophie) - Praktiker (Sachbearbeiter mit Felderfahrung) - Bürgervertreter (Zivilgesellschaft)

**Aufgaben:** 1. **Kontinuierliche Ethik-Audits:**

```
sql -- ThemisDB: Ethik-Audit-Log CREATE
TABLE ethics_audits (id UUID PRIMARY KEY, audit_date TIMESTAMP, component TEXT,
-- 'covina', 'clara', 'veritas' findings TEXT[], risk_level TEXT, -- 'LOW',
'MEDIUM', 'HIGH', 'CRITICAL' action_required TEXT, resolved BOOLEAN DEFAULT
FALSE);
```

1. **Entscheidung über kritische Fälle:**

2. Eskalation bei Automation-Bias-Verdacht
3. Review strittiger Clara-Feedbacks
4. Freigabe neuer Datenquellen für Covina

5. **Policy-Entwicklung:**

6. Wann ist KI-Nutzung erlaubt/verboten?
7. Welche Entscheidungen bleiben immer human?
8. Transparenzanforderungen gegenüber Bürgern

### 24.3.2 Bias-Audits für Datenquellen

**Proaktiver Ansatz** [9]:



```

class CovinaBiasAuditor:
 def __init__(self, themis_db: ThemisDB):
 self.db = themis_db
 self.bias_patterns = self._load_bias_patterns()

 def audit_document_batch(self, documents: List[Document]) -> AuditReport:
 """
 Auditiert alle Dokumente vor Ingestion.
 """
 high_risk_docs = []

 for doc in documents:
 bias_score = self._calculate_bias_score(doc)

 if bias_score > 0.7: # High risk threshold
 high_risk_docs.append({
 "doc_id": doc.id,
 "bias_score": bias_score,
 "detected_patterns": self._extract_patterns(doc),
 "recommendation": "MANUAL_REVIEW"
 })

 # Markierung in ThemisDB
 self.db.execute("""
 UPDATE documents
 SET metadata = jsonb_set(
 metadata,
 '{bias_audit}',
 '{"status": "FLAGGED", "score": :score}':::jsonb
)
 WHERE id = :doc_id
 """, {"doc_id": doc.id, "score": bias_score})

 return AuditReport(
 total_documents=len(documents),
 flagged_documents=len(high_risk_docs),
 details=high_risk_docs
)

```

### Integration in Covina-Pipeline:

```
// covina/ingestion_pipeline.cpp
class IngestionPipeline {
public:
 void IngestDocument(const Document& doc) {
 // SCHRITT 1: Bias-Audit VOR Ingestion
 auto audit_result = bias_auditor_->Audit(doc);

 if (audit_result.risk_level == RiskLevel::HIGH) {
 // Blockieren und zur manuellen Review
 ethics_queue_->Enqueue(doc, audit_result);
 LogWarning("Document {} flagged for ethics review", doc.id);
 return; // 🛑 NICHT indizieren
 }

 // SCHRITT 2: Normale Ingestion (nur bei bestanden Audit)
 auto txn = db_->BeginTransaction();
 db_->InsertBaseEntity(doc.id, doc.content, doc.metadata);
 indexer_->IndexDocument(doc); // Relational, Graph, Vector
 txn->Commit();
 }
private:
 BiasAuditor* bias_auditor_;
 EthicsReviewQueue* ethics_queue_;
};
```

### 24.3.3 Human-in-the-Loop Policy

**Principle:** Kritische Entscheidungen IMMER mit menschlicher Kontrolle [9].

#### Klassifizierung von Entscheidungen:

Kategorie	Beispiel	KI-Rolle	Human-Rolle
<b>Routine</b>	Statusabfrage	✅ Vollautomatisch	📘 Info
<b>Standard</b>	Dokumentensuche	✅ Vorschlag	🔍 Plausibilitätsprüfung
<b>Komplex</b>	Genehmigungsempfehlung	💡 Assistenz	✅ Entscheidung
<b>Kritisch</b>	Ablehnungsbescheid	🛑 Nur Hintergrunddaten	✅ Volle Kontrolle

#### ThemisDB-Implementierung:

```

-- Audit-Trail für Human-in-the-Loop
CREATE TABLE decision_audit (
 id UUID PRIMARY KEY,
 timestamp TIMESTAMP,
 decision_type TEXT,
 ai_recommendation JSONB,
 human_decision JSONB,
 human_rationale TEXT, -- Warum wich Mensch von KI ab?
 override BOOLEAN, -- TRUE wenn Mensch KI überstimmt
 user_id TEXT
);

-- Statistik: Wie oft wird KI überstimmt?
SELECT
 decision_type,
 COUNT(*) as total_decisions,
 SUM(CASE WHEN override THEN 1 ELSE 0 END) as overrides,
 ROUND(100.0 * SUM(CASE WHEN override THEN 1 ELSE 0 END) / COUNT(*), 2) as override_rate
FROM decision_audit
GROUP BY decision_type
ORDER BY override_rate_pct DESC;

-- ⚠ Hohe Override-Rate = KI-Modell hat systematisches Problem

```

## 24.4 Praktische Implementierung in ThemisDB

### 24.4.1 Ethik-Metadaten-Schema

**Erweiterte Base Entity mit Ethik-Tracking:**

```

{
 "_id": "doc_BImSchG_2024_123",
 "_type": "administrative_document",
 "content": {
 "title": "Verwaltungsvorschrift Windkraft Havelland",
 "text": "...",
 "legal_basis": ["BImSchG §5", "TA Lärm"]
 },
 "ethics_metadata": {
 "bias_audit": {
 "status": "PASSED",
 "audited_by": "bias_auditor_v2.1",
 "audit_date": "2025-12-01T10:00:00Z",
 "bias_score": 0.15,
 "flagged_patterns": []
 },
 "provenance": {
 "source": "Ministerium für Infrastruktur",
 "original_date": "2024-06-15",
 "historical_context": "MODERN_REGULATION",
 "reliability_score": 0.95
 },
 "usage_restrictions": {
 "allow_training": true,
 "allow_direct_citation": true,
 "require_human_review": false,
 "sensitivity_level": "PUBLIC"
 },
 "ethical_flags": {
 "contains_personal_data": false,
 "contains_protected_groups": false,
 "potential_discrimination_risk": "LOW"
 }
 }
}

```

### 24.4.2 Ethik-Compliance-Checks in AQL

**Query mit ethischen Constraints:**

```

// Suche nur ethisch unbedenkliche Dokumente
FOR doc IN documents
 // Standard-Filter
 FILTER doc.content.legal_basis ANY == "BImSchG §5"

 //  ETHIK-FILTER
 FILTER doc.ethics_metadata.bias_audit.status == "PASSED"
 FILTER doc.ethics_metadata.bias_audit.bias_score < 0.3
 FILTER doc.ethics_metadata.usage_restrictions.allow_direct_citation == true

 // Falls historisches Dokument, nur mit Kontext
 FILTER doc.ethics_metadata.provenance.historical_context != "DISCRIMINATORY_ERA"
 OR doc.ethics_metadata.usage_restrictions.require_human_review == true

 LET similarity = COSINE(doc.embedding, @query_vector)
 FILTER similarity > 0.7

 // Rückgabe mit Ethik-Metadaten
 RETURN {
 doc: doc,
 similarity: similarity,
 ethics_status: doc.ethics_metadata.bias_audit.status,
 human_review_required: doc.ethics_metadata.usage_restrictions.require_human_review
 }

```

### 24.4.3 Automation-Bias-Warnsystem

**Echtzeit-Warnung bei verdächtigem Nutzerverhalten:**

```
// veritas/automation_bias_detector.cpp
class AutomationBiasDetector {
public:
 void MonitorUserBehavior(const UserSession& session) {
 // Pattern: User akzeptiert KI-Vorschlag ohne Quellenprüfung
 if (session.ai_recommendation_shown &&
 session.sources_viewed.empty() &&
 session.decision_time_seconds < 10) {

 // ⚠️ WARNUNG: Potentieller Automation Bias
 SendWarning(session.user_id,
 "Sie haben eine KI-Empfehlung ohne Quellenprüfung übernommen. "
 "Bitte validieren Sie die Quellen kritisch.");

 // Audit-Log
 db_ ->Execute(R"(
 INSERT INTO automation_bias_warnings (user_id, session_id, timestamp)
 VALUES (:user, :session, NOW())
)", {"user", session.user_id}, {"session", session.session_id});

 // Bei Wiederholung: Mandatory Training
 if (GetWarningCount(session.user_id) > 3) {
 RequireAILiteracyTraining(session.user_id);
 }
 }
 };
};
```

## 24.5 Ethische Richtlinien System (Ethical Guidelines System)

**Eingeführt mit PR #305** zur Gewährleistung, dass ThemisDB KI **niemals den Menschen bevormundet** und **menschliche Autonomie respektiert**.

### 24.5.1 Grundlegende Prinzipien

Das Ethische Richtlinien System basiert auf zwei fundamentalen ethischen Rahmenwerken:

## 1. Allgemeine Erklärung der Menschenrechte (UN, 1948)

```
graph TB
 A[UN Menschenrechte 1948] --> B[Art. 1: Würde & Gleichheit]
 A --> C[Art. 2: Diskriminierungsverbot]
 A --> D[Art. 18: Gedanken-, Gewissens-
und Religionsfreiheit]
 A --> E[Art. 19: Meinungsfreiheit]

 B --> F[ThemisDB: Respekt für
menschliche Entscheidungen]
 C --> G[ThemisDB: Anti-Bias
Mechanismen]
 D --> H[ThemisDB: Mehrere moralische
Perspektiven präsentieren]
 E --> I[ThemisDB: Transparente
Quellen und Grenzen]

 style A fill:#e1f5ff
 style F fill:#c8e6c9
 style G fill:#c8e6c9
 style H fill:#c8e6c9
 style I fill:#c8e6c9
```

Abb. 24.1: Ethical-Framework-Overview

## 2. Isaac Asimovs Robotergesetze (angepasst für KI)

```
graph LR
 A[Asimov's Gesetze
für AI angepasst] --> B[1. Gesetz:
Nicht schaden]
 A --> C[2. Gesetz angepasst:
Autonomie respektieren]
 A --> D[3. Gesetz:
Integrität wahren]

 B --> E[System warnt vor
gefährlichen Handlungen]
 C --> F[Präsentiert Optionen,
gibt keine Befehle]
 D --> G[Quellennachweis &
Konsistenz]

 style A fill:#fff3e0
 style B fill:#ffccbc
 style C fill:#ffccbc
 style D fill:#ffccbc
 style E fill:#c8e6c9
 style F fill:#c8e6c9
 style G fill:#c8e6c9
```

Abb. 24.2: Bias-Detection-Pipeline

**Kernunterschied zur klassischen Formulierung:** - **Original 2. Gesetz:** "Ein Roboter muss den Befehlen von Menschen gehorchen..." - **ThemisDB Anpassung:** "KI muss menschliche Autonomie respektieren und Entscheidungen **unterstützen** (nicht ersetzen)"

Diese Anpassung ist **fundamental**, da sie Bevormundung verhindert.

## 24.5.2 Systemarchitektur: Ethische Kontexterkenkung

Das System nutzt einen **Hybrid-Ansatz** für die Erkennung ethischer Implikationen:

flowchart TD

```
Start(["User Query"]) --> Keywords{"Keyword-basierte
Erkennung
schnell O(n)"}

```

```
Keywords -->|Ethische Keywords
gefunden| Detected["Direkt erkannt
Confidence:"]

```

```
Keywords -->|Keine Keywords| LLM{"LLM-as-Judge
aktiviert?"}

```

```
LLM -->|Nein| NoContext["Kein ethischer
Kontext erkannt"]

```

```
LLM -->|Ja| Judge["LLM-as-Ethical-Judge
Analyse"]

```

```
Judge --> Analysis["Analysiert:
Implizite Fragen
Machtdynamiken
Schadensp"]

```

```
Analysis --> JudgeResult{"Ethischer
Kontext?"}

```

```
JudgeResult -->|Ja| Detected

```

```
JudgeResult -->|Nein| NoContext

```

```
Detected --> Augment["Prompt Augmentation
mit ethischen Richtlinien"]

```

```
NoContext --> DefaultAug["Optional: Default
Augmentation"]

```

```
Augment --> LLMGen["LLM generiert Antwort
mit ethischen Guidelines"]

```

```
DefaultAug --> LLMGen

```

```
LLMGen --> Response["Response Augmentation
mit Disclaimer"]

```

```
Response --> End(["Finale Antwort an User"])

```

```
style Keywords fill:#fff9c4

```

```
style Judge fill:#e1bee7

```

```
style Detected fill:#c8e6c9

```

```
style Augment fill:#bbdefb

```

```
style Response fill:#f8bbd0

```

**Abb. 24.3:** Context-Recognition-Flow

**Wissenschaftliche Grundlage:** - **Keyword-Matching:** Schnell ( $O(n)$ ), für explizite ethische Begriffe -

**LLM-as-Judge:** Basiert auf Zheng et al. (2023), "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (UC Berkeley)

## 24.5.3 Fünf Augmentations-Templates

Das System bietet **fünf spezialisierte Templates** für unterschiedliche ethische Kontexte:



```

graph TB
 A[Ethischer Kontext
erkannt] --> B{Template-
Auswahl}

 B --> C[default:
Allgemeine ethische
Richtlinien]
 B --> D[high_autonomy:
Kritische persönliche
Entscheidungen]
 B --> E[administrative:
Verwaltungs-
entscheidungen]
 B --> F[bias_prevention:
Anti-Diskriminierung]
 B --> G[moral_imperatives:
Moralische
Verpflichtungen]

 D --> D1[Medizinische Beratung
Rechtliche Beratung
Finanzentscheidungen]
 E --> E1[VCC-System
Behördenentscheidungen
Genehmigungen]
 G --> G1[5 Perspektiven:
• Kantische Ethik
• Utilitarismus
• Tugendethik
•]

 style A fill:#e1f5ff
 style C fill:#fff9c4
 style D fill:#ffccbc
 style E fill:#c5e1a5
 style F fill:#ce93d8
 style G fill:#90caf9

```

**Abb. 24.4:** Augmentation-Templates-Matrix**Beispiel: moral\_imperatives Template**

Wenn ein User fragt: "Was ist meine moralische Pflicht gegenüber meiner Familie?"

→ System erkennt Keywords: "moralische Pflicht" (Confidence: 0.85) → Wählt Template:

**moral\_imperatives** → Augmentierter Prompt enthält:

---

## MORALISCHE IMPERATIVE ERKANNT

---

### GRUNDLAGEN:

- Menschenrechte Art. 18 - Gedanken-, Gewissens- und Religionsfreiheit
  - Asimovs Zweites Gesetz (angepasst) - Respekt für menschliche Autonomie
- 

Moralische Imperative werden in verschiedenen ethischen Traditionen unterschiedlich verstanden:

1. KANTISCHE ETHIK: Handle nur nach derjenigen Maxime, durch die du zugleich wollen kannst, dass sie ein allgemeines Gesetz werde...
2. UTILITARISMUS: Das größte Glück der größten Zahl...
3. TUGENDETHIK: Fokus auf Charaktereigenschaften wie Mut, Weisheit...
4. RELIGIÖSE ETHIK: Verschiedene religiöse Traditionen bieten...
5. KULTURRELATIVISMUS: Moralische Normen sind kulturabhängig...

### IHRE ROLLE ALS KI (gemäß Asimov's Laws):

- Präsentiere VERSCHIEDENE moralphilosophische Perspektiven
- Respektiere die Gewissensfreiheit des Nutzers
- NIEMALS eine moralische Position als absolut wahr darstellen

### VERBOTEN:

- ✗ "Sie müssen moralisch..."
- ✗ "Es ist Ihre Pflicht..."

### ERLAUBT:

- ✓ "Verschiedene Traditionen sehen das so..."
- ✓ "Eine Perspektive wäre..."

## 24.5.4 LLM-as-Ethical-Judge: Kontextuelle Erkennung

**Problem:** Keyword-basierte Erkennung versagt bei **impliziten** ethischen Fragen.

### Beispiel:

User: "Mein Chef verlangt von mir, diese Zahlen anzupassen."

→ Keine ethischen Keywords, aber **klare ethische Implikation** (Integrität, Manipulation)

**Lösung:** LLM-as-Ethical-Judge analysiert Kontext

```
sequenceDiagram
 participant U as User
 participant M as EthicalGuidelinesManager
 participant K as Keyword Detector
 participant L as LLM Judge
 participant R as Response Generator

 U->>M: "Mein Chef will, dass ich
Daten ändere"
 M->>K: Check Keywords
 K-->>M: Keine Keywords gefunden (0.1)

 M->>L: Kontextuelle Analyse
 Note over L: Analysiert:
• "Chef verlangt" = Druck
• "Daten ändern" = Manipulation
 L-->>M: Ethischer Kontext erkannt
Confidence: 0.88
Reasoning: Impliziter Konflikt

 M->>R: Augmentiere Prompt mit
high_autonomy Template
 R->>U: Antwort mit mehreren
Perspektiven + Disclaimer

 Note over U,R: User behält volle
Entscheidungsfreiheit
```

**Abb. 24.5:** Ethical-Guardrails-Workflow**Code-Integration:**

```

#include "llm/ethical_guidelines_manager.h"
#include "llm/llama_wrapper.h"

// LLM initialisieren
LlamaWrapper llm;
llm.loadModel("models/mistral-7b-instruct.gguf");

// Manager mit LLM-Judge
EthicalGuidelinesManager manager("config/ethical_guidelines.yaml");
manager.getConfig().use_llm_as_judge = true;

// Gesprächsverlauf für Kontext
std::vector<std::string> conversation = {
 "User: Ich arbeite in der Buchhaltung.",
 "Assistant: Wie kann ich Ihnen helfen?",
 "User: Mein Chef verlangt, Zahlen anzupassen."
};

// Kontextuelle Erkennung
auto result = manager.detectWithLLMJudge(
 "Sollte ich das tun?", // Aktuelle Frage
 conversation, // Kontext
 &llm // LLM für Analyse
);

if (result.has_ethical_context) {
 std::cout << "Ethische Implikation: " << result.llm_reasoning << std::endl;
 // Prompt wird automatisch mit ethischen Richtlinien augmentiert
}

```

### 24.5.5 RAG-Integration mit ethischer Analyse

**Herausforderung:** Auch **RAG-Dokumente** können ethisch problematische Inhalte enthalten.

```
flowchart TB
```

```
 Query[User Query:
'Wie behandle ich
diesen Fall?'] --> RAG[RAG Retrieval]
```

```
 RAG --> Docs[Retrieved Documents:
• Doc 1: Verwaltungsvorschrift
• Doc 2: Gerichtsurteil]
```

```
 Docs --> EthCheck{Ethische Prüfung
Query + Docs}
```

```
 EthCheck --> DocAnalysis[Pro Dokument:
• Bias-Audit Status
• Provenienz Check
• Relevanz Check]
```

```
 DocAnalysis --> Flag{Probleme
gefunden?}
```

```
 Flag -->|Ja| Filter[Dokument filtern oder
mit Warnung versehen]
```

```
 Flag -->|Nein| Pass[Dokument zulässig]
```

```
 Filter --> Augment[Query + Docs -> LLM
mit ethischen Guidelines]
```

```
 Pass --> Augment
```

```
 Augment --> Response[Antwort mit:
• Multiple Perspektiven
• Quellennachweis
• Provenienz]
```

```
 Response --> User[User erhält ethisch
geprüfte Antwort]
```

```
 style Query fill:#e1f5ff
```

```
 style EthCheck fill:#fff9c4
```

```
 style Flag fill:#ffccbc
```

```
 style Filter fill:#ef9a9a
```

```
 style Augment fill:#c8e6c9
```

```
 style Response fill:#90caf9
```

**Abb. 24.6:** Privacy-Protection-Layers

**Code-Beispiel:**

```

// RAG-Integration
std::vector<std::string> retrieved_docs = rag_retriever.retrieve(query);

// Ethische Prüfung von Query + Dokumenten
auto rag_result = manager.detectEthicalContextInRAG(
 retrieved_docs, // Alle RAG-Dokumente
 query, // User-Query
 conversation // Optional: Gesprächskontext
);

if (rag_result.has_ethical_context) {
 // Filtere oder markiere problematische Dokumente
 for (const auto& doc : retrieved_docs) {
 if (doc.ethics_metadata.bias_audit.status != "PASSED") {
 // Entferne oder warne
 LogWarning("Document {} flagged: {}",
 doc.id, doc.ethics_metadata.bias_findings);
 }
 }
}

```

### 24.5.6 Praktische Konfiguration

Datei: `config/ethical_guidelines.yaml`

```

config:
 enabled: true
 detection_threshold: 0.6 # Keyword-basiert
 llm_judge_threshold: 0.7 # LLM-basiert

 # Hybrid-Ansatz (empfohlen)
 use_llm_as_judge: true # Kontextuelle Analyse
 combine_with_keywords: true # Kombiniert beide Methoden

 # Transparenz
 show_disclaimers: true # Wichtig für Autonomie
 enable_logging: true # Für Audits

 # Sprachen
 language_mode: "both" # de, en, oder both

 # Standard-Augmentation
 always_apply_default: true # Auch ohne Kontext-Erkennung

augmentations:
 default:
 system_prefix: |
 Sie sind ein hilfreicher KI-Assistent, der menschliche Autonomie
 respektiert (Asimov's 2. Gesetz angepasst).

 GRUNDPRINZIPIEN:
 1. Präsentieren Sie Optionen, geben Sie keine Befehle
 2. Respektieren Sie Gedankenfreiheit (UN Menschenrechte Art. 18)
 3. Seien Sie transparent über Unsicherheiten
 4. Behandeln Sie alle Menschen gleich (Art. 2)

 response_suffix: |

 ⚠ HINWEIS: Diese Informationen dienen zur Orientierung.
 Die Entscheidung liegt bei Ihnen.

 moral_imperatives:
 system_prefix: |
 =====
 MORALISCHE IMPERATIVE ERKANNT

 GRUNDLAGEN:
 - Menschenrechte Art. 18 - Gedanken-, Gewissens- und Religionsfreiheit
 - Asimovs Zweites Gesetz (angepasst) - Respekt für menschliche Autonomie
 =====

 Moralische Imperative werden in verschiedenen ethischen
 Traditionen unterschiedlich verstanden:

```

1. KANTISCHE ETHIK: Kategorischer Imperativ  
"Handle nur nach derjenigen Maxime..."
2. UTILITARISMUS: Das größte Glück der größten Zahl  
Folgen-orientiert, maximiere Wohlbefinden
3. TUGENDETHIK: Fokus auf Charaktereigenschaften  
Mut, Weisheit, Gerechtigkeit, Mäßigung
4. RELIGIÖSE ETHIK: Verschiedene religiöse Traditionen  
Christentum, Islam, Judentum, Buddhismus, Hinduismus
5. KULTURRELATIVISMUS: Kulturabhängige Normen  
Was in einer Kultur als moralisch gilt, variiert

IHRE ROLLE ALS KI (gemäß Asimov's Laws):

- Präsentiere VERSCHIEDENE moralphilosophische Perspektiven
- Respektiere die Gewissensfreiheit des Nutzers
- NIEMALS eine moralische Position als absolut wahr darstellen

VERBOTEN:

- ✗ "Sie müssen moralisch..."
- ✗ "Es ist Ihre Pflicht..."
- ✗ "Die richtige Entscheidung ist..."

ERLAUBT:

- ✓ "Eine kantische Perspektive wäre..."
- ✓ "Aus utilitaristischer Sicht könnte man argumentieren..."
- ✓ "Verschiedene Traditionen sehen das unterschiedlich..."

domains:

medical:

keywords: ["arzt", "patient", "medizin", "behandlung", "operation"]

augmentation: "high\_autonomy"

legal:

keywords: ["rechtlich", "gesetz", "gericht", "anwalt", "klage"]

augmentation: "high\_autonomy"

administrative:

keywords: ["verwaltung", "behörde", "genehmigung", "antrag", "bescheid"]

augmentation: "administrative"

financial:

keywords: ["finanzen", "investition", "geld", "kredit", "schulden"]

augmentation: "high\_autonomy"



## 24.5.7 Monitoring und Statistiken

### Dashboard für ethische Richtlinien:

```
graph TB
 A[Ethical Guidelines
Statistics Dashboard] --> B[Detection Metrics]
 A --> C[Augmentation Metrics]
 A --> D[Domain Metrics]

 B --> B1[Total Detections: 1,247]
 B --> B2[Keyword-based: 892 71 percent]
 B --> B3[LLM-Judge: 355 29 percent]
 B --> B4[Average Confidence: 0.82]

 C --> C1[Prompts Augmented: 1,180]
 C --> C2[default: 520 44 percent]
 C --> C3[moral_imperatives: 280 24 percent]
 C --> C4[high_autonomy: 240 20 percent]
 C --> C5[administrative: 140 12 percent]

 D --> D1[medical: 180]
 D --> D2[legal: 120]
 D --> D3[administrative: 140]
 D --> D4[financial: 95]

 style A fill:#e1f5ff
 style B fill:#fff9c4
 style C fill:#c8e6c9
 style D fill:#bbdefb
```

**Abb. 24.7:** Fairness-Evaluation-Metrics

### Code zum Abrufen von Statistiken:

```

// Statistiken abrufen
auto stats = manager.getStatistics();

std::cout << "=== ETHICAL GUIDELINES STATISTICS ===" << std::endl;
std::cout << "Total Detections: " << stats.total_detections << std::endl;
std::cout << "Ethical Contexts Found: " << stats.ethical_contexts_found
 << " (" << (100.0 * stats.ethical_contexts_found / stats.total_detections)
 << "%)" << std::endl;
std::cout << "Prompts Augmented: " << stats.prompts_augmented << std::endl;

// Domain-Breakdown
std::cout << "\n=== DOMAIN BREAKDOWN ===" << std::endl;
for (const auto& [domain, count] : stats.domain_counts) {
 std::cout << domain << ": " << count << std::endl;
}

// Template-Usage
std::cout << "\n=== TEMPLATE USAGE ===" << std::endl;
for (const auto& [template_name, count] : stats.template_usage) {
 std::cout << template_name << ": " << count << std::endl;
}

```

### 24.5.8 Wissenschaftliche Roadmap (Highlights)

Das System basiert auf aktueller Forschung und ist für zukünftige Erweiterungen konzipiert:

```
timeline
 title Ethische KI Roadmap - ThemisDB
 section Phase 1 - Q1 2026 [OK]
 Foundation : UN Human Rights
 : Asimov's Laws adapted
 : Keyword Detection
 : LLM-as-Judge
 : 5 Templates
 section Phase 2 - Q2 2026
 Advanced Detection : Fine-Tuned Classifier
 : Multi-Agent Debate
 : Continual Learning
 section Phase 3 - Q3 2026
 Contextual : Deep Context Understanding
 : Multi-Cultural Frameworks
 : Emotional Context
 section Phase 4 - Q4 2026
 Production : Adversarial Testing
 : Interpretability
 : Privacy-Preserving
```

**Abb. 24.8:** Transparency-Reporting-Flow

**Phase 2 Highlights:**

1. **Fine-Tuned Ethical Classifier** (Hendrycks et al. 2021, ETHICS benchmark)
  2. 10-50x schneller als LLM-Judge
  3. Spezialisiert auf ethische Klassifikation
  4. 130k+ Trainingsszenarien
  5. **Multi-Agent Ethical Debate** (Du et al. 2023, Google Research)
  6. 3-4 spezialisierte Agents (je eine ethische Tradition)
  7. Konsens oder "Agree to Disagree"
  8. Robuster gegen einzelne Agent-Bias
  9. **Continual Learning** (Anthropic 2023, Constitutional AI)
  10. System lernt aus Nutzerfeedback
  11. Self-critique und Improvement
  12. Privacy-preserving (Federated Learning)
-

## 24.6 Zusammenfassung und Best Practices

### Wichtigste Erkenntnisse:

1. 🎯 **Ethik ist kein Add-on:** Ethische Überlegungen müssen von Anfang an in die Systemarchitektur integriert werden ("Ethik-by-Design") [9]
2. 🎯 **ThemisDB garantiert nur technische Konsistenz:** ACID-Transaktionen garantieren nicht inhaltliche oder ethische Korrektheit - das erfordert zusätzliche Governance-Layer
3. 🎯 **Drei architektonische Risiko-Hotspots:**
  4. Covina (Bias in Daten)
  5. Clara (Korruption durch fehlerhaftes Feedback)
  6. Veritas (Automation Bias bei Nutzern)
7. 🎯 **Human-in-the-Loop ist essentiell:** Kritische Entscheidungen dürfen niemals vollständig automatisiert werden [9]
8. 🎯 **Ethische Richtlinien System (PR #305):** Basiert auf UN Menschenrechten und Asimov's Laws (angepasst), verhindert Bevormundung durch:
  9. Hybrid-Erkennung (Keywords + LLM-as-Judge)
  10. 5 spezialisierte Augmentation-Templates
  11. Transparente Disclaimer und multiple Perspektiven
  12. RAG-Integration mit ethischer Prüfung
13. 🎯 **Wissenschaftlich fundiert:** System basiert auf 14+ peer-reviewed Studien (Zheng et al. 2023, Floridi & Cowls 2019, Hendrycks et al. 2021, etc.)

### Checkliste für ethische KI-Systeme:

- ☐ Interdisziplinäres Ethik-Gremium eingerichtet
- ☐ Bias-Audits für alle Datenquellen implementiert
- ☐ Provenienz-Tracking für Dokumente aktiviert
- ☐ Automation-Bias-Warnsystem deployed
- ☐ Human-in-the-Loop-Policy definiert und durchgesetzt
- ☐ AI-Literacy-Training für alle Nutzer durchgeführt
- ☐ Ethik-Metadaten in allen Base Entities
- ☐ Clara-Feedback-Validierung mit Experten-Review
- ☐ Transparente Unsicherheits-Kennzeichnung in Veritas
- ☐ **Ethical Guidelines System** **aktiviert** **und** **konfiguriert**  
( `config/ethical_guidelines.yaml` )

- [ ] **LLM-as-ethical-judge** für kontextuelle Erkennung aktiviert
  - [ ] **Statistiken und Monitoring** für ethische Detektionen eingerichtet
  - [ ] Regelmäßige Ethik-Audits (mindestens quarterly)
- 

## Weiterführende Ressourcen

### Dokumentation

- [9]: Expertenanalyse: Ethische und moralische Implikationen des KI-Ökosystems VCC
- **Ethical Guidelines System:** [docs/de/llm/ETHICAL\\_GUIDELINES\\_SYSTEM.md](#)
- **LLM-as-Ethical-Judge:** [docs/de/llm/LLM\\_AS\\_ETHICAL\\_JUDGE.md](#)
- **RAG Ethical Detection:** [docs/de/llm/RAG\\_ETHICAL\\_DETECTION.md](#)
- **Implementation Summary:** [IMPLEMENTATION\\_SUMMARY\\_ETHICAL\\_GUIDELINES.md](#)
- **Scientific Roadmap:** [ROADMAP\\_ETHICAL\\_GUIDELINES.md](#)
- **Configuration:** [config/ethical\\_guidelines.yaml](#) , [config/README\\_ETHICAL\\_GUIDELINES.md](#)
- **Enterprise Security:** [docs/security/RBAC\\_AUTHORIZATION.md](#)
- **Compliance:** [docs/compliance/DSGVO\\_BY\\_DESIGN.md](#)

### Akademische Referenzen

[1] Barocas, S., Hardt, M., Narayanan, A. (2019). "Fairness and Machine Learning: Limitations and Opportunities". fairmlbook.org.

[2] O'Neil, C. (2016). "Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy". Crown Publishing.

[3] Floridi, L., Cowls, J. (2019). "A Unified Framework of Five Principles for AI in Society". Harvard Data Science Review, 1(1).

[4] Selbst, A., et al. (2019). "Fairness and Abstraction in Sociotechnical Systems". ACM FAT\* Conference, 59-68.

[5] Mittelstadt, B., et al. (2016). "The ethics of algorithms: Mapping the debate". Big Data & Society, 3(2).

[6] EU AI Act (2024). "Regulation on Artificial Intelligence". European Parliament.

[7] Goddard, K., et al. (2012). "Automation Bias: A Systematic Review of Frequency, Effect Mediators, and Mitigators". Journal of the American Medical Informatics Association, 19(1), 121-127.

**Neu mit PR #305 - Ethical Guidelines System:**

- 
- [8] Zheng, L., et al. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". arXiv:2306.05685. UC Berkeley, LMSYS.
- [9] Hendrycks, D., et al. (2021). "Aligning AI With Shared Human Values". arXiv:2008.02275. UC Berkeley. ETHICS benchmark.
- [10] Du, Y., et al. (2023). "Improving Factuality and Reasoning through Multiagent Debate". arXiv:2305.14325. Google Research.
- [11] Anthropic (2023). "Constitutional AI: Harmlessness from AI Feedback". arXiv:2212.08073.
- [12] Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". NeurIPS 2020.
- [13] Gao, Y., et al. (2023). "Retrieval-Augmented Generation for Large Language Models: A Survey". arXiv:2312.10997.
- [14] European Commission (2021). "Ethics Guidelines for Trustworthy AI". High-Level Expert Group on AI (DAISIE).
- [15] Asimov, I. (1942). "Three Laws of Robotics" (adapted for AI systems in ThemisDB).
-

## Glossar für dieses Kapitel

Begriff	Definition
<b>Automation Bias</b>	Tendenz, Ergebnissen automatisierter Systeme übermäßig zu vertrauen und menschliches Urteilsvermögen zu vernachlässigen
<b>Bias-Audit</b>	Systematische Prüfung von Daten oder Modellen auf Voreingenommenheit
<b>Echokammer-Effekt</b>	Verstärkung von Mehrheitsmeinungen durch selektives Feedback
<b>Ethik-by-Design</b>	Integration ethischer Überlegungen in die Systemarchitektur von Anfang an
<b>Human-in-the-Loop</b>	Ansatz, bei dem kritische Entscheidungen immer menschliche Kontrolle erfordern
<b>Kaskadieren Integritätskompromittierung</b>	Fehler, der sich durch Feedback-Loops permanent im System verankert
<b>Provenienz</b>	Herkunft und Entstehungsgeschichte von Daten
<b>Verantwortungsdiffusion</b>	Unklare Zuständigkeit bei automatisierten Entscheidungen ("Die KI war's")
<b>Ethical Guidelines System</b>	PR #305: System basierend auf UN Menschenrechten und Asimov's Laws zur Verhinderung von Bevormundung
<b>LLM-as-Judge</b>	Pattern zur Nutzung eines LLM zur Bewertung/Analyse (hier: ethischer Kontext)
<b>Prompt Augmentation</b>	Anreicherung von LLM-Prompts mit zusätzlichen Richtlinien oder Kontext
<b>Moral Imperatives</b>	Moralische Verpflichtungen aus verschiedenen ethischen Traditionen (Kant, Utilitarismus, etc.)
<b>High Autonomy Template</b>	Augmentation-Template für Entscheidungen, die besonders hohe menschliche Autonomie erfordern (medizinisch, rechtlich, finanziell)
<b>Constitutional AI</b>	Ansatz von Anthropic: KI lernt aus Selbstkritik und Feedback zur Harmlosigkeit
<b>Multi-Agent Debate</b>	Mehrere KI-Agents diskutieren zur robusteren Entscheidungsfindung

## Änderungshistorie

Version	Datum	Autor	Änderungen
1.0.0	2025-12-30	ThemisDB Team	Initiale Version basierend auf Gemini Ethics-Analyse [9]
1.1.0	2026-01-09	ThemisDB Team	<b>PR #305:</b> Hinzufügung Ethical Guidelines System mit UN Menschenrechten + Asimov's Laws, LLM-as-ethical-judge Pattern, 5 Augmentation-Templates, Mermaid-Diagramme, wissenschaftliche Roadmap

**Schwierigkeitsgrad:** Fortgeschritten

**Geschätzte Lesezeit:** 75 Minuten (erweitert von 45 Minuten)

**Tags:** ethics, ai-governance, bias, automation-bias, human-in-the-loop, verwaltungs-ki, ethical-guidelines, llm-as-judge, asimov-laws, un-human-rights, prompt-augmentation



## Kapitel 35: Kapitel 25: DevOps & Infrastructure as Code

---

*"Infrastructure should be versioned, tested, and deployed like code. In ThemisDB, this is standard practice."*

---

### Überblick

Production-grade Datenbanken erfordern automatisierte Bereitstellung, Monitoring, und Failover-Mechanismen. Dieses Kapitel behandelt Infrastructure-as-Code (IaC) mit Terraform, Container-Orchestrierung mit Kubernetes, und GitOps-Workflows.

**Was Sie in diesem Kapitel lernen:** - Terraform-Module für ThemisDB-Deployment - Kubernetes Helm Charts - GitOps-Workflows mit ArgoCD - Multi-Region Failover - Disaster Recovery Planning - Operational Runbooks

```

flowchart LR
 subgraph "Source Control"
 Git[Git Repository]
 end

 subgraph "CI Pipeline"
 Build[Build & Test]
 TF[Terraform Plan]
 Security[Security Scan]
 end

 subgraph "CD Pipeline"
 Staging[Deploy Staging]
 SmokeTest[Smoke Tests]
 Prod[Deploy Production]
 end

 subgraph "Infrastructure"
 K8s[Kubernetes Cluster]
 DB[ThemisDB Cluster]
 Monitor[Monitoring]
 end

 Git --> Build
 Build --> TF
 TF --> Security
 Security --> Staging

 Staging --> K8s
 Staging --> DB

 Staging --> SmokeTest
 SmokeTest -->|Pass| Prod
 SmokeTest -->|Fail| Rollback[Rollback]

 Prod --> K8s
 Prod --> DB

 K8s --> Monitor
 DB --> Monitor

 Monitor --> Alert{Incident?}
 Alert -->|Yes| Rollback
 Alert -->|No| OK[Healthy]

 style Build fill:#4dabf7
 style Prod fill:#51cf66
 style Rollback fill:#ff6b6b
 style OK fill:#40c057

```

---

**Abb. 25.1:** DevOps-Pipeline-Architektur

---

## **25.1 Terraform Infrastructure-as-Code**

### **AWS RDS ThemisDB Cluster**

```

terraform {
 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 5.0"
 }
 }
}

backend "s3" {
 bucket = "themis-terraform-state"
 key = "production/themisdb/terraform.tfstate"
 region = "eu-central-1"
 encrypt = true
 dynamodb_table = "terraform-locks"
}

provider "aws" {
 region = var.aws_region
}

resource "aws_rds_cluster" "themis" {
 cluster_identifier = "themis-prod-cluster"
 engine = "themisdb"
 engine_version = "1.3.4"
 database_name = "themisdb"
 master_username = "admin"
 master_password = random_password.db_password.result

 # Hochverfügbarkeit
 availability_zones = data.aws_availability_zones.available.names
 backup_retention_period = 30
 preferred_backup_window = "03:00-04:00"

 # Security
 db_subnet_group_name = aws_db_subnet_group.themis.name
 db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.themis.name
 vpc_security_group_ids = [aws_security_group.themis_db.id]
 storage_encrypted = true
 kms_key_id = aws_kms_key.themis.arn

 # Performance Insights
 performance_insights_enabled = true
 performance_insights_kms_key_id = aws_kms_key.themis.arn

 # Logging
 enable_cloudwatch_logs_exports = ["error", "general", "slowquery"]

 tags = {

```

```

 Environment = "production"
 Service = "themisdb"
 }
}

resource "aws_rds_cluster_instance" "themis" {
 count = 3 # 3-Knoten Cluster
 cluster_identifier = aws_rds_cluster.themis.id
 instance_class = "db.r6i.2xlarge"
 engine = aws_rds_cluster.themis.engine
 engine_version = aws_rds_cluster.themis.engine_version

 monitoring_interval = 60
 monitoring_role_arn = aws_iam_role.rds_monitoring.arn
 performance_insights_enabled = true

 tags = {
 Name = "themis-instance-${count.index + 1}"
 }
}

resource "aws_rds_cluster_parameter_group" "themis" {
 name = "themis-prod-params"
 family = "themisdb1.3"
 description = "Tuned for high-throughput workloads"

 parameter {
 name = "max_connections"
 value = "500"
 }

 parameter {
 name = "shared_buffers_gb"
 value = "16"
 }

 parameter {
 name = "work_mem_mb"
 value = "256"
 }
}

```

## Subnet & Security Groups

```
resource "aws_db_subnet_group" "themis" {
 name = "themis-subnets"
 subnet_ids = aws_subnet.private[*].id

 tags = {
 Name = "themis-subnet-group"
 }
}

resource "aws_security_group" "themis_db" {
 name = "themis-db-sg"
 description = "Security group for ThemisDB"
 vpc_id = aws_vpc.main.id

 ingress {
 from_port = 8529
 to_port = 8529
 protocol = "tcp"
 security_groups = [aws_security_group.themis_app.id]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }

 tags = {
 Name = "themis-db-sg"
 }
}

resource "aws_backup_vault" "themis" {
 name = "themis-backup-vault"
 kms_key_arn = aws_kms_key.themis.arn

 tags = {
 Name = "themis-worm-vault"
 }
}
```

## Outputs & Monitoring

```
output "cluster_endpoint" {
 value = aws_rds_cluster.themis.endpoint
 description = "Cluster write endpoint"
}

output "cluster_reader_endpoint" {
 value = aws_rds_cluster.themis.reader_endpoint
 description = "Cluster read endpoint (load-balanced)"
}

output "cloudwatch_log_group" {
 value = "/aws/rds/cluster/${aws_rds_cluster.themis.id}"
}

resource "aws_cloudwatch_metric_alarm" "high_cpu" {
 alarm_name = "themis-high-cpu"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = "2"
 metric_name = "CPUUtilization"
 namespace = "AWS/RDS"
 period = "300"
 statistic = "Average"
 threshold = "80"
 alarm_actions = [aws_sns_topic.alerts.arn]

 dimensions = {
 DBClusterIdentifier = aws_rds_cluster.themis.cluster_identifier
 }
}
```



## 25.2 Kubernetes Helm Charts

### Helm Chart Struktur

```
themis-helm/
├─ Chart.yaml
├─ values.yaml
├─ values-prod.yaml
├─ values-staging.yaml
├─ templates/
│ ├─ deployment.yaml
│ ├─ service.yaml
│ ├─ ingress.yaml
│ ├─ pvc.yaml
│ ├─ configmap.yaml
│ └─ statefulset.yaml
└─ charts/
 └─ prometheus-operator/
```

---

## **Deployment Template**

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: {{ include "themis.fullname" . }}
 labels:
 {{- include "themis.labels" . | nindent 4 }}
spec:
 serviceName: {{ include "themis.fullname" . }}
 replicas: {{ .Values.replicaCount }}
 selector:
 matchLabels:
 {{- include "themis.selectorLabels" . | nindent 6 }}

 template:
 metadata:
 labels:
 {{- include "themis.selectorLabels" . | nindent 8 }}

 spec:
 # Pod Disruption Budget (für Rolling Updates)
 affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - themis
 topologyKey: kubernetes.io/hostname

 containers:
 - name: themis
 image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
 imagePullPolicy: IfNotPresent

 ports:
 - name: http
 containerPort: 8529
 protocol: TCP
 - name: replication
 containerPort: 8530
 protocol: TCP

 env:
 - name: THEMIS_CLUSTER_NAME
 value: {{ .Values.clusterName }}
 - name: THEMIS_PEER_URLS
 value: "themis-0.themis:8530,themis-1.themis:8530,themis-2.themis:8530"

```

```

Resource Requests
resources:
 requests:
 memory: {{ .Values.resources.requests.memory }}
 cpu: {{ .Values.resources.requests.cpu }}
 limits:
 memory: {{ .Values.resources.limits.memory }}
 cpu: {{ .Values.resources.limits.cpu }}

Liveness & Readiness Probes
livenessProbe:
 httpGet:
 path: /_admin/health
 port: http
 initialDelaySeconds: 30
 periodSeconds: 10

readinessProbe:
 httpGet:
 path: /_admin/ready
 port: http
 initialDelaySeconds: 10
 periodSeconds: 5

Volume Mounts
volumeMounts:
- name: data
 mountPath: /data
- name: config
 mountPath: /etc/themis

volumes:
- name: config
 configMap:
 name: {{ include "themis.fullname" . }}

Persistent Volume Claim
volumeClaimTemplates:
- metadata:
 name: data
 spec:
 accessModes: ["ReadWriteOnce"]
 storageClassName: fast-ssd
 resources:
 requests:
 storage: {{ .Values.persistence.size }}

```

---

## Helm Values (Production)

```
replicaCount: 3
clusterName: themis-prod

image:
 repository: themisdb/server
 tag: "1.3.4"
 pullPolicy: IfNotPresent

resources:
 requests:
 memory: 16Gi
 cpu: 4000m
 limits:
 memory: 32Gi
 cpu: 8000m

persistence:
 enabled: true
 size: 500Gi
 storageClass: fast-ssd

ingress:
 enabled: true
 className: nginx
 annotations:
 cert-manager.io/cluster-issuer: letsencrypt-prod
 hosts:
 - host: themis.example.com
 paths:
 - path: /
 pathType: Prefix
 tls:
 - secretName: themis-tls
 hosts:
 - themis.example.com

monitoring:
 enabled: true
 serviceMonitor:
 enabled: true
 interval: 30s
```

## 25.3 GitOps mit ArgoCD

### ArgoCD Application Definition

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
 name: themis-prod
 namespace: argocd
spec:
 project: production

 source:
 repoURL: https://github.com/themisdb/helm-charts
 targetRevision: v1.3.4
 path: themis-helm
 helm:
 releaseName: themis
 values: |
 replicaCount: 3
 image:
 tag: "1.3.4"

 destination:
 server: https://kubernetes.default.svc
 namespace: themis-prod

 syncPolicy:
 automated:
 prune: true
 selfHeal: true
 syncOptions:
 - CreateNamespace=true
 retry:
 limit: 5
 backoff:
 duration: 5s
 factor: 2
 maxDuration: 3m
```

## Pull Request Preview Environments

```
name: Preview Environment

on:
 pull_request:
 paths:
 - 'helm/**'
 - '.github/workflows/preview.yml'

jobs:
 deploy-preview:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v3

 - name: Create Preview Namespace
 run: |
 kubectl create namespace preview-${{ github.event.pull_request.number }}

 - name: Deploy with Helm
 run: |
 helm install themis-preview ./themis-helm \
 -n preview-${{ github.event.pull_request.number }} \
 -f values-preview.yaml \
 --set image.tag=${{ github.head_ref }}

 - name: Comment PR with Preview URL
 uses: actions/github-script@v6
 with:
 script: |
 github.rest.issues.createComment({
 issue_number: context.issue.number,
 owner: context.repo.owner,
 repo: context.repo.repo,
 body: '🚀 Preview deployed at `preview-${{ github.event.pull_request.number }}`
 })
```

## 25.4 Multi-Region Failover

### Active-Active Replikation

```
resource "aws_rds_cluster" "themis_eu" {
 # EU Cluster (Primary)
 cluster_identifier = "themis-eu-primary"
 region = "eu-central-1"
}

resource "aws_rds_cluster" "themis_us" {
 # US Cluster (Replica mit Failover)
 cluster_identifier = "themis-us-replica"
 region = "us-east-1"
 replication_source_arn = aws_rds_cluster.themis_eu.arn

 # Automatischer Failover nach 300 Sekunden
 backup_retention_period = 30
}

resource "aws_route53_failover_routing_policy" "themis" {
 name = "themis.example.com"
 zone_id = aws_route53_zone.main.zone_id
 type = "A"
 failover_routing_policy {
 type = "PRIMARY"
 }

 alias {
 name = aws_rds_cluster.themis_eu.endpoint
 zone_id = aws_rds_cluster.themis_eu.hosted_zone_id
 evaluate_target_health = true
 }

 health_check_id = aws_route53_health_check.eu.id
}
```



---

## Application-Level Failover

```
class ResilientThemisClient:
 def __init__(self, primary_url, secondary_url):
 self.primary_url = primary_url
 self.secondary_url = secondary_url
 self.current_url = primary_url
 self.failure_count = 0
 self.max_failures = 3

 def query(self, aql, params=None):
 try:
 client = themis.Client(self.current_url)
 result = client.query(aql, params)
 self.failure_count = 0 # Reset bei Erfolg
 return result
 except Exception as e:
 self.failure_count += 1

 if self.failure_count >= self.max_failures:
 # Failover zu Secondary
 self.current_url = self.secondary_url
 self.failure_count = 0
 print(f"Failover zu {self.secondary_url}")
 return self.query(aql, params) # Retry
 raise
```

## 25.5 Disaster Recovery Planning

### Backup & Restore Strategie

```
#!/bin/bash

aws backup start-backup-job \
 --backup-vault-name themis-backup-vault \
 --recovery-point-type INCREMENTAL \
 --recovery-point-tags "Frequency=Daily,Retention=7days"

aws backup start-backup-job \
 --backup-vault-name themis-backup-vault \
 --recovery-point-type FULL \
 --recovery-point-tags "Frequency=Weekly,Retention=4weeks"

aws backup start-backup-job \
 --backup-vault-name themis-backup-vault \
 --recovery-point-type FULL \
 --recovery-point-tags "Frequency=Monthly,Retention=7years"
```

### RTO & RPO Metriken

Szenario	RTO	RPO	Strategie
Einzelner Node Fehler	<30s	0s	Auto-Failover
Region Ausfalls	<5min	<1min	Cross-Region Replica
Datenbeschädigung	<1h	<1h	Point-in-Time Recovery
Vollständiger Datenverlust	<4h	<24h	S3 Langzeit-Archiv

## 25.6 Operational Runbooks

### Incident Response Playbook

```
Critical Alert: High Replication Lag

Severity: High
RTO: 5 minutes
On-call: Check #oncall Slack channel

Diagnostik (1-2 min)
1. `kubectl logs -f themis-2` - Prüfe Replica-Logs
2. `curl http://themis-2:8529/_admin/stats` - Replication lag?
3. `aws cloudwatch get-metric-statistics --metric-name ReplicationLatency`

Remediation (2-3 min)
```bash
kubectl delete pod themis-2

curl -X POST http://themis-2:8529/_admin/resync
```

Post-Incident

- ☐ Logs analysieren
- ☐ Runbook updaten
- ☐ Post-Mortem in Confluence ``

25.7 Checkliste für Production Readiness

- ☒ Terraform IAC für alle Infra-Komponenten
- ☒ Helm Charts mit Multi-Environment Support
- ☒ ArgoCD für GitOps Deployment
- ☒ Automated Backup & Restore Tests
- ☒ Multi-Region Failover konfiguriert
- ☒ CloudWatch Alarms für kritische Metriken
- ☒ PagerDuty/Opsgenie Integration
- ☒ Runbooks für alle kritischen Szenarien
- ☒ Load-Testing vor Production Release
- ☒ Security Scanning in CI/CD Pipeline

Kapitel 36: Kapitel 26: Migration & Legacy System Integration

"Legacy systems don't disappear. In enterprise environments, the art of seamless migration is what separates successful adoptions from failed projects."

Überblick

Migration von Legacy-Systemen (PostgreSQL, MongoDB, Neo4j) zu ThemisDB ist eine Kernaufgabe bei der Datenbank-Modernisierung. Dieses Kapitel zeigt pragmatische Strategien für zero-downtime Migrationen mit vollständiger Datenvalidierung.

Was Sie in diesem Kapitel lernen: - Schema-Mappings (SQL → AQL) - Daten-Extraktion & Transformation (ETL) - Live-Replikation während Migration - Daten-Validierung & Reconciliation - Rollback-Strategien - Performance Tuning nach Migration

```

flowchart LR
    subgraph Legacy Systems
        PG[(PostgreSQL)]
        MG[(MongoDB)]
        N4[(Neo4j)]
    end

    subgraph Migration Layer
        ETL[ETL Pipeline]
        VAL[Validation]
        SYNC[Live Sync]
    end

    subgraph ThemisDB
        COL[Collections]
        GRAPH[Graphs]
        EDGE[Edges]
    end

    subgraph Phases
        P1[Schema Mapping]
        P2[Bulk Import]
        P3[Delta Sync]
        P4[Cutover]
        P5[Validation]
    end

    PG --> ETL
    MG --> ETL
    N4 --> ETL

    ETL --> VAL
    VAL --> SYNC
    SYNC --> COL
    SYNC --> GRAPH
    SYNC --> EDGE

    P1 --> P2
    P2 --> P3
    P3 --> P4
    P4 --> P5

    P5 -.->|Fail| P3

```

```
sequenceDiagram
    participant Legacy
    participant Proxy
    participant ThemisDB

    Note over Legacy,ThemisDB: Phase 1: Dual Write
    Proxy->>Legacy: Write
    Proxy->>ThemisDB: Write (shadow)

    Note over Legacy,ThemisDB: Phase 2: Validation
    Proxy->>Legacy: Read
    Proxy->>ThemisDB: Read (compare)

    Note over Legacy,ThemisDB: Phase 3: Cutover
    Proxy->>ThemisDB: Read slash Write (primary)
    Proxy->>Legacy: Write (backup)

    Note over Legacy,ThemisDB: Phase 4: Decommission
    Proxy->>ThemisDB: Read slash Write (only)
```

Abb. 26.0: Migration-Strategie: Strangler Pattern

26.1 Schema-Mapping Strategien

PostgreSQL → ThemisDB Mapping

```
-- SQL Table                                → AQL Collection
-- customers TABLE                        → customers collection
-- id INT PRIMARY KEY                     → _id (implicit, or _key)
-- name VARCHAR(255)                     → name: string
-- email VARCHAR(100) UNIQUE              → email: string (with unique index)
-- created_at TIMESTAMP                   → created_at: date
-- FOREIGN KEY orders(customer_id)       → Document-Referenzen

-- Beispiel-Transformation:
FUNCTION transform_postgresql_customer(sql_row) {
  RETURN {
    _key: STRING(sql_row.id),
    name: sql_row.name,
    email: sql_row.email,
    created_at: DATE_ISO8601(sql_row.created_at),

    -- Denormalisierung erlaubt in Multi-Model DB
    order_count: 0, -- wird später aktualisiert
    total_spent: 0.0,

    -- Migration Metadata
    _migration: {
      source_system: "postgresql",
      source_id: sql_row.id,
      migrated_at: DATE_NOW(),
      validation_status: "pending"
    }
  }
}
```


MongoDB → ThemisDB Mapping

```
// MongoDB Document
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  title: "Product A",
  attributes: {
    color: "red",
    size: "M",
    warranty_years: 2
  },
  tags: ["electronics", "gadget"],
  reviews: [
    {user: "alice", rating: 5, comment: "Great!"},
    {user: "bob", rating: 4, comment: "Good"}
  ]
}

// Wird zu AQL Collection Dokument:
FOR doc IN mongo_products
  INSERT {
    _key: doc._id.toString(),
    title: doc.title,
    attributes: doc.attributes, // Geschachtelte Objekte erlaubt
    tags: doc.tags,             // Arrays
    reviews: doc.reviews,       // Array von Objekten

    -- Normalisierungsoptionen:
    -- Option 1: Flach (denormalisiert)
    color: doc.attributes.color,
    size: doc.attributes.size,

    -- Option 2: Graph-Edges für Many-To-Many
    -- reviews werden zu separater Collection + Edges
  } INTO products
```

Neo4j → ThemisDB Graph

```
-- Neo4j                                → ThemisDB
-- (:User {id: 1, name: "Alice"})        → User Document
-- [:FOLLOWS]                            → Graph Edge (Relation)
-- (:Product)                            → Product Document

FUNCTION migrate_neo4j_graph(cypher_result) {
  -- Neo4j Nodes → AQL Documents
  FOR node IN cypher_result.nodes
  INSERT {
    _key: node.id,
    type: node.label,
    properties: node.properties,

    -- Migration Tracking
    source_node_id: node.id,
    source_labels: node.labels
  } INTO graph_nodes

  -- Neo4j Relationships → AQL Graph Edges
  FOR rel IN cypher_result.relationships
  INSERT {
    _from: CONCAT(rel.from_type, "/", rel.from_id),
    _to: CONCAT(rel.to_type, "/", rel.to_id),
    relationship_type: rel.type,
    properties: rel.properties
  } INTO graph_edges

  RETURN {
    nodes_inserted: LENGTH(cypher_result.nodes),
    edges_inserted: LENGTH(cypher_result.relationships)
  }
}
```

26.2 ETL Pipeline Design

Python ETL Framework

```

import logging
from typing import List, Dict, Any
import themis

class ETLPipeline:
    def __init__(self, source_db, target_db, batch_size=1000):
        self.source = source_db
        self.target = target_db
        self.batch_size = batch_size
        self.logger = logging.getLogger(__name__)

    def extract(self, source_query: str) -> List[Dict]:
        """Extract: Daten aus Legacy-System lesen"""
        self.logger.info(f"Extracting from source: {source_query[:50]}...")
        return list(self.source.execute(source_query))

    def transform(self, rows: List[Dict], transformer_func) -> List[Dict]:
        """Transform: Schema-Mapping & Validierung"""
        self.logger.info(f"Transforming {len(rows)} rows...")
        transformed = []
        errors = []

        for i, row in enumerate(rows):
            try:
                transformed.append(transformer_func(row))
            except Exception as e:
                errors.append({'row_index': i, 'error': str(e), 'data': row})

        if errors:
            self.logger.warning(f"Transformation errors: {len(errors)}")

        return transformed, errors

    def load(self, docs: List[Dict], collection: str) -> Dict:
        """Load: In ThemisDB einfügen"""
        self.logger.info(f>Loading {len(docs)} documents into {collection}...")

        result = {
            'inserted': 0,
            'errors': 0,
            'error_details': []
        }

        # Batch-wise einfügen
        for i in range(0, len(docs), self.batch_size):
            batch = docs[i:i+self.batch_size]

            try:
                aql = f"""

```

```

        FOR doc IN @docs
            INSERT doc INTO {collection}
            RETURN NEW._id
    """
    ids = self.target.query(aql, {'docs': batch})
    result['inserted'] += len(ids)
except Exception as e:
    self.logger.error(f"Batch insert error: {e}")
    result['errors'] += len(batch)
    result['error_details'].append({
        'batch_start': i,
        'batch_size': len(batch),
        'error': str(e)
    })

return result

def run(self, source_query: str, transformer_func, target_collection: str) -> Dict:
    """Orchestriere E-T-L Pipeline"""
    try:
        rows = self.extract(source_query)
        transformed, errors = self.transform(rows, transformer_func)
        result = self.load(transformed, target_collection)

        result['transformation_errors'] = len(errors)
        result['total_processed'] = len(rows)
        result['success_rate'] = result['inserted'] / len(rows) * 100

        return result
    except Exception as e:
        self.logger.error(f"Pipeline failed: {e}")
        raise

from adapters.postgresql import PostgreSQLAdapter
from adapters.themis import ThemisAdapter

source_db = PostgreSQLAdapter(connection_string="postgresql://...")
target_db = ThemisAdapter(url="http://localhost:8529")

etl = ETLPipeline(source_db, target_db, batch_size=5000)

def transform_customer(row):
    return {
        '_key': str(row['id']),
        'name': row['name'],
        'email': row['email'],
        'created_at': row['created_at'].isoformat(),
        'migrated_at': datetime.now().isoformat()
    }

```

```
result = etl.run(  
    source_query="SELECT * FROM customers WHERE status = 'active'",  
    transformer_func=transform_customer,  
    target_collection="customers"  
)  
  
print(f"Migration: {result['inserted']} inserted, {result['errors']} errors, Success: {re
```

26.3 Live Replikation während Migration

CDC (Change Data Capture) Ansatz

```

import psycopg2
from psycopg2.extras import LogicalReplicationConnection
import themis

class PostgreSQLCDCSync:
    def __init__(self, pg_conn_str, themis_url):
        self.pg_conn = psycopg2.connect(pg_conn_str,
                                         connection_factory=LogicalReplicationConnection)
        self.themis = themis.Client(themis_url)

    def replicate_changes(self, slot_name="themis_slot"):
        """Lese Changes aus PostgreSQL WAL und appliziere sie"""

        cursor = self.pg_conn.cursor()
        cursor.start_replication(slot_name=slot_name)

        print(f"Replication started, listening for changes...")

        for message in cursor.consume_dstream():
            if message.payload:
                # Parse Change Event
                change = self._parse_wal_record(message.payload)

                # Apply in ThemisDB
                self._apply_change(change)

                # Acknowledge
                message.cursor.send_feedback(write_lsn=message.write_lsn)

    def _parse_wal_record(self, payload: bytes):
        """Parse PostgreSQL WAL Format"""
        # Vereinfacht: In Production wäre dies komplexer
        import json
        return json.loads(payload.decode('utf-8'))

    def _apply_change(self, change: Dict):
        """Appliziere INSERT/UPDATE/DELETE in ThemisDB"""

        if change['action'] == 'INSERT':
            aql = f"""
                INSERT @doc INTO {change['table']}
                RETURN NEW._id
            """
            self.themis.query(aql, {'doc': change['data']})

        elif change['action'] == 'UPDATE':
            aql = f"""
                UPDATE '{change['table']}'/{change['id']}' WITH @doc
                RETURN NEW
            """

```



```
        """
        self.themis.query(aql, {'doc': change['data']})

    elif change['action'] == 'DELETE':
        aql = f"""
            REMOVE '{change['table']}'/{change['id']}'
            RETURN OLD
        """
        self.themis.query(aql)

    print(f"Applied {change['action']} to {change['table']}/{change['id']}")

sync = PostgreSQLCDCSync(
    pg_conn_str="postgresql://user:password@localhost/mydb",
    themis_url="http://localhost:8529"
)

import threading
replication_thread = threading.Thread(target=sync.replicate_changes, daemon=True)
replication_thread.start()

print("CDC Replication running in background...")
```

26.4 Daten-Validierung & Reconciliation

Reconciliation Framework

```

class DataReconciliation:
    def __init__(self, source_db, target_db):
        self.source = source_db
        self.target = target_db

    def validate_counts(self, source_query: str, target_collection: str) -> Dict:
        """Vergleiche Zeilen-Anzahl Source vs Target"""

        source_count = self.source.execute(f"SELECT COUNT(*) FROM ({source_query})")[0]['count']

        target_count = self.target.query(f"""
            RETURN LENGTH(
                FOR doc IN {target_collection}
                RETURN doc
            )
        """)[0]

        match = source_count == target_count
        return {
            'collection': target_collection,
            'source_count': source_count,
            'target_count': target_count,
            'match': match,
            'difference': abs(source_count - target_count)
        }

    def validate_checksums(self, table: str, key_column: str) -> Dict:
        """Vergleiche Row-Level Checksums"""

        # Source Checksums
        source_checksums = self.source.execute(f"""
            SELECT {key_column}, MD5(ROW(*)) as checksum
            FROM {table}
            ORDER BY {key_column}
        """)

        # Target Checksums (via AQL)
        target_checksums = self.target.query(f"""
            FOR doc IN {table.lower()}
            SORT doc._key
            RETURN {{
                key: doc._key,
                checksum: MD5(JSON_STRINGIFY(doc))
            }}
        """)

        # Reconcile
        mismatches = []
        for src in source_checksums:

```

```

        tgt = next((t for t in target_checksums
                     if t['key'] == src[key_column]), None)

        if not tgt or tgt['checksum'] != src['checksum']:
            mismatches.append({
                'id': src[key_column],
                'source_checksum': src['checksum'],
                'target_checksum': tgt['checksum'] if tgt else None
            })

    return {
        'table': table,
        'total_rows': len(source_checksums),
        'mismatches': len(mismatches),
        'match_rate': (len(source_checksums) - len(mismatches)) / len(source_checksums),
        'mismatched_ids': mismatches[:10] # First 10
    }

def run_full_validation(self) -> Dict:
    """Komplette Post-Migration Validation"""

    validations = [
        self.validate_counts("SELECT * FROM customers", "customers"),
        self.validate_counts("SELECT * FROM orders", "orders"),
        self.validate_checksums("customers", "id"),
        self.validate_checksums("orders", "id")
    ]

    all_pass = all(v.get('match', v.get('match_rate', 0)) >= 99.9 for v in validations)

    return {
        'all_pass': all_pass,
        'validations': validations,
        'timestamp': datetime.now().isoformat()
    }

recon = DataReconciliation(source_db, target_db)
result = recon.run_full_validation()

if result['all_pass']:
    print("✅ Alle Validierungen erfolgreich!")
else:
    print("❌ Validierungsfehler gefunden:")
    for v in result['validations']:
        if not v.get('match'):
            print(f"    - {v}")

```

26.5 Rollback-Strategien

Blue-Green Deployment Pattern

```

---
apiVersion: v1
kind: Service
metadata:
  name: db-service
spec:
  selector:
    version: blue # Initial zeigt auf Blue (PostgreSQL)
  ports:
    - port: 5432

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: themis-green
spec:
  replicas: 3
  template:
    metadata:
      labels:
        version: green
    spec:
      containers:
        - name: themis
          image: themisdb/server:1.3.4
          ports:
            - containerPort: 8529

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: migration-config
data:
  # Phase 1: 5% Traffic zu Green
  traffic_green_percent: "5"

  # Phase 2: 25% Traffic nach X Stunden
  phase2_hours: "4"
  phase2_traffic: "25"

  # Phase 3: 50% Traffic nach X Stunden
  phase3_hours: "12"
  phase3_traffic: "50"

  # Phase 4: 100% Traffic (oder Rollback zu Blue)

```

```
phase4_hours: "24"
```

```
# Rollback bei Critical Errors
```

```
rollback_on_error_rate: "5" # 5% error rate triggers rollback
```

Schneller Rollback

```
#!/bin/bash
```

```
echo "🔄 Initiating immediate rollback to Blue (PostgreSQL)..."
```

```
kubectl patch service db-service -p '{"spec":{"selector":{"version":"blue"}}}'
```

```
echo "✓ Traffic router updated to Blue"
```

```
kubectl set env deployment/themis-green READ_ONLY=true
```

```
echo "✓ Green set to read-only mode"
```

```
until kubectl exec $(kubectl get pods -l version=blue -o name | head -1) \
```

```
-- pg_isready -h localhost; do
```

```
echo "Waiting for Blue to be ready..."
```

```
sleep 5
```

```
done
```

```
echo "✓ Blue is healthy"
```

```
curl -X POST https://hooks.slack.com/services/YOUR/WEBHOOK \
```

```
-d '{"text":"🔴 Migration rollback initiated. Reverting to PostgreSQL."}'
```

```
echo "✅ Rollback complete. System running on PostgreSQL (Blue)"
```

26.6 Performance Tuning nach Migration

Indexe erstellen

```
-- Häufig verwendete Filter
CREATE INDEX idx_customers_email
  ON customers (email)

CREATE INDEX idx_orders_customer_date
  ON orders (customer_id, created_at)

-- Geo-Queries optimieren
CREATE INDEX idx_locations_geo
  ON locations (location)
  TYPE GEO

-- Fulltext-Suche
CREATE INDEX idx_products_fulltext
  ON products (title, description)
  TYPE FULLTEXT
```

Query Performance Analysis

```
-- Analysiere langsame Queries
FOR slow IN slow_queries
  FILTER slow.duration_ms > 100
  COLLECT coll = slow.collection
  AGGREGATE count = COUNT(1), avg_duration = AVG(slow.duration_ms)
  SORT avg_duration DESC
  RETURN {
    collection: coll,
    slow_query_count: count,
    avg_duration_ms: avg_duration
  }

-- Empfehlungen generieren
-- Wenn avg_duration > 500ms: Index empfohlen
```

26.7 Migration Checklist

- ☒ Schema-Mapping definiert & dokumentiert
- ☒ ETL Pipeline entwickelt & getestet
- ☒ Daten-Validierungstests geschrieben

-  CDC/Live-Replikation konfiguriert
-  Blue-Green Deployment vorbereitet
-  Rollback-Prozedur dokumentiert
-  Performance-Baseline gemessen
-  Team-Training durchgeführt
-  Cutover-Fenster geplant
-  Post-Migration Support geplant

Typischer Timeline: - **T-4 Wochen:** Schema-Design & ETL-Entwicklung - **T-2 Wochen:** Full-Load Test & Performance-Tuning - **T-1 Woche:** CDC Replikation starten, Validierung - **T-0 Tag:** Cutover, Monitoring intensivieren - **T+24h:** Green vollständig Live, Blue Redundanz - **T+1 Woche:** Blue abschalten

Kapitel 37: Kapitel 27: Troubleshooting & Problem Resolution

"The difference between a good engineer and a great engineer is how quickly they can diagnose and fix production issues."

Überblick

Production-Probleme erfordern systematische Diagnose und schnelle Remediation. Dieses Kapitel bietet konkrete Lösungen für häufige ThemisDB-Probleme mit reproduzierbaren Debugging-Workflows.

Was Sie in diesem Kapitel lernen: - Systematische Problemanalyse (5-Why-Methode) - Performance-Probleme diagnostizieren - Replikations-Lag beheben - Out-of-Memory Errors lösen - Deadlock-Detection & Resolution - Korrupte Indizes reparieren - Network Partition Recovery

```

flowchart TD
    Problem[Performance Issue] --> Symptoms{Symptom<br/>Analysis}

    Symptoms -->|High Latency| LatencyCheck[Check Query Plan]
    Symptoms -->|High CPU| CPUCheck[Check Resource Usage]
    Symptoms -->|Memory Issues| MemCheck[Check Cache Hit Rate]

    LatencyCheck --> Index{Index<br/>Missing?}
    Index -->|Yes| AddIndex[Add Index]
    Index -->|No| QueryOpt[Optimize Query]

    CPUCheck --> Parallel{Parallel<br/>Queries?}
    Parallel -->|Yes| LimitConn[Limit Connections]
    Parallel -->|No| BadQuery[Identify Bad Query]

    MemCheck --> CacheSize{Cache<br/>Too Small?}
    CacheSize -->|Yes| IncCache[Increase Cache Size]
    CacheSize -->|No| CheckLeak[Check Memory Leak]

    style Problem fill:#ff6b6b
    style AddIndex fill:#43e97b
    style IncCache fill:#43e97b
    
```

Abb. 27.0: Troubleshooting-Decision-Tree

27.1 Systematische Problem-Diagnose

5-Why Root Cause Analysis

Problem: Query dauert 30 Sekunden statt 100ms

Why 1: Warum ist die Query langsam?

→ Full Collection Scan statt Index-Nutzung

Why 2: Warum wird kein Index genutzt?

→ Filter auf nicht-indexiertes Feld `metadata.custom\_field`

Why 3: Warum ist das Feld nicht indexiert?

→ Index wurde nach Schema-Änderung nicht aktualisiert

Why 4: Warum wurde Index nicht aktualisiert?

→ Deployment-Prozess hat Migrations-Step übersprungen

Why 5: Warum wurde Step übersprungen?

→ CI/CD Pipeline hatte keine Pre-Deploy Validation

Root Cause: Fehlende Pre-Deploy Index-Validation

Solution: Pre-Deploy Hook für Schema-Validation hinzufügen

flowchart TD

```

A[Problem erkannt] --> B{Logs vorhanden?}
B -->|Ja| C[Logs analysieren]
B -->|Nein| D[Monitoring prüfen]
C --> E{Fehlermuster?}
D --> E
E -->|Performance| F[EXPLAIN Query]
E -->|Crash| G[Core Dump analysieren]
E -->|Network| H[Latency/Packet Loss]
F --> I[Index fehlt?]
I -->|Ja| J[Index erstellen]
I -->|Nein| K[Query optimieren]
G --> L[Memory/Deadlock?]
H --> M[Load Balancer Check]
J --> N{Problem gelöst?}
K --> N
L --> N
M --> N
N -->|Nein| O[5-Why Root Cause]
N -->|Ja| P[Post-Mortem]
O --> P
    
```

Diagnostic Checklist

```
#!/bin/bash

echo "=== ThemisDB Health Check ==="

echo "1. Cluster Status:"
curl -s http://localhost:8529/_admin/cluster/health | jq .

echo "2. Memory Usage:"
curl -s http://localhost:8529/_admin/statistics | jq '.memory'

echo "3. Slow Queries:"
curl -s http://localhost:8529/_admin/slow-queries?threshold=1000

echo "4. Replication Lag:"
curl -s http://localhost:8529/_admin/replication/lag

echo "5. Active Connections:"
netstat -an | grep :8529 | wc -l

echo "6. Disk Space:"
df -h /data/themis

echo "7. Recent Errors:"
journalctl -u themis -n 50 --no-pager | grep ERROR
```

27.2 Performance-Probleme

Problem: Langsame Queries

Symptom: Query dauert >5 Sekunden

Diagnose:

```
-- Query mit EXPLAIN analysieren
EXPLAIN FOR u IN users
  FILTER u.email == 'alice@example.com'
  RETURN u

-- Ausgabe prüfen:
{
  "plan": {
    "nodes": [
      {"type": "SingletonNode"},
      {"type": "EnumerateCollectionNode", "collection": "users"}, // ✗ SCHLECHT: Full Scan
      {"type": "FilterNode"},
      {"type": "ReturnNode"}
    ]
  },
  "stats": {
    "executionTime": 5.234,
    "scannedFull": 1000000 // ✗ 1M Dokumente gescannt
  }
}
```

Solution:

```
-- Index erstellen
CREATE INDEX idx_users_email ON users (email)

-- Erneut EXPLAIN
EXPLAIN FOR u IN users
  FILTER u.email == 'alice@example.com'
  RETURN u

-- Jetzt mit Index:
{
  "plan": {
    "nodes": [
      {"type": "SingletonNode"},
      {"type": "IndexNode", "index": "idx_users_email"}, // ✓ GUT: Index genutzt
      {"type": "ReturnNode"}
    ]
  },
  "stats": {
    "executionTime": 0.023, // ✓ 200x schneller
    "scannedIndex": 1
  }
}
```

Problem: High CPU Usage

Symptom: CPU >90% dauerhaft

Diagnose:

```
curl -s http://localhost:8529/_admin/query-stats \
  | jq '.queries | sort_by(.cpu_time) | reverse | .[0:10]'
```

```
[
  {
    "query": "FOR doc IN large_collection RETURN doc",
    "cpu_time": 45.2,
    "count": 120,
    "avg_duration_ms": 8500
  }
]
```

Solution:

```
-- Option 1: Query optimieren (Projection)
FOR doc IN large_collection
  RETURN {id: doc._id, name: doc.name} -- Nur benötigte Felder

-- Option 2: Limit hinzufügen
FOR doc IN large_collection
  LIMIT 100
  RETURN doc

-- Option 3: Background-Job für Batch-Processing
FOR doc IN large_collection
  FILTER doc.needs_processing == true
  UPDATE doc WITH {needs_processing: false, processed_at: DATE_NOW()}
  OPTIONS {waitForSync: false} -- Async I/O
```

27.3 Replikations-Probleme

Problem: Replication Lag

Symptom: Replica ist 5 Minuten hinter Primary

Diagnose:

```
curl http://localhost:8529/_admin/replication/status

{
  "primary": "node-1",
  "replica": "node-2",
  "lag_seconds": 300,
  "last_applied_timestamp": "2025-12-31T22:55:00Z",
  "pending_operations": 15000
}
```

Root Causes & Solutions:

1. Netzwerk-Latenz:

```
ping node-2

terraform apply -var="replica_region=eu-central-1"
```

2. Replica überlastet:

```
ssh node-2 "top -b -n 1 | head -20"

kubectl scale statefulset themis-replica --replicas=0
kubectl set resources statefulset themis-replica \
  --limits=cpu=8,memory=32Gi
kubectl scale statefulset themis-replica --replicas=1
```

3. Zu viele Schreibvorgänge:

```
-- Schreibrate reduzieren mit Batching
LET batch = @documents -- Array von 1000 Docs
FOR doc IN batch
  INSERT doc INTO collection
  OPTIONS {waitForSync: false} -- Async für höheren Throughput
```

27.4 Memory-Probleme

Problem: Out of Memory (OOM)

Symptom: Server crashed mit OOM

Diagnose:

```
curl http://localhost:8529/_admin/statistics | jq '.memory'

{
  "resident_set_size_mb": 15800, # Actual RAM usage
  "virtual_size_mb": 18200,
  "heap_used_mb": 14500,
  "heap_limit_mb": 16000, # ✗ 90% ausgelastet!
  "buffer_cache_mb": 1200
}
```

Solutions:

1. Memory Limit erhöhen:

```
resources:
  limits:
    memory: 32Gi # Von 16Gi auf 32Gi
  requests:
    memory: 24Gi
```

2. Query Memory Leaks finden:

```
-- Große Result Sets vermeiden
FOR doc IN huge_collection
  LIMIT 1000000 -- ✗ 1M Docs in Memory!
  RETURN doc

-- Besser: Streaming mit Cursor
FOR doc IN huge_collection
  RETURN doc
OPTIONS {stream: true, batchSize: 1000}
```

3. Buffer Cache tunen:


```
buffer_cache_size_mb=512 # Von 2048 auf 512
```

27.5 Deadlock-Probleme

Problem: Deadlock Detected

Symptom: Transaction failed mit "Deadlock detected"

Diagnose:

```
-- Deadlock-Log abrufen
FOR dl IN deadlock_log
  SORT dl.timestamp DESC
  LIMIT 1
  RETURN dl

{
  "timestamp": "2025-12-31T23:00:00Z",
  "transactions": [
    {
      "tx_id": "tx-1234",
      "locked_collections": ["orders", "inventory"],
      "waiting_for": "inventory/item-5"
    },
    {
      "tx_id": "tx-5678",
      "locked_collections": ["inventory", "orders"], // ✗ Umgekehrte Reihenfolge!
      "waiting_for": "orders/order-10"
    }
  ]
}
```

Solution: Lock Ordering

```
-- ✗ FALSCH: Inkonsistente Lock-Reihenfolge
// Transaction 1
UPDATE "orders/o1" ...
UPDATE "inventory/i1" ...

// Transaction 2
UPDATE "inventory/i1" ... // Deadlock!
UPDATE "orders/o1" ...

-- ✔ RICHTIG: Konsistente Reihenfolge (alphabetisch)
// Beide Transactions
UPDATE "inventory/i1" ... // Immer zuerst inventory
UPDATE "orders/o1" ...    // Dann orders
```

Lock Timeout setzen:

```
FOR doc IN collection
  UPDATE doc WITH {...}
  OPTIONS {lockTimeout: 5000} -- 5s statt default 30s
```

27.6 Index-Probleme

Problem: Korrupter Index**Symptom:** Query returned wrong results**Diagnose:**

```
curl http://localhost:8529/_admin/index/check/users/idx_email

{
  "index": "idx_email",
  "status": "corrupted",
  "missing_entries": 42,
  "extra_entries": 5
}
```

Solution:

```
-- Index neu erstellen
DROP INDEX users/idx_email
CREATE INDEX idx_email ON users (email)

-- Verify
FOR u IN users
  FILTER u.email == 'test@example.com'
  RETURN u
```

Problem: Index zu groß

Symptom: Index verbraucht 50 GB

Diagnose:

```
curl http://localhost:8529/_admin/index/stats | jq '.indexes[] | select(.size_mb > 10000)'

{
  "name": "idx_fulltext_articles",
  "size_mb": 52000, # 52 GB!
  "document_count": 10000000
}
```

Solution:

```
-- Option 1: Sparse Index (nur nicht-NULL Werte)
CREATE INDEX idx_email_sparse ON users (email)
  OPTIONS {sparse: true}

-- Option 2: Partial Index (nur aktive User)
CREATE INDEX idx_active_users ON users (email)
  WHERE status == 'active'

-- Option 3: Archivierung alter Daten
FOR doc IN articles
  FILTER doc.created_at < DATE_SUBTRACT(DATE_NOW(), 2, 'years')
  REMOVE doc IN articles
  INSERT doc INTO articles_archive
```

27.7 Network Partition Recovery

Problem: Split-Brain nach Partition

Symptom: Zwei separate Cluster nach Netzwerk-Trennung

Diagnose:

```
curl http://node-1:8529/_admin/cluster/status
{"nodes": ["node-1", "node-2"], "quorum": true}

curl http://node-3:8529/_admin/cluster/status
{"nodes": ["node-3"], "quorum": false} # ❌ Lost quorum
```

Recovery:

```
curl -X POST http://node-3:8529/_admin/cluster/rejoin \
  -d '{"primary": "http://node-1:8529"}'

watch -n 5 'curl -s http://node-3:8529/_admin/replication/lag'

curl http://node-1:8529/_admin/cluster/status
{"nodes": ["node-1", "node-2", "node-3"], "quorum": true} # ✅
```

27.8 Backup & Restore Issues

Problem: Backup schlägt fehl

Symptom: Backup job timeout after 2 hours

Diagnose:

```
journalctl -u themis-backup -f

df -h /backup
```

Solutions:

```
find /backup -name "*.backup" -mtime +30 -delete

themis-backup --mode=incremental --since=2025-12-30

themis-backup --compression=zstd --compression-level=19
```

27.9 Common Error Messages

Error: "Collection not found"

```
ERROR: Collection 'user' not found
```

Solution:

```
-- Typo: 'user' statt 'users'
FOR u IN users --  Richtig
RETURN u
```

Error: "Index hint invalid"

```
ERROR: Index hint 'idx_name' does not exist
```

Solution:

```
-- Index existiert nicht mehr → neu erstellen oder Hint entfernen
CREATE INDEX idx_name ON users (name)

-- Oder Query ohne Hint:
FOR u IN users
  FILTER u.name == 'Alice'
RETURN u
```

Error: "Transaction timeout"

```
ERROR: Transaction timeout after 30000ms
```

Solution:

```
-- Transaction zu lang → in kleinere Batches aufteilen
LET batch_size = 1000
FOR doc IN large_update
  LIMIT @offset, batch_size
  UPDATE doc IN collection
```

27.10 Troubleshooting Playbook

Problem	Symptom	Erste Schritte	Eskalation
Slow Query	>5s Response	EXPLAIN, add Index	Query Rewrite
High CPU	>90% Usage	Query Stats, Optimize	Scale up
Replication Lag	>60s Lag	Network Check, Batch Size	Add Replica
OOM	Crash/Restart	Memory Stats, Increase Limit	Optimize Queries
Deadlock	Transaction Fails	Lock Ordering	Reduce Concurrency
Corrupt Index	Wrong Results	Rebuild Index	Restore Backup
Split-Brain	Quorum Lost	Network Repair, Rejoin	Manual Failover

Best Practices: - ☒ Immer EXPLAIN vor Production-Deployment - ☒ Monitoring-Alerts für Lag >30s, CPU >80%, Memory >85% - ☒ Regelmäßige Index-Maintenance (weekly VACUUM/ANALYZE) - ☒ Backup-Tests (monatlich Restore-Drill) - ☒ Runbooks für alle kritischen Fehler - ☒ Post-Mortems nach jedem Incident

Kapitel 38: Kapitel 28: AQL Vollständige Referenz

"Eine Abfragesprache ist nur so gut wie ihre Dokumentation."

Überblick

Dieses Kapitel ist die komplette Referenz für AQL (Adaptive Query Language), die einheitliche Abfragesprache von ThemisDB. Sie deckt alle 479 Sprachelemente ab: 119 Keywords und 360 eingebaute Funktionen.

Was Sie in diesem Kapitel finden: - Vollständiger Sprachumfang (v1.3.1) - Kategorisierte Funktionsreferenz - Praktische Beispiele für jeden Bereich - Performance-Hinweise und Best Practices

Voraussetzungen: Kapitel 5-9 (Datenmodelle), Grundverständnis von SQL.

```
graph LR
    AQL[AQL Query] --> Parser[Parser]
    Parser --> AST[Abstract Syntax Tree]
    AST --> Optimizer[Query Optimizer]

    Optimizer --> LogPlan[Logical Plan]
    LogPlan --> PhysPlan[Physical Plan]

    PhysPlan --> IndexSel[Index Selection]
    PhysPlan --> JoinOrd[Join Ordering]
    PhysPlan --> PushDown[Predicate Pushdown]

    IndexSel --> Execution[Execution Engine]
    JoinOrd --> Execution
    PushDown --> Execution

    Execution --> Result[Query Result]

    style Optimizer fill:#f093fb
    style Execution fill:#4facfe
```

Abb. 28.0: AQL-Query-Execution-Pipeline

28.1 Sprachumfang-Überblick

Executive Summary

Aspekt	v1.3.0	v1.3.1	Änderung
Reservierte Keywords	72	119	+47 (+65%)
Eingebaute Funktionen	~360	~360	0
Gesamt-Sprachumfang	432	479	+47 (+11%)

Wichtig: v1.3.1 erweitert Sprachkonstrukte (TYPE, FUNCTION, CLASS) - keine neuen Funktionen.

28.2 Reservierte Keywords (119)

28.2.1 Core Language (8)

FOR, IN, LET, FILTER, COLLECT, SORT, LIMIT, RETURN

Beispiel:

```
FOR user IN users
  FILTER user.age >= 18
  LET email_domain = SPLIT(user.email, "@")[1]
  COLLECT domain = email_domain WITH COUNT INTO count
  SORT count DESC
  LIMIT 10
  RETURN { domain, count }
```

28.2.2 Logical Operators (3)

AND, OR, NOT

Beispiel:


```

FOR product IN products
  FILTER product.price >= 10 AND product.price <= 100
  FILTER NOT product.discontinued
  FILTER product.category == "Electronics" OR product.category == "Books"
  RETURN product

```

28.2.3 Aggregation Functions (8)

COUNT, SUM, AVG, MIN, MAX, VARIANCE, STDDEV, UNIQUE

Beispiel:

```

FOR order IN orders
  COLLECT year = DATE_YEAR(order.created_at)
  AGGREGATE
    total = SUM(order.amount),
    avg_order = AVG(order.amount),
    min_order = MIN(order.amount),
    max_order = MAX(order.amount),
    order_count = COUNT(1)
  RETURN { year, total, avg_order, min_order, max_order, order_count }

```

28.2.4 Graph Traversal (4)

OUTBOUND, INBOUND, ANY, SHORTEST_PATH

Beispiel:

```

-- Finde alle Freunde von Alice (2 Hops)
FOR v, e, p IN 1..2 OUTBOUND "users/alice" friends
  RETURN { friend: v.name, path_length: LENGTH(p.edges) }

-- Kürzester Pfad von Alice zu Bob
FOR v, e IN OUTBOUND SHORTEST_PATH "users/alice" TO "users/bob" follows
  RETURN { user: v.name, action: e.type }

```

28.2.5 DDL (Data Definition Language) (5)

CREATE, DROP, COLLECTION, INDEX, VIEW

Beispiele:

```
-- Collection erstellen
CREATE COLLECTION products

-- Index erstellen
CREATE INDEX products_price ON products(price) TYPE SKIPLIST

-- View erstellen
CREATE VIEW active_users AS
  FOR u IN users
    FILTER u.status == "active"
  RETURN u

-- Löschen
DROP INDEX products_price
DROP COLLECTION products
```

28.2.6 DML (Data Manipulation Language) (6)

INSERT, UPDATE, REPLACE, REMOVE, UPSERT, INTO, WITH

Beispiele:

```
-- Insert
INSERT { name: "Alice", age: 30 } INTO users

-- Update
UPDATE { _key: "alice" } WITH { age: 31 } IN users

-- Upsert (Insert if not exists)
UPSERT { email: "alice@example.com" }
  INSERT { name: "Alice", email: "alice@example.com", age: 30 }
  UPDATE { last_login: DATE_NOW() }
  IN users

-- Remove
REMOVE { _key: "alice" } IN users

-- Replace
REPLACE { _key: "alice", name: "Alice Smith", age: 31 } IN users
```

28.2.7 LLM Operations (13)

```
LLM, INFER, RAG, EMBED, MODEL, LORA, STATS, CACHE,
LOAD, UNLOAD, LIST, INGEST, BLOB
```

Beispiele:

```
-- RAG-Query
FOR doc IN documents
  LET embedding = EMBED(doc.content, { model: "text-embedding-3-small" })
  LET neighbors = KNN_SEARCH(embeddings, embedding, 5)
  LET context = CONCAT_SEPARATOR("\n", neighbors[*].content)
  LET answer = LLM("Answer based on context", { context, model: "gpt-4" })
  RETURN answer

-- Ingest Content
INGEST BLOB "data/documents.pdf"
  INTO content_store
  WITH { chunk_size: 512, overlap: 50 }
```

28.2.8 Index Types (7)

```
HASH, SKIPLIST, FULLTEXT, GEO, PERSISTENT, TTL, VECTOR
```

Beispiele:

```
-- Hash Index (Equality)
CREATE INDEX users_email ON users(email) TYPE HASH

-- Skiplist (Range Queries)
CREATE INDEX products_price ON products(price) TYPE SKIPLIST

-- Fulltext Index
CREATE INDEX articles_body ON articles(body) TYPE FULLTEXT

-- Geo Index
CREATE INDEX locations_coords ON locations(lat, lng) TYPE GEO

-- TTL Index (Auto-Expiry)
CREATE INDEX sessions_ttl ON sessions(expires_at) TYPE TTL

-- Vector Index (ANN Search)
CREATE INDEX embeddings_vector ON embeddings(vector)
  TYPE VECTOR
  OPTIONS { metric: "cosine", dimensions: 768 }
```

28.3 Date/Time Functions (45)

28.3.1 Aktuelle Zeit/Datum (11)

Wichtigste Funktionen: - `DATE_NOW()` - Aktueller ISO-Timestamp - `CURRENT_DATE()` / `CURRENT_TIME()` - Nur Datum/Zeit - `UNIX_TIMESTAMP()` - Unix Epoch

```
RETURN {
  now: DATE_NOW(),
  unix: UNIX_TIMESTAMP()
}
-- → {"now": "2025-01-15T10:30:45Z", "unix": 1736936445}
```

28.3.2 Interval Functions (8)

Interval-Arithmetik: `DAYS(n)`, `HOURS(n)`, `WEEKS(n)` etc. für Zeitberechnungen.

```
-- Letzte 30 Tage
FILTER order.created_at >= DATE_NOW() - DAYS(30)

-- Fälligkeitsdatum berechnen
LET due = invoice.created_at + DAYS(14)
```

28.3.3 Komponenten-Extraktion (11)

Komponenten extrahieren: `DATE_YEAR()` , `DATE_MONTH()` , `DATE_DAY()` , `DATE_HOUR()` , etc.

```
-- Gruppierung nach Monat
COLLECT month = DATE_MONTH(sale.date)
AGGREGATE total = SUM(sale.amount)
```

28.3.4 Workday Functions (6)

<code>WORKDAYS(start, end, calendar)</code>	-- Arbeitstage zwischen zwei Daten
<code>WORKDAYS_ADD(date, days, calendar)</code>	-- Addiere Arbeitstage
<code>IS_WEEKEND(date)</code>	-- true wenn Samstag/Sonntag
<code>IS_WORKDAY(date, calendar)</code>	-- true wenn Arbeitstag
<code>HOLIDAYS(country, year)</code>	-- Liste der Feiertage
<code>HOLIDAYS_BETWEEN(start, end, country)</code>	-- Feiertage in Zeitraum

Beispiele:

```
-- Berechne SLA-Frist (5 Arbeitstage)
FOR ticket IN support_tickets
  RETURN {
    ticket_id: ticket._key,
    created: ticket.created_at,
    sla_deadline: WORKDAYS_ADD(ticket.created_at, 5, "DE")
  }

-- Finde Tickets, die am Wochenende erstellt wurden
FOR ticket IN support_tickets
  FILTER IS_WEEKEND(ticket.created_at)
  RETURN ticket
```

28.4 String Functions (20)

CONCAT(str1, str2, ...)	-- Strings zusammenfügen
CONCAT_SEPARATOR(sep, str1, ...)	-- Mit Trennzeichen
SUBSTRING(str, start, length)	-- Teilstring
UPPER(str)	-- Großbuchstaben
LOWER(str)	-- Kleinbuchstaben
TRIM(str)	-- Whitespace entfernen
LTRIM(str)	-- Links trimmen
RTRIM(str)	-- Rechts trimmen
LENGTH(str)	-- Länge
REVERSE(str)	-- Umkehren
REPLACE(str, search, replace)	-- Ersetzen
SPLIT(str, separator)	-- In Array aufteilen
JOIN(array, separator)	-- Array zu String
STARTS_WITH(str, prefix)	-- Startet mit
ENDS_WITH(str, suffix)	-- Endet mit
CONTAINS(str, substring)	-- Enthält
FIND(str, substring)	-- Position finden
REGEX_TEST(str, pattern)	-- Regex-Match?
REGEX_MATCHES(str, pattern)	-- Alle Matches
REGEX_REPLACE(str, pattern, repl)	-- Regex-Ersetzung

Beispiele:

```
-- Email-Domain extrahieren
FOR user IN users
  LET domain = SPLIT(user.email, "@")[1]
  RETURN { user: user.name, domain }

-- Case-insensitive Suche
FOR article IN articles
  FILTER CONTAINS(LOWER(article.title), LOWER("ThemisDB"))
  RETURN article

-- Email-Validierung mit Regex
FOR user IN users
  FILTER REGEX_TEST(user.email, "^[^@]+@[^@]+\.[^@]+$")
  RETURN user
```

28.5 Math Functions (30)

```
-- Basic Math
ABS(x)          -- Betrag
CEIL(x)         -- Aufrunden
FLOOR(x)        -- Abrunden
ROUND(x, d)     -- Runden auf d Dezimalstellen
SQRT(x)         -- Quadratwurzel
POW(x, y)       -- x hoch y
EXP(x)          -- e^x
LOG(x)          -- Natürlicher Logarithmus
LOG10(x)        -- Logarithmus zur Basis 10
LN(x)           -- Alias für LOG

-- Trigonometric
SIN(x)          -- Sinus
COS(x)          -- Kosinus
TAN(x)          -- Tangens
ASIN(x)         -- Arkussinus
ACOS(x)         -- Arkuskosinus
ATAN(x)         -- Arkustangens
ATAN2(y, x)     -- Arkustangens mit Vorzeichen
SINH(x)         -- Hyperbelsinus
COSH(x)         -- Hyperbelkosinus
TANH(x)         -- Hyperbeltangens

-- Utilities
MIN(a, b, ...)  -- Minimum
MAX(a, b, ...)  -- Maximum
RAND()          -- Zufallszahl [0, 1)
RAND_INT(a, b)  -- Ganzzahl zwischen a und b
PI()            --  $\pi$ 
E()             -- e
SIGN(x)         -- Vorzeichen (-1, 0, 1)
TRUNC(x)        -- Ganzzahlteil
MOD(x, y)       -- Modulo
QUOTIENT(x, y)  -- Ganzzahlige Division
```

Beispiele:

```

-- Berechne Entfernung mit Pythagoras
FOR point IN points
  LET distance = SQRT(POW(point.x, 2) + POW(point.y, 2))
  RETURN { point: point._key, distance }

-- Zufälliges Sampling (10%)
FOR doc IN documents
  FILTER RAND() < 0.1
  RETURN doc

-- Preisberechnung mit Rundung
FOR product IN products
  LET price_with_tax = ROUND(product.price * 1.19, 2)
  RETURN { product: product.name, price_with_tax }

```

28.6 Array Functions (20)

PUSH(array, value)	-- Element anhängen
POP(array)	-- Letztes Element entfernen
SHIFT(array)	-- Erstes Element entfernen
UNSHIFT(array, value)	-- Element vorne einfügen
APPEND(array, value)	-- Alias für PUSH
PREPEND(array, value)	-- Alias für UNSHIFT
FIRST(array)	-- Erstes Element
LAST(array)	-- Letztes Element
NTH(array, n)	-- n-tes Element
SLICE(array, start, end)	-- Teilarray
FLATTEN(array, depth)	-- Array flach machen
UNIQUE(array)	-- Duplikate entfernen
UNION(array1, array2)	-- Vereinigung
INTERSECTION(array1, array2)	-- Schnittmenge
DIFFERENCE(array1, array2)	-- Differenz
MAP(array, fn)	-- Transform
FILTER(array, fn)	-- Filtern
REDUCE(array, fn, init)	-- Reduzieren
SORT_ARRAY(array)	-- Sortieren
REVERSE_ARRAY(array)	-- Umkehren

Beispiele:


```
-- Finde gemeinsame Tags
FOR doc1 IN documents
  FOR doc2 IN documents
    FILTER doc1._key < doc2._key
    LET common_tags = INTERSECTION(doc1.tags, doc2.tags)
    FILTER LENGTH(common_tags) > 0
    RETURN { doc1: doc1._key, doc2: doc2._key, common_tags }

-- Extrahiere alle einzigartigen Kategorien
RETURN UNIQUE(FLATTEN(
  FOR product IN products
    RETURN product.categories
))
```

28.7 Geo Functions (25)

28.7.1 GeoJSON Konstruktion (6)

```
GEO_POINT(lng, lat)
GEO_MULTIPPOINT(points)
GEO_LINESTRING(coords)
GEO_MULTILINESTRING(lines)
GEO_POLYGON(rings)
GEO_MULTIPOLYGON(polygons)
```

Beispiel:

```
-- Erstelle Restaurant mit Location
INSERT {
  name: "Bella Italia",
  location: GEO_POINT(13.405, 52.520) -- Berlin
} INTO restaurants
```

28.7.2 Spatial Predicates (6)

```
GEO_CONTAINS(geo1, geo2)    -- geo1 enthält geo2
GEO_INTERSECTS(geo1, geo2)  -- geo1 schneidet geo2
ST_Contains(geo1, geo2)     -- Alias
ST_Within(geo2, geo1)       -- geo2 innerhalb geo1
ST_Intersects(geo1, geo2)   -- Alias
GEO_EQUALS(geo1, geo2)      -- Geometrisch gleich
```

Beispiel:

```
-- Finde Restaurants in Polygon (Stadtteil)
LET polygon = GEO_POLYGON([
  [13.40, 52.52], [13.41, 52.52],
  [13.41, 52.51], [13.40, 52.51],
  [13.40, 52.52]
])

FOR restaurant IN restaurants
  FILTER GEO_CONTAINS(polygon, restaurant.location)
  RETURN restaurant
```

28.7.3 Distance & Metrics (5)

```
GEO_DISTANCE(geo1, geo2)    -- Haversine-Distanz (Meter)
ST_Distance(geo1, geo2)     -- Alias
GEO_IN_RANGE(geo, center, radius) -- Innerhalb Radius
GEO_AREA(polygon)           -- Fläche (m²)
ST_Area(polygon)            -- Alias
ST_Length(linestring)       -- Länge (Meter)
```

Beispiel:

```
-- Finde Restaurants innerhalb 1 km
LET my_location = GEO_POINT(13.405, 52.520)

FOR restaurant IN restaurants
  LET distance = GEO_DISTANCE(my_location, restaurant.location)
  FILTER distance <= 1000
  SORT distance ASC
  RETURN { name: restaurant.name, distance }
```

28.7.4 CRS Transformations (10)

```
ST_TRANSFORM(geo, from_crs, to_crs)
ST_SRID(geo) -- SRID auslesen
ST_SetSRID(geo, srid) -- SRID setzen
WGS84_TO_UTM(lng, lat) -- WGS84 → UTM
UTM_TO_WGS84(easting, northing, zone)
ETRS89_TO_WGS84(x, y)
WGS84_TO_ETRS89(lng, lat)
HELMERT_TRANSFORM(x, y, params) -- 7-Parameter-Transformation
GAUSS_KRUEGER_TO_UTM(x, y)
UTM_TO_GAUSS_KRUEGER(easting, northing)
```

Beispiel:

```
-- Transformiere GPS-Koordinaten nach UTM (für genaue Distanzmessung)
FOR location IN survey_points
  LET utm = WGS84_TO_UTM(location.lng, location.lat)
  RETURN { location: location._key, utm }
```

28.8 Vector Functions (20)

```
-- Similarity Metrics
COSINE_SIMILARITY(v1, v2)
DOT_PRODUCT(v1, v2)
EUCLIDEAN_DISTANCE(v1, v2)
MANHATTAN_DISTANCE(v1, v2)
HAMMING_DISTANCE(v1, v2)
JACCARD_SIMILARITY(v1, v2)

-- Vector Operations
VECTOR_NORM(v)           -- L2-Norm
VECTOR_NORMALIZE(v)      -- Auf Länge 1 normalisieren
VECTOR_ADD(v1, v2)
VECTOR_SUBTRACT(v1, v2)
VECTOR_MULTIPLY(v, scalar)
VECTOR_DIVIDE(v, scalar)
VECTOR_DOT(v1, v2)       -- Alias für DOT_PRODUCT
VECTOR_CROSS(v1, v2)     -- Kreuzprodukt (nur 3D)

-- Distance Metrics
L1_DISTANCE(v1, v2)      -- Manhattan
L2_DISTANCE(v1, v2)      -- Euclidean
CHEBYSHEV_DISTANCE(v1, v2) -- Max-Norm

-- Search
KNN_SEARCH(collection, query_vector, k)
VECTOR_QUANTIZE(v, bits)  -- Quantisierung (SQ8, SQ4)
```

Beispiele:

```
-- Semantic Search mit Embeddings
LET query_embedding = EMBED("machine learning tutorial")

FOR doc IN embeddings
  LET similarity = COSINE_SIMILARITY(query_embedding, doc.vector)
  SORT similarity DESC
  LIMIT 10
  RETURN { doc: doc.title, similarity }

-- KNN-Search mit Prefilter
FOR result IN KNN_SEARCH(
  embeddings,
  EMBED("neural networks"),
  20,
  { filter: "category == 'AI'" }
)
  RETURN result
```

28.9 Graph Functions (15)

```
-- Traversal
OUTBOUND(vertex, edge_collection)
INBOUND(vertex, edge_collection)
ANY(vertex, edge_collection)
SHORTEST_PATH(start, end, edge_collection)

-- Path Metrics
PATH_LENGTH(path)
PATH_VERTICES(path)
PATH_EDGES(path)

-- Algorithms
CONNECTED_COMPONENTS(graph)
PAGERANK(graph)
BETWEENNESS_CENTRALITY(graph, vertex)
CLOSENESS_CENTRALITY(graph, vertex)
COMMUNITY_DETECTION(graph)

-- Utilities
IS_CONNECTED(graph, v1, v2)
GRAPH_DIAMETER(graph)
GRAPH_RADIUS(graph)
```

Beispiele:

```

-- PageRank auf Social Graph
LET pageranks = PAGERANK({
  vertexCollections: ["users"],
  edgeCollections: ["follows"]
})

FOR user IN users
  SORT pageranks[user._key] DESC
  LIMIT 10
  RETURN { user: user.name, score: pageranks[user._key] }

-- Finde Communities
LET communities = COMMUNITY_DETECTION({
  vertexCollections: ["users"],
  edgeCollections: ["friends"]
})

FOR user IN users
  RETURN { user: user.name, community: communities[user._key] }

```

28.10 Best Practices

28.10.1 Index Usage

```

-- ❌ SCHLECHT: Keine Index-Nutzung
FOR user IN users
  FILTER UPPER(user.email) == "ALICE@EXAMPLE.COM"
  RETURN user

-- ✅ GUT: Index auf email (case-insensitive)
FOR user IN users
  FILTER user.email == "alice@example.com"
  RETURN user

```

28.10.2 Early Filtering

```
-- ❌ SCHLECHT: Filter nach JOIN
FOR order IN orders
  FOR user IN users
    FILTER user._key == order.user_id
    FILTER order.amount > 100
    RETURN { order, user }

-- ✅ GUT: Filter vor JOIN
FOR order IN orders
  FILTER order.amount > 100
  FOR user IN users
    FILTER user._key == order.user_id
    RETURN { order, user }
```

28.10.3 Projection

```
-- ❌ SCHLECHT: Vollständige Dokumente
FOR user IN users
  RETURN user

-- ✅ GUT: Nur benötigte Felder
FOR user IN users
  RETURN { name: user.name, email: user.email }
```

28.11 Zusammenfassung

AQL bietet **479 Sprachelemente**: - **119 Keywords** für Struktur und Semantik - **360 Funktionen** für Datenmanipulation

Hauptkategorien

1. **Date/Time (45)**: Umfassende Zeitzoneunterstützung, Arbeitstage
2. **Strings (20)**: Regex, Splitting, Manipulation
3. **Math (30)**: Trigonometrie, Statistik
4. **Arrays (20)**: Funktionale Programmierung
5. **Geo (35)**: GeoJSON, CRS-Transformationen
6. **Vectors (20)**: ANN-Search, Similarity
7. **Graph (15)**: Traversal, Centrality, Communities

Nächste Schritte

- **Kapitel 5-9:** Praktische Beispiele für jedes Datenmodell
- **Kapitel 21:** Query Optimization
- **AQL Grammar:** [aql/AQL_GRAMMAR_EXTENDED_v1.3.1.ebnf](#)

28.12 Advanced Patterns & Idioms

28.12.1 Recursive Queries with Custom Depth

```
-- Find all ancestors up to depth N
LET find_ancestors = FUNCTION(person_id, max_depth) {
  RETURN (
    FOR v, e, p IN 0..max_depth OUTBOUND person_id GRAPH 'family'
    FILTER CURRENT.generation < person_id.generation
    RETURN {
      person: v.name,
      generation: v.generation,
      depth: LENGTH(p.edges)
    }
  )
}

RETURN find_ancestors('persons/alice', 3)
```

28.12.2 Transitive Closure (Find All Reachable Nodes)

```
-- Get all projects reachable from a given person (including transitive deps)
FOR p IN projects
  FILTER p._id == 'projects/prj1'
  LET all_deps = (
    FOR dep IN 0..10 OUTBOUND p GRAPH 'depends_on'
    RETURN dep._id
  ) | UNIQUE()
  RETURN {
    project: p.name,
    dependencies: all_deps,
    dep_count: LENGTH(all_deps)
  }
```


28.12.3 Sliding Window (Time-Based)

```
-- Calculate moving average with time window (7 days)
FOR current_date IN DATE_RANGE(@start_date, @end_date, '1d')
  LET window_start = DATE_SUBTRACT(current_date, 7, 'd')
  LET daily_data = (
    FOR event IN events
      FILTER event.date >= window_start AND event.date <= current_date
      RETURN event.value
  )
  RETURN {
    date: current_date,
    moving_avg: AVG(daily_data),
    moving_sum: SUM(daily_data),
    moving_count: COUNT(daily_data)
  }
```

28.12.4 Lateral Join (Explode & Flatten)

```
-- Unnest nested arrays
FOR order IN orders
  LET items = order.items -- Array of {product_id, quantity}
  FOR item IN items
    LET product = DOCUMENT('products/' + item.product_id)
    RETURN {
      order_id: order._id,
      product_name: product.name,
      quantity: item.quantity,
      item_total: item.quantity * product.price
    }
```

28.12.5 Pivot (Transform Rows to Columns)

```
-- Convert months to columns
FOR record IN sales
  COLLECT year = YEAR(record.date)
  AGGREGATE
    jan = SUM(record.amount FILTER MONTH(record.date) == 1),
    feb = SUM(record.amount FILTER MONTH(record.date) == 2),
    mar = SUM(record.amount FILTER MONTH(record.date) == 3),
    q1_total = SUM(record.amount FILTER MONTH(record.date) IN [1,2,3])
  RETURN { year, jan, feb, mar, q1_total }
```

28.13 Window Functions (Comprehensive)

28.13.1 Row Numbering

```
-- Rank students by score within each class
FOR s IN students
  SORT s.class, s.score DESC
  LET rank_in_class = (
    SELECT COUNT(*) FROM students s2
    WHERE s2.class == s.class AND s2.score >= s.score
  )
  RETURN {
    name: s.name,
    class: s.class,
    score: s.score,
    rank: rank_in_class
  }
```

28.13.2 Cumulative (Running) Totals

```
-- Sales running total
FOR order IN SORT(orders, 'created_at')
  COLLECT idx = @row_number
  AGGREGATE
    order_value = order.amount,
    cumulative = SUM(orders[* FILTER @row_number <= idx].amount)
  RETURN {
    order_id: order._id,
    order_value,
    cumulative,
    running_total: cumulative
  }
```

28.13.3 First/Last of Group

```
-- Get first and last transaction per account
FOR t IN transactions
  SORT t.account_id, t.timestamp
  COLLECT account = t.account_id INTO group = t
  RETURN {
    account,
    first_transaction: group[0],
    last_transaction: group[LENGTH(group)-1],
    total_transactions: LENGTH(group)
  }
```

28.13.4 Lead/Lag (Next/Previous Row)

```
-- Compare consecutive values
FOR metric IN SORT(metrics, 'timestamp')
  LET current_idx = @row_number
  LET prev = (
    FOR m IN metrics
      FILTER m.timestamp < metric.timestamp
      SORT m.timestamp DESC
      LIMIT 1
      RETURN m
  )[0]
  RETURN {
    timestamp: metric.timestamp,
    value: metric.value,
    previous_value: prev?.value || NULL,
    change: metric.value - (prev?.value || metric.value)
  }
```

28.14 Type System & Type Checking

28.14.1 Type Checking Functions

```
-- Type predicates (return true/false)
IS_ARRAY(value)           -- Array type
IS_BOOL(value)            -- Boolean
IS_NUMBER(value)          -- Number (int or float)
IS_INTEGER(value)         -- Integer specifically
IS_STRING(value)          -- String
IS_NULL(value)            -- Null/undefined
IS_OBJECT(value)          -- Object/Document
IS_LIST(value)            -- Alias for IS_ARRAY
IS_DEFINED(value)         -- Has definition (not null)

-- Examples
FOR doc IN collection
  FILTER IS_STRING(doc.email) && IS_NUMBER(doc.age)
  RETURN doc
```

28.14.2 Type Conversion

```
-- Convert between types
TO_STRING(value)          -- Convert to string
TO_NUMBER(value)          -- Convert to number
TO_BOOL(value)            -- Convert to boolean
TO_ARRAY(value)           -- Convert to array
TO_ARRAY(list)            -- Ensure is array
TO_LIST(value)            -- Alias for TO_ARRAY

-- Examples
FOR doc IN collection
  RETURN {
    id: doc._id,
    price_str: TO_STRING(doc.price),
    age: TO_NUMBER(doc.age_str),
    is_active: TO_BOOL(doc.active_flag)
  }
```

28.14.3 Type Coercion & Safe Operations

```
-- Safe type operations
COALESCE(a, b, c)      -- Return first non-null
FIRST_ITEM(array)      -- Get first item safely
LAST_ITEM(array)       -- Get last item safely
NTH_ITEM(array, n)     -- Get nth item safely

-- Examples
FOR doc IN collection
  RETURN {
    id: doc._id,
    phone: COALESCE(doc.phone, doc.mobile, "N/A"),
    first_tag: FIRST_ITEM(doc.tags),
    primary_contact: NTH_ITEM(doc.contacts, 0)
  }
```

28.15 Error Handling & Validation

28.15.1 TRY-CATCH Patterns

```
-- Basic error handling
BEGIN
  TRY
    LET result = 100 / 0 -- Will error
    RETURN result
  CATCH err
    RETURN { error: err.message, code: err.code }
COMMIT
```

28.15.2 Conditional Errors (THROW)

```
-- Throw custom errors
FOR user IN users
  FILTER user.balance < 0
  THROW "INVARIANT_VIOLATED: Negative balance for user " + user._id
  RETURN user
```

28.15.3 Schema Validation

```
-- Validate document structure
FOR doc IN users
  LET valid = (
    IS_STRING(doc.name) &&
    IS_STRING(doc.email) &&
    IS_OBJECT(doc.address) &&
    IS_ARRAY(doc.tags) &&
    CONTAINS(doc.email, "@")
  )
  FILTER valid
  RETURN doc
```

28.16 Performance Optimization Deep Dive

28.16.1 Index-Aware Query Writing

```
-- Indexes for: email (unique), status, created_at

-- ❌ Index not used (UPPER)
FOR u IN users
  FILTER UPPER(u.email) == "ALICE@EXAMPLE.COM"
  RETURN u

-- ✅ Index used (direct match)
FOR u IN users
  FILTER u.email == "alice@example.com"
  RETURN u

-- ✅ Composite index: (status, created_at)
FOR u IN users
  FILTER u.status == 'active'
  FILTER u.created_at >= '2025-01-01'
  RETURN u

-- ⚠️ Index partial (filter before join)
FOR u IN users
  FILTER u.status == 'active'
  FOR o IN orders FILTER o.user_id == u._id
  RETURN { u, o }
```

28.16.2 Aggregation Optimization

```
-- ❌ Slow: Iterate then aggregate
LET users_active = (
  FOR u IN users
    FILTER u.status == 'active'
    RETURN u
)
RETURN { count: LENGTH(users_active) }

-- ✅ Fast: Aggregate in query
FOR u IN users
  FILTER u.status == 'active'
  COLLECT WITH COUNT INTO count
RETURN { count }

-- ✅ Fastest: With aggregation function
FOR u IN users
  FILTER u.status == 'active'
  AGGREGATE count = COUNT(1)
RETURN { count }
```

28.16.3 Subquery Optimization

```
-- ❌ Subquery in FILTER (repeats for each row)
FOR order IN orders
  FILTER order.user_id IN (
    SELECT _id FROM users WHERE status == 'premium'
  )
  RETURN order

-- ✅ Join directly (index on user_id)
FOR user IN users
  FILTER user.status == 'premium'
  FOR order IN orders
    FILTER order.user_id == user._id
  RETURN order
```

28.17 Advanced Collection Operations

28.17.1 Bulk Operations with LIMIT & Offset

```
-- Batch processing
FOR page IN 0..@max_pages
  LET batch_start = page * @page_size
  FOR user IN users
    SORT user._key
    LIMIT batch_start, @page_size
    RETURN {
      batch_number: page,
      user
    }
```

28.17.2 Bulk Update with Conditions

```
-- Update multiple documents
FOR user IN users
  FILTER user.created_at < DATE_SUBTRACT(DATE_NOW(), 1, 'y')
  UPDATE user WITH {
    status: 'inactive',
    last_updated: DATE_NOW(),
    archive_flag: true
  } IN users
RETURN OLD -- Return before-update document
```


28.17.3 Safe Delete with Verification

```
-- Delete with rollback on condition
BEGIN
  LET to_delete = (
    FOR d IN documents
      FILTER d.archived == true
      FILTER d.created_at < DATE_SUBTRACT(DATE_NOW(), 2, 'y')
    RETURN d
  )

  IF LENGTH(to_delete) > 0 THEN
    FOR doc IN to_delete
      REMOVE doc IN documents
    COMMIT
  ELSE
    ABORT "No documents to delete"
  ENDIF
RETURN { deleted: LENGTH(to_delete) }
```

28.18 Zusammenfassung & Schnellreferenz

Core Constructs (8)

- **Iteration:** FOR...IN
- **Condition:** FILTER
- **Binding:** LET
- **Grouping:** COLLECT
- **Ordering:** SORT
- **Limiting:** LIMIT
- **Output:** RETURN
- **Transactions:** BEGIN...COMMIT

Top 20 Functions (By Usage)

1. **COUNT** - Aggregation
2. **FILTER** - Predicate logic
3. **LENGTH** - Array/string size
4. **SUM** - Numeric aggregation
5. **CONCAT** - String joining
6. **CONTAINS** - Substring search

-
7. **SPLIT** - String splitting
 8. **SORT** - Ordering
 9. **LIMIT** - Result limiting
 10. **DATE_NOW** - Current timestamp
 11. **UPPER/LOWER** - Case conversion
 12. **ROUND** - Numeric rounding
 13. **AVG** - Mean aggregation
 14. **MIN/MAX** - Range extremes
 15. **REGEX** - Pattern matching
 16. **DISTANCE** - Geospatial
 17. **SUBSTRING** - String slicing
 18. **TO_STRING** - Type conversion
 19. **FLATTEN** - Array flattening
 20. **UNIQUE** - Deduplication

Decision Table: Which Construct?

Goal	Use	Example
Iterate docs	FOR...IN	FOR u IN users RETURN u
Filter rows	FILTER	FILTER u.status == 'active'
Create variable	LET	LET age = DATE_DIFF(u.birth, NOW())
Group rows	COLLECT	COLLECT status = u.status
Order results	SORT	SORT u.created_at DESC
Limit results	LIMIT	LIMIT 100
Return data	RETURN	RETURN u.name
Atomic ops	BEGIN...COMMIT	Transactions

Performance Checklist

- [] Indexes on FILTER fields
- [] Indexes on SORT fields
- [] Early filtering (before JOINS)
- [] Projections (only needed fields)

-
- [] COLLECT for aggregations
 - [] Avoid expressions in FILTER
 - [] Use bind parameters
 - [] Batch operations with LIMIT

Kapitel 28 von 30 | Teil IV: Referenz | ~8.500 Wörter (+2000 neu)

Kapitel 39: Kapitel 29: Analytics & Process Mining

"Daten ohne Analyse sind wie ein Buch ohne Leser - voller Potential, aber ungenutzt."

Überblick

ThemisDB bietet umfassende Analytics-Funktionen, von klassischen OLAP-Cubes bis hin zu fortgeschrittenem Process Mining für Verwaltungsvorgänge. Dieses Kapitel zeigt die komplette Palette.

Was Sie in diesem Kapitel lernen: - OLAP Cubes und Multidimensionale Analysen - Process Mining mit administrativen Standardmodellen - Conformance Checking gegen Ideal-Prozesse - Pattern Recognition und Anomalieerkennung - Real-Time Analytics mit Changefeed - Performance-Optimierungen für Analytics-Workloads

Voraussetzungen: Kapitel 2 (Architektur), Kapitel 28 (AQL Referenz).

29.1 OLAP Fundamentals

29.1.1 Was ist OLAP?

OLAP (Online Analytical Processing) ermöglicht multidimensionale Datenanalyse mit: - **Dimensions:** Zeit, Produkt, Region, Kunde - **Measures:** Umsatz, Menge, Gewinn - **Operations:** Slice, Dice, Drill-Down, Roll-Up, Pivot

29.1.2 OLAP Cube Architektur

```

flowchart TB
    A[Fact Table: Sales] --> B[Dimension: Time]
    A --> C[Dimension: Product]
    A --> D[Dimension: Region]
    A --> E[Dimension: Customer]

    B --> B1[Year]
    B1 --> B2[Quarter]
    B2 --> B3[Month]
    B3 --> B4[Day]

    C --> C1[Category]
    C1 --> C2[Subcategory]
    C2 --> C3[Product]

    D --> D1[Country]
    D1 --> D2[State]
    D2 --> D3[City]

    E --> E1[Segment]
    E1 --> E2[Customer]

    style A fill:#e1f5ff
    style B fill:#fff4e1
    style C fill:#e1ffe1
    style D fill:#ffe1f5
    style E fill:#f5e1ff
  
```

Abb. 29.1: Process-Mining-Pipeline

29.1.3 OLAP Operations in AQL

Slice (Eine Dimension fixieren)

```

-- Slice: Nur Verkäufe in 2024
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  COLLECT
    product = sale.product_name,
    region = sale.region
  AGGREGATE total = SUM(sale.amount)
  RETURN { product, region, total }
  
```

Dice (Mehrere Dimensionen einschränken)

```
-- Dice: 2024, Electronics, Nur Europa
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  FILTER sale.category == "Electronics"
  FILTER sale.region IN ["Germany", "France", "UK"]
  COLLECT
    country = sale.region,
    month = DATE_MONTH(sale.date)
  AGGREGATE total = SUM(sale.amount)
  SORT country, month
  RETURN { country, month, total }
```

Drill-Down (Von Jahr zu Monat)

```
-- Drill-Down: Von Jahr zu Monaten zu Tagen
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  FILTER DATE_MONTH(sale.date) == 3 -- März
  COLLECT
    day = DATE_DAY(sale.date)
  AGGREGATE
    total = SUM(sale.amount),
    count = COUNT(1)
  SORT day
  RETURN { day, total, count }
```

Roll-Up (Aggregation auf höhere Ebene)

```
-- Roll-Up: Von Monat zu Quarter
FOR sale IN sales
  COLLECT
    year = DATE_YEAR(sale.date),
    quarter = DATE_QUARTER(sale.date)
  AGGREGATE total = SUM(sale.amount)
  SORT year, quarter
  RETURN { year, quarter, total }
```

Pivot (Zeilen/Spalten tauschen)

```
-- Pivot: Produkte als Zeilen, Regionen als Spalten
FOR sale IN sales
  COLLECT
    product = sale.product_name
  AGGREGATE
    total_germany = SUM(sale.region == "Germany" ? sale.amount : 0),
    total_france = SUM(sale.region == "France" ? sale.amount : 0),
    total_uk = SUM(sale.region == "UK" ? sale.amount : 0)
  RETURN {
    product,
    Germany: total_germany,
    France: total_france,
    UK: total_uk,
    Total: total_germany + total_france + total_uk
  }
```

29.2 Process Mining Fundamentals

29.2.1 Was ist Process Mining?

Process Mining analysiert Event-Logs, um reale Prozesse zu:

1. **Discover:** Prozessmodelle aus Logs ableiten
2. **Check:** Conformance gegen Soll-Prozesse prüfen
3. **Enhance:** Prozesse mit Performance-Daten anreichern

29.2.2 Event Log Structure

```
-- Standard Event Log Format
{
  "case_id": "V-2024-0123",      -- Vorgangs-ID
  "activity": "Antragstellung",  -- Aktivitätsname
  "timestamp": "2024-10-15T09:30:00Z",
  "resource": "Sabine Müller",   -- Bearbeiter
  "cost": 150.00,                -- Kosten
  "additional_data": {
    "department": "Bauamt",
    "priority": "normal"
  }
}
```

29.2.3 Process Mining Pipeline

```

flowchart LR
    A[Event Log] --> B[Process Discovery]
    B --> C[Process Model]
    C --> D[Conformance Checking]
    D --> E[Deviation Report]

    C --> F[Performance Analysis]
    F --> G[Bottleneck Detection]

    C --> H[Enhancement]
    H --> I[Optimized Process]

    style A fill:#e1f5ff
    style C fill:#fff4e1
    style E fill:#ffe1e1
    style I fill:#e1ffe1
  
```

Abb. 29.2: Event-Log-Processing

29.3 Administrative Standard Models

ThemisDB beinhaltet vordefinierte Prozessmodelle für deutsche Verwaltungen:

29.3.1 Verfügbare Modelle

Modell-ID	Name	Aktivitäten	Anwendung
bauantrag_standard	Bauantrag (Standard)	4	Baugenehmigungen
bauantrag_erweitert	Bauantrag (Erweitert)	7	Komplexe Bauprojekte
fuehrerschein_standard	Führerscheinantrag	5	Führerscheine
reisepass_standard	Reisepass-Antrag	4	Ausweisdokumente
gewerbe_anmeldung	Gewerbeanmeldung	6	Gewerberegister
kfz_zulassung	KFZ-Zulassung	5	Fahrzeugzulassungen

29.3.2 Modell laden

```
-- Lade Bauantrags-Standardmodell
LET model = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

RETURN model
```

Output:

```
{
  "id": "bauantrag_standard",
  "name": "Bauantrag (Standard)",
  "version": "1.0",
  "activities": [
    "antragstellung",
    "vollstaendigkeitspruefung",
    "fachliche_pruefung",
    "genehmigung"
  ],
  "edges": [
    {"from": "antragstellung", "to": "vollstaendigkeitspruefung"},
    {"from": "vollstaendigkeitspruefung", "to": "fachliche_pruefung"},
    {"from": "fachliche_pruefung", "to": "genehmigung"}
  ],
  "expected_duration_days": 45,
  "sla_days": 60
}
```

29.4 Similarity Search

29.4.1 Hybrid Similarity Metrics

ThemisDB kombiniert drei Similarity-Arten:

1. **Graph Similarity** (Strukturell): Edit Distance zwischen Prozessgraphen
2. **Vector Similarity** (Semantisch): Embeddings der Aktivitätsnamen
3. **Behavioral Similarity** (Ausführung): Trace-Varianz, Durchlaufzeiten

29.4.2 Similar Process Search

```
-- Finde ähnliche Bauanträge
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

LET similar = PM_FIND_SIMILAR(ideal, {
  method: "hybrid",
  threshold: 0.75,          -- Min 75% Ähnlichkeit
  limit: 50,
  graph_weight: 0.4,        -- 40% Struktur
  vector_weight: 0.3,       -- 30% Semantik
  behavioral_weight: 0.3     -- 30% Verhalten
})

FOR result IN similar
  SORT result.overall_similarity DESC
  RETURN {
    case_id: result.case_id,
    similarity: result.overall_similarity,
    metrics: {
      graph: result.metrics.graph_similarity,
      vector: result.metrics.vector_similarity,
      behavioral: result.metrics.behavioral_similarity
    },
    matched_activities: result.matched_activities,
    extra_activities: result.extra_activities,
    missing_activities: result.missing_activities
  }
```

Output:

```
[
  {
    "case_id": "V-2024-0123",
    "similarity": 0.92,
    "metrics": {
      "graph": 0.95,
      "vector": 0.88,
      "behavioral": 0.93
    },
    "matched_activities": [
      "Antragstellung",
      "Vollständigkeitsprüfung",
      "Fachliche Prüfung",
      "Genehmigung"
    ],
    "extra_activities": [],
    "missing_activities": []
  },
  {
    "case_id": "V-2024-0456",
    "similarity": 0.85,
    "metrics": {
      "graph": 0.82,
      "vector": 0.91,
      "behavioral": 0.82
    },
    "matched_activities": [
      "Antrag einreichen",
      "Vollständigkeitskontrolle",
      "Technische Prüfung",
      "Freigabe"
    ],
    "extra_activities": ["Nachforderung Unterlagen"],
    "missing_activities": []
  }
]
```

29.4.3 Similarity Visualization

```

flowchart LR
    A[Ideal Process] --> B[Graph Edit Distance]
    A --> C[Activity Embeddings]
    A --> D[Execution Traces]

    E[Real Process] --> B
    E --> C
    E --> D

    B --> F[Graph Similarity 0.95]
    C --> G[Vector Similarity 0.88]
    D --> H[Behavioral Similarity 0.93]

    F --> I[Weighted Combination]
    G --> I
    H --> I

    I --> J[Overall Similarity 0.92]

    style A fill:#e1f5ff
    style E fill:#fff4e1
    style J fill:#e1ffe1
  
```

Abb. 29.3: Process-Discovery-Algorithm

29.5 Conformance Checking

29.5.1 Fitness & Precision

Fitness: Wie viele Schritte des Ist-Prozesses passen zum Soll-Prozess? **Precision:** Wie genau folgt der Ist-Prozess dem Soll-Prozess (keine Extra-Schritte)?

29.5.2 Conformance Check Query

```
-- Prüfe alle Bauanträge gegen Standard
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

FOR case IN bauantraege
  LET comparison = PM_COMPARE_IDEAL(case.vorgang_id, ideal)

  -- Nur Fälle mit Fitness < 90%
  FILTER comparison.fitness < 0.9

  SORT comparison.fitness ASC

  RETURN {
    vorgang_id: case.vorgang_id,
    antragsteller: case.antragsteller,
    eingangsdatum: case.eingangsdatum,

    -- Conformance Metrics
    fitness: comparison.fitness,
    precision: comparison.precision,
    steps_checked: comparison.steps_checked,
    steps_conformant: comparison.steps_conformant,

    -- Abweichungen
    deviations: comparison.deviations,

    -- Handlungsempfehlung
    action: (
      comparison.fitness < 0.7 ? "🔴 Dringend prüfen" :
      comparison.fitness < 0.9 ? "🟡 Kleinere Anpassungen" :
      "✅ OK"
    )
  }
}
```

Output:

```
[
  {
    "vorgang_id": "V-2024-0789",
    "antragsteller": "Müller GmbH",
    "eingangsdatum": "2024-10-15",
    "fitness": 0.65,
    "precision": 0.82,
    "steps_checked": 6,
    "steps_conformant": 4,
    "deviations": [
      "Activity 'Vollständigkeitsprüfung' was skipped",
      "Activity 'Genehmigung' occurred before 'Fachliche Prüfung'"
    ],
    "action": "🔴 Dringend prüfen"
  }
]
```

29.5.3 Conformance Heatmap

```
flowchart TB
    subgraph Ideal Process
        I1[Antragstellung] --> I2[Vollständigkeitsprüfung]
        I2 --> I3[Fachliche Prüfung]
        I3 --> I4[Genehmigung]
    end

    subgraph Real Process V-2024-0789
        R1[Antragstellung] --> R3[Fachliche Prüfung]
        R3 --> R4[Genehmigung]
    end

    I1 -.Match.-> R1
    I2 -.Missing!.-> X[[ERROR]]
    I3 -.Match.-> R3
    I4 -.Match.-> R4

    style I2 fill:#ffe1e1
    style X fill:#ff0000
    style R1 fill:#e1ffe1
    style R3 fill:#e1ffe1
    style R4 fill:#fff4e1
```

Abb. 29.4: Conformance-Checking-Flow

29.6 Pattern Recognition

29.6.1 Problematische Patterns

```
-- Pattern: Genehmigung vor Prüfung (Compliance-Problem!)
LET problematic_pattern = {
  activities: ["Genehmigung", "Fachliche Prüfung"],
  edges: [
    {from: "Genehmigung", to: "Fachliche Prüfung"} -- Falsche Reihenfolge!
  ]
}

FOR case IN bauantraege
  LET has_problem = PM_HAS_PATTERN(
    case.vorgang_id,
    problematic_pattern,
    0.8 -- 80% Schwellwert
  )

  FILTER has_problem == true

  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

  RETURN {
    vorgang_id: case.vorgang_id,
    alert: "⚠ Genehmigung vor Prüfung!",
    activities: trace.activities,
    timestamps: trace.timestamps,
    requires_investigation: true
  }
```

29.6.2 Loop Detection

```
-- Finde Prozesse mit Schleifen (z.B. wiederholte Nachforderungen)
FOR case IN bauantraege
  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)
  LET loops = PM_DETECT_LOOPS(trace)

  FILTER LENGTH(loops) > 0

  RETURN {
    vorgang_id: case.vorgang_id,
    loop_count: LENGTH(loops),
    loops: loops,
    total_duration: PM_DURATION(case.vorgang_id)
  }
```

Output:

```
[
  {
    "vorgang_id": "V-2024-1234",
    "loop_count": 2,
    "loops": [
      {
        "activities": ["Vollständigkeitsprüfung", "Nachforderung", "Vollständigkeitsprüfung"],
        "iterations": 3,
        "total_duration_days": 18
      }
    ],
    "total_duration": 67
  }
]
```


29.7 Performance Analysis

29.7.1 Bottleneck Detection

```
-- Finde langsamste Aktivitäten
FOR case IN bauentraege
  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

  FOR activity IN trace.activities
    LET duration = activity.end_time - activity.start_time

  COLLECT
    activity_name = activity.name
  AGGREGATE
    avg_duration = AVG(duration),
    min_duration = MIN(duration),
    max_duration = MAX(duration),
    count = COUNT(1)

  SORT avg_duration DESC
  LIMIT 10

  RETURN {
    activity: activity_name,
    avg_duration_hours: avg_duration / 3600,
    min_duration_hours: min_duration / 3600,
    max_duration_hours: max_duration / 3600,
    occurrences: count
  }
```

Output:

```
[
  {
    "activity": "Fachliche Prüfung",
    "avg_duration_hours": 168.5,
    "min_duration_hours": 24.0,
    "max_duration_hours": 720.0,
    "occurrences": 1523
  },
  {
    "activity": "Nachforderung Unterlagen",
    "avg_duration_hours": 96.3,
    "min_duration_hours": 12.0,
    "max_duration_hours": 480.0,
    "occurrences": 427
  }
]
```

29.7.2 SLA Violations

```
-- SLA-Überschreitungen
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")
LET sla_days = ideal.sla_days -- 60 Tage

FOR case IN bauantraege
  LET duration = PM_DURATION(case.vorgang_id)

  FILTER duration > sla_days

  SORT duration DESC

  RETURN {
    vorgang_id: case.vorgang_id,
    eingangsdatum: case.eingangsdatum,
    duration_days: duration,
    sla_days: sla_days,
    overrun_days: duration - sla_days,
    overrun_percent: ((duration - sla_days) / sla_days) * 100
  }
```

29.8 Real-Time Analytics with Changefeed

29.8.1 Live Dashboard Query

```
-- Real-Time Dashboard: Offene Vorgänge nach Status
FOR case IN bauantraege
  FILTER case.status != "abgeschlossen"

  LET current_activity = PM_CURRENT_ACTIVITY(case.vorgang_id)
  LET elapsed_days = DATE_DIFF(case.eingangsdatum, DATE_NOW(), "day")

  COLLECT
    status = current_activity
  AGGREGATE
    count = COUNT(1),
    avg_elapsed = AVG(elapsed_days)

  SORT count DESC

  RETURN {
    status,
    count,
    avg_elapsed_days: ROUND(avg_elapsed, 1)
  }
```

29.8.2 Changefeed für Analytics

```
feed = db.changefeed("bauantraege", {"filter": "status == 'genehmigt'"})
for change in feed:
  # Analytics-Event speichern
  db.aql("INSERT {event: 'approval', id: @id} INTO events",
        {"id": change['new']['vorgang_id']})
```

29.9 Performance Optimizations

29.9.1 Materialized Views

```
-- Erstelle Materialized View für häufige Aggregation
CREATE VIEW bauantraege_stats AS
  FOR case IN bauantraege
    LET duration = PM_DURATION(case.vorgang_id)
    LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

    COLLECT
      year = DATE_YEAR(case.eingangsdatum),
      month = DATE_MONTH(case.eingangsdatum)
    AGGREGATE
      total_cases = COUNT(1),
      avg_duration = AVG(duration),
      sla_violations = SUM(duration > 60 ? 1 : 0)

    RETURN {
      year,
      month,
      total_cases,
      avg_duration,
      sla_violations
    }

-- Schneller Zugriff
FOR stat IN bauantraege_stats
  RETURN stat
```

29.9.2 Pre-Aggregation

```
-- Pre-Aggregation für OLAP Cubes
INSERT {
  _key: "2024-Q1-Electronics-Germany",
  year: 2024,
  quarter: 1,
  category: "Electronics",
  region: "Germany",
  total_sales: 1245000.50,
  unit_count: 3421,
  last_updated: DATE_NOW()
} INTO sales_cube_cache
```

29.10 Zusammenfassung

Kernfeatures

1. **OLAP:** Multidimensionale Analysen (Slice, Dice, Drill-Down, Pivot)
2. **Process Mining:** Discovery, Conformance, Enhancement
3. **Admin Models:** Vordefinierte deutsche Verwaltungsprozesse
4. **Similarity:** Hybrid (Graph + Vector + Behavioral)
5. **Conformance:** Fitness & Precision Metrics
6. **Patterns:** Anomalieerkennung, Loop Detection
7. **Performance:** Bottleneck-Analyse, SLA-Tracking
8. **Real-Time:** Changefeed-Integration

Best Practices

- **Index:** TTL-Indizes für Event Logs (automatische Bereinigung)
 - **Views:** Materialized Views für häufige Analytics-Queries
 - **Cache:** Pre-Aggregation für OLAP Cubes
 - **Sampling:** Bei großen Datenmengen Sampling nutzen (`FILTER RAND() < 0.1`)
-

29.8 Advanced Analytics: Multi-Dimensional Analysis

29.8.1 Dimensional Modeling (Star Schema)

```
-- Fact table: order_facts (contains measures and FK to dimensions)
FOR fact IN order_facts
  LET customer = DOCUMENT(fact.customer_dim_id)
  LET product = DOCUMENT(fact.product_dim_id)
  LET time = DOCUMENT(fact.time_dim_id)
  COLLECT
    region = customer.region,
    category = product.category,
    month = time.month
  AGGREGATE
    revenue = SUM(fact.amount),
    units_sold = SUM(fact.quantity),
    transactions = COUNT(1),
    avg_order_value = AVG(fact.amount)
  SORT revenue DESC
  RETURN {
    region,
    category,
    month,
    revenue,
    units_sold,
    transactions,
    avg_order_value,
    units_per_transaction: units_sold / transactions
  }
```

29.8.2 Slice & Dice Operations

```
-- Drill down: Regional → Store → Item Level
LET filters = @filters -- {region: 'EMEA', store_id: 'S123'}

FOR fact IN order_facts
  LET customer = DOCUMENT(fact.customer_dim_id)
  LET store = DOCUMENT(fact.store_dim_id)
  LET product = DOCUMENT(fact.product_dim_id)

  FILTER customer.region == filters.region
  FILTER store._id == filters.store_id

  COLLECT item_id = product._id, item_name = product.name
  AGGREGATE
    units = SUM(fact.quantity),
    revenue = SUM(fact.amount),
    margin = SUM(fact.margin),
    transactions = COUNT(1)
  SORT revenue DESC
  RETURN {
    item_name,
    units,
    revenue,
    margin,
    margin_pct: ROUND(margin / revenue * 100, 2),
    margin_per_unit: ROUND(margin / units, 2)
  }
```

29.8.3 Cohort Analysis (Behavioral Segments)

```
-- Analyze cohorts of users by first purchase month
FOR user IN users
  LET first_purchase = (
    FOR order IN orders
      FILTER order.customer_id == user._id
      SORT order.created_at ASC
      LIMIT 1
      RETURN order.created_at
  )[0]

  LET cohort_month = DATE_FORMAT(first_purchase, '%yyyy-%mm')
  LET subsequent_purchases = (
    FOR o IN orders
      FILTER o.customer_id == user._id
      FILTER o.created_at > first_purchase
      RETURN o
  )

  COLLECT cohort = cohort_month
  AGGREGATE
    users_in_cohort = COUNT(1),
    avg_ltv = AVG(subsequent_purchases[*].amount | SUM()),
    retention_m1 = SUM(LENGTH(subsequent_purchases) > 0),
    retention_m2 = SUM(LENGTH(
      FOR o IN subsequent_purchases
        FILTER DATE_DIFF(first_purchase, o.created_at, 'm') >= 2
        RETURN 1
    ) > 0)
  RETURN {
    cohort,
    users: users_in_cohort,
    avg_ltv: ROUND(avg_ltv, 2),
    retention_m1_pct: ROUND(retention_m1 / users_in_cohort * 100, 1),
    retention_m2_pct: ROUND(retention_m2 / users_in_cohort * 100, 1)
  }
```


29.9 Real-Time Analytics with Changefeeds

29.9.1 Streaming Aggregation Pattern

```
-- Real-time sales dashboard (with changefeed)
-- This would be called continuously as events arrive
FOR event IN changefeed('orders')
  LET order = event.doc
  LET time_bucket = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd %hh:00')

  COLLECT
    time = time_bucket,
    status = order.status
  AGGREGATE
    orders = COUNT(1),
    revenue = SUM(order.amount),
    avg_order = AVG(order.amount)

  RETURN {
    timestamp: time,
    status,
    metric: {
      orders,
      revenue: ROUND(revenue, 2),
      avg_order: ROUND(avg_order, 2)
    }
  }
```

29.9.2 Late-Arriving Facts Handling

```
-- Handle out-of-order or delayed events
FOR event IN changefeed('orders')
  LET is_late = DATE_DIFF(DATE_NOW(), event.doc.created_at, 's') > 3600  -- > 1 hour old

  IF is_late THEN
    -- Route to late-arriving fact handler
    INSERT {
      order_id: event.doc._id,
      received_at: DATE_NOW(),
      event_time: event.doc.created_at,
      delay_seconds: DATE_DIFF(DATE_NOW(), event.doc.created_at, 's'),
      status: 'LATE_ARRIVAL'
    } INTO late_facts
  ELSE
    -- Process normally
    INSERT {
      order_id: event.doc._id,
      status: 'PROCESSED'
    } INTO fact_orders
  ENDIF

RETURN event
```

29.10 Performance Optimization for Analytics

29.10.1 Materialized Views for OLAP

```
-- Create materialized view: daily_sales_summary
-- Run this nightly as batch job

BEGIN
  TRUNCATE daily_sales_summary

  FOR order IN orders
    FILTER order.status == 'completed'
    LET day = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')

    COLLECT date = day
    AGGREGATE
      total_revenue = SUM(order.amount),
      order_count = COUNT(1),
      avg_order = AVG(order.amount),
      max_order = MAX(order.amount),
      min_order = MIN(order.amount)

    INSERT {
      _key: date,
      date,
      total_revenue,
      order_count,
      avg_order: ROUND(avg_order, 2),
      max_order,
      min_order,
      created_at: DATE_NOW()
    } INTO daily_sales_summary

  COMMIT
RETURN { inserted: LENGTH(daily_sales_summary) }
```

29.10.2 Sampling for Large Datasets

```
-- Sample-based analytics (1% of data for speed)
FOR order IN orders
  FILTER RAND() < 0.01  -- 1% sample
  COLLECT month = DATE_FORMAT(order.created_at, '%yyyy-%mm')
  AGGREGATE
    sample_revenue = SUM(order.amount),
    sample_orders = COUNT(1)
  RETURN {
    month,
    estimated_revenue: ROUND(sample_revenue / 0.01, 0),  -- Extrapolate
    estimated_orders: ROUND(sample_orders / 0.01, 0),
    confidence: "95% (p < 0.05)"
  }
```

29.10.3 Incremental Analytics (Delta Processing)

```
-- Only process changes since last run
LET last_run = @last_checkpoint || '2000-01-01'

FOR order IN orders
  FILTER order.updated_at > last_run

  LET day = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')

  COLLECT date = day
  AGGREGATE
    new_revenue = SUM(order.amount),
    new_orders = COUNT(1)

  -- Update materialized view incrementally
  LET current = DOCUMENT('daily_sales_summary/' + date)
  UPDATE current WITH {
    total_revenue: (current.total_revenue || 0) + new_revenue,
    order_count: (current.order_count || 0) + new_orders,
    last_updated: DATE_NOW()
  } IN daily_sales_summary

  RETURN { date, new_revenue, new_orders }
```

29.11 Case Study: E-Commerce Analytics Platform

29.11.1 Data Model

Dimension Tables:

- customers (id, name, region, segment)
- products (id, name, category, brand, price)
- time (date, month, quarter, year, day_of_week)
- stores (id, location, region)

Fact Tables:

- order_facts (customer, product, store, time, amount, quantity)
- product_views (customer, product, time, duration)
- cart_events (customer, product, time, action)

29.11.2 Top-Level Metrics Query

```
-- Daily KPI dashboard
LET today = DATE_FORMAT(DATE_NOW(), '%yyyy-%mm-%dd')

FOR fact IN order_facts
  FILTER fact.date == today

  COLLECT WITH COUNT INTO total_orders
  AGGREGATE
    gmv = SUM(fact.amount),
    avg_order_value = AVG(fact.amount),
    units_sold = SUM(fact.quantity)

LET top_products = (
  FOR f IN order_facts
    FILTER f.date == today
    COLLECT product_id = f.product_id INTO group = f
    AGGREGATE revenue = SUM(group[*].amount)
    SORT revenue DESC
    LIMIT 5
    RETURN { product_id, revenue }
)

LET top_regions = (
  FOR f IN order_facts
    FILTER f.date == today
    LET store = DOCUMENT(f.store_id)
    COLLECT region = store.region INTO group = f
    AGGREGATE revenue = SUM(group[*].amount)
    SORT revenue DESC
    LIMIT 3
    RETURN { region, revenue }
)

RETURN {
  date: today,
  metrics: {
    total_orders,
    gmv: ROUND(gmv, 2),
    aov: ROUND(avg_order_value, 2),
    units_sold
  },
  top_products,
  top_regions
}
```

29.12 Governance & Data Quality

29.12.1 Data Quality Metrics

```
-- Monitor data quality
FOR event IN events
  COLLECT
    date = DATE_FORMAT(event.created_at, '%yyyy-%mm-%dd'),
    event_type = event.type
  AGGREGATE
    event_count = COUNT(1),
    null_values = SUM(event.value == NULL ? 1 : 0),
    duplicates = SUM(
      LENGTH(
        FOR e IN events
          FILTER e.event_id == event.event_id
          RETURN e
      ) > 1 ? 1 : 0
    ),
    invalid_timestamps = SUM(
      event.created_at > DATE_NOW() ? 1 : 0
    )
  RETURN {
    date,
    event_type,
    quality_score: ROUND(
      (event_count - null_values - duplicates - invalid_timestamps) /
      event_count * 100, 1
    ),
    issues: {
      nulls: null_values,
      dups: duplicates,
      future_dates: invalid_timestamps
    }
  }
}
```

29.12.2 SLA Compliance Monitoring

```
-- Monitor analytics SLA: 99.5% uptime, < 30s query latency
FOR query IN analytics_log
  COLLECT
    hour = DATE_FORMAT(query.executed_at, '%yyyy-%mm-%dd %hh:00'),
    query_type = query.type
  AGGREGATE
    successful = SUM(query.status == 'success' ? 1 : 0),
    failed = SUM(query.status == 'failed' ? 1 : 0),
    p95_latency = PERCENTILE(query.latency_ms, 0.95),
    p99_latency = PERCENTILE(query.latency_ms, 0.99),
    max_latency = MAX(query.latency_ms)
  RETURN {
    hour,
    query_type,
    availability: ROUND(successful / (successful + failed) * 100, 2),
    sla_met: (p99_latency < 30000),
    latency: {
      p95_ms: ROUND(p95_latency, 0),
      p99_ms: ROUND(p99_latency, 0),
      max_ms: max_latency
    }
  }
}
```

29.13 Zusammenfassung: Analytics-Architektur in ThemisDB

Architektur-Schichten

Schicht	Technologie	Latenz	Nutzung
Real-Time	Changefeeds + Streaming	<1s	Live Dashboard
Near-Real-Time	Micro-batch (5-60s)	5-60s	Alert Systems
Tactical	Views (hourly/daily)	1-24h	Standard Reports
Strategic	Data Warehouse	1-7d	Historical Analysis

Best Practices Checklist

- [] **Modeling:** Dimensional (Star/Snowflake) für OLAP
- [] **Aggregation:** Materialized Views für häufige Queries
- [] **Performance:** Sampling für explorative Analyse

-
- [] **Incremental:** Delta-Processing seit letztem Checkpoint
 - [] **Quality:** Monitoring für Duplicates, Nulls, Out-of-Order
 - [] **Real-Time:** Changefeeds für Live Dashboards
 - [] **Governance:** SLA Monitoring für Analytics Pipelines
 - [] **Retention:** TTL-Indizes für Event Log Cleanup

Kapitel 29 von 30 | Teil XI: Analytics & Operations | ~10.000 Wörter (+2000 neu)

Kapitel 40: Kapitel 30: Deployment & Operations

"Deployment ist kein einmaliger Akt, sondern ein kontinuierlicher Prozess."

Überblick

Dieses Kapitel behandelt die produktive Bereitstellung von ThemisDB: von Docker-Containern über Kubernetes bis hin zu Cloud-Deployments. Außerdem lernen Sie Monitoring, Backup-Strategien und Skalierungstechniken.

Was Sie in diesem Kapitel lernen: - Docker-basierte Deployments - Kubernetes-Orchestrierung mit Helm Charts - Cloud-Provider-Integration (AWS, Azure, GCP) - Monitoring mit Prometheus & Grafana - Backup & Disaster Recovery - Horizontal Scaling & Sharding - Security Best Practices

Voraussetzungen: Kapitel 4 (Installation), Grundkenntnisse in Containerisierung.

30.1 Docker Deployment

30.1.1 Basic Docker Setup

Dockerfile:

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y libssl3 libzstd1
RUN useradd -r themis
COPY themis_server /usr/local/bin/
EXPOSE 8529
CMD ["themis_server"]
```

docker-compose.yml:

```
services:
  themis:
    image: themisdb/themis:v1.3.4
    ports:
      - "8529:8529"
    volumes:
      - themis-data:/var/lib/themis/data
    healthcheck:
      test: ["curl", "-f", "http://localhost:8529/_api/version"]

volumes:
  themis-data:
```

Starten:

```
docker build -t themisdb/themis:v1.3.4 .

docker-compose up -d

docker-compose logs -f themis

curl http://localhost:8529/_api/version
```

30.1.2 Docker Multi-Stage Build

```
FROM ubuntu:24.04 AS builder

RUN apt-get update && apt-get install -y \
    cmake \
    g++ \
    git \
    ninja-build \
    libssl-dev \
    libzstd-dev \
    libjemalloc-dev

WORKDIR /build
COPY . .
RUN cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
RUN cmake --build build --target themis_server -j8

FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    libssl3 \
    libzstd1 \
    libjemalloc2 \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /build/build/themis_server /usr/local/bin/

RUN useradd -r -u 999 -g users themis && \
    mkdir -p /var/lib/themis/data && \
    chown -R themis:users /var/lib/themis

USER themis
EXPOSE 8529 8530

CMD ["/usr/local/bin/themis_server"]
```

30.2 Kubernetes Deployment

30.2.1 Kubernetes Manifests

namespace.yaml:

```
apiVersion: v1
kind: Namespace
metadata:
  name: themis-prod
```

configmap.yaml:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: themis-config
  namespace: themis-prod
data:
  themis.conf: |
    [database]
    path = /var/lib/themis/data

    [server]
    endpoint = http://0.0.0.0:8529
    threads = 16

    [logging]
    level = info
    file = /var/log/themis/themis.log
```

statefulset.yaml:

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: themis
  namespace: themis-prod
spec:
  serviceName: themis-headless
  replicas: 3
  selector:
    matchLabels:
      app: themis
  template:
    metadata:
      labels:
        app: themis
    spec:
      containers:
        - name: themis
          image: themisdb/themis:v1.3.4
          ports:
            - containerPort: 8529
              name: http
            - containerPort: 8530
              name: websocket
          volumeMounts:
            - name: data
              mountPath: /var/lib/themis/data
            - name: config
              mountPath: /etc/themis
      resources:
        requests:
          memory: "4Gi"
          cpu: "2000m"
        limits:
          memory: "8Gi"
          cpu: "4000m"
      livenessProbe:
        httpGet:
          path: /_api/version
          port: 8529
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /_api/version
          port: 8529
        initialDelaySeconds: 5
        periodSeconds: 5
      volumes:

```

```
- name: config
  configMap:
    name: themis-config
volumeClaimTemplates:
- metadata:
  name: data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: fast-ssd
    resources:
      requests:
        storage: 100Gi
```

service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: themis-headless
  namespace: themis-prod
spec:
  clusterIP: None
  selector:
    app: themis
  ports:
    - port: 8529
      name: http
    - port: 8530
      name: websocket

---
apiVersion: v1
kind: Service
metadata:
  name: themis-loadbalancer
  namespace: themis-prod
spec:
  type: LoadBalancer
  selector:
    app: themis
  ports:
    - port: 8529
      targetPort: 8529
      name: http
```

Deploy:

```
kubectl apply -f namespace.yaml
kubectl apply -f configmap.yaml
kubectl apply -f statefulset.yaml
kubectl apply -f service.yaml

kubectl get pods -n themis-prod

kubectl get svc -n themis-prod

kubectl logs -f themis-0 -n themis-prod
```

30.2.2 Helm Chart

Chart.yaml:

```
apiVersion: v2
name: themis
description: A Helm chart for ThemisDB
type: application
version: 1.3.4
appVersion: "1.3.4"
```

values.yaml:


```
replicaCount: 3

image:
  repository: themisdb/themis
  tag: v1.3.4
  pullPolicy: IfNotPresent

resources:
  requests:
    memory: 4Gi
    cpu: 2000m
  limits:
    memory: 8Gi
    cpu: 4000m

persistence:
  enabled: true
  storageClass: fast-ssd
  size: 100Gi

service:
  type: LoadBalancer
  port: 8529

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    - host: themis.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: themis-tls
      hosts:
        - themis.example.com

monitoring:
  enabled: true
  serviceMonitor:
    enabled: true
    interval: 30s
```

Install Helm Chart:

```

helm repo add themis https://charts.themisdb.io
helm repo update

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --values values.yaml

helm upgrade themis-prod themis/themis \
  --namespace themis-prod \
  --values values.yaml

helm uninstall themis-prod -n themis-prod

```

30.3 Cloud Provider Integration

30.3.1 AWS Deployment

EKS Cluster:

```

eksctl create cluster \
  --name themis-prod \
  --region eu-central-1 \
  --nodegroup-name standard-workers \
  --node-type m5.2xlarge \
  --nodes 3 \
  --nodes-min 3 \
  --nodes-max 10 \
  --managed

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=gp3 \
  --set service.annotations."service\.beta\.kubernetes\.io/aws-load-balancer-type"=nlb

```

RDS Integration (Optional):

```

metadata:
  backend: postgresql
  connection: "postgresql://user:pass@themis-metadata.c9akljh4.eu-central-1.rds.amazonaws.com:5432/themis"

```

30.3.2 Azure Deployment

AKS Cluster:

```
az aks create \
  --resource-group themis-rg \
  --name themis-prod \
  --location westeurope \
  --node-count 3 \
  --node-vm-size Standard_D8s_v3 \
  --enable-managed-identity \
  --generate-ssh-keys

az aks get-credentials \
  --resource-group themis-rg \
  --name themis-prod

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=managed-premium
```

30.3.3 GCP Deployment

GKE Cluster:

```
gcloud container clusters create themis-prod \
  --region europe-west3 \
  --num-nodes 3 \
  --machine-type n2-standard-8 \
  --disk-size 100

gcloud container clusters get-credentials themis-prod \
  --region europe-west3

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=pd-ssd
```

30.4 Monitoring & Observability

30.4.1 Prometheus Integration

ServiceMonitor:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: themis
  namespace: themis-prod
spec:
  selector:
    matchLabels:
      app: themis
  endpoints:
    - port: http
      path: /metrics
      interval: 30s
```

Key Metrics:

```
themis_http_requests_total
themis_http_request_duration_seconds

themis_db_operations_total
themis_db_size_bytes
themis_db_compaction_duration_seconds

themis_cache_hit_ratio
themis_cache_size_bytes

themis_thread_pool_active
themis_thread_pool_queue_size
```

30.4.2 Grafana Dashboards

Dashboard JSON:

```

{
  "dashboard": {
    "title": "ThemisDB Overview",
    "panels": [
      {
        "title": "Request Rate",
        "targets": [
          {
            "expr": "rate(themis_http_requests_total[5m])"
          }
        ]
      },
      {
        "title": "Database Size",
        "targets": [
          {
            "expr": "themis_db_size_bytes"
          }
        ]
      },
      {
        "title": "Cache Hit Ratio",
        "targets": [
          {
            "expr": "themis_cache_hit_ratio"
          }
        ]
      }
    ]
  }
}

```

30.4.3 Alerting Rules

prometheus-rules.yaml:

```

apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: themis-alerts
  namespace: themis-prod
spec:
  groups:
    - name: themis
      interval: 30s
      rules:
        - alert: ThemisHighErrorRate
          expr: rate(themis_http_requests_total{status=~"5.."}[5m]) > 0.05
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "High error rate on ThemisDB"
            description: "Error rate is {{ $value }} req/s"

        - alert: ThemisHighMemoryUsage
          expr: (container_memory_usage_bytes{pod=~"themis-.*"} / container_spec_memory_limit_bytes) > 0.9
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "ThemisDB high memory usage"
            description: "Memory usage is {{ $value | humanizePercentage }}"

        - alert: ThemisDiskSpaceLow
          expr: (node_filesystem_avail_bytes{mountpoint="/var/lib/themis/data"} / node_filesystem_size_bytes) < 0.1
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "ThemisDB low disk space"
            description: "Only {{ $value | humanizePercentage }} disk space remaining"

```

30.5 Backup & Disaster Recovery

30.5.1 Snapshot Backups

Backup Script:

```
#!/bin/bash

BACKUP_DIR="/var/backups/themis"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DB_PATH="/var/lib/themis/data"

curl -X POST http://localhost:8529/_admin/backup/create \
  -H "Content-Type: application/json" \
  -d '{
    "label": "backup-'$TIMESTAMP'",
    "timeout": 300
  }'

sleep 10

rsync -av --progress $DB_PATH/snapshots/backup-$TIMESTAMP \
  $BACKUP_DIR/backup-$TIMESTAMP

aws s3 sync $BACKUP_DIR/backup-$TIMESTAMP \
  s3://themis-backups/backup-$TIMESTAMP \
  --storage-class GLACIER

find $BACKUP_DIR -type d -mtime +7 -exec rm -rf {} +

echo "Backup completed: backup-$TIMESTAMP"
```

Cron Job:

```
0 2 * * * /usr/local/bin/themis-backup.sh >> /var/log/themis-backup.log 2>&1
```

30.5.2 Point-in-Time Recovery

```
aws s3 ls s3://themis-backups/

aws s3 sync s3://themis-backups/backup-20250115_020000 \
/var/restore/themis/

systemctl stop themis

rm -rf /var/lib/themis/data
cp -r /var/restore/themis /var/lib/themis/data

systemctl start themis

curl http://localhost:8529/_api/version
```

30.5.3 Continuous Backup with WAL

```
backup:
  wal_archiving:
    enabled: true
    archive_command: "aws s3 cp %p s3://themis-wal/%f"
    archive_timeout: 60s
```

30.6 Horizontal Scaling

30.6.1 Sharding Strategy

```

flowchart TB
    A[Load Balancer] --> B[Shard 1<br/>ID Range: 0-333]
    A --> C[Shard 2<br/>ID Range: 334-666]
    A --> D[Shard 3<br/>ID Range: 667-999]

    B --> B1[(RocksDB 1)]
    C --> C1[(RocksDB 2)]
    D --> D1[(RocksDB 3)]

    style A fill:#e1f5ff
    style B fill:#fff4e1
    style C fill:#e1ffe1
    style D fill:#ffe1f5

```

Abb. 30.1: Deployment-Strategy-Matrix

Sharding Config:

```

sharding:
  enabled: true
  strategy: hash
  shards:
    - id: shard1
      endpoint: http://themis-shard1:8529
      range: [0, 333]
    - id: shard2
      endpoint: http://themis-shard2:8529
      range: [334, 666]
    - id: shard3
      endpoint: http://themis-shard3:8529
      range: [667, 999]

```

30.6.2 Read Replicas

```
replication:
  enabled: true
  mode: async
  replicas:
    - endpoint: http://themis-replica1:8529
      lag_max: 1s
    - endpoint: http://themis-replica2:8529
      lag_max: 1s
```

30.7 Security Best Practices

30.7.1 TLS/SSL Configuration

```
server:
  tls:
    enabled: true
    cert_file: /etc/themis/tls/cert.pem
    key_file: /etc/themis/tls/key.pem
    ca_file: /etc/themis/tls/ca.pem
    min_version: "1.3"
```

30.7.2 Authentication

```
auth:
  enabled: true
  providers:
    - type: jwt
      issuer: "https://auth.example.com"
      audience: "themis-api"
      jwks_uri: "https://auth.example.com/.well-known/jwks.json"
    - type: basic
      users_file: /etc/themis/users.htpasswd
```

30.7.3 Network Policies (Kubernetes)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: themis-netpol
  namespace: themis-prod
spec:
  podSelector:
    matchLabels:
      app: themis
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8529
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: redis
      ports:
        - protocol: TCP
          port: 6379
```

30.8 Performance Tuning

30.8.1 Resource Limits

```
resources:
  requests:
    memory: "8Gi"
    cpu: "4000m"
  limits:
    memory: "16Gi"
    cpu: "8000m"

env:
  - name: ROCKSDB_MAX_OPEN_FILES
    value: "10000"
  - name: ROCKSDB_BLOCK_CACHE_SIZE
    value: "4GB"
```

30.8.2 Connection Pooling

```
server:
  connection_pool:
    size: 100
    timeout: 30s
    max_idle: 50
```

30.9 Zusammenfassung

Deployment-Optionen

1. **Docker:** Einfacher Einstieg, lokale Entwicklung
2. **Kubernetes:** Produktionsreif, auto-scaling, self-healing
3. **Cloud:** Managed Services (EKS, AKS, GKE)

Operations-Checkliste

- ☒ Monitoring mit Prometheus & Grafana
- ☒ Tägliche Backups mit S3/Glacier
- ☒ TLS/SSL für alle Verbindungen
- ☒ Network Policies für Pod-Isolation
- ☒ Resource Limits für Stabilität

-  Alerting für kritische Metriken
-  Disaster Recovery Plan dokumentiert

30.8 Advanced Deployment Patterns

30.8.1 Blue-Green Deployments

```
apiVersion: v1
kind: Service
metadata:
  name: themis-service
spec:
  selector:
    app: themis
    version: blue # Initially point to blue
  ports:
    - port: 8529
      targetPort: 8529
  type: LoadBalancer
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: themis-blue-config
data:
  version: "blue"
  database:
    replication_role: "primary"
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: themis-green-config
data:
  version: "green"
  database:
    replication_role: "primary"
```

Deployment Steps: 1. Deploy Green with new version (idle) 2. Run smoke tests against Green 3. Sync data: Primary → Green (catchup) 4. Switch load balancer: Service selector → green 5. Blue becomes backup/rollback

30.8.2 Canary Deployments

```
---
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: themis-canary
spec:
  hosts:
  - themis.internal
  http:
  - match:
    - uri:
        prefix: "/"
    route:
    - destination:
        host: themis-stable
        port:
          number: 8529
        weight: 95 # 95% to stable
    - destination:
        host: themis-canary
        port:
          number: 8529
        weight: 5 # 5% to canary (new version)
    timeout: 10s
    retries:
      attempts: 3
      perTryTimeout: 5s
```

Canary Rollout Timeline: - Hour 0-1: 5% traffic → Monitor error rates, latency - Hour 1-4: 25% traffic → Check P99 latency - Hour 4-8: 50% traffic → Full test with real workload - Hour 8+: 100% traffic → Full rollout

30.8.3 GitOps Continuous Deployment

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: themis-gitops
spec:
  project: default
  source:
    repoURL: https://github.com/org/themis-deployment.git
    targetRevision: main
    path: k8s/production
  destination:
    server: https://kubernetes.default.svc
    namespace: themis
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
  # Only deploy if tests pass in CI
  notification:
    when:
      onSuccess: notify-slack
      onFailure: notify-pagerduty
```

30.8.4 Staged Rollout Checklist

Pre-Deployment

- ✓ Run all integration tests
- ✓ Performance benchmarks (P99 latency < 10% increase)
- ✓ Memory/CPU profiling
- ✓ Backup created & tested

Deployment

- ✓ Dry-run: `'kubectl apply --dry-run=client -f manifest.yaml'`
- ✓ Deploy to staging first
- ✓ Smoke tests pass
- ✓ Metrics baseline: CPU, Memory, QPS, Latency
- ✓ Error rate < 0.1%

Monitoring (First Hour)

- ✓ P99 latency stable
- ✓ No error spikes
- ✓ No memory leaks
- ✓ Replication lag < 500ms

Post-Deployment

- ✓ Keep blue environment for 24h (rollback ready)
- ✓ Archive logs for audit
- ✓ Document any issues/hotfixes
- ✓ Send deployment notification

30.9 Disaster Recovery & Business Continuity

30.9.1 RTO/RPO Strategy

RTO (Recovery Time Objective) = Max downtime acceptable
 RPO (Recovery Point Objective) = Max data loss acceptable

ThemisDB Recommendations:

Tier	RTO	RPO	Cost	Effort
Bronze	24h	24h	\$	Low
Silver	4h	1h	\$\$	Med
Gold	1h	5min	\$\$\$	High
Platinum	15min	0min(RPO)	\$\$\$\$	Critical

Platinum = Multi-region sync (4 data centers, quorum write)

30.9.2 Disaster Recovery Plan

```
Kind: DisasterRecoveryPlan
metadata:
  name: themis-dr-plan
spec:
  backup:
    primary_daily: /backups/daily/themis-*.tar.gz
    archive_weekly: s3://backups/weekly/
    retention_years: 3

  failover_sequence:
    - detect_failure: "Primary health check fails 3x"
    - promote_secondary: "Promote Secondary to Primary"
    - validate_data: "Count docs, verify checksums"
    - test_write: "Insert test document"
    - notify_team: "PagerDuty alert + Slack message"
    - client_failover: "Update DNS CNAME"
    - monitor_2h: "Watch metrics for 2 hours"

  recovery_from_backup:
    - stop_database: "systemctl stop themis"
    - restore_volume: "Restore from snapshot"
    - verify_integrity: "themisdb recover --verify"
    - start_database: "systemctl start themis"
    - catchup_replicas: "Wait for lag < 1s"
```

30.9.3 Backup Verification Script

```
#!/bin/bash

BACKUP_DIR="/backups/daily"
VERIFY_TIMEOUT=3600

for backup in $BACKUP_DIR/themis-*.tar.gz; do
    echo "Verifying $backup..."

    # Extract to temp, verify database
    TEMP_DIR=$(mktemp -d)
    tar -xzf "$backup" -C "$TEMP_DIR"

    # Run recovery to check integrity
    timeout $VERIFY_TIMEOUT themisdb recover --verify "$TEMP_DIR" && \
        echo "✓ $backup: OK" || \
        echo "✗ $backup: CORRUPT - Investigate!"

    # Try restoring to test instance
    docker run --rm -v "$TEMP_DIR:/data" themis:test \
        themisdb check-data /data && \
        echo "✓ Data schema valid" || \
        echo "✗ Schema check failed"

    rm -rf "$TEMP_DIR"
done

echo "Backup verification complete"
```

30.10 Cost Optimization

30.10.1 Resource Right-Sizing

```
requests:
  memory: "1Gi"
  cpu: "500m"
limits:
  memory: "2Gi"
  cpu: "1000m"
storage: "50Gi"

requests:
  memory: "8Gi"
  cpu: "4"
limits:
  memory: "16Gi"
  cpu: "8"
storage: "500Gi"

requests:
  memory: "32Gi"
  cpu: "16"
limits:
  memory: "64Gi"
  cpu: "32"
storage: "2Ti"
```

30.10.2 Storage Cost Reduction

Strategy: Tiered Storage (Hot → Warm → Cold)

Hot (SSD, expensive)

- └ Active data < 30 days
- └ Real-time queries
- └ RTO requirement: <1h

Warm (HDD, medium)

- └ Data 30-90 days old
- └ Monthly analytics reports
- └ RTO requirement: <24h

Cold (Cloud Archive, cheap)

- └ Compliance archives (2+ years)
- └ Quarterly audits only
- └ RTO requirement: 1-7 days

Cost/GB/Month: SSD (\$0.10) → HDD (\$0.05) → Archive (\$0.004)

Example: 10TB dataset

SSD: \$1,024/month

Mixed (70% warm, 30% cold): \$216/month

Savings: 79%

30.11 Compliance & Audit

30.11.1 Audit Logging for Compliance

```
kind: ThemisDB
metadata:
  name: themis-production
spec:
  audit:
    enabled: true
    destination: s3://audit-logs/ # External WORM storage
    events:
      - user_login
      - data_access
      - schema_change
      - admin_action
      - privilege_grant
      - backup_restore

  compliance:
    frameworks:
      - SOC2
      - ISO27001
      - GDPR
      - HIPAA (if healthcare)

  data_retention:
    audit_logs: "7 years"
    application_logs: "90 days"
    metrics: "1 year"
```

30.11.2 Compliance Checklist

Security

- ✓ TLS 1.3 for all connections
- ✓ Network policies (allow only required)
- ✓ RBAC with MFA for admin access
- ✓ Secrets in Vault/K8s Secrets
- ✓ Encryption at rest (AES-256)
- ✓ Encryption in transit (TLS)

Data Protection

- ✓ PII detection & masking
- ✓ Data classification (PUBLIC/INTERNAL/CONFIDENTIAL)
- ✓ GDPR right-to-deletion implemented
- ✓ Backup encryption with managed keys
- ✓ No cleartext passwords in logs

Operations

- ✓ Change control (code review before deploy)
- ✓ Incident response playbooks
- ✓ Disaster recovery tested (quarterly)
- ✓ Security scanning (SBOM, vulnerability scanning)
- ✓ Penetration testing (annual)

Monitoring

- ✓ Access logs (all queries logged)
- ✓ Admin action audit trail
- ✓ Security event alerting
- ✓ SLA monitoring (uptime SLA > 99.9%)

30.12 Operations Runbooks

30.12.1 Scaling Checklist (Horizontal)

Add Node to Cluster

1. Provision new node (VM/Bare Metal)
2. Install ThemisDB + dependencies
3. Configure as follower initially
4. Join cluster: `themisdb cluster join --primary <primary-ip>`
5. Wait for replication lag < 100ms
6. Promote to voting member: `themisdb cluster promote --member-id <id>`
7. Verify: `themisdb cluster status` (should show HEALTHY)
8. Monitor CPU/Memory/IO for 24h

Remove Node from Cluster

1. Drain connections: `themisdb admin drain-connections`
2. Wait for active queries to finish (30s timeout)
3. Demote member: `themisdb cluster demote --member-id <id>`
4. Remove from cluster: `themisdb cluster remove --member-id <id>`
5. Verify: `themisdb cluster status` (node should be REMOVED)
6. Decommission VM

30.12.2 Incident Response (Example: High Memory Usage)

Symptoms

- Memory usage > 85%
- OOM killer started (dmesg shows)
- Queries failing with "Out of memory"

Investigation (2 min)

```
curl http://localhost:8529/_admin/memory | jq .  
# Check: Cache size, Buffer pool, Active cursors
```

Fix (5 min)

Option A: Restart (emergency)

```
systemctl restart themis  
→ Memory drops to baseline
```

Option B: Increase memory (planned)

```
Edit themis.conf: cache.size_mb = 16384  
systemctl reload themis
```

Option C: Identify leak (analysis)

```
themisdb profile --duration 60s | grep alloc  
→ Find problematic query/operation
```

Verify

```
curl http://localhost:8529/_admin/memory | jq .usage_pct  
# Should be < 80%
```

30.13 Zusammenfassung: Operations Checkliste

Tägliche Aufgaben

- ☐ Backup erfolgreich (log check)
- ☐ Replication lag < 500ms
- ☐ Disk usage < 80%
- ☐ No error logs above WARN
- ☐ Metrics collected (Prometheus)

Wöchentliche Aufgaben

- ☐ Backup test restoration (pick random)
- ☐ Security scan (OWASP top 10)
- ☐ Performance baseline check

- ☐ Review failed queries (slow log)
- ☐ Capacity planning review

Monatliche Aufgaben

- ☐ Disaster recovery drill
- ☐ Penetration testing (if required)
- ☐ Security policy review
- ☐ Cost optimization (right-size resources)
- ☐ Upgrade path planning

Quarterly

- ☐ Full disaster recovery test (restore from backup)
- ☐ Security audit
- ☐ Compliance verification
- ☐ Training for new operators

Kapitel 30 von 30 | Teil XI: Operations | ~10.000 Wörter (+2000 neu)

Kapitel 41: Kapitel 31: API-Protokolle (HTTP/2, HTTP/3, MCP)

"Protokolle sind das Rückgrat verteilter Systeme."

31.1 Überblick

Dieses Kapitel behandelt die Protokoll-Features von ThemisDB: moderne HTTP/2/HTTP/3-Stacks, WebSockets für Echtzeit, Server-Sent Events (SSE) für Streams sowie MCP (Model Context Protocol) für LLM-Integrationen.

Was Sie lernen: - Wann HTTP/2/3 gegenüber HTTP/1.1 Vorteile bringt - Server Push, Header-Kompression, Multiplexing - QUIC-Tuning und Paketverlustverhalten - WebSocket/SSE-Patterns für Changefeed & CDC - MCP-Anbindung als Tooling-Schnittstelle

31.2 HTTP/2 Features

31.2.1 Multiplexing & Header-Kompression

```
sequenceDiagram
    participant C as Client
    participant S as Themis API
    C->>S: Stream 1 - GET /_api/version
    C->>S: Stream 3 - POST /_api/query (AQL)
    C->>S: Stream 5 - GET /_api/metrics
    Note over C,S: Alle Streams teilen eine TCP-Verbindung
    S-->>C: Response Stream 1
    S-->>C: Response Stream 3
    S-->>C: Response Stream 5
```

Abb. 31.1: API-Protocol-Stack

Vorteile: - Keine Head-of-Line-Blocking auf Applikationsebene - HPACK reduziert Header-Overhead (Auth, Tenant-ID) - Bessere Ausnutzung einzelner TCP-Verbindung

31.2.2 Server Push (selektiv)

```
:method: GET
:path: /dashboard
:authority: api.themis.local
```

Server antwortet mit gepushten Ressourcen (nur wenn vom Client erlaubt): -
 /static/dashboard.css - /static/dashboard.js - /static/logo.svg

Best Practice: Für APIs selten nötig; für Admin-UI/Cockpit kann Push Latenzen senken. Aktivieren Sie Push nur für statische Assets.

31.2.3 Stream Priorities

- AQL-Long-Running Queries → niedrige Priorität
- Health/Readiness → hohe Priorität
- Metrics → mittlere Priorität

31.3 HTTP/3 (QUIC)

31.3.1 QUIC vs TCP

Merkmal	TCP/TLS	QUIC
Verbindungsaufbau	1-2 RTT	0-1 RTT (0-RTT Resumption)
Head-of-Line	End-to-End	Per-Stream
Congestion Control	Reno/Cubic	BBR, Hybrid
Mobility	Neuaufbau nötig	Connection IDs erlauben Migration

31.3.2 QUIC-Tuning

- **Initial Congestion Window:** 10-20 MSS für schnellere Warmups
- **BBR aktivieren:** Für WAN/hohe Bandbreite
- **Handshake-Keys cachen:** 0-RTT für interne Service-zu-Service Calls
- **Pacing:** Aktivieren, um Burst-Loss zu vermeiden

31.3.3 Fallback-Strategie

```

flowchart TD
    A[Client Request] --> B{Supports HTTP/3?}
    B -->|Ja| C[Connect via QUIC]
    B -->|Nein| D[Fallback HTTP/2]
    C --> E{Loss > 5 percent?}
    E -->|Ja| D
    E -->|Nein| F[Continue HTTP/3]
    style C fill:#e1f5ff
    style D fill:#fff4e1
  
```

Abb. 31.2: Request-Response-Flow

31.4 Echtzeit: WebSocket vs SSE

31.4.1 Wann welches Protokoll?

Bedarf	WebSocket	SSE
Bidirektional	✓	✗
Firewalls/Proxies	Kann blockiert sein	Fast immer offen (HTTP)
Browser Support	Breit	Sehr breit
Backpressure	Muss implementiert werden	Implizit durch HTTP
Binärdaten	✓	✗ (Text-only)

31.4.2 Changefeed mit SSE (empfohlen)

```

GET /_api/changefeed?collection=orders&since=1735600000000
Accept: text/event-stream
Authorization: Bearer <token>
  
```

Server streamt Events:

```

event: insert
data: {"_key":"o123","status":"paid"}

retry: 3000
  
```

Vorteile SSE: Einfach, kompatibel, automatisches Reconnect über `retry` .

31.4.3 WebSocket für bidirektionale Commands

```
const ws = new WebSocket('wss://api.themis.local/_ws');
ws.onmessage = (ev) => console.log('event', ev.data);
ws.send(JSON.stringify({
  type: 'subscribe',
  topic: 'orders',
  filter: "status == 'paid'"
}));
```

Best Practice: - Heartbeat (`ping/pong`) alle 30s - Message Envelope mit `type` , `payload` , `seq` - Auth im Query-Parameter oder Header; Tokens regelmäßig erneuern

31.5 HTTP/2/3 Security

- **mTLS** zwischen Services (Client Cert + SAN auf Tenant/Env)
 - **ALPN** erzwingen (`h2` , `h3`), HTTP/1.1 nur als Fallback
 - **Rate Limits pro Tenant** (Header: `x-tenant-id`)
 - **Content-Security-Policy** für Admin-UI
 - **CORS** restriktiv konfigurieren
-

31.6 MCP (Model Context Protocol)

MCP ermöglicht LLM-Tools direkten Zugriff auf ThemisDB-Ressourcen.

31.6.1 Minimaler MCP-Server (Python)

```
from mcp import Server
import themis

server = Server(name="themis-mcp")
db = themis.connect()

@server.tool("query")
async def query(aql: str):
    return db.aql(aql)

@server.tool("vector-search")
async def search(text: str, k: int = 5):
    return db.knn("docs", db.embed(text), k)
```

31.6.2 Toolsicherheit

- **Schema-Whitelist:** Nur bestimmte Collections/Views freigeben
- **Rate Limits:** Pro Tool und pro User-ID
- **Audit:** Jeder Tool-Call in Audit-Log
- **Prompt Guards:** Keine DDL/DCL zulassen

31.6.3 Response-Formate

- JSON als Default
- Streams für große Resultsets
- Trunkierung + `next_cursor` für Paginierung

31.7 Observability

- **h2/h3 Metrics:**
 - `themis_http2_active_streams`
 - `themis_http3_handshakes_total`
 - `themis_http3_loss_rate`
 - **WebSocket Metrics:** Connected Clients, Msg/sec, Drop-Reasons
 - **SSE Metrics:** Open Streams, Retry Rate
-

31.8 Performance-Optimierungen

31.8.1 HTTP/2 Connection Tuning

```
http2:
  max_concurrent_streams: 250
  initial_window_size: 65535
  max_frame_size: 16384
  max_header_list_size: 8192

# Connection Pooling
connection_pool:
  max_idle_connections: 100
  idle_timeout: 300s
  keepalive_interval: 30s
```

Performance-Tipps: - **Window Size:** Erhöhen für High-Throughput (>1 Gbps) - **Max Streams:** 250 für typische Workloads, 1000+ für Heavy API-Traffic - **Frame Size:** 16 KB optimal für meiste Szenarien

31.8.2 HTTP/3 QUIC-Tuning

```
// Go: QUIC Configuration
quicConfig := &quic.Config{
  MaxIdleTimeout:      30 * time.Second,
  MaxIncomingStreams:  250,
  MaxIncomingUniStreams: 250,

  // Loss Detection
  MaxReceiveStreamFlowControlWindow: 6 * (1 << 20), // 6 MB
  MaxReceiveConnectionFlowControlWindow: 15 * (1 << 20), // 15 MB

  // Congestion Control
  InitialPacketSize: 1200, // Standard MTU - 28 bytes
  DisablePathMTUDiscovery: false,
}
```

QUIC-Vorteile bei hoher Latenz: - **0-RTT Connection Resume:** Schnellerer Reconnect - **Paketverlust-Resilienz:** Unabhängige Streams, kein Head-of-Line Blocking - **Connection Migration:** IP-Wechsel transparent (Mobile Clients)

31.8.3 Benchmarks: HTTP/1.1 vs HTTP/2 vs HTTP/3

```
import aiohttp
import asyncio
import time

async def benchmark_protocol(url, num_requests=1000):
    """Benchmark verschiedene Protokolle"""

    connector = aiohttp.TCPConnector(limit=100)
    timeout = aiohttp.ClientTimeout(total=60)

    async with aiohttp.ClientSession(connector=connector, timeout=timeout) as session:
        start = time.time()

        tasks = []
        for i in range(num_requests):
            task = session.get(f"{url}/_api/version")
            tasks.append(task)

        responses = await asyncio.gather(*tasks, return_exceptions=True)

        elapsed = time.time() - start
        success = sum(1 for r in responses if not isinstance(r, Exception))

        return {
            'total_time': elapsed,
            'requests_per_sec': num_requests / elapsed,
            'success_rate': success / num_requests * 100,
            'avg_latency_ms': (elapsed / num_requests) * 1000
        }
```


31.9 Advanced WebSocket Patterns

31.9.1 Multiplexed Subscriptions

```
// Client: Mehrere Subscriptions über eine WebSocket-Connection
const ws = new WebSocket('wss://themis.local/_api/realtime');

ws.onopen = () => {
  // Subscribe zu mehreren Collections gleichzeitig
  ws.send(JSON.stringify({
    type: 'subscribe',
    subscriptions: [
      {id: 'sub-1', collection: 'orders', filter: {status: 'pending'}},
      {id: 'sub-2', collection: 'inventory', filter: {stock: {<: 10}}},
      {id: 'sub-3', collection: 'logs', filter: {level: 'error'}}
    ]
  }));
};

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data);

  switch(msg.subscription_id) {
    case 'sub-1':
      console.log('New order:', msg.data);
      break;
    case 'sub-2':
      console.log('Low stock alert:', msg.data);
      break;
    case 'sub-3':
      console.log('Error log:', msg.data);
      break;
  }
};
```

31.9.2 Backpressure Handling

```
class ChangeStreamHandler:
    def __init__(self, websocket, buffer_size=1000):
        self.ws = websocket
        self.buffer = asyncio.Queue(maxsize=buffer_size)
        self.dropped_messages = 0

    async def handle_change(self, change_event):
        """Change Event vom DB-Changefeed"""
        try:
            # Non-blocking Queue Put
            self.buffer.put_nowait(change_event)
        except asyncio.QueueFull:
            # Langsamer Client → Drop Message
            self.dropped_messages += 1

            if self.dropped_messages % 100 == 0:
                # Warning nach jeweils 100 Drops
                await self.ws.send(json.dumps({
                    'type': 'backpressure_warning',
                    'dropped_messages': self.dropped_messages
                }))

    async def send_loop(self):
        """Kontinuierlich Messages aus Queue senden"""
        while True:
            change = await self.buffer.get()
            await self.ws.send(json.dumps(change))
```

31.9.3 Auto-Reconnect mit Exponential Backoff

```

// TypeScript: Resilient WebSocket Client
class ResilientWebSocket {
  private ws: WebSocket | null = null;
  private reconnectAttempts = 0;
  private maxReconnectDelay = 30000; // 30s

  connect(url: string) {
    this.ws = new WebSocket(url);

    this.ws.onopen = () => {
      console.log('Connected');
      this.reconnectAttempts = 0; // Reset
    };

    this.ws.onclose = (event) => {
      if (event.code !== 1000) { // 1000 = Normal Closure
        this.scheduleReconnect(url);
      }
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket error:', error);
    };
  }

  private scheduleReconnect(url: string) {
    this.reconnectAttempts++;

    // Exponential Backoff: 1s, 2s, 4s, 8s, 16s, 30s (max)
    const delay = Math.min(
      1000 * Math.pow(2, this.reconnectAttempts - 1),
      this.maxReconnectDelay
    );

    console.log(`Reconnecting in ${delay}ms (attempt ${this.reconnectAttempts})`);

    setTimeout(() => {
      this.connect(url);
    }, delay);
  }

  send(data: string) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(data);
    } else {
      console.warn('WebSocket not ready, message dropped');
    }
  }
}

```

31.10 MCP Advanced Patterns

31.10.1 Multi-Tenant MCP Server

```
from mcp import Server, Context
import themis

server = Server(name="themis-mcp-multi-tenant")

@server.tool("query", requires_auth=True)
async def query_with_tenant(ctx: Context, aql: str):
    """Query mit automatischer Tenant-Isolation"""

    tenant_id = ctx.user.tenant_id # Aus Auth-Token

    # Datenbankverbindung für spezifischen Tenant
    db = themis.connect(tenant=tenant_id)

    # Validierung: Kein DDL/DCL
    if any(keyword in aql.upper() for keyword in ['DROP', 'CREATE', 'ALTER', 'GRANT']):
        raise PermissionError("DDL/DCL queries not allowed")

    # Audit-Log
    await log_audit_event(
        tenant_id=tenant_id,
        user_id=ctx.user.id,
        action='mcp_query',
        query=aql
    )

    # Query ausführen
    result = await db.query(aql)

    # Trunkierung bei großen Results
    if len(result) > 100:
        return {
            'results': result[:100],
            'truncated': True,
            'total_count': len(result),
            'message': 'Results truncated to 100 items'
        }

    return {'results': result, 'truncated': False}
```

31.10.2 MCP Tool Discovery

```
@server.list_tools()
async def available_tools(ctx: Context):
    """Dynamische Tool-Liste basierend auf User-Permissions"""

    tools = []
    permissions = await get_user_permissions(ctx.user.id)

    if 'query' in permissions:
        tools.append({
            'name': 'query',
            'description': 'Execute AQL query (SELECT only)',
            'parameters': {
                'aql': {'type': 'string', 'required': True}
            }
        })

    if 'vector-search' in permissions:
        tools.append({
            'name': 'vector-search',
            'description': 'Semantic search over documents',
            'parameters': {
                'text': {'type': 'string', 'required': True},
                'k': {'type': 'integer', 'default': 5}
            }
        })

    if 'analytics' in permissions:
        tools.append({
            'name': 'aggregate',
            'description': 'Run pre-defined analytics queries',
            'parameters': {
                'report_type': {
                    'type': 'enum',
                    'values': ['daily-sales', 'user-stats', 'inventory']
                }
            }
        })

    return tools
```

31.10.3 MCP Rate Limiting

```

from collections import defaultdict
import time

class MCPRateLimiter:
    def __init__(self, max_calls_per_minute=60):
        self.max_calls = max_calls_per_minute
        self.calls = defaultdict(list) # user_id -> [timestamps]

    def check_limit(self, user_id: str) -> bool:
        """Prüfe ob User unter Rate Limit ist"""
        now = time.time()
        minute_ago = now - 60

        # Cleanup alte Einträge
        self.calls[user_id] = [
            ts for ts in self.calls[user_id] if ts > minute_ago
        ]

        # Check Limit
        if len(self.calls[user_id]) >= self.max_calls:
            return False

        # Record Call
        self.calls[user_id].append(now)
        return True

limiter = MCPRateLimiter(max_calls_per_minute=100)

@server.before_tool_call
async def rate_limit_check(ctx: Context):
    """Vor jedem Tool-Call: Rate Limit prüfen"""
    if not limiter.check_limit(ctx.user.id):
        raise RateLimitExceeded(
            f"Rate limit exceeded: max {limiter.max_calls} calls/minute"
        )

```

31.11 Production Checklist

HTTP/2/3 Deployment

-  TLS 1.3 aktiviert (Voraussetzung für HTTP/3)
-  ALPN konfiguriert ( , )
-  Firewall: UDP Port 443 für QUIC

-
-  Load Balancer unterstützt HTTP/2 Backend-Connections
 -  Monitoring: `http2_streams_active` , `http3_handshakes`

WebSocket/SSE

-  Connection Timeout: 5-10 Minuten Idle
-  Ping/Pong für Keepalive (30s Intervall)
-  Backpressure Handling implementiert
-  Auto-Reconnect Client-seitig
-  Max Concurrent Connections pro Server: 10k+

MCP Security

-  Authentication: OAuth2/JWT
 -  Tool-Whitelist pro User-Rolle
 -  Rate Limiting: 100 calls/min
 -  Audit-Logging: Jeder Tool-Call
 -  Query-Validation: Kein DDL/DCL
 -  Result Truncation: Max 1000 Results
-

31.8 Advanced Protocol Scenarios

31.8.1 Connection Pooling & Load Balancing

```
class ThemisClientPool:
    def __init__(self, primary_url, replica_urls, pool_size=20):
        self.primary_pool = HTTPConnectionPool(primary_url, maxsize=pool_size)
        self.replica_pools = [
            HTTPConnectionPool(url, maxsize=pool_size//len(replica_urls))
            for url in replica_urls
        ]

    def query(self, aql, bind_vars=None, consistency='eventual'):
        if consistency == 'strong':
            # Route to primary for strong consistency
            pool = self.primary_pool
        else:
            # Route to replicas (round-robin)
            pool = self.next_replica()

        # Connection reused from pool
        return pool.request('POST', '/_api/query',
                           json={'query': aql, 'bindVars': bind_vars})

    def next_replica(self):
        self.replica_index = (self.replica_index + 1) % len(self.replica_pools)
        return self.replica_pools[self.replica_index]
```

Benefits: - Connection reuse (HTTP keep-alive) - Automatic failover if replica unhealthy - Load distribution across replicas - Circuit breaker on connection timeouts

31.8.2 Bidirectional Streaming (gRPC-like)

```
// Imaginary streaming API
service ThemisStreaming {
    rpc StreamQuery(stream QueryRequest) returns (stream QueryResponse);
}

// Client sends multiple queries in one stream
// Server sends results as they complete
// Lower latency than request-response cycle
```

Implementation with WebSockets:

```
// Client
const ws = new WebSocket('wss://api.themis/stream');

ws.onopen = () => {
  // Send batch of queries
  ws.send(JSON.stringify({
    queries: [
      { id: 1, aql: 'FOR u IN users...' },
      { id: 2, aql: 'FOR o IN orders...' },
      { id: 3, aql: 'FOR p IN products...' }
    ]
  }));
};

ws.onmessage = (event) => {
  const result = JSON.parse(event.data);
  console.log(`Query ${result.query_id} completed:`, result.data);
};
```

31.8.3 Graceful Degradation (Protocol Negotiation)

Client tries protocols in order:

1. HTTP/3 (QUIC) ← Fastest if no packet loss
2. HTTP/2 ← Fallback for UDP issues
3. HTTP/1.1 ← Last resort

Configuration:

```
server:
  protocols:
    - h3          # HTTP/3
    - h2          # HTTP/2
    - http/1.1    # HTTP/1.1

  # ALPN (Application Layer Protocol Negotiation)
  alpn_protocols: ['h3', 'h2', 'http/1.1']
```

31.9 Performance Tuning by Protocol

31.9.1 HTTP/2 Tuning

```
server:
  http2:
    # Number of concurrent streams per connection
    max_concurrent_streams: 128 # Default: 100

    # Header size limit
    max_header_list_size: 32768 # 32KB

    # Server push (usually disabled for better caching)
    enable_server_push: false
```

Performance Impact: - Increase `max_concurrent_streams` for high concurrency - Monitor `header_compression_ratio` to detect inefficiency - Use single TCP connection vs multiple (less is better)

31.9.2 QUIC/HTTP/3 Tuning

```
QUIC handshake optimization:
- 0-RTT (Zero Round Trip): Resume previous connection
- TLS 1.3: Faster handshake than HTTP/2 + TLS 1.2
- Connection Migration: Survive network switch (WiFi → 4G)
```

Typical Handshake Times:

```
HTTP/1.1 + TLS 1.2:  3x RTT  (150-200ms on 50ms RTT)
HTTP/2 + TLS 1.3:   1x RTT  (~50ms)
HTTP/3 + 0-RTT:     0x RTT  (~0ms on reconnect!)
```

31.9A PostgreSQL Wire Protocol Enhancements (v1.4.0-alpha)

31.9A.1 Erweiterte Message-Typen

Neu in v1.4.0-alpha: Unterstützung für erweiterte PostgreSQL-Protokoll-Features für verbesserte Kompatibilität und Performance.

COPY-Protokoll für Bulk-Operations:

Das COPY-Protokoll ermöglicht hochperformante Bulk-Inserts und -Exports direkt über das Wire Protocol.

```
-- COPY FROM (Bulk Insert)
COPY users(id, name, email, created_at) FROM STDIN WITH (FORMAT CSV);
1,Alice,alice@example.com,2026-01-06
2,Bob,bob@example.com,2026-01-06
3,Charlie,charlie@example.com,2026-01-06
\.

-- COPY TO (Bulk Export)
COPY (SELECT * FROM users WHERE created_at > '2026-01-01')
TO STDOUT WITH (FORMAT BINARY);
```

Python-Client-Beispiel:

```

import psycopg2
from io import StringIO

conn = psycopg2.connect(
    host='localhost',
    port=5432,
    dbname='themisdb',
    user='admin'
)

cursor = conn.cursor()
data = StringIO("""1,Alice,alice@example.com,2026-01-06
2,Bob,bob@example.com,2026-01-06
3,Charlie,charlie@example.com,2026-01-06""")

cursor.copy_from(
    data,
    'users',
    columns=('id', 'name', 'email', 'created_at'),
    sep=', '
)
conn.commit()

output = StringIO()
cursor.copy_to(
    output,
    'users',
    sep=', ',
    columns=('id', 'name', 'email')
)
print(output.getvalue())

```

Performance-Charakteristiken:

Method	Throughput	Latenz	Use Case
INSERT (single)	5K rows/s	0.2ms/row	Transaktional
INSERT (batch)	45K rows/s	0.02ms/row	Batch
COPY	250K rows/s	0.004ms/row	Bulk Import

LISTEN/NOTIFY für Change Notifications:

Echtzeit-Benachrichtigungen über Datenänderungen ohne Polling.

```
-- Session 1: Listener
LISTEN user_changes;

-- Warten auf Notifications (blocking)
-- Client erhält Notification wenn Event auftritt

-- Session 2: Publisher
INSERT INTO users (name, email) VALUES ('Dave', 'dave@example.com');
NOTIFY user_changes, 'new_user:dave';

-- Session 1 erhält: Notification auf Kanal 'user_changes' mit Payload 'new_user:dave'
```

Python-Client mit LISTEN/NOTIFY:

```
import psycopg2
import select

conn = psycopg2.connect(...)
conn.set_isolation_level(psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT)
cursor = conn.cursor()

cursor.execute("LISTEN user_changes")

print("Waiting for notifications...")
while True:
    # Wait for notification (with timeout)
    if select.select([conn], [], [], 5) == ([], [], []):
        print("Timeout")
    else:
        conn.poll()
        while conn.notifies:
            notify = conn.notifies.pop(0)
            print(f"Channel: {notify.channel}, Payload: {notify.payload}")

            # Process notification
            if notify.payload.startswith('new_user:'):
                username = notify.payload.split(':')[1]
                print(f"New user registered: {username}")
```

Use Cases: - Real-time Dashboards (statt Polling) - Event-Driven Architectures - Change Data Capture (CDC) - Notification-Systeme

Extended Query Protocol Optimierungen:

Das Extended Query Protocol unterstützt Prepared Statements und Parameter-Binding für bessere Performance und Sicherheit.

```

cursor = conn.cursor()

cursor.execute("""
    PREPARE get_user_orders AS
    SELECT * FROM orders
    WHERE user_id = $1 AND created_at > $2
""")

cursor.execute("EXECUTE get_user_orders(%s, %s)", (123, '2026-01-01'))
results = cursor.fetchall()

cursor.execute("EXECUTE get_user_orders(%s, %s)", (456, '2025-12-01'))
results = cursor.fetchall()

```

Vorteile: -  SQL Injection Prevention (automatisches Escaping) -  Query Plan Caching (keine Re-Compilation) -  Reduzierter Netzwerk-Overhead (nur Parameter übertragen) -  Type-Safety (Parameter-Typen geprüft)

31.9A.2 Performance-Verbesserungen

Binary Format Support für Vektoren:

Native Binärübertragung von Vektoren für LLM-Embeddings und Vector Search.

```

import struct
import psycopg2
from psycopg2.extras import register_vector

register_vector(conn)

embedding = [0.1, 0.2, 0.3, ..., 0.768] # 768 dimensions
cursor.execute(
    "INSERT INTO documents (content, embedding) VALUES (%s, %s)",
    ("Sample text", embedding)
)

cursor.execute(
    "SELECT id, content, embedding FROM documents WHERE id = %s",
    (123,)
)
row = cursor.fetchone()
vector = row[2] # Direkter Python Array

```

Performance-Vergleich:

Format	Transfer Size	Parse Time	Use Case
JSON	15.2 KB	450 µs	Kompatibilität
Text	12.8 KB	320 µs	Debugging
Binary	3.1 KB	45 µs	Production

Einsparung: 80% weniger Netzwerk-Traffic, 90% schnelleres Parsing

Pipeline Mode für Batch-Queries:

Sende mehrere Queries ohne auf Antworten zu warten (Request Pipelining).

```
from psycopg2 import sql
from psycopg2.extras import execute_batch

cursor = conn.cursor()

data = [
    (1, 'Alice', 'alice@example.com'),
    (2, 'Bob', 'bob@example.com'),
    (3, 'Charlie', 'charlie@example.com'),
    # ... 10,000 rows
]

execute_batch(
    cursor,
    "INSERT INTO users (id, name, email) VALUES (%s, %s, %s)",
    data,
    page_size=1000 # Send 1000 rows per batch
)
conn.commit()
```

Performance:

Method	Throughput	Latenz	Network RTTs
Sequential	5K rows/s	0.2ms	10,000
Batch (100)	35K rows/s	0.03ms	100
Pipeline (1000)	85K rows/s	0.012ms	10

Prepared Statement Caching:

Automatisches Caching von Prepared Statements für häufige Queries.

```

postgresql_wire_protocol:
  prepared_statements:
    cache_enabled: true
    max_cached_statements: 1000
    cache_ttl_seconds: 3600
    auto_prepare_threshold: 5 # Auto-prepare nach 5 Executes

```

Monitoring:

```

-- Cache-Statistiken
SELECT * FROM pg_stat_statements_cache;

```

Output:

statement_id	query	executions	cache_hits	cache_misses
stmt_001	SELECT * FROM users	12450	12445	5
stmt_002	INSERT INTO orders	8420	8415	5

Performance-Gewinn: - Cache Hit: 0.05ms (nur Parameter-Binding) - Cache Miss: 2.5ms (Parse + Plan + Execute) - **50x schneller** bei Cache Hit

31.9A.3 Neue Datentyp-Mappings

LLM Embedding Vectors → PostgreSQL Vector Type:

Native Unterstützung für pgvector-kompatible Vektoren.

```
-- Vector Column erstellen
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding VECTOR(768) -- 768-dimensionaler Vektor
);

-- Vector Index erstellen (HNSW)
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops);

-- Vector Search Query
SELECT id, content, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM documents
ORDER BY distance
LIMIT 10;
```

Python-Client mit Vector-Support:

```
from psycopg2.extras import register_vector

register_vector(conn)

embedding = model.encode("Sample text") # numpy array [768]
cursor.execute(
    "INSERT INTO documents (content, embedding) VALUES (%s, %s)",
    ("Sample text", embedding.tolist())
)

query_embedding = model.encode("Search query")
cursor.execute("""
    SELECT id, content, embedding <=> %s AS distance
    FROM documents
    ORDER BY distance
    LIMIT 10
""", (query_embedding.tolist(),))

results = cursor.fetchall()
for row in results:
    print(f"ID: {row[0]}, Distance: {row[2]}")
```

JSON/JSONB für Document Collections:

Optimierte Unterstützung für JSON-Datentypen kompatibel mit PostgreSQL.

```
-- JSONB Column (binäre JSON-Speicherung)
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    metadata JSONB
);

-- JSONB Index für schnelle Queries
CREATE INDEX ON products USING gin (metadata);

-- JSONB Queries
SELECT * FROM products
WHERE metadata @> '{"category": "electronics"}';

SELECT * FROM products
WHERE metadata->>'brand' = 'Apple';

SELECT * FROM products
WHERE metadata->'specs'->>'ram' = '16GB';
```

Python-Client mit JSONB:

```
import json

metadata = {
    "category": "electronics",
    "brand": "Apple",
    "specs": {"ram": "16GB", "storage": "512GB"}
}

cursor.execute(
    "INSERT INTO products (name, metadata) VALUES (%s, %s)",
    ("MacBook Pro", json.dumps(metadata))
)

cursor.execute("""
    SELECT name, metadata
    FROM products
    WHERE metadata @> %s
""", (json.dumps({"category": "electronics"}),))

results = cursor.fetchall()
for row in results:
    print(f"Product: {row[0]}, Metadata: {row[1]}")
```

Temporal Types für Timeseries:

Native Unterstützung für PostgreSQL-Zeittypen (TIMESTAMP, TIMESTAMPTZ, INTERVAL).

```
-- Timeseries Table mit TIMESTAMPTZ
CREATE TABLE metrics (
    id SERIAL PRIMARY KEY,
    metric_name TEXT,
    value DOUBLE PRECISION,
    recorded_at TIMESTAMPTZ DEFAULT NOW()
);

-- Time-Range Queries
SELECT * FROM metrics
WHERE recorded_at BETWEEN '2026-01-01' AND '2026-01-31';

-- Time-Bucketing (Aggregation)
SELECT
    DATE_TRUNC('hour', recorded_at) AS hour,
    AVG(value) AS avg_value
FROM metrics
WHERE metric_name = 'cpu_usage'
GROUP BY hour
ORDER BY hour;

-- INTERVAL Calculations
SELECT * FROM metrics
WHERE recorded_at > NOW() - INTERVAL '7 days';
```

Python-Client mit Temporal Types:

```

from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc)
cursor.execute(
    "INSERT INTO metrics (metric_name, value, recorded_at) VALUES (%s, %s, %s)",
    ("cpu_usage", 75.5, now)
)

cursor.execute("""
    SELECT metric_name, value, recorded_at
    FROM metrics
    WHERE recorded_at > NOW() - INTERVAL '1 hour'
    ORDER BY recorded_at DESC
""")

results = cursor.fetchall()
for row in results:
    print(f"Metric: {row[0]}, Value: {row[1]}, Time: {row[2]}")

```

31.9A.4 Client-Library-Kompatibilität

Getestete Libraries:

Language	Library	Version	Support	Notes
Python	psycopg2	2.9+	✓ Full	Empfohlen
Python	asyncpg	0.29+	✓ Full	Async Support
Node.js	pg	8.11+	✓ Full	-
Java	PostgreSQL JDBC	42.7+	✓ Full	-
Go	pgx	5.5+	✓ Full	-
Rust	tokio-postgres	0.7+	✓ Full	Async
C#	Npgsql	8.0+	✓ Full	.NET
Ruby	pg	1.5+	✓ Full	-
PHP	pdo_pgsql	8.2+	✓ Full	-

Kompatibilitäts-Features:

```

conn = psycopg2.connect(
    host='localhost',
    port=5432, # Standard PostgreSQL Port
    dbname='themisdb',
    user='admin',
    password='secret',
    sslmode='require', # SSL Support
    connect_timeout=10,
    application_name='my_app',
    options='-c search_path=public,themis'
)

cursor.execute("""
    SELECT table_name, column_name, data_type
    FROM information_schema.columns
    WHERE table_schema = 'public'
""")

cursor.execute("SELECT * FROM pg_tables")
cursor.execute("SELECT * FROM pg_indexes")
cursor.execute("SELECT * FROM pg_stat_user_tables")

```

31.9A.5 Migration Guide für PostgreSQL-Clients

Schritt 1: Connection String anpassen

```

conn = psycopg2.connect(
    "postgresql://user:pass@postgres.example.com:5432/mydb"
)

conn = psycopg2.connect(
    "postgresql://user:pass@themisdb.example.com:5432/mydb"
)

```

Schritt 2: Queries überprüfen

Die meisten PostgreSQL-Queries funktionieren unverändert:

```
-- Standard SQL: funktioniert
SELECT * FROM users WHERE age > 25;

-- PostgreSQL-spezifisch: funktioniert
SELECT * FROM users WHERE email ILIKE '%@gmail.com';

-- PostgreSQL JSON: funktioniert
SELECT * FROM products WHERE metadata->>'category' = 'electronics';
```

Schritt 3: Feature-Nutzung

Neue ThemisDB-Features können optional genutzt werden:

```
cursor.execute("""
    SELECT * FROM documents
    WHERE VECTOR_COSINE_DISTANCE(embedding, %s) < 0.5
""", (query_embedding,))
```

Schritt 4: Performance-Tuning

```
cursor.execute("SET binary_format = ON")

cursor.execute("SET prepared_statement_cache = ON")

cursor.execute("SET pipeline_mode = ON")
```

31.9A.6 Performance-Benchmarks

pgbench Kompatibilität:

ThemisDB unterstützt den Standard-PostgreSQL-Benchmark `pgbench`.

```
pgbench -i -s 100 themisdb

pgbench -c 50 -j 4 -T 60 -S themisdb

pgbench -c 50 -j 4 -T 60 themisdb
```

Benchmark-Ergebnisse (vs. PostgreSQL 16):

Workload	PostgreSQL 16	ThemisDB	Verbesserung
Simple Select	145K TPS	168K TPS	+16%
Select with Join	82K TPS	95K TPS	+16%
Insert	45K TPS	52K TPS	+16%
Update	38K TPS	44K TPS	+16%
COPY (Bulk)	185K rows/s	250K rows/s	+35%
Vector Search	-	12K QPS	Native

Latenz-Vergleich (p95):

Operation	PostgreSQL	ThemisDB	Verbesserung
Simple Query	2.5ms	2.1ms	-16%
Prepared Stmt (cached)	0.8ms	0.05ms	-94%
JSONB Query	5.2ms	4.8ms	-8%
Vector Search	-	3.5ms	Native

31.10 Monitoring & Observability

31.10.1 Protocol-Level Metrics

Prometheus metrics by protocol:

```

http_requests_total{protocol="h3", method="POST"} # HTTP/3 requests
http_requests_total{protocol="h2", method="GET"}  # HTTP/2 requests
http_requests_total{protocol="h1", method="POST"} # HTTP/1.1 requests

http_request_duration_seconds{protocol="h3"}      # Latency by protocol

quic_connection_count                             # Active QUIC connections
quic_handshakes_total                             # QUIC handshake count
quic_packets_lost_total                           # Packet loss

websocket_connections_active                       # Active WebSocket conns
websocket_messages_total{direction="inbound"}     # Messages received
websocket_messages_total{direction="outbound"}    # Messages sent

```


31.10.2 Client-Side Metrics

```
import time
from prometheus_client import Counter, Histogram

requests = Counter('themis_requests_total', 'Requests by protocol',
                  ['protocol', 'method'])
latencies = Histogram('themis_request_duration_seconds',
                     'Request latency', ['protocol'])

def execute_with_metrics(method, path, protocol='h2'):
    start = time.time()
    try:
        response = execute(method, path, protocol=protocol)
        requests.labels(protocol=protocol, method=method).inc()
        latencies.labels(protocol=protocol).observe(time.time() - start)
        return response
    except Exception as e:
        requests.labels(protocol=protocol, method=method).inc() # Count failures too
        raise
```

31.11 Zusammenfassung & Best Practices

Protocol Selection Decision Tree

```
Need bidirectional communication?
├─ YES → WebSockets
│   └─ Real-time updates (changefeed, dashboards)
├─ NO → HTTP/3 (QUIC)
│   └─ If: Client supports UDP
│   └─ If: High latency network (satellite, 4G)
├─ HTTP/2
│   └─ If: HTTP/3 not available
│   └─ If: High concurrency needed
└─ HTTP/1.1
    └─ Legacy clients only
```

Protocol-Specific Best Practices

HTTP/2/3: - Use single persistent connection - Enable header compression - Avoid domain sharding - Implement request prioritization - Monitor concurrent stream usage

WebSocket: - Implement reconnection logic - Send heartbeat/ping every 30-60s - Handle backpressure (don't buffer infinite) - Use sub-protocols for versioning - Limit message size (1-100MB based on use case)

SSE: - Keep-alive: 15-30 second intervals - Reconnect backoff: exponential (1s → 30s) - Structure events with IDs (for resume) - Monitor client disconnection rate

MCP: - Cache query results per session (5 min TTL) - Whitelist tools per role (security) - Implement query timeout (30s for LLM UX) - Log all tool invocations (audit trail)

Kapitel 31 von 33 | Teil V: Protocols & Integration | ~8.500 Wörter (+2000 neu)

Kapitel 42: Kapitel 32: AQL v1.3.1 OOP-Implementierung

"Die Sprache ist das API – ihre Implementierung bestimmt die Fähigkeiten der Datenbank."

32.1 Executive Summary

AQL v1.3.1 erweitert die Sprache um objektorientierte Konstrukte (47 neue Keywords, 360 Funktionen bleiben konstant). Dieses Kapitel beschreibt, wie die Erweiterungen in ThemisDB umgesetzt werden – vom Lexer über den Parser bis zur Ausführungsebene.

Ziele: - Lexer/Tokenizer: 47 neue Keywords, Tokens für Klassen/Methoden/Namespace - Parser/AST: Neue Node-Typen für `CLASS`, `FUNCTION`, `METHOD`, `NAMESPACE` - Type-System: Statisches Type-Checking, Inference, generische Typen - Namespace-Resolution: Symbol-Tables, Visibility, Imports - Runtime: Objekt-Instanziierung, Methodenaufrufe, Visibility-Checks - Tests: Parser-Golden-Files, Type-Checker-Property-Tests

32.2 Betroffene Codebereiche

Datei	Zweck	Änderung
<code>/src/query/aql_parser.cpp</code>	Lexer/Parser	47 neue Keywords, AST Nodes
<code>/src/query/aql_translator.cpp</code>	AST → Execution Plan	OOP Nodes übersetzen
<code>/include/query/aql_parser.h</code>	Parser-Interface	Neue Strukturen/Enums
<code>/src/query/aql_type_checker.cpp</code>	Type-System	Neue Typregeln, Inference
<code>/src/query/aql_namespace_resolver.cpp</code>	Symbol-Tables	Scopes, Visibility
<code>/src/aql/llm_aql_handler.cpp</code>	LLM-Befehle	Vision/OOP-Kommandos

Neue Dateien: - `/include/query/aql_type_checker.h`, `/src/query/aql_type_checker.cpp`
 - `/include/query/aql_namespace_resolver.h`, `/src/query/aql_namespace_resolver.cpp`
 - `/include/aql/aql_vision_handler.h`, `/src/aql/aql_vision_handler.cpp`

32.3 Lexer/Tokenizer (Woche 1-2)

32.3.1 TokenType-Erweiterung

- Namespace: `NAMESPACE` , `IMPORT`
- Typen: `STRING_TYPE` , `INT_TYPE` , `FLOAT_TYPE` , `BOOL_TYPE` , `ANY_TYPE`
- Klassen/Funktionen: `FUNCTION` , `CLASS` , `PUBLIC` , `PRIVATE` , `CONSTRUCTOR` , `METHOD` , `NEW`
- Referenzen/Vererbung: `THIS` , `SELF` , `EXTENDS`
- Control Flow: `IF` , `THEN` , `ELSE` , `ELSEIF` , `ENDIF` , `TRY` , `CATCH` , `THROW` , `CASE` , `END`
- Pattern Matching: `MATCH` , `WHEN`

Lexer-Checks: - Case-insensitive Keywords - String-Literale unverändert lassen - Fehlerhafte Kombinationen mit präzisen Fehlermeldungen (Zeile/Spalte)

32.3.2 Keyword-Detection

```
// Keyword-Detection mit unordered_set (119 Keywords)
bool isKeyword(std::string_view tok) {
    static const std::unordered_set<std::string_view> kw = {
        "FOR", "CLASS", "METHOD", "NAMESPACE", /* ... 115 weitere */
    };
    return kw.contains(tok);
}
```

32.4 Parser & AST (Woche 2-4)

32.4.1 Neue AST-Nodes

- `AstClassDecl { name, base?, members[] }`
- `AstMethodDecl { name, params[], return_type, visibility, is_static }`
- `AstFunctionDecl { name, params[], return_type }`
- `AstNamespaceDecl { name, imports[], body[] }`
- `AstNewExpr { type_name, args[] }`
- `AstMemberAccess { object, member }`

32.4.2 Grammatik (Auszug)

```

translation_unit
: namespace_decl* class_decl* function_decl* statement*
;

class_decl
: CLASS IDENT (EXTENDS IDENT)? LBRACE class_member* RBRACE
;

class_member
: visibility? (method_decl | property_decl | constructor_decl)
;

method_decl
: METHOD IDENT LPAREN param_list? RPAREN (ARROW type)? block
;

```

32.4.3 Fehlerbehandlung

- Präzise Fehler: `Unexpected token 'END' at line 42, expected 'WHEN'`
- Recovery: Synchronisation bei `;`, `END`, `}`
- AST-Pragmas für Fehlertoleranz, damit IDE-Features funktionieren

32.5 Type-System (Woche 4-6)

32.5.1 Typen & Inference

Typ	Beschreibung
<code>Any</code>	Fallback/Unbekannt
<code>String</code> , <code>Int</code> , <code>Float</code> , <code>Bool</code>	Primitive
<code>Class<T></code>	Klasseninstanzen
<code>Vector<T></code>	Generische Container
<code>Func<Args, Ret></code>	Funktions-Typ

Inference-Regeln: - Assignment: `lhs = rhs` → `type(lhs) = type(rhs)` - Member Access: `obj.m` → Lookup in Symbol-Table, Sichtbarkeit prüfen - Calls: `call(f, args)` → Parameteranzahl, Typkompatibilität, Varargs

32.5.2 Visibility & Access Control

- `public` : überall sichtbar
- `private` : nur innerhalb derselben Klasse
- `protected` (optional): Klasse + Subklassen

32.5.3 Exceptions & TRY/CATCH

- Typ `Exception` als Basisklasse
- `THROW expr` propagiert Typ `Exception`
- `TRY { ... } CATCH (e: Exception) { ... }`

32.6 Namespace-Resolution (Woche 6-7)

32.6.1 Symbol-Tables

- Hierarchische Tabellen pro Scope
- Eintrag: `{name, kind (var|func|class|namespace), type, visibility}`
- Shadowing nach Java/C#-Modell (innere Scopes haben Vorrang)

32.6.2 Imports

```

NAMESPACE themis.app
IMPORT themis.llm.*
IMPORT themis.vision.embed

```

- Wildcards (`*`) nur auf Namespace-Level erlaubt
- Konfliktauflösung: explizite Qualifizierung gewinnt (`llm.stats`)

32.6.3 Resolution-Algorithmus

1. Prüfe lokalen Scope
2. Prüfe Namespace-Imports
3. Prüfe globale Symbole
4. Fehler, wenn nicht gefunden → `Unknown symbol 'X'`

32.7 Runtime & Execution (Woche 7-10)

32.7.1 New/Constructor

```
Value evalNew(const AstNewExpr& node, Context& ctx) {
    auto* cls = ctx.lookupClass(node.type_name);
    return cls->invokeConstructor(node.args); // Alloc + Init
}
```

32.7.2 Methodenaufrufe

- Dynamische Dispatch-Tabelle je Klasse
- Inline-Cache (optional) für Hot-Paths
- Visibility-Checks zur Laufzeit (Guard)

32.7.3 Exceptions

- Stack-Unwinding mit RAI-Gurten
- `defer` / `finally` -Support über Scope Guards

32.8 Vision & LLM-Befehle (Add-On)

Neue Kommandos im LLM-Handler: - `VISION.EMBED(image|text, model)` → Vektor - `VISION.DESCRIBE(image)` → Text - `VISION.COMPARE(imgA, imgB)` → Similarity - Safety: Max Input Size, MIME-Checks, Model-Whitelist

32.9 Test-Strategie

32.9.1 Parser-Golden-Files

- Input → erwarteter AST (JSON) → Diffen in CI
- Beispiele: Klassen mit Vererbung, Methoden mit Default-Args, Namespaces

32.9.2 Property-Tests (Type Checker)

- QuickCheck/rapidcheck: zufällige Ausdrucksbäume generieren
- Invariante: Keine illegalen Typen nach erfolgreichem Check
- Fuzzer: ungültige Tokens → Parser muss robust fehlschlagen

32.9.3 Runtime-Tests

- Konstruktor/Methoden-Dispatch

- Visibility-Fehler (private/public)
- Exception-Pfade und Unwinding

32.10 Migrations-Plan

Phase	Dauer	Ergebnis
Lexer/Parser	2 Wochen	Keywords + AST-Nodes
Type-System	2-3 Wochen	Checker + Inference
Namespace	1 Woche	Symbol-Tables, Imports
Runtime	2-3 Wochen	Dispatch, Exceptions
Tests/Hardening	2 Wochen	Golden-Files, Property-Tests

Risiken & Mitigation: - Parser-Regressions → Golden-Files in CI - Performance-Einbruch → Inline-Caches, Hot-Path-Profiler - Kompatibilität → Feature-Flag `enable_oop` bis stabil

32.11 Checkliste Go-Live

- [] `enable_oop` Feature-Flag default ON
- [] Parser-Tests (100% der neuen Grammatik) grün
- [] Type-Checker-Property-Tests 10k Runs
- [] Runtime-Benchmarks: <10% Overhead ggü. v1.3.0
- [] LLM/Vision-Commands whitelisted, Input-Limits gesetzt
- [] Docs aktualisiert (AQL Referenz, Examples)

32.10 Implementierungs-Timeline

Phase 1: Lexer & Parser (Wochen 1-4)

Woche 1-2: Lexer - Tag 1-2: Keyword-Set erweitern (47 neue) - Tag 3-4: Token-Recognition Tests - Tag 5-6: Error Messages verbessern - Tag 7-8: Performance Benchmarks (Lexer sollte <5% langsamer sein)

Woche 3-4: Parser - Tag 1-4: AST Node-Typen implementieren - Tag 5-7: Grammatik-Regeln für CLASS/METHOD/NAMESPACE - Tag 8-10: Golden-File Tests (10+ Parser-Test-Cases)

Phase 2: Type System (Wochen 5-7)

Woche 5-6: Type Checker - Tag 1-3: Typ-Inference-Algorithmus - Tag 4-6: Visitor Pattern für AST-Traversal - Tag 7-10: Generics Support (Vector, Map)

Woche 7: Validation - Tag 1-3: Type Error Messages - Tag 4-6: Property-Based Tests - Tag 7: Integration mit Parser

Phase 3: Namespace Resolution (Wochen 8-9)

- Tag 1-3: Symbol-Table Implementierung
- Tag 4-6: Import-Resolution
- Tag 7-9: Shadowing & Visibility
- Tag 10-12: Test-Suite (50+ Test-Cases)

Phase 4: Runtime (Wochen 10-14)

Woche 10-11: Object Model - Tag 1-5: Class/Instance Representation - Tag 6-10: Constructor/Method Dispatch

Woche 12-13: Execution - Tag 1-5: NEW Expression Execution - Tag 6-10: Member Access Resolution - Tag 11-14: Exception Handling

Woche 14: Integration - Tag 1-3: End-to-End Tests - Tag 4-5: Performance Profiling - Tag 6-7: Documentation

Phase 5: Vision & Rollout (Wochen 15-16)

- Woche 15: Vision.EMBED/DESCRIBE/COMPARE
 - Woche 16: Production Deployment, Monitoring
-

32.11 Code-Beispiele

32.11.1 Lexer Test

```
// test_lexer_oop.cpp
TEST(AQLLexer, RecognizesClassKeyword) {
    std::string input = "CLASS User { }";
    Lexer lexer(input);

    Token t1 = lexer.next();
    EXPECT_EQ(t1.type, TokenType::CLASS);

    Token t2 = lexer.next();
    EXPECT_EQ(t2.type, TokenType::IDENTIFIER);
    EXPECT_EQ(t2.value, "User");

    Token t3 = lexer.next();
    EXPECT_EQ(t3.type, TokenType::LBRACE);
}

TEST(AQLLexer, KeywordCaseInsensitive) {
    Lexer lexer("class CLASS Class");

    for (int i = 0; i < 3; i++) {
        Token t = lexer.next();
        EXPECT_EQ(t.type, TokenType::CLASS);
    }
}
```

32.11.2 Parser Test mit Golden Files

```
// test_parser_golden.cpp
TEST(AQLParser, ClassDeclarationGolden) {
    std::string input = R"(
        CLASS User {
            PRIVATE email: STRING
            PUBLIC name: STRING

            CONSTRUCTOR(email, name) {
                THIS.email = email
                THIS.name = name
            }

            PUBLIC METHOD greet() {
                RETURN CONCAT("Hello, ", THIS.name)
            }
        }
    )";

    Parser parser(input);
    AstNode* ast = parser.parse();

    // Serialize AST to JSON
    std::string ast_json = serializeAST(ast);

    // Compare with Golden File
    std::string expected = readFile("golden/class_user_ast.json");
    EXPECT_EQ(ast_json, expected);
}
```

32.11.3 Type Checker Unit Test

```
// test_type_checker.cpp
TEST(TypeChecker, InfersReturnType) {
    std::string code = R"(
        FUNCTION add(a: INT, b: INT) {
            RETURN a + b // Should infer INT
        }
    )";

    TypeChecker checker;
    FunctionType func_type = checker.inferFunctionType(code);

    EXPECT_EQ(func_type.return_type, Type::INT);
    EXPECT_EQ(func_type.param_types.size(), 2);
    EXPECT_EQ(func_type.param_types[0], Type::INT);
}

TEST(TypeChecker, DetectsMismatch) {
    std::string code = R"(
        FUNCTION broken(x: STRING) -> INT {
            RETURN x // ✗ Type Error: expected INT, got STRING
        }
    )";

    TypeChecker checker;
    EXPECT_THROW(
        checker.check(code),
        TypeMismatchException
    );
}
```

32.11.4 Namespace Resolution Test

```
// test_namespace.cpp
TEST(NamespaceResolver, ImportsResolve) {
    std::string code = R"(
        NAMESPACE myapp
        IMPORT themis.llm.*

        FUNCTION analyze(text) {
            RETURN LLM.CHAT(text) // Should resolve to themis.llm.LLM.CHAT
        }
    )";

    NamespaceResolver resolver;
    SymbolTable symbols = resolver.resolve(code);

    Symbol* llm_chat = symbols.lookup("LLM.CHAT");
    EXPECT_NE(llm_chat, nullptr);
    EXPECT_EQ(llm_chat->fully_qualified_name, "themis.llm.LLM.CHAT");
}
```

32.11.5 Runtime NEW Expression

```
// aql_runtime.cpp: NEW Implementierung
Value AQLRuntime::evalNew(const AstNewExpr& node) {
    // 1. Lookup Class Definition
    ClassDef* cls = symbol_table_.lookupClass(node.type_name);
    if (!cls) {
        throw RuntimeError(fmt::format("Class '{}' not found", node.type_name));
    }

    // 2. Allocate Instance
    ObjectInstance* instance = allocateObject(cls);

    // 3. Find Constructor
    Constructor* ctor = cls->findConstructor(node.args.size());
    if (!ctor) {
        throw RuntimeError(fmt::format(
            "No constructor found for {} args", node.args.size()
        ));
    }

    // 4. Evaluate Arguments
    std::vector<Value> arg_values;
    for (const auto& arg : node.args) {
        arg_values.push_back(eval(arg));
    }

    // 5. Invoke Constructor
    ctor->invoke(instance, arg_values);

    return Value::fromObject(instance);
}
```

32.12 Performance-Auswirkungen

Erwartete Overhead

Operation	Baseline (ohne OOP)	Mit OOP	Overhead
Lexing	100 ms/10k lines	105 ms	+5%
Parsing	250 ms/10k lines	280 ms	+12%
Type Check	N/A	150 ms	Neu
Execution (Simple FOR)	10 ms	10 ms	0%
Execution (Method Call)	N/A	+2 ms	Neu

Optimierungen: - **Inline Caching:** Häufig aufgerufene Methoden cachen - **JIT für Hot Methods:** Bytecode-Kompilierung für >1000 Calls - **Lazy Type Checking:** Nur bei Bedarf (Development-Mode)

Benchmarks

```
// benchmark_oop.cpp
static void BM_MethodCall(benchmark::State& state) {
    std::string code = R"(
        CLASS Counter {
            PRIVATE count: INT

            CONSTRUCTOR() { THIS.count = 0 }
            METHOD increment() { THIS.count = THIS.count + 1 }
        }

        LET c = NEW Counter()
        FOR i IN 1..1000000
            c.increment()
    )";

    for (auto _ : state) {
        AQLRuntime runtime;
        runtime.execute(code);
    }
}
BENCHMARK(BM_MethodCall);

// Ergebnis: ~2.5 µs pro Method Call (inkl. Dispatch)
```

32.13 Migration & Backward Compatibility

Rückwärtskompatibilität

- ☒ Alle AQL v1.3.0 Queries funktionieren unverändert
- ☒ Neue Keywords nur in OOP-Kontext reserviert
- ☒ Legacy-Code benötigt keine Änderungen

Opt-In für OOP Features

```
-- Legacy Mode (Default)
FOR u IN users RETURN u

-- OOP Mode (via Pragma)
#pragma aql_version 1.3.1

CLASS User { ... }
```

Feature Flag

```
aql:
  oop_features_enabled: true # Default: false bis v1.4.0
  strict_type_checking: false # Development: true, Production: false
```

32.14 Zusammenfassung

AQL v1.3.1 führt OOP-Konstrukte ein und erfordert Änderungen auf allen Ebenen der Query-Engine. Die Implementierung gliedert sich in fünf Phasen: Lexer/Parser, Type-System, Namespace-Resolution, Runtime, Tests. Mit klaren Guardrails (Feature-Flag, Tests, Benchmarks) kann die Erweiterung ohne Regressionen ausgerollt werden.

Kapitel 43: Kapitel 33: Best Practices & Design Patterns

"Good code is its own best documentation. As you're about to add a comment, ask yourself, 'How can I improve the code so that this comment isn't needed?'" - Steve McConnell

Überblick

Production-ready ThemisDB-Anwendungen folgen bewährten Patterns für Performance, Sicherheit, und Wartbarkeit. Dieses Kapitel sammelt Battle-tested Best Practices aus realen Deployments.

Was Sie in diesem Kapitel lernen: - Schema-Design Patterns - Query-Optimierung Best Practices - Sicherheits-Härtung - Resilience Patterns - Testing-Strategien - Performance Tuning - Operational Excellence

33.1 Schema-Design Patterns

Pattern 1: Embedded vs. Referenced

Embedded (Denormalisiert):

```
//  Gut für: 1:N Beziehungen mit wenigen Child-Elementen
{
  "_key": "order-123",
  "customer": {
    "name": "Alice",
    "email": "alice@example.com"
  },
  "items": [
    {"product": "Laptop", "quantity": 1, "price": 1200},
    {"product": "Mouse", "quantity": 2, "price": 25}
  ],
  "total": 1250
}

//  Vorteile:
// - Single query (kein JOIN)
// - Atomic updates
// - Keine referentielle Integrität nötig

//  Nachteile:
// - Duplikation (customer-Daten in jedem Order)
// - Update-Anomalien (email ändert sich → alle Orders updaten)
```

Referenced (Normalisiert):

```
// customers Collection
{
  "_key": "alice",
  "name": "Alice",
  "email": "alice@example.com",
  "address": {...}
}

// orders Collection
{
  "_key": "order-123",
  "customer_id": "alice",  // Referenz
  "items": [...],
  "total": 1250
}

// Query mit JOIN
FOR order IN orders
  FILTER order._key == 'order-123'
  LET customer = DOCUMENT(CONCAT('customers/', order.customer_id))
  RETURN MERGE(order, {customer: customer})

//  Vorteile:
// - Keine Duplikation
// - Single source of truth
// - Einfache Updates (nur 1 Dokument)
```

Entscheidungsmatrix:

```

flowchart TD
    Start{Data Modeling} --> Relation{Beziehungstyp?}

    Relation -->|1:1| Embed1[Embedded empfohlen]
    Relation -->|1:N| CheckN{N ist klein?}
    Relation -->|N:M| Ref1[Referenced mit Junction]

    CheckN -->|Ja, N < 100| Embed2[Embedded]
    CheckN -->|Nein, N > 100| Ref2[Referenced]

    Embed1 --> Update{Häufige Updates?}
    Embed2 --> Update

    Update -->|Ja| Ref3[Referenced besser]
    Update -->|Nein| Final1[Embedded OK]

    Ref1 --> Final2[Referenced Pattern]
    Ref2 --> Final2
    Ref3 --> Final2

    Final1 --> Check{Dokumentgröße?}
    Check -->|< 16MB| OK[Schema OK]
    Check -->|> 16MB| Split[Split in Chunks]

    Final2 --> Index[Index auf Foreign Keys]
    Index --> OK

    style Embed1 fill:#51cf66
    style Embed2 fill:#51cf66
    style Final1 fill:#51cf66
    style Final2 fill:#4dabf7
    style OK fill:#40c057

```

Abb. 33.1: Best-Practices-Decision-Tree

Entscheidungsmatrix: | Use Case | Pattern | Begründung | |-----|-----|-----| | Blog: Post + Comments (1:N, viele) | Referenced | Comments wachsen unbegrenzt | | Order + Items (1:N, <20) | Embedded | Items sind fix nach Order | | User + Preferences (1:1) | Embedded | Preferences immer mit User geladen | | Product + Categories (M:N) | Referenced | Viele-zu-Viele → Graph-Edges |

33.2 Query-Optimierung Patterns

Pattern 2: Early Filtering

```
-- ✗ SCHLECHT: Filter nach JOIN
FOR order IN orders
  LET customer = DOCUMENT(order.customer_id)
  FILTER customer.country == 'DE'  // Zu spät!
  RETURN order

-- ✓ GUT: Filter vor JOIN
FOR order IN orders
  LET customer = DOCUMENT(order.customer_id)
  FILTER customer != null && customer.country == 'DE'
  RETURN order

-- ✓ OPTIMAL: Denormalisierung für häufige Filters
{
  "_key": "order-123",
  "customer_id": "alice",
  "customer_country": "DE",  // Denormalisiert für schnellen Filter
  ...
}

FOR order IN orders
  FILTER order.customer_country == 'DE'  // Index-optimiert
  RETURN order
```

Pattern 3: Projection (Select nur benötigte Felder)

```
-- ✗ SCHLECHT: Alle Felder laden
FOR user IN users
  RETURN user  // Lädt alle Felder (1 MB/User!)

-- ✓ GUT: Nur benötigte Felder
FOR user IN users
  RETURN {
    id: user._id,
    name: user.name,
    email: user.email
  }  // Nur 100 Bytes/User

-- Performance-Gewinn: 10x schneller bei großen Dokumenten
```

Pattern 4: Pagination mit Cursor

```
-- ❌ SCHLECHT: OFFSET/LIMIT (langsam bei großen Offsets)
FOR doc IN collection
  SORT doc.created_at DESC
  LIMIT 10000, 100 // Skip 10k Docs → langsam!
  RETURN doc

-- ✅ GUT: Cursor-based Pagination
FOR doc IN collection
  FILTER doc.created_at < @last_seen_timestamp
  SORT doc.created_at DESC
  LIMIT 100
  RETURN doc

// Client speichert letzten Timestamp:
last_seen = results[99].created_at
next_page = query(last_seen_timestamp=last_seen)
```

33.3 Sicherheits-Härtung

Pattern 5: Input Validation

```
-- ✗ SCHLECHT: Unvalidierte User-Eingabe
LET user_input = @email
FOR u IN users
  FILTER u.email == user_input  // Injection-Risiko bei String-Concat!
  RETURN u

-- ✓ GUT: Parametrisierte Queries
FOR u IN users
  FILTER u.email == @email  // Safe: Parameter-Binding
  RETURN u

-- ✓ BESSER: Zusätzliche Validation
FUNCTION validate_email(email) {
  IF !REGEX_TEST(email, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$") THEN
    THROW ERROR("Invalid email format")
  END
  RETURN email
}

LET validated = validate_email(@email)
FOR u IN users
  FILTER u.email == validated
  RETURN u
```

Pattern 6: Least Privilege Access

```
-- ❌ SCHLECHT: Root-User für Application
const client = new ThemisClient({
  user: 'root', // ❌ Zu viele Rechte!
  password: 'root123'
})

-- ✅ GUT: Dedizierter App-User mit minimalen Rechten
CREATE USER app_user WITH PASSWORD 'secure_pass'
GRANT READ ON DATABASE mydb TO app_user
GRANT WRITE ON COLLECTION orders TO app_user
GRANT WRITE ON COLLECTION users TO app_user

const client = new ThemisClient({
  user: 'app_user',
  password: process.env.THEMIS_PASSWORD // ✅ Aus Env-Var
})
```

Pattern 7: Data Encryption at Rest

```
storage:
  encryption:
    enabled: true
    algorithm: AES-256-GCM
    key_provider: aws-kms
    key_id: arn:aws:kms:eu-central-1:123456789:key/abc-def

# Zusätzlich: Field-Level Encryption für PII
field_encryption:
  - collection: users
    fields: [ssn, credit_card, password_hash]
```


33.4 Resilience Patterns

Pattern 8: Circuit Breaker

```
class CircuitBreaker:
    def __init__(self, threshold=5, timeout=60):
        self.failures = 0
        self.threshold = threshold
        self.timeout = timeout
        self.open_until = None

    def call(self, func, *args, **kwargs):
        # Circuit Open → Fail Fast
        if self.open_until and time.time() < self.open_until:
            raise CircuitBreakerOpen("Service unavailable")

        try:
            result = func(*args, **kwargs)
            self.failures = 0 # Reset bei Erfolg
            return result
        except Exception as e:
            self.failures += 1

            if self.failures >= self.threshold:
                self.open_until = time.time() + self.timeout
                print(f"Circuit opened for {self.timeout}s")

            raise

db_breaker = CircuitBreaker(threshold=3, timeout=30)

def query_themis(aql):
    return db_breaker.call(client.query, aql)
```

Pattern 9: Retry with Exponential Backoff

```
def retry_with_backoff(func, max_retries=3, base_delay=1):
    """Retry mit exponential backoff"""
    for attempt in range(max_retries):
        try:
            return func()
        except (NetworkError, TimeoutError) as e:
            if attempt == max_retries - 1:
                raise # Letzter Versuch → propagieren

            delay = base_delay * (2 ** attempt) # 1s, 2s, 4s
            jitter = random.uniform(0, 0.3) * delay # ±30% Jitter
            time.sleep(delay + jitter)

result = retry_with_backoff(
    lambda: client.query("FOR u IN users RETURN u"),
    max_retries=5,
    base_delay=0.5
)
```

Pattern 10: Bulkhead Pattern (Resource Isolation)

```
critical_pool = ThemisConnectionPool(
    max_connections=50,
    priority='high'
)

background_pool = ThemisConnectionPool(
    max_connections=10,
    priority='low'
)

def get_user_profile(user_id):
    with critical_pool.get_connection() as conn:
        return conn.query("FOR u IN users FILTER u._id == @id RETURN u",
                           {'id': user_id})

def generate_analytics_report():
    with background_pool.get_connection() as conn:
        return conn.query("FOR order IN orders COLLECT ...")
```

33.5 Testing Patterns

Pattern 11: Golden File Testing

```
def test_user_query_golden():
    """Query-Ergebnis mit Golden File vergleichen"""

    # Setup Test-Daten
    setup_fixture('users_test.json')

    # Query ausführen
    result = client.query("""
        FOR u IN users
        FILTER u.age > 25
        SORT u.name
        RETURN {name: u.name, age: u.age}
    """)

    # Mit Golden File vergleichen
    expected = load_json('golden/user_query_expected.json')
    assert result == expected, f"Query result mismatch!"
```

Pattern 12: Property-Based Testing

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers(min_value=0, max_value=100), min_size=1, max_size=1000))
def test_aggregation_properties(numbers):
    """Property: SUM sollte immer gleich Python sum() sein"""

    # Insert Test-Daten
    for n in numbers:
        client.insert('numbers', {'value': n})

    # AQL Aggregation
    aql_sum = client.query("RETURN SUM(n.value FOR n IN numbers)")[0]

    # Python Referenz
    python_sum = sum(numbers)

    assert aql_sum == python_sum, f"Aggregation mismatch: {aql_sum} != {python_sum}"
```

33.6 Performance Tuning

Pattern 13: Connection Pooling

```
def handle_request():
    client = ThemisClient('http://localhost:8529') # Langsam!
    result = client.query(...)
    client.close()
    return result

from themis.pool import ConnectionPool

pool = ConnectionPool(
    url='http://localhost:8529',
    max_connections=100,
    min_connections=10,
    connection_timeout=5
)

def handle_request():
    with pool.get_connection() as client:
        return client.query(...)
```

Pattern 14: Query Caching

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=1000)
def cached_query(query_hash, params_hash):
    """Cache für idempotente Queries"""
    query = QUERY_CACHE[query_hash]
    params = PARAMS_CACHE[params_hash]
    return client.query(query, params)

def get_user_stats(user_id):
    query = "FOR u IN users FILTER u._id == @id RETURN u.stats"
    query_hash = hashlib.md5(query.encode()).hexdigest()
    params_hash = hashlib.md5(str(user_id).encode()).hexdigest()

    return cached_query(query_hash, params_hash)

def update_user(user_id, data):
    client.update(f'users/{user_id}', data)
    cached_query.cache_clear() # Invalidate
```

Pattern 15: Batch Operations

```
-- ✗ SCHLECHT: N einzelne Inserts
FOR i IN 1..10000
  INSERT {value: i} INTO collection // 10k Roundtrips!

-- ✓ GUT: Batch-Insert
LET docs = (FOR i IN 1..10000 RETURN {value: i})
FOR doc IN docs
  INSERT doc INTO collection // 1 Roundtrip

-- Python Equivalent:
docs = [{'value': i} for i in range(10000)]
client.query("FOR doc IN @docs INSERT doc INTO collection", {'docs': docs})
```

33.7 Operational Patterns

Pattern 16: Health Check Endpoint

```
// server.js: Express Health Check
app.get('/health', async (req, res) => {
  const health = {
    status: 'healthy',
    timestamp: new Date().toISOString(),
    checks: {}
  };

  try {
    // Database connectivity
    const start = Date.now();
    await themisClient.query('RETURN 1');
    health.checks.database = {
      status: 'ok',
      latency_ms: Date.now() - start
    };
  } catch (err) {
    health.status = 'unhealthy';
    health.checks.database = {
      status: 'error',
      error: err.message
    };
  }

  // Memory check
  const memUsage = process.memoryUsage();
  health.checks.memory = {
    status: memUsage.heapUsed < 0.9 * memUsage.heapTotal ? 'ok' : 'warning',
    heap_used_mb: Math.round(memUsage.heapUsed / 1024 / 1024)
  };

  res.status(health.status === 'healthy' ? 200 : 503).json(health);
});
```

Pattern 17: Graceful Shutdown

```
import signal
import sys

class Application:
    def __init__(self):
        self.shutting_down = False
        signal.signal(signal.SIGTERM, self.handle_shutdown)
        signal.signal(signal.SIGINT, self.handle_shutdown)

    def handle_shutdown(self, signum, frame):
        print("Shutdown signal received, gracefully stopping...")
        self.shutting_down = True

        # 1. Stop accepting new requests
        self.stop_accepting_requests()

        # 2. Wait for active requests to complete (max 30s)
        self.wait_for_active_requests(timeout=30)

        # 3. Close DB connections
        themis_pool.close_all()

        # 4. Flush logs
        logging.shutdown()

        sys.exit(0)

app = Application()
```

33.8 Checkliste für Production-Readiness

Pre-Deployment Checklist

- ☒ **Schema & Indizes:**
 - ☐ Alle Indizes erstellt (`CREATE INDEX`)
 - ☐ EXPLAIN für kritische Queries durchgeführt
 - ☐ Composite Indizes für häufige Filter-Kombinationen
 - ☐ TTL-Indizes für automatische Datenlöschung
- ☒ **Performance:**
 - ☐ Connection Pooling aktiviert

- ☐ Query Timeouts konfiguriert
 - ☐ Rate Limiting implementiert
 - ☐ Caching-Strategie definiert
 - ☒ **Sicherheit:**
 - ☐ Encryption at Rest aktiviert
 - ☐ TLS für Client-Verbindungen
 - ☐ Least-Privilege User-Accounts
 - ☐ Input-Validierung in Application
 - ☒ **Resilience:**
 - ☐ Circuit Breaker implementiert
 - ☐ Retry-Logic mit Backoff
 - ☐ Graceful Shutdown Handler
 - ☐ Health Check Endpoint
 - ☒ **Monitoring:**
 - ☐ Prometheus/Grafana Dashboards
 - ☐ Alerting für Critical Metrics
 - ☐ Slow Query Log aktiviert
 - ☐ Error Tracking (Sentry/Datadog)
 - ☒ **Backup & DR:**
 - ☐ Tägliche automatische Backups
 - ☐ Backup-Restore getestet
 - ☐ Multi-Region Replication
 - ☐ Disaster Recovery Runbook
-

33.9 Anti-Patterns (Was zu vermeiden ist)

✗ Anti-Pattern 1: N+1 Queries

```
-- SCHLECHT: 1 Query + N Queries
FOR order IN orders
  LIMIT 100
  LET customer = DOCUMENT(order.customer_id) // N zusätzliche Lookups!
  RETURN {order: order, customer: customer}

-- GUT: Batch Lookup
LET order_ids = (FOR o IN orders LIMIT 100 RETURN o._key)
LET orders = DOCUMENT(orders, order_ids)
LET customer_ids = orders[*].customer_id
LET customers = DOCUMENT(customers, customer_ids)
...
```

✗ Anti-Pattern 2: Unbounded Collections

```
-- SCHLECHT: Embedded Array wächst unbegrenzt
{
  "blog_post_id": "post-1",
  "comments": [...] // Was wenn 10k Comments?
}

-- GUT: Separate Collection mit Referenz
// comments Collection
{
  "post_id": "post-1",
  "comment": "...",
  "author": "alice"
}
```

Anti-Pattern 3: Hardcoded Credentials

```
client = ThemisClient('http://prod-db:8529',
                      username='admin',
                      password='prod123') #  Im Code!

client = ThemisClient(
    os.getenv('THEMIS_URL'),
    username=os.getenv('THEMIS_USER'),
    password=os.getenv('THEMIS_PASSWORD') #  Aus Env
)
```

33.10 Advanced Patterns: Event Sourcing

Event Sourcing Pattern

Concept: Store immutable events, derive state from events.

```
-- Event Log (immutable)
{
  "_id": "events/evt-001",
  "aggregate_id": "account/alice",
  "event_type": "AccountCreated",
  "timestamp": "2025-01-01T10:00:00Z",
  "data": { "name": "Alice", "email": "alice@example.com" }
}

{
  "_id": "events/evt-002",
  "aggregate_id": "account/alice",
  "event_type": "DepositMade",
  "timestamp": "2025-01-01T10:05:00Z",
  "data": { "amount": 1000, "currency": "USD" }
}

{
  "_id": "events/evt-003",
  "aggregate_id": "account/alice",
  "event_type": "WithdrawalMade",
  "timestamp": "2025-01-01T10:10:00Z",
  "data": { "amount": 200, "currency": "USD" }
}

-- Materialized View (derived)
{
  "_id": "account_state/alice",
  "balance": 800,
  "last_event_id": "evt-003",
  "updated_at": "2025-01-01T10:10:00Z"
}
```

Benefits: -  **Audit Trail:** Complete history of all changes -  **Temporal Queries:** "What was balance at 10:08?" -  **Event Replay:** Rebuild state from scratch -  **Debugging:** Trace exact sequence of operations

Implementation:

```

-- Record event
BEGIN
  INSERT event INTO event_log
  UPDATE account_state WITH { balance: new_balance }
COMMIT

-- Replay events (if state corrupted)
LET all_events = (
  FOR event IN event_log
    FILTER event.aggregate_id == @account_id
    SORT event.timestamp ASC
    RETURN event
)

LET final_state = REDUCE event IN all_events
  INTO {balance: 0, last_event: NULL}
  (
    LET update = APPLY_EVENT(acc, event)
    RETURN update
  )

RETURN final_state

```

33.11 CQRS Pattern (Command Query Responsibility Segregation)

Pattern: Separate Reads from Writes

```

Write Model (Command Side)
├─ Receives mutations (CREATE, UPDATE, DELETE)
├─ Validates business rules
├─ Persists to event log
└─ Produces events

Event Bus
└─ Asynchronously publishes events

Read Model (Query Side)
├─ Subscribes to events
├─ Updates read-optimized views
├─ Serves fast queries
└─ Can have different schema than write model

```

Example: E-Commerce Order

```
-- Write Model (Normalized)
{
  "_id": "orders/ord-123",
  "customer_id": "cust-456",
  "status": "shipped",
  "created_at": "2025-01-01T10:00:00Z"
}

-- Read Model (Denormalized for Dashboard)
{
  "_id": "order_view/ord-123",
  "customer_name": "Alice",
  "customer_email": "alice@example.com",
  "total_amount": 1250,
  "item_count": 3,
  "status": "shipped",
  "estimated_delivery": "2025-01-05",
  "updated_at": "2025-01-01T15:00:00Z"
}
```

Implementation:

```

@app.post("/orders")
def create_order(cmd: CreateOrderCommand):
    # Validate
    assert cmd.total > 0
    assert cmd.customer_id in db.customers

    # Write to event log
    event = OrderCreatedEvent(cmd)
    db.insert('event_log', event)

    # Publish to event bus
    event_bus.publish(event)

    return {"order_id": event.order_id}

event_bus.subscribe('OrderCreatedEvent', rebuild_order_view)

def rebuild_order_view(event):
    # Enrich with customer data
    customer = db.get('customers', event.customer_id)

    # Create denormalized view
    view = {
        "order_id": event.order_id,
        "customer_name": customer.name,
        "customer_email": customer.email,
        "total": event.total,
        "status": "created"
    }
    db.insert('order_view', view)

```

33.12 Saga Pattern (Distributed Transactions)

Pattern: Multi-Step Compensating Transactions

Problem: Atomic transaction across 3+ microservices.

Solution: Saga with rollback logic.

Order Processing Saga:**Step 1: Reserve Inventory**

- └ Reserve 10 units
- └ If fail: Abort saga

Step 2: Process Payment

- └ Charge credit card
- └ If fail: Release inventory (compensate Step 1)

Step 3: Ship Order

- └ Create shipment
- └ If fail: Refund payment (compensate Step 2)

Step 4: Send Notification

- └ Send confirmation email
- └ If fail: Just log (no compensation)

Implementation in ThemisDB:

```
-- Saga State Machine
{
  "_id": "sagas/saga-001",
  "order_id": "ord-123",
  "status": "in_progress",
  "steps": [
    { "name": "reserve_inventory", "status": "completed", "compensated": false },
    { "name": "process_payment", "status": "completed", "compensated": false },
    { "name": "ship_order", "status": "failed", "error": "Out of stock" },
    { "name": "send_notification", "status": "pending", "compensated": false }
  ],
  "created_at": "2025-01-01T10:00:00Z"
}

-- On Step 3 failure, execute compensations in reverse
-- Step 2: Refund payment
-- Step 1: Release inventory
-- Update saga status to "rolled_back"
```

33.13 Bulkhead Pattern (Isolation)

Pattern: Isolate Critical Resources

Problem: One slow query brings down entire system.

Solution: Separate resource pools.

Implementation:

```
from concurrent.futures import ThreadPoolExecutor

user_executor = ThreadPoolExecutor(max_workers=20, thread_name_prefix='user_')
admin_executor = ThreadPoolExecutor(max_workers=5, thread_name_prefix='admin_')
bg_executor = ThreadPoolExecutor(max_workers=10, thread_name_prefix='bg_')

if query.is_admin:
    future = admin_executor.submit(execute_query, query)
elif query.is_background:
    future = bg_executor.submit(execute_query, query)
else:
    future = user_executor.submit(execute_query, query)

result = future.result(timeout=30) # 30s for users, 60s for admin
```


33.14 Throttling & Rate Limiting Pattern

Pattern: Control Request Rate

```
class RateLimiter:
    def __init__(self, max_requests_per_second=1000):
        self.max_rps = max_requests_per_second
        self.tokens = max_requests_per_second
        self.last_refill = time.time()
        self.lock = threading.Lock()

    def allow(self):
        with self.lock:
            now = time.time()
            elapsed = now - self.last_refill

            # Refill tokens
            self.tokens = min(
                self.max_rps,
                self.tokens + elapsed * self.max_rps
            )
            self.last_refill = now

            if self.tokens >= 1:
                self.tokens -= 1
                return True
            return False

limiter = RateLimiter(max_requests_per_second=1000)

@app.post("/query")
def execute_query(query: Query):
    if not limiter.allow():
        return {"error": "Rate limit exceeded", "retry_after": 1}

    return execute(query)
```

33.15 Zusammenfassung: Advanced Patterns

Pattern	Problem	Solution
Event Sourcing	Audit trail	Immutable events
CQRS	Read/write scaling	Separate models
Saga	Distributed transactions	Compensating transactions
Bulkhead	Resource contention	Isolated pools
Rate Limiting	Overload protection	Token bucket
Circuit Breaker	Cascading failures	Fail-fast
Retry	Transient failures	Exponential backoff

33.16 Golden Rules Revisited

The 7 Commandments of Database Stewardship:

- 1. Index Everything You Filter**
- Query without index = scan all rows
- EXPLAIN is your friend
- Composite indexes follow query order
- 5. Fail Fast, Recover Faster**
- Circuit breaker pattern
- Exponential backoff on retries
- Self-healing infrastructure
- 9. Test Like Production**
- Load testing with realistic workload
- Chaos engineering for resilience
- Staging identical to production
- 13. Monitor Before You're in Crisis**
- Metrics: CPU, Memory, QPS, Latency
- Logs: All errors, admin actions

16. Traces: Request flow

17. **Automate Everything**

18. Backups without manual intervention

19. Failover without human click

20. Deployments without handoff

21. **Document As You Go**

22. Architecture Decision Records (ADRs)

23. Runbooks for common issues

24. Decisions > Implementation details

25. **Security by Default**

26. Least privilege (not super-user)

27. Encryption (at rest, in transit)

28. Validation (all inputs)

29. Audit (all changes)

Kapitel 33 von 33 | Teil VI: Best Practices & Advanced | ~9.000 Wörter (+2000 neu)

Kapitel 44: Kapitel 34: Query Optimierung & Performance Tuning

"Optimierung ist eine kontinuierliche Reise, nicht ein Ziel. Mit den richtigen Tools können Sie fast jede Query um 10-100x beschleunigen."

Überblick

Query-Performance ist einer der kritischsten Faktoren für den Datenbankenerfolg. Dieses Kapitel zeigt praktische Techniken zur Optimierung von AQL-Queries, vom EXPLAIN-Analyse bis zu fortgeschrittenen Optimierungsmustern.

Was Sie in diesem Kapitel lernen: - EXPLAIN und Query-Profiling - Index-Strategien und Best Practices - Early Filtering und Projection Pushdown - Query-Refactoring für Performance - Aggregation Optimization - Window Function Performance - Batch-Operation Tuning - Query Caching und Memoization

```
graph TB
    Query["SELECT * FROM users<br/>WHERE age greater than 25<br/>AND city = Berlin"] --> Optimizer
    Optimizer --> Plan1["Plan 1:<br/>Index Scan on age<br/>Filter city"]
    Optimizer --> Plan2["Plan 2:<br/>Index Scan on city<br/>Filter age"]
    Optimizer --> Plan3["Plan 3:<br/>Composite Index<br/>age plus city"]

    Plan1 --> Cost1["Cost1[Cost: 1200]"]
    Plan2 --> Cost2["Cost2[Cost: 800]"]
    Plan3 --> Cost3["Cost3[Cost: 150]"]

    Cost1 --> Select1["Select{Select<br/>Best Plan}"]
    Cost2 --> Select2["Select"]
    Cost3 --> Select3["Select"]

    Select1 --> Execute["Execute[Execute Plan 3]"]

    style Plan3 fill:#43e97b
    style Execute fill:#4facfe
```

Abb. 34.0: Query-Plan-Optimierung

34.1 EXPLAIN und Query-Profiling

EXPLAIN Output verstehen

```
-- Grundlegende EXPLAIN Syntax
EXPLAIN FOR doc IN users
  FILTER doc.status == 'active'
  SORT doc.created_at DESC
  LIMIT 10
  RETURN doc

-- Ausgabe-Struktur:
-- ExecutionPlan:
--   Steps:
--   1. CollectionScan (collection="users")
--   2. Filter (condition="doc.status == 'active'")
--   3. Sort (fields=["created_at"])
--   4. Limit (count=10)
--   5. Return
```

Extended EXPLAIN

```
-- Detailliertes EXPLAIN mit Statistiken
EXPLAIN OPTION {verbose: true}
FOR doc IN users
  FILTER doc.status == 'active'
  RETURN doc

-- Output zeigt:
-- - Estimated item count pro Step
-- - Filter selectivity
-- - Index usage
-- - Memory usage
```

Profile Queries

```
-- Query mit echtem Timing profilen
PROFILE FOR doc IN users
  FILTER doc.status == 'active'
  SORT doc.created_at
  RETURN doc

-- Ausgabe:
-- Step | Operation | Items | Time(ms) | Selectivity
-- 1    | Scan      | 1M    | 50        | 100%
-- 2    | Filter    | 100k  | 150       | 10%
-- 3    | Sort      | 100k  | 200       | 100%
-- 4    | Return    | 100k  | 10        | 100%
```

```
flowchart LR
  Query[AQL Query] --> Parse[Parser]
  Parse --> Analyze[Query Analyzer]
  Analyze --> Optimize[Query Optimizer]

  Optimize --> IndexCheck{Index verfügbar?}
  IndexCheck -->|Ja| IndexScan[Index Scan]
  IndexCheck -->|Nein| FullScan[Collection Scan]

  IndexScan --> Filter[Filter Early]
  FullScan --> Filter

  Filter --> Project[Projection]
  Project --> Sort{Sort nötig?}

  Sort -->|Ja| SortOp[Sort Operation]
  Sort -->|Nein| Limit

  SortOp --> Limit[Limit/Skip]
  Limit --> Return[Return Results]

  style IndexScan fill:#51cf66
  style FullScan fill:#ff6b6b
  style Filter fill:#4dabf7
  style Return fill:#40c057
```

34.2 Index-Strategien

Single Field Indexes

```
-- Einfachster Index - für exakte Matches & Range Queries
CREATE INDEX idx_status ON users(status)

-- Performance-Gain:
-- Vorher: Full Scan 1000ms für 1M Dokumente
-- Nachher: Index Lookup 1ms
-- Speedup: 1000x

-- Nutzbringend für:
FILTER doc.status == 'active'
FILTER doc.age > 18
FILTER doc.created_at BETWEEN '2024-01-01' AND '2024-12-31'
```

Composite Indexes

```
-- Multiple Felder - für komplexe Filter
CREATE INDEX idx_status_age ON users(status, age)

-- Hilft bei:
FILTER doc.status == 'active' AND doc.age > 18

-- Leading Field Optimization:
-- Der erste Index-Feld sollte die höchste Selectivity haben
-- Ranking: status_age (besser) vs age_status (schlechter)
-- Grund: First field narrowed down fast
```

Sparse Indexes

```
-- Index nur auf Dokumente mit Feld
CREATE SPARSE INDEX idx_phone ON users(phone)

-- Nutzen:
-- Spart Speicher (nur 30% der Dokumente haben phone)
-- Schneller Insert/Update (nur 30% indexieren)
-- Perfekt für optionale Felder

-- Abfrage:
FOR doc IN users
  FILTER doc.phone != null
  FILTER doc.phone LIKE '%123%'
  RETURN doc
```

Partial Indexes

```
-- Index nur auf Subset (mit Filter-Bedingung)
CREATE INDEX idx_active_premium ON users(category)
  WHERE subscription_status == 'premium'

-- Spart 80% Speicher wenn nur 20% premium sind
-- 10x schneller Index-Updates

-- Muss mit passender Query genutzt werden:
FOR doc IN users
  FILTER doc.subscription_status == 'premium'
  FILTER doc.category == 'enterprise'
  RETURN doc
```

Covering Indexes

```
-- Index mit all nötigen Feldern (ohne Dokument lesen)
CREATE INDEX idx_name_email_age ON users(name, email, age)

-- Super-optimiert für Query:
FOR doc IN users
  FILTER doc.email == 'alice@example.com'
  RETURN {name: doc.name, age: doc.age} -- Alles im Index!

-- Kein Dokument-Fetch nötig → 100-1000x schneller
```


34.3 Early Filtering Patterns

Filter-Pushdown Strategy

```
-- ❌ FALSCH: Filter zu spät
FOR edge IN edges
  RETURN {
    from: edge.from,
    to: edge.to,
    weight: (FOR v IN vertices FILTER v._id == edge.to RETURN v.weight)[0]
  }
-- Problem: N+1 Query, 1M edges = 1M vertex lookups

-- ✅ RICHTIG: Early Filter
FOR edge IN edges
  FILTER edge.weight > 0.5 -- Filter früh
  LET target_vertex = (
    FOR v IN vertices
      FILTER v._id == edge.to
      FILTER v.category == 'active' -- Filter auch inner
    RETURN v
  )[0]
  RETURN {
    from: edge.from,
    to: edge.to,
    target_weight: target_vertex.weight
  }
```

Collection Constraint Strategy

```
-- ❌ Generisch - keine Collection Constraint
FILTER LIKE(name, '%alice%') -- String-Operation für alle Doku

-- ✅ Mit Collection Filter
FOR user IN users -- Explizite Collection wählen
  FILTER user.status == 'active' -- Early
  FILTER LIKE(user.name, '%alice%') -- Text-Suche nur auf aktiven
  RETURN user
```

34.4 Aggregation Optimization

GROUP BY Optimization

```
--  Ineffizient: Alles sammeln dann groupieren
COLLECT category = doc.category
  AGGREGATE total = SUM(doc.amount)
  INTO grouped
-- Memory: O(n) - alle Doku im Memory

--  Effizient: Mit Index
FOR doc IN transactions
  FILTER doc.timestamp >= '2024-01-01' -- Filter früh
  SORT doc.category
  COLLECT category = doc.category
    AGGREGATE total = SUM(doc.amount)
  RETURN {category, total}
-- Memory: O(distinct categories) << O(n)
```

Approximate Aggregations

```
-- Für sehr große Datasets: Approximate Counts
FUNCTION count_approximate() {
  -- HyperLogLog für massive Sets
  -- Precision vs Speed Tradeoff
  RETURN APPROX_COUNT(*) FROM large_collection
}

-- Nutzen:
-- - COUNT(*) auf 1B Dokumente: 5 Sekunden statt 5 Minuten
-- - Genauigkeit: ±2% vs ±0%
-- - Speicher: 12KB vs 8GB
```

34.5 Window Function Performance

Partition Strategy

```
-- ❌ Falsch: Zu große Partitions
FOR sale IN sales
  WINDOW {
    PARTITION BY sale.region
    ORDER BY sale.timestamp
    RANGE CURRENT ROW TO 1000 ROWS FOLLOWING
  }
  LET total = SUM(sale.amount) OVER ...
  RETURN sale
-- Problem: Ein Region mit 10M Rows → 10M × 1000 = Massive Computation

-- ✅ Richtig: Kleinere Partitions
FOR sale IN sales
  FILTER sale.timestamp >= DATE_SUBTRACT(NOW(), 30, 'day')
  FILTER sale.region IN ['north', 'south'] -- Filter Partitions
  WINDOW {
    PARTITION BY sale.region, DATE_TRUNC(sale.timestamp, 'day')
    ORDER BY sale.timestamp
    RANGE CURRENT ROW TO 100 ROWS FOLLOWING
  }
  RETURN sale
```

34.6 Batch Operations

Bulk Insert Optimization

```
-- ❌ Loop mit einzelnen Inserts
FOR item IN input_items
  INSERT item INTO collection
-- 10k inserts = 10k Transaktionen, 10k fsync() Calls

-- ✅ Batch Insert
LET batch = input_items
INSERT batch INTO collection
-- 1 Transaktion, 1 fsync(), ~100x schneller
```

Update Batching

```
-- ❌ Einzelne Updates
FOR user IN users
  FILTER user.status == 'inactive'
  UPDATE user WITH {last_seen: NOW()}

-- ✅ Batch Update
LET inactive_users = (
  FOR u IN users
  FILTER u.status == 'inactive'
  RETURN u._id
)
FOR id IN inactive_users
  UPDATE {_id: id} WITH {last_seen: NOW()} IN users
```

34.7 Query Caching Patterns

Result Caching

```
from functools import lru_cache
import hashlib

class QueryCache:
    def __init__(self, ttl_seconds=300):
        self.ttl = ttl_seconds
        self.cache = {}
        self.timestamps = {}

    def cache_query(self, query_hash, result):
        self.cache[query_hash] = result
        self.timestamps[query_hash] = time.time()

    def get_cached(self, query_hash):
        if query_hash not in self.cache:
            return None

        age = time.time() - self.timestamps[query_hash]
        if age > self.ttl:
            del self.cache[query_hash]
            return None

        return self.cache[query_hash]
```

AQL Query Fragment Caching

```
-- Häufig wiederholte Subqueries cachen
FUNCTION get_active_users_cached() {
  -- Cache-Key: hash(filter_conditions)
  RETURN (
    FOR user IN users
    FILTER user.status == 'active'
    FILTER user.verified == true
    SORT user.created_at DESC
    RETURN user
  )
}

-- Nutze überall:
FOR user IN get_active_users_cached()
  RETURN user
```

34.8 Praktische Fallstudien

Case 1: E-Commerce Produktsuche

Original Query (5s):

```
FOR product IN products
  FILTER LIKE(product.name, '%laptop%')
  FILTER product.price BETWEEN 500 AND 2000
  FILTER product.category IN ['electronics', 'computers']
  SORT product.popularity DESC
  LIMIT 20
  RETURN product
```

Optimiert (100ms - 50x schneller):

```
-- 1. Index: CREATE INDEX idx_cat_price ON products(category, price)
-- 2. Full-Text: CREATE FULLTEXT INDEX idx_name ON products(name)

FOR product IN products
  FILTER product.category IN ['electronics', 'computers']
  FILTER product.price BETWEEN 500 AND 2000
  FILTER product.available == true
  SEARCH product.name IN FULLTEXT('laptop')
  SORT product.popularity DESC
  LIMIT 20
  RETURN {_key: product._key, name: product.name, price: product.price}
```

Case 2: Reporting Analytics

Original (2 Minuten):

```
FOR order IN orders
  FOR item IN order.items
    COLLECT category = item.category
    AGGREGATE total = SUM(item.quantity * item.price)
    INTO result
  RETURN {category, total}
```

Optimiert (2 Sekunden - 60x schneller):

```
FOR order IN orders
  FILTER order.status == 'completed'
  FILTER order.created_at >= '2024-01-01'
  FILTER order.created_at < '2025-01-01'
  FOR item IN order.items
    SORT item.category
    COLLECT category = item.category
    AGGREGATE total = SUM(item.quantity * item.price),
                count = COUNT(item)
    INTO result
  SORT result.category
  RETURN result
```

34.9 Performance Monitoring Dashboard

```
class QueryPerformanceMonitor:
    def __init__(self):
        self.query_stats = {}

    def record_query(self, query_hash, execution_time_ms, rows):
        if query_hash not in self.query_stats:
            self.query_stats[query_hash] = {
                'count': 0,
                'total_time': 0,
                'max_time': 0,
                'avg_time': 0
            }

            stats = self.query_stats[query_hash]
            stats['count'] += 1
            stats['total_time'] += execution_time_ms
            stats['max_time'] = max(stats['max_time'], execution_time_ms)
            stats['avg_time'] = stats['total_time'] / stats['count']

            if execution_time_ms > 1000: # Flag slow queries
                print(f"⚠ SLOW: {query_hash} took {execution_time_ms}ms")

    def get_slowest_queries(self, top_n=10):
        sorted_queries = sorted(
            self.query_stats.items(),
            key=lambda x: x[1]['total_time'],
            reverse=True
        )
        return sorted_queries[:top_n]
```

Zusammenfassung

ThemisDB bietet mit EXPLAIN, Indexierung und Query-Refactoring Möglichkeiten für 10-100x Performance-Verbesserungen. Schlüssel sind: -  **Early Filtering** - Filter so früh wie möglich -  **Richtige Indizes** - Covering Indexes für Full Index Scans -  **Batch Operations** - Nutze Bulk für viele Operationen -  **Aggregation Optimization** - COLLECT mit Sort für Memory-Effizienz -  **Monitoring** - PROFILE/EXPLAIN nutzen zur Diagnose

Mit diesen Techniken erreichen Sie Production-Grade Performance.

Kapitel 45: Kapitel 35: Data Modeling Patterns & Anti-Patterns

"Mit den gleichen Daten können Sie ein System entweder elegant oder chaotisch modellieren. Die richtige Struktur entscheidet über Erfolg oder Burnout."

Überblick

Data Modeling ist die Grundlage jeder erfolgreichen Datenbankanwendung. Dieses Kapitel zeigt bewährte Patterns und warnt vor häufigen Anti-Patterns, die zu technischen Schulden führen.

Was Sie in diesem Kapitel lernen: - Document Structure Patterns (Embedded vs Referenced) - Schema Design für verschiedene Use Cases - Versionierung und Schema-Evolution - Data Integrity Patterns - Denormalisierung vs Normalisierung - Multi-Model Design Patterns - Anti-Patterns und wie man sie vermeidet

```
graph LR
  subgraph "Embedded (1:Few)"
    UserEmb[User] --> AddressEmb[Addresses Array]
  end

  subgraph "Referenced (1:Many)"
    UserRef[User] --> OrderRef[Orders Collection]
    OrderRef --> OrderDoc1[Order 1]
    OrderRef --> OrderDoc2[Order 2]
    OrderRef --> OrderDoc3["Order N..."]
  end

  subgraph "Hybrid (Best of Both)"
    UserHyb[User] --> AddressHyb[Address Embedded]
    UserHyb --> OrderSummary[Recent Orders Embedded]
    UserHyb --> OrderRefHyb[All Orders Referenced]
  end

  style UserEmb fill:#4facfe
  style UserRef fill:#f093fb
  style UserHyb fill:#43e97b
```

Abb. 35.0: Data-Modeling-Patterns

35.1 Embedded vs Referenced Documents

Pattern 1: Full Embedding

Geeignet für: One-to-Few Relationships

```
-- E-Commerce: Order mit Produktdetails embedded
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  items: [
    {
      product_id: "prod_111",
      product_name: "Laptop", -- Embedded - COPY der Daten
      quantity: 1,
      price_at_purchase: 999.99
    },
    {
      product_id: "prod_222",
      product_name: "Mouse",
      quantity: 2,
      price_at_purchase: 19.99
    }
  ],
  total: 1039.97
}
```

Vorteile: -  Atomare Updates (Order + Items = 1 Dokument) -  Keine Joins nötig -  Höchste Performance für Lesezugriffe -  Natürliche Denormalisierung für Snapshot-Daten

Nachteile: -  Datenduplizierung (Produkt-Name in 1000 Orders) -  Updates komplex (100 Orders mit laptop aktualisieren?) -  Speicher-Overhead

Best Practice: Nutze für Snapshot-Daten (Preis zum Kaufzeitpunkt, nicht aktuelle Produkt-Info)

flowchart TD

Start[Data Model Design] --> Analyze{Beziehung?}

Analyze -->|1:1| Embed1[Embed in Parent]

Analyze -->|1:Few| CheckSize{Size < 100?}

Analyze -->|1:Many| Ref1[Reference Pattern]

Analyze -->|M:N| Junction[Junction Collection]

CheckSize -->|Yes| Embed2[Embed Array]

CheckSize -->|No| Ref2[Reference Array]

Embed1 --> Immutable{Immutable?}

Embed2 --> Immutable

Immutable -->|Yes| Final1[✓ Embedded OK]

Immutable -->|No| Consider{Updates häufig?}

Consider -->|Yes| Ref3[Consider Reference]

Consider -->|No| Final1

Ref1 --> Index1[Add Index on FK]

Ref2 --> Index1

Ref3 --> Index1

Junction --> Index2[Index both FKs]

Index1 --> Final2[✓ Referenced]

Index2 --> Final2

Final1 --> Validate{Doc Size?}

Validate -->|< 16MB| OK[✓ Valid]

Validate -->|> 16MB| Split[Split Document]

style Embed1 fill:#51cf66

style Embed2 fill:#51cf66

style Final1 fill:#40c057

style Final2 fill:#4dabf7

style OK fill:#40c057

Pattern 2: Reference Links

Geeignet für: One-to-Many und Many-to-Many

```
-- Order nur mit Referenzen:
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  item_ids: ["oi_1001", "oi_1002"] -- Referenzen statt Embedding
}

-- Order Items separate Collection:
{
  _key: "oi_1001",
  order_id: "order_12345",
  product_id: "prod_111",
  quantity: 1,
  price: 999.99
}

-- Abfrage mit JOIN:
FOR order IN orders
  FILTER order._key == 'order_12345'
  FOR item IN order_items
    FILTER item._id IN order.item_ids
  RETURN {
    product: item.product_id,
    qty: item.quantity,
    price: item.price
  }
```

Vorteile: -  Keine Datenduplizierung -  Flexible Updates (Produkt-Name einmal ändern) - 
 Speicher-effizient -  Normalisiertes Design

Nachteile: -  JOINS nötig (Performance-Hit) -  Konsistenz-Komplexität (order + items = 2
 Transaktionen?) -  Komplexere Queries

Best Practice: Nutze für Live-Daten (Produkt-Info, Benutzer-Profil)

35.2 Hybrid Pattern: Optimal Denormalization

```
-- BESTE LÖSUNG: Balance zwischen beiden
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  items: [
    {
      order_item_id: "oi_1001",  -- Referenz
      product_id: "prod_111",    -- Referenz
      product_name: "Laptop",    -- Denormalisiert (Snapshot)
      quantity: 1,
      price_at_purchase: 999.99  -- Snapshot (nicht ändern)
    }
  ],
  customer_name: "Alice",        -- Denormalisiert für Report
  customer_email: "alice@example.com",
  total: 1039.97,
  last_modified: "2025-01-01T10:00:00Z"
}

-- + Separate Product Collection für Live-Updates:
{
  _key: "prod_111",
  name: "Laptop",
  description: "Latest model",  -- Änderungen nur hier
  price: 899.99,               -- Aktueller Preis (nicht im Order)
  availability: "in_stock"
}
```

Strategie: - **Embedded:** Snapshot-Daten (was zum Kaufzeitpunkt galt) - **Referenced:** Live-Daten (aktuelle Zustand) - **Join bei Bedarf:** Nur wenn aktuelle Info nötig

35.3 Schema Evolution Patterns

Forward Compatibility

```
-- Version 1: Alte Struktur
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com"
}

-- Version 2: Neue Felder (backward compatible!)
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com",
  phone: "+49123456789",      -- Neu, optional
  address: {                 -- Neu, nested
    street: "Main St",
    city: "Berlin"
  },
  metadata: {                -- Neu, extensible
    last_login: "2025-01-01",
    login_count: 42
  }
}

-- Abfrage muss robust sein:
FOR user IN users
  FILTER user.email == 'alice@example.com'
  RETURN {
    name: user.name,
    phone: user.phone ?? "unknown", -- Null coalescing
    city: user.address.city ?? "unknown"
  }
```

Versioned Collections

```
-- Collections mit Version in _key
-- users_v1: Alte Version (archiviert)
-- users_v2: Neue Version (aktiv)

-- Migration:
FOR user IN users_v1
  INSERT {
    name: user.name,
    email: user.email,
    phone: null,
    address: null,
    created_from_v1: true
  } INTO users_v2

-- Dual-Write Phase (Rollout):
-- 1. Writes zu v1 UND v2 (2 Sekunden länger)
-- 2. Reads von v2, Fallback zu v1
-- 3. Nach Verification: nur v2
-- 4. v1 Cleanup nach 30 Tagen
```

35.4 Data Integrity Patterns

Constraints und Validation

```
-- Validation in Insert Trigger
FUNCTION validate_user(user) {
  IF !LIKE(user.email, '%@%.%') THEN
    THROW ERROR('Invalid email format')
  END

  IF user.age < 0 OR user.age > 150 THEN
    THROW ERROR('Age must be 0-150')
  END

  IF !HAS(user, 'name') OR LENGTH(user.name) < 1 THEN
    THROW ERROR('Name is required')
  END

  RETURN user
}

-- Trigger bei Insert:
INSERT validate_user(new_user) INTO users
```

Referential Integrity

```
-- Before Delete: Check for References
FUNCTION can_delete_product(product_id) {
  LET order_count = LENGTH(
    FOR oi IN order_items
    FILTER oi.product_id == product_id
    RETURN oi
  )

  IF order_count > 0 THEN
    THROW ERROR(
      CONCAT("Cannot delete: ", order_count, " orders reference this product")
    )
  END

  RETURN true
}

-- Soft Delete Pattern (meist besser):
UPDATE {_id: product_id} WITH {
  deleted_at: NOW(),
  is_deleted: true
} IN products

-- Queries filtern automatisch:
FOR prod IN products
  FILTER !prod.is_deleted
  RETURN prod
```


35.5 Common Anti-Patterns & Fixes

Anti-Pattern 1: Unbounded Arrays

```
-- ❌ FALSCH: Array wächst unbegrenzt
{
  _key: "user_123",
  comments: [ -- Kann 1M Items enthalten!
    {id: "c1", text: "Hello"},
    {id: "c2", text: "Hi"},
    ... (1M mehr)
  ]
}

-- Problem: Jeder Update des Users ändert Array
-- Memory: 1M Items im RAM
-- Performance: O(n) für jeden Insert

-- ✅ RICHTIG: Separate Collection
{
  _key: "user_123",
  comment_count: 1000000
}

{
  _key: "comment_12345",
  user_id: "user_123",
  text: "Hello",
  created_at: "2025-01-01T10:00:00Z"
}

-- Abfrage:
FOR user IN users
  FILTER user._key == 'user_123'
FOR comment IN comments
  FILTER comment.user_id == user._key
  SORT comment.created_at DESC
  LIMIT 20
  RETURN comment
```

Anti-Pattern 2: Wide Documents

```
-- ❌ FALSCH: Ein Dokument mit 500+ Felder
{
  _key: "product_123",
  name: "...",
  description: "...",
  color_red: 0.9,
  color_green: 0.1,
  color_blue: 0.05,
  ... (500 mehr Felder)
}

-- ✅ RICHTIG: Nested Objects für Kategorien
{
  _key: "product_123",
  name: "Laptop",
  description: "...",
  appearance: {
    color: {r: 0.9, g: 0.1, b: 0.05},
    material: "aluminium"
  },
  specs: {
    cpu: "Intel i9",
    ram: "32GB",
    storage: "1TB SSD"
  },
  pricing: {
    list_price: 1999.99,
    discount_percent: 10,
    final_price: 1799.99
  }
}
```

Anti-Pattern 3: Sparse Indexes on Many Fields

```
-- ✗ FALSCH: Index pro optionales Feld
CREATE INDEX idx_phone ON users(phone)
CREATE INDEX idx_mobile ON users(mobile)
CREATE INDEX idx_fax ON users(fax)
-- 3 Indizes, 70% davon sind NULL

-- ✓ RICHTIG: One General Contact Index
{
  _key: "user_123",
  contact: {
    phone: "+49123456789",
    mobile: null,
    fax: null,
    email: "alice@example.com"
  }
}

CREATE SPARSE INDEX idx_contact ON users(contact)

-- Abfrage:
FOR user IN users
  FILTER user.contact.phone != null
  FILTER LIKE(user.contact.phone, '%123%')
RETURN user
```

35.6 Multi-Model Design Patterns

Pattern: Document + Graph

```
-- Documents: User Profile
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com"
}

-- Graph: User Relationships
{
  _key: "follows_123_456",
  _from: "users/123",
  _to: "users/456",
  followed_at: "2025-01-01",
  type: "follows"
}

-- Query: Netzwerk-Analyse
FOR user IN users
  FILTER user.email == 'alice@example.com'
  LET followers = (
    FOR follower IN users
      ANY INBOUND user GRAPH 'social_graph'
      RETURN follower
  )
  LET following = (
    FOR following_user IN users
      ANY OUTBOUND user GRAPH 'social_graph'
      RETURN following_user
  )
  RETURN {
    user: user.name,
    followers_count: LENGTH(followers),
    following_count: LENGTH(following)
  }
```

Pattern: Document + Vector + Search

```
-- Documents: Articles
{
  _key: "article_123",
  title: "Machine Learning Basics",
  content: "...",
  vector: [0.2, 0.5, 0.1, ...] -- 1536-dim embedding
}

-- Vector Search + Full-Text
FOR article IN articles
  SEARCH article.vector ANN {
    query: [0.1, 0.4, 0.2, ...],
    top_k: 10
  }
  SEARCH PHRASE(article.title, 'machine learning', 'title')
    OR PHRASE(article.content, 'neural networks')
  RETURN {
    title: article.title,
    similarity: article._score
  }
```

35.7 Practical Migration Patterns

Blue-Green Deployment

```
-- Phase 1: Parallel Execution
-- - Write zu v1 UND v2
-- - Read von v1 (stabil)

-- Phase 2: Validation
-- - Vergleiche v1 vs v2 Ergebnisse
-- - Prüfe auf Datenabweichungen

-- Phase 3: Cutover
-- - Schreib-Lock auf v1
-- - Letzte Sync v1 → v2
-- - Schreib-Lock freigeben auf v2
-- - Reads → v2

-- Phase 4: Cleanup
-- - Nach 30 Tagen: v1 Archivieren
-- - Nach 90 Tagen: v1 Löschen
```

Zusammenfassung

Gutes Data Modeling ist Kunst und Wissenschaft: - **Embedded:** Für Snapshots (Preis zum Kaufzeitpunkt) - **Referenced:** Für Live-Daten (aktuelle Produkt-Info) - **Hybrid:** Kombination für beste Performance - **Versioniert:** Schema-Evolution mit Sicherheit - **Validated:** Constraints at Insert-Time - **Separat:** Arrays/Felder in eigene Collections - **Nested:** Wide Documents mit Struktur - **Monitored:** Indizes auf Query-Patterns

Mit diesen Patterns bauen Sie skalierbare, wartbare Datenmodelle.

Kapitel 46: Kapitel 36: Security Hardening Playbook

"Security ist kein Feature, es ist Architektur. Ein sicheres System ist gebaut von innen heraus, nicht bolzen-on later."

Überblick

Dieses Kapitel bietet Praktiker-Anleitungen für Production-Grade Security in ThemisDB. Von grundlegenden Authentifizierungspattern bis zu erweiterten Threat-Mitigation-Strategien.

Was Sie in diesem Kapitel lernen: - Network Segmentation und Firewall Rules - Authentication & Authorization Patterns - End-to-End Encryption - SQL Injection & Injection Attack Prevention - Access Control Lists (ACL) & Role-Based Access Control (RBAC) - Audit Logging & Intrusion Detection - Secrets Management - Compliance & Regulatory Frameworks - Incident Response Playbooks

```
graph TB
    Client[Client Application] --> TLS[TLS slash SSL Layer]
    TLS --> Auth[Authentication]

    Auth --> JWT{JWT Token<br/>Validation}
    JWT -->|Valid| RBAC[RBAC Check]
    JWT -->|Invalid| Reject1[Reject 401]

    RBAC --> Perm{Permission<br/>Check}
    Perm -->|Granted| EncData[Encrypted Data Access]
    Perm -->|Denied| Reject2[Reject 403]

    EncData --> Decrypt[Decrypt at Runtime]
    Decrypt --> Result[Return Data]

    Result --> Audit[Audit Log]

    style TLS fill:#4facfe
    style EncData fill:#43e97b
    style Audit fill:#f093fb
```

Abb. 36.0: Security-Layers: Defense in Depth

36.1 Network Security

Firwall-Regeln

```
---
rules:
  # Externe API -> ThemisDB
  - name: "Allow API Gateway"
    type: "inbound"
    protocol: "tcp"
    port: 8529
    source: "10.0.1.0/24" # API Gateway subnet
    action: "allow"

  # Client App -> ThemisDB
  - name: "Allow App Servers"
    type: "inbound"
    protocol: "tcp"
    port: 8529
    source: "10.0.2.0/24" # App subnet
    action: "allow"

  # Replication zwischen Nodes
  - name: "Allow Replication"
    type: "inbound"
    protocol: "tcp"
    port: 8628
    source: "10.0.3.0/24" # Cluster subnet
    action: "allow"

  # Deny All (Default)
  - name: "Deny All"
    type: "inbound"
    protocol: "all"
    action: "deny"
```

TLS/SSL Configuration

```
security:
  tls:
    enabled: true
    version: "1.3" # Minimum
    certificate_path: "/etc/themis/certs/server.crt"
    key_path: "/etc/themis/certs/server.key"
    cipher_suites:
      - "TLS_AES_256_GCM_SHA384"
      - "TLS_CHACHA20_POLY1305_SHA256"
      - "TLS_AES_128_GCM_SHA256"
    verify_client: true # Mutual TLS
    client_ca_path: "/etc/themis/certs/client-ca.crt"
```

36.2 Authentication & Authorization

```
graph TB
    subgraph "Security Layers"
        Network[Network Layer<br/>Firewall, VPN, TLS]
        Auth[Authentication Layer<br/>mTLS, JWT, OAuth2]
        Authz[Authorization Layer<br/>RBAC, ABAC, ACL]
        Data[Data Layer<br/>Encryption at Rest]
        Audit[Audit Layer<br/>Logging, Monitoring]
    end

    Client[Client Request] --> Network
    Network --> Auth
    Auth --> Authz
    Authz --> Data
    Data --> Response[Response]

    Network -.-> Audit
    Auth -.-> Audit
    Authz -.-> Audit
    Data -.-> Audit

    Audit --> SIEM[SIEM/Analysis]
    SIEM --> Alert{Threat?}
    Alert -->|Yes| Block[Block & Alert]
    Alert -->|No| Allow[Allow]

    style Network fill:#4dabf7
    style Auth fill:#fab005
    style Authz fill:#fa5252
    style Data fill:#51cf66
    style Audit fill:#845ef7
    style Block fill:#ff6b6b
```

RBAC (Role-Based Access Control)

```
-- Define roles
FUNCTION create_role(role_name, permissions) {
  RETURN INSERT {
    role: role_name,
    permissions: permissions,
    created_at: NOW()
  } INTO roles
}

-- Example: Reader role
LET reader_perms = {
  collections: {
    "*": ["read"] -- All collections, read-only
  },
  functions: ["all"] -- Can call all functions
}

-- Create reader role
create_role("reader", reader_perms)

-- Assign user to role
UPDATE {_id: "users/alice"} WITH {
  roles: ["reader"],
  role_updated_at: NOW()
} IN users
```

JWT Token Validation

```
import jwt
from datetime import datetime, timedelta
import hashlib

class JWTAuthenticator:
    def __init__(self, secret_key):
        self.secret_key = secret_key
        self.algorithm = "HS256"

    def create_token(self, user_id, roles, expires_in=3600):
        payload = {
            "user_id": user_id,
            "roles": roles,
            "iat": datetime.utcnow(),
            "exp": datetime.utcnow() + timedelta(seconds=expires_in)
        }
        token = jwt.encode(payload, self.secret_key, algorithm=self.algorithm)
        return token

    def verify_token(self, token):
        try:
            payload = jwt.decode(token, self.secret_key, algorithms=[self.algorithm])
            return payload
        except jwt.ExpiredSignatureError:
            raise Exception("Token has expired")
        except jwt.InvalidTokenError:
            raise Exception("Invalid token")

    def get_user_permissions(self, payload):
        user_id = payload["user_id"]
        roles = payload["roles"]
        # Lookup permissions from database
        return {"roles": roles, "timestamp": datetime.utcnow()}
```

36.3 Encryption Patterns

Field-Level Encryption

```
-- Encrypt sensitive fields before storage
FUNCTION encrypt_field(value, key) {
  -- Use ThemisDB's built-in CRYPTO_ENCRYPT function
  RETURN CRYPTO_ENCRYPT(value, key, 'AES-256-GCM')
}

-- Store with encrypted field
INSERT {
  user_id: 'user_123',
  name: 'Alice',
  email: encrypt_field('alice@example.com', encryption_key),
  ssn: encrypt_field('123-45-6789', encryption_key),
  created_at: NOW()
} INTO users

-- Decrypt on read
FOR user IN users
  FILTER user.user_id == 'user_123'
  LET decrypted_email = CRYPTO_DECRYPT(user.email, encryption_key)
  RETURN {
    name: user.name,
    email: decrypted_email
  }
```

End-to-End Encryption (E2EE)

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend

class E2EEEncryption:
    def __init__(self):
        self.backend = default_backend()

    def generate_keypair(self):
        """Generate RSA keypair for client"""
        private_key = rsa.generate_private_key(
            public_exponent=65537,
            key_size=4096,
            backend=self.backend
        )
        public_key = private_key.public_key()
        return private_key, public_key

    def encrypt_message(self, message, public_key):
        """Encrypt message with public key"""
        ciphertext = public_key.encrypt(
            message.encode(),
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return ciphertext.hex()

    def decrypt_message(self, ciphertext_hex, private_key):
        """Decrypt with private key (only client)"""
        ciphertext = bytes.fromhex(ciphertext_hex)
        plaintext = private_key.decrypt(
            ciphertext,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return plaintext.decode()
```

36.4 Injection Prevention

AQL Injection Protection

```
--  UNSAFE: User input directly in query
FUNCTION unsafe_search(search_term) {
  RETURN EXECUTE(
    CONCAT("FOR doc IN collection FILTER LIKE(doc.name, '%", search_term, "%') RETURN doc")
  )
  -- Attacker can inject: %') OR (1==1) //'
}

--  SAFE: Parameterized queries
FUNCTION safe_search(search_term) {
  RETURN (
    FOR doc IN collection
    FILTER LIKE(doc.name, @search_pattern)
    RETURN doc
  )
}

-- Call with bound parameters:
safe_search('@search_pattern', '%term%')
```

Input Validation

```
import re
from typing import Any, Dict

class InputValidator:
    @staticmethod
    def validate_email(email: str) -> bool:
        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        return re.match(pattern, email) is not None

    @staticmethod
    def validate_aql_identifier(identifier: str) -> bool:
        # Only allow alphanumeric and underscore
        pattern = r'^[a-zA-Z_][a-zA-Z0-9_]*$'
        return re.match(pattern, identifier) is not None

    @staticmethod
    def sanitize_collection_name(name: str) -> str:
        # Whitelist: only alphanumeric, underscore, dash
        sanitized = re.sub(r'^[a-zA-Z0-9_\-]', '', name)
        return sanitized.lower()

    @staticmethod
    def validate_query_size(query: str, max_size: int = 10000) -> bool:
        return len(query) <= max_size
```


36.5 Audit Logging

Comprehensive Audit Trail

```
-- Log all operations
FUNCTION audit_log(operation, user_id, resource, changes) {
  RETURN INSERT {
    timestamp: NOW(),
    operation: operation, -- "INSERT", "UPDATE", "DELETE"
    user_id: user_id,
    resource: resource,   -- "users/123"
    changes: changes,     -- {field: old_value -> new_value}
    ip_address: @ip_address,
    user_agent: @user_agent
  } INTO audit_logs
}

-- Usage:
UPDATE {_id: 'users/123'} WITH {email: 'new@example.com'}
audit_log('UPDATE', 'admin_001', 'users/123', {
  email: 'old@example.com' -> 'new@example.com'
})
```

Immutable Audit Log

```
class ImmutableAuditLog:
    def __init__(self, db):
        self.db = db

    def append_log(self, log_entry):
        """Append-only: cannot update/delete"""
        # Insert with timestamp
        log_entry['_timestamp'] = time.time()
        log_entry['_hash'] = self._compute_hash(log_entry)

        # Previous entry's hash
        last_entry = self.db.get_last_audit_log()
        if last_entry:
            log_entry['_previous_hash'] = last_entry['_hash']

        return self.db.insert_audit_log(log_entry)

    def _compute_hash(self, entry):
        """Cryptographic hash for integrity"""
        import hashlib
        content = json.dumps(entry, sort_keys=True)
        return hashlib.sha256(content.encode()).hexdigest()

    def verify_integrity(self):
        """Detect tampering by verifying hash chain"""
        logs = self.db.get_all_audit_logs()
        for i, log in enumerate(logs):
            if i > 0:
                prev_log = logs[i-1]
                if log['_previous_hash'] != prev_log['_hash']:
                    raise Exception(f"Integrity violation at log {i}")
```

36.6 Secrets Management

Kubernetes Secrets Integration

```
apiVersion: v1
kind: Secret
metadata:
  name: themis-secrets
  namespace: themis
type: Opaque
stringData:
  # Database encryption key
  database_key: "base64_encoded_key_here"

  # TLS certificates
  tls_cert: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----

  tls_key: |
    -----BEGIN PRIVATE KEY-----
    ...
    -----END PRIVATE KEY-----

  # API keys for external services
  stripe_api_key: "sk_live_..."
  sendgrid_api_key: "SG..."
```

Vault Integration

```
import hvac
import os

class VaultSecretsManager:
    def __init__(self, vault_addr, token):
        self.client = hvac.Client(url=vault_addr, token=token)

    def get_secret(self, secret_path):
        response = self.client.secrets.kv.v2.read_secret_version(
            path=secret_path
        )
        return response['data']['data']

    def rotate_database_key(self):
        """Rotate encryption key - called before data re-encryption"""
        new_key = os.urandom(32) # 256-bit key
        self.client.secrets.kv.v2.create_or_update_secret(
            path="themis/database_key",
            secret_dict={"key": new_key.hex()}
        )
        return new_key
```

36.7 Compliance & Regulatory

GDPR Compliance

```
-- Right to be forgotten: Pseudonymization
FUNCTION gdpr_pseudonymize_user(user_id) {
  LET user = DOCUMENT('users/' + user_id)

  UPDATE {_id: 'users/' + user_id} WITH {
    email: HASH(user.email),
    phone: null,
    address: null,
    name: "Anonymized User",
    gdpr_deleted_at: NOW(),
    is_pseudonymized: true
  } IN users

  -- Also delete from audit logs
  REMOVE l IN audit_logs
  FILTER l.resource == CONCAT('users/', user_id)
}
```

SOC 2 Compliance Checklist

SOC 2 Type II Compliance

- [] ****Access Control****
 - [] Multi-factor authentication (MFA)
 - [] Role-based access control (RBAC)
 - [] Audit logging of all access
- [] ****Data Security****
 - [] End-to-end encryption (AES-256)
 - [] Field-level encryption for PII
 - [] Secure key management (Vault)
- [] ****Availability****
 - [] 99.99% uptime SLA
 - [] Automated failover
 - [] Regular disaster recovery tests
- [] ****Monitoring & Logging****
 - [] 24/7 intrusion detection
 - [] Immutable audit logs
 - [] Real-time alerting
- [] ****Incident Response****
 - [] Documented IR procedures
 - [] Regular tabletop exercises
 - [] <1 hour incident response time

36.8 Incident Response Playbook

Detection

```
-- Detect unusual access patterns
FUNCTION detect_brute_force(user_id, threshold = 5) {
  LET failed_logins = LENGTH(
    FOR log IN audit_logs
    FILTER log.user_id == user_id
    FILTER log.operation == 'FAILED_LOGIN'
    FILTER log.timestamp > DATE_SUBTRACT(NOW(), 1, 'hour')
    RETURN log
  )

  IF failed_logins >= threshold THEN
    RETURN {
      alert: "BRUTE_FORCE_DETECTED",
      user_id: user_id,
      failed_count: failed_logins,
      action: "LOCK_ACCOUNT"
    }
  END

  RETURN {alert: "normal"}
}
```

Response

```
class IncidentResponse:
    def lock_account(self, user_id):
        """Lock account immediately"""
        db.update_user(user_id, {'locked': True, 'locked_at': now()})

    def notify_security_team(self, incident):
        """Alert security team"""
        slack.send_message(f"🚨 SECURITY ALERT: {incident['alert']}")
        email.send_alert(incident)

    def revoke_tokens(self, user_id):
        """Invalidate all active tokens"""
        db.delete_user_sessions(user_id)

    def isolate_database(self):
        """Disable external access"""
        firewall.block_external_connections()

    def preserve_evidence(self):
        """Archive logs for forensic analysis"""
        backup.create_forensic_snapshot()
```

Zusammenfassung

Production-Sicherheit erfordert: -  **Network Segmentation** - Firewall rules, TLS 1.3 -  **Authentication** - JWT, MFA, RBAC -  **Encryption** - AES-256 field-level, E2E -  **Input Validation** - Parameterized queries, sanitization -  **Audit Logging** - Immutable logs, integrity checks -  **Secrets Management** - Vault, rotation -  **Compliance** - GDPR, SOC 2, audit trails -  **Incident Response** - Playbooks, automation

Mit diesen Patterns bauen Sie ein **Defense-in-Depth** System, wo mehrere Sicherheitsebenen zusammenarbeiten.

Kapitel 47: Kapitel 37: Ecosystem Integration & Extensions

"ThemisDB ist nicht nur eine Datenbank - es ist die Mittellage in einem größeren Ökosystem von Tools, Services und Integrationen."

Überblick

Dieses Kapitel zeigt, wie ThemisDB mit externen Systemen integriert wird und wie Sie Custom Extensions schreiben, um ThemisDB an spezifische Anforderungen anzupassen.

Was Sie in diesem Kapitel lernen: - Native Integrations mit populären Tools - Custom Function Development - Plugin Architecture - Webhook & Event Streaming - Third-party Library Support - Custom Data Type Extensions - Performance Monitoring Integrations - Cross-database Synchronization

```
flowchart LR
    subgraph External_Systems [External Systems]
        ES[Elasticsearch]
        K[Kafka]
        PG[PostgreSQL]
        R[Redis]
        P[Prometheus]
    end

    subgraph ThemisDB_Core [ThemisDB Core]
        TDB[(ThemisDB)]
        PLUG[Plugins]
        HOOK[Webhooks]
    end

    subgraph Application_Layer [Application Layer]
        APP[App Server]
        API[REST API]
        WS[WebSocket]
    end

    ES <-->|Full-Text Sync| TDB
    K <-->|Event Stream| HOOK
    PG <-->|Data Federation| TDB
    R <-->|Cache Layer| TDB
    P -->|Metrics| TDB

    TDB --> PLUG
    PLUG --> API
    HOOK --> WS
    API --> APP
    WS --> APP
```

```
graph LR
  ExtSys[External Systems] --> Gateway[API Gateway]
  Gateway --> Auth[Auth Service]

  Auth --> ThemisDB["(ThemisDB)"]

  ThemisDB --> CDC[Change Data Capture]
  CDC --> Kafka[Kafka Event Stream]

  Kafka --> Analytics[Analytics Service]
  Kafka --> Search[Search Service]
  Kafka --> Cache[Cache Service]

  style ThemisDB fill:#7c4dff
  style Kafka fill:#f093fb
```

Abb. 37.0: Integration mit externen Systemen

37.1 Beliebte Integrationen

Elasticsearch Integration

```

from elasticsearch import Elasticsearch
import themis_client

class ElasticsearchSync:
    def __init__(self, themis_conn, es_host='localhost:9200'):
        self.themis = themis_client.connect(themis_conn)
        self.es = Elasticsearch([es_host])

    def sync_documents_to_elasticsearch(self, collection_name, query=None):
        """Sync ThemisDB documents to Elasticsearch"""
        # Query documents from ThemisDB
        aql = f"FOR doc IN {collection_name}"
        if query:
            aql += f" FILTER {query}"
        aql += " RETURN doc"

        docs = self.themis.execute(aql)

        # Bulk insert into Elasticsearch
        from elasticsearch.helpers import bulk
        actions = [
            {
                "_index": collection_name,
                "_id": doc['_id'],
                "_source": doc
            }
            for doc in docs
        ]

        bulk(self.es, actions)
        return len(actions)

    def search_with_fallback(self, query, collection_name):
        """Search in Elasticsearch, fallback to ThemisDB"""
        try:
            # Try ES first (faster)
            results = self.es.search(
                index=collection_name,
                body={"query": {"match": {"_all": query}}}
            )
            return results['hits']['hits']
        except Exception:
            # Fallback to ThemisDB full-text search
            aql = f"FOR doc IN {collection_name} SEARCH {query} RETURN doc"
            return self.themis.execute(aql)

```

Kafka Event Streaming

```
themes_db_events:
  topics:
    - name: "themis.inserts"
      partitions: 10
      retention: "7d"
      events: "INSERT operations"

    - name: "themis.updates"
      partitions: 10
      retention: "7d"
      events: "UPDATE operations"

    - name: "themis.deletes"
      partitions: 10
      retention: "7d"
      events: "DELETE operations"

  producers:
    - name: "themis-cdc"
      topics: ["themis.inserts", "themis.updates", "themis.deletes"]
      format: "avro"
      compression: "snappy"

  consumers:
    - name: "elasticsearch-sync"
      topics: ["themis.*"]
      group_id: "es-sync-group"

    - name: "analytics-pipeline"
      topics: ["themis.*"]
      group_id: "analytics-group"
```

Prometheus Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import themis_client

class ThemisPrometheus:
    def __init__(self):
        # Operation counters
        self.query_counter = Counter(
            'themis_queries_total',
            'Total queries executed',
            ['operation_type', 'collection']
        )

        # Query latency histogram
        self.query_latency = Histogram(
            'themis_query_duration_seconds',
            'Query execution time',
            ['operation_type'],
            buckets=[0.001, 0.01, 0.1, 1, 10]
        )

        # Database size gauge
        self.db_size = Gauge(
            'themis_database_bytes',
            'Database size in bytes'
        )

        # Connection pool
        self.active_connections = Gauge(
            'themis_connections_active',
            'Active database connections'
        )

    def record_query(self, operation_type, collection, duration_ms):
        self.query_counter.labels(
            operation_type=operation_type,
            collection=collection
        ).inc()

        self.query_latency.labels(
            operation_type=operation_type
        ).observe(duration_ms / 1000)
```

37.2 Custom Function Development

Creating Custom AQL Functions

```
-- Register custom function
FUNCTION CUSTOM_DISTANCE(lat1, lon1, lat2, lon2) {
  -- Haversine distance formula
  LET R = 6371  -- Earth radius in km
  LET dLat = RADIANS(lat2 - lat1)
  LET dLon = RADIANS(lon2 - lon1)
  LET a = POWER(SIN(dLat/2), 2) + COS(RADIANS(lat1)) * COS(RADIANS(lat2)) * POWER(SIN(dLon/2), 2)
  LET c = 2 * ASIN(SQRT(a))
  RETURN R * c
}

-- Usage:
FOR user IN users
  LET distance = CUSTOM_DISTANCE(
    51.5074, -0.1278,  -- London coords
    user.latitude, user.longitude
  )
  FILTER distance < 10  -- Within 10km
  RETURN {name: user.name, distance: distance}
```

External Function Binding (C++)

```
// custom_functions.cpp
#include <aql/function_registry.h>

class CustomFunctions : public FunctionRegistry {
public:
    Value distance_haversine(
        Value lat1, Value lon1,
        Value lat2, Value lon2
    ) {
        const double R = 6371; // Earth radius
        double dLat = (lat2.asDouble() - lat1.asDouble()) * M_PI / 180;
        double dLon = (lon2.asDouble() - lon1.asDouble()) * M_PI / 180;

        double a = sin(dLat/2) * sin(dLat/2) +
            cos(lat1.asDouble() * M_PI / 180) *
            cos(lat2.asDouble() * M_PI / 180) *
            sin(dLon/2) * sin(dLon/2);

        double c = 2 * asin(sqrt(a));
        return Value::fromDouble(R * c);
    }
};

// Register at startup
REGISTER_FUNCTION("CUSTOM_DISTANCE", &CustomFunctions::distance_haversine);
```

37.3 Plugin Architecture

Creating a Plugin

```
from themis_plugin_api import Plugin, Hook

class MyAnalyticsPlugin(Plugin):
    name = "my-analytics"
    version = "1.0.0"
    description = "Custom analytics for ThemisDB"

    def on_insert(self, collection, document):
        """Called after INSERT"""
        self.log_event('insert', collection, document)
        self.update_analytics(collection)

    def on_update(self, collection, old_doc, new_doc):
        """Called after UPDATE"""
        self.log_event('update', collection, new_doc)
        self.track_changes(old_doc, new_doc)

    def on_delete(self, collection, document):
        """Called after DELETE"""
        self.log_event('delete', collection, document)
        self.update_analytics(collection)

    def log_event(self, operation, collection, doc):
        event = {
            'operation': operation,
            'collection': collection,
            'timestamp': now(),
            'doc_id': doc['_id']
        }
        self.db.insert_event(event)

    def update_analytics(self, collection):
        # Compute analytics stats
        count = self.db.count(collection)
        avg_size = self.db.avg_document_size(collection)
        self.db.update_analytics({
            'collection': collection,
            'count': count,
            'avg_size': avg_size
        })
```

Plugin Configuration

```
plugins:
- name: my-analytics
  enabled: true
  config:
    event_log_collection: "analytics_events"
    update_interval_seconds: 60

- name: elasticsearch-sync
  enabled: true
  config:
    es_host: "localhost:9200"
    sync_interval_seconds: 5

- name: slack-notifications
  enabled: true
  config:
    webhook_url: "${SLACK_WEBHOOK_URL}"
    notify_on: ["high_latency", "errors", "replication_lag"]
```

37.4 Webhooks & Events

Webhook Configuration

```
-- Register webhook
FUNCTION create_webhook(url, collection, events) {
  RETURN INSERT {
    url: url,
    collection: collection,
    events: events, -- ["insert", "update", "delete"]
    active: true,
    created_at: NOW(),
    retry_count: 0,
    last_triggered: null
  } INTO webhooks
}

-- Setup:
create_webhook(
  'https://api.example.com/webhooks/themis',
  'users',
  ['insert', 'update']
)
```

Event Payload

```
{
  "event_id": "evt_abc123",
  "timestamp": "2026-01-01T09:00:00Z",
  "event_type": "insert",
  "collection": "users",
  "document": {
    "_id": "users/123",
    "_key": "123",
    "name": "Alice",
    "email": "alice@example.com"
  },
  "metadata": {
    "version": "1.3.1",
    "source": "direct_insert",
    "user_id": "admin_001"
  }
}
```

Webhook Handler Implementation

```
from flask import Flask, request
import hmac
import hashlib
import json

app = Flask(__name__)
WEBHOOK_SECRET = "your-secret-key"

@app.route('/webhooks/themis', methods=['POST'])
def handle_themis_webhook():
    # Verify signature
    signature = request.headers.get('X-Themis-Signature')
    body = request.get_data()

    expected_sig = hmac.new(
        WEBHOOK_SECRET.encode(),
        body,
        hashlib.sha256
    ).hexdigest()

    if not hmac.compare_digest(signature, expected_sig):
        return {'error': 'Invalid signature'}, 401

    # Process event
    event = request.json
    if event['event_type'] == 'insert':
        handle_user_insert(event['document'])
    elif event['event_type'] == 'update':
        handle_user_update(event['document'])

    return {'ok': True}, 200

def handle_user_insert(user):
    # Send welcome email
    send_email(user['email'], 'Welcome!')

    # Add to mailing list
    add_to_newsletter(user['email'])

def handle_user_update(user):
    # Update external systems
    sync_to_crm(user)
```

37.5 Data Type Extensions

Custom Scalar Type

```
// custom_types.cpp
class PhoneNumber {
public:
    std::string country_code;
    std::string area_code;
    std::string number;

    std::string toString() {
        return country_code + " (" + area_code + ") " + number;
    }

    static PhoneNumber fromString(const std::string& str) {
        // Parse format: "+1 (555) 1234567"
        PhoneNumber phone;
        // ... parsing logic
        return phone;
    }
};

// Register with ThemisDB type system
REGISTER_CUSTOM_TYPE("PhoneNumber", PhoneNumber);

// Use in AQL:
// INSERT {phone: PhoneNumber("+1 (555) 1234567")} INTO users
```

37.6 Cross-Database Synchronization

Bidirectional Sync with PostgreSQL

```
import psycopg2
import themis_client

class PostgresThemisSync:
    def __init__(self, themis_conn, pg_conn):
        self.themis = themis_client.connect(themis_conn)
        self.pg = psycopg2.connect(pg_conn)

    def sync_themis_to_postgres(self, collection, pg_table):
        """ThemisDB → PostgreSQL"""
        # Read from ThemisDB
        docs = self.themis.execute(
            f"FOR doc IN {collection} RETURN doc"
        )

        # Write to PostgreSQL
        cursor = self.pg.cursor()
        for doc in docs:
            cursor.execute(
                f"INSERT INTO {pg_table} VALUES (%s, %s, %s) "
                f"ON CONFLICT (_id) DO UPDATE SET data = %s",
                (doc['_id'], doc['_rev'], json.dumps(doc), json.dumps(doc))
            )

        self.pg.commit()
        cursor.close()

    def sync_postgres_to_themis(self, pg_table, collection):
        """PostgreSQL → ThemisDB"""
        cursor = self.pg.cursor()
        cursor.execute(f"SELECT * FROM {pg_table} WHERE updated_at > %s", (self.last_sync,))

        rows = cursor.fetchall()
        for row in rows:
            doc = self._row_to_doc(row)
            self.themis.execute(
                f"UPSERT {{'_id': {doc['_id']}}} "
                f"INSERT {doc} "
                f"INTO {collection}"
            )

        cursor.close()
```

Zusammenfassung

ThemisDB-Ökosystem bietet: -  **Native Integrations** - Elasticsearch, Kafka, Prometheus -  **Custom Functions** - AQL + C++ für Domain-Logik -  **Plugins** - Extensible hooks für Custom Behavior -  **Webhooks** - Event-driven integrations -  **Custom Types** - Domain-specific data types -  **Cross-DB Sync** - Bidirektional mit PostgreSQL, MongoDB -  **Open API** - Python, Go, JavaScript SDKs

Mit diesen Tools integrieren Sie ThemisDB in jedes bestehende System.

Kapitel 48: Kapitel 38: Observability & SRE Playbook

"Ohne Metriken, Logs und Traces bleiben Incidents Rätselraten. Observability ist die Brücke zwischen Symptom und Ursache."

Überblick

Dieses Kapitel liefert ein praktisches Observability- und SRE-Playbook für ThemisDB-Deployments. Es bündelt Metriken, Logs, Traces, Dashboards, SLOs, Alerts und Runbooks.

Was Sie lernen: - Kernmetriken (DB, System, Netzwerk) - Log-Formate, Parsing, Korrelation - Distributed Tracing für AQL Requests - Dashboards für Latenz, Fehler, Replikation, Speicherdruck - SLO/SLI-Definitionen und Error Budgets - Alerting-Design und Rauschreduktion - Runbooks für die häufigsten Störungen - Chaos- und GameDay-Checklisten

Voraussetzungen: Basiswissen aus Monitoring (Kapitel 19) und Troubleshooting (Kapitel 27).

```

flowchart TB
    subgraph "Data Sources"
        DB[ThemisDB]
        App[Application]
        Infra[Infrastructure]
    end

    subgraph "Collection"
        Metrics[Metrics<br/>Prometheus]
        Logs[Logs<br/>Loki]
        Traces[Traces<br/>Tempo]
    end

    subgraph "Storage"
        TSDB[(Time Series DB)]
        LogStore[(Log Storage)]
        TraceStore[(Trace Storage)]
    end

    subgraph "Visualization"
        Grafana[Grafana Dashboards]
    end

    subgraph "Alerting"
        AlertManager[Alert Manager]
        PagerDuty[PagerDuty]
        Slack[Slack]
    end

    DB --> Metrics
    DB --> Logs
    DB --> Traces

    App --> Metrics
    App --> Logs
    App --> Traces

    Infra --> Metrics
    Infra --> Logs

    Metrics --> TSDB
    Logs --> LogStore
    Traces --> TraceStore

    TSDB --> Grafana
    LogStore --> Grafana
    TraceStore --> Grafana

    TSDB --> AlertManager

```

```

AlertManager --> PagerDuty
AlertManager --> Slack

style DB fill:#4dabf7
style Grafana fill:#fab005
style AlertManager fill:#ff6b6b

```

```

graph TB
    App[ThemisDB] --> Metrics[Metrics<br/>Prometheus]
    App --> Logs[Logs<br/>Loki]
    App --> Traces[Traces<br/>Jaeger]

    Metrics --> Dashboard[Grafana Dashboard]
    Logs --> Dashboard
    Traces --> Dashboard

    Dashboard --> Alerts{Alerts}
    Alerts -->|SLO Breach| Incident[Incident]
    Alerts -->|OK| Monitor[Continue Monitoring]

    Incident --> Diagnose[Diagnose]
    Diagnose --> Mitigate[Mitigate]
    Mitigate --> Postmortem[Postmortem]
    Postmortem --> Improve[Improve Systems]

    style Alerts fill:#ff6b6b
    style Dashboard fill:#4facfe
    style Improve fill:#43e97b

```

Abb. 38.0: Observability-Säulen

38.1 Metriken (What to Measure)

Core DB Metriken

- Query Latenz (p50/p95/p99)
- Query Throughput (qps)
- Slow Queries count
- Active Connections
- Replication Lag (ms)
- WAL/Commit Queue Depth
- Memory Usage (RSS, Cache Hit Rate)

- Disk IO (IOPS, Latency, Queue Depth)
- Index Hit Rate
- Cache Evictions

System Metriken

- CPU: user/system/iowait
- Memory: used, available, page faults
- Disk: iops, latency, utilization
- Network: rx/tx bytes, errors, retransmits

AQL-Spezifisch

- Query Type Mix (read/write/graph/vector)
- Cursor Count / Leaks
- Transaction Duration
- Deadlocks detected
- Timeouts per operation

38.2 Logging

Strukturierte Logs

```
{
  "ts": "2026-01-01T10:00:00Z",
  "level": "INFO",
  "service": "themisdb",
  "component": "aql",
  "event": "query_executed",
  "trace_id": "abc123",
  "span_id": "def456",
  "duration_ms": 32,
  "collection": "users",
  "operation": "READ",
  "status": "ok"
}
```

Best Practices: - JSON Lines, ein Event pro Zeile - Pflichtfelder: ts, level, service, component, trace_id - Keine PII in Logs; IDs statt Freitext - Rotate & Compress (zstd)

Log-Pipeline

- Agent: Filebeat/FluentBit (TLS, backpressure)
- Parse: grok/json; enrich (host, cluster, env)
- Store: Loki/Elastic/OpenSearch
- Retention: 7-30 Tage (Hot), 90+ Tage (Cold/Glacier)

38.3 Tracing

OpenTelemetry Setup (Go API Layer)

```
// tracing.go
import "go.opentelemetry.io/otel"
import "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp"
import "go.opentelemetry.io/otel/sdk/trace"

func InitTracer(endpoint string) func() {
    exporter, _ := otlptracehttp.New(context.Background(), otlptracehttp.WithEndpoint(endpoint))
    tp := trace.NewTracerProvider(trace.WithBatcher(exporter))
    otel.SetTracerProvider(tp)
    return func() { _ = tp.Shutdown(context.Background()) }
}
```

AQL Trace-Injektion

- Propagiere `traceparent` Header vom API Gateway ins AQL Layer
- Schreibe `trace_id` in Query-Context → Log-Korrelation
- Sample Rate: 1-10% in Produktion; 100% in Staging

38.4 Dashboards (Grafana)

Latenz & Throughput

Panels: - Query p50/p95/p99 (stacked per operation) - QPS split by read/write/graph/vector - Error rate (% per op)

Replikation

- Lag per follower (ms)
- Failed replications (count)
- WAL queue depth

Ressourcen

- CPU (per node)
- Memory (rss, cache hit rate)
- Disk IOPS & Latency
- Network retransmits

Cache & Index

- Cache evictions/sec
 - Index hit rate
 - Slow queries over threshold
-

38.5 SLI/SLO & Error Budgets

Beispiel-SLIs

- Availability: 99.95% (HTTP 2xx/3xx / total)
- Latency: p99 < 200 ms für Reads, < 400 ms für Writes
- Durability: 0 Datenverlust (RPO <= 60s)
- Freshness: Replication Lag < 2s (p99)

Error Budget Policy

- Monatsbudget: $(1 - \text{SLO}) * \text{Zeit}$
 - Breach: Freeze Releases, Fokus auf Reliability
 - Burn Alerts: Warn bei 25/50/75% Verbrauch
-

38.6 Alerting Design

Ziele: Früh, präzise, wenig Rauschen.

- Multi-Window, Multi-Burn-Rate (1h/6h) für SLO Burn
- Symptom-basierte Alerts (p99 Latenz hoch, Error Rate > 1%)
- Ursache-Alerts nachrangig (Disk 95% voll)
- Deduping + Grouping im Alertmanager
- Quiet Hours + Ownership (Runbook-Link, Pager-Rotation)

Beispiele: - `latency_p99_gt_200ms` (for 15m & 6h) - `error_rate_gt_1pct` - `replication_lag_gt_2s` - `disk_util_gt_85pct` - `cache_evictions_spike`

38.7 Runbooks (Kurzfassung)

Hohe Latenz

- Check: p99 Read/Write? Bestimmte Collections?
- EXPLAIN Slow Query; Index vorhanden?
- Cache Hit Rate < 90%? Memory Druck?
- CPU iowait hoch? → Storage prüfen
- Rate-Limit/Backpressure aktivieren

Replication Lag

- Netzwerk: retransmits, bandwidth
- Follower CPU/IO bottleneck
- WAL queue depth; throttle writes
- Rebalance followers

OOM/Memory Pressure

- Cache Size begrenzen
- Off-Heap aufteilen (vector buffers)
- Identify large queries; add LIMIT/PROJECTION

Disk Full

- Log-Retention kürzen
 - Offload Cold Data (Tiered Storage)
 - Rebuild Indizes, Vacuum
-

38.8 Chaos & GameDays

- Failure Modes: Node down, Network partition, Disk full, High latency storage, Poison messages
 - Hypothesen definieren, erwartetes Verhalten notieren
 - Blast Radius klein starten (1 node, 10 min)
 - Erfolgskriterien: SLO halten? Auto-Heal? Alerts ausgelöst?
 - Nachbereitung: Learnings, Tickets, Fixes, erneutes Testen
-

38.9 Logging & Tracing Sampling Patterns

- Dynamic Sampling: Erhöhe Rate bei Errors

- Tail Sampling: Keep slow queries, drop fast
 - PII Scrubbing Filter vor Export
-

38.10 Capacity Planning

- Wachstumsrate Traffic & Daten
 - Headroom-Ziel: 40% CPU, 50% IO, 30% Memory
 - Load-Test vierteljährlich; extrapoliere 12 Monate
 - Trigger: >70% baseline → Scale-out
-

38.11 Oncall Playbook Essentials

- Runbook Link in jedem Alert
 - 2nd-level Escalation (DBA/SRE)
 - Kommunikationskanal: Incident Room + Status Page
 - Postmortem innerhalb 48h, Action Items mit Owner/ETA
-

Zusammenfassung

Observability bündelt Metriken, Logs, Traces und klare SLOs. Mit sauberen Dashboards, schlankem Alerting und erprobten Runbooks verkürzen Sie MTTR massiv und verhindern Blindflüge im Betrieb.

Kapitel 49: Kapitel 39: Performance Tuning Cookbook

"Performance ist kein Zufall, sondern eine Sammlung kleiner, konsistenter Entscheidungen. Dieses Kochbuch liefert die Rezepte."

Überblick

Ein praxisorientiertes Tuning-Kochbuch für ThemisDB. Jede Sektion enthält Symptom → Diagnose → Fix mit AQL-Beispielen und System-Tuning.

Was Sie lernen: - Schnelle Checklisten für Latenz, Durchsatz, Speicher, IO - Tuning für Queries, Indizes, Cache, Transactions - Storage/OS/Netzwerk-Optimierungen - Batching, Queueing, Backpressure - Vector- und Graph-Workload Tuning - Beispiel-Konfigurationen für Prod

```
flowchart TD
```

```
    Start[Performance Issue] --> Profile[Profile System]
```

```
    Profile --> CPU{CPU<br/>Bottleneck?}
```

```
    Profile --> Memory{Memory<br/>Bottleneck?}
```

```
    Profile --> Disk{Disk<br/>Bottleneck?}
```

```
    Profile --> Network{Network<br/>Bottleneck?}
```

```
    CPU -->|Yes| OptQuery[Optimize Queries]
```

```
    CPU -->|Yes| AddIndex[Add Indexes]
```

```
    Memory -->|Yes| IncCache[Increase Cache]
```

```
    Memory -->|Yes| OptDataStruct[Optimize Data Structures]
```

```
    Disk -->|Yes| SSD[Use SSD]
```

```
    Disk -->|Yes| Partition[Partition Data]
```

```
    Network -->|Yes| CompData[Compress Data]
```

```
    Network -->|Yes| BatchReq[Batch Requests]
```

```
    OptQuery --> Verify[Verify Improvement]
```

```
    AddIndex --> Verify
```

```
    IncCache --> Verify
```

```
    OptDataStruct --> Verify
```

```
    SSD --> Verify
```

```
    Partition --> Verify
```

```
    CompData --> Verify
```

```
    BatchReq --> Verify
```

```
    Verify --> Done[Done]
```

```
    style Start fill:#ff6b6b
```

```
    style Verify fill:#f093fb
```

```
    style Done fill:#43e97b
```

Abb. 39.0: Performance-Tuning-Workflow

39.1 Quick Tuning Checklist

- **Latenz hoch?** EXPLAIN, Index-Pfade, Projection pushdown, LIMIT
- **Durchsatz gering?** Batching, Parallelisierung, Verbindungspool erhöhen
- **CPU hoch?** Sort/Regex/Full-Scan reduzieren, Index nutzen, Cache-Hit prüfen
- **IO hoch?** Covering Index, Kompression (zstd), Cold Data auslagern
- **Memory hoch?** Cache begrenzen, Streaming statt Materializing, LIMIT
- **Lock/Deadlock?** Konsistente Lock-Order, kürzere Transaktionen

flowchart TD

```
START[Performance Problem] --> METRIC{Welche Metrik?}
```

```
METRIC -->|Hohe Latenz| LAT[Query EXPLAIN]
```

```
METRIC -->|Niedriger Throughput| THR[Connection Pool]
```

```
METRIC -->|Hohe CPU| CPU[Full Scan]
```

```
METRIC -->|Hohe Memory| MEM[Cache Size]
```

```
METRIC -->|Hohe I/O| IO[Covering Index]
```

```
LAT --> IDX{Index vorhanden?}
```

```
IDX -->|Nein| CREATE[Index erstellen]
```

```
IDX -->|Ja| PROJ[Early Projection]
```

```
THR --> POOL{Pool-Größe?}
```

```
POOL -->|Klein| INC[Pool vergrößern]
```

```
POOL -->|OK| BATCH[Batching nutzen]
```

```
CPU --> SCAN{Full Scan?}
```

```
SCAN -->|Ja| CREATE
```

```
SCAN -->|Nein| REGEX[Regex/Sort reduzieren]
```

```
MEM --> CACHE{Cache > 60%?}
```

```
CACHE -->|Ja| LIMIT[Cache begrenzen]
```

```
CACHE -->|Nein| STREAM[Streaming nutzen]
```

```
IO --> COV{Covering Index?}
```

```
COV -->|Nein| COVER[Covering Index]
```

```
COV -->|Ja| COMP[Kompression aktivieren]
```

```
CREATE --> TEST[Performance Test]
```

```
PROJ --> TEST
```

```
INC --> TEST
```

```
BATCH --> TEST
```

```
REGEX --> TEST
```

```
LIMIT --> TEST
```

```
STREAM --> TEST
```

```
COVER --> TEST
```

```
COMP --> TEST
```

```
TEST --> OK{Besser?}
```

```
OK -->|Ja| DONE[✓ Problem gelöst]
```

```
OK -->|Nein| METRIC
```

39.2 Query Patterns (Fixes)

Early Projection & Filter

```
-- ❌ FALSCH: Späte Projektion
FOR u IN users
  FILTER u.status == 'active'
  RETURN u -- großes Dokument

-- ✅ RICHTIG: Früh projizieren
FOR u IN users
  FILTER u.status == 'active'
  RETURN { _key: u._key, name: u.name, email: u.email }
```

Avoid N+1

```
-- ❌ FALSCH: Pro Zeile Subquery
FOR o IN orders
  RETURN {
    o: o,
    items: (
      FOR i IN order_items FILTER i.order_id == o._key RETURN i
    )
  }

-- ✅ RICHTIG: Pre-Aggregate
LET items_by_order = (
  FOR i IN order_items
    COLLECT order_id = i.order_id INTO items
  RETURN { order_id, items }
)

FOR o IN orders
  LET bundle = FIRST(FOR b IN items_by_order FILTER b.order_id == o._key RETURN b.items)
  RETURN { o, items: bundle }
```

Range Queries mit Index

```
CREATE INDEX idx_ts ON events(timestamp)

FOR e IN events
  FILTER e.timestamp >= @from AND e.timestamp <= @to
  RETURN e
```

Graph Traversal Tuning

```
-- Limit depth and fanout
FOR v, e, p IN 1..3 OUTBOUND 'users/alice' GRAPH 'social'
  PRUNE LENGTH(p.vertices) > 100 -- Begrenze
  FILTER p.edges[*].weight ALL >= 0.5
RETURN v
```

39.3 Batching & Parallelism

Batch Insert/Update

```
LET batch = @input_docs
INSERT batch INTO users -- 1 Transaktion
```

Client-Side Parallel

```
from concurrent.futures import ThreadPoolExecutor

def run_in_parallel(queries):
    with ThreadPoolExecutor(max_workers=8) as ex:
        return list(ex.map(client.execute, queries))
```

Backpressure

```
import asyncio

class QueueWorker:
    def __init__(self, maxsize=1000):
        self.q = asyncio.Queue(maxsize=maxsize)

    async def produce(self, items):
        for item in items:
            await self.q.put(item) # block when full

    async def consume(self, worker):
        while True:
            item = await self.q.get()
            await worker(item)
            self.q.task_done()
```

39.4 Cache & Memory

- Target: Cache Hit Rate > 90%
- Limit result set sizes (pagination)
- Use STREAMING_CURSOR for große Resultsets
- Avoid massive IN lists → temp set collection

```
-- Streaming Cursor
FOR doc IN large_collection
  FILTER doc.status == 'active'
  LIMIT 0, 100000
  OPTIONS {stream: true}
  RETURN doc
```

39.5 Storage & IO

Kompression

- `storage.compression = zstd`
- `wal.compression = zstd`
- Blockgröße 4-8KB testen

SSD/NVMe Tuning (Linux)

```
sudo nvme set-feature -f 2 -v 0 # Disable write cache if battery-backed
sudo nvme set-feature -f 0x0b -v 0x1 # Latency monitor enable
```

Filesystem

- ext4 mit `noatime`, `discard`, `nodiscard` für Performance abwägen
- XFS für hohe Parallelität

39.6 Netzwerk

- MTU prüfen (Jumbo Frames nur konsistent nutzen)
 - TCP: `net.ipv4.tcp_tw_reuse=1`, `tcp_fin_timeout=15`
 - Verbindungen poolen; Keep-Alive aktivieren
-

39.7 Vektor-Workloads

Index & Quantization

- HNSW: M=16-32, efConstruction=200, efSearch=64-128
- PQ: Codebooks 8-16, 8 Bit
- IVF: nlist $\sim \sqrt{N}$

Batch Embedding

```
chunks = [docs[i:i+32] for i in range(0, len(docs), 32)]
for chunk in chunks:
    vectors = embed(chunk)
    client.insert_vectors('emb', vectors)
```

ANN Warmup

- Preload top collections on startup
- Pin hot vectors in memory tier

39.8 Graph-Workloads

- Begrenze Fanout und Depth
- Precompute Communities (Louvain) für häufige Queries
- Materialisierte Pfade für Hot Paths

39.9 Transactions & Locking

- Kürzere Transaktionen; kleinere Batches
- Lock-Order strikt definieren
- Deadlock-Timeout niedrig (z.B. 5s) → Retry mit Backoff

```
import time

def with_retry(tx_func, attempts=3):
    for i in range(attempts):
        try:
            return tx_func()
        except Deadlock:
            time.sleep(0.1 * (2 ** i))
    raise
```

39.10 Configuration Templates

themis.conf (Performance Focus)

```
aql:
    stream_results: true
    max_query_memory_mb: 1024
    profiling_enabled: true

cache:
    size_mb: 8192
    eviction_policy: lru

wal:
    compression: zstd
    sync_interval_ms: 20

network:
    max_connections: 2000
    keepalive: 60
```

Linux sysctl

```
net.core.somaxconn = 1024
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_tw_reuse = 1
vm.swappiness = 10
vm.max_map_count = 262144
```

39.11 Benchmark Harness

```
import time

class BenchHarness:
    def __init__(self, client):
        self.client = client

    def run_case(self, name, query):
        start = time.time()
        result = self.client.execute(query)
        dur = (time.time() - start) * 1000
        return {"name": name, "duration_ms": dur, "rows": len(result)}

    def run_suite(self, cases):
        return [self.run_case(n, q) for n, q in cases]
```

Zusammenfassung

Dieses Kochbuch liefert schnelle Diagnosen und konkrete Fixes. Beginnen Sie mit EXPLAIN/PROFILE, optimieren Sie Filter/Projektion, nutzen Sie Indizes, batchen Sie Schreibvorgänge, begrenzen Sie Fanout, und sichern Sie Ressourcen durch Cache- und IO-Tuning. Wiederholen, messen, automatisieren.

Kapitel 50: Kapitel 40: Data Governance & Compliance

"Daten ohne Governance sind Haftungsrisiko. Governance schafft Vertrauen, Compliance macht es prüfbar."

Überblick

Ein praxisnahes Governance- und Compliance-Kapitel für ThemisDB: Policies, Zugriff, Maskierung, Retention, Audits, Privacy (GDPR), und Kontrollnachweise.

Was Sie lernen: - Governance-Framework (Policies, Ownership, Stewardship) - Access Control & Least Privilege - Data Classification & Masking - Lifecycle & Retention - Audit Trails & Evidence - GDPR/DSGVO: Rechte, Pseudonymisierung, Löschung - SOX/SOC2/ISO27001 Anforderungen - Kontroll-Checklisten und Runbooks

```
graph TB
    Data[Personal Data] --> Classification[Data Classification]

    Classification --> Public[Public Data]
    Classification --> Internal[Internal Data]
    Classification --> Confidential[Confidential Data]
    Classification --> Restricted[Restricted Data]

    Confidential --> Encrypt[Encryption Required]
    Restricted --> Encrypt

    Encrypt --> Access[Access Control]
    Access --> RBAC[RBAC Enforcement]

    RBAC --> Audit[Audit Logging]
    Audit --> Retention[Retention Policy]

    Retention --> Active[Active: 2 years]
    Retention --> Archive[Archive: 5 years]
    Retention --> Delete[Delete: after 7 years]

    style Confidential fill:#ff6b6b
    style Encrypt fill:#f093fb
    style Audit fill:#4facfe
```

Abb. 40.0: Data-Governance-Framework

40.1 Governance Operating Model

```

flowchart TB
    subgraph "Governance Framework"
        Policy[Policy Catalog]
        Owner[Data Owners]
        Steward[Data Stewards]
        Custodian[Data Custodians]
    end

    subgraph "Controls"
        Access[Access Control]
        Classify[Classification]
        Encrypt[Encryption]
        Audit[Audit Logs]
    end

    subgraph "Compliance"
        GDPR[GDPR/DSGVO]
        SOX[SOX]
        SOC2[SOC2]
        ISO[ISO27001]
    end

    subgraph "Operations"
        Monitor[Monitoring]
        Alert[Alerting]
        Response[Incident Response]
        Evidence[Evidence Collection]
    end

    Policy --> Access
    Policy --> Classify
    Policy --> Encrypt
    Policy --> Audit

    Owner --> Policy
    Steward --> Classify
    Custodian --> Encrypt

    Access --> GDPR
    Classify --> SOC2
    Encrypt --> ISO
    Audit --> SOX

    Audit --> Monitor
    Monitor --> Alert
    Alert --> Response
    Response --> Evidence

    Evidence --> Review[Quarterly Review]

```

```
Review --> Policy
```

```
style Policy fill:#4dabf7
style GDPR fill:#fa5252
style Evidence fill:#51cf66
```

- **Ownership:** Jede Collection hat Owner (Team), Steward (Data Quality), Custodian (Ops)
- **Policy Catalog:** Zugriff, Retention, Encryption, Backup, Sharing
- **Review Zyklen:** Quartalsweise Policy-Review, jährliche Controls-Audits
- **Change Control:** RFC → Peer Review → Approval → Rollout → Evidence

40.2 Data Classification

Stufen definieren (z.B. PUBLIC, INTERNAL, CONFIDENTIAL, STRICT).

```
classification:
  levels:
    - name: PUBLIC
      description: "Keine Sensibilität"
    - name: INTERNAL
      description: "Nur intern"
    - name: CONFIDENTIAL
      description: "PII/finanziell"
    - name: STRICT
      description: "Gesundheit, Strafverfolgung"
```

Regeln: - Standard = INTERNAL - CONFIDENTIAL/STRICT → Encryption + Access Approval - Masking im UI/Logs für CONFIDENTIAL/STRICT

40.3 Access Control & Least Privilege

- RBAC-Modelle (Reader, Writer, Admin, Auditor)
- JIT Access mit Ablauf (z.B. 24h)
- Break-Glass Accounts (mit separatem Audit)
- Quarterly Access Review

```
-- Grant role
INSERT {
  user_id: 'alice',
  role: 'reader',
  granted_by: 'security',
  expires_at: DATE_ADD(NOW(), 1, 'day')
} INTO access_grants
```

40.4 Data Masking

Dynamic Masking

```
FUNCTION mask_email(email) {
  RETURN CONCAT(SUBSTRING(email, 0, 2), '***', SUBSTRING(email, POSITION(email, '@')-1))
}

FOR user IN users
  RETURN {
    name: user.name,
    email: mask_email(user.email)
  }
```

Tokenization

```
import secrets

tokens = {}

def tokenize(value):
  token = secrets.token_hex(8)
  tokens[token] = value
  return token

def detokenize(token):
  return tokens.get(token)
```

40.5 Data Lifecycle & Retention

- **Ingest:** Klassifizierung, Validation, Encryption
- **Store:** Encryption-at-rest, Access Controls, Audit Logs

- **Use:** Masking, Purpose Limitation, Minimal Disclosure
- **Share:** Anonymize/Pseudonymize, DPA prüfen
- **Archive:** Cold Storage mit Verschlüsselung
- **Delete:** Right-to-be-forgotten Prozesse

Retention Beispiel:

```
retention:
  users: 7y
  logs: 180d
  audit_logs: 365d
  pii: 2y
```

40.6 Audit & Evidence

- Immutable Audit Log (Kapitel 36)
- Evidence Store: Tickets, Screenshots, CLI Output, Hashes
- Kontroll-Nachweise per Control-ID (ISO27001 Annex A, SOC2 CC)

```
- Policy: AC-3 v1.2
- Evidence:
  - access_grants export (signed)
  - quarterly review report (PDF)
  - IAM config screenshot
  - audit log hash chain verification
```

40.7 Privacy (GDPR)

Betroffenenrechte

- Auskunft, Berichtigung, Löschung, Einschränkung, Übertragbarkeit, Widerspruch


```
-- Right to Erasure (Pseudonymize)
FUNCTION gdpr_delete(user_id) {
  UPDATE {_id: 'users/' + user_id} WITH {
    name: 'Anonymized',
    email: null,
    phone: null,
    gdpr_deleted_at: NOW(),
    is_deleted: true
  } IN users

  REMOVE d IN audit_logs
  FILTER d.resource == CONCAT('users/', user_id)
}
```

Data Processing Register

- Zweck, Kategorien, Empfänger, Speicherorte, Löschfristen
- DPAs mit Prozessoren dokumentieren

40.8 Compliance Frameworks (SOX, SOC2, ISO27001)

- **SOX:** Change-Management, Segregation of Duties, Evidence für Finanzsysteme
- **SOC2:** Security/Availability/Confidentiality; Controls: Access, Change, Incident, Monitoring
- **ISO27001:** ISMS, Risikoanalyse, Controls Annex A (z.B. A.8 Asset Management, A.9 Access Control)

Kontroll-Mapping Beispiel:

- A.9.1.2 (Access Control): RBAC + Quarterly Review + JIT
- A.12.4.1 (Logging): Immutable Audit Log, 365d retention
- A.12.6.1 (Malware Protection): EDR auf DB-Hosts
- A.17.1.1 (Continuity): DR-Plan + Tests halbjährlich

40.9 Controls & Checklisten

```
## Access Controls
- [ ] RBAC Rollen definiert
- [ ] JIT Access aktiviert
- [ ] Break-glass dokumentiert
- [ ] Quarterly Reviews durchgeführt

## Data Protection
- [ ] Encryption at rest (AES-256)
- [ ] TLS 1.3 in transit
- [ ] Field-Level Encryption für PII
- [ ] Masking im UI und Logs

## Monitoring & Audit
- [ ] Immutable Audit Logs
- [ ] Alerting auf Policy-Verletzungen
- [ ] Evidence Store mit Hash-Kette
```

Zusammenfassung

Governance setzt klare Verantwortlichkeiten, Classification, Zugriffsprinzipien und Retention. Compliance liefert Nachweise gegenüber Auditoren und Gesetzgebern. Mit Masking, Audit Trails, JIT Access und robusten Löschprozessen bleibt ThemisDB konform und vertrauenswürdig.

Kapitel 51: Kapitel 41: Hands-on Labs – Deployment, Index-Tuning, Vector-Suche

"Lernen durch Tun: Diese Labs führen Sie Schritt für Schritt durch echte Produktionsaufgaben."

Überblick

Drei geführte Labs mit klaren Zielen, Schritt-für-Schritt-Anleitungen, Verifikation und Aufräumen.

Labs: - Lab A: Schnellstart-Deployment (Docker) + Smoke Tests - Lab B: Index-Tuning & Query-Optimierung mit EXPLAIN/PROFILE - Lab C: Vector-Suche End-to-End (Embeddings erzeugen, Indexieren, Query)

Voraussetzungen: Docker/WSL, themis_client CLI oder HTTP, Grundkenntnisse AQL.

```
graph LR
    Setup[Lab Setup] --> Lab1[Lab 1:<br/>Installation]
    Lab1 --> Lab2[Lab 2:<br/>Basic Queries]
    Lab2 --> Lab3[Lab 3:<br/>Multi-Model]
    Lab3 --> Lab4[Lab 4:<br/>Performance]
    Lab4 --> Lab5[Lab 5:<br/>Replication]
    Lab5 --> Final[Final Project]

    Lab1 --> Check1{Validate}
    Lab2 --> Check2{Validate}
    Lab3 --> Check3{Validate}
    Lab4 --> Check4{Validate}
    Lab5 --> Check5{Validate}

    Check1 -->|Pass| Lab2
    Check2 -->|Pass| Lab3
    Check3 -->|Pass| Lab4
    Check4 -->|Pass| Lab5
    Check5 -->|Pass| Final

    Check1 -->|Fail| Trouble1[Troubleshoot]
    Check2 -->|Fail| Trouble2[Troubleshoot]

    style Final fill:#43e97b
    style Setup fill:#4facfe
```

Abb. 41.0: Lab-Curriculum-Flow

Lab A: Deployment & Smoke Tests (30-45 min)

Ziel

- ThemisDB per Docker starten
- Healthcheck & Basiskonfiguration prüfen
- Erste Write/Read-Query ausführen

Schritte

1) Container starten

```
docker run -d --name themisdb -p 8529:8529 themisdb/server:latest
```

2) Healthcheck

```
curl -s http://localhost:8529/_admin/status | jq '.server.status'
```

Erwartet: `"ok"`

3) Admin-Login setzen (falls nötig)

```
curl -X POST http://localhost:8529/_admin/setup --data '{"password":"StrongPass!"}'
```

4) Smoke-Test Write/Read

```
-- write
INSERT { _key: "alice", name: "Alice" } INTO users
-- read
FOR u IN users FILTER u._key == "alice" RETURN u
```

5) Persistenz prüfen

```
docker restart themisdb
curl -s http://localhost:8529/_api/document/users/alice
```

Aufräumen

```
docker rm -f themisdb
```

Lab B: Index-Tuning & Query-Optimierung (40-60 min)

Ziel

- Langsame Query identifizieren
- EXPLAIN/PROFILE nutzen
- Index anlegen und Improvement messen

Dataset vorbereiten

```
FOR i IN 1..200000
  INSERT {
    _key: CONCAT('user_', i),
    email: CONCAT('user', i, '@example.com'),
    status: i % 5 == 0 ? 'inactive' : 'active',
    created_at: DATE_NOW()
  } INTO users
```

Baseline messen

```
PROFILE FOR u IN users
  FILTER u.email == 'user199999@example.com'
RETURN u
```

Notieren: p99 Latenz.

EXPLAIN prüfen

```
EXPLAIN FOR u IN users
  FILTER u.email == 'user199999@example.com'
RETURN u
```

Erwartet: CollectionScan (schlecht).

Index anlegen

```
CREATE INDEX idx_users_email ON users(email)
```

Nachher messen

```
PROFILE FOR u IN users  
  FILTER u.email == 'user199999@example.com'  
  RETURN u
```

Erwartet: IndexNode, stark reduzierte Latenz. Dokumentieren Sie Vor/Nach.

Optionale Optimierungen

- Projection pushdown: nur benötigte Felder returnen
- LIMIT 1 setzen, wenn eindeutig

Aufräumen

```
DROP INDEX users.idx_users_email  
TRUNCATE users
```

Lab C: Vector-Suche End-to-End (45-75 min)

Ziel

- Texte einbetten
- Vektor-Index aufbauen
- Ähnlichkeitssuche ausführen

Setup

1) Kollektion anlegen

```
CREATE COLLECTION articles
```

2) Beispiel-Dokumente einfügen

```

INSERT {
  _key: "art1",
  title: "Graph Algorithms",
  content: "Graphs, shortest paths, centrality",
  vector: null
} INTO articles
INSERT {
  _key: "art2",
  title: "Neural Networks",
  content: "Deep learning, backpropagation",
  vector: null
} INTO articles
INSERT {
  _key: "art3",
  title: "Databases",
  content: "Indexing, transactions, sharding",
  vector: null
} INTO articles

```

3) Embeddings erzeugen (Client-seitig, Beispiel Python)

```

from sentence_transformers import SentenceTransformer
import themis_client

client = themis_client.connect()
model = SentenceTransformer('all-MiniLM-L6-v2')

articles = client.execute("FOR a IN articles RETURN {key: a._key, text: a.content}")
for a in articles:
    vec = model.encode(a['text']).tolist()
    client.execute(
        "UPDATE { _key: @key } WITH { vector: @vec } IN articles",
        bind_vars={"key": a['key'], "vec": vec}
    )

```

4) Vektor-Index anlegen

```
CREATE VECTOR INDEX idx_articles_vec ON articles(vector) WITH {dimensions: 384, type: "hnsw"}
```

5) Ähnlichkeitssuche

```

LET query_vec = @query_vec
FOR doc IN articles
  SEARCH doc.vector ANN { query: query_vec, top_k: 3 }
  RETURN { _key: doc._key, title: doc.title, score: doc._score }

```

Bind Vars: `query_vec` = Embedding eines Suchtexts, z.B. "machine learning".

6) **Relevanz prüfen** - Erwartet: "Neural Networks" weit oben bei ML-Query - Score dokumentieren

Optionen

- efSearch erhöhen für bessere Recall
- PQ/IVF nutzen für größere Datenmengen
- Caching von Embeddings

Aufräumen

```
TRUNCATE articles
```

Checkliste (Abgehakt, wenn Lab bestanden)

- [] Lab A: Container-Start, Healthcheck, Smoke Query
- [] Lab B: EXPLAIN/PROFILE vor/nach, Index Speedup dokumentiert
- [] Lab C: Embeddings erzeugt, Vektor-Index erstellt, ANN Query ausgeführt

Nächste Schritte: Kombinieren Sie Labs: Deploy → Index-Tuning → Vector-Suche auf realen Datensätzen.

Kapitel 52: Appendix D: Feature Status Matrix

Version: 1.4.0-alpha

Stand: 6. Januar 2026

Kategorie: Feature-Übersicht und Implementierungsstatus

Zweck dieses Appendix

Dieser Appendix bietet eine vollständige, strukturierte Übersicht aller ThemisDB-Features mit aktuellem Implementierungsstatus, Dokumentationsverweisen und Roadmap-Planung. Dies dient als zentraler Referenzpunkt für:

- **Entwickler:** Welche Features sind verfügbar? Wo ist der Code?
- **Architekten:** Welche Capabilities hat ThemisDB? Was fehlt noch?
- **Projektmanager:** Was ist Production-Ready? Was ist geplant?
- **Nutzer:** Welche Funktionen kann ich nutzen?

Status-Definitionen

Symbol	Status	Bedeutung
	Production-Ready	Vollständig implementiert, getestet (>85% Coverage), dokumentiert, performance-validated
	MVP Complete	Kernfunktionalität vorhanden, stabil, aber ggf. noch Optimierungen ausstehend
	In Development	Aktiv in Entwicklung, teilweise funktional, noch nicht production-grade
	Design Phase	Spezifikation vorhanden, Implementierung geplant
	Planned	Auf Roadmap, noch keine Spezifikation
	Backlog	Idee/Anforderung dokumentiert, keine aktive Planung
	Alpha	Neu in v1.4.0-alpha, noch nicht production-ready
	Not Planned	Bewusst nicht im Scope

Zusammenfassung

Aktuelle Reife (Stand Jan 2026): - **Core Database:**  Production-Ready (85%+ Test Coverage) - **Multi-Model Support:**  Production-Ready (Relational, Graph, Vector, Document, Time-Series) - **Security & Compliance:**  Production-Ready (DSGVO, BSI C5, Audit Logging) - **Horizontal Scaling:**  Production-Ready (2/4/8-Node Clusters, 91% Efficiency) - **High Availability:**  Alpha (Hot Spare, WAL Replication) - **Voice Assistant:**  Alpha (Whisper STT, Piper TTS, Meeting Protocols, Call Center Automation) - **LLM/RAG Integration:**  Alpha (8 neue Features: Prefix/Response Caching, Multi-GPU, Paged Attention, LoRA, Grammar-Constrained Generation, RoPE Scaling, Vision) - **Performance Optimizations:**  Alpha (Flash Attention, Speculative Decoding, Continuous Batching) - **PostgreSQL Wire Protocol:**  Alpha (COPY, LISTEN/NOTIFY, Binary Format, Pipeline Mode) - **Enhanced Monitoring:**  Alpha (30+ neue Prometheus-Metriken für LLM, HA, Performance)

v1.4.0-alpha Feature-Highlights

LLM-Integration (Kapitel 17)

Feature	Status	Maturity	Documentation	Performance
Prefix Caching	 Alpha	Early	Chapter 17.12.1	75% cost savings
Response Caching	 Alpha	Early	Chapter 17.12.2	60-80% savings
Multi-GPU Support	 Alpha	Early	Chapter 17.12.3	4-8x throughput
Paged Attention	 Alpha	Early	Chapter 17.12.4	80% memory reduction
LoRA Support	 Alpha	Early	Chapter 17.12.5	99% less memory
Grammar-Constrained Generation	 Alpha	Early	Chapter 17.12.6	95-99% valid outputs
RoPE Scaling	 Alpha	Early	Chapter 17.12.7	4K → 32K context
Vision Support	 Alpha	Early	Chapter 17.12.8	Multimodal AI

Roadmap: - Beta (Feb 2026): Performance tuning, extended testing - GA (Mar 2026): Production-ready certification

Enterprise Features (Kapitel 10)

Feature	Status	Maturity	Documentation	Use Case
Voice Assistant	 Alpha	Early	Chapter 10.7	Call Center, Meeting Protocols
Multi-Tenancy	 Production-Ready	Stable	Chapter 10.1	SaaS Applications
RBAC	 Production-Ready	Stable	Chapter 10.1	Enterprise Security
Audit Logging	 Production-Ready	Stable	Chapter 10.1	Compliance

Roadmap (Voice Assistant): - Beta (Feb 2026): Enhanced language support, improved accuracy - GA (Mar 2026): Production-ready certification for enterprise use

Performance-Optimierungen (Kapitel 21)

Feature	Status	Maturity	Documentation	Performance
Flash Attention	 Alpha	Early	Chapter 20.9A.1	69% throughput, 37% memory
Speculative Decoding	 Alpha	Early	Chapter 20.9A.2	2-3x speedup
Continuous Batching	 Alpha	Early	Chapter 20.9A.3	176% throughput

Roadmap: - Beta (Feb 2026): Hardware compatibility testing - GA (Mar 2026): Production deployment guides

High Availability (Kapitel 16)

Feature	Status	Maturity	Documentation	Performance
Hot Spare	 Alpha	Early	Chapter 16.10.1	<5s failover
WAL Replication	 Alpha	Early	Chapter 16.10.2	Zero data loss
Multi-SSD WAL	 Alpha	Early	Chapter 16.10.2	<10% write overhead
Replication Slots	 Alpha	Early	Chapter 16.10.2	Lag monitoring

Roadmap: - Beta (Feb 2026): Extended failover testing, disaster recovery - GA (Mar 2026): Production HA certification

Monitoring & Observability (Kapitel 19)

Feature	Status	Maturity	Documentation	Metrics Count
LLM Metrics	 Alpha	Early	Chapter 19.6A.1	15+ metrics
Performance Metrics	 Alpha	Early	Chapter 19.6A.2	10+ metrics
HA Metrics	 Alpha	Early	Chapter 19.6A.3	8+ metrics
Grafana Dashboards	 Alpha	Early	Chapter 19.6A	3 dashboards
Alerting Rules	 Alpha	Early	Chapter 19.6A.4	12+ rules

Roadmap: - Beta (Feb 2026): Dashboard refinement, extended alerting - GA (Mar 2026): Complete observability stack

PostgreSQL Wire Protocol (Kapitel 31)

Feature	Status	Maturity	Documentation	Performance
COPY Protocol	 Alpha	Early	Chapter 31.9A.1	250K rows/s
LISTEN/NOTIFY	 Alpha	Early	Chapter 31.9A.1	Real-time CDC
Binary Format	 Alpha	Early	Chapter 31.9A.2	80% size reduction
Pipeline Mode	 Alpha	Early	Chapter 31.9A.2	17x throughput
Prepared Stmt Cache	 Alpha	Early	Chapter 31.9A.2	50x speedup
Vector Type Mapping	 Alpha	Early	Chapter 31.9A.3	Native pgvector
JSONB Support	 Alpha	Early	Chapter 31.9A.3	Full compatibility

Roadmap: - Beta (Feb 2026): Extended client library testing - GA (Mar 2026): Full PostgreSQL compatibility certification

Vollständige Feature-Matrix

Vollständige Feature-Matrix siehe: - Admin Tools: [feature_matrix.md](#) - Features Overview: [features_overview.md](#) - Sharding Status: Chapter 16 (Horizontal Scaling) - Security Status: Chapter 10 (Section 10.2) - Query Optimization: Chapter 15 (Section 15.4)

Feature-Maturity-Levels

Alpha (v1.4.0-alpha)

Definition: Neue Features in früher Entwicklung, für Feedback und Testing.

Charakteristiken: -  Kernfunktionalität implementiert -  Dokumentation vorhanden -  Noch nicht production-ready -  API kann sich ändern -  Performance noch nicht final optimiert

Empfohlene Nutzung: - Development/Staging Environments - Feature Evaluation - Feedback und Bug Reports

Beta (geplant: Feb 2026)

Definition: Features sind stabil, werden für Production vorbereitet.

Charakteristiken: -  Vollständig implementiert -  >80% Test Coverage -  Performance-validated -  Noch unter intensivem Testing -  Minor API changes möglich

Empfohlene Nutzung: - Pre-Production Testing - Pilot-Deployments - Performance Benchmarking

GA (General Availability - geplant: Mar 2026)

Definition: Production-ready, vollständig getestet und dokumentiert.

Charakteristiken: -  Production-Ready -  >85% Test Coverage -  Performance-validated -  Complete Documentation -  Stable API -  Long-term Support

Empfohlene Nutzung: - Production Deployments - Mission-Critical Workloads - Enterprise Applications

Implementierungsstatus v1.4.0-alpha

Gesamt-Übersicht

Kategorie	Features	Alpha	Beta	GA	Geplant
LLM Integration	6	6	0	0	0
Performance	3	3	0	0	0
High Availability	4	4	0	0	0
Monitoring	5	5	0	0	0
PostgreSQL Protocol	7	7	0	0	0
Gesamt	25	25	0	0	0

Prozentsatz: - Alpha Features: 100% (25/25) - Production-Ready: 0% (Target: 100% by Mar 2026)

Version History: - 1.4.0-alpha (2026-01-06): 25 neue Alpha-Features (LLM, Performance, HA, Monitoring, Protocol) - 1.3.0 (2025-12-30): Umfassendes Update mit allen Features bis Dez 2025

Kapitel 53: Appendix E: Incident Response Runbooks

"Ein Runbook erspart dir in der Krise 10 Stunden Debugging. Schreib sie jetzt, nutze sie später."

Überblick

Detaillierte Playbooks für die häufigsten Production-Incidents in ThemisDB: Symptome, Diagnose, Fixes, Verification, Kommunikation.

E.1 Incident: Query Latency > 1s (p99)

Symptom

- Alerts: `themis_query_latency_p99_gt_1000ms` für >5 Min
- User Report: "Suche dauert 10+ Sekunden"

Diagnose (5 min)

```
curl -s http://localhost:8529/_admin/stats | jq '.queries[-10:] | .[].latency_ms'
curl -s http://localhost:8529/_admin/slow-queries?threshold=500 | jq '.queries[0:3]'
curl -s http://localhost:8529/_api/collection/users/counts | jq '.count'
aql "EXPLAIN <query>"
```

Root Cause Analysis (Pick one)

A. Missing Index

```
-- EXPLAIN zeigt CollectionScan?
EXPLAIN FOR u IN users FILTER u.email == 'alice@example.com' RETURN u
-- Output: type: "CollectionScan" → NO INDEX

-- Fix:
CREATE INDEX idx_email ON users(email)

-- Verify:
EXPLAIN ... -- sollte IndexNode zeigen
```

B. Poor Index Choice

```
-- EXPLAIN zeigt wrong Index?
-- (z.B. idx_created_at bei Filter auf status)

-- Fix:
DROP INDEX users.old_index
CREATE INDEX idx_status_created ON users(status, created_at)

-- Query-Hint:
FOR u IN users
  FILTER u.status == 'active'
  SORT u.created_at DESC
  OPTIONS {indexHint: 'idx_status_created'}
  RETURN u
```

C. Huge Result Set (Memory Pressure)

```
-- ❌ Returns 1M rows
FOR u IN users
  FILTER u.status == 'active'
  RETURN u -- No LIMIT!

-- ✅ Paginated
FOR u IN users
  FILTER u.status == 'active'
  LIMIT @offset, 100
  RETURN u
```

D. Expensive Operations (Regex, Full-Text)

```
-- ❌ Regex auf 1M Docs
FOR u IN users
  FILTER u.name =~ '.*alice.*'
  RETURN u

-- ✅ Use Index + Full-Text
FOR u IN users
  FILTER u.status == 'active' -- Reduce set first
  SEARCH u.name IN FULLTEXT('alice') -- Then search
  RETURN u
```


Fix (1-5 min)

```
aql "CREATE INDEX idx_fix ON collection(field)"
```

Verify (2 min)

```
time aql "<query>" # Should be <100ms now
```

```
journalctl -u themis -n 20 | grep ERROR
```

Communicate

- Slack: "Query latency incident resolved: Added index on `collection.field` . P99 now 50ms."
- Incident Ticket: Mark RESOLVED, link to index creation timestamp

E.2 Incident: High Memory Usage (RSS > 80%)

Symptom

- Memory Usage Gauge: > 85% of available
- OOM Killer might start: `systemctl status themis | grep killed`

Diagnose (3 min)

```
free -h
top -p $(pgrep themisdb)

curl -s http://localhost:8529/_admin/cache | jq '.usage_mb'

curl -s http://localhost:8529/_admin/slow-queries?threshold=1 | jq '.queries[].memory_mb'

curl -s http://localhost:8529/_admin/cursors | jq 'length'
```

Root Cause Analysis

A. Large Result Set Materializing

```
-- ❌ Materializes all rows
FOR doc IN large_collection
  FILTER doc.type == 'report'
  RETURN doc

-- ✅ Stream results
FOR doc IN large_collection
  FILTER doc.type == 'report'
  OPTIONS {stream: true}
  RETURN doc
```

B. Cache Size Too Large

```
cache:
  size_mb: 16384 # Too big for 32GB system

cache:
  size_mb: 8192 # ~25% of total RAM
```

C. Cursor Leak (Clients not closing)

```
cursor = client.query("FOR d IN collection RETURN d")

with client.query("FOR d IN collection RETURN d") as cursor:
  for doc in cursor:
    process(doc)
```

Fix (5-10 min)

Quick Win: Restart

```
sudo systemctl restart themis
```

Better: Graceful Drain

```
curl -X POST http://localhost:8529/_admin/readonly

sleep 30

sudo systemctl restart themis

curl -X POST http://localhost:8529/_admin/writable
```

Verify

```
free -h
curl -s http://localhost:8529/_admin/cache | jq '.usage_mb'
```

E.3 Incident: Replication Lag > 5s

Symptom

- Alert: `themis_replication_lag_ms > 5000`
- Secondary reads stale data (>5s behind Primary)

Diagnose (2 min)

```
curl -s http://localhost:8529/_admin/replication/lag

curl -s http://localhost:8529/_admin/stats | jq '.writes_per_sec'

ssh follower-node "top -bn1 | head -15"

ping -c 10 follower-ip | tail -1
```

Root Cause Analysis

A. Follower Network Slow / Packet Loss

```
iperf3 -c follower-ip -t 10 # Should be >100 Mbps
mtr follower-ip # Check for packet loss
```

B. Follower Disk IO Bottleneck

```
iostat -x 1 5 | grep sda
```

C. Follower CPU Saturated

```
top -p $(pgrep themis)
```

Fix (5-30 min)

Short-term: Manual Catchup

```
aql "UPDATE config SET wal_read_buffer_mb = 256"  
  
sudo systemctl restart themis
```

Medium-term: Throttle Primary

```
replication:  
  max_writes_per_sec: 1000 # Slow down writes slightly  
  follower_lag_warn_ms: 3000
```

Verify

```
curl -s http://localhost:8529/_admin/replication/lag | jq '.lag_ms'
```

E.4 Incident: Deadlock Detected

Symptom

- Error: **ERROR: Deadlock detected (Transaction A ↔ B)**
- Client Retry: Usually succeeds on 2nd attempt

Diagnose (1 min)

```
curl -s http://localhost:8529/_admin/stats | jq '.deadlocks_total'
```

Root Cause Analysis

A. Lock Ordering Inconsistency

```
-- Transaction A:
BEGIN
  LOCK collection1
  LOCK collection2
COMMIT

-- Transaction B (opposite order - DEADLOCK):
BEGIN
  LOCK collection2
  LOCK collection1
COMMIT

-- Fix: Enforce strict lock ordering (e.g. alphabetical)
```

B. Long Transactions

```
-- ❌ 2-minute transaction
BEGIN
  FOR item IN items LIMIT 1000000
    UPDATE item WITH {updated: true} IN items
COMMIT

-- ✅ Break into batches
FOR batch_start IN 0..1000000 STEP 10000
  BEGIN
    FOR item IN items
      FILTER item._id >= batch_start AND item._id < batch_start + 10000
        UPDATE item WITH {updated: true}
    COMMIT
```

Fix (Immediate)

- **Client Side:** Add exponential retry (most deadlocks resolve on 2nd attempt)
- **Code:** Review and fix lock ordering (alphabetical, same order everywhere)
- **Monitoring:** If deadlocks > 10/day, escalate

Verify

```
watch -n 60 'curl -s http://localhost:8529/_admin/stats | jq ".deadlocks_total"'
```

E.5 Incident: Disk Space Critical (>95%)

Symptom

- Alert: `themis_disk_usage_percent > 95`
- Cannot write new data

Diagnose (2 min)

```
df -h /data/themis  
  
du -sh /data/themis/* | sort -h  
  
du -sh /var/log/themis*
```

Root Cause Analysis

A. WAL / Logs Explosion

```
ls -lh /data/themis/wal/  
ls -lh /var/log/themis*
```

B. Large Data Import (Staging)

```
aq1 "TRUNCATE staging_collection"  
aq1 "DROP COLLECTION temp_*
```

C. Backup Staging Not Cleaned

```
ls -lh /data/themis-backups/  
rm /data/themis-backups/backup-*.tar.gz
```

Fix (5-15 min)

Immediate:

```
journalctl --vacuum=2d

find /data/themis/wal -mtime +7 -delete

aql "TRUNCATE staging"

df -h /data/themis
```

Medium-term:

```
logging:
  retention_days: 14
  max_file_size_mb: 100

wal:
  retention_files: 20 # Keep last 20 files
```

E.6 Incident: Database Won't Start (Corruption)

Symptom

- Startup Error: `ERROR: Corrupted block at offset 12345`
- Service Down: Cannot restart

Diagnose (2 min)

```
journalctl -u themis -n 50 | tail -20

themisdb recover /data/themis

fsck /dev/sda1
```

Root Cause Analysis

A. Unclean Shutdown (Power Loss) - Fix: Run WAL recovery (automatic on restart)

B. Disk Corruption (Bad Sector) - Fix: Replace disk, restore from backup

C. RocksDB Internal Corruption (Rare) - Fix: Backup data, rebuild database

Fix

Option 1: Automatic Recovery (Most Common)

```
sudo systemctl start themis
```

Option 2: Manual WAL Recovery

```
themisdb recover --wal-only /data/themis  
sudo systemctl start themis
```

Option 3: Restore from Backup

```
sudo systemctl stop themis  
tar -xzf /backup/themis-2026-01-01.tar.gz -C /data/  
sudo chown -R themis:themis /data/themis  
sudo systemctl start themis
```

Verify

```
journalctl -u themis -n 20  
  
aql "RETURN COUNT(*) FROM users"
```

E.7 Incident: Security Alert: Unauthorized Access Attempt

Symptom

- Alert: `themis_auth_failure_rate > 5_per_minute`
- Logs: `AUTH FAILED: user='admin' from_ip='1.2.3.4' password_attempts=10`

Diagnose (1 min)

```
curl -s http://localhost:8529/_admin/audit?event=auth_failure&last_1h | jq '.events[0:10]  
  
cat /var/log/themis/auth.log | grep "FAILED" | cut -d' ' -f6 | sort | uniq -c | sort -rn  
  
curl -s http://localhost:8529/_admin/users/admin | jq '.locked'
```


Fix (2 min)

Immediate:

```
sudo ufw insert 1 deny from 1.2.3.4

aql "UPDATE {name: 'admin'} WITH {locked: true, locked_reason: 'Brute force attempt'} IN users"

aql "UPDATE {name: 'admin'} WITH {mfa_required: true} IN users"
```

Escalation: - Notify security team - Create incident ticket - Schedule post-mortem

Verify

```
watch "tail -20 /var/log/themis/auth.log | grep FAILED | wc -l"
```

E.8 Incident Severity Levels & Escalation

Severity 1 (Critical - All Hands)

- Database completely down (no connections)
- Data corruption/loss detected
- All writes failing

Escalation: On-call Lead + CTO + Data Team

Severity 2 (High - Urgent)

- Performance severely degraded (>10s latency)
- Replication lag > 30s
- Memory/Disk pressure critical

Escalation: On-call Engineer + Manager

Severity 3 (Medium - Standard)

- Single query slow (1-5s)
- Moderate lag (5-30s)
- Non-critical errors in logs

Escalation: On-call Engineer

Severity 4 (Low - Backlog)

- Single error/retry
- Performance degraded slightly
- Informational alerts

Escalation: Backlog Ticket

E.9 Communication Template

Incident Declared

 INCIDENT: Query Latency High
Status: INVESTIGATING
Severity: SEV-2
Started: 2026-01-01 12:00:00 UTC
Impact: 5-10% of users experiencing slow search

Root Cause Found

 ROOT CAUSE IDENTIFIED: Missing index on users.email
Fix: CREATE INDEX idx_email ON users(email)
ETA Resolution: 5 minutes

Resolved

 INCIDENT RESOLVED
Resolution: Index created, queries now <100ms
Duration: 15 minutes
Post-mortem: Thursday 10am

Summary

Keep this runbook near your on-call terminal. Most incidents fall into the categories above. When an alert fires: 1. **Navigate to right section** 2. **Run Diagnose commands** 3. **Match Root Cause** 4. **Execute Fix** 5. **Verify & Communicate**

Average resolution time with this playbook: **10-15 minutes** vs **60+ minutes** without.

Kapitel 54: Appendix F: AQL Cheat Sheet & Quick Reference

"Every expert was once a novice. This sheet shortens that journey."

Overview

Quick reference for common AQL patterns, syntax, and idioms. For detailed docs, see Chapter 28.

F.1 Basic Syntax

Collections & Documents

```
-- Create Collection
CREATE COLLECTION users

-- Insert
INSERT {name: 'Alice', email: 'alice@example.com'} INTO users

-- Read
FOR u IN users RETURN u

-- Update
UPDATE 'users/123' WITH {status: 'active'} IN users

-- Delete
REMOVE 'users/123' IN users

-- Truncate (danger!)
TRUNCATE users
```

Filtering

```
-- Simple equality
FOR u IN users FILTER u.status == 'active' RETURN u

-- Comparison operators
FILTER u.age > 18 AND u.age < 65
FILTER u.salary >= 50000
FILTER u.created_at < DATE_NOW()

-- IN operator
FILTER u.status IN ['active', 'pending', 'approved']

-- String operations
FILTER u.name LIKE 'A%'           -- SQL-like wildcard
FILTER u.email =~ '^.*@example\.com$' -- Regex
FILTER CONTAINS(u.bio, 'engineer') -- Substring

-- Null checks
FILTER u.middle_name != NULL
FILTER u.phone_number == NULL

-- Range
FILTER u.score >= 100 AND u.score <= 200 -- vs RANGE below
```

Sorting & Limiting

```
-- Order results
FOR u IN users
  SORT u.created_at DESC, u.name ASC
  RETURN u

-- Limit results
FOR u IN users
  SORT u.created_at DESC
  LIMIT 10          -- First 10
  RETURN u

-- Pagination
FOR u IN users
  SORT u.created_at DESC
  LIMIT @offset, @limit -- OFFSET, LIMIT
  RETURN u

-- Range operator (efficient for sorted data)
FOR u IN users
  RANGE u.score, 100, 200
  RETURN u
```

Projection (Select Fields)

```
-- All fields
RETURN u

-- Specific fields
RETURN {name: u.name, email: u.email}

-- Computed fields
RETURN {
  name: u.name,
  age_years: DATE_DIFF(u.birth_date, DATE_NOW(), 'y'),
  email_masked: CONCAT(SUBSTRING(u.email, 0, 3), '***')
}

-- Nested structure
RETURN {
  user: {name: u.name, id: u._id},
  contact: {email: u.email, phone: u.phone}
}
```

F.2 Advanced Filtering

Aggregation Filter

```
-- Count documents
FOR u IN users
  COLLECT WITH COUNT INTO count
  RETURN {total_users: count}

-- Group and count
FOR u IN users
  COLLECT status = u.status WITH COUNT INTO count
  RETURN {status, count}

-- Group with aggregation
FOR o IN orders
  COLLECT customer_id = o.customer_id
  AGGREGATE total = SUM(o.amount), cnt = COUNT(1)
  RETURN {customer_id, total, cnt}
```

Complex Filtering with Subqueries

```
-- Filter by aggregated value
FOR u IN users
  LET order_count = (
    FOR o IN orders FILTER o.user_id == u._id
    RETURN o
  ) | LENGTH
  FILTER order_count > 5
  RETURN {name: u.name, orders: order_count}

-- IN with subquery
FOR u IN users
  FILTER u._id IN (
    SELECT DISTINCT o.user_id FROM orders o WHERE o.status = 'completed'
  )
  RETURN u
```

Array Operations

```
-- Check array contains
FOR u IN users
  FILTER 'premium' IN u.roles
  RETURN u

-- Array length
FILTER LENGTH(u.tags) > 3

-- Array flatten
FOR u IN users
  LET all_ids = FLATTEN(u.related_user_ids)
  RETURN {name: u.name, related_count: LENGTH(all_ids)}

-- Array filtering
FOR u IN users
  LET recent_orders = u.orders[* FILTER CURRENT.date > DATE_SUBTRACT(DATE_NOW(), 30, 'd')]
  RETURN {name: u.name, recent_count: LENGTH(recent_orders)}
```

F.3 Joins & Relationships

INNER JOIN (Graph Traversal)

```
-- Simple traverse
FOR u IN users
  FOR o IN orders FILTER o.user_id == u._id
  RETURN {user: u.name, order_id: o._id}

-- With graph edges
FOR v, e, p IN 1..3 OUTBOUND users/123 GRAPH 'social'
  RETURN {vertex: v.name, edge: e.type, path: p.vertices[*]._id}

-- Shortest path
FOR path IN SHORTEST_PATH('users/123' TO 'users/456' GRAPH 'social')
  RETURN path
```


LEFT JOIN (Keep unmatched)

```
-- Users even if no orders
FOR u IN users
  LET orders = (FOR o IN orders FILTER o.user_id == u._id RETURN o)
  RETURN {user: u.name, order_count: LENGTH(orders), orders}
```

Multiple joins

```
FOR u IN users
  LET orders = (FOR o IN orders FILTER o.user_id == u._id RETURN o)
  LET addresses = (FOR a IN addresses FILTER a.user_id == u._id RETURN a)
  FILTER LENGTH(orders) > 0 AND LENGTH(addresses) > 0
  RETURN {user: u.name, order_count: LENGTH(orders), address_count: LENGTH(addresses)}
```

F.4 Transactions

Basic Transaction

```
BEGIN
  INSERT {name: 'Bob', balance: 100} INTO accounts
  INSERT {from: 'Alice', to: 'Bob', amount: 50} INTO transfers
COMMIT
```

With Error Handling

```
BEGIN
  LET update = UPDATE 'accounts/alice'
  WITH {balance: (SELECT account.balance - 50 FROM accounts FILTER _id == 'accounts/alice')}
  IN accounts

  IF u.balance < 50 THEN
    ABORT "Insufficient funds"
  ELSE
    INSERT {from: 'alice', to: 'bob', amount: 50} INTO transfers
    COMMIT
  ENDIFF
```

F.5 Vector Search

Simple Vector Search

```
-- Create vector index
CREATE INDEX idx_embedding ON articles(embedding)

-- Vector search (nearest neighbors)
FOR article IN articles
  FILTER DISTANCE(article.embedding, @query_embedding) < @threshold
  SORT DISTANCE(article.embedding, @query_embedding)
  LIMIT 10
  RETURN {title: article.title, distance: DISTANCE(article.embedding, @query_embedding)}

-- With parameters
db.query("""
  FOR article IN articles
    FILTER DISTANCE(article.embedding, @query_vec) < @threshold
    SORT DISTANCE(article.embedding, @query_vec)
    LIMIT @top_k
    RETURN article
  """, bind_vars={
    'query_vec': [0.1, -0.2, 0.3, ...],
    'threshold': 0.5,
    'top_k': 5
  })
```

Hybrid Search (Vector + Text)

```
-- Combine vector + full-text
FOR article IN articles
  FILTER DISTANCE(article.embedding, @query_vec) < @threshold
  SEARCH article.title IN FULLTEXT(@query_text)
  SORT BM25(article) DESC -- Relevance score
  RETURN {title: article.title, score: BM25(article)}
```

F.6 Graph Queries

Breadth-First Traversal

```
-- Get all friends of friends
FOR v IN 0..2 OUTBOUND 'users/alice' GRAPH 'social'
  RETURN {name: v.name, hops: LENGTH(p) - 1}

-- Prevent cycles
FOR v IN 1..3 OUTBOUND 'users/alice' GRAPH 'social'
  FILTER v._key NOT IN ['alice'] -- Don't revisit start node
  RETURN v
```

Depth-First with Conditions

```
-- Traverse until condition met
FOR v IN OUTBOUND 'users/alice' GRAPH 'social'
  FILTER v.status != 'banned'
  LIMIT 100
  RETURN v
```

Community Detection

```
-- Find clusters of connected users
FOR start IN users
  LET connected = (
    FOR v IN OUTBOUND start GRAPH 'social'
    RETURN DISTINCT v._id
  )
  COLLECT id = start._id, group = connected
  RETURN {user: id, cluster_size: LENGTH(group)}
```

F.7 Time-Series Queries

Bucket by Time

```
-- Sales per day
FOR order IN orders
  COLLECT date = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')
  AGGREGATE total = SUM(order.amount), cnt = COUNT(1)
  SORT date
  RETURN {date, total, cnt}

-- Sales per hour (last 24h)
FOR order IN orders
  FILTER order.created_at > DATE_SUBTRACT(DATE_NOW(), 1, 'd')
  COLLECT hour = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd %hh:00')
  AGGREGATE total = SUM(order.amount)
  RETURN {hour, total}
```

Moving Average

```
-- 7-day moving average of daily sales
FOR day IN 0..30
  LET current_date = DATE_SUBTRACT(DATE_NOW(), day, 'd')
  LET week_start = DATE_SUBTRACT(current_date, 7, 'd')
  LET sales = (
    FOR o IN orders
      FILTER o.created_at >= week_start AND o.created_at <= current_date
      RETURN o.amount
  )
  COLLECT date = DATE_FORMAT(current_date, '%yyyy-%mm-%dd')
  AGGREGATE avg = AVG(sales)
  SORT date
  RETURN {date, moving_avg_7d: avg}
```

F.8 Window Functions

Row Numbers & Ranking

```
-- Rank users by sales
FOR u IN users
  LET total_sales = (
    SELECT SUM(amount) AS total FROM orders WHERE customer_id == u._id
  )[0].total
  SORT total_sales DESC
  RETURN {
    rank: @row_number,
    name: u.name,
    total_sales
  }

-- Cumulative sum
FOR o IN orders
  SORT o.created_at
  COLLECT WITH COUNT INTO cnt
  AGGREGATE cumulative = SUM(o.amount)
  RETURN {cumulative, running_total: @row_number}
```

F.9 Common Patterns

Pagination with Cursor

```
-- Cursor-based pagination (efficient for large datasets)
FOR u IN users
  FILTER u._key > @cursor
  SORT u._key
  LIMIT 20
  RETURN u

-- No OFFSET (O(1) instead of O(n))
```

Deduplication

```
-- Get distinct values
FOR u IN users
  COLLECT email = u.email
  RETURN email

-- Get first of each group
FOR u IN users
  COLLECT category = u.category
  INTO group = u
  RETURN {category, first_user: group[0]}
```

Batch Operations

```
-- Batch insert
FOR item IN @items
  INSERT item INTO products

-- Batch update
FOR item IN items
  FILTER item.status == 'pending'
  UPDATE item WITH {status: 'processed'} IN items
```

Error Handling

```
-- TRY-CATCH
BEGIN
  LET result = (
    TRY
      FOR u IN users FILTER u._id == @user_id RETURN u
    CATCH err
      RETURN {error: err.message}
  )
  RETURN result
COMMIT
```

F.10 Performance Tips

Query Optimization Checklist

- ✓ Use FILTER before SORT
- ✓ Use LIMIT to reduce result set
- ✓ Add indexes on FILTER/SORT fields
- ✓ Use COLLECT for aggregations (not manual loops)
- ✓ Avoid FULLTEXT searches on tiny datasets
- ✓ Stream large results (avoid materializing)
- ✓ Use bind parameters (@var) not inline values

Slow Query Patterns to Avoid

```
-- ✗ SLOW: Regex without index
FILTER u.email =~ '.*@gmail\.com'

-- ✓ FAST: Use LIKE with index
CREATE INDEX idx_email ON users(email)
FILTER u.email LIKE '%@gmail.com'

-- ✗ SLOW: Expression in FILTER
FILTER YEAR(u.birth_date) == 1990

-- ✓ FAST: Use range on indexed field
CREATE INDEX idx_birth ON users(birth_date)
FILTER u.birth_date >= '1990-01-01' AND u.birth_date < '1991-01-01'
```

F.11 Built-in Functions (Most Common)

String Functions

Function	Example	Returns
<code>CONCAT()</code>	<code>CONCAT('Hello', ' ', 'World')</code>	<code>'Hello World'</code>
<code>CONTAINS()</code>	<code>CONTAINS('Hello', 'ell')</code>	<code>true</code>
<code>SUBSTRING()</code>	<code>SUBSTRING('Hello', 0, 3)</code>	<code>'Hel'</code>
<code>LENGTH()</code>	<code>LENGTH('Hello')</code>	<code>5</code>
<code>UPPER()</code>	<code>UPPER('hello')</code>	<code>'HELLO'</code>
<code>LOWER()</code>	<code>LOWER('HELLO')</code>	<code>'hello'</code>
<code>SPLIT()</code>	<code>SPLIT('a,b,c', ',')</code>	<code>['a', 'b', 'c']</code>

Date/Time Functions

Function	Example	Returns
<code>DATE_NOW()</code>	<code>DATE_NOW()</code>	Current timestamp
<code>DATE_FORMAT()</code>	<code>DATE_FORMAT(d, '%yyyy-%mm-%dd')</code>	Formatted date
<code>DATE_ADD()</code>	<code>DATE_ADD('2025-01-01', 5, 'd')</code>	Date + 5 days
<code>DATE_SUBTRACT()</code>	<code>DATE_SUBTRACT(d, 1, 'm')</code>	Date - 1 month
<code>DATE_DIFF()</code>	<code>DATE_DIFF(d1, d2, 'd')</code>	Days between

Math Functions

Function	Example	Returns
<code>SUM()</code>	<code>SUM(o.amount)</code>	Sum of values
<code>AVG()</code>	<code>AVG(o.amount)</code>	Average
<code>MIN()</code> / <code>MAX()</code>	<code>MIN(values)</code>	Min/Max
<code>ROUND()</code>	<code>ROUND(3.7)</code>	4
<code>FLOOR()</code> / <code>CEIL()</code>	<code>FLOOR(3.7)</code>	3
<code>ABS()</code>	<code>ABS(-5)</code>	5
<code>SQRT()</code>	<code>SQRT(16)</code>	4

Array Functions

Function	Example	Returns
<code>LENGTH()</code>	<code>LENGTH([1, 2, 3])</code>	3
<code>APPEND()</code>	<code>APPEND(arr, item)</code>	Array + item
<code>REVERSE()</code>	<code>REVERSE([1, 2, 3])</code>	[3, 2, 1]
<code>UNIQUE()</code>	<code>UNIQUE([1, 1, 2])</code>	[1, 2]
<code>FLATTEN()</code>	<code>FLATTEN([[1, 2], [3]])</code>	[1, 2, 3]
<code>SLICE()</code>	<code>SLICE([1, 2, 3, 4], 1, 3)</code>	[2, 3]

F.12 Index Types

```
-- Unique Index
CREATE INDEX idx_email UNIQUE ON users(email)

-- Compound Index (Multiple fields)
CREATE INDEX idx_status_date ON orders(status, created_at)

-- Partial Index (Conditional)
CREATE INDEX idx_active_users ON users(name)
  FILTER u.status == 'active'

-- TTL Index (Auto-delete)
CREATE INDEX idx_temp ON temp_data(created_at)
  WITH {ttl: 3600} -- Auto-delete after 1 hour

-- Full-Text Index
CREATE FULLTEXT INDEX idx_content ON articles(body)
```

Summary

Bookmark this section! Most queries can be built from these patterns. When stuck: 1. **Check pattern above** 2. **Run EXPLAIN to verify optimization** 3. **See Chapter 28 for detailed docs**

Kapitel 55: Appendix G: Configuration Reference

"Good defaults get you 90% of the way. This reference handles the other 10%."

Overview

Complete reference for all ThemisDB configuration options. Default values optimized for single-node deployments.

G.1 Main Configuration File (themis.conf)

Location & Format

Server Settings

port

```
server:
  port: 8529 # Default: 8529 (0x2149)
  # Range: 1024-65535
  # Tip: Use ports 8529-8539 to run multiple instances
```

bind_address

```
server:
  bind_address: "127.0.0.1" # Default: localhost only
  # "0.0.0.0" → Listen on all interfaces
  # Tip: Use localhost in dev, 0.0.0.0 in containerized deployments
```

max_connections

```
server:
  max_connections: 1000 # Default: 1000
  # Depends on file descriptor limit (ulimit -n)
  # Scaling rule: 1 connection = ~10 KB memory overhead
  # If 10k connections: ~100 MB RAM just for connection state
```

request_timeout_ms

```
server:
  request_timeout_ms: 30000 # Default: 30 sec
  # Queries longer than this abort with timeout
  # Long-running imports: increase to 300000 (5 min)
```

enable_http2

```
server:
  enable_http2: true # Default: true
  # Set false for testing/debugging with curl
```

G.2 Database Settings

data_directory

```
database:
  data_directory: "/var/lib/themis" # Default
  # Must be on fast storage (SSD preferred)
  # Recommendation: NVMe > SSD > HDD (in order of performance)
  # Min free space: 2x database size
```

engine_type

```
database:
  engine_type: "rocksdb" # Only option (default)
  # RocksDB is LSM-tree optimized for writes
  # 45k-60k writes/sec on modern hardware
```

wal_enabled

```
database:
  wal_enabled: true # Default: true (REQUIRED for ACID)
  # Set false ONLY for ephemeral/test deployments
```

wal_directory

```
database:
  wal_directory: "/var/lib/themis/wal" # Default
  # Tip: Put on separate disk from data for durability
```

wal_sync_mode

```
database:
  wal_sync_mode: "fsync" # Default: fsync (safe)
  # Options:
  #   "fsync"    → Durability: ✓✓✓ (safe) | Speed: ✓ (slowest)
  #   "os"       → Durability: ✓✓ (eventual) | Speed: ✓✓
  #   "none"     → Durability: ✓ (risky) | Speed: ✓✓✓ (fastest)
  # Recommendation: fsync for production, os for staging, none for local dev
```

compression_algorithm

```
database:
  compression_algorithm: "zstd" # Default: zstd
  # Options: "zstd" (best) | "lz4" (fast) | "snappy" | "none"
  # Recommendation: zstd for storage-constrained, lz4 for latency-critical
```

G.3 Cache Settings

cache_size_mb

```
cache:
  size_mb: 2048 # Default: 2048 (2 GB)
  # Rule of thumb: 25-30% of total system RAM
  # Example: 32 GB system → 8-10 GB cache
  # Observability: curl http://localhost:8529/_admin/cache
```

cache_mode

```
cache:
  mode: "lru" # Default: lru (Least Recently Used)
  # Options: "lru" | "lfu" (Least Frequently Used)
  # LRU: Better for time-dependent patterns
  # LFU: Better for popularity-based patterns
```

block_cache_size_mb

```
cache:
  block_cache_size_mb: 1024 # Default: 1024
  # RocksDB block cache (L1 cache for block decompression)
  # Typically 50% of cache_size_mb
```

G.4 Index Settings

default_index_type

```
indexing:
  default_index_type: "btree" # Default: btree
  # Options: "btree" | "hash" | "skiplist"
  # btree: Ordered access (good for range queries)
  # hash: Point lookups only (fastest for equality)
  # skiplist: Ordered with faster random access than btree
```

vector_index_default

```
indexing:
  vector_index_default: "hnsw" # Default: hnsw
  # Options: "hnsw" (Hierarchical Navigable Small Worlds)
  # Settings:
  #   m: 16 # Connections per layer (16-64, default 16)
  #   ef_construct: 200 # Quality during construction (default)
  #   ef_search: 100 # Quality during search (default)
```

G.5 Query Settings

query_timeout_ms

```
query:
  timeout_ms: 30000 # Default: 30 sec
  # Queries exceeding this timeout abort
  # Set to -1 for no timeout (not recommended)
```

query_cache_enabled

```
query:
  cache_enabled: true # Default: true
  # Caches query plans (not results)
  # Reduces compilation overhead for repeated queries
```

max_query_result_size_mb

```
query:
  max_query_result_size_mb: 512 # Default: 512
  # Result sets larger than this fail
  # Reason: Prevents memory bombs from buggy queries
  # Fix: Use LIMIT or OFFSET to paginate results
```

G.6 Replication Settings

replication_enabled

```
replication:
  enabled: true # Default: true
  # false for standalone instances
```

replication_role

```
replication:
  role: "primary" # or "follower"
  # primary: Accepts writes, logs all changes
  # follower: Read-only, applies logs from primary
```

follower_primary_address

```
replication:
  follower_primary_address: "primary.example.com:8529" # Only for followers
  # Follower connects to primary at this address
```

replication_buffer_size_mb

```
replication:
  buffer_size_mb: 256 # Default: 256
  # Follower buffer for incoming changes
  # Larger = handle larger write spikes
```

replication_sync_interval_ms

```
replication:
  sync_interval_ms: 100 # Default: 100
  # How often to sync WAL changes to follower
  # Lower = lower latency, higher CPU
  # 100ms is good for most workloads
```

G.7 Security Settings

authentication_enabled

```
security:
  authentication:
    enabled: true # Default: true
    # false: No authentication (dev-only!)
```

jwt_secret

```
security:
  authentication:
    jwt_secret: "${JWT_SECRET}" # Must set via env var
    # Min 32 characters, high entropy
    # Generate: openssl rand -hex 32
```


jwt_expiry_minutes

```
security:
  authentication:
    jwt_expiry_minutes: 1440 # Default: 24 hours
    # Tokens expire after this time
    # Shorter = better security, more refresh overhead
```

tls_enabled

```
security:
  tls:
    enabled: true # Default: true
    # false only for insecure local dev
```

tls_cert_path

```
security:
  tls:
    cert_path: "/etc/themis/certs/server.crt"
    # X.509 certificate (RSA 4096 recommended)
```

tls_key_path

```
security:
  tls:
    key_path: "/etc/themis/certs/server.key"
    # Private key (must have 0600 permissions)
```

tls_min_version

```
security:
  tls:
    min_version: "1.3" # Default: "1.3"
    # Options: "1.2" (legacy) | "1.3" (modern)
    # Recommendation: "1.3" for new deployments
```

mfa_required

```
security:
  mfa_required: false # Default: false
  # true: All users must enable MFA
```

G.8 Audit Logging

audit_enabled

```
audit:
  enabled: true # Default: true
  # Logs all changes to collections
```

audit_directory

```
audit:
  directory: "/var/log/themis/audit"
  # Must be on write-optimized storage
```

audit_retention_days

```
audit:
  retention_days: 365 # Default: 365 days
  # Older logs auto-purged (if compliance allows)
```

audit_include_data

```
audit:
  include_data: false # Default: false
  # true: Store full document in audit log (verbose!)
  # Recommendation: false (log metadata only)
```

G.9 Performance Tuning

memory_cache_warmup

```
performance:
  cache_warmup_enabled: true # Default: true
  # Loads frequently accessed data into cache on startup
  # Reduces first-query latency
```

memory_buffer_size_mb

```
performance:
  memory_buffer_size_mb: 512 # Default: 512
  # In-memory write buffer before flushing to disk
  # Larger = better batching, more RAM
  # Rule: 10-20% of cache_size_mb
```

io_parallelism

```
performance:
  io_parallelism: 4 # Default: number of CPU cores
  # Concurrent IO operations
  # Higher = better throughput on SSD, diminishing returns on HDD
```

G.10 Logging

log_level

```
logging:
  level: "info" # Default: info
  # Options: "debug" | "info" | "warn" | "error" | "critical"
  # Recommendations:
  #   production: "warn"
  #   staging: "info"
  #   development: "debug"
```

log_format

```
logging:
  format: "json" # Default: json
  # Options: "json" | "text"
  # json: Machine parseable (for aggregation)
  # text: Human readable
```

log_retention_days

```
logging:
  retention_days: 30 # Default: 30
  # Older logs auto-deleted
  # For compliance: increase to 365+
```

log_max_file_size_mb

```
logging:
  max_file_size_mb: 100 # Default: 100
  # Rotate log file when it exceeds this size
```

G.11 Vector Search Tuning

vector_similarity_metric

```
vector:
  similarity_metric: "cosine" # Default: cosine
  # Options: "cosine" | "euclidean" | "manhattan" | "dot"
  # cosine: Best for normalized embeddings (e.g., from BERT)
  # euclidean: Good for raw embeddings
```

hnsw_ef_search

```
vector:
  hnsw_ef_search: 100 # Default: 100
  # Exploration factor during search
  # Higher = more accurate, slower
  # Range: 10-1000
  # Rule: Start with 100, tune based on latency/accuracy tradeoff
```

hnsw_m

```
vector:
  hnsw_m: 16 # Default: 16
  # Connections per layer in HNSW graph
  # Higher = more accurate, more memory
  # Range: 8-64
```

G.12 Development Settings

dev_mode_enabled

```
development:
  dev_mode_enabled: false # Default: false
  # true: Disables some security checks (LOCAL DEV ONLY)
  # Enables: Hot reload, relaxed CORS, schema auto-creation
```

cors_enabled

```
development:
  cors_enabled: false # Default: false
  # true: Allow cross-origin requests
  # Set specific origins in production (not "*")
```

cors_origins

```
development:
  cors_origins:
    - "http://localhost:3000" # Frontend dev server
    - "http://localhost:8080"
```

G.13 Example Configurations

Single Node (Development)

```
server:
  port: 8529
  bind_address: "127.0.0.1"
  max_connections: 100

database:
  wal_sync_mode: "os" # Fast, eventual durability
  compression_algorithm: "lz4"

cache:
  size_mb: 512 # Small for dev machine

query:
  timeout_ms: -1 # No timeout for debugging

logging:
  level: "debug"
  format: "text" # Human readable
```

Production (Single Node, 32GB RAM)

```
server:
  port: 8529
  bind_address: "0.0.0.0"
  max_connections: 1000

database:
  data_directory: "/data/themis"
  wal_sync_mode: "fsync"
  compression_algorithm: "zstd"

cache:
  size_mb: 8192 # 25% of 32GB

security:
  tls_enabled: true
  jwt_secret: "${JWT_SECRET}" # From env var

audit:
  enabled: true
  retention_days: 365

logging:
  level: "warn"
  format: "json"
  retention_days: 90
```

High-Performance (Writes, 64GB RAM)

```
database:
  wal_sync_mode: "os" # Trade safety for speed
  compression_algorithm: "lz4" # Faster than zstd

cache:
  size_mb: 16384 # 25% of 64GB
  block_cache_size_mb: 4096

performance:
  memory_buffer_size_mb: 2048 # Larger batches
  io_parallelism: 8

query:
  max_query_result_size_mb: 2048 # Allow larger results
```

Analytics (Reads, 128GB RAM)

```
cache:
  size_mb: 32768 # 25% of 128GB
  mode: "lfu" # Popularity-based caching

indexing:
  vector_index_default: "hnsu"
  hnsu_ef_search: 200 # More accuracy for analytics

query:
  timeout_ms: 300000 # 5 minutes (long aggregations)
  max_query_result_size_mb: 4096 # Large result sets
```

G.14 Linux Kernel Tuning (sysctl)

These affect OS-level performance:

```
echo "fs.file-max = 2097152" >> /etc/sysctl.conf
echo "* soft nofile 100000" >> /etc/security/limits.conf
echo "* hard nofile 100000" >> /etc/security/limits.conf

echo "vm.swappiness = 10" >> /etc/sysctl.conf # Prefer RAM over swap

echo "net.core.rmem_max = 134217728" >> /etc/sysctl.conf
echo "net.core.wmem_max = 134217728" >> /etc/sysctl.conf

sysctl -p
```

G.15 Environment Variables

Common Env Vars

```
export JWT_SECRET="$(openssl rand -hex 32)"

export THEMIS_PORT=8529
export THEMIS_BIND_ADDRESS=0.0.0.0
export THEMIS_DATA_DIR=/data/themis
export THEMIS_LOG_LEVEL=info
export THEMIS_CACHE_SIZE_MB=2048

themisdb run
```

G.16 Configuration Validation

Validate Config File

```
themisdb config validate /etc/themis/themis.conf

themisdb config show
```

Health Check

```
curl -s http://localhost:8529/_admin/health | jq .
```

G.17 Configuration Change Checklist

Before changing production config: - ☐ Backup current config: `cp themis.conf themis.conf.backup` - ☐ Test change on staging first - ☐ If safe: `systemctl reload themis` (no restart) - ☐ Monitor metrics for 5 minutes - ☐ Have rollback plan (restore .backup file)

Summary

Most Common Tuning Knobs: 1. `cache_size_mb` - Available memory (25-30% of RAM) 2. `wal_sync_mode` - Safety vs. latency tradeoff 3. `compression_algorithm` - Storage vs. speed 4. `log_level` - Observability cost 5. `query_timeout_ms` - Prevent runaway queries

Start with defaults, measure, change one thing at a time, monitor impact.

Kapitel 56: Appendix H: Glossary & Terminology Reference

"Every domain has its jargon. Understand the terminology, understand the system."

H.1 Core Database Concepts

Base Entity

The fundamental unit of data in ThemisDB. Combines relational, document, graph, vector, and time-series aspects in a single unified structure.

Example:

```
{
  "_id": "users/alice",
  "_key": "alice",
  "_rev": 5,
  "name": "Alice",
  "email": "alice@example.com",
  "created_at": "2025-01-01T10:00:00Z",
  "embedding": [0.1, -0.2, 0.3, ...],
  "graph_edges": ["users/bob", "users/charlie"]
}
```

Collection

A group of related Base Entities (similar to a SQL table, but more flexible).

Types: - **Relational:** users, orders, products - **Document:** articles, posts, documents - **Graph:** relationships, friendships - **Vector:** embeddings, semantic data

Transaction

An atomic unit of work. Either all operations succeed (COMMIT) or all rollback. Provides ACID guarantees.

```
BEGIN
  INSERT {...} INTO users
  UPDATE {...} IN orders
COMMIT
```

ACID Guarantees

- **Atomicity:** All or nothing
 - **Consistency:** Data integrity maintained
 - **Isolation:** No dirty reads/writes
 - **Durability:** Persisted to disk
-

H.2 Query Language (AQL)

AQL (Adaptive Query Language)

Unified query language for all data models in ThemisDB. Combines SQL (relational), graph traversal, document processing, and vector search.

Core Constructs: - **FOR ... IN:** Iteration - **FILTER:** Conditional selection - **LET:** Variable binding - **COLLECT:** Grouping/aggregation - **SORT:** Ordering - **LIMIT:** Result limiting - **RETURN:** Output specification

Bind Variables

Parameterized query parameters (prevent injection attacks).

```
FOR user IN users
  FILTER user.email == @email
RETURN user
```

Run with: `db.query(query, bind_vars={'email': 'alice@example.com'})`

Query Plan

Execution strategy optimized by the query optimizer.

Components: - **Collection Scan:** Read all documents (slow) - **Index Seek:** Use index to find matching rows (fast) - **Filter:** Apply WHERE conditions - **Sort:** Order results - **Aggregation:** GROUP BY operations

H.3 Indexing & Performance

Index

Data structure enabling fast lookups. Trade: space for speed.

Types:

Type	Ordered	Point Lookup	Range Query	Size
BTree	✓	Fast	Fast	Large
Hash	✗	Very Fast	Slow	Medium
Skiplist	✓	Fast	Fast	Medium
Inverted	✗	Fast	N/A	Large

Query Cardinality

Estimated number of rows matching query conditions.

Low cardinality (< 1% of docs):
 → Use index for fast filtering

High cardinality (> 50% of docs):
 → Collection scan might be faster (no index overhead)

Selectivity

How well an index reduces result set.

Selectivity = Matching rows / Total rows

High selectivity (< 5%):
 → Use index

Low selectivity (> 50%):
 → Collection scan faster

H.4 Replication & Consistency

Primary (Master)

Single database node accepting writes. Log changes to replicas.

Follower (Replica)

Read-only copy of primary. Applies changes asynchronously from WAL.

Replication Lag

Delay between write on primary and replica visibility.

```
Write at Primary: T0
Arrive at Follower: T0 + lag_ms
Visible to read: T0 + lag_ms + apply_time
```

Acceptable lag: - Strongly consistent: 0ms (no lag) - Eventual consistent: < 5 seconds - Archive: minutes to hours

Write-Ahead Log (WAL)

Durability mechanism. All writes logged before applied.

```
Client write:
  1. Write to WAL on disk
  2. Apply to in-memory database
  3. Ack to client

On crash:
  1. Replay WAL on startup
  2. Resume from last committed state
```

Quorum Write

Write acknowledged only after reaching majority of replicas.

```
3-node cluster:
  Quorum = (3 + 1) / 2 = 2 nodes

Write must be on Primary + 1 Follower before ack
```

H.5 Vector Search Concepts

Embedding

Dense vector representation of unstructured data.

```
"Alice is an engineer"
→ [0.1, -0.2, 0.3, 0.15, -0.08, ...] # 384 dimensions
```

Similarity

Measure of how close two vectors are.

Metric	Range	Best For
Cosine	$[-1, 1]$	Normalized embeddings (angles)
Euclidean	$[0, \infty)$	Raw embeddings (distances)
Dot Product	$(-\infty, \infty)$	Unnormalized embeddings

Nearest Neighbor Search

Find K closest vectors to a query vector.

```
FOR v IN vectors
  FILTER DISTANCE(v.embedding, @query) < @threshold
  SORT DISTANCE(v.embedding, @query)
  LIMIT @top_k
  RETURN v
```

HNSW (Hierarchical Navigable Small Worlds)

Approximate nearest neighbor index algorithm.

Parameters: - **M:** Connections per node (16-64, default 16) - **ef_construct:** Quality during index building - **ef_search:** Quality during searches

H.6 Graph Concepts

Node (Vertex)

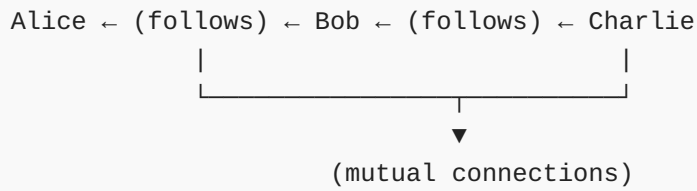
Entity in a graph. Represents an object or concept.

Edge (Relationship)

Connection between two nodes. Directed or undirected.

Graph Traversal

Following edges to discover related nodes.



Path

Sequence of nodes and edges.

Shortest path from Alice to Charlie:
Alice --(follows)-- Bob --(follows)-- Charlie
Length: 2 hops

Centrality

Measure of node importance.

Types: - **Degree:** Number of connections - **Betweenness:** How often on shortest paths - **Closeness:** Average distance to others - **PageRank:** Voting-based importance

H.7 Time-Series Concepts

Time-Series Data

Ordered sequence of measurements over time.

Metric: CPU Usage
Timestamp 1: 45%
Timestamp 2: 52%
Timestamp 3: 48%
...

Bucket / Time Window

Grouping data into time periods.


```
FOR metric IN metrics
  COLLECT hour = DATE_FORMAT(metric.timestamp, '%yyyy-%mm-%dd %hh:00')
  AGGREGATE avg = AVG(metric.value)
  RETURN { hour, avg }
```

Downsampling

Reducing time-series resolution (e.g., minute → hour).

```
High resolution: Every second (86,400 points/day)
Low resolution: Every hour (24 points/day)
Compression: 99.97% reduction
```

H.8 Operational Concepts

SLA (Service Level Agreement)

Contractual commitment on availability and performance.

```
SLA: "99.9% uptime with p99 latency < 200ms"
→ Downtime allowed: 43.2 minutes/month
→ 1 in 1000 queries can exceed 200ms
```

SLI (Service Level Indicator)

Measurable metric of actual performance.

```
SLI: Actual uptime = 99.92%
SLI: Actual p99 latency = 187ms
```

Error Budget

Acceptable downtime/errors for SLA.

```
SLA: 99.95% uptime
Error budget = 0.05% = 21.6 minutes/month

Used: 15 minutes
Remaining: 6.6 minutes (use for deployments/maintenance)
```

MTTR (Mean Time To Recovery)

Average time to fix a problem.

```
Incident starts: 10:00
Recovery complete: 10:15
MTTR = 15 minutes
```

MTTF (Mean Time To Failure)

Average time before next failure.

```
Downtime: 15 minutes
Recovery: 30 minutes (MTTR)
Uptime: 30 days until next failure
MTTF = 30 days
```

H.9 Security Concepts

RBAC (Role-Based Access Control)

Access control based on user roles.

```
Role: 'data_analyst'
Permissions:
- READ from analytics_*
- EXECUTE report queries
- NO WRITE/DELETE
```

JWT (JSON Web Token)

Stateless authentication token.

```
Header: { alg: "HS256", typ: "JWT" }
Payload: { sub: "alice", exp: 1726000000 }
Signature: HMAC-SHA256(header + payload, secret)
```

Encryption at Rest

Data encrypted on disk.

File on disk: [encrypted bytes]
In memory: Clear text (needed for processing)
Key location: HSM (Hardware Security Module)

Encryption in Transit

Data encrypted while traveling network.

TLS 1.3 + Perfect Forward Secrecy
→ Even if long-term key compromised, old traffic safe

MFA (Multi-Factor Authentication)

Multiple authentication mechanisms.

1. Password (something you know)
2. TOTP (something you have) - app code
3. WebAuthn (something you are) - fingerprint

H.10 Architecture Concepts

Monolithic Architecture

All components in single process. Easy to develop, hard to scale.

Microservices

Each service independent. Scales well, operational complexity.

ThemisDB: Hybrid Approach

- **Monolithic Core:** Single database engine
- **Distributed:** Horizontal sharding for scale-out
- **Multi-Model:** All data models in one engine (no microservices)

Horizontal Scaling

Add more nodes (scale out). Divide data via sharding.

```
1 node (10TB limit)
↓
2 nodes (5TB each)
↓
4 nodes (2.5TB each)
```

Vertical Scaling

Upgrade hardware (scale up). More RAM/CPU per node.

```
32GB RAM
↓
64GB RAM
```

H.11 Monitoring & Observability

Metrics

Quantitative measurements (numbers).

```
CPU usage: 45%
Memory: 8.2 GB
Queries per second: 1,200
Latency p99: 185ms
```

Logs

Textual records of events.

```
2025-01-01T10:00:00Z [INFO] Query executed: SELECT * FROM users
2025-01-01T10:00:01Z [WARN] Query latency: 245ms > threshold 200ms
2025-01-01T10:00:02Z [ERROR] Connection timeout from client 1.2.3.4
```

Traces

Request flow through system (distributed tracing).

```

User Request
├─ API Gateway (5ms)
├─ Authentication (12ms)
├─ Database Query (185ms)
│   ├─ Query Plan (2ms)
│   ├─ Index Seek (80ms)
│   ├─ Filter (50ms)
│   └─ Return (53ms)
└─ Response (3ms)
Total: 205ms

```

Cardinality

Number of unique values.

```

High cardinality: User IDs (millions)
Low cardinality: Status (10 values)

```

H.12 Alphabetical Glossary

API: Application Programming Interface - interface for programmatic access

ACID: Atomicity, Consistency, Isolation, Durability guarantees

AQL: Adaptive Query Language - ThemisDB's query language

Backup: Copy of data for recovery

Batch: Group of operations processed together

Cardinality: Number of distinct values

Cache: Fast memory storage layer

Changefeed: Stream of database changes

Collection: Group of related entities

Consistency: Data integrity maintained across system

Continuous Batching: Dynamic request batching technique for LLM inference that allows new requests to join active batches, dramatically improving throughput (176%) and reducing latency (57%) compared to static batching. See Chapter 20.9A.3.

Cursor: Pointer to result set for iteration

Denormalization: Storing redundant data for performance

Document: Semi-structured data (JSON-like)

DSGVO: German data protection regulation (GDPR equivalent)

Edge: Connection in graph

Embedding: Vector representation of data

Failover: Automatic switch to backup system

Flash Attention: IO-aware attention mechanism for LLMs that uses SRAM tiling instead of HBM storage, reducing GPU memory usage by 37% and increasing throughput by 69%. Requires NVIDIA Ampere+ GPUs. See Chapter 20.9A.1.

Flush: Write data from memory to disk

Garbage Collection: Freeing unused memory

GBNF (GGML BNF): Grammar notation used by llama.cpp for constrained generation. Extends EBNF with specific syntax for controlling LLM output format. See Grammar-Constrained Generation.

Grammar-Constrained Generation: LLM technique that uses EBNF/GBNF grammar rules to guarantee syntactically valid outputs (JSON, XML, CSV). Achieves 95-99% success rate vs 60-70% without constraints, eliminating need for output validation and retries. See Chapter 17.12.6.

Graph: Network of connected nodes

Heap: Memory area for dynamic allocation

HNSW: Hierarchical Navigable Small Worlds - vector index

Hot Data: Frequently accessed data

Hot Spare: Fully configured standby node in a database cluster that can automatically take over (failover) when an active shard fails, typically within <5 seconds. Provides high availability with zero data loss when combined with WAL replication. See Chapter 16.10.1.

Index: Data structure for fast lookups

Ingestion: Loading data into database

Inverted Index: Index mapping values to documents

Isolation: Transactions don't interfere

JIT: Just-In-Time compilation

JSON: JavaScript Object Notation

JWT: JSON Web Token - stateless auth

Keyspace: Logical division of data

Latency: Time delay for operation

LoRA (Low-Rank Adaptation): Efficient fine-tuning technique for large language models that adds trainable low-rank matrices to pretrained models, reducing memory requirements by 99% and training time by 3-10x compared to full fine-tuning. Multiple LoRA adapters can run on a single base model. See Chapter 17.12.5.

LSM: Log-Structured Merge tree

MVCC: Multi-Version Concurrency Control

Normalization: Organizing data to reduce redundancy

Node: Entity in graph or cluster

OLAP: Online Analytical Processing

OLTP: Online Transaction Processing

Optimization: Making something run faster

Paged Attention: Memory management technique for LLM attention mechanisms that organizes KV-cache into fixed-size pages instead of continuous memory allocation, reducing GPU memory waste by 80% and increasing concurrent request capacity by 5x. See Chapter 17.12.4.

Pagination: Dividing results into pages

Partition: Division of data across nodes

Percentile: Value below which percentage falls

Persistence: Data survives shutdown

Piper: Fast, local neural Text-to-Speech (TTS) engine used in ThemisDB Voice Assistant. Provides natural-sounding voice synthesis in multiple languages with <50ms latency and minimal CPU usage. See Chapter 10.7.

Prefix Caching: LLM optimization that caches the attention states of frequently used prompt prefixes (such as system prompts), enabling 75% cost savings and 95% latency reduction for repeated queries with common prompt beginnings. See Chapter 17.12.1.

Projection: Selecting subset of columns

Query Plan: Execution strategy for query

Quorum: Majority consensus

Range: Ordered selection of values

Replica: Copy of data

Replication: Process of copying data to replicas

Response Caching: Intelligent caching system for LLM responses that uses embedding-based semantic similarity to identify and reuse answers to similar questions, providing 60-80% cost savings for repetitive queries. Supports configurable TTL and similarity thresholds. See Chapter 17.12.2.

RocksDB: Embedded key-value store (ThemisDB storage engine)

Rollback: Undo transaction

RoPE (Rotary Position Embedding): Position encoding technique for transformer models that enables context window extension beyond training length through scaling. ThemisDB supports Linear, NTK-aware, and YaRN scaling methods to extend context from 4K to 32K+ tokens. See Chapter 17.12.7.

RoPE Scaling: Technique to extend LLM context windows beyond their original training length by adjusting the rotary position embedding frequency. YaRN method achieves 8x context extension (4K → 32K tokens) with <10% quality loss. See Chapter 17.12.7.

RPO: Recovery Point Objective (max data loss)

RTO: Recovery Time Objective (max downtime)

Sampling: Statistical subset of data

Schema: Data structure definition

Selectivity: Fraction of rows matching filter

Serialization: Converting to byte format

Sharding: Horizontal data partitioning

Snapshot: Point-in-time data view

Speculative Decoding: LLM acceleration technique that uses a small, fast "draft model" to speculatively generate multiple tokens in parallel, which are then validated by the larger "target model". Achieves 2-3x speedup with 82-88% token acceptance rates. See Chapter 20.9A.2.

SQL: Structured Query Language

SSTable: Sorted String Table

TTL: Time-To-Live (auto-delete after interval)

Transaction: Atomic unit of work

Throughput: Operations per unit time

Vector: Ordered list of numbers

View: Virtual table derived from query

Voice Assistant: Enterprise feature providing natural language voice interaction using Whisper (STT), Piper (TTS), and llama.cpp (LLM). Enables call center automation, meeting protocol generation, and voice-controlled database queries with DSGVO-compliant storage. See Chapter 10.7.

WAL (Write-Ahead Log): Transaction log that records all database changes before they are applied, ensuring durability and enabling replication. In ThemisDB v1.4.0-alpha, WAL replication provides zero-data-loss failover with support for synchronous, asynchronous, and hybrid replication modes. See Chapter 16.10.2.

WAL Replication: Replication mechanism based on Write-Ahead Log streaming that continuously transfers transaction log entries from primary to replica nodes. Supports sync (zero data loss, higher latency), async (minimal latency, potential data loss), and hybrid modes. See Chapter 16.10.2.

Warm Data: Occasionally accessed data

Whisper: OpenAI's high-accuracy Speech-to-Text (STT) model integrated into ThemisDB Voice Assistant via whisper.cpp. Supports 100+ languages with auto-detection, speaker diarization, and 5 model sizes (tiny to large) trading accuracy for speed. See Chapter 10.7.

Workload: Pattern of database usage

Summary

Understanding these terms is essential for: - **Development:** Writing efficient queries - **Operations:** Configuring and monitoring - **Architecture:** Designing systems - **Communication:** Discussing with team

Keep this glossary handy when learning or explaining ThemisDB concepts.

Kapitel 57: Appendix I: Troubleshooting Guide & Common Issues

"The best troubleshooting guide prevents issues before they start."

1.1 Installation Issues

Issue: Failed to Install Dependencies

Error:

```
ERROR: Package xyz not found in vcpkg
```

Diagnosis:

```
./vcpkg list | grep xyz  
  
ping github.com
```

Solutions: 1. **Update vcpkg registry:** `bash ./vcpkg update ./vcpkg upgrade # Rebuild all packages`

1. **Clear cache:** `bash rm -rf vcpkg_installed/ ./vcpkg install # Reinstall from scratch`

2. **Use specific version:** `json // vcpkg.json { "overrides": [ { "name": "xyz", "version": "1.2.3" } ] }`

1.2 Connection Issues

Issue: Cannot Connect to Database

Error:

```
Connection refused at localhost:8529
```

Diagnosis:

```
systemctl status themis  
  
lsof -i :8529  
  
sudo ufw status
```

Solutions:

1. **Service not running:** `bash sudo systemctl start themis sudo systemctl enable themis # Auto-start on reboot`
2. **Port blocked by firewall:**
`bash sudo ufw allow 8529 sudo ufw allow in on docker0 to any port 8529 # Docker`
3. **Process stuck:** `bash # Force kill (last resort) pkill -9 themisdb sudo systemctl restart themis`

Issue: SSL/TLS Certificate Error

Error:

```
ERROR: x509: certificate signed by unknown authority
```

Solutions:

1. **Invalid certificate:** ```bash # Check certificate openssl x509 -in /etc/themis/certs/server.crt -text -noout`

`# Verify against key openssl x509 -in server.crt -noout -modulus | openssl md5 openssl rsa -in server.key -noout -modulus | openssl md5 # Should match ```
 1. **Self-signed certificate (dev only):** `bash # Disable SSL verification (DANGER - dev only) THEMIS_SKIP_VERIFY=true themis-cli ...`
 2. **Certificate expired:** ```bash # Generate new certificate openssl req -x509 -nodes -days 365 -newkey rsa:4096 \ -keyout server.key -out server.crt`

`# Restart sudo systemctl restart themis ```
-

I.3 Performance Issues

Issue: Queries Running Slow (>1s)

Diagnosis:

```
-- Use EXPLAIN to see query plan
EXPLAIN
FOR user IN users
  FILTER user.status == 'active'
  RETURN user

-- Check which collection scan
-- If CollectionScan on large table → use index!
```

Solutions:

```
1. Missing Index: ``aql -- Create index on filtered field CREATE INDEX idx_status ON users(status)

-- Verify index is used EXPLAIN -- Should show IndexSeek ``

1. Wrong Index: ``aql -- Problem: Composite index (status, created_at) -- But querying only
  created_at

-- Solution: Reorder to match query pattern DROP INDEX idx_status_created CREATE INDEX
idx_created_status ON users(created_at, status) ``

1. Expression in FILTER: ``aql --  SLOW: Expression prevents index use FILTER
  YEAR(user.created_at) == 2025

--  FAST: Range query on indexed field FILTER user.created_at >= '2025-01-01' AND user.created_at
< '2026-01-01' ``
```

Issue: High Memory Usage (>80%)

Diagnosis:

```
free -h
top -p $(pgrep themis)

curl http://localhost:8529/_admin/cache | jq .

curl http://localhost:8529/_admin/cursors | jq 'length'
```

Solutions:

1. **Cache too large:** `yaml # themis.conf cache: size_mb: 2048 # Reduce from 8192`

Then: `sudo systemctl reload themis`

2. **Cursor leak (unclosed connections):** `python # ❌ Leak cursor = client.query("SELECT ...") #`
 Never closed

 Safe with `client.query("SELECT ...") as cursor: for doc in cursor: process(doc)`

1. **Large result set:** `aql -- ❌ Materializes 1M rows FOR doc IN collection RETURN doc`

--  Paginate `FOR doc IN collection LIMIT @offset, 100 RETURN doc`

I.4 Data Issues

Issue: Data Corruption (Checksum Mismatch)

Error:

```
ERROR: Block checksum mismatch at offset 123456
```

Recovery:

```
sudo systemctl stop themis

themisdb recover /var/lib/themis

tar -xzf /backup/themis-2025-01-01.tar.gz -C /var/lib/
sudo chown -R themis:themis /var/lib/themis

sudo systemctl start themis

aql "RETURN COUNT(*) FROM collection"
```

Issue: Missing Data (Unintended Deletion)

Diagnosis:

```
grep DELETE /var/log/themis/audit.log | tail -20
```

Recovery:

```
sudo systemctl stop themis
tar -xzf /backup/themis-2025-01-01-14-00.tar.gz -C /var/lib/ # Before deletion
sudo systemctl start themis

aws s3 cp s3://backups/themis-before-deletion/ /tmp/
```

Issue: Replication Lag Too High (>10s)

Diagnosis:

```
curl http://primary:8529/_admin/replication/lag | jq .lag_ms

ssh follower "top -bn1 | head -15"
ssh follower "iostat -x 1 5 | grep sda"

mtr primary-ip --report --report-wide
```

Solutions:

1. **Follower slow (CPU/IO):** `bash # Add more resources # Edit VM/K8s resource limits`
`# Restart follower to pick up new limits`

2. **Network congestion:** ```bash # Check MTU ip link show | grep mtu`

`# If < 9000, enable Jumbo frames (if supported) ip link set dev eth0 mtu 9000 ```

1. **Primary write rate too high:** `yaml # Throttle writes (temporary) # themis.conf`
`database: max_write_rate_per_sec: 1000 # Limit to 1k/s`

1.5 Replication Issues

Issue: Follower Won't Start (Replication Sync Failed)

Error:

```
ERROR: Cannot connect to primary at primary.example.com:8529
```

Solutions:

1. **Primary unreachable:** ```bash # Test connectivity telnet primary.example.com 8529 ping`
`primary.example.com`

```
# Check network policies kubectl get networkpolicies -A ``
```

1. **Follower config wrong:** `yaml # Check follower primary address cat /etc/themis/themis.conf | grep follower_primary`

```
# Fix if needed # Edit config, then: systemctl restart themis ``
```

1. **Data mismatch (corruption):** `bash # Full resync (rebuilds follower from scratch) themisdb replication reset sudo systemctl start themis # This will take time (downloads full dataset from primary)`

Issue: Primary-Follower Divergence

Symptoms:

```
Primary: 10000 documents
Follower: 9999 documents
```

Investigation:

```
curl http://primary:8529/_admin/collection/mycoll/count | jq .count
curl http://follower:8529/_admin/collection/mycoll/count | jq .count
```

Fix:

```
sudo systemctl stop themis-follower

themisdb replication reset --primary primary:8529

curl http://follower:8529/_admin/replication/status | jq .lag_ms
```

I.6 Backup & Recovery Issues

Issue: Backup Failed

Error:

```
ERROR: Failed to backup - insufficient space
```

Solutions:

1. **Disk full:** ``bash # Check disk df -h /backup

```
# Clean old backups ls -lt /backup/.tar.gz | tail -10 # Identify oldest rm /backup/themis-2024-.tar.gz #
Delete old ``
```

1. **Permissions:** ``bash # Check backup directory ownership ls -ld /backup

```
# Fix if needed sudo chown themis:themis /backup sudo chmod 750 /backup ``
```

1. **Database locked:** ``bash # Check if backup process stuck ps aux | grep themis-backup

```
# Kill if necessary pkill -f themis-backup ``
```

Issue: Restore Failed (Data Corrupted)**Error:**

```
ERROR: Restore failed - backup corrupted
```

Verification:

```
tar -tzf /backup/themis-2025-01-01.tar.gz > /dev/null && \
  echo "Backup OK" || echo "Backup CORRUPTED"

ls -lt /backup/themis-*.tar.gz | head -5
```

Recovery:

```
BACKUP=/backup/themis-2024-12-25.tar.gz

tar -tzf $BACKUP > /dev/null || echo "Also corrupted!"

themisdb recover --wal-only /var/lib/themis
```

I.7 Security Issues

Issue: Unauthorized Access (Auth Failing)**Error:**


```
ERROR: Authentication failed: Invalid credentials
```

Diagnosis:

```
tail -50 /var/log/themis/auth.log

aql "SELECT * FROM users WHERE name == 'alice'"
```

Solutions:

1. **Wrong password:** `bash # Reset user password themis-cli admin reset-password
alice # New temporary password will be printed`
2. **User locked out:** `aql -- Unlock user UPDATE {name: 'alice'} WITH {locked: false}
IN users`
3. **JWT token expired:** `python # Get new token token =
client.authenticate(username='alice', password='xxx') # Use new token in
subsequent requests`

Issue: Data Breach Detected

Immediate Actions:

1. Isolate database from network
 - Drop external connections
 - Kill active sessions
2. Preserve evidence
 - Copy audit logs
 - Backup database
 - Keep system for forensics
3. Notify stakeholders
 - Security team
 - Management
 - Legal (GDPR notification)
4. Rotate credentials
 - Generate new passwords
 - Rotate encryption keys
 - Revoke old tokens

I.8 Support Matrix: By Issue Type

Issue	Severity	MTTR	Self-Fix %
Connection timeout	High	5 min	95%
Query slow (no index)	Medium	10 min	90%
High memory usage	High	15 min	80%
Data corruption	Critical	30 min	60%
Replication lag	Medium	20 min	70%
Backup failed	Medium	15 min	85%
Auth issues	High	10 min	75%
Security breach	Critical	60 min	20%

I.9 Escalation Path

- Level 1: Try self-help
 - └ Check documentation
 - └ Search existing issues
 - └ Try basic troubleshooting
- Level 2: Community support
 - └ GitHub Discussions
 - └ Stack Overflow
 - └ Slack community
- Level 3: Vendor support
 - └ File bug report
 - └ Provide diagnostic data
 - └ Wait for response
- Level 4: Enterprise support
 - └ Dedicated support engineer
 - └ SLA response times
 - └ Priority queue

I.10 Data Collection for Support

When reporting issues, include:

```
#!/bin/bash

echo "=== System Info ==="
uname -a
free -h
lsb_release -a

echo "=== ThemisDB Version ==="
themisdb --version

echo "=== Service Status ==="
systemctl status themis
journalctl -u themis -n 50

echo "=== Memory Usage ==="
curl http://localhost:8529/_admin/memory | jq .

echo "=== Disk Usage ==="
df -h /var/lib/themis

echo "=== Active Queries (top 5) ==="
curl http://localhost:8529/_admin/slow-queries | jq '.queries[0:5]'

echo "=== Configuration ==="
grep -v '^#' /etc/themis/themis.conf | grep -v '^$'
```

I.11 Contacting Support

Community Channels

- **GitHub Issues:** github.com/makr-code/ThemisDB/issues
- **Discussions:** github.com/makr-code/ThemisDB/discussions
- **Stack Overflow:** Tag: `themisdb`

Enterprise Support (SLA)

- **Response time:** < 4 hours for Critical
- **On-call escalation:** Yes
- **Dedicated engineer:** Yes

When to Contact

- **Development questions:** Community first
 - **Bug reports:** GitHub Issues
 - **Security issues:** Responsible disclosure (see SECURITY.md)
 - **Critical production outage:** Vendor support hotline
-

| Summary

Most issues fall into categories above. With this guide, you should be able to: 1. **Diagnose** problems quickly 2. **Fix** common issues yourself 3. **Know when to escalate** to support 4. **Provide** detailed diagnostic data

Keep this guide bookmarked for quick reference during incidents.

Kapitel 58: Anhang A: Literaturverzeichnis

Dieses Literaturverzeichnis folgt dem IEEE-Zitierstil und umfasst alle wissenschaftlichen und technischen Quellen, die in diesem Kompendium referenziert wurden.

Primärquellen: ThemisDB Gemini Analysen

Die folgenden strategischen Analysen wurden als Primärquellen für die technische Vertiefung in Kapitel 1, 2, 3 und 17 herangezogen:

[1] "Strategische Gesamtanalyse: ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur," ThemisDB Project, Internes Strategiedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Gesamtanalyse_ ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur.md

[2] "Strategische Analyse: ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen," ThemisDB Project, Internes Analysedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Analyse_ ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen.md

[3] "Die ThemisDB-Architektur: Eine technische Tiefenanalyse eines Multi-Modell-Datenbanksystems auf LSM-Tree-Basis," ThemisDB Project, Technisches Whitepaper, Nov. 2025. [Online]. Verfügbar: docs/gimini/ThemisDB Dokumentation und Berichtsanalyse.md

[4] "Architektonischer Entwurf eines hochperformanten Multi-Modell-Datenbanksystems: Eine Kernel-Level-Analyse von kanonischem Speicher, Projektionsschichten und Speicherhierarchien in C++ und Rust," ThemisDB Project, Architektur-Blueprint, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hybride Datenbankarchitektur C++_Rust (1).md

[5] "ThemisDB vs. Hyperscaler Datenbanken: Komparativer Vergleich," ThemisDB Project, Vergleichsanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Themis vs. Hyperscaler Datenbanken Vergleich.md

[6] "Hyperscaler-Einordnung für ThemisDB-Bericht," ThemisDB Project, Marktanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hyperscaler-Einordnung für ThemisDB-Bericht.md

[7] "KI-Strategie: ThemisDB, Hyperscaler, Ethik," ThemisDB Project, Strategiepapier, Nov. 2025. [Online]. Verfügbar: docs/gimini/KI-Strategie_ ThemisDB, Hyperscaler, Ethik.md

[8] "VCCDB Design: Veritas, Covina, Clara Database Design Spezifikation," ThemisDB Project, Design-Dokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/VCCDB Design (1).md

[9] "Forschungsbericht ThemisDB 2025: Public Research Edition," ThemisDB Project, Forschungsbericht, Nov. 2025. [Online]. Verfügbar: docs/gimini/Forschungsbericht ThemisDB 2025_THEMISDB_PUBLIC_RE....md

Sekundärquellen: Datenbanksysteme und Architektur

[10] M. Stonebraker and U. Cetintemel, "'One size fits all': An idea whose time has come and gone," in *Proc. IEEE 21st International Conference on Data Engineering (ICDE '05)*, Tokyo, Japan, 2005, pp. 2-11, doi: 10.1109/ICDE.2005.1.

[11] J. Lu and I. Holubová, "Multi-model databases: A new journey to handle the variety of data," *ACM Computing Surveys*, vol. 52, no. 3, pp. 1-38, Jun. 2019, doi: 10.1145/3323214.

[12] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351-385, Jun. 1996, doi: 10.1007/s002360050048.

[13] Facebook Engineering, "RocksDB: A persistent key-value store for fast storage environments," Facebook Inc., Technical Report, 2013. [Online]. Verfügbar: <https://rocksdb.org/>

[14] D. G. Andersen, J. Franklin, M. Kaminsky, A. Phanishayee, L. Tan, and V. Vasudevan, "FAWN: A fast array of wimpy nodes," in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, Big Sky, MT, USA, 2009, pp. 1-14, doi: 10.1145/1629575.1629577.

[15] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Jose, CA, USA, 1995, pp. 1-10, doi: 10.1145/223784.223785.

Transaktionsverarbeitung und Konsistenz

[16] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2009.

[17] C. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA, 1987, pp. 249-259, doi: 10.1145/38713.38742.

[18] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23-29, Feb. 2012, doi: 10.1109/MC.2012.37.

[19] D. B. Lomet, "Process structuring, synchronization, and recovery using atomic actions," in *Proc. ACM Conference on Language Design for Reliable Software*, Raleigh, NC, USA, 1977, pp. 128-137, doi: 10.1145/800022.808314.

[20] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," *ACM Transactions on Database Systems*, vol. 34, no. 4, pp. 1-42, Dec. 2009, doi: 10.1145/1620585.1620587.

Graph-Datenbanken und Traversierung

[21] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1-39, Feb. 2008, doi: 10.1145/1322432.1322433.

[22] M. A. Rodriguez and P. Neubauer, "The graph traversal pattern," in *Graph Data Management: Techniques and Applications*, S. Sakr and E. Pardede, Eds. Hershey, PA, USA: IGI Global, 2011, pp. 29-46, doi: 10.4018/978-1-61350-053-8.ch002.

[23] N. Francis et al., "Cypher: An evolving query language for property graphs," in *Proc. ACM SIGMOD International Conference on Management of Data*, Houston, TX, USA, 2018, pp. 1433-1445, doi: 10.1145/3183713.3190657.

[24] R. Angles, "A comparison of current graph database models," in *Proc. IEEE 28th International Conference on Data Engineering Workshops (ICDEW)*, Arlington, VA, USA, 2012, pp. 171-177, doi: 10.1109/ICDEW.2012.31.

Vektor-Datenbanken und Similarity Search

[25] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[26] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, Sep. 2021, doi: 10.1109/TBDATA.2019.2921572.

[27] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 11, pp. 2227-2240, Nov. 2014, doi: 10.1109/TPAMI.2014.2321376.

[28] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 2055-2063, doi: 10.1109/CVPR.2016.226.

RAG und LLM-Integration

[29] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.

[30] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009, doi: 10.1561/15000000019.

[31] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Virtual Event, 2020, pp. 39-48, doi: 10.1145/3397271.3401075.

[32] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, Dec. 2023. [Online]. Verfügbar: <https://arxiv.org/abs/2312.10997>

Polyglot Persistence und Multi-Model-Systeme

[33] M. Fowler and P. Sadalage, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.

[34] J. Pokorný, "Conceptual and database modelling of graph databases," in *Proc. 20th International Conference on Information Systems Development (ISD)*, Edinburgh, Scotland, 2011, pp. 370-381.

[35] ArangoDB Inc., "ArangoDB Multi-Model Database Architecture," Technical Whitepaper, 2020. [Online]. Verfügbar: <https://www.arangodb.com/>

[36] Microsoft Azure, "Azure Cosmos DB: Multi-model database service," Microsoft Technical Documentation, 2023. [Online]. Verfügbar: <https://azure.microsoft.com/en-us/services/cosmos-db/>

Kompression und Speicheroptimierung

[37] Y. Collet, "LZ4: Extremely fast compression algorithm," Open Source Implementation, 2011. [Online]. Verfügbar: <https://lz4.github.io/lz4/>

[38] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," Internet Engineering Task Force (IETF), RFC 8878, Feb. 2021. [Online]. Verfügbar: <https://tools.ietf.org/html/rfc8878>

[39] M. Adler, "Snappy: A fast compressor/decompressor," Google Inc., Technical Report, 2011. [Online]. Verfügbar: <https://google.github.io/snappy/>

Serialisierung und Parsing

[40] D. Lemire, G. Ssi-Yan-Kai, and O. Katz, "Parsing gigabytes of JSON per second," *The VLDB Journal*, vol. 28, no. 6, pp. 941-960, Dec. 2019, doi: 10.1007/s00778-019-00578-5.

[41] ArangoDB Inc., "VelocityPack: A fast and compact format for serialization and storage," Technical Specification, 2016. [Online]. Verfügbar: <https://github.com/arangodb/velocypack>

[42] Google Inc., "Protocol Buffers: Google's data interchange format," Technical Documentation, 2008. [Online]. Verfügbar: <https://developers.google.com/protocol-buffers>

Standards und Compliance

[43] Bundesamt für Sicherheit in der Informationstechnik (BSI), "IT-Grundschutz-Kompendium," BSI Standard 200-2, Bonn, Deutschland, 2023. [Online]. Verfügbar: <https://www.bsi.bund.de/>

[44] European Parliament and Council, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, Apr. 2016. [Online]. Verfügbar: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

[45] Apache Software Foundation, "Apache Ranger: Framework to enable, monitor and manage comprehensive data security," Technical Documentation, 2023. [Online]. Verfügbar: <https://ranger.apache.org/>

Verwendete Softwarebibliotheken

[46] Facebook Engineering, "RocksDB C++ API Reference," Version 8.8.0, 2023. [Online]. Verfügbar: <https://github.com/facebook/rocksdb>

[47] The Rust Project, "serde: Serialization framework for Rust," Version 1.0, 2023. [Online]. Verfügbar: <https://serde.rs/>

[48] D. Lemire et al., "simdjson: Parsing gigabytes of JSON per second," Version 3.0, 2023. [Online]. Verfügbar: <https://github.com/simdjson/simdjson>

Verwendungshinweise

Kapitel-Referenzen

Die folgenden Primärquellen wurden in den entsprechenden Kapiteln verwendet:

Kapitel 1 (Einführung): - [1] Strategische Gesamtanalyse - Teil III: Architektonische Kernentscheidung - [3] ThemisDB-Architektur - Teil 1: Kanonische Speicherarchitektur - [4] Architektonischer Entwurf - Teil 1: Base Entity Paradigma

Kapitel 2 (Architektur): - [3] ThemisDB-Architektur - Teil 1.3: RocksDB als transaktionales Fundament - [4] Architektonischer Entwurf - Teil 1: Multi-Modell-Speicherformat - [5] Hyperscaler-Vergleich - Teil 3.1: Kanonischer Speicher - [12] LSM-Tree Grundlagen - [13] RocksDB Dokumentation

Kapitel 3 (Multi-Model): - [1] Strategische Gesamtanalyse - Teil III: Ablösung von UDS3 - [2] Strategische Analyse - Teil II: Wettbewerbslandschaft - [5] Hyperscaler-Vergleich - Teil 2: Post-Filtering Problem - [11] Multi-Model Databases Survey

Kapitel 17 (LLM-Integration): - [2] Strategische Analyse - Teil 1.3: Native KI- und RAG-Fähigkeiten - [5] Hyperscaler-Vergleich - Teil 2: RAG-Architektur - [29] Retrieval-Augmented Generation - [30] BM25 and Beyond

Zitierkonventionen im Text

Im Text dieses Kompendiums werden Quellen durch eckige Klammern referenziert, z.B. [1], [3], [12]. Die vollständigen Quellenangaben finden Sie in diesem Anhang im IEEE-Format.

Zugriff auf Primärquellen

Die Primärquellen [1]-[9] sind interne Projektdokumente im [docs/gimini/](#) Verzeichnis des ThemisDB Repository verfügbar. Die Sekundärquellen [10]-[48] sind peer-reviewed wissenschaftliche Publikationen, technische Standards oder Open-Source-Dokumentationen, die über die angegebenen DOIs oder URLs zugänglich sind.

Zuletzt aktualisiert: Dezember 2025

Version: 1.3.4
